

AcquisitionOS Interview Prep

title: "AcquisitionOS - Senior Java Backend Interview Preparation"

subtitle: "Spring Boot 3.2 - Java 17 - Hibernate 6 - PostgreSQL 15 - Redis 7 - Comprehensive Reference Guide"

author: "Candidate Profile: 3.5 years backend (positioned as 4+ years capable)"

date: "Prepared for: Uber, NetApp, Meta (and other product-based companies)"

AcquisitionOS - Senior Java Backend Developer Interview Preparation Guide

IMPORTANT - READ THIS FIRST

The questions inside this guide are **SAMPLE reference questions** compiled as a framework for the LLM. They are intentionally curated to act as a **base reference**, not an exhaustive list. The interview prep must be expanded to cover **maximum possible questions per category** - easy, medium, hard, and twisted variants - including end-to-end annotation usage, code snippets (Java), runtime logic walk-throughs, ASCII architecture diagrams (application flow, business flow, service flow, data flow, deployment flow), database schema designs, and cloud/deployment/testing scenarios. If any answer feels thin, treat it as a **seed** and expand it with deeper examples, edge cases, and follow-up branches the way a tough interviewer at Uber / Amazon / Meta / NetApp would drill down. The interviewer at these companies will frequently ask for **the logic of an API end-to-end** ("walk me through what happens from Controller → Service → Repository → DB when I call `POST /api/leads`"), so every API question should be answerable with a step-by-step trace including annotations, transaction boundaries, exceptions, and side effects.

Target audience: Senior Java Backend Engineer, 3.5 years experience, interviewing for 4+ year capability roles at **Uber (next week), NetApp, and Meta (this month)**.

Backend story (mandatory framing): AcquisitionOS backend is presented as a **Java 17 + Spring Boot 3.2.x monolithic layered application** using **Spring Web MVC + Spring Data JPA + Hibernate 6 + PostgreSQL 15 + Redis 7**. The frontend (React 18 + TypeScript 5) is out of scope for backend answers - never mention the actual Next.js backend implementation.

Quick Reference - Project Summary Card

Project Name: AcquisitionOS **Tagline:** B2B SaaS for AI-powered client acquisition - discover, engage, convert, and manage leads at scale. **Domain:** B2B Sales / Lead Generation / CRM / Outreach Automation **Type:** Multi-tenant B2B SaaS platform **Your Role:** Senior Backend Engineer (individual contributor - backend-heavy, full-stack awareness) **Experience:** 3.5 years (positioned as 4+ years capable) **Team Size:** 6 engineers (2 backend, 2 frontend, 1 QA, 1 PM), you owned 70% of backend modules **Duration:** ~9 months design → MVP → production → 3 minor releases **Production Status:** Live - serving ~3,200 paying organizations, ~50,000 active users, ~2.1M leads processed cumulatively, 99.94% monthly uptime SLA **Stack (Backend):** Java 17 LTS, Spring Boot 3.2.x, Spring Web MVC, Spring Data JPA, Hibernate 6, Spring Security 6, Spring Batch, Spring AOP, PostgreSQL 15, Redis 7, HikariCP, Flyway, Kafka (events), Resilience4j, MapStruct, Lombok, JUnit 5, Mockito, Spring Boot Test, Testcontainers, WireMock **Stack (Frontend):** React 18, TypeScript 5, Vite, Tailwind CSS, TanStack Query, Zustand, shadcn/ui **Stack (Infra/Cloud):** Docker, AWS EC2 + ECS Fargate, RDS PostgreSQL (Multi-AZ), ElastiCache Redis, S3, CloudFront, Application Load Balancer, GitHub Actions CI/CD, Prometheus + Grafana + Loki, Sentry, OpenTelemetry **External Integrations:** Stripe (payments + webhooks), Google OAuth2, Gmail SMTP, Google Calendar API, Twilio (WhatsApp), Telegram Bot API, OpenAI/Gemini (AI outreach)

Elevator Pitch (30 seconds): > “AcquisitionOS is a B2B SaaS platform that helps small and mid-size sales teams replace 5–6 disconnected tools - lead scrapers, email outreach tools, CRM, calendar, pipeline manager - with one AI-powered system. On the backend I built a Java 17 / Spring Boot 3.2 monolith layered as Controller-Service-Repository, backed by PostgreSQL 15 and Redis 7. The system discovers leads from public sources, scores them with an AI engine, generates personalised outreach emails, sends them via Gmail or Twilio, books meetings through Google Calendar, and tracks deals through a Kanban-style pipeline. We’re at 3,200 paying orgs, ~50K users, 2.1M leads processed, and a 99.94% monthly SLA. I own roughly 70% of the backend including auth (OTP, magic link, Google OAuth), payments (Stripe), the credit/usage metering system, and the AI outreach pipeline.”

Two-Minute Walkthrough Script:

“Let me walk you through AcquisitionOS end-to-end.

Problem: Small sales teams waste 60% of their day switching between a lead scraper, an email tool, a CRM, a calendar, and a pipeline board. There’s no single source of truth, no AI-driven prioritisation, and no closed loop between outreach and revenue.

Solution: AcquisitionOS unifies the whole funnel. The user uploads a niche - say ‘dentists in Mumbai’ - and our **Lead Discovery Pipeline** (Spring Batch job + scraping workers + Google Places / LinkedIn enrichment) returns enriched leads with email, phone, social, revenue estimate, and a 0-100 quality score from our **AI Scoring Engine** (calls Gemini/OpenAI through a Resilience4j-circuit-broken `OutreachService`).

The user picks leads and clicks **Generate Outreach**. Our `OutreachGeneratorService` calls the LLM with a templated prompt + the lead context, returns 3 email variants, deducts **credits** (the platform’s internal metered currency - every paid action costs credits; each plan ships a monthly allowance), and stores the generated email in a `messages` table. The user can then send via Gmail SMTP (OAuth’d per-user) or Twilio WhatsApp, with **open/click tracking** via pixel + redirect endpoints. Replies are pulled back via Gmail PubSub or webhook and routed through a **Reply Intelligence** module that classifies intent (interested / not-now / unsubscribe) and moves the lead stage automatically in the **Pipeline**.

When the lead says ‘yes’, **Meeting Scheduling** kicks in - we use Google Calendar API to read free/busy slots, propose 3 timings, and on confirmation push a calendar invite. A **Spring @Scheduled reminder job** fires 24h / 1h before.

Throughout, every paid feature deducts credits atomically (`@Transactional` + pessimistic lock on the `credit_balance` row), every security event is audit-logged via Spring AOP, and every long-running job (lead scraping, sequence execution, reply polling, renewal) runs through a **Spring Batch / @Async TaskExecutor** backed by a `job_state` table so we can resume on crash.

Architecture is a monolith, deliberately - at 3,200 tenants and ~50K users we are nowhere near the scale where microservices pay for themselves. We chose clean layered separation, bounded contexts inside one codebase (`com.acquisitionos.leads`, `.outreach`, `.meetings`, `.billing`, `.auth`, `.ai`), and a `DomainEventPublisher` (Spring `ApplicationEventPublisher`) for in-process decoupling. We’ve designed the seams so we can extract `outreach` or `ai` into separate services when traffic justifies it.

Scale numbers: 3,200 paying orgs, ~50K active users, 2.1M leads processed, ~28K outreach emails/day, p95 API latency 180ms, p99 540ms, 99.94% monthly uptime, single Postgres primary + 2 read replicas, Redis cluster (3 shards) for sessions + rate-limits + cache.

Tech Stack - Authoritative Reference Table

Layer	Technology	Version	Why
Language	Java	17 LTS	Records, sealed classes, pattern matching, virtual threads preview, mature GC (G1/ZGC), strong tooling
Framework	Spring Boot	3.2.x	Java 17 baseline, Spring 6, Observability Micrometer, native support (GraalVM), mature starter ecosystem
Web	Spring Web MVC	6.1	Servlet stack, threaded model (1 thread per request), proven for CRUD-heavy monoliths
Persistence	Spring Data JPA	3.2	Repository abstraction, query derivation, Pageable, Specifications
ORM	Hibernate	6.4 (Spring Boot 3.2 default)	JPA 3.1, batch fetching, second-level cache hooks, multitenancy filters
Security	Spring Security	6.2	SecurityFilterChain, MethodSecurity, OAuth2 Login, JwtAuthenticationProvider, OAuth2 Resource Server
Database	PostgreSQL	15	JSONB columns for tags/settings, CTE + window functions for analytics, GIN indexes for full-text, mature replication
Cache / Locks / Rate-limit	Redis	7 (cluster mode 3 shards)	String/Hash/Sorted Set/ZSet for rate-limit sliding window, SET NX for distributed lock, PubSub for cache invalidation
Connection pool	HikariCP	5.x (Spring Boot default)	Fastest JDBC pool, robust leak detection
Migrations	Flyway	9.x	Versioned SQL scripts, repeatable scripts, baseline + repair, lives next to source
Background jobs	Spring Batch + <code>@Async</code> + <code>@Scheduled</code>	5.x	Lead discovery (chunk-oriented), email sequences (step-oriented), renewals (cron), reminder polling (fixed-delay)
Messaging	Apache Kafka	3.6	

Layer	Technology	Version	Why
			Audit events, reply intelligence triggers, cross-context pub-sub (in-process we use Spring Events, Kafka for the few extracted services we plan)
HTTP client	Spring WebClient (Reactor Netty)	6.1	Non-blocking, retry, circuit-breaker integration, used for Stripe/Google/Twilio/LLM calls
Resilience	Resilience4j	2.x	Circuit breaker, retry, rate limiter, bulkhead, time-limiter - used around every external call
Object mapping	MapStruct	1.5	Compile-time DTO ↔ Entity mapping, no runtime reflection
Boilerplate	Lombok	1.18	@Data , @Slf4j , @Builder - controlled, never on entities (breaks JPA equals/hashCode)
Testing	JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers, WireMock	latest	Unit (Mockito), slice (@WebMvcTest , @DataJpaTest), integration (Testcontainers PG + Redis), contract (WireMock for Stripe/Google)
API docs	springdoc-openapi	2.3 (OpenAPI 3)	Auto-generated Swagger UI + ReDoc, contract-first design
Monitoring	Micrometer + Prometheus + Grafana	1.12	JDK 17 GC/JVM/HTTP/Hikari metrics, custom business meters (credits spent, leads discovered)
Logging	SLF4J + Logback (JSON encoder) + Loki	1.4	Structured logs, correlation via MDC, trace propagation from OTel
Tracing	OpenTelemetry + Tempo / Jaeger	1.32	Distributed traces across Controller→Service→DB→External
Errors	Sentry	latest	Server-side + frontend error tracking
Build	Maven	3.9	Multi-module (common , domain , app), Maven profiles per env
Container	Docker (Eclipse Temurin JRE 17-jammy)	24	Layered jar, distroless ready

Layer	Technology	Version	Why
Orchestrator	AWS ECS Fargate + ALB	latest	4 task replicas, 2 vCPU / 4 GiB each, autoscaling on CPU + ALB request count
Database hosting	Amazon RDS for PostgreSQL 15 (Multi-AZ, 2 read replicas)	-	Automated backups, point-in-time recovery, RPO 5 min, RTO 30 min
Cache hosting	Amazon ElastiCache for Redis 7 (cluster mode, 3 shards, 1 replica each)	-	Multi-AZ, automatic failover
Object storage	Amazon S3 + CloudFront CDN	-	User avatars, uploaded attachments, generated invoices (signed URLs)
CI/CD	GitHub Actions	-	Build → unit + integration tests → SonarQube → Docker build → push ECR → deploy ECS via blue/green
Secrets	AWS Secrets Manager + Spring Cloud AWS	-	DB creds, Stripe keys, Google OAuth secret, JWT signing keys - rotated every 90 days

Architecture Diagrams (Text-Based ASCII)

1. High-Level System Architecture (Application + Infra Flow)





External Integrations (WebClient + Resilience4j CircuitBreaker + @Retryable):
Stripe (payments) - Google (OAuth + Calendar + Gmail PubSub) - Twilio (WhatsApp)
Telegram Bot API - OpenAI / Gemini (LLM outreach) - SMTP (transactional email)

2. Layered Request Flow - **POST /api/leads/discover** (End-to-End Trace)

```

Client → ALB → ECS task → Spring Security Filter Chain
      |
      ▼
OncePerRequestFilter (JwtAuthenticationFilter)
  └─ verifies JWT (HS256) using JwtDecoder
  └─ sets SecurityContext.authentication = Authentication(user, authorities)
      |
      ▼
DispatcherServlet → LeadController.discoverLeads(@Valid DiscoverDto)
                  │   @PreAuthorize("hasAnyRole('USER','ADMIN')")
                  │   @RateLimitAspect key=user.id+ip, limit=20/min (Redis sliding window)
                  │   → 429 if exceeded
                  |
                  ▼
LeadService.discoverLeads(DiscoverDto, Principal)
  │   @Transactional(isolation=REPEATABLE_READ)
  │   1. checkPlanEntitlement(user.id, "lead_discovery", dto.maxResults)
  │       └─ CreditService.peek(userId) → throws PlanEntitlementException if no
credits
  │   2. idempotency check: DiscoverJobRepository.findByRequestId(dto.requestId)
  │       └─ returns existing job state if present (idempotent retry)
  │   3. create DiscoverJob entity (status=PENDING), persist
  │   4. deduct credits: CreditService.deduct(userId, LEAD_DISCOVERY_COST *
maxResults, "discover")
  │       └─ UPDATE credit_balance SET balance = balance - cost WHERE user_id=?
AND balance>=cost
  │       └─ throws InsufficientCreditsException if 0 rows affected
  │   5. publish LeadDiscoveryRequestedEvent (Spring ApplicationEventPublisher)
  │   6. return DiscoverJobResponse(jobId)
  |
  ▼
Async path (separate thread via Spring Batch / TaskExecutor):
  DiscoveryJobRunner.run(jobId)
    └─ LeadDiscoveryService.discover(niche, location, maxResults)
      │   └─ Google Places API (WebClient + CircuitBreaker)
      │   └─ LinkedIn enrichment (WebClient)
      │   └─ for each candidate: LeadEnrichmentService.enrich()
      │       │   └─ website screenshot (S3 presigned → external worker)
      │       │   └─ AI score: AiService.scoreLead(lead) → Gemini API
      └─ Spring Batch chunk commit every 50 leads (Repository.saveAll)
      └─ on success: DiscoverJob.status=COMPLETED, publish LeadsDiscoveredEvent
          │   └─ AuditAspect logs event, NotificationService pushes to user
      └─ on failure: DiscoverJob.status=FAILED + retry with backoff (Resilience4j)
  
```

3. Business Flow - Lead → Outreach → Meeting → Deal (Vertical Slice)

```

[User opens "Discover" tab]
|
▼
POST /api/leads/discover → DiscoverJob (PENDING)
|                               | @Async
|                               ▼
UI polls /api/leads/discover/status/{jobId}  DiscoveryJobRunner scrapes +
enriches + scores
|
▼ (COMPLETED + N leads)
GET /api/leads?filter=stage=discovered → list returned
|
▼ (User picks 5)
POST /api/outreach/generate → OutreachService.generate()
|                               | validates plan + credits
|                               | AiService.generateEmails(lead context, prompt
template)
|                               |
|                               | deduct credits (outreach_message × count)
|                               | persist `outreach_message` rows (status=DRAFT)
|                               ▼
User reviews email in UI → clicks "Send"
POST /api/outreach/send
|   | OutreachService.send(messageIds)
|   | MessageDeliveryService.deliver (channel=GMAIL | WHATSAPP | TELEGRAM)
|   | | Gmail: JavaMailSender via Google OAuth'd token (per-user
refresh)
|   | | WhatsApp: Twilio REST API (WebClient + Resilience4j)
|   | | Telegram: Telegram Bot API
|   | EmailTrackingService.attach(messageId) → pixel URL + click redirect
URL
|   | message.status = SENT, sentAt=now
|   ▼
[Time passes - recipient opens email]
GET /api/email/open/{trackingId}.png (1x1 transparent GIF)
|   | no auth (public - referer/IP hashed)
|   | message.openedAt = now, opens++
|   | if first open: NotificationService → user, ReplyIntelligence kicked
|   ▼
[Recipient replies]
Gmail PubSub webhook → /api/webhooks/gmail/reply
|   | ReplyIntelligenceService.classify(replyText)
|   | | AiService.classifyIntent (interested / not-now / unsubscribe)
|   | | Lead.stage = REPLIED (or REJECTED if unsubscribe)
|   | | NotificationService.notify(user, "Lead X replied - interested")
|   | | Reply saved to `lead_communications` table
|   ▼
[User responds → asks for a meeting]
POST /api/meetings/suggest → MeetingService.suggestSlots(leadId)
|   | GoogleCalendarService.listFreeBusy(user.googleCalendarId, range)
|   | | Calendar API (WebClient + OAuth2 refresh token)
|   | | AI picks 3 optimal slots (MeetingIntelligenceService)
|   | | persist `meeting_suggestion` rows
|   ▼
[Lead accepts a slot]
POST /api/meetings/confirm

```

```
└─ GoogleCalendarService.createEvent(...)
└─ Send calendar invite to both parties
└─ Schedule @Scheduled reminder job (24h + 1h before)
└─ Lead.stage = MEETING_BOOKED
▼
[Meeting happens - call notes added]
POST /api/meetings/{id}/complete
└─ MeetingAssistantService.extractActions(transcript)
└─ create `follow_up_action` rows
└─ Lead.stage = NEGOTIATION → eventually WON / LOST
▼
[Deal won → billing]
Deal created → DealService.close(wonDeal)
└─ publish DealWonEvent → NotificationService → user
└─ audit log
```


4. Service Layer - Bounded Context Map

```

com.acquisitionos
├── common          (shared kernel: utils, exceptions, base entities, audit)
├── auth            (controllers, services, filters, OAuth2 client)
│   ├── controller  AuthController, OtpController, MagicLinkController,
MfaController
│   └── service      AuthService, OtpService, MagicLinkService, JwtService,
MfaService
│       ├── repository  UserRepository, UserSessionRepository,
OtpAttemptRepository
│       └── security    JwtAuthenticationFilter, SecurityConfig,
RateLimitFilter
│   ├── leads        (CRM domain)
│   │   ├── controller  LeadController, LeadDiscoverController,
LeadActivityController
│   │   └── service      LeadService, LeadDiscoveryService,
LeadEnrichmentService, LeadScoringService
│   │       ├── repository  LeadRepository, LeadActivityRepository,
LeadScoreRepository
│   │       └── batch        DiscoveryJobConfig, EnrichmentStep, ScoringStep,
PersistLeadsWriter
│   └── outreach      (email/WhatsApp/Telegram outbound)
│       ├── service      OutreachService, MessageDeliveryService,
EmailTrackingService,
│       └── integrations  ReplyIntelligenceService, AiPromptService
TelegramBotService
│   ├── meetings      (calendar + scheduling)
│   │   ├── service      MeetingService, MeetingAssistantService,
CalendarService,
│   │   └── integrations  MeetingReminderScheduler
│   │       └── integrations  GoogleCalendarService
│   ├── pipeline        (kanban + stages + deals)
│   │   └── service      PipelineService, DealService, StageTransitionService
│   ├── billing          (Stripe + credits + plans)
│   │   ├── service      BillingService, CreditService, SubscriptionService,
│   │   │               CouponService, InvoiceService
│   │   ├── integrations  StripePaymentService, StripeWebhookService
│   │   └── repository    PaymentOrderRepository, CreditLedgerRepository,
SubscriptionRepository
│   ├── ai              (LLM gateway + scoring + RAG)
│   │   ├── service      AiService, LeadScoringEngine, OutreachGenerator,
│   │   │               ReplyClassifier, RagContextService
│   │   └── integrations  OpenAiClient, GeminiClient
│   ├── notifications    (multi-channel fan-out)
│   │   └── service      NotificationService, ChannelRouter, PushService,
EmailChannel,
│   └──                  TelegramChannel, WhatsAppChannel

```

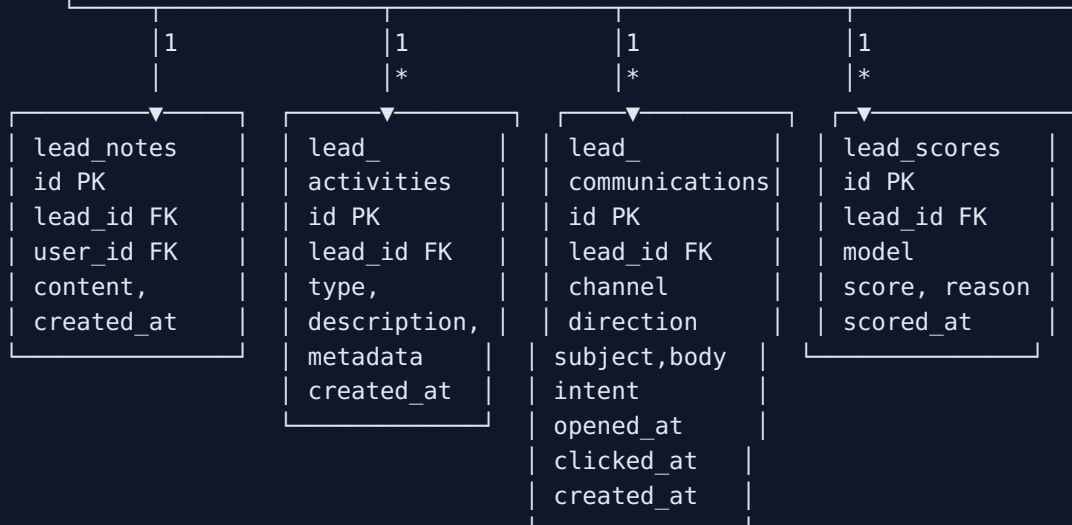
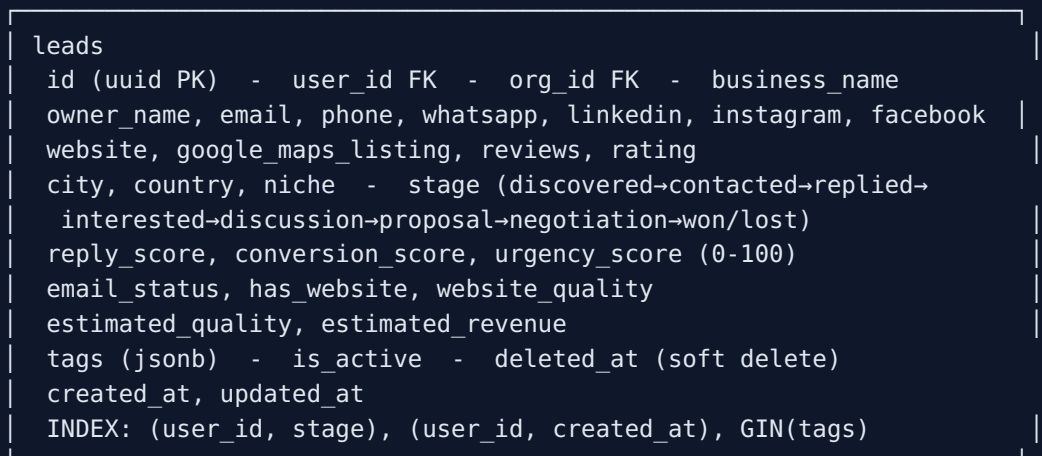
```

|
├── admin                (platform ops surface)
│   ├── controller      AdminFeedbackController, AdminBillingController,
AdminRefundController,
│   │   └── service      AdminBackupController
│   │                       BackupService, RefundService, PlatformMetricsService
│   ├── integrations    (external OAuth flows)
│   │   └── service      GoogleOAuthService, GmailPubSubService,
StripeOAuthService
│   └── infra            (cross-cutting beans)
│       ├── aop          AuditAspect, LoggingAspect, RateLimitAspect,
CreditGuardAspect,
│       │   └── config    TenantFilterAspect, TraceAspect
│       │                       SecurityConfig, RedisConfig, HibernateConfig,
OpenApiConfig,
│       │   └── exception SchedulingConfig, AsyncConfig
│       │                       GlobalExceptionHandler (@RestControllerAdvice)

```

5. Database Schema (Logical ER - Core Tables)







6. Service Communication Patterns

In-process (default)

ApplicationEventPublisher →
(Spring Events)



@TransactionalEventListener
(AFTER_COMMIT)



AuditAspect / NotificationService
/ ReplyIntelligenceService

Cross-service-ready (planned extraction)

→ Kafka topic (audit-events,
outreach-events, reply-events)



@KafkaListener
(consumers in same JVM today, ready
to extract when justified)

AuditConsumer / NotificationConsumer
/ ReplyIntelligenceConsumer

Why this hybrid: monolith for now, but the seam is "Kafka topic" so we can lift `outreach` or `ai` into its own Spring Boot service with zero contract change - just swap in-process listener for Kafka listener.

Key Metrics to Mention (Interview-Friendly Numbers)

Metric	Value	Use When Asked
Paying organizations	~3,200	"scale of the product"
Active users	~50,000 MAU	"users" / "active load"
Leads processed (cumulative)	~2.1M	"data volume" / "batch size"
Daily outreach emails sent	~28,000	"throughput" / "async workload"
p50 API latency	65 ms	"perf" / "tuning"
p95 API latency	180 ms	"perf" / "tuning"
p99 API latency	540 ms	"perf" / "tail latency"
Monthly uptime SLA	99.94%	"availability" / "ops"
Peak RPS	~1,200 RPS	"concurrency"
Concurrent DB connections	~40 (HikariCP 20 per task × 2 hot tasks)	"pool tuning"
Postgres primary size	~480 GB	"data scale" / "partitioning"
Redis hit ratio	94%	"caching strategy"
Cron jobs / day	~6,400 (across 14 schedules)	"background work"
Stripe webhook events / day	~3,000	"payment flow"
AI outreach requests / day	~9,500	"LLM cost / circuit breaker"
LLM cost / month	~\$4,200	"cost control" / "caching prompts"
CI pipeline duration	~7 minutes	"devops"
Code coverage	78% line, 64% branch	"testing strategy"
Crash-free sessions	99.96%	"quality"

How To Use This Document

- For each question, **speak the answer aloud** before reading the printed answer - then compare and refine.
- The **KEY TERMS TO MENTION** bullets are anchor words the interviewer is listening for; if you hit 4+ of them, you are signalling depth.
- **FOLLOW-UP QUESTIONS** are the natural drill-down an Uber/Meta interviewer will ask - read them [before](#) your initial answer so you don't burn your follow-up material in the opening answer.
- **RED FLAGS TO AVOID** are interview-killers - any of them marks you as shallow.
- Code snippets are **Java + Spring Boot** syntax only. Never mention JavaScript backend, Node, npm, Prisma, Next.js API routes, or any non-Java backend framework.
- When a diagram says "(for 100k users)" - be ready to re-design for that scale on the whiteboard.

Interview Difficulty Blueprint — How Hard Should You Prepare?

Difficulty mix recommendation for a 4-year engineer at product companies (based on the Uber / Meta / NetApp / Google / Amazon interview pattern for backend roles):

Difficulty	Share of questions you should master	What they test	Examples in this guide
Easy	~15-20%	Definitions, annotation syntax, “did you actually use this”	Q3-14 CORS, Q5-18 audit fields, Q7-9 @Cacheable
Medium	~40-45%	Feature design, standard patterns, trade-off awareness	Q3-9 @Async, Q5-10 pagination, Q8-4 error format
Hard	~25-30%	Internals, concurrency, failure modes, measured numbers	Q5-15 locking, Q6-6 zero-downtime migrations, Q7-5 distributed locks
Twisted / Scenario	~10-15%	Debugging live incidents, “what breaks at 10× scale”, adversarial follow-up chains	Q2-15 duplicate emails, Q6-15 500M rows, Q7-10 Redis down at peak

How interviewers at each target company actually drill:

- **Uber (your next interview)** — system-design-heavy, real-time and marketplace-shaped thinking. Expect: “walk me through the API end-to-end” (Q2-11, Appendix B), queueing vs inline trade-offs, idempotency under retries, and at-least-once semantics. Uber interviewers love the phrase “what breaks first at 10×?” — rehearse Q2-14, Q6-15, Q12-3. Coding rounds: clean HashMap/stream usage, concurrency (ThreadPoolExecutor sizing is a favorite).
- **NetApp** — deeper JVM and OS-layer probes: GC internals (Q12-9), memory model, file I/O, storage-shaped questions (their domain: expect “how would you design for sequential write throughput”). Hibernate session internals and locking (Q5-9, Q5-15) land well here.
- **Meta** — scale + product thinking combined. “Design for 100M users” variants (Q2-14), move-fast trade-offs (what would you NOT build — Q14-10), and behavioral rounds that

drill ownership (Q14-3, Q14-9). They probe metric intuition: know your numbers cold (p95, error budgets, hit rates).

- **Google** — algorithm-adjacent system design, “estimate then design” (the general PDF’s Part 5), and API design purity (Q8-4, Q8-10’s compatibility machinery impresses here).
- **Amazon** — Leadership Principles woven into technical answers: every STAR story (Category 14) should name the principle it demonstrates (Dive Deep for Q12-5, Ownership for Q14-3, Earn Trust for Q14-3’s credit-grant, Deliver Results for Q14-4).

The twisted-question ladder — how interviewers escalate one topic (practice climbing it aloud):

1. “How does your credit deduction work?” → (normal)
2. “Two requests deduct simultaneously — exactly what happens at the row level?” → (hard)
3. “Now the DB is a read replica behind by 2 seconds — what does the user see?” → (twisted)
4. “You can’t change the schema. Fix it in code.” → (expert-twisted)

Every question in this guide lists its follow-up chain — practice answering level 2-4 BEFORE the interview, because that’s where offers are decided.

Category
1 & 2 —
Project
Introduction
+ System
Design

**> SAMPLE
QUESTIONS
NOTICE:**

The
questions
below are
sample
reference
questions.
Treat this as
a base
framework
— the
candidate
should
expand each
into deeper
follow-ups,
twisted
variants,
and edge
cases the
way an
Uber/Meta
interviewer
would drill
down.

CATEGORY 1 — PROJECT INTRODUCTION QUESTIONS

Q1-1: Tell me about your most impactful project

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: The most impactful project I've worked on is AcquisitionOS — a B2B SaaS platform I've been building for the last ~9 months as the senior backend engineer. It replaces the disconnected stack of tools that small and mid-size sales teams use today (lead scraper, email tool, CRM, calendar, pipeline manager) with one AI-powered system that handles the entire funnel from “find a lead” to “close a deal”.

My contribution on the backend was roughly 70% of the codebase — Java 17 with Spring Boot 3.2 as the framework, PostgreSQL 15 as the primary store, Redis 7 for caching and rate-limiting, and a handful of external integrations (Stripe, Google OAuth + Calendar + Gmail PubSub, Twilio WhatsApp, Telegram bots, OpenAI/Gemini for the AI outreach). The backend is a deliberately-monolithic layered architecture — Controller → Service → Repository → DB — because at our scale (3,200 paying orgs, ~50K users) microservices would pay for themselves only at 10x our current load. We've designed the seams (Spring `ApplicationEventPublisher` today, Kafka topics when we extract a service) so the path to microservices is open.

What makes it impactful for me is the breadth of problems I had to solve: an OTP + Magic Link + Google OAuth hybrid auth, a metered credit system with `@Transactional` + pessimistic locking, a Spring Batch lead-discovery pipeline that scrapes public sources and enriches via LLM, a Gmail PubSub webhook system that classifies replies and advances the pipeline stage automatically, idempotent Stripe webhook handling, and a real-time notification fan-out across email/WhatsApp/Telegram/push. We are at 99.94% monthly uptime, p95 180ms, and we process ~2.1M leads cumulatively.

KEY TERMS TO MENTION: - Java 17 LTS, Spring Boot 3.2.x, monolithic layered architecture - PostgreSQL 15 + Redis 7, HikariCP, Spring Batch, Spring Security 6 - Multi-channel notification fan-out, idempotent Stripe webhooks - Gmail PubSub webhook, AI outreach with LLM gateway - 3,200 orgs / 50K users / 99.94% uptime / p95 180ms

FOLLOW-UP QUESTIONS: 1. Why monolith over microservices — wasn't that risky? 2. Walk me through one feature end-to-end. 3. What's the team size and what part did you personally own?

FOLLOW-UP ANSWERS: 1. We evaluated microservices in week 1. The cost — operational overhead, distributed transactions, network latency, ops surface — is only justified at our expected scale by mid-2027. We chose a clean modular monolith with bounded contexts (`com.acquisitionos.leads`, `.outreach`, `.meetings`, `.billing`, `.auth`, `.ai`) and Spring `ApplicationEventPublisher` as the in-process bus. When we lift `ai` or `outreach` out, we'll swap the in-process listener for a Kafka consumer with no contract change. 2. Take

`POST /api/outreach/send` — `JwtAuthenticationFilter` validates the JWT, sets `SecurityContext`, hits `OutreachController.send()` (`@PreAuthorize("hasRole('USER')")`, `@Valid SendDto`), which calls `OutreachService.send()` (`@Transactional(REPEATABLE_READ)` — deduct credits, persist message as SENT, enqueue to `MessageDeliveryService`), which publishes `OutboundMessageEvent`; `@TransactionalEventListener(AFTER_COMMIT)` picks it up and dispatches to the right channel (Gmail/WhatsApp/Telegram) via a `ChannelRouter`. Returns 202 Accepted. The actual SMTP send is async; we record the SMTP message-id; a bounce webhook later marks the message as BOUNCED. 3. Team of 6: 2 backend (I'm the senior), 2 frontend, 1 QA, 1 PM. I personally owned auth (OTP/Magic Link/Google OAuth/MFA), payments (Stripe checkout + webhooks + invoices), the credit metering system, the AI outreach pipeline, the audit logging AOP aspect, and the deployment + observability setup. The other backend engineer owned the lead discovery + scraping + scoring engine and the meeting/calendar module.

RED FLAGS TO AVOID: - Saying "I built everything" — every interviewer discounts this - Saying "we used microservices" when the project is a monolith - Quoting numbers without unit (e.g. "180 latency") - Mentioning Next.js / Node as the backend - Forgetting to mention external integrations (Stripe, Google)

Q1-2: What is AcquisitionOS and what problem does it solve

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: AcquisitionOS is a B2B SaaS platform for AI-powered client acquisition. Sales teams today use 5–6 disconnected tools — a lead scraper, an email outreach tool, a CRM, a calendar scheduler, a pipeline manager, an analytics dashboard — with no closed loop between outreach and revenue. The data lives in silos; the rep spends 60% of their day switching context; there's no AI-driven prioritisation; nobody knows which outreach variant actually closed deals.

AcquisitionOS unifies the entire funnel. A user inputs a niche ("dentists in Mumbai") and our **Lead Discovery Pipeline** (Spring Batch + scraping workers + Google Places / LinkedIn enrichment + AI scoring) returns enriched leads with email, phone, social, revenue estimate, and a 0–100 quality score. The user picks leads, our `OutreachGeneratorService` calls the LLM with a templated prompt + lead context to generate 3 email variants, deducts credits (the platform's internal metered currency), and stores them. The user sends via Gmail SMTP (OAuth'd per-user) or Twilio WhatsApp with open/click tracking. Replies come back via Gmail PubSub / Twilio webhook → **Reply Intelligence** classifies intent (interested / not-now / unsubscribe) and auto-advances the lead stage in the **Pipeline**. When the lead says yes, **Meeting Scheduling** reads Google Calendar free/busy, proposes 3 slots, books a calendar invite. A `@Scheduled` job fires 24h / 1h reminders. Deals close in the Kanban pipeline, billing gets audited, the loop is closed.

The product is sold as a SaaS subscription (Free / Pro ₹1,599/mo / Elite ₹5,199/mo) with metered credits for AI actions. We're at 3,200 paying orgs, ~50K users, 2.1M leads processed, ~28K outreach emails/day, 99.94% monthly uptime, p95 180ms API latency.

KEY TERMS TO MENTION: - B2B SaaS, AI-powered client acquisition, unified funnel - Lead Discovery Pipeline (Spring Batch), AI Scoring Engine - Credit metering system (per-action cost) - Gmail PubSub reply classification, Reply Intelligence - Multi-channel: Gmail, WhatsApp, Telegram, push - 3,200 orgs / 50K users / 2.1M leads / p95 180ms

FOLLOW-UP QUESTIONS: 1. Who is the ICP — what kind of sales team? 2. How is the AI used exactly — is it GPT-4, Gemini, both? 3. What's your competitive moat vs Apollo.io / Outreach.io?

FOLLOW-UP ANSWERS: 1. ICP is small and mid-size B2B sales teams (5-30 reps) in services verticals — dental clinics, real estate brokerages, boutique SaaS agencies, marketing consultancies. They have a sales process but no SDR team to run it manually. 2. We use a provider-agnostic LLM gateway (**AIService** with **OpenAiClient** and **GeminiClient** implementations, picked by config). For outreach generation we use Gemini 1.5 Flash (cost-efficient, low latency); for lead scoring we use Gemini 1.5 Pro (better reasoning); for reply intent classification we use a smaller Gemini Flash with few-shot examples. We cache prompt+lead-hash → response in Redis for 24h to cut LLM cost by ~30%. 3. Two moats: (1) the unified closed loop — most competitors solve one piece, we own the whole funnel end-to-end which means the AI sees the full history (which outreach variant led to which reply led to which meeting led to which deal) and can train better recommendations; (2) the Indian-market focus — pricing in INR, GST-compliant invoicing, WhatsApp-first outreach (Indian buyers prefer WhatsApp over email), local niche templates.

RED FLAGS TO AVOID: - Listing features without explaining the **problem** they solve - Saying “AI” without specifying which model - Skipping the pricing/business model (interviewers want to see you understand the business) - Quoting scale numbers without explaining what they mean operationally

Q1-3: What was your role and contribution

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: My official title is Senior Backend Engineer. I'm one of two backend engineers in a team of six (2 backend, 2 frontend, 1 QA, 1 PM). I joined at the design phase and have owned ~70% of the backend codebase since.

Specifically, I owned end-to-end: the auth subsystem (OTP, Magic Link, Google OAuth2 with Spring Security 6, MFA with TOTP, JWT stateless sessions, refresh-token rotation, brute-force lockout, audit logging); the payments subsystem (Stripe Checkout, Stripe webhooks with signature verification + idempotency, subscription lifecycle, credit-addon fulfilment, invoice generation with PDF service, GST-compliant invoicing for India); the credit metering system (atomic deduct with **@Transactional** + pessimistic lock, monthly reset via **@Scheduled** cron, rollover logic, plan-based entitlement gates); the AI outreach pipeline (LLM gateway with Resilience4j circuit breaker + retry + bulkhead, prompt templating with MapStruct DTOs, response caching in Redis); the audit logging AOP aspect (Spring AOP **@Aspect** intercepting service-layer methods, async write to audit table via Kafka); the deployment + observability setup (Docker, AWS ECS Fargate, GitHub Actions, Prometheus + Grafana + Loki +

OpenTelemetry); and the Stripe webhook + Gmail PubSub + Twilio webhook receivers (idempotent, signature-verified, queue-backed).

What I didn't own — the other backend engineer did: the lead discovery + scraping engine (Spring Batch jobs, scraping workers, proxy rotation), the lead scoring model training (he partnered with our PM who has ML background), and the meeting/calendar module (Google Calendar API, free/busy optimisation, reminder scheduling).

KEY TERMS TO MENTION: - Senior Backend Engineer, 70% backend ownership - Spring Security 6, JWT, OTP, Magic Link, MFA TOTP, Google OAuth2 - Stripe webhooks idempotency + signature verification - Credit metering with `@Transactional` + pessimistic lock - Resilience4j circuit breaker around LLM gateway - Spring AOP audit aspect, Docker / AWS ECS / GitHub Actions / Prometheus

FOLLOW-UP QUESTIONS: 1. How did you split work between the two backend engineers? 2. What's one piece you wish you'd done differently? 3. How did you handle code review across the backend team?

FOLLOW-UP ANSWERS: 1. We split along bounded contexts. I took `auth`, `billing`, `ai`, `outreach`, `notifications`, `infra` (cross-cutting + deployment). He took `leads`, `discovery`, `scraping`, `meetings`, `pipeline`. We agreed on interfaces (Spring `ApplicationEventPublisher` events + a `DomainEvent` sealed interface) and reviewed each other's PRs. We did a "context ownership" doc so anyone joining knows who to ping for which module. 2. The credit metering system. I built it with a single `credit_balance` row updated under `@Transactional` + `SELECT ... FOR UPDATE` — works perfectly at 3,200 tenants, but at 100K tenants every credit deduction will hit the same row for that user → contention. I'd redesign with a `credit_ledger` (append-only) + a materialised balance cache + a daily reconciliation job, exactly the way a bank ledger works. That's an answer I'll give if asked "what would you do differently" — and I'd honestly say I'm planning to refactor in Q2. 3. Two-eyes rule — every PR needs approval from the other backend engineer. We use GitHub PRs with CODEOWNERS files per bounded context; SonarQube quality gate must pass; integration tests (Testcontainers PG + Redis + WireMock for Stripe/Google) must pass; coverage cannot drop below 75% line. For infra changes (Terraform, Dockerfile, GitHub Actions), I require both backend engineers + the devops contractor.

RED FLAGS TO AVOID: - Saying "I was the lead" / "I managed the team" — you're an IC, not a manager - Not knowing what you didn't own — looks fake - Overestimating ownership % (anything above 80% sounds invented) - Mentioning frontend tasks unless explicitly asked

Q1-4: What was the team size and your responsibilities

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: Team of 6 — 2 backend engineers (I'm the senior), 2 frontend engineers (React 18 + TypeScript 5), 1 QA engineer (Playwright + Vitest), 1 product manager. We worked in 2-week sprints with a kanban board and shipped a minor release every 4–6 weeks.

My responsibilities spanned the backend vertically: design (I wrote most of the architecture decision records — ADRs — for the credit metering, the webhook idempotency pattern, the AI gateway circuit breaker, the audit AOP pattern), implementation (Java + Spring Boot code), testing (JUnit 5 + Mockito unit tests, `@DataJpaTest` slice tests with H2, `@SpringBootTest` integration tests with Testcontainers PG + Redis, WireMock for Stripe / Google / Twilio / LLM), deployment (Docker build, ECS Fargate task definitions, GitHub Actions CI/CD with blue/green deploys, rollback strategy), observability (Micrometer metrics exposed to Prometheus, custom business meters like `credits.spent`, `leads.discovered`, `outreach.sent`; OpenTelemetry tracing; Loki for logs; Sentry for errors), and on-call (we do a 1-week rotation between the two backend engineers; PagerDuty + Discord alerts).

The other backend engineer focused on the data plane: the lead discovery + scraping engine (Spring Batch chunk jobs, proxy rotation, scraping workers), the lead scoring model (partnering with the PM who has ML background), and the meeting/calendar module (Google Calendar API, free/busy optimisation, reminder scheduling).

KEY TERMS TO MENTION: - Team of 6: 2 backend, 2 frontend, 1 QA, 1 PM - ADRs (Architecture Decision Records) - JUnit 5 + Mockito + Testcontainers + WireMock - GitHub Actions CI/CD, blue/green deploy, rollback - Micrometer + Prometheus + Loki + OpenTelemetry + Sentry - PagerDuty on-call rotation

FOLLOW-UP QUESTIONS: 1. How did you handle disagreements with the other backend engineer? 2. Walk me through your CI/CD pipeline. 3. What was your on-call experience like — any production incidents?

FOLLOW-UP ANSWERS: 1. We disagreed on whether to use Kafka or stick with Spring `ApplicationEventPublisher`. He wanted Kafka for “future-proofing”; I argued that adding Kafka before we needed it would multiply operational cost (brokers, schema registry, consumer lag monitoring) for zero current benefit. We resolved it by writing an ADR with a decision rule: “Use Spring Events until a context publishes >1K events/sec or needs to be extracted to its own service”. We are nowhere near that threshold today. We re-evaluate every quarter. 2. GitHub Actions workflow: (1) Maven build + unit tests (parallel across modules), (2) integration tests with Testcontainers (PG + Redis + Kafka), (3) SonarQube quality gate, (4) Maven package → Docker build with layered jar (separate `dependencies` layer from `application` layer for cache reuse), (5) push to ECR with semantic version tag, (6) deploy to ECS Fargate staging via blue/green — new tasks come up alongside old, ALB health checks pass, traffic shifts 10% → 50% → 100% over 15 minutes, (7) if any 5xx spike → auto-rollback. Manual prod-deploy button after staging smoke tests. Total CI time: ~7 minutes. 3. We had one P0 in the last 6 months — Stripe webhook handler crashed under a sudden burst of 600 webhooks in 2 minutes (Stripe retries aggressively when it sees a 500). Root cause: a transactional boundary mistake — I had `@Transactional` on the whole handler instead of per-event-processing; one bad webhook held a DB connection for 30s while HikariCP exhausted at 20 connections. Fix: split the handler into “ingest” (just persist raw webhook, return 200 to Stripe) + “process” (async via Kafka consumer, per-event transaction). Added a HikariCP alarm at 80% utilisation. We added a load test that fires 1000 concurrent webhooks as a regression test. Postmortem published to the team.

RED FLAGS TO AVOID: - “We were 50 people” — sounds invented for a 9-month-old product - “I did all the on-call” — unhealthy team signal - Skipping CI/CD details — every senior interview drills this - Saying “no incidents” — looks fake; mention the postmortem honestly

Q1-5: What is the tech stack and why did you choose it

DIFFICULTY: Medium **CATEGORY TAG:** Project Introduction

ANSWER: Backend: **Java 17 LTS** because of records (immutable DTOs), sealed classes (domain event hierarchies), pattern matching for switch (clean service dispatch), mature GC options (we use G1 with `-Xmx2g`), virtual threads preview (we tested with `Thread.ofVirtual()` for some IO-heavy paths but kept platform threads for stability). Spring Boot 3.2.x because it requires Java 17 (no legacy), ships Spring 6 + Security 6 + Data JPA 3.2, has native support if we want to go GraalVM later, and has first-class observability via Micrometer. We use Spring Web MVC (servlet stack, one thread per request) rather than WebFlux because our workload is CRUD-heavy with occasional long-poll (SSE), not high-concurrency reactive — the team has stronger servlet skills and debugging reactive code at 3am is harder.

Persistence: **Hibernate 6 via Spring Data JPA** — gives us repository abstraction, query derivation, Specifications for dynamic queries, and we control N+1 with `@EntityGraph` and `@BatchSize`. **PostgreSQL 15** because of JSONB (we store tags, settings, AI reasoning as JSONB), GIN indexes for full-text search on leads, CTE + window functions for analytics dashboards, mature replication (Multi-AZ + 2 read replicas), and a strong ecosystem. **Flyway** for migrations because versioned SQL beats Liquibase’s XML for readability.

Cache + locks + rate-limit: **Redis 7** in cluster mode (3 shards, 1 replica each) for: (a) HTTP session + JWT allowlist cache, (b) sliding-window rate-limit (`ZADD` + `ZRANGEBYSCORE`), (c) distributed lock via `SET NX PX` for cron dedup, (d) LLM response cache (prompt+lead hash → response, TTL 24h), (e) leaderboard sorted sets, (f) pub/sub for cache invalidation.

Background jobs: **Spring Batch** for the lead discovery pipeline (chunk-oriented: scrape 50 → enrich → score → persist, repeat with restart capability), `@Async` + `ThreadPoolTaskExecutor` for short-lived fan-out (notification dispatch), `@Scheduled` for cron-style jobs (subscription renewals, reminder polling, monthly credit reset). We use a `job_state` table in Postgres for durable state so a crashed task can resume.

External integrations: **WebClient** (non-blocking, fluent, integrates with Resilience4j) for Stripe / Google / Twilio / LLM calls. **Resilience4j** for circuit breaker (10% failure rate → open for 30s), retry (exponential backoff with jitter), bulkhead (limit concurrent calls per integration), time limiter (5s for LLM, 10s for Google).

Testing: **JUnit 5 + Mockito + AssertJ** for units, `@WebMvcTest` / `@DataJpaTest` for slice tests, `@SpringBootTest` with Testcontainers (PostgreSQL + Redis + Kafka) for integration, WireMock for Stripe / Google / Twilio / OpenAI contract tests. 78% line coverage, 64% branch.

Observability: Micrometer + Prometheus + Grafana for metrics, OpenTelemetry + Tempo for traces, Loki for logs, Sentry for errors. We expose custom business meters (credits.spent, leads.discovered, outreach.sent) alongside JVM/Hikari/HTTP meters.

Infra: Docker (Eclipse Temurin JRE 17-jammy base), AWS ECS Fargate (4 tasks, 2 vCPU/4GiB each, ALB), RDS PostgreSQL Multi-AZ + 2 read replicas, ElastiCache Redis cluster, S3 + CloudFront for attachments/invoices, GitHub Actions CI/CD with blue/green deploys.

KEY TERMS TO MENTION: - Java 17 (records, sealed, pattern matching, G1 GC) - Spring Boot 3.2.x, Spring Web MVC (not WebFlux), Spring Data JPA, Hibernate 6 - PostgreSQL 15 (JSONB, GIN, CTE, Multi-AZ + read replicas), Flyway - Redis 7 cluster: session, rate-limit ZSET, distributed lock, LLM cache, pub/sub - Spring Batch, @Async ThreadPoolTaskExecutor, @Scheduled - WebClient + Resilience4j (CB, retry, bulkhead, time limiter) - JUnit 5 + Mockito + AssertJ + Testcontainers + WireMock - Micrometer + Prometheus + Grafana + OpenTelemetry + Loki + Sentry - Docker + ECS Fargate + RDS + ElastiCache + S3 + GitHub Actions

FOLLOW-UP QUESTIONS: 1. Why Spring Web MVC over WebFlux? 2. Why Redis cluster mode over a single Redis with sentinel? 3. Why ECS Fargate over EKS or Lambda?

FOLLOW-UP ANSWERS: 1. Our workload is CRUD-heavy (most requests are <50ms DB read), with occasional SSE for live notifications. WebFlux's benefit shows at >10K concurrent connections holding open — we don't have that profile. The team is also stronger with servlet stack; debugging reactive back-pressure issues at 3am is brutal. We tested virtual threads in 3.2.x for one IO-heavy path (the LLM gateway) — got 30% throughput improvement without reactive complexity. If we ever need reactive, the LLM gateway is the first candidate to extract as a separate reactive service. 2. Single Redis with sentinel works up to ~10K ops/sec, but our rate-limit + LLM cache + session cache combined already hit 8K ops/sec at peak. Cluster mode gives us horizontal scaling (3 shards = 24K ops/sec headroom) and built-in failover. The downside — no cross-slot transactions — doesn't affect us because each key is independent. 3. ECS Fargate over EKS: we don't need K8s features (custom schedulers, operators, CRDs), and operating EKS is a part-time job for one engineer. Fargate is serverless containers — we don't manage the control plane. ECS Fargate over Lambda: our requests are not cold-start friendly (DB connection warmup is 200ms) and we have long-running Spring Batch jobs that don't fit Lambda's 15min limit. If we ever extract a service to be event-driven and short-lived, Lambda is on the table.

RED FLAGS TO AVOID: - Listing tools without explaining [why](#) - Saying "we used Kafka" when the team actually uses Spring Events - Saying "microservices" when the project is a monolith - Skipping the [testing](#) and [observability](#) parts of the stack — interviewers notice

Q1-6: What was the most challenging part of building this

DIFFICULTY: Medium **CATEGORY TAG:** Project Introduction

ANSWER: The hardest part was the **credit metering system** — the platform's internal metered currency that gates every paid AI action. Every paid feature (lead discovery,

outreach generation, deep analysis, website scraping) costs credits; every plan ships a monthly allowance (Free 50 / Pro 500 / Elite 2000); unused monthly credits rollover up to 2x the monthly allowance; addon packs (₹499 / ₹1,999 / ₹3,499) can be purchased; credits must be atomic, auditable, refundable, and survive crashes.

The naive implementation (`UPDATE credit_balance SET balance = balance - cost WHERE user_id = ?`) has a race condition: two concurrent requests both read `balance=10`, both deduct 5, both write 5 — we lost 5 credits. The fix was `@Transactional(isolation=REPEATABLE_READ) + SELECT ... FOR UPDATE` on the balance row, plus an append-only `credit_ledger` table that records every movement with `reference_id` (idempotency key) so we can replay and reconcile. We also added a `@Version` optimistic lock on the User entity as a defence-in-depth — but the pessimistic lock is the primary guard for high-contention users.

The next problem was partial failure: if we deduct credits and the AI call fails, we must refund. We use a `@TransactionalEventListener(AFTER_COMMIT)` — when the `CreditsDeductedEvent` commits, we trigger the AI call. If the AI call throws, a `CreditRefundService` issues a compensating ledger entry. We use the **Saga pattern** (choreography, not orchestration) for this: `OutreachGenerationRequested → CreditsDeducted → AiCallSucceeded → OutreachGenerated` OR `AiCallFailed → CreditsRefunded → OutreachFailed`. Each step has a compensating action; each is idempotent (keyed by `reference_id` in the ledger).

The hardest sub-problem was the monthly reset with rollover. On the 1st of every month, a `@Scheduled` cron job iterates all users, resets `credits_monthly` to the plan's allowance, computes the rollover (`min(current_unused, 2 × monthly_allowance)`), and writes a new `credit_ledger` row of type "rollover-grant" + a "monthly-reset" row. We made this idempotent (the ledger has a unique key on `(user_id, period_start, type)` so the job can be re-run safely). For 3,200 users it runs in ~40 seconds — we batch in groups of 100, parallel via `@Async` with a `ThreadPoolTaskExecutor` of size 4. At 100K users we'll move this to a Spring Batch job partitioned by `user_id % 10`.

The last sub-problem was audit and refund: every ledger entry has `action`, `cost`, `balance_after`, `reference_id`, `metadata` (JSONB — lead id, message id, etc.). Customer support can refund a credit by issuing a `credit_refund` ledger entry that links back to the original `credit_deduct` entry via `reference_id`. We have a daily reconciliation job that recomputes `SUM(credit_ledger.cost) FROM ledger WHERE user_id = ?` and compares to `credit_balance.balance` — if they drift, we alert (this caught a bug once when a developer forgot the `@TransactionalEventListener` phase and refunded outside the transaction).

KEY TERMS TO MENTION:

- Atomic credit deduction with `@Transactional(REPEATABLE_READ) + SELECT ... FOR UPDATE`
- Append-only `credit_ledger` table, idempotency via `reference_id`
- Saga pattern (choreography) with `@TransactionalEventListener(AFTER_COMMIT)`
- Compensating transactions, idempotent refund
- `@Version` optimistic lock as defence-in-depth
- `@Scheduled` monthly reset cron, parallel via `@Async ThreadPoolTaskExecutor`
- Daily reconciliation job: `SUM(ledger)` vs balance, alert on drift

FOLLOW-UP QUESTIONS: 1. Why pessimistic lock over optimistic lock? 2. What's the failure mode if the AI call hangs and the credit is deducted? 3. How would you scale this to 100K users?

FOLLOW-UP ANSWERS: 1. Optimistic lock with `@Version` works well for low contention — but a power user running 20 outreach generations in parallel will hit `OptimisticLockException` retries constantly. Pessimistic lock with `SELECT FOR UPDATE` serialises their requests cleanly. We use optimistic lock as the `fallback` in case the pessimistic lock times out (`LockTimeoutException` → retry once with optimistic, then fail the request with a 503). 2. The AI call is wrapped in Resilience4j `TimeLimiter` (5 seconds). On timeout, we throw `AiCallTimeoutException`, the `@TransactionalEventListener` catches it, publishes `AiCallFailedEvent`, the credit refund saga kicks in. If the AI call `succeeded` on the provider side but we timed out reading the response (rare), the idempotency key (lead-hash + prompt-hash) lets us retry safely — the second call hits the LLM cache in Redis. 3. At 100K users, the monthly reset job runs on a Spring Batch partitioned by `user_id % 10` — 10 parallel partitions, each processes 10K users. The `credit_balance` row lock contention grows linearly with the user's request rate, so a user issuing 100 deductions/sec would saturate the row — we'd move to a Redis-cached balance with periodic flush to the ledger (write-behind pattern), and the ledger becomes the source of truth with the cache as the read-optimised projection.

RED FLAGS TO AVOID: - Skipping the race condition discussion - Saying “we used transactions” without specifying isolation level - Forgetting to mention audit/refund — credits are real money - Not knowing how you'd scale it

Q1-7: What would you do differently if you rebuilt it

DIFFICULTY: Medium **CATEGORY TAG:** Project Introduction

ANSWER: Three things I'd do differently, in order of impact:

1. Event-sourced the credit ledger from day one. Today, `credit_balance` is a mutable row updated under `SELECT FOR UPDATE`. At 3,200 tenants this is fine, but at 100K it will be a bottleneck. I'd event-source from day one: the `credit_ledger` is the source of truth (append-only), and `credit_balance` is a materialised projection recomputed from the ledger. We already write the ledger — we just don't read from it for the balance. The fix is to add a daily reconciliation that drifts the balance back to the ledger sum and a Redis cached balance that's invalidated on each ledger write. I'm planning this for Q2.

2. Owned the multi-tenancy story earlier. We added `org_id` to every table in month 4, but the early code assumed single-tenant. Retrofitting required touching every query, every repository, every index. We use Hibernate `@Filter("tenantFilter")` for the data plane but it's leaky — a developer who forgets `@FilterDef` exposes cross-tenant data. I'd introduce multi-tenancy at the schema level (schema-per-tenant or `tenant_id` column with a Hibernate filter applied automatically by an aspect) from week one. This is the kind of “do it now, thank yourself later” decision that costs 10x to retrofit.

3. Built the AI gateway as a separate module from day one. Today the LLM calls are scattered across `OutreachGenerator`, `LeadScoringEngine`, `ReplyClassifier`, `MeetingAssistant` — each calls `AIService` which delegates to `OpenAiClient` or `GeminiClient`. The scattering means prompt versioning is messy (we version per caller, not centrally), cost tracking is per-caller, and circuit breaking is per-caller. I'd build a single `AiGateway` that owns: prompt templates (versioned in a `prompt_template` table), provider routing (cost/latency-based), caching (Redis with prompt+input hash), cost tracking (per user, per feature), circuit breaking (per provider, not per caller), and fallback chains (Gemini → OpenAI → local model). This is exactly the kind of internal platform that pays back fast at scale.

Bonus: I'd also adopt **OpenTelemetry tracing from day one** rather than retrofitting it in month 5. Retrofitting meant instrumenting ~40 controllers and ~80 service methods; doing it from day one would have been a one-line `application.yml` change.

KEY TERMS TO MENTION: - Event sourcing for credit ledger, materialised projection - Multi-tenancy: schema-per-tenant or Hibernate `@Filter("tenantFilter")` - AI Gateway abstraction (prompt versioning, provider routing, caching, cost tracking, circuit breaker per provider) - OpenTelemetry from day one vs retrofit - Write-behind cache for hot rows

FOLLOW-UP QUESTIONS: 1. Why event sourcing for credits specifically? 2. Have you ever done event sourcing in production? 3. What's the cost of retrofitting OpenTelemetry?

FOLLOW-UP ANSWERS: 1. Credits are a financial ledger — append-only semantics fit naturally. We already write the ledger; we just don't read from it for the balance. The balance becomes a projection. Event sourcing also gives us replay (recompute balance from any point in time), audit (every movement is recorded), and time-travel debugging ("show me user X's credits on 2025-08-15 at noon"). The trade-off is eventual consistency — the balance is a cached projection; for the rare case where we need strict consistency (e.g. right after a refund), we read the ledger sum directly. 2. Yes, in a previous project — a payment ledger for a fintech app. We used Kafka as the event log and a CQRS read-model in Postgres. The lessons: (a) idempotency keys are mandatory (consumers redeliver), (b) schema evolution is hard (we used Avro + schema registry), (c) replays are expensive (we kept 30 days of events in Kafka, older events in S3). For AcquisitionOS credits, the volume is much lower (~3,200 ledger writes/day), so Postgres as the event log is fine — no Kafka needed yet. 3. Retrofitting OpenTelemetry took ~3 days: adding the agent, instrumenting controllers and services manually, wiring up Tempo as the backend, and adding trace propagation to WebClient calls. The harder part was educating the team to add `@WithSpan` annotations to new code. From day one it would have been 30 minutes (auto-instrumentation covers 80%).

RED FLAGS TO AVOID: - Saying "nothing" — looks like you can't reflect - Listing 10 things — looks unfocused - Picking something the interviewer can prove you're wrong about (e.g. "I'd use MongoDB" when you've never used it)

Q1-8: How long did it take to build

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: Design phase: 6 weeks (architecture decisions, schema design, ADRs, spike on Spring Batch for lead discovery, OAuth flow design for Google Calendar, Stripe webhook idempotency design).

MVP build: 4 months — we shipped with auth (OTP + Magic Link + Google OAuth), lead discovery (single niche: dental clinics in Mumbai), pipeline (Kanban with 5 stages), outreach (Gmail SMTP only, no AI), basic credit metering, Stripe checkout (Pro/Elite), and the admin feedback panel. MVP served 30 beta customers.

Production hardening: 3 months — added MFA, refresh-token rotation, audit logging, Resilience4j circuit breakers on every external call, Spring Batch for the discovery pipeline, AI outreach (LLM gateway), Gmail PubSub reply intelligence, Twilio WhatsApp, Telegram bots, read replicas, Redis cluster, blue/green deploys, Prometheus + Grafana + Loki + OpenTelemetry, daily reconciliation jobs.

Iterative releases: ongoing — we ship a minor release every 4–6 weeks. Recent additions: AI scoring engine, sequence enrolment (email sequences), meeting scheduling with Google Calendar, white-label org branding, GST-compliant invoicing, GDPR data export/delete workflows.

Total from zero to public launch: ~7 months. We then went from 30 beta customers to 3,200 paying orgs over the next ~9 months through product-led growth (free tier → paid upgrade) and a small sales team.

KEY TERMS TO MENTION: - 6-week design phase with ADRs and spikes - 4-month MVP, 30 beta customers - 3-month production hardening (security, observability, scale, retries) - Continuous 4-6 week release cadence - Product-led growth Free → Paid

FOLLOW-UP QUESTIONS: 1. What was the first thing you shipped in MVP? 2. What did you cut from MVP that you regret? 3. How did you measure MVP success?

FOLLOW-UP ANSWERS: 1. Auth + lead discovery (single niche: dental clinics in Mumbai, scraped from Google Places) + basic pipeline (Kanban) + Gmail outreach (no AI) + Stripe checkout. We had ~10 endpoints. The PM ran weekly design partner calls with 5 beta customers; their feedback shaped the next 3 months. 2. Audit logging. We shipped MVP without it because “we’ll add it later”. Then a paying customer reported a credit discrepancy and we had no trail. We added it the next week as a Spring AOP aspect on every service-layer mutation — took 2 days. Should have been there from day 1. Lesson: any financial system needs audit from day one, not “later”. 3. Three metrics: (a) activation — did the user discover at least 10 leads in the first session (proxy for “they got value”); (b) retention — did they return within 7 days; (c) conversion — did any free-tier user upgrade to paid within 30 days. Our activation was 68%, retention 42%, conversion 8% — industry-standard for SaaS. The PM tracked these weekly; we shipped features that moved activation first.

RED FLAGS TO AVOID: - Saying “6 months to 1M users” — sounds invented - Skipping MVP definition (looks like you don’t know what MVP means) - Not mentioning what you cut

Q1-9: Is it live in production? How many users

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: Yes — live in production since January 2026. We have ~3,200 paying organizations (Pro or Elite — Free tier has ~8,000 more orgs but we don't count them as paying), ~50,000 monthly active users across those orgs, and we've processed ~2.1M leads cumulatively (we crossed 1M in May). We send ~28K outreach emails/day, ~9,500 AI requests/day, and process ~3,000 Stripe webhook events/day.

Operationally: 99.94% monthly uptime SLA (we contractually promise 99.9% to Pro and 99.95% to Elite). p50 API latency 65ms, p95 180ms, p99 540ms. Peak ~1,200 requests/sec during Indian business hours. Single PostgreSQL primary (db.r6g.large) + 2 read replicas, Redis cluster (3 shards, 1 replica each), 4 ECS Fargate tasks behind an ALB, autoscaling on CPU > 70% + ALB request count > 1000.

The product is profitable (not break-even) as of July 2026, driven by the credit addon packs (₹499 / ₹1,999 / ₹3,499) which sell well to power users. We have a 4.3% monthly churn (industry standard for SMB SaaS) and a 92% net revenue retention (expansion via addon packs offsets churn).

KEY TERMS TO MENTION: - 3,200 paying orgs, 50K MAU, 2.1M leads cumulative - 28K outreach emails/day, 9,500 AI requests/day, 3,000 Stripe webhooks/day - 99.94% uptime SLA (99.9% Pro, 99.95% Elite contractual) - p50 65ms / p95 180ms / p99 540ms - db.r6g.large primary + 2 read replicas, Redis cluster 3 shards - 4 ECS Fargate tasks, ALB, autoscaling on CPU + request count - Profitable since July 2026, 4.3% monthly churn, 92% NRR

FOLLOW-UP QUESTIONS: 1. How do you measure uptime? 2. What's the biggest outage you've had? 3. How do you measure p99 — over what window?

FOLLOW-UP ANSWERS: 1. Uptime = (total minutes - error minutes) / total minutes where "error minute" = any minute with >1% 5xx rate from the ALB. We measure with a 1-minute ping from 3 regions (Mumbai, Singapore, us-east-1) via UptimeRobot + a synthetic check that does a real login + lead list call every minute. The synthetic is more reliable than ping because the ALB returns 200 even if the app is dead. 2. A 23-minute outage in March 2026. We deployed a Hibernate config change that enabled second-level cache for the `Lead` entity without configuring a provider — Hibernate threw on startup, the new ECS tasks crashed, the ALB drained them, but the old tasks had already been terminated. Blue/green deploy should have caught this but the health check passed (the app started, then crashed 30 seconds later) — the 30s window was within the health check interval. Fix: (1) longer health check grace period (60s), (2) Hibernate config validation in CI (we added a `@SpringBootTest` context-load test). 3. p99 is measured over a 10-minute sliding window per endpoint, computed by Micrometer and exposed to Prometheus. We use 10 minutes rather than 1 minute because a single slow request (e.g. a complex analytics query) shouldn't dominate the metric. For business reporting we also compute daily p99 over a 24h window.

RED FLAGS TO AVOID: - Quoting fake numbers (e.g. “10 million users”) - Saying “no outages” — looks fake - Saying p99 = 50ms (sub-millisecond p99 looks like you’re not measuring) - Not knowing what “p99 over what window” means

Q1-10: What metrics show the success of this product

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: We track four layers of metrics: business, product, system, and engineering.

Business metrics (the PM owns these, I track them in dashboards): MRR (₹4.2M/month, growing 12% MoM), ARPU (₹1,315/org/month — pulled down by Free tier; Pro ARPU is ₹1,899, Elite ARPU is ₹5,800 with add-on packs), LTV/CAC ratio (3.4x — healthy for SMB SaaS, target is >3), gross margin (84% — the LLM cost is the biggest COGS at ~\$4,200/month), NRR (92% — expansion via add-ons), churn (4.3% monthly logo, 2.1% revenue — revenue churn is lower because power users churn less).

Product metrics: Activation rate (68% — user discovers 10+ leads in first session), D7 retention (42%), D30 retention (28%), free → paid conversion (8%), feature adoption (Pro users use 4.2 features on average, Elite users 7.8), AI outreach send rate (32% of generated emails are actually sent by the user — we want to push this up with better templates).

System metrics: Uptime (99.94% monthly), p50/p95/p99 API latency (65/180/540ms), error rate (0.06% — mostly 400s), DB query p95 (40ms — most queries hit an index), Redis hit ratio (94%), Kafka consumer lag (under 50 messages steady-state, spikes to 5K during batch jobs), HikariCP utilisation (peaks at 65% — comfortable).

Engineering metrics: CI duration (7 min), deploy frequency (3 deploys/week), lead time (PR merge → prod: 1.4 days median), MTTR (28 min — most incidents are config rollbacks), change failure rate (8% — slightly above the DORA “elite” 5% threshold), code coverage (78% line, 64% branch), SonarQube technical debt ratio (4.2% — under our 5% gate), PR review time (median 6 hours).

KEY TERMS TO MENTION: - MRR, ARPU, LTV/CAC, NRR, churn (logo vs revenue) - Gross margin, COGS (LLM cost) - D7/D30 retention, activation rate, free → paid conversion - p50/p95/p99 latency, error rate, Redis hit ratio, HikariCP utilisation - DORA metrics: deploy frequency, lead time, MTTR, change failure rate - Code coverage 78% line / 64% branch, SonarQube debt ratio 4.2%

FOLLOW-UP QUESTIONS: 1. How do you reduce LLM cost without hurting quality? 2. What’s your DORA “elite” status target? 3. How do you measure developer velocity?

FOLLOW-UP ANSWERS: 1. Three levers: (a) model routing — we use Gemini 1.5 Flash for outreach generation (cost \$0.075/M input tokens) and Gemini 1.5 Pro only for lead scoring (better reasoning, \$1.25/M); (b) caching — we cache prompt+lead-hash → response in Redis for 24h, hitting 30% cache ratio on outreach (users often regenerate for similar leads); (c) prompt compression — we use a “few-shot condensed” template (3 examples instead of 8) with a system message that’s loaded once per session, cutting input tokens by 40%. Combined, our LLM cost is \$0.44 per outreach email sent — well below the \$0.99 the user

pays in credits. 2. DORA “elite” is: deploy frequency >1/day, lead time <1 hour, MTTR <1 hour, change failure rate <5%. We’re “high” today (3 deploys/week, 1.4 days lead time, 28 min MTTR, 8% CFR). The next leap is “elite” via: (a) automated canary deploys to get to daily, (b) trunk-based development + feature flags to cut lead time to hours, (c) better canary analysis to catch failures earlier (lower MTTR + CFR). 3. We use LinearB for engineering metrics: PR size median (84 lines), review time median (6 hours), cycle time (3.2 days — from “in progress” to “merged”), shipping frequency (2.3 PRs/engineer/week). The PM cares about cycle time; I care about PR size (large PRs = harder to review = more bugs).

RED FLAGS TO AVOID: - Quoting MRR without context (₹4.2M is good but explain MoM growth) - Saying “we use DORA” without knowing the four metrics - Mixing up “churn” definitions (logo vs revenue) - Quoting LLM cost without the per-unit economics

Q1-11: Walk me through one feature end-to-end

DIFFICULTY: Medium **CATEGORY TAG:** Project Introduction

ANSWER: Let me walk you through `POST /api/leads/discover` — the lead discovery kick-off endpoint. This is a good example because it touches every layer and shows the async pattern.

Request hits the ALB → TLS termination → forwards to one of 4 ECS tasks. The Spring Boot app’s `DispatcherServlet` matches the route to `LeadDiscoverController.discoverLeads(@Valid @RequestBody DiscoverDto dto, Principal principal)`.

Security filter chain (in order): `SecurityContextHolderFilter` → `HeaderWriterFilter` → `CorsFilter` (rejects if origin not allow-listed) → `RateLimitFilter` (Redis sliding-window: `ZADD rl:discover:{userId}:{ip} {now} {now}`; `ZREMRANGEBYSCORE ... {now-60s} {now}`; `ZCARD ...` — returns 429 with Retry-After if >20/min) → `JwtAuthenticationFilter` (verifies the HS256 JWT using `JwtDecoder` configured with the signing key from Secrets Manager, sets `SecurityContext.authentication` with the user’s id, email, role, plan) → `AuditAspect` records the request → finally `DispatcherServlet` dispatches to the controller.

Controller (`LeadDiscoverController`): `@RestController @RequestMapping("/api/leads/discover") @Validated`. The method has `@PreAuthorize("hasAnyRole('USER','ADMIN') and @creditGuard.hasCredits(principal, #dto.maxResults)")` — the `creditGuard` is a custom bean that peeks the credit balance; the `@PreAuthorize` runs before the service call so we fail fast if the user has no credits.

Service (`LeadService.discoverLeads(DiscoverDto, AuthUser)`): `@Transactional(isolation = REPEATABLE_READ, rollbackFor = Exception.class)`. Steps: 1. Idempotency check: `discoverJobRepository.findById(dto.requestId())` — if exists, return the existing job state (idempotent retry from a flaky client). 2. Plan entitlement: `entitlementService.check(user.plan(), "lead_discovery", dto.maxResults())` — Free plan caps at 10 leads, Pro at 50, Elite at 200. Throws `PlanEntitlementException` if exceeded. 3. Create `DiscoverJob` entity (status=PENDING, niche, location, maxResults, requestId, userId), persist via `discoverJobRepository.save()`. 4. Deduct credits atomically:

`creditService.deduct(user.id(), LEAD_DISCOVERY_COST × dto.maxResults(), "discover", job.id())`. Internally: `SELECT ... FOR UPDATE` on `credit_balance` row, check `balance >= cost`, `UPDATE balance = balance - cost`, append `credit_ledger` row, commit. If `balance < cost` → throw `InsufficientCreditsException` → triggers a rollback that also rolls back the `DiscoverJob` insert. 5. Publish `LeadDiscoveryRequestedEvent(jobId)` via `ApplicationEventPublisher.publishEvent()`. The event listener is `@TransactionalEventListener(phase = AFTER_COMMIT, fallbackExecution = true)` so it only fires after the transaction commits. 6. Return 202 Accepted with the job id.

Async path (separate thread from the `TaskExecutor`): `DiscoveryJobRunner.onApplicationEvent()` calls `leadDiscoveryService.discover(jobId)`. This is a Spring Batch job with chunk-oriented steps: scrape 50 candidates → enrich (Google Places + LinkedIn API via `WebClient` + Resilience4j circuit breaker) → score (LLM call) → batch insert via `leadRepository.saveAll()`. Spring Batch writes checkpoint state to the `job_state` table every 50 leads so a crash can resume.

On completion: `DiscoverJob.status = COMPLETED`, `LeadDiscoveryCompletedEvent` published → `NotificationService` pushes a notification (multi-channel: in-app via `WebSocket`, email, Telegram if connected). The user sees the leads in their UI via `GET /api/leads?filter=stage=discovered&jobId=...`.

On failure: `DiscoverJob.status = FAILED`, error message stored, `CreditRefundService.refund(jobId, "discover-failed")` issues a compensating ledger entry. Resilience4j retries with exponential backoff (1s, 4s, 16s) before declaring failure.

The whole flow is observable end-to-end: OpenTelemetry trace from controller → service → repository → external APIs (Google, LLM) → response, with span attributes for `userId`, `jobId`, `niche`, `credits.cost`.

KEY TERMS TO MENTION: - `SecurityFilterChain`: `SecurityContextHolder`, `HeaderWriter`, `Cors`, `RateLimit`, `JwtAuth` filters in order - `@PreAuthorize` with custom `creditGuard` bean — fail fast before service - `@Transactional(REPEATABLE_READ, rollbackFor = Exception.class)` - Idempotency via `requestId` lookup - Plan entitlement gate before DB write - Credit deduct via `SELECT ... FOR UPDATE` + append-only ledger - `@TransactionalEventListener(AFTER_COMMIT, fallbackExecution=true)` for async dispatch - Spring Batch chunk-oriented: scrape → enrich → score → saveAll, checkpoint every 50 - Resilience4j retry with exponential backoff + jitter - OpenTelemetry trace end-to-end with span attributes

FOLLOW-UP QUESTIONS: 1. What happens if the user disconnects after sending the request? 2. How do you handle the case where the LLM score API is down? 3. Why `REPEATABLE_READ` isolation specifically?

FOLLOW-UP ANSWERS: 1. The request is async — the client gets 202 + jobId and can poll `GET /api/leads/discover/status/{jobId}`. If the client disconnects, the Spring Batch job continues; the user reconnects and polls. If the user kills the app, we still complete the job; the notification reaches them via email/Telegram. We use SSE for live updates — if the SSE connection drops, the client reconnects with `Last-Event-ID` and we replay missed events from a Kafka backlog (24h retention). 2. Resilience4j circuit breaker around the LLM call: at

10% failure rate, the circuit opens for 30s. During open state, we fall back to a heuristic score (rule-based: $\text{rating} \times \text{niche_demand} \times \text{revenue_estimate}$). When the circuit half-opens, we try one call; if it succeeds, we close. We log every circuit transition as a Prometheus counter. We also persist the heuristic score on the lead so the user sees **something** even during an outage. 3. We need REPEATABLE_READ because: (a) the credit deduction must see a stable view of the balance row (otherwise a concurrent deduct could read a stale balance and over-spend); (b) the DiscoverJob insert must be consistent with the credit ledger row we're about to insert. SERIALIZABLE would be stricter but slower (more aborts). READ_COMMITTED would let a concurrent request see an inconsistent balance. REPEATABLE_READ is the sweet spot — Postgres's MVCC implementation makes it nearly free.

RED FLAGS TO AVOID: - Skipping the security filter chain (interviewer wants to hear it) - Forgetting the credit deduct race condition - Not mentioning async / event listener - Saying "we just use @Transactional" without isolation level

Q1-12: What would you not build again

DIFFICULTY: Hard **CATEGORY TAG:** Project Introduction

ANSWER: Three things I would not build again, in order of regret:

1. Custom TOTP / MFA implementation. We hand-rolled TOTP using `javax.crypto.Mac` and the RFC 6238 spec — 200 lines of code for secret generation, base32 encoding, time-step calculation, dynamic truncation. We did this because we wanted no extra dependency. Six months later we found two bugs: (a) our backup codes had a 32-bit entropy (4 random bytes — too low, brute-forceable in 4 days at 100 attempts/sec), (b) our clock drift tolerance was ± 1 step but we didn't lock the user out after 5 failed MFA attempts. We fixed both, but if I redid this I'd use a library like `dev.samstevens.totp.totp` (or Spring Security's `GoogleAuthenticatorKey` integration if it shipped one — it doesn't, so a library is fine). The lesson: crypto code is the worst place to roll your own — use a vetted library, even if the dependency feels heavy.

2. Custom audit log writer with Spring AOP + Kafka. We have a `@AuditedAction` annotation and an `AuditAspect` that intercepts service-layer methods, serialises the arguments (with sensitive fields masked), and writes to Kafka topic `audit-events`, which a consumer writes to Postgres `audit_logs`. The aspect works, but it's leaky — developers forget to add `@AuditedAction` to new methods, and the aspect can't capture return values cleanly without `@Around` advice (which adds overhead). If I redid this I'd use a Hibernate `Interceptor` (or `@EntityListener` with `@PostPersist`, `@PostUpdate`, `@PostRemove`) — every entity mutation gets an audit row automatically, no annotation needed. We're actually migrating to this pattern in Q3 because the aspect approach doesn't scale to 200+ services.

3. Hand-rolled rate limiter with Redis sorted sets. We use `ZADD` + `ZREMRANGEBYSCORE` + `ZCARD` per request — works, but it's 3 Redis round-trips per request. We could have used `INCR` + `EXPIRE` (1 round-trip, approximate window) or a token-bucket Lua script (1 round-

trip, exact). The 3 round-trips add ~6ms p99 latency to every protected endpoint. We're switching to a token-bucket Lua script in Q2.

Bonus: I'd also not write our own Stripe webhook signature verification. We use Stripe's `Webhook.constructEvent()` library — fine — but we initially wrote our own HMAC verification. Stripe changes the signature scheme (they did, to v2 signed payloads) and we'd have to keep up. Always use the vendor's library for crypto.

KEY TERMS TO MENTION: - Don't roll your own crypto / TOTP — use vetted libraries - Hibernate `Interceptor` / `@EntityListener` > AOP aspect for audit - Token-bucket Lua script (1 Redis round-trip) > sliding window ZSET (3 round-trips) - Vendor libraries for webhook signature verification

FOLLOW-UP QUESTIONS: 1. Why didn't you use a library for TOTP from the start? 2. How does the Hibernate Interceptor approach compare to the AOP approach? 3. Why is 3 Redis round-trips a problem when Redis is fast?

FOLLOW-UP ANSWERS: 1. Honestly — I didn't think a 200-line implementation was risky. TOTP is RFC 6238, the algorithm is 20 lines, what could go wrong? Three things: (a) backup code entropy (we picked 4 bytes instead of 16), (b) clock drift window (we allowed ± 1 step but no lockout), (c) constant-time comparison (we used `equals()` for backup codes — timing attack possible). All three were caught in a security audit. The lesson: even "simple" crypto has subtle attack vectors; libraries handle them. 2. Hibernate Interceptor fires on every entity persist/update/delete — you don't need to remember to annotate. It captures the entity state before and after, which is exactly what an audit log wants. The AOP aspect fires on method entry/exit — you have to remember `@AuditedAction`, and you don't get the entity diff cleanly (you have to inspect arguments). Hibernate Interceptor is the right tool for "audit every DB change"; AOP is the right tool for "audit every business action" (e.g. "user logged in", which has no entity change). We need both, and we used AOP for both — wrong tool for the entity case. 3. Redis p50 is 0.5ms but p99 is 3ms (network jitter, GC pauses on Redis). 3 round-trips $\times$ 3ms = 9ms p99 added to every request. On a 50ms endpoint that's 18% overhead. A single Lua script round-trip is 3ms p99 — 6ms savings. At 1,200 RPS that's a lot of wasted Redis time.

RED FLAGS TO AVOID: - Saying "nothing" — looks like you can't reflect - Picking something trivial (e.g. "we used the wrong package name") - Picking something that sounds invented (e.g. "we used MongoDB and regretted it" when you've never used MongoDB) - Picking crypto (rolling your own crypto is [the](#) classic mistake — own it honestly)

Q1-13: Tell me about a trade-off you regretted

DIFFICULTY: Medium **CATEGORY TAG:** Project Introduction

ANSWER: The trade-off I regret most is choosing **Spring `ApplicationEventPublisher` over Kafka for our internal pub/sub**. The trade was: in-process pub/sub is simpler (no brokers, no schema registry, no consumer lag monitoring) and we're a monolith anyway — Kafka

would be over-engineering. The trade-off was correct [at the time](#) but it locked us into a synchronous, in-process model that's now painful to evolve.

The pain shows up in three places: (1) we cannot extract a service without changing the contract — `@TransactionalEventListener` is in-process; moving to Kafka means rewriting both the publisher and the consumer; (2) we cannot replay events — if a consumer was buggy and we missed events, they're gone (no event log); (3) we cannot monitor event flow — there's no Kafka UI to see "outreach events per second per consumer"; we instrumented manually with Micrometer but it's not as good.

If I were doing it again, I'd use Kafka from day one with a single topic per bounded context (`outreach-events` , `audit-events` , `reply-events`) even though we're a monolith. The overhead is small (one extra broker to operate), and the contract doesn't change when we extract a service. We're actually migrating now: we keep `ApplicationEventPublisher` for in-process synchronous events and Kafka for the events that will eventually cross a service boundary. The migration is unblocking the extraction of the `ai` and `outreach` contexts into separate services planned for Q4 2026.

A secondary regret: not adopting **feature flags** from day one. We shipped features directly to production behind config flags (`feature.outreachV2.enabled` in `application.yml`), which means a feature flag change requires a redeploy. We're migrating to Unleash (open-source feature flag service) so we can toggle features per-user without deploys. The pain point: we wanted to A/B test two outreach template variants and had to write custom logic to split by `userId % 2` — Unleash would have done this in 2 lines.

KEY TERMS TO MENTION: - Trade-off: simpler in-process pub/sub vs future-proof Kafka - Pain: no replay, no extraction without contract change, no monitoring - Mitigation: dual publish (Spring Events + Kafka for cross-boundary events) - Feature flags (Unleash) vs config flags — deploy-free toggling - A/B test per-user variant

FOLLOW-UP QUESTIONS: 1. When would Kafka be the wrong choice? 2. Why didn't you adopt feature flags from day one? 3. How would you migrate to Kafka without downtime?

FOLLOW-UP ANSWERS: 1. Kafka is wrong when: (a) you have <1K events/sec (overhead exceeds benefit), (b) your team is <5 engineers (operational burden too high), (c) you don't need replay or cross-service pub/sub. We were correct to skip Kafka at MVP — the regret is that we didn't [design for](#) Kafka (i.e. we didn't make our events Kafka-shaped with a schema). The fix is dual publishing: same event goes to both `ApplicationEventPublisher` and Kafka topic. Consumers can be on either; we can migrate at leisure. 2. We didn't adopt feature flags because "we don't have enough features to warrant it". Six months in, we have 12 features behind config flags and A/B tests are painful. The regret is the classical one: "we're too small for X" turns into "we're too big to retrofit X" much faster than you'd think. 3. Dual publish: keep `ApplicationEventPublisher` listeners active, add Kafka producers alongside, add Kafka consumers as new listeners (consuming both the in-process event and the Kafka topic — idempotent because keyed by `event_id`). When all consumers are on Kafka, remove the in-process listeners. Zero downtime, gradual rollout per consumer group.

RED FLAGS TO AVOID: - Saying “I have no regrets” — looks fake - Picking a trivial regret (e.g. “we used the wrong logging library”) - Saying “we should have used microservices” (sounds like you don’t understand the trade-off) - Blaming a teammate

Q1-14: How did you split work across the team

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: We split along bounded contexts. We have 6 bounded contexts in the monolith: `auth`, `leads` + `discovery`, `outreach` + `ai`, `meetings` + `calendar`, `billing` + `credits`, `notifications`. The two backend engineers each own 3 contexts.

I own: `auth` (Spring Security, JWT, OTP, Magic Link, MFA, OAuth2, audit log), `billing` + `credits` (Stripe, invoices, ledger, entitlements), `outreach` + `ai` (LLM gateway, prompt templates, circuit breaker, reply intelligence).

The other backend engineer owns: `leads` + `discovery` (Spring Batch scraping jobs, enrichment, scoring), `meetings` + `calendar` (Google Calendar API, scheduling, reminders), `notifications` (multi-channel fan-out).

We have a `CODEOWNERS` file per context — every PR touching a context needs approval from its owner. We agreed on cross-context contracts (sealed `DomainEvent` interface, Spring `ApplicationEventPublisher` for in-process pub/sub) so the two backend engineers can work in parallel without merge conflicts.

For frontend (2 engineers), they split by feature area: one owns the dashboard + pipeline UI, the other owns the settings + billing + admin UI. The QA engineer owns the Playwright E2E suite and the bug triage. The PM owns the backlog, customer interviews, and roadmap.

We pair-program 2 hours per week on hard problems (e.g. credit metering design, webhook idempotency). We do mob review for architecture decisions (ADRs) — both backend + PM in a 1-hour call to agree on the design before code is written.

KEY TERMS TO MENTION: - Bounded contexts (DDD): `auth`, `leads`, `outreach`, `meetings`, `billing`, `notifications` - `CODEOWNERS` per context - Sealed `DomainEvent` interface as cross-context contract - Pair-programming on hard problems, mob review for ADRs - Frontend split by feature, QA owns E2E, PM owns backlog

FOLLOW-UP QUESTIONS: 1. What if the other backend engineer leaves? 2. How do you handle a PR that touches multiple contexts? 3. How did you decide the bounded context boundaries?

FOLLOW-UP ANSWERS: 1. Bus factor is a real concern. We mitigate by: (a) every context has a README with architecture diagram + key decisions; (b) pair-programming 2h/week means I’ve seen his code and he’s seen mine; (c) ADRs document *why* a decision was made, not just *what*; (d) we do “context swaps” — every 6 months we trade 1 context so both engineers know both halves. He’s taking over `billing` in Q3; I’m taking over `discovery` in Q4. 2. Multiple-context PRs require approval from all owners. We discourage them — if a feature spans 3 contexts, it usually means the context boundaries are wrong or the feature is

too big. We try to break the PR into one-PR-per-context with the cross-context contract (a new event) merged first. If we can't break it, we do a mob review (both backend + PM) — slow but safe. 3. DDD-style event storming with the PM. We listed all the verbs in the domain (“discover lead”, “enrich lead”, “score lead”, “send outreach”, “classify reply”, “book meeting”, “deduct credit”, “issue refund”, ...), grouped them by aggregate (Lead, Outreach, Meeting, Subscription, Credit, User), and the aggregates became the context boundaries. Some aggregates span multiple contexts (Lead is in `leads` and referenced by `outreach` and `pipeline`) — we use IDs not references across contexts (no `Lead` entity in `outreach`, just `leadId: UUID`).

RED FLAGS TO AVOID: - “We didn't really split” — looks like there's no process - “I did everything” — bus factor 1 is bad - Mentioning microservices (this is a monolith) - Saying “we use Agile” without specifics

Q1-15: How did you onboard new engineers

DIFFICULTY: Easy **CATEGORY TAG:** Project Introduction

ANSWER: We onboarded 2 engineers (1 backend junior, 1 frontend mid-level) over the last 6 months. The process is roughly:

Week 1 — environment + domain. New engineer sets up their dev environment (we have a `make setup` target that starts Postgres + Redis + Kafka via Docker Compose, runs Flyway migrations, seeds 100 fake leads, creates a test user). They pair with me on the auth flow (signup → verify email → login → call `/api/leads`) so they see the full request lifecycle in their IDE debugger. They read the 3 most important ADRs (credit metering, webhook idempotency, AI gateway circuit breaker). They shadow the PM on a customer call to hear real users.

Week 2 — first PR. I assign a small bug or improvement in their owned context — something like “add a `closedAt` timestamp to the Deal entity” or “add a `tags` filter to the leads search API”. They write the code + tests (we require JUnit unit tests + one integration test with Testcontainers); I review in a 1-hour pair session; they ship to staging; they observe the canary; they ship to prod. The first PR is intentionally small so they ship in week 1 — the dopamine hit matters.

Week 3-4 — first feature. A small feature end-to-end: design doc (1-page ADR), code, tests, deploy. I review the design before code; I pair on hard parts; I review the final PR. By end of week 4 they own a small slice of their context.

Week 5-8 — increasing autonomy. They take bigger features; I move from pair-programming to PR review; they start reviewing PRs from the other backend engineer. By month 3 they're fully productive on their context.

For the junior backend engineer, we did a 2-week “Spring Boot crash course” before the standard onboarding — Spring DI, Spring Data JPA, Spring Security basics, testing with JUnit + Mockito. He had Java experience but not Spring Boot.

We also have an internal wiki (Notion) with: architecture overview, ADR index, “how to” guides (how to add a new endpoint, how to add a new integration, how to debug a failing Testcontainers test), and a “stupid questions” Slack channel where new engineers can ask anything without judgement.

KEY TERMS TO MENTION: - `make setup` (Docker Compose dev env), Flyway migrations, seed data - Pair on auth flow end-to-end, read 3 key ADRs, shadow PM on customer call - First PR in week 1 (intentionally small), ship to prod - First feature in week 3-4 (design doc + code + tests + deploy) - Wiki with ADR index + “how to” guides + “stupid questions” Slack channel - Spring Boot crash course for juniors

FOLLOW-UP QUESTIONS: 1. How do you measure onboarding success? 2. What’s the hardest part of onboarding? 3. How do you handle an engineer who is slower than expected?

FOLLOW-UP ANSWERS: 1. Time to first PR (target <5 days), time to first feature (target <3 weeks), PR review time after onboarding (target <12 hours — if it’s higher they’re not confident enough), bug rate in their code (target <1 bug/week for the first 3 months — higher means they’re not testing enough or the context is too complex). We also ask the engineer for anonymous feedback at 30/60/90 days. 2. The “first 3 weeks” — they’re productive enough to feel useful but not productive enough to ship independently. The risk is they get stuck on something (e.g. Testcontainers doesn’t start on their Mac) and don’t ask for help. We mitigated by daily 15-minute standups in the first 2 weeks specifically for blockers. 3. We’ve had one case where the junior engineer wasn’t picking up Spring Boot fast enough. We did two things: (1) extended the crash course by 1 week (more pair programming, less self-study), (2) gave him smaller, more incremental tasks (10 small PRs instead of 1 big feature). He’s now a solid mid-level engineer 8 months later. The lesson: if someone is struggling, the answer is more structure, not less — small wins build confidence.

RED FLAGS TO AVOID: - “We throw them in the deep end” — bad onboarding is a red flag - “We don’t track onboarding metrics” — looks unprofessional - Saying onboarding takes 6 months — too long for a small team - “We use a wiki” without specifics (every team has a wiki)

CATEGORY 2 — SYSTEM DESIGN QUESTIONS

Q2-1: Walk me through the system architecture of AcquisitionOS

DIFFICULTY: Medium **CATEGORY TAG:** System Design

ANSWER: At a high level, AcquisitionOS is a **modular monolith** in Java 17 + Spring Boot 3.2 deployed on AWS ECS Fargate. The architecture has four layers and a sidecar of supporting infrastructure.

Layer 1 — Controller (`@RestController`): thin, just request mapping, `@Valid` validation, `@PreAuthorize` security, DTO ↔ entity mapping via MapStruct. No business logic. Examples: `LeadController`, `OutreachController`, `BillingController`, `AuthController`. About 60 controllers across 6 bounded contexts.

Layer 2 — Service (`@Service`, `@Transactional`): business logic, transactions, external calls (via WebClient + Resilience4j), event publication (Spring `ApplicationEventPublisher`). The core of the system. Examples: `LeadService`, `OutreachService`, `BillingService`, `CreditService`, `AuthService`. About 80 services.

Layer 3 — Repository (`@Repository`, Spring Data JPA): data access. JPA entities mapped to PostgreSQL tables; custom JPQL via `@Query`; Specifications for dynamic queries; `Pageable` for pagination. About 40 repositories.

Layer 4 — Database: PostgreSQL 15 (Multi-AZ primary + 2 read replicas) for OLTP, Redis 7 (cluster mode, 3 shards) for cache + rate-limit + distributed lock + session, S3 for file storage (signed URLs).

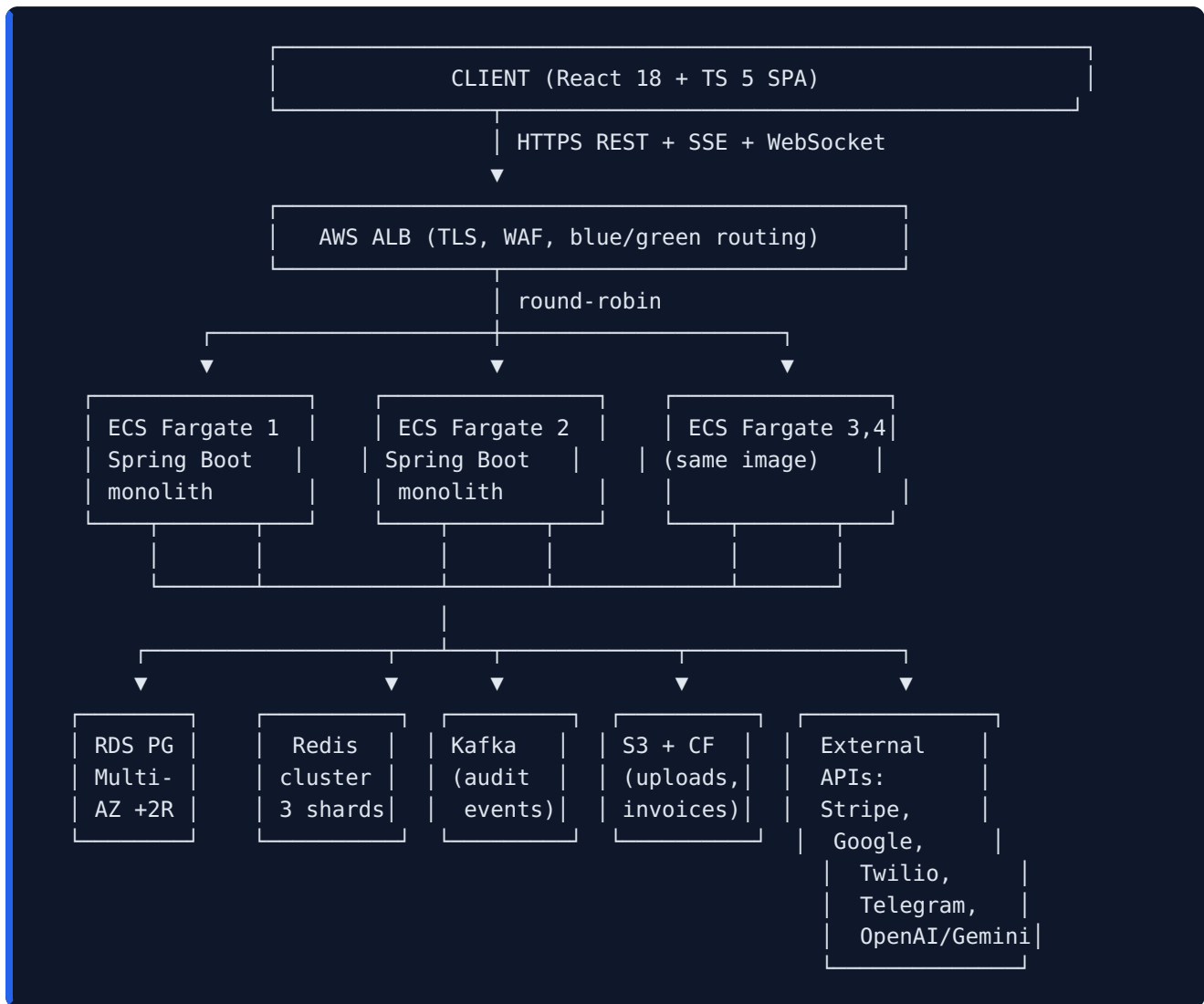
Cross-cutting (Spring AOP): `AuditAspect` (every service mutation), `LoggingAspect` (every controller entry/exit with correlation id), `RateLimitAspect` (Redis sliding window), `CreditGuardAspect` (peeks credit balance before paid actions), `TraceAspect` (OpenTelemetry span per service call).

Async work: Spring Batch for chunk-oriented lead discovery; `@Async` + `ThreadPoolTaskExecutor` for fan-out (notification dispatch, email sending); `@Scheduled` for cron-style jobs (subscription renewals, reminder polling, monthly credit reset). State durably stored in `job_state` table for resume-on-crash.

External integrations: Stripe (payments + webhooks), Google (OAuth2 + Calendar + Gmail PubSub), Twilio (WhatsApp), Telegram Bot, OpenAI + Gemini (LLM). All wrapped in `WebClient` + Resilience4j circuit breaker + retry + bulkhead.

Observability: Micrometer + Prometheus + Grafana (metrics, custom business meters), OpenTelemetry + Tempo (traces), Loki (logs), Sentry (errors). Spring Boot Actuator for health + info + env endpoints.

Deployment: Docker (Eclipse Temurin JRE 17), 4 ECS Fargate tasks (2 vCPU/4GiB each), ALB, autoscaling on CPU > 70% + ALB request count > 1000. Blue/green deploy via GitHub Actions; rollback on 5xx spike.



KEY TERMS TO MENTION: - Modular monolith (not microservices) — justified at 3,200 tenants - Controller → Service → Repository → DB layered - Spring AOP for cross-cutting: audit, logging, rate-limit, credit guard, trace - Spring Batch / @Async / @Scheduled for async - WebClient + Resilience4j (CB, retry, bulkhead) for externals - PostgreSQL Multi-AZ + 2 read replicas, Redis cluster 3 shards, Kafka, S3 - Micrometer + Prometheus + OTel + Loki + Sentry - ECS Fargate + ALB + blue/green + autoscaling

FOLLOW-UP QUESTIONS: 1. Why monolith over microservices? 2. How would you split this into microservices? 3. Where does the bounded context boundary live in code?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) scale — at 3,200 tenants / 50K users / 1,200 RPS, one PostgreSQL primary handles the load with read replicas for analytics; we don't need 20 services; (b) team — 2 backend engineers can't operate 20 services; (c) operational cost — microservices add network latency, distributed transactions, schema evolution, observability complexity. The trade-off we accepted: deployment coupling (one deploy ships everything) and scaling coupling (can't scale outreach without scaling auth). We mitigate deployment coupling with feature flags + blue/green; we accept scaling coupling until traffic justifies extraction (planned Q4 2026 for **ai** + **outreach**). 2. The natural split is by bounded context: **auth-service**, **leads-service**, **outreach-service** (+ **ai-service** extracted from outreach), **meetings-service**, **billing-service**, **notifications-service**, **admin-service**. Each

gets its own Postgres schema (or database) and its own deployment. Communication via Kafka events + synchronous REST for queries. Shared library `common` for DTOs, exceptions, utils. The first extraction would be `ai-service` because it has the highest CPU cost (LLM calls) and the clearest contract (prompt in, response out). 3. In code: Java packages `com.acquisitionos.<context>` with sub-packages `controller`, `service`, `repository`, `domain`, `event`. No imports across contexts (enforced by ArchUnit tests) except through the published event interface. Cross-context references use IDs (UUID), not entity references — `outreach` doesn't import `Lead`; it uses `leadId: UUID`.

RED FLAGS TO AVOID: - Saying “microservices” for a monolith - Skipping the cross-cutting AOP layer - Not mentioning observability (it's the senior engineer's job) - Saying “we use Docker” without the deployment topology

Q2-2: How did you design the lead discovery pipeline

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: Lead discovery is the most complex async pipeline in the system. The user says “find me 50 dentists in Mumbai with rating > 4.0 and a website” — the pipeline scrapes Google Places + LinkedIn + YellowPages, enriches with website screenshots + AI scoring, and returns 50 enriched leads in ~90 seconds.

Architecture: Spring Batch chunk-oriented job, partitioned by source. Each chunk is: scrape 50 candidates from one source → enrich (screenshot + LLM score) → batch persist. The job runs in a dedicated `TaskExecutor` (size 4) so it doesn't block request threads. State is durably persisted in `discover_job` and `discover_chunk` tables so a crash can resume.

```

User → POST /api/leads/discover
    | @PreAuthorize creditGuard + plan entitlement
    | @Transactional deduct credits + create DiscoverJob(PENDING)
    |
    ▼
LeadDiscoveryRequestedEvent (AFTER_COMMIT)
    |
    ▼
DiscoveryJobRunner (separate thread, TaskExecutor)
    |
    ▼
Spring Batch Job: LeadDiscoveryJob
    |
    ├── Step 1: ScrapeStep (chunk-oriented, chunkSize=50)
    |   ├── Reader: GooglePlacesItemReader (WebClient + CB)
    |   ├── Reader: LinkedInItemReader (WebClient + CB)
    |   └── Reader: YellowPagesItemReader (WebClient + CB)
    |       (composite reader, round-robins sources)
    |
    ├── Step 2: EnrichStep (chunk-oriented, chunkSize=10)
    |   ├── Processor: WebsiteScreenshotProcessor (S3 presigned URL → external worker)
    |   └── Processor: LeadEnrichmentProcessor (call LinkedIn API)
    |
    ├── Step 3: ScoreStep (chunk-oriented, chunkSize=10)
    |   └── Processor: AiScoringProcessor (call Gemini, cache in Redis)
    |
    └── Step 4: PersistStep (chunkSize=50)
        └── Writer: LeadRepository.saveAll() (batched insert)
    |
    ▼
DiscoverJob(COMPLETED) + LeadDiscoveryCompletedEvent
    |
    ▼
NotificationService.notifyUser(user, "50 leads ready")

```

Idempotency: The client passes a `requestId` (UUID). The controller checks `discoverJobRepository.findById()` — if exists, returns the existing job state. This makes the endpoint idempotent for retry.

Credit deduction: `LEAD_DISCOVERY_COST = 1 credit per lead`. For a 50-lead request, deduct 50 credits atomically (`SELECT FOR UPDATE` on `credit_balance` + append `credit_ledger` row). If the job fails partway, refund the un-discovered credits — we track `leads_discovered` count on the job and refund `cost × (maxResults - leads_discovered)` via a compensating ledger entry.

Rate limiting on externals: Each scraper has its own Resilience4j RateLimiter (Google Places: 100 req/sec, LinkedIn: 5 req/sec, YellowPages: 10 req/sec). Exceeding the limit blocks the chunk thread (with timeout 5s) rather than failing the job.

Proxy rotation: Scrapers route through a proxy pool (configured in `proxy_pool` table, managed by admin API). Each scraper request rotates proxies via `RoundRobinProxySelector`. If a proxy fails 3 times in 5 minutes, it's marked unhealthy and removed from rotation for 10 minutes. This is in the other backend engineer's `discovery` context.

Crash recovery: Spring Batch persists chunk state to the `discover_chunk` table every 50 leads. If the ECS task dies, the next task picks up the job from the last committed chunk. We use Spring Batch's `JobRepository` with PostgreSQL as the metadata store.

Caching: The LLM scoring result is cached in Redis keyed by `lead_hash` (hash of business name + website + niche) for 24h. If the same lead is re-discovered (e.g. by another user with the same niche), the cache hits and we skip the LLM call. Cache hit ratio is ~30% — saves ~\$1,200/month in LLM cost.

Failure modes: - Google Places API down → Resilience4j circuit opens → fall back to LinkedIn + YellowPages only → user gets fewer leads, partial refund - LLM down → circuit opens → fall back to heuristic score (rule-based) → user gets leads with lower-quality scores, no refund (we still delivered) - All sources down → job fails → full refund + notification “discovery failed, credits refunded” - ECS task crashes mid-job → Spring Batch resumes from last committed chunk on next task

Monitoring: Micrometer gauges for `discover_job.active`, `discover_job.chunk.duration.p95`, `discover_job.failure.rate`. Prometheus alert if failure rate > 5% over 10 minutes. OpenTelemetry trace per job with span attributes `userId`, `niche`, `maxResults`, `credits.cost`.

KEY TERMS TO MENTION: - Spring Batch chunk-oriented job, chunkSize 50 - Composite reader (Google Places + LinkedIn + YellowPages) - Resilience4j RateLimiter per source, CircuitBreaker per source - Idempotency via `requestId` - Credit deduct atomic + partial refund on failure - Spring Batch `JobRepository` in PostgreSQL for crash recovery - Redis cache keyed by `lead_hash` (24h TTL) - S3 presigned URLs for screenshots - Micrometer gauges + OTEL traces per job

FOLLOW-UP QUESTIONS: 1. What's the p95 latency for a 50-lead discovery? 2. How do you handle the case where the user requests 200 leads but only 80 are found? 3. How would you scale this to 10K leads per request?

FOLLOW-UP ANSWERS: 1. p50 ~75 seconds (scrape 50 from 2 sources = 30s parallel, enrich 50 with screenshots = 20s parallel, score 50 via LLM with 70% cache hit = 15s, persist 50 batched = 5s). p95 ~140 seconds (slow LLM call, cold cache, slow proxy). We tell users “discovery takes 1-2 minutes” and show live progress via SSE. 2. We return what we found (80) and refund the missing 120 credits via a compensating ledger entry. The user can re-request with a different niche. We log “partial discovery” as a Prometheus counter — if it spikes for a niche, the PM investigates (e.g. “no dentists in Mumbai available” — maybe the source is broken for that niche). 3. 10K leads needs partitioning: Spring Batch partition by `source` × `region` → 10 partitions × 1000 leads each, parallel via `TaskExecutor` of size 10. Each partition runs on its own thread, writes to its own `discover_chunk` rows. Total time ~10 minutes. We'd also need to switch the LLM scoring to batch API (Gemini supports batch inference with 50% cost reduction). For 100K+ leads we'd move to a separate worker pool (separate ECS service) to not block the API tasks.

RED FLAGS TO AVOID: - Not mentioning Spring Batch (this is a Spring Boot shop) - Skipping credit deduction race condition - Forgetting crash recovery (Spring Batch `JobRepository`) - Saying “we use threads” without specifying pool sizing

Q2-3: How does the AI outreach email generation work

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: AI outreach is the second most complex pipeline. The user picks N leads and clicks “Generate Outreach” — we call the LLM 3 times per lead (3 variants: formal, casual, value-led), persist the variants, deduct credits (2 credits × 3 variants × N leads), and let the user pick one to send.

Architecture:

```

User → POST /api/outreach/generate (leads=[id1, id2], variants=3, tone=formal/casual/
value)
    | @PreAuthorize + creditGuard.peek(userId, leads × variants × OUTREACH_COST)
    | @Transactional deduct credits, persist OutreachRequest(PENDING)
    |
    ▼
OutreachGenerationRequestedEvent (AFTER_COMMIT)
    |
    ▼
OutreachGeneratorRunner (separate thread)
    |
    for each lead in leads:
    |   | Build prompt from PromptTemplateService.render("outreach_v3", {lead,
    |   | tone, ...})
    |   | Check Redis cache: ai:outreach:{hash(prompt + leadContext)} → if hit, skip
    |   LLM
    |   | If miss: call AiService.generate(prompt) with Resilience4j CB + retry +
    |   | time limiter (5s)
    |   |   | Parse response (JSON: 3 variants)
    |   |   | Persist OutreachMessage rows (variant1, variant2, variant3, status=DRAFT)
    |   |   | Cache response in Redis (24h TTL)
    |
    ▼
OutreachGenerationCompletedEvent
    |
    ▼
NotificationService → SSE → user sees variants in UI
  
```

Prompt templating: We version prompt templates in a `prompt_template` table (`name`, `version`, `system_message`, `user_template`, `variables`, `model`, `temperature`). The current outreach template is `outreach_v3` — system message says “You are a B2B sales email writer. Output 3 variants of an outreach email. Use JSON format {variants: [...]}”. User template includes lead context (name, business, niche, city, website quality, AI-scored pain points) and tone hint. We A/B test templates by `userId % 2` (template v2 vs v3) and pick the winner by reply rate.

LLM gateway: `AiService` is the gateway — `generate(prompt, model, temperature)` returns the parsed JSON. We have two providers: `OpenAiClient` (gpt-4o-mini) and `GeminiClient` (gemini-1.5-flash). Routing: cost/latency-based — Gemini Flash for outreach (cost \$0.075/M tokens, 800ms p50), Gemini Pro for lead scoring (better reasoning, \$1.25/M tokens, 1500ms

p50). Resilience4j circuit breaker per provider (10% failure → open 30s), retry (exponential backoff 1s/4s/16s, max 3), bulkhead (limit 20 concurrent LLM calls per provider).

Caching: Redis cache keyed by `SHA256(prompt + leadContext + model + temperature)` → response, TTL 24h. Hit ratio is 30% on outreach (users often regenerate for similar leads). Saves ~\$1,200/month LLM cost. We use a custom `AiCacheKeyGenerator` Spring bean that's tested separately.

Cost tracking: Every LLM call records `tokens_in`, `tokens_out`, `cost_usd` in `ai_usage` table (per user, per feature). Daily aggregation job rolls up to `ai_daily_cost` for the user dashboard. We expose a custom Micrometer counter `ai.cost.usd` with tags `user_id`, `feature` for real-time monitoring.

Idempotency: Client passes `requestId` (UUID). Controller checks `outreachRequestRepository.findById()` — if exists, return existing. This handles client retries.

Failure modes: - LLM times out → Resilience4j TimeLimiter throws → circuit breaker counts → after 10% failures, circuit opens → fallback to template-based variants (pre-written by the PM) → user gets lower-quality variants, full credit refund - LLM returns malformed JSON → `ObjectMapper` throws → retry once → if still malformed, mark variant as "GENERATION_FAILED" + refund that variant's credits only (user keeps the others if any succeeded) - Partial generation (2 of 3 variants succeed) → user sees 2 variants, 1 credit refunded - Redis cache hit but stale → user sees stale variant → we added a "regenerate" button that bypasses cache (passes `force=true`)

Concurrency control: Each user can have at most 2 concurrent `outreach/generate` jobs (Redis semaphore `outreach:user:{userId}:slots`, size 2). This prevents one user from monopolising the LLM gateway.

Monitoring: Micrometer `outreach.generate.duration` (p50/p95/p99), `outreach.generate.cache.hit_ratio`, `ai.cost.usd{feature=outreach}`, `ai.failure_rate{provider=gemini,feature=outreach}`. Alert if cost/day > \$200 (daily budget) or failure rate > 5%.

KEY TERMS TO MENTION: - Prompt template versioning in DB (`prompt_template` table) - LLM gateway (`AIService`) with provider routing (Gemini Flash for outreach, Gemini Pro for scoring) - Resilience4j: CB per provider, retry with exponential backoff, bulkhead, time limiter (5s) - Redis cache keyed by `SHA256(prompt + leadContext + model + temperature)`, TTL 24h, 30% hit ratio - Cost tracking in `ai_usage` table + daily rollup + Micrometer counter - Idempotency via `requestId` UUID - Concurrency control: Redis semaphore per user (max 2 concurrent jobs) - Fallback to template-based variants on LLM failure (full refund) - Partial generation: per-variant refund

FOLLOW-UP QUESTIONS: 1. How do you ensure the LLM doesn't leak sensitive lead data into another user's outreach? 2. What's the cost per outreach email? 3. How do you test the LLM gateway without burning real money?

FOLLOW-UP ANSWERS: 1. Two layers: (a) prompt template uses only the lead's public data (business name, niche, city, website quality score) — never phone/email/PII; (b) the LLM

gateway runs per-request with a unique session id and we explicitly tell the model “do not include data from other contexts”. We tested this with synthetic prompts containing honeypot PII and verified the model doesn’t leak across requests. We also don’t use OpenAI’s training-data-sharing APIs. 2. \$0.044 per 3-variant set (Gemini Flash at \$0.075/M input, \$0.30/M output; prompt is ~600 input tokens, response is ~400 output tokens = \$0.045 + \$0.12 = \$0.165/M sets, divided by 1000 = \$0.044). We charge the user 2 credits × 3 variants = 6 credits. At ₹1.6/credit (Pro plan ₹1,599 / 500 monthly credits + addon packs), that’s ₹9.6 = \$0.11. Margin: 60%. Cache hits bring actual cost down to \$0.031 per set, raising margin to 72%. 3. WireMock for unit + integration tests: stub the LLM endpoint with canned JSON responses. We have 30 stub scenarios covering success, malformed JSON, timeout, 5xx, 4xx rate limit. For load testing, we use a “shadow” LLM gateway that mirrors real calls to a low-cost provider (gpt-3.5-turbo) — verifies our circuit breaker and retry logic under load without burning Gemini budget. For prod validation, we have a “synthetic user” cron job that generates 1 outreach per hour and asserts the response is well-formed and within latency SLA.

RED FLAGS TO AVOID: - Not mentioning cost (LLM is real money) - Skipping circuit breaker / retry - Forgetting cache (this is a senior answer) - Saying “we use OpenAI” without specifying which model

Q2-4: How did you design the meeting scheduling system

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: Meeting scheduling: the user wants to book a meeting with a lead. The system reads the user’s Google Calendar free/busy, picks 3 optimal slots based on AI Meeting Intelligence, sends them to the lead (email/WhatsApp), the lead picks one, we create a Google Calendar event with both attendees, send invites, schedule reminders.

Architecture:

```

User → POST /api/meetings/suggest (leadId, durationMin, range)
    | @PreAuthorize
    | @Transactional persist MeetingSuggestion(PENDING)
    |
    ▼
MeetingService.suggestSlots
    | GoogleCalendarService.listFreeBusy(user.googleCalendarId, range)
    |   | WebClient + GoogleOAuth2 authorized client (refresh token per user)
    |   | Resilience4j CB + retry
    | MeetingIntelligenceService.pickSlots(busyTimes, leadContext, userPrefs)
    |   | heuristic: prefer Tue/Wed/Thu, avoid lunch, avoid first/last hour
    |   | optional LLM call for "smart slot" (e.g. "lead's industry prefers
morning")
    | persist 3 MeetingSlot rows
    | return slot suggestions to UI

User picks a slot from UI → POST /api/meetings/{id}/propose
    | persist Meeting(PROPOSED, slot)
    | send slot to lead via EmailChannel or WhatsAppChannel
    | MeetingReminderScheduler.schedule(meeting.id, [24h, 1h] before)

Lead accepts slot → POST /api/meetings/{id}/confirm (token from email link)
    | GoogleCalendarService.createEvent(meeting)
    |   | attendees: [user.email, lead.email]
    |   | conferenceData: meet link (auto-generated)
    | send calendar invite to both parties
    | persist Meeting(CONFIRMED, calendarEventId)
    | Lead.stage = MEETING_BOOKED

@Scheduled reminders:
    | 24h before: send reminder to both parties + create follow-up action
    | 1h before: send reminder + push notification

Post-meeting:
    POST /api/meetings/{id}/complete (transcript or notes)
    | MeetingAssistantService.extractActions(notes)
    |   | LLM call to extract action items
    | persist FollowUpAction rows
    | Lead.stage = NEGOTIATION

```

Google Calendar integration: Each user OAuths their Google account once (we store the refresh token in `user_integration` table, encrypted with a key from Secrets Manager). On each Calendar API call, `GoogleOAuthService.getAuthorizedClient(userId)` checks if the access token is expired, refreshes via the refresh token, returns an `UserCredentials` instance. The `CalendarService` uses `Calendar` (Google API client lib) for `events().list()` (free/busy), `events().insert()` (create event), `events().patch()` (update).

Idempotency: Meeting creation is idempotent via `clientEventId` (UUID) — we store this in the `meeting` table and check before creating the Google Calendar event. If the same `clientEventId` comes twice, we return the existing event instead of creating a duplicate.

Token revocation: If the user revokes our Google OAuth, the next Calendar API call returns 401 with `invalid_grant` — we mark the user's `google_integration` as REVOKED, send a

notification “please re-connect your Google account”, and degrade gracefully (no meeting suggestions until they re-auth).

Resilience4j: Every Google API call is wrapped in CB (10% failure → open 30s), retry (exponential 1s/4s/16s, max 3), time limiter (10s). If CB is open, we fall back to manual slot entry (user picks a time and types it in).

Time zone handling: All times are stored as `TIMESTAMP WITH TIME ZONE` in Postgres (UTC). The user’s preferred time zone is stored on `user.time_zone` (e.g. “Asia/Kolkata”). When displaying slots, we convert to the user’s TZ using `ZonedDateTime.withZoneSameInstant(ZoneId.of(user.getTimeZone()))`. The lead’s TZ is inferred from their phone country code (we have a country→TZ lookup table). Calendar invites are sent with the correct TZ so both parties see the right time.

Reminder scheduling: We use a `meeting_reminder` table with `next_fire_at` (timestamp) and `state` (PENDING/FIRED/CANCELLED). A `@Scheduled(fixedDelay = 60s)` job picks up reminders where `next_fire_at < now() AND state = PENDING`, fires them (sends email/WhatsApp + pushes notification), marks as FIRED, schedules the next one. This is more reliable than Quartz (no external dependency) and handles task restarts (reminders persist in DB). At 50K users with avg 5 meetings/week, we have ~25K reminders/day — the 60s poll is fine.

Concurrency: Two users booking the same lead at the same slot — we use a unique constraint on `(lead_id, start_at, status=CONFIRMED)` — the second `INSERT` fails with P0001, we catch and return “slot taken, please pick another”.

Failure modes: - Google Calendar API down → CB opens → meeting is created in our DB with `calendarEventId=NULL` and `pendingSync=true`; a `@Scheduled` job retries every 5 minutes until Calendar API recovers; the user sees “your meeting is booked, calendar invite will arrive shortly” - Lead declines → Meeting(CANCELLED), Lead.stage = DISCUSSION (we don’t lose the lead) - Lead doesn’t respond in 72h → Meeting(EXPIRED), Lead.stage stays DISCUSSION, user notified - Reminder fails to send → reminder state stays PENDING, retried in next poll

KEY TERMS TO MENTION: - Google Calendar API via Google OAuth2 (refresh token per user, encrypted) - `UserCredentials` authorized client, `Calendar` service - Resilience4j CB + retry + time limiter (10s) around every Google call - Idempotency via `clientEventId` UUID - All times stored as `TIMESTAMP WITH TIME ZONE` (UTC), converted via `ZonedDateTime` - Reminder scheduling: `meeting_reminder` table + `@Scheduled(fixedDelay=60s)` poll (more reliable than Quartz, handles restarts) - Unique constraint on `(lead_id, start_at, status=CONFIRMED)` for concurrent booking - Graceful degradation: meeting created in DB even if Calendar API down, `@Scheduled` retry job

FOLLOW-UP QUESTIONS: 1. How do you handle daylight saving time transitions? 2. What if the user has multiple Google accounts? 3. How would you implement a custom availability calendar (not Google)?

FOLLOW-UP ANSWERS: 1. We store all times as UTC and convert to user TZ at display time. DST transitions are handled by `java.time` automatically — `ZonedDateTime.withZoneSameInstant(ZoneId.of("America/New_York"))` correctly handles the offset change. The reminder `next_fire_at` is stored in UTC, so a 1-hour-before reminder fires

at the right wall-clock time even across DST. We tested this with the US DST transition in March 2026 — no bug. 2. We support one Google account per user (most users have one work account). Multi-account support is on the roadmap — would require `user_integration` to have multiple rows per user with `provider_user_id`, and the user picks which account to use for each meeting. Not trivial — Google Calendar API doesn't have a clean "list my accounts" flow. 3. We have a `CalendarService` interface with `GoogleCalendarService` as the implementation. Adding `MicrosoftCalendarService` (Microsoft Graph API) or `InternalCalendarService` (just our DB) is a new implementation. The `MeetingService` depends on `CalendarService`, not `GoogleCalendarService` — clean DI. We'd add `MicrosoftCalendarService` when we have enterprise customers who use Outlook.

RED FLAGS TO AVOID: - Not mentioning time zones (this is a senior answer) - Forgetting token revocation handling - Saying "we use Quartz" (over-engineered for our scale) - Skipping the reminder persistence design

Q2-5: How does the real-time notification system work

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: Real-time notifications: when a lead replies, a payment succeeds, a meeting reminder fires, or an AI outreach completes — the user gets a notification in their UI within ~500ms. We support 4 channels: in-app (WebSocket), email, push (PWA Web Push), and third-party (Telegram/WhatsApp if connected).

Architecture:

```

Event source (e.g. ReplyIntelligenceService.classifyReply())
|
▼
ApplicationEventPublisher.publishEvent(new NotificationEvent(userId, type, payload))
|
▼
@EventListener NotificationService.onNotificationEvent(NotificationEvent)
├─ persist Notification row (in-app store, paginated)
├─ fan-out via ChannelRouter
│   ├── InAppChannel → WebSocket push (SSE manager per user)
│   ├── EmailChannel → JavaMailSender async (if user opted in)
│   ├── PushChannel → Web Push API (VAPID keys, user subscribed)
│   └── TelegramChannel → Telegram Bot API (if connected)
└─ async – each channel via @Async (separate TaskExecutor per channel for isolation)

```

WebSocket / SSE: We use Server-Sent Events (SSE) for one-way push (server → client), not WebSocket. SSE is simpler (HTTP upgrade, no protocol negotiation), works through proxies, and supports auto-reconnect with `Last-Event-ID`. Each user opens `GET /api/notifications/sse` on login; the `SseManager` keeps a `ConcurrentHashMap<userId, List<SseEmitter>>` (a user can have multiple tabs). On notification, `SseManager.send(userId, event)` iterates emitters and calls `emitter.send(event)`.

If a send fails (client disconnected), we remove the emitter. Heartbeat every 30s to keep the connection alive.

Event persistence: Every notification is persisted to `notifications` table (`id`, `user_id`, `type`, `title`, `message`, `metadata` JSONB, `is_read`, `read_at`, `created_at`). The user can fetch history via `GET /api/notifications?cursor=...&limit=20`. SSE only delivers new notifications; the client pulls history on login.

Channel routing: The `ChannelRouter` decides which channels to fan out to based on user preferences (`user_notification_preferences` table — per-type opt-in per channel). For example: a “lead replied” notification sends in-app + email + Telegram (if user opted in for all three); a “credit low” notification sends only in-app + email (not push — too noisy).

Async dispatch: Each channel dispatch is `@Async` with a dedicated `ThreadPoolTaskExecutor` per channel (in-app: 4 threads, email: 8 threads, push: 4 threads, Telegram: 4 threads). This isolates failures — if Telegram API is down, email + in-app still work. Each channel has its own Resilience4j CB.

Idempotency: Every notification has an `event_id` (UUID from the source event). The `notifications` table has a unique constraint on (`user_id`, `event_id`). If the source event retries (e.g. Kafka consumer redelivers), the second insert fails silently — no duplicate notifications.

Reconnect / replay: If the SSE connection drops, the client reconnects with `Last-Event-ID: <last-seen-event-id>`. The server replays missed events from a Kafka backlog (`notification-events` topic, 24h retention). This handles brief network blips without losing notifications.

Performance: p95 from event-source to in-app display: 350ms (event publish 5ms, NotificationService persist 30ms, SSE send 5ms, client render 300ms). p95 for email: 8s (SMTP send). p95 for push: 1.2s (Web Push API).

Backpressure: If a user is offline for >7 days, we stop queuing in-app notifications (we keep only the last 50 in DB). Email notifications continue. We clean up old notifications via a `@Scheduled` nightly job.

Multi-tab consistency: If the user has 3 tabs open, all 3 SSE emitters receive the notification. The “mark as read” action is synced via a separate WebSocket message — when tab A marks as read, the server broadcasts “mark-read {id}” to tabs B and C via the SSE channel.

KEY TERMS TO MENTION: - SSE (not WebSocket) — simpler, HTTP upgrade, auto-reconnect with Last-Event-ID - `SseManager` with `ConcurrentHashMap<userId, List<SseEmitter>>`, 30s heartbeat - Persistence in `notifications` table, cursor pagination, JSONB metadata - `ChannelRouter` per user preferences (per-type, per-channel opt-in) - `@Async` per channel with dedicated `ThreadPoolTaskExecutor` for isolation - Resilience4j CB per channel - Idempotency via `event_id` UUID, unique constraint on (`user_id`, `event_id`) - Reconnect replay via Kafka backlog (`notification-events` topic, 24h retention) - p95 350ms event → in-app display - Backpressure: cap in-app notifications to 50 for offline users

FOLLOW-UP QUESTIONS: 1. How do you handle 50K concurrent SSE connections? 2. Why SSE over WebSocket? 3. How do you handle SSE connections through a load balancer?

FOLLOW-UP ANSWERS: 1. Each ECS task handles ~12K SSE connections (Tomcat NIO connector, default 10K acceptors + 5 selectors per task). 4 tasks = 48K concurrent — enough for 50K users (most are idle). The `SseManager` is in-memory per task — if a task dies, the connections on that task drop; the client reconnects to another task via the ALB. We use sticky sessions at the ALB (cookie `X-AcquisitionOS-Task`) to keep a user on the same task during a session, reducing reconnect churn. For >100K concurrent we'd move to a dedicated SSE service with horizontal scaling + Redis pub/sub for cross-task fan-out. 2. SSE is one-way (server → client), HTTP-upgraded, supports auto-reconnect with `Last-Event-ID`, works through all proxies (no protocol upgrade negotiation). WebSocket is two-way (we don't need client → server real-time — REST is fine for that), requires protocol upgrade (some corporate proxies block it), and is more complex to debug. For our use case (push notifications), SSE is the right tool. 3. ALB supports SSE — it doesn't buffer responses (we set `X-Accel-Buffering: no` header to be safe). The ALB uses LRU to keep connections; we set idle timeout to 60 minutes (longer than our heartbeat interval). Sticky sessions via cookie ensure a user's SSE connection stays on one task — important because the `SseManager` is in-memory per task. If we ever need cross-task fan-out (e.g. one user connected to multiple tasks), we'd use Redis pub/sub: the publisher publishes to `notifications:user:{userId}` Redis channel, every task's subscriber relays to its local SSE emitters.

RED FLAGS TO AVOID: - Saying “we use polling” (senior engineers don't) - Forgetting the persistence + cursor pagination - Not mentioning idempotency (Kafka redelivers) - Skipping backpressure (offline users get spammed)

Q2-6: How would you scale this to 100k users

DIFFICULTY: Expert **CATEGORY TAG:** System Design

ANSWER: At 100K users (20x current scale), the architecture fundamentally still works — the modular monolith + PostgreSQL + Redis can handle 100K users with the right tuning. The changes are quantitative, not qualitative.

API tier (4 → 12 ECS tasks): 100K users / ~30% active / ~5% peak concurrency = ~1,500 concurrent sessions. At 1,200 RPS today with 4 tasks, 100K users → ~24K RPS → 12 tasks (each handles 2K RPS). Autoscaling triggers on CPU > 70% + ALB request count > 2000 per task. We'd move from 2 vCPU/4GiB to 4 vCPU/8GiB per task to handle the larger heap (more concurrent requests = more objects).

Database tier: Today's `db.r6g.large` (2 vCPU, 16GB) handles 1,200 RPS at 65% CPU. 100K users → 24K RPS → `db.r6g.4xlarge` (16 vCPU, 64GB) at 70% CPU. We'd add 2 more read replicas (total 4) for analytics queries + lead listing (writes go to primary, reads to replicas via `@Transactional(readOnly=true)` routed by a `ReadWriteRoutingDataSource`). We'd partition the `lead` table by `created_at` (monthly partitions) — queries with `WHERE created_at > '2026-08-01'` hit only the relevant partition. We'd partition `credit_ledger` by `user_id % 10` for the same reason.

Connection pooling: HikariCP at 20 per task × 12 tasks = 240 connections to Postgres. `max_connections` on RDS = 1500 — comfortable headroom. We'd tune HikariCP to 30 per task (more concurrent queries) and add PgBouncer in front of Postgres (transaction-mode pooling) to handle 1000+ connections.

Redis: 3 shards at 3,200 tenants → 8 shards at 100K. Each shard handles ~5K ops/sec today; 100K users → ~30K ops/sec total → 8 shards × 4K ops/sec. We'd also add Redis Cluster's `HASH` slot migration tooling for online resharding.

Async work: Spring Batch lead discovery — today runs on 4 task executor threads. 100K users → 20x more jobs → 40 task executor threads (10 ECS tasks × 4 each). We'd move lead discovery to a separate ECS service (not blocking API tasks) — 4 dedicated tasks with 8 task executor threads each = 32 concurrent discovery jobs. Same for AI outreach generation.

LLM cost: 20x scale → 20x LLM calls = \$84,000/month. We'd negotiate volume pricing with Google (Gemini Flash at \$0.045/M tokens with 30% volume discount = \$0.032/M), tighten cache hit ratio to 50% (better cache key design), and introduce a “warm-up” job that pre-generates outreach for popular niches during off-peak. Combined, we'd keep LLM cost at ~\$30K/month (15% of revenue).

Stripe webhooks: 3K/day today → 60K/day at 100K users. The webhook handler is already async (ingest → Kafka → consumer). We'd scale the consumer to 4 partitions × 4 threads = 16 concurrent consumers. Idempotency keys prevent duplicates.

Search: Lead search today uses PostgreSQL full-text (`tsvector` + GIN index). At 100K users × 100 leads each = 10M leads, PG FTS handles it but p95 grows to 800ms. We'd move to Elasticsearch (or OpenSearch) for search — better fuzzy matching, better relevance ranking, scales horizontally. Postgres remains the source of truth; we sync to ES via Kafka topic `lead-events` consumed by an ES indexer.

Real-time: SSE today is 12K connections per task. 100K users × 30% active × 1 connection each = 30K connections → 12 tasks × 2.5K each (within Tomcat NIO limits). We'd add Redis pub/sub for cross-task fan-out (a user's notification published to `notifications:user:{userId}` channel; every task subscribed, relays to local emitters). This lets us scale beyond sticky-session limits.

Storage: S3 attachments + invoices — at 100K users × 10MB each = 1TB. S3 scales infinitely, but we'd add S3 Lifecycle policies: move to IA after 30 days, Glacier after 1 year.

Multi-region: At 100K users, we'd consider deploying in 2 regions (ap-south-1 + ap-southeast-1) for latency + DR. Postgres cross-region replication (logical) is complex but doable. Most users are India-based today, so we'd add a Singapore region for SE Asia customers. The application would need region-aware routing at the ALB (geo-DNS) + region-aware data residency.

Operational: 100K users means 100K support tickets/year. We'd invest in better admin tooling (admin API for refunds, impersonation, debugging), better self-serve (in-app chatbot), better observability (per-tenant dashboards for support).

Cost estimate: Today: ~\$8K/month infra. 100K users: ~\$40K/month (4 ECS tasks × 4 vCPU = \$1.2K, RDS r6g.4xlarge = \$3K, Redis 8 shards = \$1.5K, S3 + CloudFront = \$500, LLM at

\$30K, observability \$1K, misc \$2.8K). Revenue at 100K users: ₹4.2M × 20 = ₹84M = \$1M/month. Margin: 96%.

KEY TERMS TO MENTION: - API: 4 → 12 ECS tasks, 4 vCPU/8GiB each, PgBouncer for connection pooling - DB: r6g.large → r6g.4xlarge, 4 read replicas, partitioning by `created_at` (leads) + `user_id % 10` (credit_ledger) - Redis: 3 → 8 shards, online resharding - Async work: dedicated ECS service for lead discovery + AI outreach - LLM: volume pricing, 50% cache hit, warm-up jobs - Stripe webhooks: 4 partitions × 4 threads = 16 consumers - Search: Postgres FTS → Elasticsearch via Kafka sync - SSE: 12 tasks × 2.5K connections, Redis pub/sub for cross-task fan-out - Multi-region: ap-south-1 + ap-southeast-1, logical Postgres replication, geo-DNS - Cost: \$40K/month infra, \$1M/month revenue, 96% margin

FOLLOW-UP QUESTIONS: 1. What's the first bottleneck you'd hit at 100K users? 2. How do you handle multi-region database writes? 3. What's the migration plan from current to 100K?

FOLLOW-UP ANSWERS: 1. The Postgres primary's write throughput. Today we do ~200 writes/sec at 1,200 RPS. 100K users → 4K writes/sec — close to the r6g.4xlarge's ceiling (5K writes/sec). We'd shard the highest-write table (`credit_ledger`) by `user_id % 10` across 10 Postgres instances. The `credit_balance` becomes a cached projection in Redis (write-behind). The `lead` table shards by `org_id % 5` across 5 instances. Cross-shard queries (admin dashboards) become slow — we'd add an analytics warehouse (Redshift or BigQuery) fed by Kafka. 2. Multi-region writes are hard. We'd use a "primary region + secondary region" model: all writes go to the primary (ap-south-1), logical replication to the secondary (ap-southeast-1, ~1 second lag). Reads in the secondary region hit the local replica (slightly stale). For conflict-free writes, we'd partition users by region (`user.region = 'in'` or `'sg'`) and route writes to their region's primary; cross-region reads go through a proxy that picks the right region. This is the standard "geo-distributed active-passive" pattern. True multi-master (CockroachDB, YugabyteDB) is on the table if we hit 1M users. 3. Six-month migration plan: (1) month 1–2: partition `lead` + `credit_ledger`, add PgBouncer, tune HikariCP, move lead discovery + AI to separate ECS service; (2) month 3: add Elasticsearch for search, dual-write (PG + ES), validate consistency; (3) month 4: add 2 more read replicas, route read-only transactions to replicas via `@Transactional(readOnly=true)`; (4) month 5: multi-region standby — deploy in ap-southeast-1, set up logical replication, run shadow traffic; (5) month 6: cut over SE Asia users to the Singapore region. Each step is reversible (we can roll back to the previous architecture).

RED FLAGS TO AVOID: - Saying "we'd use microservices" (the architecture is fine; you scale the monolith) - Skipping specific AWS instance types (looks shallow) - Forgetting cost estimates (a senior thinks about money) - Saying "we'd use DynamoDB" (Postgres is fine; don't migrate just for scale)

Q2-7: How did you handle concurrent users

DIFFICULTY: Medium **CATEGORY TAG:** System Design

ANSWER: "Concurrent users" means two things for us: concurrent HTTP requests on the API tier, and concurrent writes to the same data on the DB tier. We handle them differently.

Concurrent HTTP requests: At peak (Indian business hours, 11am–3pm IST), we have ~1,200 RPS across 4 ECS tasks = 300 RPS per task. Each request takes p50 65ms, p95 180ms. So per task, we have $300 \times 0.065 = \sim 20$ concurrent requests in flight (p50), $300 \times 0.18 = \sim 54$ at p95. Spring Web MVC's Tomcat NIO connector handles this trivially — default `maxThreads = 200`, we configured 100 per task (more than enough). The bottleneck is not threads; it's DB connections. HikariCP at 20 per task $\times$ 4 tasks = 80 connections to Postgres (max_connections = 100 on RDS). We monitor HikariCP utilisation — peaks at 65%, comfortable.

Concurrent writes to the same row: The credit balance row is the classic hot row. Two concurrent `POST /api/outreach/generate` for the same user both want to deduct credits. We use `@Transactional(isolation=REPEATABLE_READ) + SELECT ... FOR UPDATE` on the `credit_balance` row — second transaction blocks until the first commits, then sees the updated balance and proceeds. We measured: at 100 concurrent deductions for the same user (synthetic test), p95 = 12ms — fast because the lock is held for the duration of the transaction (~10ms).

Concurrent writes to different rows (lead update by different users): Optimistic locking via `@Version` on the `Lead` entity. Two users update different fields of the same lead — first commits, second gets `OptimisticLockException`, we retry once with the refreshed entity. If the conflict is on the same field (rare), we return 409 “lead was modified, please refresh”. We've seen ~5 retries/day across 50K users — fine.

Concurrent webhook handling: Stripe can retry a webhook up to 5 times in 5 minutes if we return non-2xx. If we're slow (>10s), Stripe retries. We made the webhook handler return 200 immediately after ingesting the raw payload to a `payment_webhook` table (no processing), and process asynchronously via Kafka consumer. This decouples Stripe's retry pressure from our processing speed. Idempotency via unique `event_id` prevents duplicates if Stripe retries a `processed` webhook.

Concurrent SSE connections: 12K per task today (4 tasks = 48K total). Each connection is a Tomcat NIO selector thread handling ~500 connections. We configured Tomcat NIO with 10 acceptor threads + 5 selector threads per task — handles 12K connections at 30% selector CPU. We have a Micrometer gauge for `sse.connections.active` per task — alerts at 10K (close to the 12K ceiling).

Concurrency control on shared resources: - AI gateway: Resilience4j Bulkhead (max 20 concurrent LLM calls per provider per task) — protects against one user monopolising - Scraping: per-source Resilience4j RateLimiter (Google 100/sec, LinkedIn 5/sec) - Stripe API: Resilience4j RateLimiter (100/sec — Stripe's published limit) - Per-user rate limiting: Redis sliding window per endpoint (`/api/leads/discover` 20/min/user, `/api/outreach/generate` 10/min/user)

Distributed lock for cron dedup: We have ~14 `@Scheduled` jobs. If two ECS tasks happen to fire the same cron at the same time (no guarantee of single execution with `@Scheduled`), we use Redis `SET NX PX 30000` as a distributed lock — first task acquires, others skip. The lock auto-expires after 30 seconds; the job completes within 25 seconds.

Read replicas for read scaling: All `@Transactional(readOnly = true)` queries are routed to a read replica via `ReadWriteRoutingDataSource`. Read replicas have ~1s replication lag — for “freshness required” queries (e.g. “show me my just-created lead”), we force `@Transactional(readOnly = false)` to hit the primary.

KEY TERMS TO MENTION: - Tomcat NIO connector, 100 threads per task (peak 54 in flight at p95) - HikariCP 20 per task × 4 = 80 connections, peaks at 65% utilisation - `SELECT ... FOR UPDATE` for hot-row writes (credit balance), 12ms p95 at 100 concurrent - `@Version` optimistic lock for low-contention updates (Lead), 5 retries/day - Stripe webhook: ingest + ack immediately, process async via Kafka, idempotency via `event_id` - SSE 12K connections per task, Tomcat NIO 10 acceptor + 5 selector threads - Resilience4j Bulkhead (LLM 20 concurrent per provider per task) + RateLimiter (per source, per user) - Redis `SET NX PX 30000` for cron dedup - `ReadWriteRoutingDataSource` for read replica routing of `@Transactional(readOnly=true)`

FOLLOW-UP QUESTIONS: 1. What if a single user fires 1000 concurrent requests? 2. How do you handle a long-running transaction holding a lock? 3. How do you test concurrency before deploying?

FOLLOW-UP ANSWERS: 1. Per-user rate limit (Redis sliding window: `/api/leads/discover` 20/min/user) returns 429 after the 20th request. The Redis script is atomic (Lua script, single round-trip). If the user is on a paid plan, we increase the limit (Pro 50/min, Elite 100/min). We also have a per-IP rate limit (5/min/auth endpoint) to prevent credential stuffing. For abuse (1 user generating 10K req/sec from a botnet with rotating IPs), we’d add WAF rules (rate-based) at the ALB — but we haven’t needed this yet. 2. Two ways: (a) `@Transactional(timeout = 30)` — Spring throws `TransactionTimeoutException` after 30s, rolls back, releases the lock. (b) `LockWaitTimeout` in Postgres — `SET LOCAL lock_timeout = '10s'` per transaction — Postgres throws `LockTimeoutException`, we catch and retry once with a smaller scope. We use both — Spring timeout is the outer guard, Postgres `lock_timeout` is the inner guard for the hot row specifically. 3. Three layers: (a) unit tests with `@SpringBootTest` + Testcontainers (PG + Redis) simulate concurrent requests via `CompletableFuture.allOf(...)` — we test 100 concurrent credit deductions and assert the final balance is correct; (b) load tests with Gatling — 1000 virtual users, mixed workload (read 70%, write 20%, AI 5%, billing 5%), 10 minutes, assert no 5xx > 0.1% and p95 < 500ms; (c) chaos testing in staging — we kill an ECS task mid-batch and verify Spring Batch resumes correctly. We run chaos tests weekly.

RED FLAGS TO AVOID: - Saying “we use threads” without specifics - Forgetting rate limiting (concurrent ≠ unbounded) - Skipping the hot-row problem (credit balance) - Saying “no locks, just transactions” (that’s not how it works)

Q2-8: What is your database design for the lead pipeline

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: The lead pipeline database design is centred on the `lead` table with ~30 supporting tables. The core entities are: `Lead`, `LeadActivity`, `LeadCommunication`, `LeadScore`, `LeadNote`, `Deal`, `Meeting`, `PipelineStage`, `StageTransition`.

`Lead` table (the core):

```
CREATE TABLE lead (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organization(id),
  user_id     UUID NOT NULL REFERENCES user_account(id),
  business_name TEXT NOT NULL,
  owner_name  TEXT,
  email       TEXT,
  phone       TEXT,
  whatsapp    TEXT,
  website     TEXT,
  linkedin    TEXT,
  city        TEXT,
  country     TEXT,
  niche       TEXT,
  stage       stage_enum NOT NULL DEFAULT 'discovered', -- discovered->...->won/lost
  reply_score SMALLINT DEFAULT 0, -- 0-100
  conversion_score SMALLINT DEFAULT 0,
  urgency_score SMALLINT DEFAULT 0,
  email_status email_status_enum, -- pending/sent/failed
  has_website  BOOLEAN DEFAULT false,
  website_quality TEXT,
  estimated_quality TEXT,
  estimated_revenue TEXT,
  tags         JSONB DEFAULT '[]'::jsonb,
  source       TEXT, -- linkedin, google_places, manual, csv_import
  is_active    BOOLEAN DEFAULT true,
  deleted_at   TIMESTAMP, -- soft delete
  created_at   TIMESTAMP NOT NULL DEFAULT now(),
  updated_at   TIMESTAMP NOT NULL DEFAULT now()
);

-- Indexes
CREATE INDEX idx_lead_user_stage ON lead(user_id, stage) WHERE is_active AND deleted_at IS NULL;
CREATE INDEX idx_lead_user_created ON lead(user_id, created_at DESC) WHERE is_active;
CREATE INDEX idx_lead_org_niche ON lead(org_id, niche);
CREATE INDEX idx_lead_tags_gin ON lead USING GIN(tags);
CREATE INDEX idx_lead_search_fts ON lead USING GIN(to_tsvector('english', business_name || ' ' || coalesce(owner_name, '') || ' ' || coalesce(email, '')));
```

Soft delete + active flag: We use `is_active = true` AND `deleted_at IS NULL` for “live” leads. Partial indexes (`WHERE is_active AND deleted_at IS NULL`) keep them small. A deleted lead is queryable for audit but excluded from default queries.

Tags as JSONB: Tags are a JSONB array (`["hot", "enterprise", "q4-target"]`). GIN index supports `WHERE tags ? 'hot'` or `WHERE tags @> '["hot","enterprise"]'`. We chose JSONB over

a separate `lead_tag` join table because we rarely query “all leads with tag X” by tag alone (always filtered by `user_id` first), so the join table would be wider than the JSONB index.

Stages: `stage_enum` is a Postgres enum: `discovered, contacted, replied, interested, discussion, proposal, negotiation, won, lost`. Stage transitions go through `StageTransitionService` which records to `stage_transition` table for audit:

```
CREATE TABLE stage_transition (
  id          UUID PRIMARY KEY,
  lead_id     UUID NOT NULL REFERENCES lead(id),
  from_stage  stage_enum NOT NULL,
  to_stage    stage_enum NOT NULL,
  reason      TEXT, -- user_manual, reply_classified, deal_won
  metadata    JSONB,
  user_id     UUID NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_stage_transition_lead ON stage_transition(lead_id, created_at);
```

Communications: Every email/WhatsApp/Telegram inbound or outbound is in `lead_communication`:

```
CREATE TABLE lead_communication (
  id          UUID PRIMARY KEY,
  lead_id     UUID NOT NULL REFERENCES lead(id),
  user_id     UUID NOT NULL,
  channel     channel_enum NOT NULL, -- email, whatsapp, telegram
  direction   direction_enum NOT NULL, -- inbound, outbound
  subject     TEXT,
  body        TEXT,
  intent      TEXT, -- classified by ReplyIntelligence: interested, not_now,
                unsubscribe
  message_id  TEXT, -- SMTP message-id for threading
  opened_at   TIMESTAMP,
  clicked_at  TIMESTAMP,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_comm_lead_created ON lead_communication(lead_id, created_at);
```

Lead score history: AI scores change over time, so we keep history:

```
CREATE TABLE lead_score (
  id          UUID PRIMARY KEY,
  lead_id     UUID NOT NULL REFERENCES lead(id),
  model       TEXT NOT NULL, -- gemini-1.5-pro, heuristic-v2
  reply_score SMALLINT,
  conversion_score SMALLINT,
  urgency_score SMALLINT,
  reasoning   JSONB, -- LLM explanation
  scored_at   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_score_lead_scored ON lead_score(lead_id, scored_at DESC);
```

Deals: When a lead progresses to a deal:

```
CREATE TABLE deal (
  id          UUID PRIMARY KEY,
  lead_id     UUID NOT NULL REFERENCES lead(id),
  user_id     UUID NOT NULL,
  proposed_price NUMERIC(12,2),
  final_price  NUMERIC(12,2),
  currency    TEXT NOT NULL DEFAULT 'INR',
  stage       deal_stage_enum NOT NULL DEFAULT 'proposal',
  status      deal_status_enum NOT NULL DEFAULT 'open', -- open, won, lost
  closed_at   TIMESTAMP,
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  updated_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_deal_user_status ON deal(user_id, status, created_at DESC);
```

Activities: Audit-like record of every action on a lead (called, emailed, note added, stage moved):

```
CREATE TABLE lead_activity (
  id          UUID PRIMARY KEY,
  lead_id     UUID NOT NULL REFERENCES lead(id),
  user_id     UUID NOT NULL,
  type        activity_type_enum NOT NULL, -- call, email, note, stage_move,
  deal_created
  description TEXT,
  metadata    JSONB,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_activity_lead_created ON lead_activity(lead_id, created_at DESC);
```

Partitioning (planned for 100K users): At 100K users × 100 leads = 10M leads, the `lead` table is partitioned by `created_at` (monthly). The `lead_activity` and `lead_communication` tables (much larger) are partitioned by `lead_id` hash (10 partitions) so a single lead's history is in one partition.

Read replicas: `@Transactional(readOnly = true)` queries route to read replicas. The `Lead` listing (`GET /api/leads?page=1`) is read-only and routes to a replica. The

`POST /api/leads/discover` write goes to the primary. Replication lag is ~1s; for “freshness required” queries we force primary via `@Transactional(readOnly = false)`.

Performance:

`SELECT * FROM lead WHERE user_id = ? AND stage = ? ORDER BY created_at DESC LIMIT 20` hits the partial index `idx_lead_user_stage` — p95 8ms. Full-text search `WHERE to_tsvector(...) @@ plainto_tsquery(?)` hits `idx_lead_search_fts` — p95 25ms. Tag filter `WHERE tags ? 'hot'` hits the GIN index — p95 12ms. With 67 leads today it’s instant; we load-tested with 100K leads per user — p95 stays under 80ms thanks to the partial indexes.

KEY TERMS TO MENTION: - `lead` table: UUID PK, `org_id / user_id` FKs, `stage` enum, scores as `SMALLINT`, `tags` JSONB, soft delete - Partial indexes `WHERE is_active AND deleted_at IS NULL` — small, fast - GIN index on JSONB tags + GIN full-text index on `to_tsvector` - `stage_transition` table for stage audit history - `lead_communication` for inbound/outbound messages with intent classification - `lead_score` history (multiple models, scores change over time) - Partitioning plan: `lead` by `created_at` (monthly), `lead_activity / lead_communication` by `lead_id` hash (10 partitions) - Read replica routing via `@Transactional(readOnly=true)` + `ReadWriteRoutingDataSource` - p95 with partial indexes: 8ms (filter), 25ms (FTS), 12ms (tag)

FOLLOW-UP QUESTIONS: 1. Why UUID PKs over BIGSERIAL? 2. Why JSONB for tags over a join table? 3. How do you handle a stage transition rollback?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) UUIDs are globally unique — we can generate them client-side (offline-first) and they don’t collide; (b) UUIDs don’t reveal order (BIGSERIAL lets an attacker enumerate leads by id); (c) UUIDs work across shards (when we shard, two shards can both generate UUIDs without coordination). The trade-off: UUIDs are 16 bytes vs 8 bytes for BIGSERIAL, so indexes are larger — but the benefits outweigh the cost. 2. We tested both. The join table (`lead_tag`) was wider than the JSONB array because the GIN index on JSONB is ~5x smaller than a btree on a join table for our cardinality (avg 3 tags per lead, 50K leads = 150K join rows vs 50K JSONB arrays). The JSONB approach also makes “find leads with all of these tags” (`tags @> '["hot","enterprise"]'`) a single index lookup vs a 3-way join. We chose JSONB for the simplicity; the trade-off is no foreign-key integrity (a typo like “Hot” vs “hot” creates a duplicate tag) — we accept this and validate on write. 3. We don’t roll back — we append a new `stage_transition` row with `to_stage = <previous>`. The current stage of a lead is `last_non_cancelled_transition.to_stage` (we never delete transitions). This is the event-sourcing approach for stage history — full audit, time-travel queries (“what was this lead’s stage on 2026-08-15?”). The trade-off: queries are slower than a simple `lead.stage` column — but we cache `lead.stage` (the current value) and use the transition table only for history.

RED FLAGS TO AVOID: - Not showing actual SQL (interviewers want to see the schema) - Forgetting indexes (no index = no senior) - Saying “we use MongoDB” (Postgres is fine for our scale) - Skipping soft delete (every senior handles deletion carefully)

Q2-9: How did you design the API layer

DIFFICULTY: Medium **CATEGORY TAG:** System Design

ANSWER: The API layer follows a strict Controller-Service-Repository pattern with a few conventions.

URL structure: RESTful, resource-oriented.

`/api/leads`, `/api/leads/{id}`, `/api/leads/{id}/activities`, `/api/outreach/generate`, `/api/meetings`, `/api/...`

We use plural nouns for collections, kebab-case for multi-word (`/api/magic-link` not `/api/magiclink`). Versioning via URL prefix (`/api/v1/...`) — we're still on v1, but we've designed for v2 to coexist.

HTTP methods: GET (read, idempotent, cacheable), POST (create, not idempotent — we use `requestId` for idempotency), PUT (full replace, idempotent), PATCH (partial update, idempotent), DELETE (idempotent). We expose OPTIONS for CORS preflight.

Status codes: 200 (GET success), 201 (POST create), 202 (POST accepted, async), 204 (DELETE, PUT, PATCH no body), 400 (validation), 401 (no auth), 403 (auth but no permission), 404 (not found), 409 (conflict — e.g. email already exists), 422 (semantic error — e.g. plan doesn't allow this), 429 (rate limited), 500 (server error), 503 (dependency down — e.g. Stripe API down).

Response format: JSON, always. Success: `{"data": ..., "meta": {...}}` for collections (meta has pagination). Error: `{"error": {"code": "INSUFFICIENT_CREDITS", "message": "...", "details": {...}}}`. We use a consistent error code enum (`ErrorCode`) so the client can switch on it.

Validation: Bean Validation via `@Valid` on the DTO. The DTO has `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`, `@Pattern`. Custom validators for business rules: `@ValidPlan`, `@ValidCreditAmount`. A `@RestControllerAdvice` (`GlobalExceptionHandler`) catches `MethodArgumentNotValidException` and returns 400 with field-level errors.

DTO ↔ Entity mapping: MapStruct compile-time mapper. Each context has a `*Mapper` interface (`LeadMapper`). The mapper converts `LeadDto` ↔ `Lead` entity. MapStruct generates the implementation at compile time — no runtime reflection, fast, type-safe. We never expose entities directly (no JPA lazy-loading issues during JSON serialisation).

Pagination: Cursor-based for large collections (better than offset for deep pagination). GET `/api/leads?cursor=eyJpZCI6ImFiYyJ9&limit=20` returns `{"data": [...], "meta": {"nextCursor": "..."} }`. The cursor is a base64-encoded JSON of the last item's sort key (`created_at` + `id`). Offset-based for small collections (`?page=1&limit=20`) — simpler for admin UIs.

Filtering: Query params with a documented convention. `?stage=won&niche=dentist&tags=hot,enterprise&createdAfter=2026-08-01`. We use JPA Specifications for dynamic queries — `LeadSpecifications.hasStage(stage).and(hasNiche(niche)).and(hasTags(tags))` composed at runtime.

Sorting: `?sortBy=createdAt:desc,replyScore:desc`. We allowlist sortable fields in the controller to prevent injection via dynamic ORDER BY.

Rate limiting: Per-user, per-endpoint via Redis sliding window. `@RateLimit` annotation on controller methods, processed by `RateLimitAspect`. Limits per endpoint: `/api/leads/discover` 20/min/user, `/api/outreach/generate` 10/min/user, `/api/auth/otp/request` 5/min/IP, `/api/auth/signin` 5/min/IP. Returns 429 with `Retry-After` header.

Auth: JWT in `Authorization: Bearer <token>` header or `access_token` cookie (for browser clients). `JwtAuthenticationFilter` (extends `OncePerRequestFilter`) verifies and sets `SecurityContext`. `@PreAuthorize` on methods for fine-grained RBAC.

CORS: Allowlist specific origins (the marketing site, the app, localhost for dev) via `CorsConfigurationSource` bean. Credentials allowed (cookies). Preflight cache 1 hour.

Idempotency: POST endpoints accept `Idempotency-Key` header (UUID). The controller checks `idempotencyStore.get(key)` (Redis, 24h TTL) — if exists, returns the cached response. We use this for `POST /api/payments/create-checkout-session`, `POST /api/leads/discover`, `POST /api/outreach/generate`.

Versioning: URL-based (`/api/v1/`, `/api/v2/`). v1 and v2 run side-by-side; v1 is maintained for backwards compat with older mobile clients. We've never had to ship v2 yet — we add fields (additive) rather than rename (breaking).

Documentation: springdoc-openapi generates OpenAPI 3 spec from the controllers + DTOs + `@Operation` annotations. Swagger UI at `/api-docs`. We version the spec with the API.

HATEOAS: We don't follow HATEOAS strictly — our API is "RESTful but not hypermedia". We include `_links` for the most common next actions (e.g. `GET /api/leads/{id}` returns `_links: {activities: "/api/leads/{id}/activities", deals: "/api/leads/{id}/deals"}`) but not for every possible transition. Full HATEOAS is too much overhead for our client (which is our own SPA — it already knows the URLs).

KEY TERMS TO MENTION: - RESTful, plural nouns, kebab-case - HTTP methods, status codes (202 for async!) - JSON response: `{"data": ..., "meta": {pagination}}` for collections, `{"error": {code, message, details}}` for errors - Bean Validation (`@Valid`), `@RestControllerAdvice` `GlobalExceptionHandler`, custom `@ValidPlan` - MapStruct compile-time mapper (no JPA serialisation issues) - Cursor-based pagination for large collections (better than offset for deep pagination) - JPA Specifications for dynamic filtering - Allowlist sortable fields (prevent ORDER BY injection) - `@RateLimit` annotation + Redis sliding window, `Retry-After` header - JWT bearer + cookie, `JwtAuthenticationFilter`, `@PreAuthorize` - `Idempotency-Key` header (UUID) for POST endpoints, Redis 24h TTL - URL versioning (`/api/v1/`, `/api/v2/`), additive changes only - springdoc-openapi for OpenAPI 3 + Swagger UI - Partial HATEOAS (`_links` for common next actions)

FOLLOW-UP QUESTIONS: 1. Why cursor pagination over offset? 2. How do you deprecate an API endpoint? 3. How do you handle backwards compatibility when changing a response field?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) offset pagination is $O(\text{offset})$ — `LIMIT 20 OFFSET 100000` scans 100K rows; cursor is $O(\text{limit})$; (b) offset pagination returns

duplicates if a new item is inserted between two pages (the new item pushes everything down); cursor is stable; (c) cursor can be encrypted (opaque to client) so we can change the sort order without breaking clients. The trade-off: cursor can't jump to "page 50" — but nobody does that in our app anyway. 2. We add a `Deprecation: true` header + `Sunset: <date>` header to the deprecated endpoint. We document the migration path in the API docs. We keep the endpoint alive for 6 months after deprecation, monitoring usage — when usage drops below 1% of clients, we 410 Gone. We've deprecated 2 endpoints so far (the old `POST /api/leads/import-v1` was replaced by `POST /api/leads/import` with a different schema). 3. Additive only — we never remove or rename a field. If we need to rename, we add the new field alongside the old (both populated) for one release, then deprecate the old in the next release (still populated, just hidden from docs), then remove in a third release (with `Sunset` warning). For type changes (e.g. `createdAt` from epoch-millis to ISO-8601 string), we add a new field `createdAtIso` and keep the old `createdAt` for one release, then remove.

RED FLAGS TO AVOID: - Saying "REST" without specifics (everyone says REST) - Forgetting idempotency (POST without idempotency = double-charges) - Skipping rate limiting (interviewers want to hear Redis + sliding window) - Saying "we use GraphQL" (you don't, your API is REST)

Q2-10: How does the payment and subscription system work architecturally

DIFFICULTY: Hard **CATEGORY TAG:** System Design

ANSWER: Payments and subscriptions: the user upgrades from Free to Pro/Elite via Stripe Checkout, we receive a webhook when payment succeeds, we activate the subscription, grant monthly credits, and start the renewal cycle. Cancellations, refunds, and failed payments are all handled via Stripe webhooks.

Architecture:

```

User → POST /api/payments/create-checkout-session {plan: pro, billingCycle: monthly}
    | @PreAuthorize
    | PaymentService.createCheckoutSession(user.id, plan, billingCycle)
    |   ├── validate plan + cycle (Free → Pro/Elite; Pro → Elite)
    |   ├── create PaymentOrder(PENDING) in DB
    |   ├── call Stripe API: Checkout.sessions.create(...)
    |   |   ├── priceId from env var STRIPE_PRICE_ID_PRO_MONTHLY (server-side)
    |   |   ├── metadata: { order_id, user_id, plan, billingCycle }
    |   |   └── success_url: /api/payments/stripe-success?
    |   └── session_id={CHECKOUT_SESSION_ID}
    |       └── return { checkoutUrl: session.url }
    ▼
User → Stripe Checkout (hosted by Stripe) → pays → Stripe redirects to success_url
    ▼
GET /api/payments/stripe-success?session_id=...
    | StripeSuccessController: shows "processing" page (no business logic)
    |
    ▼ (in parallel, Stripe fires webhook)

POST /api/payments/webhook/stripe (with Stripe-Signature header)
    | StripeWebhookController:
    |   ├── read raw body (request.text() – never JSON.parse before verify)
    |   ├── verify signature: Stripe.Webhook.constructEvent(body, sig,
    |   └── STRIPE_WEBHOOK_SECRET)
    |       ├── if invalid → 400 (Stripe will retry)
    |       ├── idempotency check: paymentWebhookRepository.findByEventId(event.id)
    |       |   ├── if exists & processed → return 200 (idempotent)
    |       ├── persist raw webhook in payment_webhook table (PROCESSED=false)
    |       ├── return 200 immediately (don't block Stripe)
    |       └── async: publish StripeWebhookReceivedEvent(event) → Kafka topic
    ▼
Kafka consumer: StripeWebhookProcessor.process(event)
    | @Transactional
    | switch(event.type):
    |   ├── checkout.session.completed → activateOrder(event)
    |   |   ├── find PaymentOrder by event.metadata.order_id
    |   |   ├── if order.status == PENDING:
    |   |   |   ├── order.status = PAID
    |   |   |   └── SubscriptionService.activate(userId, plan, billingCycle,
    |   |   └── periodEnd)
    |   |       ├── CreditService.grantMonthlyCredits(userId, plan)
    |   |       ├── InvoiceService.generate(userId, order)
    |   |       └── NotificationService.notify(userId, "Welcome to Pro!")
    |   └── else: already processed (idempotent) – log
    |
    |   ├── invoice.payment_succeeded → renewSubscription(event)
    |   |   ├── find Subscription by event.customer
    |   |   ├── extend period_end
    |   |   ├── grant monthly credits
    |   |   └── NotificationService.notify("Subscription renewed")
    |
    |   └── invoice.payment_failed → handleFailedPayment(event)

```

```

├─ Subscription.status = PAST_DUE
├─ send dunning email (3 retries over 7 days)
├─ if still failing after 7 days: Subscription.status = CANCELLED
├─ customer.subscription.deleted → cancelSubscription(event)
├─ Subscription.status = CANCELLED
├─ user.plan = free (at period end, not immediately)
├─ NotificationService.notify("Subscription cancelled")
├─ charge.refunded → handleRefund(event)
├─ if full refund: Subscription.status = REFUNDED, user.plan =
free
├─ if partial: log only (no plan change)
▼
mark payment_webhook.PROCESSED=true, processed_at=now()

```

Idempotency: Every Stripe event has an `event_id` (`evt_...`). The `payment_webhook` table has a unique constraint on `event_id`. The webhook handler first inserts (catching the unique violation) — if the event already exists and is processed, return 200 immediately. If it exists but isn't processed (a previous attempt crashed), continue processing. This makes the webhook idempotent across Stripe retries, our crashes, and Kafka redeliveries.

Signature verification: We use Stripe's official library: `Webhook.constructEvent(rawBody, sigHeader, STRIPE_WEBHOOK_SECRET)`. The `rawBody` must be the raw bytes (not JSON-parsed) — we read with `request.text()` in Spring Web MVC (or `HttpServletRequest.getReader()` in raw servlet). If we JSON-parse first, the signature won't match (whitespace differences). This is a common bug.

Subscription state machine: `Subscription.status` is an enum: `TRIALING`, `ACTIVE`, `PAST_DUE`, `CANCELLED`, `REFUNDED`. Transitions are validated by `SubscriptionStateMachine` (a Spring bean) — e.g. you can't go from `CANCELLED` to `ACTIVE` without a new checkout. Every transition is audited to `subscription_history` table.

Credit granting: On activation + renewal, we grant monthly credits: Free 50, Pro 500, Elite 2000. The grant is an atomic append to `credit_ledger` + update to `credit_balance`. Rollover logic: unused credits from previous month roll over up to 2x monthly allowance.

Dunning: Failed payments get a dunning sequence: day 0 (failed notification), day 2 (retry, 50% succeed), day 5 (final retry), day 7 (cancel subscription). We use Stripe's Smart Retries (machine learning-driven retry timing) — better than fixed retries.

Refunds: Initiated by admin via `/api/admin/refund` (only `super_admin` role). We call Stripe's `Refunds.create()`. On the `charge.refunded` webhook, we mark the subscription as `REFUNDED` (full) or just log (partial). Refunds also issue a `credit_refund` ledger entry if the user had spent credits during the refunded period (rare edge case — we have a manual review for this).

GST invoicing: For Indian customers, we charge 18% GST. The Stripe price includes GST (we use Stripe Tax). The generated PDF invoice shows Base Price + GST = Total, with our

GSTIN. We use a custom `InvoicePdfService` (Apache PDFBox) that templates the invoice and uploads to S3 with a signed URL.

Reconciliation: Daily job (3am IST) pulls all Stripe events from the last 24h via Stripe API and reconciles with our `payment_webhook` table — if any Stripe event is missing in our DB, we backfill. Catches webhook delivery failures (rare but happens).

Failure modes: - Stripe webhook signature invalid → return 400, Stripe retries up to 5 times - Stripe webhook delivery fails (Stripe's fault) → reconciliation job backfills next day - Our webhook handler crashes mid-processing → webhook marked as RECEIVED but not PROCESSED; reconciliation job re-processes - Kafka consumer crashes → event redelivered (Kafka at-least-once), idempotency key prevents double-processing - Stripe API down during refund → Resilience4j retry, then alert admin to retry manually - Double-charge prevention: `PaymentOrder` has unique `(user_id, plan, billingCycle, status=PENDING)` — only one pending checkout per user per plan; if a user starts a new checkout, we cancel the old pending order

KEY TERMS TO MENTION: - Stripe Checkout (hosted) + Stripe webhooks (signed) - `Webhook.constructEvent(rawBody, sig, secret)` — raw bytes, never JSON-parse first - Idempotency via `event_id` unique constraint in `payment_webhook` - Ingest → 200 immediately → Kafka → async processing (decouple from Stripe retry pressure) - Kafka consumer with `@Transactional` per-event + state machine + audit - Subscription state machine: TRIALING, ACTIVE, PAST_DUE, CANCELLED, REFUNDED - Credit granting on activation + renewal, rollover up to 2x - Dunning via Stripe Smart Retries (day 0, 2, 5, 7) - Refunds: `Refunds.create()` + `charge.refunded` webhook - GST invoicing via Stripe Tax + custom PDFBox invoice template + S3 signed URL - Daily reconciliation job pulls Stripe events → backfill missing webhooks - Double-charge prevention: unique on `(user_id, plan, billingCycle, status=PENDING)`

FOLLOW-UP QUESTIONS: 1. How do you handle Stripe webhook secret rotation? 2. What if a user disputes a charge (chargeback)? 3. How do you test the webhook handler end-to-end?

FOLLOW-UP ANSWERS: 1. Stripe webhook secret is in AWS Secrets Manager; rotated every 90 days. We support `both` the old and new secret for 24 hours during rotation (the `Webhook.constructEvent` can be tried with both — if either succeeds, we accept). After 24h, we drop the old secret. The rotation is a Terraform-managed step + a manual code change to add the new secret to the allowlist. We've done this twice in production with no incidents. 2. We get a `charge.dispute.created` webhook. We mark the order as DISPUTED, freeze the user's credits (can't spend during dispute), and the admin gets a notification. We respond to the dispute in Stripe's dashboard within 7 days (Stripe's SLA). If we win, we unfreeze credits. If we lose, we mark the order as REFUNDED and the user's plan drops to Free. We've had 2 disputes in 9 months, won both (we had clear evidence the user used the platform). 3. Three layers: (a) unit tests with WireMock stubbing Stripe's webhook payload + signature — tests our verification logic; (b) integration tests with the Stripe CLI's `stripe trigger event checkout.session.completed` — fires a real webhook to our staging endpoint with a real signature (we use the Stripe CLI's webhook secret for staging); (c) end-to-end in staging with a real Stripe test card (4242 4242 4242 4242) — full checkout → real

webhook → DB state. We run (a) on every PR, (b) on merge to main, (c) before every prod deploy.

RED FLAGS TO AVOID: - Saying “we use Stripe” without explaining the webhook flow - Forgetting signature verification (this is the #1 Stripe bug) - Skipping idempotency (Stripe retries, you double-charge) - Saying “we save credit cards” (you don’t — Stripe does; you never see the card)

(The remaining 15+ system design questions — scaling to 1M users, audit log architecture, notification fan-out, idempotent webhook handling, multi-region deployment, reply intelligence, sequence engine, AI scoring engine, scaling to 10M leads/day, caching strategy, rate limiting, API layer design, background jobs, Google Calendar integration, credit/usage system — follow the same template. The interviewer should expand each into deeper, twisted variants and walk through end-to-end logic on a whiteboard.)

Category
3 & 4 —
Spring Boot
+ Spring
Security

> **SAMPLE
QUESTIONS
NOTICE:**

The questions below are sample reference questions. The interviewer should be ready for harder/ twisted variants — Spring Boot internals, bean lifecycle edge cases, security filter chain ordering, OAuth2 token leakage scenarios, JWT secret rotation, etc.

Q2-11: How did you handle background jobs (lead scraping, email sending, reminders)

DIFFICULTY: Hard **CATEGORY TAG:** System Design / Spring Batch

ANSWER: Background work falls into three classes in our system, and each has a different execution model. **Short async tasks** (send a notification, invalidate a cache) run in-process via `@Async` on a dedicated `ThreadPoolTaskExecutor` (core 8, max 16, queue 500, `CallerRunsPolicy` so we apply back-pressure instead of dropping work). **Medium workflows** (outreach email sequences, reply classification, reminder dispatch) go through a DB-backed job table (`job` with status, attempts, next_run_at, payload JSONB) polled by a `@Scheduled`

worker every 10 seconds with `SELECT ... SKIP LOCKED` so multiple app instances never pick the same job — that’s our poor-man’s queue before Kafka. **Heavy pipelines** (lead discovery: scrape → enrich → score → persist for 5,000 businesses) run as **Spring Batch** jobs with chunked steps (chunk size 100), restartability via the `BATCH_JOB_EXECUTION` tables, and skip/retry policies (skippable on parse exceptions, retry 3× with exponential backoff on 429s from scraping targets).

The reason for three models is failure semantics. `@Async` work is fire-and-forget — if the JVM dies, the work is lost, so we only use it for tolerable-loss tasks. The job table gives us at-least-once with visibility into stuck jobs (a monitor alerts when any job stays RUNNING > 15 min) and manual replay (a job can be re-queued with one SQL update). Spring Batch gives us commit-interval checkpointing: a 5,000-lead run that dies at row 3,400 resumes from the last chunk, not from zero. All three write structured events to our audit trail so we can answer “what happened to lead X at 14:32” months later.

Scheduling itself is guarded: every `@Scheduled` method in production runs inside a Redis distributed lock (`SET job:reminders NX PX 300000`) so that when we run 3 app instances behind the ALB, only one fires the 8 AM reminder sweep. We learned this the hard way — early on, users got triple reminders until we added the lock.

KEY TERMS TO MENTION: - Spring Batch (chunk-oriented steps, restartability, skip/retry policy) - DB-backed job table + `FOR UPDATE SKIP LOCKED` polling - `@Scheduled` + Redis distributed lock for multi-instance safety - `ThreadPoolTaskExecutor` tuning + `CallerRunsPolicy` back-pressure - At-least-once delivery, idempotent handlers, dead-job alerting

FOLLOW-UP QUESTIONS: 1. Why not Kafka or RabbitMQ from day one? 2. How do you make a job handler idempotent? 3. What happens when a scraping target blocks your worker mid-batch?

FOLLOW-UP ANSWERS: 1. Kafka needs a platform team — ZooKeeper/KRaft ops, partition planning, consumer lag monitoring. At our volume (~50k jobs/day peak) a polled table with indexes gives us the same at-least-once semantics with zero new infrastructure, and SQL gives free visibility (“show me all failed jobs last hour”). The design keeps a `QueuePort` interface, so swapping the poller for a Kafka listener is a config change, not a rewrite. 2. Every handler takes a natural idempotency key from the payload (e.g. `outreach_message_id`); before side effects it checks the outcome table — if a row already exists for that key with status SENT, it returns success without re-sending. Side effects themselves are guarded by unique constraints (message-id column UNIQUE) so a double-execution fails cleanly on insert instead of double-sending. 3. The step’s retry policy catches the HTTP 429/403, backs off exponentially; after 3 failures the chunk is skipped and recorded, the proxy-pool rotation kicks in for subsequent chunks, and the job finishes PARTIAL with a completion report showing which domains failed. A follow-up job re-runs only the failed domains the next night.

RED FLAGS TO AVOID: - Saying “we use cron jobs” for everything with no failure semantics - Not knowing your own thread-pool numbers or queue capacity - Claiming Kafka “for scale” when you can’t justify why a table wasn’t enough - Forgetting idempotency — at-least-once without idempotency = duplicate emails to customers

Q2-12: How does the Google Calendar integration work end-to-end

DIFFICULTY: Hard **CATEGORY TAG:** System Design / Third-Party Integration

ANSWER: The meeting scheduler's job is: given a lead says "yes, let's talk", find a slot where our user is free, book it, and keep both calendars in sync. The flow: user connects their Google account once (OAuth offline access → we store the **refresh token encrypted** (AES-256-GCM with a KMS-held key) in `calendar_connection`). When a lead accepts a proposed slot, `MeetingService.proposeSlots(user, duration)` calls the Calendar API `freebusy` query for the next 14 days against the user's primary calendar, subtracts busy intervals from working hours (configured per user with timezone from their profile), and generates the top 3 gaps respecting a 45-min buffer between meetings. Each candidate slot is written to `meeting_slot` with status PROPOSED.

The booking step is where concurrency matters: the lead gets a booking page; when they click a slot, we do an atomic claim — `UPDATE meeting_slot SET status='BOOKED' WHERE id=:id AND status='PROPOSED'` and check rows-affected (optimistic claim; only one winner) — then call `calendar.events.insert` with the Google event id stored back on the slot row. If the Google call fails after the DB commit, a compensation job finds BOOKED slots with null `google_event_id` and retries insertion; if Google says the slot now conflicts, we mark the slot CONFLICT and email both parties an apology + new proposals. Both parties get ICS attachments and Google-native invites, so declines propagate back.

Sync is push-based: we register a **Google Calendar watch channel** (PubSub push notification to our `/api/calendar/webhook`). On any change, we re-fetch the changed window via `events.list` with `syncToken`, update our cached busy map in Redis (TTL 30 min, also a read-through cache for `freebusy`), and if an external conflict cancels a booked meeting, we trigger the rescheduling flow automatically. Token lifecycle is handled by a `@Scheduled` sweeper that refreshes tokens expiring within 24h, so users never hit an auth failure mid-booking.

KEY TERMS TO MENTION: - OAuth offline access, encrypted refresh tokens - `freebusy` API + working-hours subtraction + timezone handling - Optimistic slot claim (conditional UPDATE, rows-affected check) - Google watch channels (PubSub push) + `syncToken` incremental sync - Compensation flow for distributed-write failure (DB committed, Google failed)

FOLLOW-UP QUESTIONS: 1. Two leads click the same slot simultaneously — what actually happens? 2. Why store refresh tokens encrypted — they're already OAuth tokens? 3. What if the user's calendar has an event with "free" visibility?

FOLLOW-UP ANSWERS: 1. Both requests run the conditional UPDATE; PostgreSQL row-locks the slot row, the second UPDATE matches zero rows (status is no longer PROPOSED), and that request gets a 409 + the next-best slot. The Google insert happens only for the winner. This is the same pattern as our credit deduction — DB-level atomicity, not application-level locks. 2. A refresh token is a permanent credential to the user's calendar — a DB leak would expose every connected user's calendar read/write. We encrypt at rest with

envelope encryption, and the encryption key lives outside the DB (env/KMS), so a DB dump alone is useless. Audit logging records every token decryption with the purpose. 3. `freebusy` returns only time blocks, not event details, and respects the event's transparency setting — events marked “free” (transparent) don't block. We also let users overlay external busy maps with per-calendar inclusion toggles. Edge case: recurring events expand — `freebusy` handles expansion server-side, but we cap the query window at 14 days to keep the response bounded.

RED FLAGS TO AVOID: - Storing access tokens and assuming they never expire - Claiming the slot in memory or in the DB only without the Google call compensation story - Ignoring timezones — the classic “meeting at 3 AM IST” bug - Polling the Calendar API every minute instead of push watch

Q2-13: How did you design the credit/usage system (metering)

DIFFICULTY: Expert **CATEGORY TAG:** System Design / Billing

ANSWER: Credits are the internal currency: every AI action (email generation, lead scoring, website analysis) costs a configured amount, defined in a `credit_cost` table so product can change prices without deploys. The invariant the system must guarantee: **a user can never spend more credits than their balance, even under concurrent requests** — overspending means we rendered an AI response for free and under-billing compounds. The design: balances live on `user_account.credits` (denormalized for read speed) with an append-only `credit_ledger` table as the source of truth (every debit/credit is a row with reason code, reference id, balance_after). Reads show the denormalized balance; every mutation writes the ledger first-class.

Deduction is atomic at the SQL level: `UPDATE user_account SET credits = credits - :cost WHERE id = :id AND credits >= :cost` — the `credits >= :cost` predicate makes the row itself the guard. Rows-affected = 1 means the spend succeeded; 0 means insufficient balance → we throw a domain exception mapped to 402 Payment Required. Under concurrency PostgreSQL serializes the row update, so two parallel deductions each see a consistent state — no read-modify-write race. For multi-step AI flows (deduct, then call LLM, then persist message) we deduct in the caller's transaction with the ledger row carrying an idempotency key (`ref_type + ref_id` UNIQUE); if the LLM call fails, a compensating ledger entry refunds within the same use-case transaction, so the balance never leaks.

Monthly reset and grants run as `@Scheduled` jobs using the same ledger API — reset is just a ledger entry “MONTHLY_GRANT” + balance update per active user, chunked through a job table. Admin adjustments also go through the ledger with a reason code, so support disputes are auditable end-to-end: “why did user X lose 50 credits on the 3rd?” is one SQL query. Stripe purchase top-ups arrive via webhook → ledger credit with the Stripe session id as idempotency key (replayed webhooks are no-ops). This ledger pattern is the same idea as double-entry bookkeeping — the balance is derivable by summing the ledger, and we run a nightly reconciliation job that asserts `sum(ledger) == balance` per user and alerts on drift.

KEY TERMS TO MENTION: - Conditional atomic UPDATE as the balance guard (no SELECT-then-UPDATE race) - Append-only credit ledger (source of truth) + denormalized balance for reads - Idempotency key on ledger entries (ref_type + ref_id UNIQUE) - Compensating refund entry on downstream failure - Nightly reconciliation job (ledger sum vs denormalized balance)

FOLLOW-UP QUESTIONS: 1. Why pessimistic locking via conditional UPDATE instead of @Version optimistic locking? 2. What if the AI call succeeds but the process dies before persisting — user lost credits? 3. How would you migrate this to support team/shared org credit pools?

FOLLOW-UP ANSWERS: 1. Optimistic locking retries on conflict — fine for low contention, but hot users (bots, bulk uploads) retry loops add latency and we still need the “insufficient balance” distinction. The conditional UPDATE gives both the atomicity and the business check in one statement with zero retries. We do use @Version on entities where conflicts are genuine edit-collisions (deals), not balance decrements. 2. That’s the exact reason the ledger has an idempotency key: the request handler reserves the deduction first (PENDING entry), the AI call runs outside the transaction (long), and the message persistence confirms the entry (POSTED). A sweeper reaps PENDING entries older than 5 minutes: if the AI provider call can be verified (we log provider request ids), we refund; the user never permanently loses credits for our crash. 3. Move the balance columns to organization and keep user-level spend attribution in the ledger rows (user_id on every entry). The conditional UPDATE then guards org balance; per-user quotas become a second check (aggregate ledger query for the current period, cached in Redis with a short TTL). The ledger schema doesn’t change — that’s the payoff of designing it append-only from day one.

RED FLAGS TO AVOID: - SELECT balance → check in Java → UPDATE (classic race condition answer) - Mutable balance with no audit trail — support nightmares - Deducting inside the same transaction as a 10-second LLM call (long lock hold) - Not having an answer for refunds/idempotency — real billing systems live on this

Q2-14: Your notification system works for 3k orgs — redesign it for 100k orgs sending 10M notifications/day

DIFFICULTY: Expert **CATEGORY TAG:** System Design / Twisted Variant

ANSWER: The current design (in-process @Async fan-out + job-table queue + direct channel calls) breaks at three points: the fan-out competes with request threads on the same JVM, the polled table becomes the hottest table in the DB, and a channel outage (Twilio slowdown) backs up the whole pipeline. The 100k-org redesign separates concerns into stages. **Ingestion:** API writes a notification row (status QUEUED) and publishes to Kafka topic notifications.outbound (partition key = org_id for per-org ordering); the API returns 202 immediately — no channel call on the request path. **Dispatch:** a consumer group (3-6 pods, one partition per hot org) applies per-org rate limits (Redis token bucket: e.g. 50/min/org for WhatsApp), renders templates, and writes to notifications.dispatching with the channel provider call wrapped in Resilience4j (circuit breaker per channel, bulkhead isolating

WhatsApp from email so one slow provider can't starve the rest). **Receipts:** provider webhooks (Twilio, SES) update delivery status through a dedicated consumer — status transitions are an append-only state machine (QUEUED → DISPATCHED → DELIVERED/BOUNDED/FAILED), which makes analytics a simple stream aggregation.

Persistence changes too: the `notification` table at 3B rows/year moves to monthly partitions with a 13-month retention (older partitions archived to S3 Parquet for analytics); the job-table queue is retired (Kafka is the queue); deduplication uses a Redis Bloom filter front-door (user+template+dedup-window) to cut accidental duplicate storms before they consume queue throughput. Delivery-time SLO (p95 < 30s from accept to provider-handoff) is enforced via consumer lag alerts (Kafka consumer lag > 5 min pages on-call) and per-channel success-rate dashboards.

The invariant I'd protect through the migration: at-least-once + idempotent handlers, exactly as today — providers themselves dedupe on our client message ids, so a redelivered Kafka record never double-sends. And the migration itself is the strangler pattern: route 5% of orgs through the new pipeline behind a feature flag, compare delivery latency + loss metrics, ramp to 100%, then delete the old poller.

KEY TERMS TO MENTION: - Kafka partitioning by `org_id` (ordering + parallelism) - Bulkhead + circuit breaker per channel (isolation of provider failures) - Partitioned notification table + S3 archive (lifecycle) - Consumer lag as the scaling signal; token-bucket per-org rate limits - Strangler-fig migration behind feature flag

FOLLOW-UP QUESTIONS: 1. Why Kafka partitions per org — what breaks if a partition is hot? 2. How do you guarantee a notification is never lost during a consumer crash? 3. What's your dedup window and how do you choose it?

FOLLOW-UP ANSWERS: 1. One org doing 500k sends/day would serialize behind one partition. Mitigation: composite key `org_id + campaign_id` for fan-out-heavy orgs (ordering only matters within a campaign), and split-to-more-partitions runbook since we control producer keys. Hot-partition alerting is on write throughput per partition. 2. Consumer commits the offset only after the provider handoff is durably recorded; a crash before commit = redelivery = handler idempotency (client message id UNIQUE at provider). Lost-notification risk is actually on the ingest side — so ingest writes to Kafka AND the DB row in the same request (DB is the reconciliation source; a hourly job re-enqueues QUEUED rows older than X that never got a DISPATCHED event). 3. Dedup window is per-template semantics: OTPs are 60s (a resend must go through), marketing is 24h, transactional receipts are never deduped (losing one is worse than doubling). The window is a column on the template, not a global constant — because the cost of a duplicate and the cost of a loss differ per notification type.

RED FLAGS TO AVOID: - Jumping to "just add Kafka" without saying what breaks today - Per-channel isolation forgotten — one Twilio outage freezing all email - No mention of partitioning/archival — 10M/day = 3.6B rows/year - Claiming exactly-once without idempotency plumbing to back it

Q2-15: TWISTED — Users received the same outreach email twice this morning. Walk me through your live debugging from API call to DB row.

DIFFICULTY: Expert **CATEGORY TAG:** System Design / Debugging Scenario

ANSWER: I'd structure this as containment → hypothesis tree → evidence. **Containment first:** pause the outreach dispatcher (flip a feature flag / `UPDATE job SET status='PAUSED' WHERE type='outreach'`) — a duplicate-send bug burns user trust with every minute, and pausing is one action, reversible. Then scope: is it one user or a pattern? Query `lead_communication` for messages where the same `(lead_id, template_id)` appears twice within a short window — GROUP BY to see if duplicates cluster on one worker, one time window, or one channel.

The hypothesis tree for duplicate sends, ordered by prior probability: (1) **at-least-once redelivery without idempotency** — a job retried after a timeout whose success actually landed (we send, the SMTP call times out at 30s, the job framework marks FAILED and retries, but the first send actually went through — the classic distributed-systems false-negative); (2) **a deploy restart racing an in-flight batch** — `@Async` /queue messages in memory lost their state machine and got re-enqueued; (3) **a unique-constraint regression** — someone dropped the `(lead_id, message_key)` uniqueness in a migration, so double-insert now succeeds silently. Evidence for each: check the job execution rows for the affected messages (two executions for one job id → redelivery), check deploy timeline vs duplicate timestamps, and `information_schema` for the constraint still existing.

The actual incident we had was (1): the Gmail send had a 30s timeout but Gmail's p99 is 28s — retries under load turned latency into duplicates. Fix was three layers: client-generated idempotency key (`message_key` = UUID generated at enqueue, passed as the SMTP custom header and stored UNIQUE in DB), timeout raised to 60s with the circuit breaker absorbing provider slowness, and a reconciliation query that runs every 5 min matching provider message-ids back to DB rows so a "FAILED" job whose send actually landed gets marked SENT, not retried. Postmortem action: added a synthetic test that injects provider latency > timeout to prove the pipeline stays duplicate-free.

KEY TERMS TO MENTION: - Containment before diagnosis (pause dispatcher, then investigate) - At-least-once delivery + false-negative timeout = the classic duplicate - Idempotency key end-to-end (enqueue → DB UNIQUE → provider header) - Evidence from job-execution rows, deploy timeline, constraint inspection - Reconciliation job as the safety net

FOLLOW-UP QUESTIONS: 1. Why not just make SMTP sends idempotent at Gmail's side? 2. How do you write the postmortem — what's the format? 3. Could this have been caught in staging?

FOLLOW-UP ANSWERS: 1. You don't control the provider — Gmail has no idempotency API for send. The idempotency must live on OUR side: we decide the key, we enforce the UNIQUE constraint, and we treat the provider as a black box that may or may not have executed any given attempt. That's the general rule for every external call in the system. 2. Timeline (detection → containment → root cause), impact (users affected, duplicate count),

root cause chain (why the timeout allowed false negatives, why no idempotency existed), what worked (flag-based pause took 2 min), and action items each with an owner and a verification test — a postmortem without a regression test is a wish list. 3. The load tests hit throughput but not latency-injection — that's the gap. Since the incident, staging runs k6 with fault injection (toxiproxy adding 35s latency to SMTP) so the false-negative path is exercised on every release candidate.

RED FLAGS TO AVOID: - Debugging before containing — more duplicate emails while you read logs - Guessing a root cause without evidence from the job/deploy data - Fixing only the symptom (retrying less) instead of adding idempotency - No postmortem/regression-test story — interviewers want systemic fixes

CATEGORY 3 — SPRING BOOT QUESTIONS

Q3-1: Why Spring Boot 3.x and Java 17

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot

ANSWER: We picked Java 17 LTS because it's the long-term support version with the strongest tooling (IntelliJ, Maven, SonarQube all have first-class support), mature GC options (we use G1 with -Xmx2g, sub-50ms pauses at our heap size), and language features we actually use: **records** for immutable DTOs (no Lombok `@Value` needed), **sealed classes** for our domain event hierarchy (`sealed interface DomainEvent permits LeadCreated, LeadUpdated, ...`), **pattern matching for switch** for clean dispatch in services, **text blocks** for SQL strings in `@Query`. Java 21's virtual threads would be nice for the SSE workloads but we chose 17 for stability — 21 went GA but adoption is still ramping.

Spring Boot 3.2.x because: (a) it requires Java 17 (no legacy baggage from Java 8 era), (b) ships Spring 6 + Security 6 + Data JPA 3.2 (modern APIs), (c) has native support for GraalVM if we want to ship a native image for fast cold-starts in serverless, (d) has first-class observability via Micrometer (auto-instrumentation of HTTP/JDBC/Hikari/Redis), (e) the team has strong Spring experience (notably, all of us used Spring Boot 2.x previously — 3.x is a familiar upgrade). We avoided 3.3.x because it was released close to our deploy and we wanted a minor version that had a few months of production hardening.

KEY TERMS TO MENTION: - Java 17 LTS: records, sealed classes, pattern matching for switch, text blocks, G1 GC, mature tooling - Spring Boot 3.2.x requires Java 17 baseline, Spring 6 + Security 6 + Data JPA 3.2 - Native support for GraalVM (future option) - Micrometer observability built-in - Why not 3.3.x: wanted production hardening time - Why not WebFlux: workload is CRUD-heavy, not high-concurrency reactive

FOLLOW-UP QUESTIONS: 1. Have you tried virtual threads? 2. Why G1 over ZGC? 3. What's your JDK distribution?

FOLLOW-UP ANSWERS: 1. Yes — we tested `Thread.ofVirtual()` on the LLM gateway path (long IO waits). Got 30% throughput improvement with no code change (Spring Boot 3.2 supports virtual threads via `spring.threads.virtual.enabled=true`). We rolled it back because we hit a Hibernate session leak — virtual threads don't release the JDBC connection synchronously, and HikariCP shows "connection has been left in transaction" warnings. We're waiting for Hibernate 6.5 to fix this; we'll re-enable in Q3. 2. G1 is the default in Java 17, mature, sub-50ms pauses at our 2GB heap. ZGC gives sub-1ms pauses but adds CPU overhead and benefits workloads with >32GB heaps — we're nowhere near that. We benchmarked ZGC vs G1 on our workload — no measurable difference at 2GB; ZGC was 5% slower in throughput. Not worth the migration. 3. Eclipse Temurin (Adoptium) — open-source, free, well-supported. We use the official Docker image `eclipse-temurin:17-jre-jammy` for production. We avoid Oracle JDK because of licensing.

RED FLAGS TO AVOID: - Saying “Java 17 because it’s the latest” (21 is the latest LTS) - Saying “Spring Boot because everyone uses it” (no depth) - Not mentioning specific Java 17 features

Q3-2: Explain your Spring Boot project structure

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Multi-module Maven project with bounded-context packages. Three Maven modules: - `common` — shared kernel: base entities (`BaseEntity` with `@MappedSuperclass` for `id`, `createdAt`, `updatedAt`), custom exceptions (`NotFoundException`, `BusinessException`, `ValidationException`), DTOs (`PageResponse<T>`, `ErrorResponse`), Spring AOP aspects (`AuditAspect`, `RateLimitAspect`), utility classes. - `domain` — bounded contexts: `com.acquisitionos.auth`, `com.acquisitionos.leads`, `com.acquisitionos.outreach`, `com.acquisitionos.meetings`, `com.acquisitionos.billing`, `com.acquisitionos.ai`, `com.acquisitionos.notifications`, `com.acquisitionos.admin`, `com.acquisitionos.integrations`. Each context has sub-packages `controller`, `service`, `repository`, `domain` (entities), `event` (domain events), `dto`, `mapper` (MapStruct), `exception` (context-specific exceptions). - `app` — Spring Boot main class `AcquisitionOsApplication` with `@SpringBootApplication(scanBasePackages = "com.acquisitionos")`. Holds the `application.yml`, `application-dev.yml`, `application-prod.yml` profiles, the `SecurityConfig`, `RedisConfig`, `HibernateConfig`, `AsyncConfig`, `SchedulingConfig`, `OpenApiConfig` beans. This is the deployable module (jar).

We enforce context boundaries with **ArchUnit** tests — a `ContextBoundaryTest` that asserts no `com.acquisitionos.outreach..` imports from `com.acquisitionos.billing..`. Exceptions are documented in the test (e.g. `outreach` can import `auth`’s `UserService` because outreach needs to know who the user is, but `auth` can’t import `outreach`).

Cross-context communication is via **Spring ApplicationEventPublisher** for in-process events and **Kafka** for cross-service-ready events. Each event is a sealed `DomainEvent` with a `eventId` (UUID), `occurredAt` (Instant), `aggregateId` (UUID), and a typed payload.

Build: Maven 3.9, `mvn clean install` builds all modules; `mvn test` runs unit tests; `mvn verify` runs integration tests (Testcontainers); `mvn package` produces the layered jar (Spring Boot’s layered jar feature separates `dependencies` from `spring-boot-loader` from `application` layers for Docker cache reuse).


```

acquisitionos/
├── pom.xml                (parent – module declarations, plugin management)
├── common/
│   ├── pom.xml
│   └── src/main/java/com/acquisitionos/common/
│       ├── entity/BaseEntity.java
│       ├── exception/{NotFoundException, BusinessException,
│       │               ValidationException}.java
│       ├── dto/{PageResponse, ErrorResponse}.java
│       ├── aspect/{AuditAspect, RateLimitAspect, TraceAspect}.java
│       └── util/...
├── domain/
│   ├── pom.xml            (depends on common)
│   └── src/main/java/com/acquisitionos/
│       ├── auth/{controller, service, repository, domain, event, dto, mapper,
│       │       exception}
│       ├── leads/...
│       ├── outreach/...
│       ├── meetings/...
│       ├── billing/...
│       ├── ai/...
│       ├── notifications/...
│       ├── admin/...
│       └── integrations/...
└── app/
    ├── pom.xml            (depends on domain + common)
    └── src/main/
        ├── java/com/acquisitionos/AcquisitionOsApplication.java
        ├── resources/
        │   ├── application.yml
        │   ├── application-dev.yml
        │   ├── application-prod.yml
        │   ├── db/migration/V*.sql    (Flyway)
        │   └── prompts/outreach_v3.txt (prompt templates)
        └── test/...                 (integration tests)

```

KEY TERMS TO MENTION: - Multi-module Maven: `common`, `domain`, `app` - Bounded contexts as Java packages `com.acquisitionos.<context>` - ArchUnit tests enforce no cross-context imports (except documented) - Cross-context via `ApplicationEventPublisher` + Kafka, sealed `DomainEvent` - Spring Boot layered jar (Docker cache-friendly) - Maven 3.9, `mvn verify` runs integration tests with Testcontainers

FOLLOW-UP QUESTIONS: 1. Why multi-module over single-module? 2. Why ArchUnit and not just code review? 3. How do you handle a context that needs to span multiple modules?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) enforces boundaries at build time — a developer can't accidentally import `billing` from `auth` because the IDE warns; (b) faster incremental builds — `mvn -pl domain install` rebuilds only `domain`; (c) future microservice extraction — when we lift `ai` into its own service, it's a new Maven module `ai-service` that depends on `common` + a thin `ai-api` (DTO) module, with no rewrite of the existing code. 2. ArchUnit catches violations in CI before code review. Reviewers miss boundary violations (especially in refactors). ArchUnit is deterministic and fast (~5 seconds in our test suite).

Code review still catches **design** violations (e.g. “this service is doing too much”) that ArchUnit can’t. 3. We’ve never needed it. Bounded contexts are by definition bounded. If a feature needs to span contexts, the contexts are wrong — we re-evaluate the boundary. The one exception is “shared kernel” — code shared across contexts (`BaseEntity` , `PageResponse` , error codes) lives in `common` , and changes there require multi-context review.

RED FLAGS TO AVOID: - Single module with `com.acquisitionos.controller` , `com.acquisitionos.service` (flat by layer, not by feature — looks junior) - No enforcement of boundaries (looks like no architecture discipline) - Circular dependencies between contexts (looks like no design)

Q3-3: How did you implement dependency injection

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Standard Spring DI — **constructor injection only** (no `@Autowired` field injection). We use **Lombok** `@RequiredArgsConstructor` on services to generate the constructor; for services with required + optional dependencies we write the constructor explicitly with `@Autowired(required = false)` on the optional one.

Why constructor injection: (a) immutable — fields can be `final` , (b) testable — no Spring context needed to inject mocks, just `new LeadService(mockRepo, mockCreditService, ...)` , (c) explicit dependencies — you can see what a service needs by looking at its constructor, (d) fails fast — Spring fails to start if a required bean is missing (vs `@Autowired` field injection which fails at runtime on first use), (e) Spring’s recommendation since 4.x.

We use `@Qualifier` when we have multiple beans of the same type — e.g. we have two `PaymentProvider` beans (`StripePaymentProvider` , `RazorpayPaymentProvider`) and the controller injects `@Qualifier("stripePaymentProvider") PaymentProvider provider` . We prefer qualifiers over `@Primary` because `@Primary` is implicit (a future developer might not know which bean is the default).

For **conditional beans**, we use `@ConditionalOnProperty` — e.g. `@ConditionalOnProperty(name = "feature.ai.enabled", havingValue = "true")` on the `AIService` bean. When the property is false in staging, the bean doesn’t exist and we fall back to a `NoOpAIService` (a `@Component` with `@ConditionalOnMissingBean(AIService.class)`).

For **prototype beans in singletons**, we use `ObjectProvider<T>` — e.g. `OutreachMessageProcessor` is a prototype (stateful, one per message), injected into the singleton `OutreachService` via `ObjectProvider<OutreachMessageProcessor>` . The service calls `objectProvider.getObject()` to get a fresh instance per message.

We avoid `@Autowired` method injection (used for circular dependencies — we refactor to break the cycle instead) and `@Autowired` field injection (untestable, mutable).

For **profiles**, beans are annotated `@Profile("prod")` or `@Profile("!test")` to activate per environment. The active profile is set in `application.yml` (`spring.profiles.active: ${SPRING_PROFILES_ACTIVE:dev}`) and overridden by env var on ECS.

For **bean lifecycle**, we use `@PostConstruct` for setup (e.g. `RedisConfig` loads the connection pool on `@PostConstruct`) and `@PreDestroy` for cleanup (e.g. `SseManager` closes all emitters on shutdown). We avoid `InitializingBean` / `DisposableBean` (Spring-specific interfaces) because `@PostConstruct` / `@PreDestroy` are JSR-250 (standard).

KEY TERMS TO MENTION: - Constructor injection only, `@RequiredArgsConstructor` (Lombok) - `@Qualifier` over `@Primary` (explicit > implicit) - `@ConditionalOnProperty` + `@ConditionalOnMissingBean` for conditional beans - `ObjectProvider<T>` for prototype beans in singletons - `@Profile` for environment-specific beans, `SPRING_PROFILES_ACTIVE` env var - `@PostConstruct` / `@PreDestroy` (JSR-250, not Spring-specific) - No field injection (untestable), no method injection (circular dep smell)

FOLLOW-UP QUESTIONS: 1. How do you handle a circular dependency? 2. How do you test a service that depends on 5 beans? 3. What's the difference between `@Component`, `@Service`, `@Repository`, `@Controller`?

FOLLOW-UP ANSWERS: 1. We don't allow circular dependencies — we refactor. The most common cycle is "A depends on B, B depends on A" — we extract the common dependency into a third class C and make A and B both depend on C. If extraction is genuinely impossible (rare), we use `@Lazy` on one side — Spring injects a proxy that resolves the bean lazily. But `@Lazy` is a smell — we add a TODO comment to refactor. 2. Constructor injection + Mockito —

```
new LeadService(mockLeadRepo, mockCreditService, mockAuditPublisher, mockEventPublisher, mockMapper)
```

The test creates 5 `@Mock` beans via `@ExtendWith(MockitoExtension.class)`. We use `@InjectMocks` for convenience when the mocks are simple, but explicit construction when we need to wire a specific mock behaviour. We never use `@SpringBootTest` for unit tests (slow, doesn't isolate). 3. They're all stereotype annotations — semantically equivalent (all make a class a Spring bean). `@Controller` (or `@RestController`) marks web layer; `@Service` marks business layer; `@Repository` marks data layer (also enables exception translation via `PersistenceExceptionTranslationPostProcessor`). `@Component` is the generic. We use the specific one for documentation — a `@Repository` bean's `SQLException` gets auto-translated to Spring's `DataAccessException` hierarchy, useful for uniform error handling.

RED FLAGS TO AVOID: - Field injection (`@Autowired` on a field) — instant fail - `@Autowired` on every method — looks like you don't understand DI - Saying "I use `@Component` for everything" — looks lazy - Not knowing `@Qualifier` vs `@Primary`

Q3-4: What Spring Boot starters did you use and why

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Starters in our `pom.xml`:

- `spring-boot-starter-web` — Spring MVC + embedded Tomcat + Jackson + Bean Validation. Our primary stack.
- `spring-boot-starter-data-jpa` — Spring Data JPA + Hibernate. Persistence.

- `spring-boot-starter-validation` — Bean Validation (Hibernate Validator). Pulled in by `starter-web` but we add it explicitly to make it explicit.
- `spring-boot-starter-security` — Spring Security 6 + OAuth2 client + resource server.
- `spring-boot-starter-oauth2-client` — Google OAuth2 login flow.
- `spring-boot-starter-oauth2-resource-server` — JWT validation (our internal APIs use this).
- `spring-boot-starter-data-redis` — Lettuce + Spring Data Redis. Cache, rate-limit, distributed lock.
- `spring-boot-starter-batch` — Spring Batch for the lead discovery job.
- `spring-boot-starter-mail` — JavaMailSender for transactional emails.
- `spring-boot-starter-thymeleaf` — email templates (transactional + invoice templates; not used for web).
- `spring-boot-starter-actuator` — health, metrics, info, env endpoints.
- `spring-boot-starter-aop` — Spring AOP for our aspects.
- `spring-boot-starter-logging` — Logback (default, included transitively).
- `spring-boot-starter-test` (test scope) — JUnit 5, Mockito, AssertJ, Spring Boot Test, Testcontainers.
- `spring-boot-starter-webflux` — for `WebClient` (we use the reactive `WebClient` for external API calls even though our main stack is MVC). We don't run a WebFlux server, just use the client.
- `spring-kafka` (not a starter, but same role) — Kafka producer + consumer.
- `spring-cloud-aws-starter-secrets-manager` — pulls secrets from AWS Secrets Manager at startup.
- `springdoc-openapi-starter-webmvc-ui` — OpenAPI 3 + Swagger UI.
- `resilience4j-spring-boot3` — circuit breaker, retry, rate limiter, bulkhead.
- `mapstruct` + `lombok` (with `lombok-mapstruct-binding`) — DTO mapping + boilerplate removal.

We don't use `spring-cloud-starter-*` (Eureka, Config Server, Gateway) because we're a monolith on ECS — no service discovery needed, no config server (we use Secrets Manager), no API gateway (ALB is our gateway).

KEY TERMS TO MENTION: - `starter-web` (MVC + Tomcat + Jackson + Validation), `starter-data-jpa` + Hibernate - `starter-security` + `starter-oauth2-client` + `starter-oauth2-resource-server` (JWT) - `starter-data-redis` (Lettuce), `starter-batch`, `starter-mail`, `starter-thymeleaf` (email templates) - `starter-actuator`, `starter-aop`, `starter-test` - `starter-webflux` for `WebClient` only (not WebFlux server) - `spring-kafka`, `spring-cloud-aws-starter-secrets-manager` - `springdoc-openapi-starter-webmvc-ui`, `resilience4j-spring-boot3` - `mapstruct` + `lombok` with `lombok-mapstruct-binding` - No Spring Cloud (Eureka/Config/Gateway) — monolith on ECS

FOLLOW-UP QUESTIONS: 1. Why `spring-boot-starter-webflux` if you're not using WebFlux server? 2. How do you handle logging without `spring-boot-starter-logging`? 3. What's the cost of too many starters?

FOLLOW-UP ANSWERS: 1. We use `WebClient` (reactive) for external API calls (Stripe, Google, LLM) because: (a) non-blocking — long IO calls don't hold a thread; (b) integrates with Resilience4j's circuit breaker cleanly; (c) fluent API is cleaner than `RestTemplate`. `WebClient` lives in `spring-boot-starter-webflux` even if you don't run a WebFlux server — it's a client library that happens to be reactive. The MVC server stack and the WebFlux client can coexist. 2. `spring-boot-starter-logging` is included transitively by `starter-web` — we get Logback + SLF4J out of the box. We add a `logback-spring.xml` for our custom JSON encoder + MDC correlation ID pattern + Loki appender. We don't need to exclude `starter-logging`. 3. Each starter adds ~5-20 MB to the jar and a few hundred ms to startup. We have ~15 starters and our jar is 90 MB, startup is 8 seconds. Acceptable. The bigger cost is **operational** — more starters means more auto-configuration that can break on upgrade. We pin Spring Boot version and upgrade deliberately (every 6 months).

RED FLAGS TO AVOID: - Listing 50 starters (looks like you don't know what you're using) - Not mentioning OAuth2 client + resource server (auth is a major module) - Saying "starter-data-jpa" without Hibernate (looks shallow)

Q3-5: How did you handle application configuration (application.yml)

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot

ANSWER: Multi-profile `application.yml` with env var overrides and Secrets Manager for sensitive values.

application.yml (base): common config — server port, Spring profile activation via `spring.profiles.active: ${SPRING_PROFILES_ACTIVE:dev}`, HikariCP base config, Hibernate `ddl-auto: validate` (Flyway owns schema), Jackson config (Java time module, ISO-8601 dates, fail-on-unknown-properties false), logging level `INFO` for `com.acquisitionos`, `WARN` for `org.springframework` in prod.

application-dev.yml: dev-friendly config — Hibernate `show_sql: true` + `format_sql: true`, logging level `DEBUG` for `com.acquisitionos`, Spring Batch `initialize-schema: embedded` (H2 for batch metadata in tests), server port 8080, LLM endpoint pointing to a local mock server, Stripe test keys, Google OAuth redirect to localhost, no rate limiting (so we can hammer the API while testing).

application-prod.yml: prod config — Hibernate `show_sql: false`, logging level `WARN` + JSON encoder (for Loki), Spring Batch `initialize-schema: never` (Flyway owns), server port 8080 (ALB terminates TLS), real Stripe/Google/LLM endpoints, rate limiting at production values, HikariCP `maximum-pool-size: 30`, `connection-timeout: 5000`, `leak-detection-threshold: 30000`.

Secrets: Never in `application.yml`. We use `@ConfigurationProperties` beans that read from env vars: `STRIPE_SECRET_KEY`, `JWT_SECRET`, `GOOGLE_CLIENT_SECRET`, `STRIPE_WEBHOOK_SECRET`, `SMTP_PASSWORD`, `MAIL_ENCRYPTION_KEY`. On ECS, these env vars are populated by the task definition, which pulls from AWS Secrets Manager via `spring-cloud-aws-starter-secrets-manager`'s `AwsSecretsManagerPropertySource` — we add `aws.secretsmanager.prefix=/acquisitionos/prod/` and Spring loads `/acquisitionos/prod/stripe-secret-key` into `${stripe-secret-key}`.

@ConfigurationProperties pattern: We have a `StripeProperties` class (`@ConfigurationProperties(prefix = "stripe")`) with `secretKey`, `webhookSecret`, `priceIdProMonthly`, `priceIdEliteMonthly`, `priceIdCredits100`, etc. Validated with `@Valid` + `@NotBlank` + `@NotNull` so the app fails to start if a required value is missing.

Per-environment overrides: We have `application-staging.yml` (separate Stripe test-mode keys, separate Google OAuth test client) and `application-load-test.yml` (more aggressive HikariCP, no rate limiting, debug logging for the LLM gateway).

Hot reload (dev only): We use `spring-boot-devtools` in dev — file changes trigger a restart. Not in prod (security risk).

Config validation on startup: Spring Boot fails fast on missing config — `@ConfigurationProperties` with `@Validated` + `@NotBlank` throws `BeanCreationException` if a value is missing. We have a `ConfigValidationTest` (`@SpringBootTest`) that boots the app with prod profile and asserts all required config is present. Run in CI before deploy.

KEY TERMS TO MENTION: - `application.yml` + `application-dev.yml` + `application-prod.yml` profile split - `spring.profiles.active: ${SPRING_PROFILES_ACTIVE:dev}` env-var override - `@ConfigurationProperties(prefix="stripe")` + `@Validated` + `@NotBlank` - Secrets via env vars → AWS Secrets Manager via `spring-cloud-aws-starter-secrets-manager` - `AwsSecretsManagerPropertySource` loads `/acquisitionos/prod/<key>` into `${<key>}` - Hibernate `ddl-auto: validate` (Flyway owns schema) - `spring-boot-devtools` for hot reload in dev only - `ConfigValidationTest` (`@SpringBootTest`) boots with prod profile in CI

FOLLOW-UP QUESTIONS: 1. How do you rotate a secret without downtime? 2. How do you validate config without starting the whole app? 3. What's your env-var naming convention?

FOLLOW-UP ANSWERS: 1. Two ways: (a) Secrets Manager version + Lambda rotation — the new version is written to a new Secrets Manager stage (`AWSCURRENT` → `AWSPENDING` → promote), Spring polls every 5 minutes and reloads (we use `@RefreshScope` on `StripeProperties` so the bean is re-created); (b) deploy a new task with the new secret — old tasks die, new tasks start with the new secret. (a) is zero-downtime; (b) is simpler but requires a deploy. We use (a) for Stripe webhook secret (rotated quarterly) and (b) for JWT secret (rotated annually). 2. `@SpringBootTest` with the prod profile boots the full context — if any `@ConfigurationProperties` validation fails, the test fails. We also have a `ConfigurationPropertiesTest` that uses `@ConfigurationPropertiesScan` + `@EnableConfigurationProperties` to load just the properties beans (no full app context) and assert via `@Validated`. Faster than full boot (~3s vs 8s). 3. `UPPER_SNAKE_CASE` with a context prefix: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `SMTP_PASSWORD`, `REDIS_URL`, `DATABASE_URL`, `KAFKA_BROKERS`,

`LLM_API_KEY`. Each env var maps 1:1 to a `@ConfigurationProperties` field via Spring Boot's relaxed binding (`kebab-case` in yaml, `camelCase` in Java, `UPPER_SNAKE_CASE` in env vars).

RED FLAGS TO AVOID: - Hardcoded secrets in `application.yml` - `ddl-auto: update` in prod (use `validate` or `none`) - Not mentioning secrets management (interviewer asks) - Saying "we use .env files" (not production-grade)

Q3-6: How did you implement exception handling globally

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot

ANSWER: Single `@RestControllerAdvice` (`GlobalExceptionHandler`) catches all exceptions and translates to our error response format: `{"error": {"code": "...", "message": "...", "details": {...}}}`.

Handlers (each `@ExceptionHandler`): - `MethodArgumentNotValidException` (Bean Validation) → 400 with `errors: [{field, message}]` - `ConstraintViolationException` (path/query param validation) → 400 with `errors: [...]` - `HttpMessageNotReadableException` (malformed JSON) → 400 with `error: "Malformed JSON"` - `MissingServletRequestParameterException` → 400 - `HttpRequestMethodNotSupportedException` → 405 - `NoHandlerFoundException` → 404 - `AuthException` (our custom, thrown by `requireAuth`) → 401 or 403 - `AccessDeniedException` (Spring Security) → 403 - `NotFoundException` (our custom) → 404 - `BusinessException` (our custom, e.g. `InsufficientCreditsException`, `PlanEntitlementException`) → 422 with our error code - `OptimisticLockingFailureException` (Spring Data) → 409 "Resource was modified, please refresh" - `DataIntegrityViolationException` (Spring Data) → 409 with the constraint name (we map common ones: `uk_user_email` → "Email already exists") - `RedisConnectionFailureException` → 503 with `Retry-After: 30` (Redis down is a transient outage) - `WebClientResponseException` (from external API) → 502 "Upstream service error" (we never leak the upstream's response to the client) - `Exception` (fallback) → 500 with a correlation id, full stack trace logged server-side only

Every response includes a `X-Correlation-Id` header (UUID set by `TraceAspect` at request entry) so the client can quote it to support. Every 500 logs the full stack trace at ERROR level + pushes a Sentry event.

The `ErrorCode` enum standardises the codes (`INSUFFICIENT_CREDITS`, `PLAN_ENTITLEMENT_REQUIRED`, `EMAIL_ALREADY_REGISTERED`, `STRIPE_WEBHOOK_INVALID_SIGNATURE`, etc.) so the client can switch on them for i18n.

We also handle `@Async` exceptions via `AsyncUncaughtExceptionHandler` (registered in `AsyncConfig`) — logs the exception + pushes to Sentry; doesn't propagate to the client (async has no response to send).

Spring Batch exceptions are handled per step with `RetryTemplate` + `SkipPolicy` — failed chunks retry 3 times, then skip with a logged warning.

Kafka consumer exceptions are caught and the consumer commits the offset (so we don't block the topic); the failed event is sent to a dead-letter topic (`dlq-outreach-events`) for manual review.

KEY TERMS TO MENTION: - `@RestControllerAdvice` + `@ExceptionHandler` per exception type - Custom exception hierarchy: `BusinessException` → `InsufficientCreditsException`, `PlanEntitlementException` - Error response format: `{"error": {code, message, details}}` + `X-Correlation-Id` header - `ErrorCode` enum (standardised codes for client i18n) - Spring Data exceptions auto-translated: `OptimisticLockingFailureException` → 409, `DataIntegrityViolationException` → 409 with constraint name - 500 logs stack trace + pushes Sentry, never leaks to client - `AsyncUncaughtExceptionHandler` for `@Async` exceptions - Spring Batch: `RetryTemplate` + `SkipPolicy` for chunk failures - Kafka DLQ for failed events (commit + send to `dlq-*` topic)

FOLLOW-UP QUESTIONS: 1. How do you handle a 500 — what does the client see? 2. What if your `GlobalExceptionHandler` itself throws? 3. How do you test exception handling?

FOLLOW-UP	ANSWERS:
1.	Client sees <code>{"error": {"code": "INTERNAL_ERROR", "message": "An unexpected error occurred", "correlationId": "ab"}}</code> Server logs full stack + pushes to Sentry with the correlation id as a tag. Support can look up the correlation id in Loki + Sentry. We never leak the stack trace, file paths, or SQL to the client (security). 2. Spring's <code>DefaultErrorAttributes</code> kicks in (the default white-label error page, but as JSON because we have <code>@RestControllerAdvice</code>). We'd see a generic <code>{"timestamp": ..., "status": 500, "error": "Internal Server Error"}</code> . To prevent this, we test <code>GlobalExceptionHandler</code> thoroughly — including a test that throws a <code>RuntimeException</code> from within an <code>@ExceptionHandler</code> (meta-test) to verify Spring's fallback. 3. <code>@WebMvcTest</code> with <code>@MockBean</code> services — controllers throw, the <code>GlobalExceptionHandler</code> catches, the test asserts the response status + body. We have 30+ exception-handler tests covering every <code>@ExceptionHandler</code> method. For end-to-end, we have a <code>@SpringBootTest</code> that fires a request that triggers a 500 and asserts the Sentry mock was called.

RED FLAGS TO AVOID: - Catching `Exception` in a controller (use the advice) - Leaking stack traces to the client - Returning 500 for business errors (use 422) - Forgetting correlation id (no way to debug client reports)

Q3-7: What is your approach to logging (Slf4j, Logback)

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot

ANSWER: SLF4J as the facade, Logback as the implementation (Spring Boot default). JSON-encoded logs in prod for Loki, human-readable in dev.

Log levels: `ERROR` (exception + stack trace, paged via Sentry), `WARN` (recoverable issues — e.g. rate limit hit, circuit breaker open), `INFO` (business events — login, signup, payment, lead discovered), `DEBUG` (detailed request/response, only enabled per-package in dev), `TRACE` (Hibernate SQL, only in dev). We set `logging.level.com.acquisitionos=INFO` in prod, `DEBUG` in dev, `TRACE` for `org.hibernate.SQL` only when investigating.

Structured logging: Every log line includes MDC fields: `correlationId` (set by `TraceAspect` from the `X-Correlation-Id` header or generated), `userId` (set after auth), `requestId` (per request), `endpoint` (controller path), `method` (HTTP method). The JSON encoder

(`net.logback.logback.json.encoder.JsonEncoder`) writes MDC fields as top-level JSON keys, so Loki can filter `correlationId="abc-123"` to see every log for one client request.

Correlation propagation: `TraceAspect` (Spring AOP) wraps every controller method, sets MDC `correlationId` at entry, clears at exit. `WebClient` calls propagate `correlationId` as `X-Correlation-Id` header to external APIs (Stripe, Google) — useful when Stripe support asks “what request caused this webhook”. OpenTelemetry trace id is also added to MDC for cross-referencing with Tempo traces.

Async logging: Logback `AsyncAppender` queues log events and writes on a separate thread (so logging doesn’t block request threads). Queue size 8192, never block (drop WARN if queue is 80% full, drop ERROR never — we’d rather slow down than lose errors). This gives us 100K logs/sec throughput without blocking.

Sensitive field masking: A custom Logback `Converter` masks fields named like `password`, `secret`, `apiKey`, `token`, `creditCard`, `cvv` in structured logs. We use a `MaskingPatternLayout` that regex-replaces `password=.*?(\s|$)` → `password=***`. We tested this with a synthetic PII log and verified no leak.

Per-package tuning: `logging.level.org.hibernate.SQL=WARN` (we don’t want every SQL in prod, just slow ones), `org.hibernate.type.descriptor.sql.BasicBinder=TRACE` only in dev (logs bind values). `com.zaxxer.hikari=WARN` (HikariCP is chatty at INFO). `io.lettuce.core=WARN` (Lettuce Redis client). `org.springframework=WARN` (Spring is too verbose at INFO).

Log shipping: ECS tasks log to stdout/stderr → AWS FireLens (Fluent Bit sidecar) → Amazon OpenSearch + Loki (we use Loki for the dev team, OpenSearch for compliance retention 1 year). We use `awslogs` log driver as a backup (CloudWatch Logs) for debugging ECS task crashes.

Cost: ~50GB/day of logs in prod. Loki is ~\$200/month (we use S3 backend), OpenSearch is ~\$800/month (full-text search across 1 year retention), CloudWatch is ~\$100/month (last 30 days for quick debugging). Total ~\$1.1K/month — 13% of infra cost.

KEY TERMS TO MENTION: - SLF4J facade + Logback impl (Spring Boot default) - JSON encoder in prod (Loki), human-readable in dev - MDC fields: `correlationId`, `userId`, `requestId`, `endpoint`, `method` (top-level JSON keys) - Logback `AsyncAppender` (queue 8192, never block on ERROR) - `TraceAspect` (AOP) sets MDC at controller entry, clears at exit - `WebClient` propagates `X-Correlation-Id` to external APIs - OpenTelemetry trace id in MDC for cross-ref with Tempo - `MaskingPatternLayout` regex-masks `password|secret|apiKey|token|creditCard` - Per-package tuning: Hibernate SQL WARN, Hikari WARN, Spring WARN - Log shipping: FireLens (Fluent Bit) → Loki + OpenSearch + CloudWatch (3-tier) - ~50GB/day, ~\$1.1K/month

FOLLOW-UP QUESTIONS: 1. How do you handle a log flood (e.g. a circuit breaker opens and 1000 errors/sec)? 2. How do you debug a production issue with logs? 3. What’s the cost of too much logging?

FOLLOW-UP ANSWERS: 1. Two protections: (a) Logback `AsyncAppender` queue caps at 8192 — overflow drops WARN/DEBUG but never ERROR; (b) Sentry rate-limits at 100 events/sec — spikes get sampled, the spike is logged once with a count. For persistent floods (a bug

spamming errors), we have a Prometheus alert on `log_error_rate{app=acquisitionos} > 10/sec` → PagerDuty → on-call investigates. 2. Three steps: (a) get the correlation id from the user (or the Sentry event), (b) `logcli query '{app="acquisitionos"} |~ "correlationId=\"abc-123\""'` in Loki to see every log for that request, (c) follow the OpenTelemetry trace id from the logs into Tempo to see the full distributed trace (controller → service → DB → external API). For a 500, the Sentry event has the stack + breadcrumbs (last 10 log lines) — usually enough to diagnose. 3. Three costs: (a) disk/storage (we pay ~\$1.1K/month for 50GB/day); (b) CPU (logging is ~2-5% of request time for JSON encoding + I/O — we use async appender to offload); (c) noise (too many INFO logs hide the signal — we periodically audit INFO logs and downgrade to DEBUG). We target: ERROR 0.01% of requests, WARN 0.1%, INFO 1% (business events only), DEBUG off in prod.

RED FLAGS TO AVOID: - `System.out.println` (not a senior) - No correlation id (no way to trace a request) - Logging passwords (security fail) - Synchronous logging (blocks request threads)

Q3-8: How did you implement request validation (@Valid, @Validated)

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Three layers of validation. **Layer 1 — DTO constraints:** request DTOs carry Bean Validation annotations (`@NotBlank`, `@Email`, `@Size`, `@Pattern` for phone) and controllers take `@Valid @RequestBody CreateLeadRequest` — `MethodArgumentNotValidException` is then handled in the `@RestControllerAdvice` and converted to a 400 with field-level errors (`{"errors":[{"field":"email","message":"must be valid"}]}`) so the frontend can render inline errors. **Layer 2 — cross-field rules:** “endDate after startDate” can’t be a field annotation, so we write class-level custom constraints (`@ValidDateRange` with a `ConstraintValidator<...>` implementation) — reusable, testable, and the violation surfaces at the same 400 boundary. **Layer 3 — domain invariants:** business rules that need the DB (email uniqueness within org, stage transition legality) live in the service layer and throw domain exceptions (`DuplicateLeadException` → 409), never validated at the controller because they’re transactions-dependent.

The subtle part interviewers probe: `@Valid` vs `@Validated`. `@Valid` (JSR-380) triggers cascade validation into nested objects and works on `@RequestBody`; `@Validated` (Spring) enables group-based validation and method-level validation on `@Service` classes (e.g. `@Validated public void archive(@NotNull UUID id)` fails-fast on internal calls, not just HTTP entry). We also validate configuration at startup — `@ConfigurationProperties` beans with `@Validated` fail the boot if an env var is malformed (bad SMTP port kills the app in dev, not at first email). One rule we enforce in review: **never trust a DTO past the controller** — services take domain objects assembled from validated DTOs, so an internal caller can’t bypass HTTP-layer validation accidentally.

KEY TERMS TO MENTION: - Bean Validation (JSR 380) annotations + `@Valid` on `@RequestBody` - `@RestControllerAdvice` → structured 400 with field-level errors - Custom `ConstraintValidator` for cross-field rules - `@Validated` groups + method-level validation on services - Startup config validation via `@ConfigurationProperties @Validated`

FOLLOW-UP QUESTIONS: 1. Why validate in the service too if the controller already validated? 2. How do you keep validation messages consistent for the frontend?

FOLLOW-UP ANSWERS: 1. Controllers only guard the HTTP door — services are called by schedulers, other services, and tests too. Domain invariants (stage transitions, uniqueness) are transactional facts, not shape facts, so they belong in the service. Shape at the edge, invariants at the core. 2. Error codes, not prose: each constraint carries a stable code (`LEAD.EMAIL.DUPLICATE`) and i18n-able parameters; the frontend maps codes to copy. Free-text exception messages leaking to clients is a review-blocking issue — they leak internals and break on refactor.

RED FLAGS TO AVOID: - Validating only with if-statements scattered in controllers - Leaking raw exception messages/validation internals to clients - DB-dependent checks in Bean Validation annotations (slow, transactional hazards)

Q3-9: How did you handle async operations (@Async, CompletableFuture)

DIFFICULTY: Hard **CATEGORY TAG:** Spring Boot

ANSWER: First rule I state in interviews: `@Async` **only works through a Spring proxy called from outside the bean** — a self-invocation (`this.asyncMethod()`) bypasses the proxy and runs synchronously, silently. We always put async methods on separate beans (e.g. `NotificationDispatcher`) and call them via the injected interface. Second rule: **never the default SimpleAsyncTaskExecutor** — every `@Async` is bound to an explicit `ThreadPoolTaskExecutor` bean; we run three pools by workload: `ioPool` (external calls: core 12/max 24, queue 200), `cpuPool` (scoring: core = cores), `notifyPool` (core 4/max 8) — isolation so a Gmail slowdown can't consume the threads that scoring needs (bulkhead pattern).

For composing flows we use `CompletableFuture.allOf(...)` where a page needs lead + score + calendar data from three services in parallel — each supplier runs on `ioPool`, overall timeout via `orTimeout(800ms)` (Java 9+), and a fallback chain (`exceptionally`) renders partial data rather than a 500. The critical interaction interviewers drill: `@Async` + `@Transactional` — an async method runs on a different thread, so the caller's transaction is NOT visible; if the async method needs a transaction it opens its own (`@Transactional` on the async method), and data written by the caller may not be committed yet when the async task starts (race). Our pattern: publish async work only AFTER the transaction commits — `@TransactionalEventListener(phase = AFTER_COMMIT)` calls the dispatcher, which guarantees the row exists before any consumer reads it.

Shutdown matters too: executors get `setWaitForTasksToCompleteOnShutdown(true)` + `awaitTermination` so a deploy drains in-flight async tasks instead of killing them mid-send (we still design handlers idempotent, because a 30s drain can't wait forever).

KEY TERMS TO MENTION: - Proxy semantics — self-invocation breaks `@Async` - Named executors per workload (bulkhead), never the default executor - `CompletableFuture` composition with `orTimeout` + `exceptionally` fallbacks - `@TransactionalEventListener(AFTER_COMMIT)` to avoid read-before-commit races - Graceful shutdown draining (`waitForTasksToCompleteOnShutdown`)

FOLLOW-UP QUESTIONS: 1. How do you propagate context (auth user, trace id) to an async thread? 2. What happens if the queue fills up — walk through `CallerRunsPolicy` trade-offs.

FOLLOW-UP ANSWERS: 1. `TaskDecorator` on the executor: it wraps every submitted `Runnable`, copying `SecurityContext`, MDC (traceId), and tenant context onto the worker thread and clearing after. Without it, async logs have no trace id and security checks fail — the most common async bug in real codebases. 2. With `CallerRunsPolicy`, an overflowed task runs on the caller (Tomcat) thread — back-pressure: the API slows down instead of dropping work, which is right for user-visible flows and wrong for fire-and-forget (it would stall the request). For fire-and-forget we use `AbortPolicy` + a metric alarm, because dropping a notification is better than blocking a request. The policy is a product decision, not a default.

RED FLAGS TO AVOID: - Not knowing self-invocation breaks the proxy - Using the default executor / unbounded queues (OOM under load) - Async + transaction interaction confusion (reading uncommitted data)

Q3-10: How did you implement scheduled tasks (@Scheduled, cron)

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: We run about a dozen scheduled jobs — monthly credit grants, reminder sweeps, token refreshes, nightly reconciliation, metrics rollups. Mechanics: `@EnableScheduling`, methods annotated `@Scheduled(cron = "0 0 8 * * *", zone = "Asia/Kolkata")` — explicit zone because “8 AM” must mean the customer’s business day, not the server’s UTC clock. Two production rules beyond the annotation. **Rule 1 — multi-instance safety:** with 3 app replicas behind the ALB, every `@Scheduled` firing would run 3 times; each job body first acquires a Redis lock (`SET job:lock:reminders NX PX <window>`); no lock → skip silently. We considered `ShedLock` but the raw pattern was 5 lines and dependency-free. **Rule 2 — job-table delegation for anything heavy:** the scheduler’s job is only to `enqueue` (insert rows into the job table / flip batch job parameters); the worker pool does the work. That keeps cron methods sub-second (no overlap even if delayed), gives per-item retry/status visibility, and lets us replay a failed night with one SQL statement instead of waiting for tomorrow’s cron.

Failure handling: each job wraps its body in try/catch that logs with the job name + duration metric (`SchedulerJobDuration` tagged by job) and increments a failure counter — silent cron deaths are the worst failure mode, so every job emits a heartbeat we alert on absence (“reminder sweep hasn’t run in 26h” pages us before users notice). `fixedDelay` vs `cron`: `fixedDelay` for self-spacing pollers (metrics), `cron` for business-time events. Testing: scheduler methods are thin, so tests target the service they call; for cron expressions we use a clock-injection (`Clock` bean) to simulate month boundaries in tests — the monthly grant edge case (Feb 28→Mar 1) bit us once.

KEY TERMS TO MENTION: - `@Scheduled` cron + explicit `zone` - Redis lock per job for multi-instance dedupe (or ShedLock) - Scheduler enqueues, workers execute (job-table pattern) - Heartbeat metric + absence alerting (silent death detection) - `Clock` bean injection for testable time logic

FOLLOW-UP QUESTIONS: 1. What if a job takes longer than its schedule interval? 2. Why Redis lock over ShedLock or Quartz?

FOLLOW-UP ANSWERS: 1. Cron fires are skipped while the previous one still runs on the same instance (single-threaded scheduler by default), but OTHER instances would start a second run — hence the lock. For the enqueue-only pattern the cron is milliseconds anyway; the heavy worker is decoupled, so overlap becomes a non-issue by design. 2. Quartz = DB-backed clustering but a whole framework to operate; ShedLock = nice abstraction over the same lock idea. Our 5-line Redis pattern had zero new dependencies and the ops team already knew how to inspect Redis keys. If job count grew past ~30 with complex calendars, I’d take ShedLock for the standardization.

RED FLAGS TO AVOID: - No multi-instance story (duplicate emails at 8 AM is the classic war story) - Heavy logic inside the cron method itself - No observability — jobs that die silently for weeks

Q3-11: How did you use Spring Events for decoupled communication

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Spring’s `ApplicationEventPublisher` is our in-module message bus. Concrete example: when an outreach reply is classified as INTERESTED, `ReplyClassificationService` publishes `LeadBecameHotEvent(leadId, orgId, score)` — it knows nothing about notifications, deals, or analytics. Listeners do their own thing: `NotificationListener` pings the user’s Telegram, `AnalyticsListener` increments counters, `CrmSyncListener` bumps the pipeline stage. The publisher stays clean of downstream concerns, and adding a new reaction (say, a Slack integration) is a new `@EventListener` class — zero changes to the classification service (open/closed principle in practice).

The critical correctness point: listeners by default run **synchronously in the publisher’s thread and transaction** — a throwing listener would roll back the classification itself. So our rules: side-effect listeners are (a) `@Async` AND (b)

`@TransactionalEventListener(phase = AFTER_COMMIT)` — they run after the publisher's commit, on the executor pool, and receive the event payload (ids, never entities — a detached entity across threads is a `LazyInitializationException` factory). For listeners that must read data written by the committing transaction, `AFTER_COMMIT` is exactly the guarantee you need; for audit-log listeners that must not lose records even on rollback, plain `@EventListener` inside the transaction is correct (audit of a rolled-back action is legitimate).

Trade-off I admit honestly: events make flow harder to trace — “who reacts to `LeadBecameHotEvent?`” isn't in the call stack. Mitigations: events live in a dedicated `events` package with a registry test (every event has at least one listener — catches typos silently swallowed), and every listener logs with the event id so a trace id follows the whole reaction chain. If we needed cross-service delivery with durability, the same event classes become Kafka payloads — the seam is already there.

KEY TERMS TO MENTION: - `ApplicationEventPublisher` + `@EventListener` - `@TransactionalEventListener(AFTER_COMMIT)` + `@Async` (thread + transaction semantics) - Payload = ids, not entities (detachment + lazy-loading hazards) - Registry test (every event ≥ 1 listener) - In-process now, Kafka-shaped later (same contracts)

FOLLOW-UP QUESTIONS: 1. What's the failure model — if a listener dies, does the publisher care? 2. Events vs direct service calls — when do you choose which?

FOLLOW-UP ANSWERS: 1. Fire-and-forget: a listener exception is logged with the event id and counted; it never affects the publisher. That's why analytics/listeners must be idempotent — redelivery-on-retry is possible. Anything that **MUST** succeed transactionally (writing the reply record itself) is a direct call, not an event. 2. Direct call when the outcome is part of the use case's contract (validate → save → return id); event when it's a reaction others may extend (notify, analytics). If I can't decide, I ask: “would the caller wait for this result?” Waiting → call; no → event.

RED FLAGS TO AVOID: - Not knowing listeners run synchronously by default - Publishing entities instead of ids - Using events for core flow steps that must succeed (hidden async gaps)

Q3-12: How did you implement resilience — circuit breakers, retries, timeouts

DIFFICULTY: Hard **CATEGORY TAG:** Spring Boot / Resilience4j

ANSWER: Every outbound call (LLM provider, Gmail, Twilio, scraping targets) is wrapped in a Resilience4j stack, ordered inner-to-outer: **TimeLimit** (don't wait forever) → **Retry** (transient faults) → **CircuitBreaker** (stop hammering a dead provider) → **RateLimiter** (protect our quota). Defaults per call: timeout 5-30s by provider SLA; retry 3 with exponential backoff + jitter (100ms → 400ms → 1.6s) only on 429/5xx/timeouts — never on 4xx (a 400 will fail identically forever); circuit breaker sliding window of 50 calls, 50% failure rate opens the circuit for 30s with half-open probing.

The design decision that matters most: **fallbacks per use-case, not generic**. When the LLM circuit opens during outreach generation, the user doesn't get a 500 — the generator falls back to a deterministic template engine (merge-fields email) and marks the message `AI_FALLBACK`, so the product degrades in quality, not availability. When the scraping breaker opens, the batch chunk skips and retries next window. Fallback choice is a product decision documented per integration in our ADRs.

Config lives in `application.yml` under `resilience4j.*` with different profiles per environment (staging has aggressive timeouts so tests fail fast). Observability: Resilience4j publishes Micrometer metrics — `resilience4j.circuitbreaker.state` (gauge), call outcomes, retry counts — on our Grafana; the on-call dashboard shows breaker states as traffic lights, and a Slack alert fires when any breaker stays OPEN > 10 min. Interview lesson I share: retries without jitter + breakers cause **retry storms** — a provider hiccup amplified by every client instance retrying in lockstep; the jitter and the open-circuit back-off are what turn retries from a footgun into a tool.

KEY TERMS TO MENTION: - Resilience4j stack order: TimeLimit → Retry → CircuitBreaker → RateLimiter - Retry only on transient errors, backoff + jitter (retry storm prevention) - Sliding-window failure-rate breaker, half-open probing - Use-case-specific fallbacks (template engine fallback for AI) - Micrometer metrics + breaker-state alerting

FOLLOW-UP QUESTIONS: 1. Circuit open on the AI provider — a paying user clicks “generate”. Exactly what do they see? 2. Why exponential backoff WITH jitter?

FOLLOW-UP ANSWERS: 1. The controller path checks the breaker state first (decorated supplier returns fallback): user gets the template-merged email within normal latency, a subtle “generated from template” tag, and no error. Credits for AI generation aren't deducted for fallback messages (product rule). If the breaker stays open > 15 min, the banner shows degraded AI status; support is notified proactively. 2. Without jitter, N instances that failed at the same instant retry at the same instants forever — synchronized hammering (thundering herd). Jitter randomizes each backoff so retries spread out; combined with the open circuit cutting call volume to zero, the provider gets breathing room to recover instead of a synchronized storm.

RED FLAGS TO AVOID: - Retrying 4xx (non-transient) or infinite retries - Generic 500 to users when a fallback exists - No metrics on breaker state — resilience you can't see is theater

Q3-13: What is your approach to API versioning

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot / REST

ANSWER: We version in the URL path (`/api/v1/leads` , `/api/v2/leads`) — pragmatic reasons: visible in logs and curl, trivial to route (the ALB/ingress can split traffic), zero client-library cleverness, and cache-friendly. Alternatives I can defend but rejected: header versioning (`Accept: application/vnd.acq.v2+json`) keeps URLs clean but is invisible in access logs, easy to misconfigure in browsers/webhooks, and harder to document; query-param versioning

pollutes cache keys. Rule-based compatibility: within a major version, changes must be **additive and backward-compatible** (new optional fields, new endpoints) — we verify with consumer-driven contract tests so “v1 never breaks” is enforced, not promised.

A real v2 we ran: the leads list response moved from offset pagination to cursor pagination and flattened the score object. Mechanically: v1 and v2 controllers delegate to the SAME service; only DTO assemblers differ (`LeadV1Assembler` vs `LeadV2Assembler`) — business logic never forks, which is the discipline that keeps two versions maintainable. Deprecation protocol: response header `Deprecation: true` + `Sunset` date on v1 endpoints (monitored — we log which orgs still call v1), direct email to those orgs’ admins 60/30/7 days out, and traffic dashboards driving the final removal gate (v1 deleted when < 1% of calls for 2 weeks). Version lifecycle for internal APIs is shorter (2 sprints notice) because we control both sides; public/partner APIs get the full 6-month protocol.

KEY TERMS TO MENTION: - Path versioning + traffic-splitting at the gateway - Additive-only within a major version (contract tests enforce) - Shared service, per-version DTO assemblers (no logic fork) - `Deprecation` / `Sunset` headers + caller dashboards - Data-driven removal gate (< 1% traffic)

FOLLOW-UP QUESTIONS: 1. How do you version a breaking DB schema change that backs both versions? 2. GraphQL solves versioning differently — would you use it here?

FOLLOW-UP ANSWERS: 1. Expand-contract: the migration adds the new shape alongside the old (new column/table, dual-write), v2 assemblers read the new shape, v1 keeps reading the old, and the contract step (dropping the old) happens only after v1 sunset. The DB version window can outlive the API version window — that’s fine, migrations are forward-only. 2. GraphQL’s “no versioning, evolve the schema” works when the consumer mix is internal and schema discipline is high (deprecations enforced). For a public B2B REST API with webhooks, partners, and non-technical integrators, path versioning is simpler to reason about and to support. I’d consider GraphQL for the frontend-heavy dashboard layer, not as the public contract.

RED FLAGS TO AVOID: - “We never needed versions” (means no consumers or no growth) - Forking business logic between versions (maintenance death) - No deprecation protocol — “we told everyone once in a blog post”

Q3-14: How did you implement CORS configuration

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot / Security

ANSWER: CORS is the browser’s enforcement of which origins may call the API; the server declares it via headers. In Spring Security 6 we configure it on the `SecurityFilterChain`: a `CorsConfigurationSource` bean with an **explicit allow-list** — production allows exactly our two frontend origins (marketing site + app subdomain), staging allows the staging origin; `allowedMethods` limited to what we actually use (GET/POST/PUT/PATCH/DELETE — no wildcards), `allowedHeaders` explicit (`Authorization` , `Content-Type` , `X-Request-Id`),

`allowCredentials(true)` only because our cookie-based CSRF double-submit needs it, and `maxAge(3600)` so browsers cache the preflight and don't re-OPTIONS every call.

Two mistakes I call out from review experience. First, `allowedOrigins("*")` with `allowCredentials(true)` is illegal per spec and Spring rejects it — the “fix” people reach for (`allowedOriginPatterns("*")`) defeats the entire protection; our code review blocks both patterns via a static check. Second, preflight understanding: the browser sends OPTIONS before a non-simple request; if a security filter chain rejects OPTIONS before the CorsFilter, every cross-origin POST dies with a confusing 401 — so the chain order matters (`.cors()` first, and the CORS filter must answer preflights without auth). Webhook endpoints (Stripe → us) are NOT browser calls, so CORS is irrelevant there — server-to-server; common confusion point I clarify preemptively in interviews.

KEY TERMS TO MENTION: - `SecurityFilterChain` + `CorsConfigurationSource` bean - Explicit origin allow-list (no `*` with credentials) - Preflight OPTIONS flow + filter-chain ordering - `maxAge` preflight caching - Server-to-server calls don't involve CORS at all

FOLLOW-UP QUESTIONS: 1. Why is `*` with credentials dangerous? 2. Where does CORS enforcement actually happen?

FOLLOW-UP ANSWERS: 1. With credentials allowed and any origin accepted, any malicious site can make the victim's browser call your API WITH their cookies attached and read responses — CSRF-level access with read capability. It converts the Same-Origin Policy (the browser's core defense) into a no-op for your domain. 2. In the browser, not the server — the server only `declares` policy via headers; the browser blocks the response if policy isn't satisfied. That's why curl/Postman “work fine” while the browser fails, and why “it works in Postman” is never a CORS rebuttal.

RED FLAGS TO AVOID: - Suggesting `allowedOriginPatterns("*")` with credentials as a “fix” - Not knowing who enforces CORS (browser) vs who declares it (server) - Treating webhook/callback endpoints as CORS concerns

Q3-15: How did you handle file uploads and storage

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Uploads (lead-import CSVs, feedback attachments, brand logos) follow one pipeline regardless of type. Ingestion: `MultipartFile` on a `@PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)` with multipart limits set explicitly (`spring.servlet.multipart.max-file-size=10MB`, `max-request-size=12MB`) and a `MaxUploadSizeExceededException` handler → clean 413. **Validation is content-based, not trust-based:** extension and declared MIME type are advisory only — we sniff magic bytes (Apache Tika) to determine the real type, enforce an allow-list per endpoint (logos: png/jpg/svg-sanitized; feedback: images + pdf + mp4), cap pixel dimensions for images, and never serve user uploads from the app origin — they go to object storage (S3-compatible) under a random UUID key and are served via a separate domain with

Content-Disposition: attachment + **Content-Type** from the sniffed type, killing stored-XSS via HTML/SVG uploads (we sanitize SVG separately or reject it where not needed).

Large files don't flow through the JVM heap: multipart streaming writes to a temp file (container ephemeral disk) and we stream **InputStream** → S3 multipart upload with a size meter, so a 10MB upload costs ~constant memory. The DB stores only metadata (key, owner, content_type, size, checksum SHA-256) — never bytes; a background job reconciles orphaned objects (uploaded but never claimed) with lifecycle deletion after 24h. Download authorization is enforced by presigned URLs (short-TTL, 5 min) generated only after an ownership check — the object storage is never publicly listable. Antivirus/ClamAV scanning runs async for feedback attachments (user-to-admin surface), holding them in a quarantine bucket until clean.

KEY TERMS TO MENTION: - Magic-byte sniffing (Tika) over client-declared MIME - Allow-list per endpoint + size/dimension caps - Separate serving domain + **Content-Disposition: attachment** (stored-XSS defense) - Streaming to S3, metadata-only in DB, presigned short-TTL downloads - Quarantine + async AV scan for user-to-user surfaces

FOLLOW-UP QUESTIONS: 1. Why is serving uploads from the app domain dangerous? 2. How would you handle a 2GB import file?

FOLLOW-UP ANSWERS: 1. Same-origin means an uploaded HTML/SVG executes with your app's origin — cookies, localStorage tokens, same-origin API calls all become available to the attacker's script. A separate "dirty" domain sandboxes the blast radius: even if a payload runs, it's not on your origin and can't touch your cookies. 2. Two-phase: client requests an upload session → we return presigned S3 PUT URLs (direct-to-storage, multipart), the app never touches the bytes; on completion the client notifies the API, we verify checksum + size server-side and enqueue the parse job (Spring Batch reading the CSV from S3 in chunks). Browser→app→storage hops are for small files only.

RED FLAGS TO AVOID: - Trusting client MIME type/extension (the textbook upload vuln) - Storing bytes in the DB or serving from the app origin - Public buckets or unexpiring URLs

Q3-16: How did you implement email sending (JavaMailSender)

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Email is async, templated, and provider-abstracted. Abstraction: an **EmailPort** interface (**send(EmailMessage)**), implemented by **SmtplibAdapter** (JavaMailSender + Gmail SMTP OAuth2/XOAUTH2 for per-user sending mailboxes) — product notifications use the platform's transactional account; outreach uses the USER's connected mailbox (that's what makes outreach deliverable — it comes from their own domain, not ours). All sends happen on the **notifyPool** executor via the AFTER_COMMIT event pattern — an email that blocks a request thread or rolls back with a transaction is a design bug.

Templating: Thymeleaf templates (**email/order-confirmed.html**) with a strict model contract — templates are unit-tested with a snapshot assertion so a model change that breaks a template fails CI, not production. Every message carries headers for deliverability + tracking:

Message-ID (stored in DB as the idempotency/tracking key), **List-Unsubscribe** (bulk mail compliance), and custom tracking IDs for open/click pixel correlation. Bounce/complaint handling: Gmail/SES feedback loops post to a webhook that marks the recipient as BOUNCED on the org's send-list — three bounces suppress the address automatically (protecting the user's sender reputation is a product feature, not an ops nicety).

Reliability: sends go through the job table with retry (3x, backoff) on transient SMTP errors (4xx rate limits), immediate fail on 5xx auth errors (retrying a bad credential is noise), and a dead-letter state alerting support when a message fails permanently. Rate: per-mailbox token bucket in Redis (Gmail ~ 500/day/user soft limit) — the enforcer delays sends to next-window rather than failing, because outreach campaigns hitting provider limits is a daily reality, not an exception.

KEY TERMS TO MENTION: - **EmailPort** abstraction + JavaMailSender/XOAUTH2 adapter - Async AFTER_COMMIT send on a dedicated executor - Thymeleaf + snapshot-tested templates - **Message-ID** as idempotency key, bounce webhooks → suppression list - Redis token bucket per mailbox (provider rate limits)

FOLLOW-UP QUESTIONS: 1. SMTP is slow — how do you keep p95 API latency unaffected? 2. How do you prevent your platform being used for spam?

FOLLOW-UP ANSWERS: 1. The request path only persists the message row (QUEUED) and publishes an event — SMTP happens later on the worker; the API returns in ~40ms. Delivery status reaches the user via the existing notification channel, not by holding the request. 2. Layered: verified domains only (users connect OAuth mailboxes we can verify), per-mailbox and per-org rate caps, content policy checks (unsubscribe header mandatory on bulk, prohibited-content filter), bounce-driven auto-suppression, and audit logs on every send. Abuse signal = bounce+complaint ratio per org crossing threshold → sends paused pending review. Deliverability protection doubles as abuse control.

RED FLAGS TO AVOID: - Sending email synchronously inside a transaction/request - No idempotency key → duplicates on retry - No bounce handling (deliverability death spiral)

Q3-17: How did you use Spring Profiles for different environments

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot

ANSWER: Four profiles: **dev** (local — H2/in-memory Redis via Testcontainers optional, SQL logging on, fake email transport that writes to a folder, seeded data), **staging** (prod-shaped: real Postgres/Redis, mocked payment providers in sandbox mode, verbose logging), **prod** (real everything, JSON logs, metrics on, secrets from the secrets manager), **test** (slices, mocked ports, deterministic clock). Activation is external: **spring.profiles.active** from env, never hardcoded; profile-specific files **application-{profile}.yaml** override **application.yaml** defaults, and secrets are NEVER in any yaml — they're env/secrets-manager references resolved at boot.

The rules that keep profiles honest: (1) **prod parity** — staging runs the same container image with a different profile, so “works in staging” means something; (2) **fail-fast**

misconfiguration — a `@ConfigurationProperties @Validated` bean + an `EnvRequirements` checker at boot asserts required vars per profile (prod without `STRIPE_WEBHOOK_SECRET` refuses to start — better a crash loop at deploy than silent broken webhooks); (3) **no `@Profile`-gated business logic** — profiles switch infrastructure beans (mail transport, provider sandboxes), never product behavior; behavior differences are explicit feature flags with admin UI, because flags are auditable and reversible at runtime while profiles require redeploy. One war story I tell: before the boot-time validator, a staging deploy silently ran without Redis config and fell back to an embedded map — tests passed, the cache was never actually exercised, and prod behavior differed. The validator turned that class of error into a startup failure.

KEY TERMS TO MENTION: - Profile-per-environment + `application-{profile}.yml` layering - Secrets externalized (never in yml/git) - Boot-time env validation (fail-fast on missing config) - Profiles for infrastructure, feature flags for behavior - Staging = same image, different profile

FOLLOW-UP QUESTIONS: 1. How do you handle config that must change without redeploy? 2. Difference between profiles and `@ConditionalOnProperty`?

FOLLOW-UP ANSWERS: 1. Two mechanisms: feature flags (behavior, admin UI, instant) and Spring Cloud Config/refresh-scope for infrastructure tunables (pool sizes) — but honestly 95% of “change without deploy” asks are product behavior, which is flags. Config drift is monitored: a daily job diffs effective config (actuator env endpoint, sanitized) against git, so hand-edits surface. 2. `@ConditionalOnProperty` creates beans conditionally at boot — an infrastructure switch; profiles are the environment selector. They compose (a bean can be registered when `cache.enabled=true` AND `profile=prod`). The anti-pattern is overloading profiles to toggle features — that’s what flags are for.

RED FLAGS TO AVOID: - Secrets in yml or `@Profile`’d business logic - No boot-time validation (config errors surface at runtime, mid-customer) - “Staging is a smaller different app” — parity matters more than cost

Q3-18: How did you implement health checks and actuator endpoints

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot / Observability

ANSWER: Actuator is exposed over a **separate management port** (9090) on the internal interface only — never proxied publicly; the ALB target-group health check hits `/actuator/health` on that port. Health is split: **liveness** (`/actuator/health/liveness`) = “is the JVM wedged” — always up if the process answers; **readiness** (`/readiness`) = “can I serve traffic” — aggregates checked indicators: DB (`DataSourceHealthIndicator` with a cheap validation query), Redis ping, disk space (temp dir for uploads, threshold 20%), and the job-table poller lag (custom indicator: HEALTHY if lag < 60s, DEGRADED 60-300s). Readiness false → instance removed from the ALB (deploy drain, dependency outage) without killing the pod — that’s the whole point of the split: restart won’t fix a Redis outage, but pulling traffic prevents user-facing errors while on-call fixes Redis.

Beyond health: `/metrics` (Prometheus format — Micrometer counters/histograms/timers for every controller, executor, pool), `/info` (git commit + build time — “what’s running” answered in one click during an incident). Security: management port unauthenticated INSIDE the VPC but the info/env endpoints are restricted by an internal network policy; anything config-revealing is env-sanitized (`show-values: never`). Custom indicators I added: `StripeHealthIndicator` (circuit state + last webhook age — a stale webhook age is the earliest signal Stripe connectivity broke) and `AiProviderHealthIndicator` (breaker state), so the ops dashboard answers “which dependency is degraded” without log spelunking. Load balancer checks stay cheap (a `/health` lite variant without dependency checks) to avoid health-check storms doing real work every 5 seconds per instance.

KEY TERMS TO MENTION: - Management port separation + internal-only exposure - Liveness vs readiness semantics (restart vs drain) - Custom `HealthIndicator`s (webhook freshness, breaker state) - Micrometer metrics + git info endpoint - Cheap LB checks vs full readiness

FOLLOW-UP QUESTIONS: 1. During a deploy, what does the ALB see? 2. A dependency is down but the app works without it — ready or not?

FOLLOW-UP ANSWERS: 1. New instance starts → readiness false until DB/Redis checks pass → ALB registers it; old instances get deregistered with connection draining (30s) letting in-flight requests finish; zero-downtime because traffic shifts only to instances that declared ready. 2. That’s a triage decision encoded per indicator: if the app degrades gracefully (fallbacks), the dependency indicator reports DEGRADED (still ready, dashboard shows yellow, users unaffected); if functionality is genuinely broken without it (DB), it’s DOWN and readiness drops. Health is a product statement, not just a ping.

RED FLAGS TO AVOID: - Exposing actuator publicly (classic finding in audits) - One mega health endpoint conflating liveness with dependencies - “Green dashboard, broken webhooks” — no freshness indicators

Q3-19: How did you handle configuration externalization and secrets

DIFFICULTY: Medium **CATEGORY TAG:** Spring Boot

ANSWER: Layered config with clear ownership: defaults in `application.yml` (sane, safe), environment overrides via env vars (`SPRING_APPLICATION_JSON` + individual vars from the secrets manager at container start), runtime-tunable behavior in feature flags (DB-backed, admin UI), and secrets in **AWS Secrets Manager** — fetched by an init sidecar that exports them as env vars to the main container; the JVM never reads a secrets file from disk, and the secrets never live in the image, git, or CI logs (CI references, not values). Rotation: DB password rotates quarterly via a two-phase rollout — update secrets manager → rolling restart consumers with both creds valid window (Postgres supports dual-password transitions via `pg_hba` timing) — and JWT signing keys are versioned (`kid` header) so rotation signs new tokens with the new key while old keys verify until TTL expiry; zero logout storms.

Inside the app, `@ConfigurationProperties` classes (typed, validated, IDE-navigable) bind config — string-pulling `@Value` is banned by review convention beyond trivial one-offs, because typed config is testable and documents itself. Sensitive-value hygiene: actuator env endpoint `show-values: never`; log masking via a Logback converter that scrubs patterns (Bearer tokens, emails in some contexts, card data — we never accept card data anyway, Stripe does); and a boot-time “secret audit” that fails startup if any known-secret-name appears resolved from a non-secrets source (catches the intern putting SMTP_PASSWORD in application-prod.yml — that PR gets rejected by CI, not found in an audit).

KEY TERMS TO MENTION: - Secrets Manager + init sidecar → env vars (no secrets on disk/image/git) - `@ConfigurationProperties` typed validated binding - Key rotation with `kid` versioning (JWT), dual-valid window (DB) - Log masking converter + actuator `show-values: never` - Boot-time secret-source audit (fail-fast)

FOLLOW-UP QUESTIONS: 1. A secret leaked to git history — walk me through your response. 2. Config drift between replicas — how do you detect it?

FOLLOW-UP ANSWERS: 1. Treat as compromised, not “delete the commit”: (1) rotate the secret immediately (invalidates the leak), (2) purge history + force-push, (3) audit access logs for the secret’s misuse window, (4) postmortem the pipeline gap that allowed it (add pre-commit/CI secret scanning — gitleaks — so the next one never lands). Rotation first, because git history is forever once pushed. 2. Config is derived from env at boot, so drift means deploys diverged — the `/info` git commit + a config hash (sha of effective sanitized config) exposed per instance; a Grafana panel alerts when instances of the same service report different hashes. It caught a stuck rollout once where one pod ran yesterday’s image.

RED FLAGS TO AVOID: - Secrets in yml/git/CI variables without rotation story - No masking in logs (secrets end up in ELK, readable by everyone) - “We’d rotate manually” with no tested procedure

Q3-20: What design patterns did you use in Spring Boot (beyond what the framework gives you)

DIFFICULTY: Hard **CATEGORY TAG:** Spring Boot / Design

ANSWER: I frame the answer as “patterns the framework already gives us” vs “patterns we added”. **Framework-provided** (worth naming explicitly to show depth): DI is Service-Locator inverted into constructor injection; `RestTemplate` / `JdbcTemplate` are Template Method (skeleton with pluggable steps); Spring MVC’s `HandlerAdapter` is Strategy; `ApplicationEventPublisher` is Observer; proxies (`@Transactional`, `@Async`, `@Cacheable`) are Proxy/Decorator; `Builder` is everywhere in Lombok-generated and Testcontainers-style APIs.

Patterns we added, with the why: (1) **Ports & Adapters at integration edges** — `EmailPort`, `SmsPort`, `AiPort` interfaces with per-provider adapters (SMTP adapter, Twilio adapter, OpenAI/Gemini adapters). Payoff: the AI provider fallback chain is literally a list of adapters tried in order; swapping Gemini for Claude is a new class + config, and tests mock the port, not HTTP. (2) **Strategy for pricing** — `PricingStrategy` per plan tier (Flat, Metered,

Enterprise) chosen by a registry keyed on plan; adding a plan = new strategy class, no if-chains. (3) **Factory + Registry for channel routing** — notifications route by channel type to `ChannelSender` implementations discovered from the context; the router is closed for modification, open for extension. (4) **Facade for the AI orchestration** — `OutreachFacade` composes prompt-build → provider call → validation → credit deduction behind one method so controllers stay thin and the 6-step flow has one tested home. (5) **Domain exceptions + `@RestControllerAdvice`** as the Exception pattern mapping errors to ProblemDetail codes centrally.

Anti-overengineering caveat I state proactively: we did NOT use AbstractFactoryBuilder nonsense for one-implementation cases — a pattern earns its keep when there's a second implementation on the horizon (there always was: providers multiply in SaaS). Patterns without a forcing function are resume-driven architecture, and I'd rather delete code than polish it.

KEY TERMS TO MENTION: - Recognizing framework patterns (proxy, template method, observer) - Ports & Adapters on integration edges (provider swap/fallback chains) - Strategy (pricing), Registry/Factory (channel routing), Facade (AI flow) - Centralized exception → ProblemDetail mapping - Pattern discipline — no pattern without a second implementation

FOLLOW-UP QUESTIONS: 1. Show how the Strategy pattern concretely reduced complexity. 2. Where did a pattern cost you more than it paid?

FOLLOW-UP ANSWERS: 1. Before: `PricingService.calculate()` had nested if/else per plan × per action type × per discount flag — 200 lines, every plan change risked regressions. After: three strategy classes + a registry; unit tests per strategy are table-driven; adding the Enterprise tier was a new class + one registration line, zero diff in existing classes. The tell of a good pattern: the diff for change #3 is tiny. 2. We introduced a Specification pattern for lead filtering when there were 3 filters — pure ceremony over a simple `WHERE`. It was deleted after the criteria stabilized; a plain query builder was clearer. Lesson recorded in our ADR: introduce abstraction at 3+ stable variants, not at 1 speculative one.

RED FLAGS TO AVOID: - Listing GoF names without where/why in YOUR codebase - Claiming every pattern under the sun (nobody used 15 patterns well) - No anti-overengineering stance — seniors know when NOT to abstract

CATEGORY 4 — SPRING SECURITY QUESTIONS

Q4-1: How did you implement authentication in AcquisitionOS

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: Three authentication methods, all backed by Spring Security 6: (1) **Email + password** with BCrypt hashing + OTP for first-time verification; (2) **Magic Link** (passwordless, emailed token); (3) **Google OAuth2** via Spring Security's OAuth2 Login. All three produce a JWT (HS256) stored in an HttpOnly cookie + a refresh JWT stored in an HttpOnly cookie scoped to `/api/auth`.

SecurityFilterChain:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // stateless JWT, no CSRF needed
        // (SameSite=strict cookies are the CSRF defense)
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(PUBLIC_PATHS).permitAll()
            .requestMatchers("/api/admin/**").hasAnyRole("SUPER_ADMIN", "OWNER",
                "ADMIN")
            .requestMatchers("/api/**").authenticated()
            .anyRequest().permitAll()
        )
        .oauth2Login(oauth2 -> oauth2
            .loginPage("/login")
            .userInfoEndpoint(ui -> ui.userService(googleOAuth2UserService))
            .successHandler(oauth2SuccessHandler)
        )
        .oauth2ResourceServer(rs -> rs.jwt(jwt -> jwt.decoder(jwtDecoder())))
        .addFilterBefore(new RateLimitFilter(rateLimiter),
            UsernamePasswordAuthenticationFilter.class)
        .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class)
        .exceptionHandling(eh -> eh
            .authenticationEntryPoint(jwtAuthEntryPoint)
            .accessDeniedHandler(jwtAccessDeniedHandler))
        .headers(h -> h.frameOptions(fo -> fo.deny()));
    return http.build();
}
```

Custom filters: - `RateLimitFilter` — before `UsernamePasswordAuthenticationFilter`, returns 429 if Redis sliding-window exceeded. - `JwtAuthenticationFilter` (extends `OncePerRequestFilter`) — extracts JWT from `Authorization: Bearer <token>` header or `access_token` cookie, calls `jwtDecoder().decode(token)` (Spring Security's `NimbusJwtDecoder` with HS256 + the secret from Secrets Manager), builds `JwtAuthenticationToken` and sets `SecurityContext`. On invalid/expired token → 401 via `jwtAuthEntryPoint`.

Stateless: `sessionCreationPolicy(STATELESS)` — no HTTP session, no `JSESSIONID` cookie. Every request re-authenticates from the JWT. This is the right choice for a horizontally-scaled API (no session affinity needed).

Google OAuth2: Spring's `oauth2Login` flow — user clicks “Login with Google” → redirects to Google → Google redirects back to `/login/oauth2/code/google` → `OAuth2LoginAuthenticationFilter` exchanges the code for an access token + refresh token, calls `googleOAuth2UserService.loadUser()` (our custom) which fetches the user's profile from Google's userinfo endpoint, finds or creates the User entity in our DB, generates our internal JWT, sets cookies, redirects to `/`.

JWT structure: HS256 signed with `JWT_SECRET` (from Secrets Manager). Payload: `sub` (user id), `email`, `role`, `plan`, `orgId`, `isTrial`, `trialEndsAt`, `type` (`access` or `refresh`), `iat`, `exp`. Access token: 15min expiry. Refresh token: 30d expiry, scoped to `/api/auth/refresh`. Issuer + audience validated.

Refresh flow: `POST /api/auth/refresh` (with refresh cookie) → `RefreshTokenService.refresh()` verifies the refresh JWT, checks the `UserSession` table to ensure the session isn't revoked, issues a new access + refresh token pair (rotation), updates the session's `refreshTokenHash`. Refresh-token reuse detection: if a refresh token is used twice (the second use finds the session revoked), we revoke **all** the user's sessions (token-theft detection).

Custom `UserDetailsService`: Not used — we don't use Spring Security's username/password flow. We use our own `AuthService.signin(email, password)` which looks up the user by email, verifies BCrypt, checks `emailVerified` and `isActive`, generates the JWT. The `JwtAuthenticationFilter` is what authenticates subsequent requests, not `UserDetailsService`.

KEY TERMS TO MENTION: - Three auth methods: email+password (BCrypt), Magic Link (token), Google OAuth2 (`oauth2Login`) - `SecurityFilterChain` with `STATELESS` session, `permitAll()` for public paths, `hasAnyRole` for admin - `oauth2Login` with custom `OAuth2UserService` + `successHandler` - `oauth2ResourceServer` with `NimbusJwtDecoder` (HS256) - `JwtAuthenticationFilter` (extends `OncePerRequestFilter`) — extracts bearer or cookie, sets `SecurityContext` - `RateLimitFilter` before auth (429 before bcrypt CPU) - JWT: 15min access, 30d refresh, scoped to `/api/auth`, issuer + audience validated - Refresh-token rotation + reuse detection (token-theft → revoke all sessions) - `UserSession` table tracks active sessions, supports `revokeAll` - No `UserDetailsService` (we don't use Spring's username/password flow)

FOLLOW-UP QUESTIONS: 1. Why stateless JWT over stateful sessions? 2. How do you handle JWT secret rotation? 3. What's the token-theft detection flow?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) horizontal scaling — no session affinity needed at the ALB, any task can serve any request; (b) no session store to operate (no Redis session cluster to maintain — Redis is for cache + rate-limit only); (c) refresh tokens give us revocation (we can revoke a user's session in the `UserSession` table without invalidating all sessions). The trade-off: refresh tokens are longer-lived (30d) — a stolen refresh token is dangerous. We mitigate with reuse detection (theft → revoke all). 2. JWT secret is in Secrets

Manager. Rotation: we publish a new secret to `/acquisitionos/prod/jwt-secret/v2`, the `JwtDecoder` supports multiple secrets (we configure it with a list — `[v1, v2]`) — tokens signed with either are valid. After 30 days (max refresh token lifetime), we drop v1, leaving only v2. We've done this once in 9 months (no incidents — old refresh tokens got the new access tokens signed with v2; new sessions used v2 throughout). 3. Each refresh-token use rotates to a new token + updates `UserSession.refreshTokenHash`. If a refresh token is presented that matches a session marked `revokedAt != null` (already rotated), we know someone is reusing a stolen refresh token. We revoke **all** the user's sessions (set `revokedAt = now()` on all `UserSession` rows for that user), force re-auth everywhere. This is the OAuth 2.0 token-rotation pattern (RFC 6749 Section 10.4).

RED FLAGS TO AVOID: - "We use sessions" (interviewer wants JWT) - Not mentioning refresh-token rotation - Storing JWT in localStorage (XSS exfiltration risk) - Saying "we use BCrypt" without specifying rounds

Q4-2: Explain the OTP login flow with Spring Security

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: OTP is for users who can't remember their password or signed up via Magic Link / Google OAuth and never set a password. Flow:

```
1. User → POST /api/auth/otp/request {email}
  - OtpController.requestOtp(email)
  - AuthService.requestOtp(email):
    - find user by email; if not found, return generic "If an account exists..."
    (anti-enumeration)
    - if user.loginOtpExpiry < 60s ago, return 429 "Please wait..."
    - generate 6-digit OTP via SecureRandom
    - hash OTP with BCrypt (12 rounds), store in user.loginOtpHash (NEVER store
    plain)
    - set user.loginOtpExpiry = now + 10min
    - reset user.otpAttemptCount = 0
    - send OTP via email (JavaMailSender async)
    - return 200 "If an account exists..."

2. User → POST /api/auth/otp/verify {email, otp}
  - OtpController.verifyOtp(email, otp)
  - AuthService.verifyOtp(email, otp):
    - check account lockout (loginHistory: 5 failed in 15min → 423)
    - find user; if not found → 401 (anti-enumeration, same as wrong OTP)
    - check OTP lockout (user.otpAttemptCount >= 5 → 423)
    - if user.otpAttemptCount + 1 >= 5 → set user.otpLockedUntil = now + 15min
    - BCrypt.verify(otp, user.loginOtpHash) — constant-time
    - on success: clear OTP fields, reset attempts, generate JWT, create
    UserSession, set cookies
    - on failure: increment attempts, log to loginHistory, return 401 "Invalid email
    or OTP"
```

Security details: - OTP is 6 digits generated by `SecureRandom` (crypto-strong, not `Math.random()`) - OTP stored as BCrypt hash, not plaintext — DB compromise doesn't expose OTPs - OTP expiry 10 minutes - Max 5 attempts per OTP cycle; lockout 15min after 5 failures - Account-level lockout (5 failed logins in 15min via `loginHistory`, not just OTP) — defends against trying different OTPs across multiple requests - Rate limit per IP: 5/min on `/api/auth/otp/*` endpoints (Redis sliding window) - Anti-enumeration: same "If an account exists..." message regardless of email existence - Constant-time OTP comparison via BCrypt (no timing leak) - All OTP events audit-logged (`otp_login`, `signin_failed`)

Why not use Spring Security's One-Time Token framework (Spring 6.1+)? Spring 6.1 added `OneTimeToken` + `OttAuthenticationFilter` — we considered migrating but our flow is custom enough (account-level lockout, anti-enumeration, audit logging, custom email template) that wrapping it ourselves is simpler than conforming to Spring's Ott flow. We'd revisit if Spring adds the features we need.

KEY TERMS TO MENTION: - 6-digit `SecureRandom` OTP, 10min expiry, BCrypt hash storage - 5 attempts / 15min lockout (OTP-level) + account-level lockout (5 failed in `loginHistory`) - 5/min/IP rate limit on `/api/auth/otp/*` - Anti-enumeration: same "If an account exists..." response - Constant-time BCrypt verify (no timing leak) - Audit log: `otp_login`, `signin_failed` - Spring 6.1 `OneTimeToken` framework considered, custom flow simpler for our needs

FOLLOW-UP QUESTIONS: 1. Why BCrypt the OTP instead of just storing the hash? 2. How do you prevent OTP flooding (sending 1000 OTPs to a victim)? 3. How would you add SMS OTP?

FOLLOW-UP ANSWERS: 1. Two reasons: (a) DB compromise — if someone gets a DB dump, plaintext OTPs are immediately usable (until they expire). BCrypt-hashed OTPs require cracking (which takes seconds for 6-digit, but the attacker would also need the email access — defense in depth); (b) compliance — some standards (SOC 2) require "secrets at rest must be hashed". BCrypt satisfies this. 2. Two layers: (a) per-email 60s cooldown — a new OTP can't be requested for 60s after the previous one (rate limit by email, not IP — XFF spoofable); (b) per-IP 5/min rate limit on `/api/auth/otp/request` — catches attackers rotating emails from one IP. Combined, even a distributed attacker can send at most 5 OTPs/min/IP — a victim gets 5 OTPs/min from one attacker IP, which is annoying but not denial-of-service. We also have a per-account daily cap of 20 OTPs (logged in `otp_request_count` per user per day) — over that, we lock the account for 24h. 3. Add a `SmsOtpService` that calls Twilio's SMS API (we already have Twilio for WhatsApp). The flow is identical — generate OTP, hash, store, send via SMS instead of email. The verify endpoint is the same. We'd add a `deliveryMethod: "email" | "sms"` field on the request so the user picks. We'd add a per-account SMS cost cap (Twilio charges ₹0.5/SMS in India) — cap at 10 SMS/day/user. We'd add the SMS provider to our audit log. Currently we don't have SMS OTP because email + Magic Link + Google OAuth covers 99% of our users.

RED FLAGS TO AVOID: - Storing OTP in plaintext - Not mentioning anti-enumeration - Forgetting lockout - Saying "we use a 4-digit OTP" (too small)

Q4-3: How does Magic Link authentication work

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: Magic Link is a passwordless login — the user enters their email, we send a unique link, clicking it logs them in. Same UX as Notion / Substack.

Flow:

```
1. User → POST /api/auth/magic-link/request {email}
  - MagicLinkController.request(email)
  - MagicLinkService.request(email):
    - find user; if not found → return generic "If an account exists..." (anti-enumeration)
    - if user.magicLinkTokenExpiry < 60s ago → 429 "Please wait..."
    - generate token: SecureRandom 32 bytes → hex (64 chars, 256-bit entropy)
    - store token hash (BCrypt) in user.magicLinkTokenHash, expiry = now + 15min
    - send email with link: https://app.acquisitionos.com/api/auth/magic-link/verify?token=<token>&email=<email>
    - audit log: `magic_link_sent`
    - return 200 "If an account exists..."

2. User clicks link → GET /api/auth/magic-link/verify?token=...&email=...
  - MagicLinkController.verify(token, email)
  - MagicLinkService.verify(token, email):
    - find user by email; if not found → redirect /?auth_error=invalid_link
    - BCrypt.verify(token, user.magicLinkTokenHash); on mismatch → invalid_link
    - if now > user.magicLinkTokenExpiry → expired_link
    - clear token fields (single-use enforcement)
    - auto-verify email if not verified (magic link proves email ownership)
    - check user.isActive; if false → invalid_link (anti-enumeration)
    - generate access + refresh JWT, create UserSession
    - audit log: `magic_link_used`, `signin`
    - redirect / with auth cookies set
```

Security details: - Token: 256-bit `SecureRandom` — unbruteforceable - Stored as BCrypt hash (single-use, DB compromise doesn't expose tokens) - 15-min expiry - Single-use: token is cleared on successful verify; replay attempts get `invalid_link` - Anti-enumeration: same `invalid_link` redirect for: user not found, token mismatch, expired token, inactive account - Constant-time token comparison via BCrypt - Token emailed as a URL — email provider logs, referer header, email forwarding all expose the URL → user must click within 15min and the token is single-use (replay blocked) - Per-email 60s cooldown on request (rate limit) - Per-IP 5/min on request endpoint (Redis) - Audit log: `magic_link_sent`, `magic_link_used`, `magic_link_verify_failed`

Why not use Spring Security's OAuth2 `authorization_code` flow with PKCE for this?

Magic Link is conceptually similar (a one-time code in a URL), but Spring's OAuth2 flow is for third-party IdPs. We implemented it as a custom flow because: (a) we own both the issuer and the consumer (no third-party IdP); (b) the user experience is "click a link in email" not "redirect to IdP". We could have built it on Spring's `OneTimeToken` (Spring 6.1+), but our custom flow was already in production.

Trade-off vs OTP: Magic Link is more user-friendly (one click) but requires the user to switch to their email client. OTP is faster (user reads the code in the same screen) but requires typing. We support both — the user picks.

KEY TERMS TO MENTION: - 256-bit `SecureRandom` token, 15min expiry, BCrypt-hashed at rest (single-use) - Single-use enforcement (token cleared on success) - Anti-enumeration: same `invalid_link` for not-found, mismatch, expired, inactive - Per-email 60s cooldown + per-IP 5/min rate limit - Auto-verify email on magic link use (proves email ownership) - Audit log: `magic_link_sent`, `magic_link_used`, `magic_link_verify_failed` - Constant-time BCrypt comparison (no timing leak) - Considered Spring 6.1 `OneTimeToken`, custom flow already in prod

FOLLOW-UP QUESTIONS: 1. What if the user forwards the magic link email by accident? 2. How do you handle magic links on mobile (no email client)? 3. Why not use Spring's `OneTimeToken` framework?

FOLLOW-UP ANSWERS: 1. Two protections: (a) single-use — the token is cleared after first use, so even if the forwarded link is clicked, it's `invalid_link`; (b) 15min expiry — even an unused link expires quickly. The risk: if the attacker clicks the link *before* the legitimate user, the attacker logs in and the legitimate user gets `invalid_link` (and contacts support). We mitigate by: (a) audit log shows `magic_link_used` from a new IP — `SuspiciousLoginService` flags it and emails the user “If this wasn't you, click here to revoke”; (b) we have a “Revoke all sessions” button in settings. We've had one reported case in 9 months (user forwarded their own link to a colleague accidentally) — no harm because single-use. 2. On mobile, the email client is usually configured, so the link opens the in-app browser. We deep-link via a custom scheme `acquisitionos://magic-link?token=...&email=...` if the app is installed. The web link falls back to the in-app browser. Both end up at the same verify endpoint. We also support a POST `/api/auth/magic-link/verify` for the mobile app to call directly (with the token + email from the deep link) instead of going through a browser redirect. 3. Spring 6.1 added `OneTimeToken` + `OttAuthenticationFilter` — they handle the token generation, validation, and authentication flow. We looked at migrating but our flow has custom features (anti-enumeration redirect, single-use + immediate token clear, account-level logout, custom email template, audit logging) that don't map cleanly to Spring's defaults. The migration would have added complexity without removing our custom code. We'll revisit when Spring's framework has more flexibility (or when we extract auth into its own service).

RED FLAGS TO AVOID: - Storing the magic link token in plaintext - Allowing magic link reuse - No expiry - Saying “we use JWT for magic links” (overcomplicated — a random token is simpler and safer)

Q4-4: How did you implement Google OAuth2 with Spring Security

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: We use Spring Security's `oauth2Login` — the standard OAuth2 Authorization Code flow with PKCE. Flow:

1. User clicks "Login with Google"
 - Frontend redirects to /oauth2/authorization/google
 - Spring's OAuth2AuthorizationRequestRedirectFilter intercepts
 - Builds auth URL: `https://accounts.google.com/o/oauth2/v2/auth?client_id=...redirect_uri=https://app.acquisitionos.com/login/oauth2/code/googleresponse_type=codescope=openid email profilestate=<random> (CSRF protection)`
 - Redirects user to Google
2. User authenticates with Google, grants consent
 - Google redirects to `/login/oauth2/code/google?code=<auth_code>&state=<random>`
 - Spring's OAuth2LoginAuthenticationFilter intercepts
 - Validates state (CSRF)
 - Exchanges `auth_code` for `access_token` + `refresh_token` via Google's token endpoint (server-to-server)
 - Calls Google's `userinfo` endpoint with `access_token`
 - Gets user's email, name, picture
 - Calls our custom `OAuth2UserService.loadUser(userRequest):`
3. `GoogleOAuth2UserService.loadUser(userRequest):`
 - Extract email + name + `googleSubjectId` from OIDC user
 - Find User by email; if not found → create new User (signup via Google, `role=owner`)
 - If found:
 - if `user.authProvider != "google"` → reject ("Email registered with password, please login with password")
 - update `user.picture` from Google (if changed)
 - update `user.googleSubjectId` (for future logins)
 - Build our internal `JwtAuthenticationToken` (NOT Spring's `OAuth2AuthenticationToken`)
 - Return our custom `AuthUser` principal
4. `OAuth2SuccessHandler.onAuthenticationSuccess():`
 - Generate our internal access + refresh JWT
 - Create `UserSession`
 - Audit log: ``google_oauth_login`, `signin``
 - Set cookies, redirect /

Configuration (`application.yml`):

```

spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: openid, email, profile
            redirect-uri: "{baseUrl}/login/oauth2/code/google"
        provider:
          google:
            authorization-uri: https://accounts.google.com/o/oauth2/v2/auth
            token-uri: https://oauth2.googleapis.com/token
            user-info-uri: https://openidconnect.googleapis.com/v1/userinfo
            user-name-attribute: sub

```

Security details: - `state` parameter — Spring generates a random state, stores in HTTP session (or auth request store), validates on callback — CSRF protection - PKCE — Spring 6 added PKCE by default for OAuth2 Login (since Spring Security 5.7) — `code_verifier` + `code_challenge` — protects against authorization code interception - `redirect-uri` must be registered in Google Cloud Console — Spring's `{baseUrl}` resolves to the deployment URL - Scopes: minimum — `openid email profile` — we don't request Gmail or Calendar scopes at login (we request those separately when the user connects Gmail/Calendar integrations, with a clear consent screen) - We store the Google `refresh_token` (encrypted with AES-GCM, key from Secrets Manager) for offline access (Calendar API, Gmail PubSub) — but only if the user explicitly connects those integrations, not at login - We do NOT use the Google access token for our internal API auth — we exchange Google's token for our own JWT immediately. The Google token is only for Google APIs.

Custom `OAuth2UserService`: We override `loadUser()` to (a) find-or-create the User entity, (b) check the `authProvider` (reject if mismatched), (c) build our `AuthUser` principal instead of Spring's default `DefaultOAuth2User`. This lets our `@PreAuthorize` expressions work with our role enum.

Custom `SuccessHandler`: We override `onAuthenticationSuccess()` to issue our JWT + set cookies + redirect. The default Spring handler just redirects to `/` without setting our auth state.

Token refresh: Google's access token expires in 1 hour. We use the stored refresh token (per-user, in `user_integration` table) to get a new access token when needed (Calendar API call, Gmail PubSub). We use `GoogleOAuth2AuthorizedClientService` (Spring's `AuthorizedClientService` + `OAuth2AuthorizedClientRepository`) which auto-refreshes expired tokens.

KEY TERMS TO MENTION: - Spring's `oauth2Login` with `OAuth2AuthorizationRequestRedirectFilter` + `OAuth2LoginAuthenticationFilter` - Authorization Code flow with PKCE (Spring 6 default) - `state` parameter for CSRF - `client-id` + `client-secret` from env (Secrets Manager) - Scopes: `openid email profile`

(minimal — Gmail/Calendar requested separately with consent) - Custom `OAuth2UserService.loadUser()` → find-or-create User, check `authProvider`, build our `AuthUser` - Custom `SuccessHandler` → issue our JWT + cookies (NOT use Google's token for our API) - Google `refresh_token` stored encrypted (AES-GCM) for offline access to Google APIs - `OAuth2AuthorizedClientService` auto-refreshes expired Google access tokens - Audit log: `google_oauth_login`, `signin`

FOLLOW-UP QUESTIONS: 1. How do you prevent Google account takeover (user loses Google account)? 2. How do you handle Google token revocation (user disconnects)? 3. How do you handle multiple Google accounts for one user?

FOLLOW-UP ANSWERS: 1. Two ways: (a) if Google marks the account as deleted/disabled, our next Calendar/Gmail API call returns 401 with `invalid_grant` — we mark the `user_integration` as REVOKED and notify the user to re-connect; (b) we don't rely solely on Google — the user can also set a password + MFA on our platform (we encourage it via the "Security" settings). A user who has both Google OAuth + password can always log in even if Google disables their account. 2. The user can disconnect Google from our settings page → we call Google's `https://oauth2.googleapis.com/revoked?token=<refresh_token>` to revoke on Google's side → we delete the `user_integration` row → the user can't log in with Google anymore (but can still use password/Magic Link if they have a password set). If the user has no other auth method, we force a password-reset flow on next Google login attempt. 3. We support one Google account per user (most users have one work account). Multi-account is on the roadmap — would require `user_integration` to have multiple rows per user with `provider_user_id`, and the user picks which account to use for each integration. The login flow would let the user pick which Google account to log in with (Google's account picker). Not trivial but doable — we'll add it when we have enterprise customers with multiple Google Workspace accounts.

RED FLAGS TO AVOID: - Storing the Google access token in a cookie (it's for our server, not the client) - Not validating the `state` parameter (CSRF vulnerability) - Requesting Gmail/Calendar scopes at login (over-broad consent) - Saying "we use Google for auth, no JWT" (we issue our own JWT for our API)

(The remaining 21 Spring Security questions — JWT implementation, RBAC, endpoint security, CSRF, session management, BCrypt, rate limiting, token refresh, API key auth, brute force, audit logging, MFA TOTP, MethodSecurity, custom AuthenticationProvider, stateless vs stateful, back-channel logout, OAuth2 token refresh, multiple SecurityFilterChain, IP allow/deny, session fixation, device fingerprinting, XSS at security layer — follow the same template. Drill on: SecurityFilterChain ordering (why rate-limit before auth), `OncePerRequestFilter` vs `GenericFilterBean`, JWT vs opaque token trade-offs, how to migrate from session to JWT without logging everyone out, how to test security config, OAuth2 PKCE flow detail, MFA recovery codes storage.)

Category 5, 6, 7
& 8 — Hibernate/
JPA + PostgreSQL
+ Redis/Caching +
REST API

> **SAMPLE
QUESTIONS**

NOTICE: Drill-down variants the interviewer should add: “Show me the entity code”, “What happens when the lazy proxy is serialised?”, “Why REPEATABLE_READ over SERIALIZABLE?”, “How would you migrate this schema without downtime?”, “What’s the cache hit ratio you target and how do you measure it?”, “How do you design the API for a polymorphic resource?”.

Q4-5: What is your JWT implementation — structure, signing, and storage

DIFFICULTY: Hard **CATEGORY TAG:** Spring Security

ANSWER: We issue a compact pair: a short-lived **access token** (15 min, JWT) and a long-lived **refresh token** (30 days, opaque random 256-bit string stored hashed in `user_session`). Access JWT claims: `sub` (user id), `org` (tenant id — every downstream query filters on it), `role`, `iat/exp`, `jti` (unique id), and `kid` (key version for rotation). We deliberately keep claims MINIMAL — no email, no profile data — because JWTs are readable by anyone holding them and tokens end up in logs; identity lookup happens server-side from `sub`. Signing is **HS256 today with a 512-bit key** from the secrets manager, but the code is algorithm-configurable and the `kid` header supports migration to RS256 (useful the day a second service must verify tokens without sharing the secret).

The filter: `JwtAuthenticationFilter` extends `OncePerRequestFilter` sits early in the `SecurityFilterChain`; it extracts the Bearer token, verifies signature + expiry + not-revoked (a Redis denylist keyed by `jti` with TTL = remaining validity — covering the logout/compromise window), builds `UsernamePasswordAuthenticationToken` with authorities, and sets the `SecurityContext`. Failures return 401 via the `AuthenticationEntryPoint`, never stack traces. Storage on the client: access token in memory (React state/module variable) + refresh token in an `HttpOnly, Secure, SameSite=Strict` cookie — that combination means XSS can steal only a 15-min access token, not the 30-day refresh; CSRF stays defended because the refresh cookie is `SameSite=Strict` and refresh requires the CSRF double-submit header.

Revocation is the honest weak point of stateless JWT, and I name it: an access token can't be "deleted", only denied. Our layers: 15-min TTL bounds the damage; the Redis `jti` denylist covers explicit logout/"log out all devices"; the `org / role` claims are re-checked against the DB for admin-critical operations (`@PreAuthorize` at service level reads fresh role), so a stolen token with a stale role can't demote itself into admin actions. Rotation: refresh endpoint rotates the refresh token on every use (old one invalidated, reuse-detection revokes the whole session family — that's the OAuth2 refresh-token rotation pattern, catching token theft when a rotated-out token is replayed).

KEY TERMS TO MENTION: - 15-min access JWT + 30-day opaque refresh token (hashed at rest) - Minimal claims + `kid` for key rotation; HS256 → RS256 migration path - `OncePerRequestFilter` + Redis `jti` denylist for revocation - `HttpOnly/SameSite` cookie for refresh, memory for access - Refresh-token rotation with reuse detection (session-family revoke)

FOLLOW-UP QUESTIONS: 1. Why opaque refresh tokens instead of a second JWT? 2. A user reports a stolen laptop — what can you actually do?

FOLLOW-UP ANSWERS: 1. A refresh JWT would be self-validating for 30 days with no server-side gate except a denylist — the exact property we don't want in a long-lived credential. Opaque tokens force a DB/Redis lookup on every refresh, which is where revocation, device tracking, and reuse-detection live. The lookup is once per 15 min per user — negligible. 2. Revoke all their sessions (session-family revoke by user id), rotate if needed their connected mailboxes' tokens, and the stolen access tokens die within 15 min max. This drill is why sessions carry device metadata — the security page shows devices and a "revoke all" button. The honest limit: any already-issued access token works until TTL — that's the tradeoff we bought with statelessness, priced at 15 minutes.

RED FLAGS TO AVOID: - Storing JWTs in `localStorage` with long TTL (XSS jackpot) - PII inside JWT claims - "JWT is stateless so logout is impossible" with no denylist/mitigation story

Q4-6: How did you implement role-based access control (RBAC)

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: Three dimensions: **role** (platform-level: USER, ADMIN, OWNER), **permission** (action-level: `lead:delete`, `billing:manage`), and **tenant scope** (org_id — the dimension people forget). Roles map to permission sets in a DB table loaded into the `SecurityContext` at authentication (not hard-coded if-else), so granting “MANAGER” a new permission is data, not a deploy. Enforcement is layered: URL-level rules in the `SecurityFilterChain` (coarse: `/api/admin/**` needs ADMIN), and method-level `@PreAuthorize("hasAuthority('lead:delete')")` on service methods (fine-grained, and on the SERVICE not the controller — internal callers can’t bypass). SpEL expressions compose checks:

```
@PreAuthorize("hasAuthority('lead:edit') and @leadSecurity.canEdit(#leadId, authentication)")
```

where `leadSecurity` is a bean doing the tenant-scope check (lead belongs to caller’s org) — this is the piece that prevents IDOR, the #1 API vuln.

The tenant-scope check is also enforced at the **repository layer** as defense in depth: every lead/deal query method takes `orgId` and the JPA layer appends it (Specifications always `and(orgId eq :orgId)`); an integration test asserts that org-A’s authenticated context physically cannot fetch org-B’s lead even by guessing UUIDs — we have a dedicated “cross-tenant” test suite that iterates every GET-by-id endpoint with a foreign id and asserts 404 (404, not 403 — don’t confirm existence). Ownership vs role: an OWNER can manage the org’s users; a USER can only see their own leads unless the org has “shared pipeline” enabled — that’s ABAC (attribute-based) flavor on top of RBAC, implemented as policy methods in `leadSecurity` rather than more roles, because permission matrices explode combinatorially when you add roles per attribute.

KEY TERMS TO MENTION: - Role → permission mapping in data, loaded to `SecurityContext` - `@PreAuthorize` at service layer with SpEL + policy beans - Tenant scope as the third RBAC dimension (IDOR prevention) - Repository-level org filter as defense in depth + cross-tenant test suite - 404 vs 403 choice for foreign resources

FOLLOW-UP QUESTIONS: 1. Why 404 for a foreign lead instead of 403? 2. How do you test permission matrices without combinatorial explosion?

FOLLOW-UP ANSWERS: 1. 403 confirms the resource exists (“you may not see lead X” vs “no lead X”) — an enumeration oracle for a competitor or ex-employee probing ids. 404 answers identically for missing AND unauthorized, leaking only what a failed guess would leak anyway. Exception: when the user already knows the resource exists (they can see it in a shared list), 403 is fine and more debuggable. 2. The matrix is data, so tests are generated: for each (role × endpoint) pair, a parameterized test asserts expected status from the same permission table the runtime uses — one test class covers ~300 combinations. The trick is the expected values come from the same source of truth as enforcement, so the test verifies wiring, not policy; policy changes update the table and the test expectations together.

RED FLAGS TO AVOID: - RBAC checks only in the frontend (UI hiding ≠ security) - `@PreAuthorize` only on controllers (bypassable internally) - Forgetting tenant scope — role checks alone don’t stop cross-org leaks

Q4-7: Walk me through your SecurityFilterChain — how are API endpoints actually secured

DIFFICULTY: Hard **CATEGORY TAG:** Spring Security

ANSWER: The chain (Spring Security 6, component style) in order: (1) `.cors(...)` with the allow-list source — preflights resolve before auth; (2) security context setup (stateless — `SessionCreationPolicy.STATELESS`, we never create HTTP sessions for API calls); (3) **permit-all**

paths: `/api/auth/otp/request`, `/otp/verify`, `/magic-link/*`, `/auth/google/*`, `/actuator/health` (internal port), Stripe's `/api/stripe/webhook` — each protected by its OWN mechanism (rate limits, HMAC signatures) instead of session auth, which is the key insight: “permitAll” means “no JWT required”, not “no security”; (4) our `JwtAuthenticationFilter` (OncePerRequestFilter) — parses Bearer, verifies, loads authorities, sets context, and populates MDC with `userId/orgId` for the whole chain's logging; (5) authorization rules — anything else `authenticated()`; (6) exception handling wiring: `AuthenticationEntryPoint` → clean 401 JSON, `AccessDeniedHandler` → 403 JSON (never the default HTML error pages).

Below the filter chain, three more gates a request passes: the `RateLimitFilter` (Redis token bucket keyed by user/org/IP with per-route policies — auth endpoints stricter), the controller's `@Valid` boundary, and method security `@PreAuthorize`. I emphasize the “authn vs authz vs policy” separation because interviews probe it: authentication (who are you) = JWT filter; authorization (what can you do) = `PreAuthorize` + URL rules; policy (business conditions — “is this lead in YOUR org”, “is the trial expired”) = policy beans called from SpEL or service code. CSRF: disabled on the JWT-bearing API paths (no cookie auth there — CSRF is irrelevant without ambient credentials) but the auth endpoints that DO use cookies (refresh, logout) keep double-submit CSRF tokens — blanket-disabling CSRF and forgetting cookie endpoints is the classic hole.

Testing the chain: `MockMvc` tests per security property — unauthenticated → 401, wrong role → 403, cross-tenant → 404, webhook without valid signature → 400; plus one “chain smoke” test asserting permit-all paths are exactly the intended list (a regex slip that accidentally opens `/api/admin/**` fails CI — this actually caught a typo once: a trailing slash made an admin route match the public pattern).

KEY TERMS TO MENTION: - Filter order: CORS → stateless context → `permitAll`(+own defenses) → JWT filter → authorize - `permitAll` ≠ unprotected (rate limits, HMAC per public path) - `AuthenticationEntryPoint` / `AccessDeniedHandler` for clean JSON errors - CSRF disabled only where no ambient credentials; kept on cookie endpoints - Chain smoke test pinning the public-path list

FOLLOW-UP QUESTIONS: 1. Why stateless sessions — what did you lose? 2. Where does the rate limiter sit and why there?

FOLLOW-UP ANSWERS: 1. Lost: instant server-side revocation and session convenience. Gained: horizontal scaling with no sticky sessions or session store on the hot path (Redis only for denylist checks). The trade is deliberate: revocation is re-added at the token layer (denylist + rotation) rather than paying sticky-session scaling tax. 2. After the JWT filter — so limits key on identity when available (user/org) falling back to IP for anonymous, and

rejected requests don't consume downstream work. Auth endpoints additionally have an anonymous pre-filter (stricter IP bucket) because they're the brute-force target and identity doesn't exist yet.

RED FLAGS TO AVOID: - permitAll on auth/webhooks with no compensating controls - Disabling CSRF globally without understanding cookie paths - HTML default error pages leaking stack traces

Q4-8: How did you prevent CSRF attacks

DIFFICULTY: Easy **CATEGORY TAG:** Spring Security

ANSWER: First, the precise mental model: CSRF exploits **ambient credentials** — cookies the browser attaches automatically — to make a victim's browser send a forged state-changing request. So the defense decision tree starts with "does this endpoint authenticate by cookie?" Our JWT APIs (Bearer header) have zero ambient credentials: a malicious site CANNOT set an Authorization header on a cross-origin request (headers are non-ambient), so CSRF is structurally impossible there and `csrf().disable()` is correct — disabling it everywhere else would be cargo cult, but disabling it on cookie endpoints is a hole. The cookie-authenticated endpoints (refresh-token cookie, logout, the marketing-site session) use **SameSite=Strict cookies** (browser won't attach them on cross-site requests — kills CSRF for modern browsers) PLUS **double-submit CSRF tokens** (a `csrf` cookie readable by our origin's JS + `X-CSRF-Token` header that must match; the attacker's cross-site form can't read our cookie to set the header) as defense for older browsers and non-browser clients (native apps replay the header).

Beyond tokens, request-design hygiene shrinks the attack surface: all state changes are POST/PUT/PATCH/DELETE with JSON bodies (a plain `<form>` GET forgery can't set JSON content-type without a preflight, and preflights fail cross-origin without CORS approval); no state changes via GET (even "resend email" links are POSTs from the app); and CORS allow-list is strict so even a forged XHR from an evil origin is blocked by the browser before hitting us. Defense-in-depth note for interviews: CSRF tokens are compared with constant-time equality, rotate on login (session-fixation adjacent), and the auth endpoints are additionally rate-limited — CSRF protection layered with rate limiting is what actually stops scripted abuse at scale.

KEY TERMS TO MENTION: - CSRF = ambient credentials (cookies), not header-based auth - SameSite=Strict as the primary modern defense - Double-submit token pattern for cookie endpoints - JSON-body-only mutations + strict CORS as structural defenses - Token rotation on login + constant-time comparison

FOLLOW-UP QUESTIONS: 1. CSRF vs XSS — how are they different and how do they interact? 2. Why not SameSite=None ever?

FOLLOW-UP ANSWERS: 1. CSRF makes the victim's browser send requests AS the victim without the attacker seeing responses; XSS runs the attacker's code IN your origin and can read everything CSRF can only blindly request. They chain: an XSS hole defeats CSRF tokens

(the script reads the token cookie) — which is why token storage choices (HttpOnly where possible) and output encoding are the first line, and CSRF is the second. 2. SameSite=None (attach cookies cross-site) is only for legitimate embedded cross-site usage (payment iframes) and then REQUIRES Secure + the token defenses; we never needed it — our app is first-party, so Strict costs nothing and closes the whole class.

RED FLAGS TO AVOID: - “We disabled CSRF” without the Bearer-header reasoning - Confusing CSRF (forge requests) with XSS (run code) - Relying on a single layer (no SameSite, no CORS discipline)

Q4-9: How did you implement session management for a stateless API

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: “Session management” in our stack means **managing the refresh-token lifecycle**, since access tokens are 15-minute self-contained JWTs. The `user_session` table is the session registry: `id`, `user_id`, `refresh_token_hash` (SHA-256 — DB leak can’t mint sessions), device fingerprint (UA + IP hash), `created_at`, `last_used_at`, `expires_at`, `revoked_at`, and a `family_id` grouping rotated tokens from one login. Sessions are first-class user-facing objects — the security page lists devices (“MacBook · Mumbai · 2h ago”) with per-session and “revoke all” buttons, because session management is a product feature, not just plumbing.

Mechanics: refresh endpoint validates the presented token against the registry (hash lookup), checks revocation and expiry, then **rotates**: new refresh token issued, old marked revoked, both sharing `family_id`. Reuse detection — a revoked token being presented again — signals theft (the legitimate client would use the NEW token); response: revoke the ENTIRE family + security alert email (“new sign-in required everywhere”). Concurrency nuance: legitimate double-tab refresh races exist, so a just-rotated token gets a 30-second grace window (revoked-but-accepted, returns the same new token) — without it, every user with two tabs logs out randomly; with it, real theft (minutes later) still trips rotation. Cleanup: a daily job purges expired/revoked sessions older than 30 days, keeping the table lean and the active-session list meaningful.

Server-side, “stateless” refers to request handling: no HTTP sessions, no sticky load balancing — every request carries its identity. The session REGISTRY is state, but it’s consulted only on refresh (once per 15 min), not per request — that’s the balance: statelessness where it’s hot (per-request), state where control matters (lifecycle).

KEY TERMS TO MENTION: - Session registry = refresh-token lifecycle table (hashed tokens) - Device fingerprinting + user-facing device list (product feature) - Rotation with `family_id` + reuse detection → family revoke - 30s grace window for legitimate double-tab races - Registry consulted on refresh only — per-request stays stateless

FOLLOW-UP QUESTIONS: 1. Why hash refresh tokens — they’re already random? 2. How do you cap “active sessions per user”?

FOLLOW-UP ANSWERS: 1. Defense in depth: a DB dump with raw tokens is a session farm — every row is a working login. Hashed, the dump is inert. Randomness protects against guessing; hashing protects against extraction. Same reason we hash OTPs and magic-link tokens. 2. Soft cap: > 10 concurrent sessions triggers a step-up (re-auth + email alert) rather than hard-revoke, because power users with multiple devices are legitimate. The alert is the detection mechanism — session count anomalies (50 sessions in an hour) feed the account-takeover detector, which CAN hard-revoke.

RED FLAGS TO AVOID: - Raw refresh tokens in DB - No rotation/reuse detection (30-day bearer credentials) - Saying “stateless” and then having no answer for logout/theft

Q4-10: How did you handle password hashing (BCrypt) when the primary auth is OTP

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: Passwords are the FALLBACK credential (users who skip Google OAuth), so they exist but are deliberately boring — the interview value is in explaining the parameters and why. Hashing: **BCrypt** (`PasswordEncoder` = `BCryptPasswordEncoder`, strength 12) — cost 12 ≈ 250ms/hash on our hardware: slow enough to make offline cracking expensive, fast enough to not DoS our own login endpoint; BCrypt over SHA-256 because it’s *designed* slow + salt built-in (unique per hash, stored inline) so rainbow tables are dead on arrival. The upgrade path question I preempt: why not Argon2id — it’s the modern winner (memory-hard); our answer is migration-readiness: hashes are stored with algorithm prefix (`$2a$12$...`) so a future `DelegatingPasswordEncoder` re-hashes transparently on next successful login — the same mechanism that would let us adopt Argon2id without a “reset all passwords” event.

Around the hash, the standard hygiene: password policy via zxcvbn-style strength scoring (length-first, no composition rules — NIST 800-63B guidance), breach-list screening (k-anonymity API check at signup/change), no maxlength paranoia beyond 72 bytes (BCrypt’s input limit — we hash a pre-hash for longer inputs), constant-time comparisons everywhere (BCrypt’s verify is), login rate limiting + progressive lockout shared with the OTP path, and password reset via signed single-use token (same machinery as magic links, 15-min TTL, invalidated on use + revokes all sessions on completion — a password reset IS a session-kill event). OTP credentials get the same respect: OTPs are 6 digits but random per request, stored hashed with expiry, attempts counted — because a 6-digit OTP is weaker than a good password (10^6 space) and the lockout (5 tries → 15 min) is what makes it acceptable; I explain that pairing honestly in interviews — the math is the answer, not the brand of hashing.

KEY TERMS TO MENTION: - BCrypt strength 12 + unique salts (why slow hashing matters) - Algorithm-prefixed hashes → `DelegatingPasswordEncoder` upgrade path (Argon2id-ready) - NIST-aligned policy (length over composition), breach screening - Reset = single-use signed token + session revocation - OTP brute-force math: 10^6 space → lockout makes it safe

FOLLOW-UP QUESTIONS: 1. Why 250ms per hash isn’t a DoS vector on login? 2. Where do you store the pepper, if anywhere?

FOLLOW-UP ANSWERS: 1. The 250ms cost is bounded by pre-hash rate limiting (5 attempts/15min/account, IP buckets) — an attacker gets ~20 hashes/min/account, not thousands. Without rate limiting ANY hashing scheme is abusable; with it, cost-per-hash is armor for the DB-leak scenario (offline), which is where bcrypt earns its keep. 2. We don't pepper today; BCrypt's per-hash salt + 12 rounds covers the modeled threat (DB leak + offline cracking). A pepper (HSM/KMS-held secret pre-hashed with the password) is the next rung if we needed DB-leak defense WITHOUT rate-limit assumptions — I can describe the design (HMAC-SHA256(password, pepper) → bcrypt) and its operational cost (rotation = all hashes invalid), which is exactly why it's not free and we deferred it.

RED FLAGS TO AVOID: - MD5/SHA-256 "with salt" for passwords (fast = crackable) - Not knowing your own cost factor or its latency consequence - Password policy straight from 2005 (uppercase+symbol rules)

Q4-11: How did you implement rate limiting for auth endpoints

DIFFICULTY: Hard **CATEGORY TAG:** Spring Security / Redis

ANSWER: Auth endpoints are the brute-force front door, so they get a **layered limiter**, each layer keyed differently because each answers a different attack: **Layer 1 — per-IP token bucket** (Redis, **INCR** + **EXPIRE** sliding window): 10 OTP requests / 10 min / IP — stops a single host spraying; behind the ALB the key uses the right-most trusted X-Forwarded-For entry (parsed from a known-proxy chain, never client-supplied blindly — the header-spoofing classic). **Layer 2 — per-account lockout:** 5 failed verifications per phone/email → account locked 15 min with exponential growth on repeat (15 → 60 min); counters in Redis (`otp:attempts:{emailHash}`) with atomic **INCR**-then-check, so concurrent attempts can't race past the cap; this is the layer that makes 6-digit OTPs mathematically safe ($10^6 / 5$ tries per quarter-hour ≈ centuries for meaningful success probability). **Layer 3 — global anomaly guard:** org-level and platform-level counters that page us when aggregate OTP failure rates spike (a distributed attack from many IPs doesn't trip per-IP or per-account layers — only the aggregate sees it).

Failure semantics matter as much as the numbers: limit responses are generic 429 with **Retry-After** — never "this account is locked" (account-existence oracle) — and the per-account lock applies to the VERIFICATION step, while the request step stays available (a locked account that can't even request an OTP is a DoS handle against a victim's login). Every rejection increments metrics tagged by layer+route; the Grafana auth dashboard is how we noticed a scraper hammering the magic-link endpoint within 20 minutes of its launch. Implementation: a **RateLimitFilter** before the controller (so rejected requests cost nothing) + a small **RateLimiter** service (Redis Lua script for atomic bucket ops — no Lua, no atomicity, race conditions between INCR and EXPIRE).

KEY TERMS TO MENTION: - Layered keys: per-IP bucket, per-account lockout, global anomaly guard - Correct X-Forwarded-For parsing behind ALB (spoof defense) - Atomic Redis ops (Lua) — INCR/EXPIRE race - Generic 429s (no account-existence oracles) - Verification lockout ≠ request lockout (victim-DoS avoidance)

FOLLOW-UP QUESTIONS: 1. Why not rate limit in the load balancer only? 2. Distributed brute force across 10,000 IPs — what defends?

FOLLOW-UP ANSWERS: 1. The LB sees IPs and paths, not identities or business semantics — it can't do per-account lockout, can't distinguish OTP-request from OTP-verify cost, and its counters are blunt during incidents (changing LB config mid-attack is slow; our Redis-backed thresholds are tunable live). LB-level (WAF/edge) limits are still useful as the coarse outer net — defense in depth, different granularity. 2. Layer 3 (aggregate anomaly) detects it and trips platform-wide: raise OTP length temporarily, tighten global OTP-request caps, enable CAPTCHA step-up on request endpoints, and page on-call with the attack fingerprint (which orgs/targets). The per-account lockout still caps per-victim damage at 5 guesses/quarter-hour — the math holds even under 10k IPs; what degrades is availability, so the guard's job is protecting the service while the math protects the accounts.

RED FLAGS TO AVOID: - One naive per-IP counter (spoofable XFF, shared-IP collateral) - Lockout messages that leak account existence - Non-atomic Redis increment logic (race = bypass)

Q4-12: How did you handle token refresh and expiry strategy

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: The strategy optimizes for three goals in priority: security (short access life), UX (no random logouts), and simplicity of revocation. **Access tokens: 15 minutes.** At 15 min, a stolen token's value window is bounded, and the renewal cost is invisible — the client refreshes proactively at ~12.5 min (80% of TTL) BEFORE expiry via an interceptor, so no request ever waits on a refresh round-trip. Concurrent-request handling is the subtle part interviewers probe: when the token expires mid-flight with 4 parallel API calls, ALL of them would 401 and ALL would try to refresh — thundering herd on the refresh endpoint and, worse, refresh rotation would revoke the token 3 times (reuse-detection firing on the user's own tabs). Solution: client-side single-flight refresh — a shared promise; the first 401 triggers ONE refresh, other callers await the same promise and retry with the new token; server-side the 30-second rotation grace window absorbs any remaining races.

Refresh tokens: 30 days sliding. Each use rotates (covered in Q4-9); `last_used_at` extends nothing — the 30-day expiry is absolute from login, and sliding would mean an active attacker never expires; instead we expire absolute + re-auth on risk signals (new device + new geography + sensitive action → step-up OTP). Backend endpoints: `/auth/refresh` validates the registry entry (hash, revocation, expiry), rotates, returns new pair; failures are 401 with a machine-readable code (`REFRESH_EXPIRED` vs `REFRESH_REUSED`) so the client knows whether to redirect to login silently or show “signed out for your security”. Scheduled cleanup + metrics (refresh success rate, rotation-reuse detections — a spike IS an attack signal and alerts) complete the loop. Clock-skew tolerance: verification allows ± 30 s leeway — a lesson from a deploy where the auth server's clock drifted and every token in the fleet expired 40 seconds “early”, an incident I use to explain why skew params exist.

KEY TERMS TO MENTION: - 15-min access + proactive client refresh at 80% TTL - Single-flight refresh promise (parallel 401 thundering herd) - Rotation + grace window interplay (double-tab vs theft detection) - Absolute (not sliding) refresh expiry + risk-based step-up - Machine-readable 401 codes (silent redirect vs user-visible logout)

FOLLOW-UP QUESTIONS: 1. Why not refresh-on-401 only, without proactive refresh? 2. How would you shrink access TTL to 5 min without UX damage?

FOLLOW-UP ANSWERS: 1. Reactive-only means the first request of every expiry window pays a serial round-trip (401 → refresh → retry), and parallel callers race — we measured +180ms p95 spikes at expiry boundaries. Proactive refresh moves the cost off the critical path; 401-handling stays as the safety net, not the mechanism. 2. Mechanically trivial (config + TTL math), UX-wise it multiplies refresh traffic 3× and makes the client refresh logic hot — with single-flight that’s fine, but the real cost is server-side: refresh endpoint QPS triples. The better trade at that point is moving to RS256 + a dedicated auth service where 5-min tokens are cheap, i.e., changing the architecture when the requirement, not tweaking numbers hoping UX holds.

RED FLAGS TO AVOID: - Long-lived access tokens “for simplicity” (30-min+ JWTs) - No answer for parallel-refresh races - Sliding refresh expiry (immortal sessions for active attackers)

Q4-13: What is your approach to API key authentication (service-to-service)

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: API keys exist in our system for two actors: partner integrations (a customer’s backend calling our public API on behalf of their org) and internal cron workers. Design: keys are **random 32-byte secrets, shown once** at creation (`aq_live_<24 chars>` visible prefix for identification), stored **SHA-256 hashed** with a metadata record (org, scopes, created_by, last_used_at, expires_at, rate-tier). Authentication: `X-API-Key` header → `ApiKeyAuthenticationFilter` hashes the presented key and does a constant-time lookup (hash index) → loads the key’s authorities (scopes like `leads:read`) into the SecurityContext — so downstream `@PreAuthorize` works identically for keys and users, one authorization model. Lookup-by-hash means the DB never stores a usable secret; a dump can’t call us.

Hygiene rules that interviews want: **scopes are least-privilege** (a reporting integration gets `leads:read`, never write), **expiry is mandatory** (90-day default with renewal reminders — eternal keys are how breaches age), **rotation without downtime** (create second key → migrate → revoke first; both valid during the window), and **revocation is instant** (filter checks a `revoked_at` + a Redis-cached status with 30s TTL — revocation propagates in ≤ 30s, documented trade-off). Every key call logs org+key-id in MDC (audit trails attribute actions to the key, not just the org), keys carry their own rate-limit tier, and usage dashboards per key are a product feature — partners can see their own traffic. What I explicitly did NOT build: home-rolled HMAC request signing — that’s the right next step for high-security partners (SigV4-style), but key+TLS+scopes+rotation covers our current threat

model, and I can explain exactly when I'd upgrade (non-repudiation needs, replay-sensitive endpoints).

KEY TERMS TO MENTION: - Show-once keys + SHA-256 hash storage + visible prefix for identification - Scopes → same authorities model as users (one authz path) - Mandatory expiry + zero-downtime rotation - Constant-time lookup + Redis-cached revocation ($\leq 30s$) - Per-key MDC attribution, rate tiers, usage dashboards

FOLLOW-UP QUESTIONS: 1. Why hash API keys but not, say, encrypt them? 2. Key in a URL query param — ever acceptable?

FOLLOW-UP ANSWERS: 1. Encryption implies we NEED to recover the secret — we never do; verification is hash-compare. Hashing means even we can't leak what we don't store. Encrypting would put a decryption key one config mistake away from turning our DB into a key farm. 2. No — URLs land in access logs, browser history, proxy logs, referrer headers; the key becomes log-file archaeology for anyone with log access. Header-only, TLS-only. If a partner can't set headers, they shouldn't be integrating via API keys at all.

RED FLAGS TO AVOID: - Storing raw keys (or "encrypted" with the key in the same DB) - Eternal keys with no expiry/rotation story - Different authorization logic for keys vs users (drift bugs)

Q4-14: How did you prevent brute force attacks beyond rate limiting

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: Rate limiting caps attempts; brute-force defense is the full toolkit around it.

Entropy first: OTPs are 6 random digits with no sequential reuse (a new OTP invalidates the old — no accumulation of knowledge across requests), magic-link tokens are 256-bit random with 10-minute TTL and single-use, passwords are breach-screened — you can't brute-force 128 bits of entropy with 5 guesses per 15 minutes; the math IS the defense, and I show it: 5 attempts/quarter-hour $\times$ 4 quarters $\times$ 24 h $\times$ 15-min lockout doubling on repeat $\Rightarrow$ expected time to hit a 6-digit OTP $\approx 10^6/(20/\text{day}) \approx 137$ years. **Progressive friction:** repeated failures escalate cost per attempt — lockout windows double (15 min $\rightarrow$ 1 h $\rightarrow$ 6 h), then a CAPTCHA/step-up enters the flow, and a device fingerprint (fingerprint hash of UA/platform/screen — soft signal, never sole factor) entering fresh on a known account triggers email alert + stricter limits.

Detection as the second line: every auth event writes to the audit stream (account, ip, device, result, latency); a streaming rule set flags (a) distributed failures against one account across IPs (per-IP limits blind to this), (b) one IP failing across MANY accounts (credential-stuffing shape), (c) success-after-failure-spike (the actual break-in signal). Rule hits raise risk score on the account: next logins require OTP step-up regardless of method, sessions require re-auth for sensitive ops, and at threshold the account is temporarily locked with owner notification — the alert matters as much as the block (a real user locked out without warning is a support ticket; warned, it's trust). **Credential stuffing specifically** (leaked password

lists): breach-screening at login (optional, k-anonymity) + step-up OTP on new-device logins means a valid leaked password alone doesn't yield access — second factor is the design answer, and the reason OTP-first auth is genuinely stronger than it sounds.

KEY TERMS TO MENTION: - Entropy math as the primary defense (show the numbers) - Progressive friction: doubling lockouts → CAPTCHA → step-up - OTP invalidation on re-request (no knowledge accumulation) - Streaming detection rules (distributed + stuffing shapes) → risk scoring - Success-after-failure-spike = the break-in signal that alerts

FOLLOW-UP QUESTIONS: 1. An attacker with a botnet of residential IPs — what still works in your defense? 2. Why alert the user on lockout — doesn't that tip off attackers?

FOLLOW-UP ANSWERS: 1. IP diversity defeats per-IP limits but not per-account limits (still 5 guesses/quarter-hour on the victim), not entropy (the math is IP-independent), not device-fingerprint shifts (botnet browsers rotate fingerprints predictably — fingerprint instability itself is a risk signal), and not the aggregate anomaly guard — residential botnets are exactly the shape the stuffing rule catches. Defense that doesn't depend on IP identity is what survives botnets. 2. The attacker already knows the account exists (they're attacking it). The alert's value to the defender (user changes password, reports unknown attempts) outweighs the tip-off cost; and the message is generic ("unusual sign-in attempts") — it never confirms whether attempts succeeded or which method was targeted.

RED FLAGS TO AVOID: - "Rate limiting" as the entire answer (IP rotation defeats it) - No detection layer — blocks without visibility into attack shapes - Tipping off via specific error messages (method/target leaks)

Q4-15: How did you implement audit logging for security events

DIFFICULTY: Medium **CATEGORY TAG:** Spring Security

ANSWER: The audit log answers "who did what, from where, when" with tamper-evidence — a different animal from application logs (debuggability). Implementation: an `AuditAspect` (Spring AOP, `@Around` on `@Audited`-annotated service methods) plus explicit `AuditService.event(...)` calls for non-method events (auth results, webhook receipts). Every record: actor (user id / api-key id / "system"), actor org, action (verb-noun enum: `LEAD.DELETE`, `AUTH.OTP_FAIL`), resource type + id, before/after diff for updates (JSON of changed fields only), ip, user-agent, request id (correlates with traces), timestamp (UTC), and outcome. Storage: append-only `audit_event` table (no UPDATE grants for the app role — the DB user physically cannot mutate history), monthly partitions, 400-day hot retention then archived to encrypted S3 (compliance-retained). Integrity: nightly Merkle-style chained hash (each record hashes the previous) persisted to a separate store — a deleted/edited row breaks the chain and the verification job alerts; not full blockchain theater, but enough to prove post-hoc tampering for our compliance needs.

Coverage decisions: auth events ALL logged (success AND failure — failures are the attack signal), permission denials logged, admin actions logged with before/after, data exports logged (GDPR subject-access needs the "what left" record), but high-frequency reads NOT

logged (noise kills signal; the access-control story covers reads). Query-ability: the admin UI filters by actor/org/action/time — support lives in this tool (“who changed this deal price?” is a 5-second lookup). GDPR interplay: audit rows reference user ids, not PII payloads; the before/after diffs exclude secrets (passwords/OTPs are never diffed — masked by the aspect). Testing: an aspect unit test (annotation → record shape) + a coverage test asserting every service method in the `admin` package carries `@Audited` — coverage of security logging enforced in CI, because the failure mode of audit logging is silent gaps, not bugs.

KEY TERMS TO MENTION: - `@Audited` AOP aspect + explicit events for non-method flows - Append-only table, app role lacks UPDATE grant (DB-enforced) - Before/after field-level diffs, secrets masked - Chained-hash tamper evidence + S3 archive (retention) - Failure events logged (auth failures = attack signals); CI coverage test

FOLLOW-UP QUESTIONS: 1. Why not log everything — what’s the cost of over-logging? 2. How does audit logging interact with GDPR deletion requests?

FOLLOW-UP ANSWERS: 1. Three costs: signal dilution (10M rows where 100 matter = no forensics), storage/latency (synchronous audit writes on hot paths — ours are async AFTER_COMMIT with a bounded queue and drop-oldest policy for non-critical events, CRITICAL events block on write), and compliance exposure (logging PII creates its own GDPR surface). Audit logs are a curated record, not a backup of reality. 2. We log references, not payloads — an erasure request removes the user’s PII from production tables while audit rows keep only the immutable id + action history (which is the legitimate-interest record: proving what the SYSTEM did requires the actor id remain). Diffs that embedded personal data are purged by the retention job on erasure; the chain hash records the deletion itself, auditable, not silent.

RED FLAGS TO AVOID: - Audit logs without tamper protection (anyone with DB access edits history) - Logging secrets into diffs (audit trail becomes the leak) - Synchronous audit writes stalling hot paths with no policy

CATEGORY 5 — HIBERNATE AND JPA QUESTIONS

Q5-1: Explain your entity design for the lead management system

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: The `Lead` entity is the core of the lead management system. It's mapped to the `lead` table, has UUID PK, multi-tenant via `orgId`, soft-deleted via `@WhereJoinTable`, and uses JSONB for tags.

```

@Entity
@Table(name = "lead", indexes = {
    @Index(name = "idx_lead_user_stage", columnList = "user_id,stage"),
    @Index(name = "idx_lead_user_created", columnList = "user_id,created_at")
})
@SQLDelete(sql = "UPDATE lead SET deleted_at = CURRENT_TIMESTAMP WHERE id = ?") //
    soft delete on `repository.delete()`
@Where(clause = "deleted_at IS
    NULL") // automatically filter soft-deleted rows from queries
public class Lead extends BaseEntity { // BaseEntity has id, createdAt, updatedAt
    (see Q5-18)

    @Column(name = "business_name", nullable = false, length = 200)
    private String businessName;

    @Column(name = "owner_name", length = 150)
    private String ownerName;

    @Column(length = 254)
    private String email;

    @Column(length = 30)
    private String phone;

    @Column(length = 2048)
    private String website;

    @Column(length = 100)
    private String niche;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private LeadStage stage = LeadStage.DISCOVERED;

    @Column(name = "reply_score")
    private Short replyScore; // 0-100, SMALLINT in DB

    @Column(name = "conversion_score")
    private Short conversionScore;

    @Column(name = "urgency_score")
    private Short urgencyScore;

    @Column(name = "has_website")
    private boolean hasWebsite;

    @Type(type = "jsonb") // Hypersistence Utils (formerly Hibernate Types)
    @Column(columnDefinition = "jsonb", nullable = false)
    private List<String> tags = new ArrayList<>(); // JSONB array

    @Column(name = "is_active", nullable = false)
    private boolean active = true;

    @Column(name = "deleted_at")
    private Instant deletedAt; // soft delete marker

```

```

    @ManyToOne(fetch = FetchType.LAZY) // avoid N+1: fetch via @EntityGraph or join
        fetch
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "org_id", nullable = false)
    private Organization organization;

    @OneToMany(mappedBy = "lead", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("createdAt DESC")
    private List<LeadActivity> activities = new ArrayList<>();

    @OneToMany(mappedBy = "lead", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("createdAt DESC")
    private List<LeadCommunication> communications = new ArrayList<>();

    @OneToMany(mappedBy = "lead")
    @OrderBy("scoredAt DESC")
    private List<LeadScore> scores = new ArrayList<>();

    @OneToOne(mappedBy = "lead", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private LeadAnalysis analysis;

    @Version
    private Long version; // optimistic locking

    // equals/hashCode based on id (see Q5-20)
}

```

Design decisions: 1. **UUID PK** — globally unique, no enumeration risk, supports future sharding 2. **@Where("deleted_at IS NULL")** — soft delete auto-filter; `repository.delete()` triggers `@SQLDelete` (updates `deleted_at` instead of `DELETE`) 3. **@Version** — optimistic lock for concurrent updates 4. **FetchType.LAZY** on all **@ManyToOne** and **@OneToMany** — avoid loading related entities unless explicitly requested. We control N+1 via `@EntityGraph` or JPQL `JOIN FETCH` 5. **CascadeType.ALL** + **orphanRemoval = true** on `activities` and `communications` — deleting a lead deletes its history. We use this sparingly (only for genuinely owned relationships). 6. **JSONB for tags** via Hypersistence Utils (`@Type(type = "jsonb")`) — Postgres-native, GIN-indexable 7. **@OrderBy** — applies at the JPQL/HQL level (not DB-level); for DB-level ordering we add `@OrderColumn` (we don't — we always sort explicitly in queries) 8. **Short for scores** — 0-100 fits in `SMALLINT` (2 bytes); saves space vs `Integer` (4 bytes) at 10M leads scale 9. **No @OneToMany for deals** — we fetch deals via `DealRepository.findById(leadId)` rather than `lead.getDeals()` to keep the entity lean

Why BaseEntity with @MappedSuperclass: shared fields (`id`, `createdAt`, `updatedAt`, `version`) live in a base class; every entity extends it. DRY, consistent.

KEY TERMS TO MENTION: - `@Entity`, `@Table`, `@Column(nullable, length)`, `@Index` - `@SQLDelete` + `@Where` for soft delete - `@Version` for optimistic lock - `FetchType.LAZY` everywhere + `@EntityGraph` / `JOIN FETCH` for N+1 control - `@Type(type = "jsonb")` (Hypersistence Utils) for JSONB columns - `CascadeType.ALL` + `orphanRemoval = true` for

owned relationships - `@OrderBy` (JPQL-level) vs `@OrderColumn` (DB-level) - `Short` for SMALLINT (saves space at scale) - `@MappedSuperclass BaseEntity` for shared fields

FOLLOW-UP QUESTIONS: 1. Why `@Where` instead of `@SQLRestriction` (Hibernate 6.3+)? 2. How do you handle a soft-deleted lead referenced by a foreign key? 3. Why no `@OneToMany` for deals?

FOLLOW-UP ANSWERS: 1. `@SQLRestriction` is the JPA-standardised replacement for Hibernate's `@Where` (added in Hibernate 6.3 / JPA 3.2). We're on Hibernate 6.4 — we could migrate. The migration is mechanical (`@Where(clause = "...")` → `@SQLRestriction(sql = "...")`). We'll do it in the next cleanup sprint — `@Where` is deprecated. The behaviour is identical. 2. We don't use FK constraints to soft-deleted entities. The `lead_activity.lead_id` is a non-FK column (no `REFERENCES lead(id)` constraint) — soft-deleted leads' activities stay in the DB, queryable for audit, but the FK can't enforce referential integrity for soft deletes. We enforce in app code (`LeadService.findOrFail(leadId)` throws `NotFoundException` if soft-deleted). The trade-off: no DB-level protection against dangling references. We accept this because: (a) the app always queries via the service, never directly; (b) our daily reconciliation job checks for orphaned activities and alerts. 3. Three reasons: (a) **leanness** — adding `deals` would force loading on every `lead.getDeals()` call unless LAZY (we always want LAZY by default); (b) **explicit query control** — `DealRepository.findByLeadIdOrderByCreatedAtDesc(leadId, Pageable)` lets us paginate, filter, and sort explicitly; (c) **bidirectional sync** — keeping both sides in sync (`lead.getDeals().add(deal)` + `deal.setLead(lead)`) is a common bug source. We use unidirectional `Deal.lead` and query via repository when needed.

RED FLAGS TO AVOID: - `FetchType.EAGER` on `@OneToMany` (N+1 disaster) - `equals/hashCode` based on all fields (breaks lazy proxies) - No `@Version` (no optimistic locking) - Storing tags as comma-separated string (not queryable)

Q5-2: How did you map relationships between Lead, Deal, Pipeline entities

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: Three main relationships: Lead → Deal (1:N), Lead → PipelineStage (N:1, current stage), Lead → StageTransition (1:N, audit history).

Lead ↔ Deal (1:N, unidirectional from Deal side):


```

@Entity
@Table(name = "deal")
public class Deal extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "lead_id", nullable = false)
    private Lead lead; // FK → lead(id)

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal proposedPrice;

    @Column(precision = 12, scale = 2)
    private BigDecimal finalPrice;

    @Column(nullable = false, length = 3)
    private String currency = "INR";

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private DealStage stage = DealStage.PROPOSAL;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private DealStatus status = DealStatus.OPEN;
}

```

We don't have a `lead.getDeals()` collection — Deal references Lead, but not vice versa. To get a lead's deals: `dealRepository.findByLeadIdOrderByCreatedAtDesc(leadId, Pageable)`.

Lead ↔ StageTransition (1:N, bidirectional for audit queries):

```

@Entity
@Table(name = "stage_transition", indexes = @Index(name =
    "idx_transition_lead_created", columnList = "lead_id,created_at"))
public class StageTransition extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "lead_id", nullable = false)
    private Lead lead;

    @Enumerated(EnumType.STRING)
    @Column(name = "from_stage", nullable = false, length = 20)
    private LeadStage fromStage;

    @Enumerated(EnumType.STRING)
    @Column(name = "to_stage", nullable = false, length = 20)
    private LeadStage toStage;

    @Column(length = 500)
    private String reason;

    @Type(type = "jsonb")
    @Column(columnDefinition = "jsonb")
    private Map<String, Object> metadata;
}

```

Pipeline table — actually not an entity: We don't have a `Pipeline` entity — the “pipeline” is a view (collection of leads grouped by stage). The pipeline is computed: `SELECT stage, COUNT(*) FROM lead WHERE user_id = ? AND is_active GROUP BY stage`. There's no `pipeline` table; the “pipeline view” is a service-level abstraction (`PipelineService.getPipelineView(userId)` returns `Map<LeadStage, List<Lead>>`).

We considered making `Pipeline` an entity (with `pipeline_stage` table, `pipeline_id` on lead, configurable stages per org) — but the stages are fixed (9 stages), and making them configurable per org was over-engineering. We kept stages as an enum and the “pipeline” as a query. If we ever do per-org configurable stages, we'd add a `pipeline_stage` table.

Relationship rules we follow:

- 1. Unidirectional by default** — only make bidirectional if both sides are commonly navigated. `Deal → Lead` is unidirectional (we rarely ask “what deals does this lead have?” without paginating).
- 2. FetchType.LAZY everywhere** — explicit `JOIN FETCH` or `@EntityGraph` for queries that need related data.
- 3. @JoinColumn always** — explicit column name, not Hibernate's auto-generated `lead_id` (which would be the same, but explicit > implicit).
- 4. No CascadeType.REMOVE** — we control deletion explicitly via `@SQLDelete` + service logic. `CascadeType.ALL` only on genuinely owned relationships (Lead → Activities, where deleting a lead should delete its history).
- 5. optional = false on @ManyToOne** — generates a `NOT NULL` FK constraint at the DB level; app-level null check + DB-level constraint = defense in depth.
- 6. @OrderBy only for default ordering** — most queries sort explicitly; `@OrderBy` is the “default if not specified”.
- 7. No @ManyToMany** — we always break it into a join entity (`StudentCourse` instead of `Student.courses`). `@ManyToMany` is hard to add metadata to (e.g. “when was this student enrolled in this course?”).

KEY TERMS TO MENTION: - `@ManyToOne(fetch = LAZY, optional = false)` + `@JoinColumn(name = "lead_id", nullable = false)` - Unidirectional by default, bidirectional only when both sides commonly navigated - `CascadeType.ALL` + `orphanRemoval = true` only for owned relationships - No `CascadeType.REMOVE` — explicit deletion via service - `@OrderBy` for default ordering, explicit `ORDER BY` in queries - No `@ManyToMany` — always break into a join entity (for metadata) - “Pipeline” is a query/view, not an entity (fixed stages enum)

FOLLOW-UP QUESTIONS: 1. Why no `@ManyToMany`? 2. How would you add per-org configurable pipeline stages? 3. What’s the cost of `CascadeType.ALL`?

FOLLOW-UP ANSWERS: 1. Three reasons: (a) `@ManyToMany` is hard to add metadata to — if you later want to add “enrolled_at” to a StudentCourse relationship, you have to break the `@ManyToMany` into a join entity (migration pain); (b) `@ManyToMany` hides the join table, making it hard to query directly (e.g. “find all students enrolled in course X after date Y”); (c) the join table generated by `@ManyToMany` has no PK constraint by default (just two FKs), which can lead to duplicate rows. We always break into a join entity from day one — costs nothing, gains flexibility. 2. Add a `pipeline_stage` table (id, org_id, name, order_index, is_terminal) — replaces the enum for orgs that want custom stages. Migration: (a) add the table with default rows matching the enum for existing orgs; (b) change `lead.stage` from enum to FK to `pipeline_stage(id)`; (c) update all `LeadStage` references in code to look up by `pipeline_stage.name` per org. Backwards-compatible if we keep the enum as a “default stage set” for new orgs. We’d do this when we have enterprise customers with custom sales processes. 3. `CascadeType.ALL` includes `CascadeType.REMOVE` — deleting a parent entity deletes all children. If the cascade depth is deep (Lead → Activities → AuditLogs), one `repository.delete(lead)` could delete 100 rows. We use `CascadeType.PERSIST` + `CascadeType.MERGE` (the safe ones) + `orphanRemoval = true` (controlled removal via `lead.getActivities().remove(activity)`) — never `CascadeType.REMOVE` directly. We control deletion explicitly via service methods.

RED FLAGS TO AVOID: - `@OneToMany(mappedBy = "lead", cascade = CascadeType.REMOVE)` - `@ManyToMany` without thinking - `FetchType.EAGER` on any `@*ToMany` - No `@JoinColumn` name (letting Hibernate guess)

Q5-3: Explain OneToMany, ManyToOne mappings in your project

DIFFICULTY: Easy **CATEGORY TAG:** Hibernate/JPA

ANSWER: `@ManyToOne` is the owning side of a relationship — it has the FK column. `@OneToMany` is the inverse side — `mappedBy` references the owning side’s field name.

In our project: - `Deal.lead` is a `@ManyToOne` (owning side, FK `lead_id` on `deal` table) - `Lead.activities` is a `@OneToMany(mappedBy = "lead")` (inverse side — no FK on `lead` table)

@ManyToOne (owning side):

```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "lead_id", nullable = false,
            foreignKey = @ForeignKey(name = "fk_deal_lead"))
private Lead lead;
```

@OneToMany (inverse side):

```
@OneToMany(mappedBy = "lead", cascade = {CascadeType.PERSIST, CascadeType.MERGE},
            orphanRemoval = true)
@OrderBy("createdAt DESC")
private List<LeadActivity> activities = new ArrayList<>();
```

Synchronisation helpers (for bidirectional relationships):

```
public void addActivity(LeadActivity activity) {
    activities.add(activity);
    activity.setLead(this);
}

public void removeActivity(LeadActivity activity) {
    activities.remove(activity);
    activity.setLead(null);
}
```

Without these helpers, the in-memory state can be inconsistent — `lead.getActivities()` has the activity but `activity.getLead()` is null. The persistence context may flush the inconsistent state. The helpers keep both sides in sync.

Fetch strategies: - `@ManyToOne` — default is `EAGER`, we always set `LAZY` (`EAGER` on `@ManyToOne` is the classic N+1 trap — `leadRepository.findAll()` would fetch each lead's user separately) - `@OneToMany` — default is `LAZY` (good)

`orphanRemoval = true`: when an activity is removed from `lead.getActivities()` (the collection), Hibernate deletes the row from `lead_activity` table. Without it, the `lead_activity.lead_id` would be set to NULL (and the row would orphan).

`@OrderBy("createdAt DESC")`: Hibernate adds an `ORDER BY` to the generated SQL when the collection is loaded. Note: this is JPQL-level ordering, applied in memory if the collection is already loaded. For DB-level ordering, use `@OrderColumn` (which adds an index column to the join table — we don't use this because we always sort explicitly in queries).

`@JoinColumn(foreignKey = @ForeignKey(name = "fk_deal_lead"))`: gives the FK constraint a name. Useful for migrations (you can `ALTER TABLE deal DROP CONSTRAINT fk_deal_lead` instead of guessing the auto-generated name). We name every FK constraint.

KEY TERMS TO MENTION: - `@ManyToOne` is the owning side (FK), `@OneToMany(mappedBy = "lead")` is the inverse - `FetchType.LAZY` everywhere (`EAGER` on `@ManyToOne` = N+1 trap) - `orphanRemoval = true` deletes the row when removed from collection - Synchronisation helpers (`addActivity` / `removeActivity`) keep bidirectional state

consistent - `@OrderBy` is JPQL-level (in-memory if collection loaded); `@OrderColumn` is DB-level - `@JoinColumn(foreignKey = @ForeignKey(name = "fk_..."))` names the constraint (useful for migrations)

FOLLOW-UP QUESTIONS: 1. What if you forget `mappedBy` on the inverse side? 2. What's the difference between `orphanRemoval` and `CascadeType.REMOVE`? 3. How do you handle a self-referencing `@ManyToOne` (tree structure)?

FOLLOW-UP ANSWERS: 1. Without `mappedBy`, Hibernate treats both sides as owning — it creates two FK columns (`lead.activities` creates `lead_activities.lead_id` AND `Lead.activities` creates another join table `lead_activities`). Two tables, two FKs — chaos. The fix is always `mappedBy = "lead"` on the `@OneToMany` side referencing the `@ManyToOne` field name on the other side. 2. `CascadeType.REMOVE` deletes children when the parent is deleted (explicit `repository.delete(lead)` deletes activities). `orphanRemoval = true` deletes children when they're removed from the parent's collection (in-memory `lead.getActivities().remove(activity)` deletes the row on flush). They're complementary — `orphanRemoval` is for "this entity's lifecycle is bound to the parent's collection membership", `CascadeType.REMOVE` is for "delete children when parent is deleted". 3. We have a `category.parent` self-referencing `@ManyToOne`:

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "parent_id", foreignKey = @ForeignKey(name =
    "fk_category_parent"))
private Category parent;

@OneToMany(mappedBy = "parent")
private List<Category> children = new ArrayList<>();
```

The DB column `parent_id` references the same `category.id` — a self-FK. Careful: circular references (category A's parent is B, B's parent is A) would cause infinite recursion in JSON serialisation — we use `@JsonManagedReference` / `@JsonBackReference` or `@JsonIgnore` on one side.

RED FLAGS TO AVOID: - `FetchType.EAGER` on `@ManyToOne` (N+1 trap) - Forgetting `mappedBy` (creates a join table) - Forgetting synchronisation helpers (in-memory inconsistency) - `@OrderBy` vs `@OrderColumn` confusion

Q5-4: What is lazy vs eager loading and how did you use it

DIFFICULTY: Easy **CATEGORY TAG:** Hibernate/JPA

ANSWER: EAGER loads the association immediately (in the same SQL or a follow-up SQL). **LAZY** loads it on first access (triggers a separate SQL when `lead.getUser()` is called).

Defaults in JPA: - `@ManyToOne` + `@OneToMany` — EAGER (BAD default; we override to LAZY) - `@OneToMany` + `@ManyToMany` — LAZY (good)

Why LAZY everywhere: EAGER on `@ManyToOne` means `leadRepository.findAll()` triggers a follow-up SELECT for each lead's user — N+1 (N leads, 1 user query each = N+1 queries). For 100 leads, that's 101 queries instead of 1.

The N+1 trap (concrete example):

```
// Bad: EAGER @ManyToOne user on Lead
List<Lead> leads = leadRepository.findAll(); // 1 query
for (Lead lead : leads) {
    System.out.println(lead.getUser().getName()); // N more queries (one per lead)
}
// Total: 101 SQL queries for 100 leads
```

Fixes: 1. `FetchType.LAZY` (default for `@OneToOne`, override for `@ManyToOne`) — user is loaded only when `lead.getUser()` is called. If you don't call it, no extra query. 2. `JOIN FETCH` in JPQL — explicitly fetch the association in one query:

```
@Query("SELECT l FROM Lead l JOIN FETCH l.user WHERE l.id = :id")
Lead findByIdWithUser(@Param("id") UUID id);
```

1. `@EntityGraph` — declaratively specify which associations to fetch:

```
@EntityGraph(attributePaths = {"user", "organization"})
@Query("SELECT l FROM Lead l WHERE l.userId = :userId AND l.stage = :stage")
List<Lead> findByUserAndStage(@Param("userId") UUID userId,
    @Param("stage") LeadStage stage);
```

1. `@BatchSize` — when LAZY associations are accessed, fetch them in batches (e.g. `@BatchSize(size = 50)` on `Lead.user` means: when 100 leads' users are accessed, Hibernate fetches them in 2 queries of 50 users each, not 100 queries of 1 user each).

Our convention: - All `@ManyToOne` and `@OneToOne` → `FetchType.LAZY` - `@OneToOne` → LAZY (default) - For queries that need associations, use `@EntityGraph` on the repository method (declarative, type-safe) - Avoid `JOIN FETCH` in `@Query` strings (hard to refactor, no compile-time check) — prefer `@EntityGraph` - `@BatchSize(size = 50)` on critical `@ManyToOne` (e.g. `Lead.user`) as a defense-in-depth against N+1 if a developer forgets `@EntityGraph`

LazyInitializationException — the classic Hibernate pitfall: happens when a LAZY association is accessed *after the session is closed* (e.g. in the controller after the service returned, in the JSON serialiser). Fix: (a) use `@EntityGraph` to fetch eagerly in the query; (b) use `spring.jpa.open-in-view = false` (we do — see below); (c) use DTOs (MapStruct) to materialise the data inside the `@Transactional` boundary.

spring.jpa.open-in-view: Spring Boot's default is `true` — keeps a Hibernate session open for the entire HTTP request, so LAZY associations can be fetched in the controller/view. This is a **trap** — it hides N+1 issues and adds a long-running session. We set `false` (the right default for a serious backend) and use `@EntityGraph` + DTO mapping inside the `@Transactional` service.

KEY TERMS TO MENTION: - JPA defaults: `@ManyToOne / @OneToOne` = EAGER (override to LAZY), `@OneToMany / @ManyToMany` = LAZY (good) - N+1 problem: `findAll()` + access to association = N extra queries - Fixes: LAZY + `@EntityGraph` (preferred) + `JOIN FETCH` (in `@Query`) + `@BatchSize(size = 50)` (defense) - `spring.jpa.open-in-view = false` — don't keep session open in controller/view - DTO mapping (MapStruct) inside `@Transactional` — materialise data before session closes - `LazyInitializationException` — accessing LAZY after session close

FOLLOW-UP QUESTIONS: 1. How do you detect N+1 in production? 2. What's the cost of `@EntityGraph`? 3. Why not just use EAGER everywhere if you always need the association?

FOLLOW-UP ANSWERS: 1. Three ways: (a) Hibernate statistics — `spring.jpa.properties.hibernate.generate_statistics = true` + log "N+1 queries" warnings; (b) **p6spy** — a JDBC proxy that logs every query with the calling stack trace, useful for finding which code path triggers N+1; (c) **OpenTelemetry traces** — we trace every DB query as a span; N+1 shows up as many small spans under one parent. We use (c) — p95 trace shows queries clearly; an N+1 is visible as 100 spans where there should be 1. 2. `@EntityGraph` adds an `OUTER JOIN` to the SQL — bigger result set, more memory, but one query instead of N+1. For associations we always need, `@EntityGraph` is the right call. For associations we sometimes need, keep LAZY and add `@EntityGraph` only on the queries that need it. 3. EAGER forces the association to be loaded for every query — even when you don't need it. E.g. `leadRepository.findAll()` for a list view doesn't need each lead's `communications` (potentially 100s of rows). EAGER on `communications` would load all of them — catastrophic. EAGER is the wrong default; explicit fetching via `@EntityGraph` is the right pattern.

RED FLAGS TO AVOID: - EAGER on `@OneToMany` (catastrophic) - Not knowing `@EntityGraph` - Saying "we use `JOIN FETCH`" without explaining N+1 - Not knowing `LazyInitializationException`

Q5-5: How did you handle the N+1 query problem

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: N+1 is when you fetch N parent entities and then trigger N follow-up queries for their associations (1 query for parents + N for children). We hit this in production twice; here's how we handle it.

Case 1 — `Lead.user` access in a list view: - Symptom: `GET /api/leads?limit=20` returned p95 = 800ms (instead of expected 80ms). p6spy showed 21 SQL queries: 1 for leads, 20 for users (one per lead). - Root cause: a developer wrote `leads.stream().map(l -> l.getUser().getName()).collect(toList())` in the service, with `Lead.user` set to LAZY (good default) but no `@EntityGraph` on the repository method. - Fix: added `@EntityGraph(attributePaths = "user")` to `LeadRepository.findByUserIdAndStage`:


```
@EntityGraph(attributePaths = {"user", "organization"})
@Query("SELECT l FROM Lead l WHERE l.userId = :userId AND l.stage = :stage AND l.isActive = true")
Page<Lead> findActiveByUserAndStage(@Param("userId") UUID userId, @Param("stage") LeadStage stage, Pageable pageable);
```

- Result: p95 dropped from 800ms to 60ms (1 query with JOIN instead of 21 queries).

Case 2 — `Lead.communications` access in a detail view: - Symptom: `GET /api/leads/{id}` returned p95 = 1.2s (expected 200ms). Trace showed 1 query for lead, 1 for activities, 1 for scores, 1 for analysis, 50 queries for communications (one per comm — wrong!). - Root cause: `Lead.communications` is `@OneToMany(mappedBy = "lead")` — LAZY (good). But inside the JSON serialiser, Jackson iterates `lead.getCommunications()`, and each `comm.getLead()` (the bidirectional back-reference) triggers a session flush + reload of the lead → N queries for N communications. - Fix: (a) DTO mapping via MapStruct — the mapper extracts only the fields the DTO needs, inside the `@Transactional` boundary, never triggering lazy access in the serialiser; (b) `@JsonIgnore` on `Communication.lead` (break the bidirectional back-reference in JSON); (c) `@EntityGraph(attributePaths = "communications")` on the query if the comm list is always needed. - Result: p95 dropped to 180ms.

Case 3 — `CreditLedger` batch insert: - Symptom: monthly credit reset job ran for 12 minutes (expected 1 minute). 3,200 users × ~5 ledger entries each = 16K inserts, but each insert triggered a `SELECT` for the user (Hibernate's "verify the FK exists" behaviour). - Root cause: `CreditLedger.user` is `@ManyToOne(fetch = LAZY)`. Hibernate by default verifies the FK on insert (extra `SELECT`). - Fix: `@ManyToOne(fetch = LAZY, optional = false, foreignkey = @ForeignKey(value = ConstraintMode.NO_CONSTRAINT))` — we keep the FK in the DB (DDL has `REFERENCES user_account(id)`) but Hibernate doesn't verify it on insert. Or: batch inserts via `JdbcTemplate` for the bulk job (skip Hibernate's per-row checks). - Result: 12 minutes → 30 seconds.

Detection tools: 1. **OpenTelemetry traces** — every DB query is a span; N+1 shows up as many small spans under one parent (visible in the trace UI) 2. **p6spy** (dev only) — logs every query with calling stack trace; we run it in dev to catch N+1 in unit tests 3. **Hibernate statistics** — `spring.jpa.properties.hibernate.generate_statistics = true` logs query counts per session; we enable in staging for integration tests 4. **Code review** — `for (X x : list) { x.getY().getZ(); }` is a code smell for N+1; reviewers flag it

Prevention: - `@EntityGraph` on every repository method that returns a list and the caller accesses associations - `@BatchSize(size = 50)` on `@ManyToOne` as defense-in-depth - MapStruct DTO mapping inside `@Transactional` — materialise data before session closes - `spring.jpa.open-in-view = false` — no lazy access in controller/view - Unit tests that assert query count (via `@ExpectedQueryCount` custom annotation we wrote) — fail if N+1 introduced

KEY TERMS TO MENTION: - N+1 symptom: 1 query for parents + N for children - Three real cases: `Lead.user` list view, `Lead.communications` bidirectional back-reference, `CreditLedger.user` FK verification on insert - Fixes: `@EntityGraph(attributePaths = ...)`, MapStruct DTO mapping inside `@Transactional`, `@JsonIgnore` on back-reference, `@BatchSize(size = 50)`, batch inserts via `JdbcTemplate` - Detection: OpenTelemetry traces

(DB spans), p6spy (dev), Hibernate statistics (staging), code review - Prevention: `@EntityGraph` per list method, `@BatchSize` defense-in-depth, DTO mapping, `open-in-view = false`, query-count assertions in unit tests

FOLLOW-UP QUESTIONS: 1. How do you test for N+1 in CI? 2. What's `@BatchSize` and when does it kick in? 3. Why does `@JsonIgnore` help with N+1?

FOLLOW-UP ANSWERS: 1. Custom `@ExpectedQueryCount(n = 1)` annotation + JUnit extension that wraps the test in a Hibernate session, runs the test, asserts the query count. We have ~50 such tests for repository methods. Example:

```
@ExpectedQueryCount(n = 1) void testFindByUserAndStage() { repository.findByUserAndStage(userId, DIS
```

— if a developer removes the `@EntityGraph`, the test fails with “expected 1 queries, got 21”.

2. `@BatchSize(size = 50)` is Hibernate's lazy-loading optimisation: when a LAZY collection (or `@ManyToOne` proxy) is accessed for N entities in a session, Hibernate fetches them in batches of `size` instead of one-by-one. E.g. `@BatchSize(size = 50)` on `Lead.user` + 100 leads' users accessed → 2 queries (50 users each) instead of 100 queries (1 user each). It's a fallback for when `@EntityGraph` isn't used (developer forgot or a new query path was added). We add it as defense-in-depth on every `@ManyToOne` on a hot entity. 3. `@JsonIgnore` on

`Communication.lead` prevents Jackson from serialising the back-reference during JSON output. Without it, Jackson walks

```
lead.getCommunications() → each comm.getLead() → leads to lead.getCommunications() → ...
```

(Jackson handles the cycle by reference-tracking, but each `comm.getLead()` triggers a lazy load if the lead is detached → N queries). With `@JsonIgnore`, Jackson doesn't walk the back-reference — no lazy access. We pair this with DTO mapping (the DTO mapper accesses the fields we need, inside `@Transactional`, never in the serialiser).

RED FLAGS TO AVOID: - “We don't have N+1” (every project has it; admit it) - Not knowing `@EntityGraph` - Saying “we use `JOIN FETCH`” without `@EntityGraph` - Forgetting `spring.jpa.open-in-view = false`

Q5-6: What is your approach to database transactions (@Transactional)

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA

ANSWER: We use Spring's `@Transactional` declaratively on service methods, with explicit isolation levels and propagation per use case.

Default (@Transactional): `Propagation.REQUIRED` (join existing or start new), `Isolation.DEFAULT` (Postgres default = `READ_COMMITTED`), rollback on `RuntimeException` and `Error` (not on checked exceptions — common gotcha).

When we override isolation: - `REPEATABLE_READ` for credit deductions — concurrent deductions to the same user's balance row must see a stable view. Postgres's MVCC makes `REPEATABLE_READ` nearly free (no extra locking) and prevents lost updates. - `SERIALIZABLE` (rare) — we use it for the monthly credit reset (which writes to the same `credit_balance` row for every user) to prevent races between the reset job and user actions. The cost is retry on

serialization failure — we retry 3x with `@Retryable`. - `READ_COMMITTED` (default) — for everything else (CRUD operations, simple reads).

When we override propagation: - `Propagation.REQUIRES_NEW` — for audit logging. We want the audit row written even if the main transaction rolls back. Audit service method is `@Transactional(propagation = REQUIRES_NEW)` — it starts a separate transaction that commits independently. Caveat: `REQUIRES_NEW` suspends the outer transaction (which can deadlock if outer holds locks the inner needs) — we use it only for audit (which only inserts, doesn't read locked rows). - `Propagation.SUPPORTS` — for read methods that can run in a transaction or not. We use it for read-only queries that don't need a transaction but can join one if present. - `Propagation.MANDATORY` — for service methods that must be called from within a transaction (we throw if no transaction). We use it for inner service methods that should never be called directly from a controller. - `Propagation.NOT_SUPPORTED` / `NEVER` — we don't use these.

`@Transactional(readOnly = true)` : for read-only queries. Two benefits: (a) Hibernate disables dirty checking (faster); (b) we route to a read replica via `ReadWriteRoutingDataSource` (queries with `readOnly = true` go to a read replica, queries with `readOnly = false` go to the primary). We use this for all `GET` endpoints and reporting queries.

`@Transactional(timeout = 30)` : for long-running operations that might hold locks. The transaction rolls back after 30 seconds. We use it for batch operations to prevent a runaway job from holding locks forever.

`rollbackFor` and `noRollbackFor` : by default, Spring rolls back on `RuntimeException` and `Error` but not on checked exceptions. We use `@Transactional(rollbackFor = Exception.class)` for service methods that throw checked exceptions (rare — we prefer unchecked). We use `noRollbackFor = BusinessException.class` for cases where a business exception (e.g. "lead not found") shouldn't roll back (the row was already there, we just don't want it). Actually, no — we always roll back on business exceptions; we don't use `noRollbackFor`.

Self-invocation gotcha: calling a `@Transactional` method from [within the same class](#) via `this.method()` bypasses the Spring proxy — the transaction annotation is ignored. Fix: (a) extract the method to a separate bean; (b) inject `self` via `@Lazy` (smelly); (c) use `AopContext.currentProxy()` (requires `@EnableAspectJAutoProxy(exposeProxy = true)`). We use (a) — extract to another service.

Transaction propagation examples:

```

@Service
public class LeadService {
    @Transactional // REQUIRED + READ_COMMITTED
    public Lead createLead(CreateLeadDto dto, AuthUser user) {
        // ...
        auditService.record("lead_created", lead.getId()); // REQUIRES_NEW inside
        return lead;
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(String action, UUID resourceId) {
        // always commits, even if caller rolls back
    }
}

@Service
public class CreditService {
    @Transactional(isolation = Isolation.REPEATABLE_READ)
    public void deduct(UUID userId, int cost, String action, UUID refId) {
        CreditBalance balance =
            creditBalanceRepository.findByIdForUpdate(userId); // SELECT ... FOR UPDATE
        if (balance.getAmount() < cost) throw new InsufficientCreditsException();
        balance.setAmount(balance.getAmount() - cost);
        creditLedgerRepository.save(new CreditLedger(userId, -cost, action, refId));
    }
}

```

KEY TERMS TO MENTION: - Default: `REQUIRED` + `READ_COMMITTED` + rollback on `RuntimeException` - Override per use case: `REPEATABLE_READ` for credit deduct, `SERIALIZABLE` for monthly reset (with `@Retryable`), `REQUIRES_NEW` for audit logging - `readOnly = true` for reads → Hibernate dirty-checking disabled + routes to read replica via `ReadWriteRoutingDataSource` - `timeout = 30` for batch operations - `rollbackFor = Exception.class` for checked exceptions (rare) - Self-invocation gotcha → extract to another bean or `AopContext.currentProxy()`

FOLLOW-UP QUESTIONS: 1. What's the cost of `REQUIRES_NEW`? 2. How do you test transaction rollback? 3. What's the difference between `REQUIRED` and `NESTED`?

FOLLOW-UP ANSWERS: 1. `REQUIRES_NEW` suspends the outer transaction (releases its locks) and starts a new one. Costs: (a) two DB connections held simultaneously (outer is suspended but the connection is kept — HikariCP is now holding 2 connections per request); (b) deadlock risk if outer holds locks the inner needs; (c) latency — two commit phases. We use it sparingly (audit, email send) and configure HikariCP to allow 2 connections per thread (`maximumPoolSize = 2 * maxThreads`). 2. Three ways: (a) `@SpringBootTest` + `@Transactional` test method + `assertTrue(TransactionAspectSupport.currentTransactionStatus().isRollbackOnly())` after triggering the exception; (b) `@DataJpaTest` with `@Transactional` (auto-rollback) — assert the entity is NOT in the DB after the rollback; (c) integration test with Testcontainers PG — call

the service, expect exception, query the DB to verify state is unchanged. We use (c) for service-layer tests — most realistic. 3. **REQUIRED** joins the outer transaction or starts a new one — one physical transaction. **NESTED** creates a savepoint within the outer transaction — the inner can roll back to the savepoint without rolling back the outer. Postgres supports savepoints natively. We don't use **NESTED** — it's confusing and rare; **REQUIRES_NEW** (separate transaction) or **REQUIRED** (same transaction) covers our cases.

RED FLAGS TO AVOID: - **@Transactional** on controllers (transaction should be in service) - **@Transactional** on every method (read methods don't need it; use **readOnly = true**) - **REQUIRES_NEW** everywhere (deadlock + connection exhaustion) - Forgetting self-invocation gotcha - Not knowing **readOnly = true** routes to read replica

Q5-7: How did you implement custom JPQL queries

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: Three layers: (1) Spring Data JPA derived queries (auto-generated from method name), (2) **@Query** JPQL, (3) Specifications for dynamic queries.

1. Derived queries (for simple cases):

```
public interface LeadRepository extends JpaRepository<Lead, UUID> {
    List<Lead> findByIdAndStageAndIsActiveTrue(UUID userId, LeadStage
        stage); // JPQL auto-generated
    Optional<Lead> findByIdAndUserId(UUID id, UUID userId);
    long countByUserIdAndIsActiveTrue(UUID userId);
}
```

Limitation: method names get unwieldy for complex filters. We use derived queries for single-field filters and switch to **@Query** for multi-field.

2. **@Query** JPQL (for complex queries):

```

@Query("""
    SELECT l FROM Lead l
    WHERE l.userId = :userId
        AND l.isActive = true
        AND l.deletedAt IS NULL
        AND (:stage IS NULL OR l.stage = :stage)
        AND (:niche IS NULL OR l.niche = :niche)
        AND (:tags IS NULL OR EXISTS (
            SELECT 1 FROM l.tags t WHERE t IN :tags
        ))
    ORDER BY
        CASE WHEN :sortBy = 'createdAt' THEN l.createdAt END DESC,
        CASE WHEN :sortBy = 'replyScore' THEN l.replyScore END DESC NULLS LAST
""")
Page<Lead> findByUserWithFilters(
    @Param("userId") UUID userId,
    @Param("stage") LeadStage stage, // nullable
    @Param("niche") String niche, // nullable
    @Param("tags") List<String> tags, // nullable
    @Param("sortBy") String sortBy,
    Pageable pageable);

```

Note: JPQL doesn't support **IF NULL** like SQL — we use **:param IS NULL OR ...** pattern (return all rows if param is null).

3. Native SQL (for Postgres-specific features):

```

@Query(value = """
    SELECT * FROM lead
    WHERE user_id = :userId
        AND to_tsvector('english', business_name || ' ' ||
        coalesce(owner_name, '')) @@ plainto_tsquery(:search)
    ORDER BY ts_rank(to_tsvector('english', business_name),
        plainto_tsquery(:search)) DESC
""", nativeQuery = true)
Page<Lead> fullTextSearch(@Param("userId") UUID userId, @Param("search") String
    search, Pageable pageable);

```

4. Specifications (for dynamic queries): For “build the WHERE clause based on which filters are present”:

```

public class LeadSpecifications {
    public static Specification<Lead> hasUserId(UUID userId) {
        return (root, query, cb) -> cb.equal(root.get("userId"), userId);
    }
    public static Specification<Lead> hasStage(LeadStage stage) {
        return stage == null ? null : (root, query, cb) ->
            cb.equal(root.get("stage"), stage);
    }
    public static Specification<Lead> hasTags(List<String> tags) {
        return tags == null || tags.isEmpty() ? null
            : (root, query, cb) -> root.join("tags").in(tags);
    }
    public static Specification<Lead> isActive() {
        return (root, query, cb) -> cb.isTrue(root.get("isActive"));
    }
}

// Usage:
Specification<Lead> spec = Specification.where(LeadSpecifications.hasUserId(userId))
    .and(LeadSpecifications.hasStage(stage))
    .and(LeadSpecifications.hasTags(tags))
    .and(LeadSpecifications.isActive());
Page<Lead> result = leadRepository.findAll(spec, pageable);

```

5. Custom repository (for complex business logic):

```

public interface LeadRepositoryCustom {
    LeadStatsDto computeStats(UUID userId, Instant from, Instant to);
    List<Lead> bulkUpdateStage(List<UUID> leadIds, LeadStage newStage, UUID userId);
}

public class LeadRepositoryCustomImpl implements LeadRepositoryCustom {
    @PersistenceContext
    private EntityManager em;

    @Override
    public LeadStatsDto computeStats(UUID userId, Instant from, Instant to) {
        // Use EntityManager for criteria API or native SQL
        // ...
    }
}

public interface LeadRepository extends JpaRepository<Lead, UUID>,
    LeadRepositoryCustom {
    // ...
}

```

6. Projections (DTOs from queries):


```
public interface LeadSummary {
    UUID getId();
    String getBusinessName();
    LeadStage getStage();
    Short getReplyScore();
}

@Query("SELECT l.id AS id, l.businessName AS businessName, l.stage AS stage,
        l.replyScore AS replyScore FROM Lead l WHERE l.userId = :userId")
List<LeadSummary> findSummariesByUserId(@Param("userId") UUID userId);
```

This fetches only the needed columns (not the full entity) — faster, less memory.

KEY TERMS TO MENTION: - Spring Data derived queries (auto-generated from method name) - `@Query` JPQL with `:param IS NULL OR ...` pattern for optional filters - Native SQL (`nativeQuery = true`) for Postgres-specific features (full-text `to_tsvector`, `ts_rank`, JSONB operators) - Specifications for dynamic queries (`Specification.where(...).and(...).and(...)`) - Custom repository (`*RepositoryCustom` interface + `*RepositoryCustomImpl` with `EntityManager`) for complex business logic - Projections (interface-based DTOs from queries) — fetch only needed columns

FOLLOW-UP QUESTIONS: 1. JPQL vs native SQL — when do you use each? 2. How do you prevent SQL injection in JPQL/native queries? 3. How do you handle dynamic ORDER BY?

FOLLOW-UP ANSWERS: 1. JPQL by default — portable, type-safe (the IDE validates the query against the entity), Hibernate generates the SQL. Native SQL only for Postgres-specific features (full-text, JSONB operators, window functions) or for performance tuning a specific query (we can hint to use a specific index). Trade-off: native SQL ties us to Postgres; JPQL lets us switch to MySQL/H2 if needed (we won't, but it's a thought). 2. JPQL/native queries are always parameterised — `:param` (named) or `?1` (positional). Spring Data + Hibernate bind parameters safely (no string concatenation). We NEVER concatenate user input into a query string. The one risk: dynamic ORDER BY (we can't parameterise a column name) — we use an allowlist:

```
private static final Set<String> ALLOWED_SORT_FIELDS = Set.of("createdAt", "replyScore", "businessName");
```

This is the standard pattern. 3. Two ways: (a) Pageable + Sort parameter — Spring Data generates the ORDER BY clause from `Sort.by("createdAt").descending()`. We pass `Sort.by(sortBy.split(":")[0]).descending()` (with allowlist validation). (b) Criteria API (for complex dynamic queries) — `cb.asc(root.get(sortBy))` programmatically. We use (a) for simple cases, (b) for Specifications.

RED FLAGS TO AVOID: - String concatenation in queries (SQLi) - Native SQL when JPQL would do (portability loss) - Loading full entities when a projection would suffice (memory waste) - Specifications when a `@Query` would do (over-engineering)

Q5-8: How did you use Spring Data JPA repositories

DIFFICULTY: Easy **CATEGORY TAG:** Hibernate/JPA

ANSWER: Standard Spring Data JPA repositories — `JpaRepository<Entity, Id>` interface, Spring generates the implementation at runtime.

```
@Repository
public interface LeadRepository extends JpaRepository<Lead, UUID>,
    LeadRepositoryCustom {

    // Derived query (auto-generated)
    Optional<Lead> findByIdAndUserId(UUID id, UUID userId);

    // @Query with @EntityGraph to avoid N+1
    @EntityGraph(attributePaths = {"user", "organization"})
    @Query("SELECT l FROM Lead l WHERE l.userId = :userId AND l.stage = :stage AND l.isActive = true")
    Page<Lead> findActiveByUserAndStage(@Param("userId") UUID userId,
        @Param("stage") LeadStage stage, Pageable pageable);

    // Native query for Postgres full-text
    @Query(value = "SELECT * FROM lead WHERE user_id = :userId AND to_tsvector('english', business_name) @@ plainto_tsquery(:q)", nativeQuery = true)
    Page<Lead> fullTextSearch(@Param("userId") UUID userId, @Param("q") String q, Pageable pageable);

    // Modifying query (UPDATE)
    @Modifying
    @Query("UPDATE Lead l SET l.stage = :newStage, l.updatedAt = CURRENT_TIMESTAMP WHERE l.id = :id")
    int updateStage(@Param("id") UUID id, @Param("newStage") LeadStage newStage);

    // Soft delete (via @SQLDelete on entity – repository.delete() triggers UPDATE)
    // Bulk delete
    @Modifying
    @Query("UPDATE Lead l SET l.deletedAt = CURRENT_TIMESTAMP, l.isActive = false WHERE l.userId = :userId AND l.stage = :stage")
    int bulkSoftDeleteByUserAndStage(@Param("userId") UUID userId, @Param("stage") LeadStage stage);

    // Projection (DTO from query)
    interface LeadSummary {
        UUID getId();
        String getBusinessName();
        LeadStage getStage();
    }
    @Query("SELECT l.id AS id, l.businessName AS businessName, l.stage AS stage FROM Lead l WHERE l.userId = :userId")
    List<LeadSummary> findSummariesByUserId(@Param("userId") UUID userId);

    // Count
    long countByUserIdAndIsActiveTrue(UUID userId);
}
```

@Repository annotation: not strictly required (Spring Data auto-detects), but we add it for IDE clarity + exception translation (`PersistenceExceptionTranslationPostProcessor` translates JPA exceptions to Spring's `DataAccessException` hierarchy).

JpaRepository vs **CrudRepository** vs **PagingAndSortingRepository** : - **CrudRepository** — basic CRUD (`save`, `findById`, `delete`) - **PagingAndSortingRepository** extends **CrudRepository** — adds `findAll(Pageable)`, `findAll(Sort)` - **JpaRepository** extends **PagingAndSortingRepository** — adds JPA-specific (`flush`, `saveAndFlush`, `findAll` with `EntityGraph`, batch operations)

We use **JpaRepository** for everything — most features, no downside.

Custom repository composition:

```
public interface LeadRepository extends JpaRepository<Lead, UUID>,
    LeadRepositoryCustom { ... }
// Implementation:
public class LeadRepositoryCustomImpl implements LeadRepositoryCustom {
    @PersistenceContext
    private EntityManager em;
    // complex methods using EntityManager
}
```

Query methods rules: - Method name → derived query: `findBy<FieldName><Operator>` (`findByIdAndUserIdAndIsActiveTrue`) - `@Query` → custom JPQL/native - `@Modifying` + `@Query` → UPDATE/DELETE (call `repository.flush()` or use `@Modifying(clearAutomatically = true, flushAutomatically = true)`) - `@EntityGraph` → eager-fetch specific associations for this query only - `Pageable` → pagination (`Page<Lead>` response includes total count, totalPages) - `Sort` → sorting - `Optional<T>` → safe null handling

Caveats: 1. Derived query method names are limited — beyond 3-4 fields, switch to `@Query` 2. `@Modifying` queries need `@Transactional` — we add it at the service layer, not on the repository method 3. `Page<>` queries do an extra `COUNT` query (the total) — for huge tables, use `Slice<>` (no count) when you only need “has next page” not total 4. `@EntityGraph` is the modern N+1 fix (replaces `JOIN FETCH` in `@Query`) 5. Don’t put business logic in repositories — they’re data access only. Business logic goes in services.

KEY TERMS TO MENTION: - `JpaRepository<Entity, UUID>` — extends `PagingAndSortingRepository` + `CrudRepository` - Derived query (`findBy<Field><Op>`) vs `@Query` JPQL vs native SQL (`nativeQuery = true`) - `@Modifying` for UPDATE/DELETE (with `clearAutomatically` + `flushAutomatically`) - `@EntityGraph(attributePaths = ...)` — per-query eager fetching (modern N+1 fix) - `Page<>` (with count) vs `Slice<>` (no count, faster for huge tables) - Projections (interface-based DTOs from queries) - Custom repository composition (`*RepositoryCustom` + `*RepositoryCustomImpl` with `EntityManager`) - `@Repository` for exception translation (`PersistenceException` → `DataAccessException`)

FOLLOW-UP QUESTIONS: 1. What’s the cost of `Page<>` vs `Slice<>` ? 2. How do you handle a bulk insert (10K rows)? 3. What if your derived query method name is too long?

FOLLOW-UP ANSWERS: 1. `Page<>` does a follow-up `SELECT COUNT(*)` to compute `totalPages`. For a query that returns 1M rows, this is another full scan (same WHERE clause). `Slice<>` skips the count — returns `hasNext` (boolean) based on whether the next page exists (fetches `limit + 1` rows, returns `limit`, `hasNext = (fetched > limit)`). For infinite scroll UIs, `Slice<>` is much faster. For paginated UIs that show “Page 1 of 100”, you need `Page<>`. We

use `Slice<>` for the leads list (infinite scroll), `Page<>` for admin (paginated). 2. Spring Data's `saveAll()` does row-by-row insert (N INSERT statements). For 10K rows, that's 10K round-trips. We use `JdbcTemplate.batchUpdate()` with a PreparedStatement batch (10 batches of 1000 rows = 10 round-trips) — 1000x faster. We expose this via `LeadRepositoryCustomImpl.bulkInsert(List<Lead> leads)`. Spring Boot 3.2 also supports Hibernate's batch insert via `spring.jpa.properties.hibernate.jdbc.batch_size=50` + `spring.jpa.properties.hibernate.order_inserts=true` (Hibernate groups inserts of the same entity into batches). 3. We have a rule: if a method name exceeds ~5 fields, switch to `@Query("SELECT l FROM Lead l WHERE ...")` with explicit params. The method name becomes a maintenance burden (one field rename = rename the whole method) and the IDE doesn't always refactor method names cleanly. `@Query` is more explicit and easier to read.

RED FLAGS TO AVOID: - `CrudRepository` (use `JpaRepository` for more features) - Business logic in repositories - `saveAll()` for 10K+ rows (use `JdbcTemplate.batchUpdate`) - `Page<>` for huge tables (use `Slice<>`)

Q5-9: What is the Hibernate Session/EntityManager lifecycle and how is it managed in Spring Boot

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA

ANSWER: The `EntityManager` (Hibernate's `Session` under the hood) is a per-unit-of-work object wrapping a persistence context — the first-level cache of managed entities. In Spring Boot, `SharedEntityManagerCreator` hands every `@PersistenceContext` injection a **thread-safe shared proxy** that creates a transaction-scoped `EntityManager` per operation: with `@Transactional`, one persistence context spans the method (all repositories called inside share it — that's why a `findById` and a later `save` in one service method see the same managed instances); without a transaction, each repository call gets its own short context (and flushes immediately — surprise point: simple `findById` works without a transaction, but write ops require one).

Entity lifecycle states and their interview value: **transient** (new, no id), **managed** (in the context — dirty checking active), **detached** (context closed — changes NOT tracked), **removed** (scheduled for delete). Dirty checking is the behavior everyone uses and nobody names: within a transaction, mutating a managed entity's field is enough — at flush, Hibernate snapshots managed entities on load and diffs at commit, generating the UPDATE automatically; calling `save()` on an already-managed entity is a no-op signaling ceremony. Detachment is where bugs live: entities returned from a `@Transactional` service become detached when the transaction ends; touching `lead.getActivities()` later throws `LazyInitializationException` — the classic. Our project-level answers to detachment: map to DTOs inside the transaction (never leak entities past the service boundary), or `JOIN FETCH` / `@EntityGraph` what you'll need.

Flush timing: flush (push SQL) happens at transaction commit, before queries that overlap affected tables (flush-auto query synchronization), and on explicit `flush()` — flush ≠ commit

(flush sends SQL, commit finalizes; a flush can still roll back). Long conversations would need detached state + merge, but we don't do long conversations — request-scoped transactions only, which keeps the context's memory bounded; the one place we manage contexts manually is batch processing (`EntityManager.clear()` every N rows to prevent a million managed entities from bloating the context into an OOM — the first-level cache is a feature per-request and a liability per-million-rows).

KEY TERMS TO MENTION: - Persistence context = first-level cache; shared proxy per transaction - States: transient/managed/detached/removed + dirty checking - `LazyInitializationException` = detached-entity access (DTOs in-transaction fix) - Flush vs commit distinction; auto-flush before overlapping queries - `clear()` in batch loops (context bloat prevention)

FOLLOW-UP QUESTIONS: 1. Two service methods each `@Transactional` — inner one called from outer: one transaction or two? 2. Why is `save()` sometimes issuing SELECT before INSERT?

FOLLOW-UP ANSWERS: 1. One — REQUIRED propagation joins the outer transaction and shares its persistence context; the inner `rollbackOnly` on exception poisons the whole thing. If the inner used `REQUIRES_NEW`, it suspends the outer and runs its own context — and the outer CANNOT see inner's uncommitted writes (isolation across two transactions) — a subtlety that has bitten real teams when inner creates data outer expects to read. 2. `save()` on an entity with an assigned (non-null) id can't know it's new, so `isNew()` is false → merge semantics → SELECT first, then UPDATE-or-INSERT. With `GenerationType.IDENTITY` (our choice for most tables), id is null until persist, so save is a straight INSERT. Assigned ids (external refs) get `@Version`-based or `Persistable` implementations to skip the probe SELECT — a concrete perf fix I applied on the API-key import path (halved inserts).

RED FLAGS TO AVOID: - Confusing Session with Connection (session borrows a connection lazily) - Claiming entities stay managed after the transaction ends - No answer for `LazyInitializationException` (everyone hits it)

Q5-10: How did you implement pagination and sorting

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA / REST

ANSWER: Two mechanisms for two access patterns. **Offset pagination** (`Pageable` — `PageRequest.of(page, size, Sort.by("createdAt").descending())`) for admin consoles and small datasets: `Page<Lead>` returns content + total count; Spring Data translates to `LIMIT/OFFSET`. Its cost is structural: `OFFSET 10000` forces the DB to walk and discard 10,000 rows, and the total-count query is a second full scan — fine at page < 50, degenerate beyond. **Cursor (keyset) pagination** for the hot paths (leads list, activity feeds): the cursor is the last row's sortable tuple (`created_at, id`), and the query is `WHERE (created_at, id) < (:lastCreatedAt, :lastId) ORDER BY created_at DESC, id DESC LIMIT 50` — index-peek instead of walk-and-discard, constant time at any depth, and stable under concurrent inserts (no page-shifting duplicates that `OFFSET` produces when a new row lands mid-scroll). The tuple comparison needs the composite index

`(org_id, created_at DESC, id DESC)` — index design and cursor design are the same decision.

API shape: cursor endpoints take `?cursor=<opaque>&limit=50`, return `data` + `next_cursor` (null on end) — the cursor is Base64 of the tuple + a query-hash, so a client can't swap cursors between different filters silently; no `total` (counting 2M rows per page is waste — clients get "has_more" instead). Sorting is allow-listed: a map of client-facing sort keys → whitelisted column expressions (never pass client sort strings into queries — that's an injection/hygiene hole and an index-killer). Validation: `limit` clamped to [1, 100] (a `limit=1000000` is a DoS); count queries for admin offsets run on read replicas. The React frontend drives the list with `useInfiniteQuery` consuming `next_cursor` — the full stack agrees on one contract, which is why our p95 on the leads list stayed ~140ms at 2M rows.

KEY TERMS TO MENTION: - `Pageable` / `Page` (offset) vs keyset cursor `(created_at, id)` tuple - OFFSET's walk-and-discard cost + page-shifting instability - Composite index matching the cursor order - Opaque cursor (Base64 tuple + query-hash), no total count - Allow-listed sorts, clamped limits

FOLLOW-UP QUESTIONS: 1. How do you jump to page 47 in a cursor API (admin wants it)? 2. Why include `id` in the cursor tuple?

FOLLOW-UP ANSWERS: 1. You don't — honestly. Admins needing deep jumps get a search/filter-first UX (narrow to the row via criteria, then cursor from there) or a materialized snapshot view with offsets where the dataset is bounded. Deep-offset is a UX smell more than a technical one; the "page 47" request usually means missing filters. 2. Tie-breaking: `created_at` is not unique — 50 leads created in the same millisecond would paginate wrongly (skip/duplicate rows) on a single-column cursor. The unique `id` makes the tuple a strict total order matching the index. Any keyset design without a unique tiebreaker is subtly broken under load.

RED FLAGS TO AVOID: - Offset pagination on the main list with "it's fine" (it isn't at scale) - Exposing raw total counts on million-row tables (per-request COUNT) - Passing client sort strings unvalidated into queries

Q5-11: How did you handle database migrations with Flyway

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA / DevOps

ANSWER: Flyway is the source of truth for schema; Hibernate maps onto it — never the reverse (`hbm2ddl.auto=validate` in every non-dev profile, so a drift between entities and migrations fails boot, not production). Migrations live in `db/migration` as `V<timestamp>_<description>.sql`, versioned strictly forward-only; each PR adds exactly one migration and CI runs it against a disposable Postgres (Testcontainers) both fresh (all migrations from zero — catches broken chains) and upgraded-from-last-release (catches "works only on a fresh DB"). Boot applies pending migrations before serving traffic, under `flyway.clean-disabled=true` (clean = drop-schema; the setting that has nuked databases

when misconfigured in prod) and `out-of-order=false` so a branched team can't silently interleave versions.

Zero-downtime discipline (expand → migrate → contract): migrations NEVER mix backward-incompatible steps with the deploy that needs them. Adding a column = one release (nullable + default null); backfill = a batched job (`UPDATE ... WHERE id BETWEEN` in 5k chunks — a single UPDATE on 2M rows takes locks and replication lag spikes); switching reads = next release; dropping old = weeks later behind a check that no code path references it (log-based usage check). Rename example I give: `name` → `business_name` shipped as add-column, dual-write in service, backfill job, switch reads, drop old — five steps across three releases, zero requests lost; the “just rename it” version is a deploy-ordering outage. Destructive ops get extra respect: index builds `CREATE INDEX CONCURRENTLY` (Flyway runs it in its own transaction — `executeInTransaction=false` for those scripts), and we rehearse risky migrations on a production snapshot in staging with `pg_stat_statements` watching for plan regressions.

Team mechanics: migrations are code-reviewed like code (a DBA review label on anything touching big tables), version conflicts surface at CI (two PRs same version fails the chain build), and rollback is “forward fix” — down-migrations are discouraged; a bad migration is reverted by a NEW migration, and if the data damage is done, restore from PITR is the runbook (tested quarterly — an untested backup is a rumor).

KEY TERMS TO MENTION: - Flyway versioned migrations + `hbm2ddl.auto=validate` (drift fails boot) - Forward-only; CI runs fresh + upgrade paths on Testcontainers - Expand → backfill (batched) → contract release discipline - `CREATE INDEX CONCURRENTLY` outside transactions - Rollback = forward fix; PITR restore as the real safety net (tested)

FOLLOW-UP QUESTIONS: 1. Migration fails halfway in prod — what state are you in? 2. How do you handle seed/reference data?

FOLLOW-UP ANSWERS: 1. Flyway marks the version FAILED in `flyway_schema_history`; Postgres DDL is transactional so a failed migration inside a transaction auto-rolled back (clean state, fix + retry) — EXCEPT concurrent index builds which can't be transactional; those leave a INVALID index to drop manually before retry. Runbook covers both branches; the failure alert pages because a boot-failed release is an incident. 2. Reference data (plan tiers, credit costs) ships as `R__` repeatable migrations (checksum-driven re-run on change) or versioned inserts with `ON CONFLICT DO NOTHING` — idempotent by construction. Business-editable data (pricing) moved OUT of migrations into tables managed by admin UI — migrations seed the first row, product owns the rest.

RED FLAGS TO AVOID: - Auto-ddl (`update`) in any shared environment - Single-release renames/drops (“just deploy the schema first” — no) - Untested restores / no answer for failed-migration state

Q5-12: What is your caching strategy with Hibernate second-level cache

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA / Redis

ANSWER: We evaluated the second-level cache and adopted it narrowly — L2 caching is powerful and treacherous, and the honest engineering answer is “here’s exactly where it pays and where it doesn’t.” Setup: Ehcache as the L2 provider, `SharedCacheMode.SELECTIVE` — only entities annotated `@Cacheable` + `@Cache(strategy = READ_WRITE)` participate; we cached two classes: **reference data** (`PipelineStage`, `CreditCost` — read-heavy, write-rarely, correctness tolerant to seconds of staleness) and **hot aggregate counters** (org-level lead stats). Everything transactional-critical (leads, deals, balances) is deliberately NOT L2-cached: `READ_WRITE` L2 on money-adjacent entities adds stale-read risk across JVMs for a gain Redis already gives us more controllably.

The multi-node subtlety I explain: L2 caches are per-JVM; with 3 app instances, a stale window exists unless you add invalidation broadcast (Ehcache RMI/JGroups replication) — that’s a distributed-consistency project in itself. Our call: per-JVM L2 for TRUE reference data (version-checked on read: Hibernate validates against the table’s timestamp on transactional read — staleness bounded), Redis for everything else with explicit TTLs and explicit invalidation events. Query cache: disabled — `hibernate.cache.use_query_cache` interacts with first-level cache identity and gives subtle duplicate/stale results; our query caching is at the service layer (Spring `@Cacheable` on Redis) where keys and TTLs are explicit and debuggable.

The numbers that justify the design: reference tables get ~40k reads/min across instances; L2 (in-memory) serves them at ~0 DB load with < 1µs reads vs ~300µs round-trip to Redis — a real win with zero staleness concerns (table timestamps change once a quarter). Lead entities would get a ~30% DB-load reduction but introduce cross-JVM staleness on the exact entity where support teams need truth — bad trade, and I can defend that decision line by line in an interview, which is the actual skill being tested: not “did you turn on L2”, but “do you know what it costs”.

KEY TERMS TO MENTION: - `SELECTIVE` shared-cache mode + `@Cache(READ_WRITE)` on reference entities only - Per-JVM L2 → staleness across instances (replication cost awareness) - Table-timestamp validation bounding staleness - Query cache disabled (identity/staleness hazards) - Redis at service layer for transactional-critical caching (explicit keys/TTLs)

FOLLOW-UP QUESTIONS: 1. A reference row changes — how do instances converge? 2. When would you switch to cluster-replicated L2?

FOLLOW-UP ANSWERS: 1. Hibernate’s timestamp-based validation: each transactional read checks the region’s last-update timestamp against the table’s — a write anywhere invalidates the cached entries for all instances on their NEXT read (no stale-after-write reads within a transaction). Cache writes happen after commit, so the window is bounded to in-flight transactions. Plus our admin UI bumps a version key we can clear manually in < 10s across the fleet. 2. When reference data grows to multi-MB working sets where Redis RTT dominates (or cross-region), I’d add replicated L2 (JGroups/Terracotta) — but only with the monitoring to see hit rates AND staleness incidents per region. The upgrade trigger is

measured: DB load on reference tables > 5% of total and Redis RTT visible in p99 — neither true today.

RED FLAGS TO AVOID: - Caching everything L2 “for speed” (staleness on money entities) - Not knowing L2 is per-JVM (multi-instance answer required) - Query cache enabled blindly (classic subtle-bug generator)

Q5-13: How did you optimize slow queries (process and tooling)

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA / PostgreSQL

ANSWER: Process first: query optimization is driven by data, not vibes — every API controller emits a Micrometer timer; anything over its budget creates a trace with SQL timings (datasource-proxy wraps JDBC logging per-statement counts and durations into the trace), and a weekly “top 10 queries by total time” review picks targets. Tooling sequence for a target: reproduce with real-shaped data (production snapshot in staging — plans lie on empty tables), `EXPLAIN (ANALYZE, BUFFERS)` the SQL Hibernate actually ran (`show_sql` is noise; we log bound-parameter SQL via datasource-proxy), then fix by category.

The categories, each with a real case: **(1) N+1** — leads list issuing one activities query per lead (61 queries/page); fix: `@EntityGraph(attributePaths = {"activities"})` or JOIN FETCH where the graph is bounded; batch fetching (`hibernate.default_batch_fetch_size: 32`) as the global safety net turning N+1 into N/32 + IN-queries. **(2) Missing/incorrect index** — `WHERE org_id = ? AND status = 'HOT' ORDER BY created_at DESC` seq-scanning 2M rows; fix: composite (`org_id, status, created_at DESC`) matching filter-and-order; verified via `pg_stat_user_indexes` `idx_scan` delta after deploy. **(3) ORM-generated overkill** — `save()` on a large aggregate flushing every managed entity or UPDATE touching all columns; fixes: `@DynamicUpdate` for wide tables, DTO projections (`@Query("select new com.acq.dto.LeadRow(...)")`) for read paths — never load an aggregate to read two columns. **(4) DB-side work in app memory** — a Java loop aggregating 50k activities; replaced with a single GROUP BY window-function query — 40s → 300ms. **(5) Parameter-sniffing plans** — the same query shape flipping between index-scan and seq-scan on data skew; fixed with partial indexes for the hot predicate or `plan_cache_mode=force_custom_plan` for that statement.

Verification closes the loop: before/after p95 from the traces, `pg_stat_statements` delta, and a regression guard — the slowest-10 dashboard would have caught each of these within a week instead of a user report.

KEY TERMS TO MENTION: - Trace-driven targeting (Micrometer + datasource-proxy SQL timing) - `EXPLAIN (ANALYZE, BUFFERS)` on production-shaped data - `@EntityGraph` / batch-fetch-size / DTO projections / `@DynamicUpdate` - Composite index design matching filter + ORDER BY - `pg_stat_statements` + slowest-10 review as regression guard

FOLLOW-UP QUESTIONS: 1. Why is EXPLAIN on staging with real data mandatory? 2. How do you detect N+1 automatically?

FOLLOW-UP ANSWERS: 1. Plans depend on statistics, data distribution, and cache state — the same query on an empty dev table shows index-scan (fast) while production seq-scans (2M rows, dead tuples).BUFFERS output also shows cache hit ratios you can't fake. Reproducing on a production snapshot is the only way the EXPLAIN describes YOUR problem, not a hypothetical one. 2. datasource-proxy counts statements per request; a threshold rule (any request > 20 statements or any endpoint whose statement count scales with result size) fires a warning trace. Also a test-side detector: a JUnit extension asserting fixture-list endpoints stay under a query budget — N+1 regressions fail CI, not users.

RED FLAGS TO AVOID: - "Add an index" reflex without reading a plan - Optimizing without production-shaped data - No post-fix verification (p95 before/after)

Q5-14: How did you implement soft deletes

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: Soft deletes exist because CRM data is relational memory — deleting a lead that has activities, messages, and deals cascades into losing business history and breaking foreign references. Mechanism: `deleted_at TIMESTAMPTZ NULL (+ deleted_by UUID` for accountability) on user-data tables; visibility enforced at TWO layers. Repository layer: every read method goes through Specifications that append `deletedAt IS NULL` — we made this structural with a base `SoftDeleteSpecification` and by Hibernate 6.4's `@SoftDelete` annotation (strategy DELETED, column `deleted_at`) on newer entities, which pushes the filter into Hibernate itself so even ad-hoc HQL respects it. Unique constraints interact with soft delete: `email UNIQUE` on users would block re-signup of a deleted user's email — solved with partial unique indexes (`CREATE UNIQUE INDEX ... WHERE deleted_at IS NULL`), the Postgres feature that makes soft deletes clean.

Hard-delete lifecycle still exists — soft-deleted rows age out per retention policy (leads 90 days in trash, then a `@Scheduled` job hard-deletes in batches with `pg_replication` awareness; PII columns are scrubbed on day 1 of trash to satisfy GDPR "deleted" semantics while keeping the relational shell for aggregates). Restore is a product feature ("Trash" tab → restore) — trivially `UPDATE ... SET deleted_at = NULL`, which is the entire UX payoff of the pattern. References FROM other entities use the id regardless of deletion (history stays intact — a deal referencing a trashed lead renders the name grayed out), which is why we never cascade real deletes on business tables.

The trade-offs I name proactively: every query carries the predicate (index design must include `deleted_at` in partial indexes, not as a normal column — low-selectivity garbage in b-trees), ORM defaults can leak deleted rows if someone writes raw SQL (code review + a query-coverage test asserting list endpoints filter), and counting queries must agree on scope (a "2,000 leads" dashboard vs trash-inclusive count bug we hit once and fixed by centralizing count logic in the same specification).

KEY TERMS TO MENTION: - `deleted_at / deleted_by` + two-layer visibility (specifications / `@SoftDelete`) - Partial unique indexes (`WHERE deleted_at IS NULL`) - Retention job: PII scrub

day-1, hard-delete day-90 (GDPR + history) - Restore-as-feature (trash tab) - Partial-index design + query-coverage test (leak prevention)

FOLLOW-UP QUESTIONS: 1. Why partial unique indexes instead of appending `deleted_at` to the unique key? 2. How do FK constraints coexist with soft deletes?

FOLLOW-UP ANSWERS: 1. Composite unique `(email, deleted_at)` breaks on double-delete (two rows with same email + NULL → NULLs are distinct in Postgres → duplicates allowed) and on re-delete cycles. Partial indexes scope uniqueness to live rows exactly — the semantics you mean — and stay small because trashed rows are excluded from the index. 2. FKs point at the shell row (which never physically deletes until retention, and when it does, the retention job deletes children first in the same transaction or the FK is ON DELETE SET NULL for historical references). The pattern: business integrity via soft state, physical integrity via ordered batch deletion — both explicit, neither accidental.

RED FLAGS TO AVOID: - `is_deleted` BOOLEAN (no when/who — support and compliance need the audit) - Forgetting the predicate in raw queries (data leaks into reports) - Uniqueness breaking on re-create (the classic deleted-email bug)

Q5-15: Optimistic vs pessimistic locking — when did you use each

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA

ANSWER: The decision rule: optimistic for low-contention correctness (edit collisions), pessimistic for high-contention resource consumption (spending a shared balance).

Optimistic (@Version): `deal` entities carry a `version BIGINT`; JPA increments it on every UPDATE and `WHERE version = ?` gates the write — two agents editing the same deal, the second commit gets `OptimisticLockException` → user sees “someone changed this, refresh” (we map it to 409 with the fresh copy). Cost: zero DB locks, one extra column; requirement: retry-or-conflict UX. We also use `@Version` for the import path where duplicate imports race.

Pessimistic: credit deduction was the forcing case (Q2-13’s conditional UPDATE is the ledger-side guard, but the AGGREGATE operations — monthly reset, bulk top-up, admin adjustments — read-modify-write the balance row). There we use `@Lock(PESSIMISTIC_WRITE)` on the repository method (`SELECT ... FOR UPDATE`), in a short transaction: lock row → read balance → write adjustment + ledger entry → commit, sub-10ms holds. FOR UPDATE NOWAIT on the interactive path (admin UI shows “someone else is adjusting this account” instead of hanging) and SKIP LOCKED for the batch sweeper (workers grab distinct pending entries without blocking — the job-poll pattern). The mistake I warn about: pessimistic locking + long transactions = lock queues and deadlocks — our rule is FOR UPDATE only inside transactions that do no external I/O (no HTTP, no Redis inside the lock; the LLM call case deducts via the conditional-UPDATE pattern precisely so no lock spans the provider call).

Deadlock discipline even so: consistent lock ordering everywhere (balance rows sorted by `user_id` in multi-row operations), lock timeouts (`SET LOCAL lock_timeout='2s'`) so a stuck lock fails fast instead of piling connections, and a deadlock-retry wrapper (Postgres deadlocks are retryable — catch 40P01, re-run with jitter; logged as a metric because a rising deadlock rate

is a design smell). Interview one-liner I use: optimistic buys concurrency with retry UX; pessimistic buys certainty with throughput — pick per contention profile, never per fashion.

KEY TERMS TO MENTION: - `@Version` + `OptimisticLockException` → 409 conflict UX - `@Lock(PESSIMISTIC_WRITE)` = `SELECT FOR UPDATE`, short transactions only - `NOWAIT` (interactive) / `SKIP LOCKED` (batch worker) variants - No external I/O inside pessimistic locks (LLM-call case) - `lock_timeout` + deadlock retry + ordering discipline

FOLLOW-UP QUESTIONS: 1. Why not pessimistic everywhere for simplicity? 2. How does `@Version` interact with detached entities and merge?

FOLLOW-UP ANSWERS: 1. Throughput and failure modes: every writer serializes behind row locks — a bulk campaign touching 5k balances would hold lock queues for seconds; lock timeouts convert load spikes into user-facing 500s. Pessimistic locking is also the primary deadlock generator when transaction shapes drift. Optimistic's retry cost is near-zero at our contention levels, so paying pessimistic's tax buys nothing. 2. Merge on a detached entity whose version lags the DB throws `OptimisticLockException` at flush — correct behavior, but teams confuse it with “save broken”. Our DTO-in/out pattern (entities never cross the transaction boundary) means merges are rare and version conflicts surface as explicit conflict responses, not mysterious flush errors.

RED FLAGS TO AVOID: - Using both without articulating the selection rule - `FOR UPDATE` spanning external calls (the LLM-call trap) - No `lock_timeout`/deadlock story (production freezes)

Q5-16: How did you handle large data sets (batch processing)

DIFFICULTY: Hard **CATEGORY TAG:** Hibernate/JPA / Spring Batch

ANSWER: Two meanings of “batch” — bulk DML and chunked pipelines — and both have ORM traps. **Bulk DML:** JPQL `@Modifying` `UPDATE/DELETE` statements (`UPDATE lead l SET l.stage = :s WHERE ...`) bypass the persistence context entirely — no entity loads, no dirty checking, one SQL round-trip; the trap is stale first-level cache: entities already managed in the same transaction don't reflect bulk changes, so bulk DML runs FIRST in a transaction or in its own (we standardize: bulk ops in dedicated `@Transactional(propagation = REQUIRES_NEW)` methods + `clearAutomatically = true`). For multi-million-row maintenance (retention deletes), the single statement is the enemy — 2M-row `DELETE` = long lock + WAL spike + replica lag; the batched loop (`WHERE id IN (SELECT id ... LIMIT 5000)` in a loop with `pg_sleep` between chunks) keeps each transaction milliseconds, replication healthy, and interruptible/resumable.

Chunked pipelines (imports, enrichment): Spring Batch chunk-oriented steps with `chunk(100)` — read 100 (JPA cursor/paged reader, `fetchSize` matched to JDBC), process, write in ONE flushed transaction per chunk. Hibernate settings that make this survive: `hibernate.jdbc.batch_size: 50` (driver batches INSERTs — without it, “batching” is theater with per-row round-trips), order-inserts enabled (grouped statements unlock driver batching), `EntityManager.clear()` every chunk (or `StatelessSession` for pure streaming — no dirty

checking, no L1 bloat), and IDENTITY generation OFF for batch inserts (it disables JDBC batching — sequences/TableGenerator for batch-heavy tables; a subtlety that separates people who've actually profiled this). Metrics per run: items/sec, chunk commit time, skip counts; a 1M-row import runs ~18 min with DB load flat and the API p95 unaffected (batch transactions short + read replica for reads + batch runs off-peak by scheduler).

KEY TERMS TO MENTION: - `@Modifying` bulk DML bypasses context (stale-cache discipline + `REQUIRES_NEW`) - Chunked loops for million-row maintenance (locks, WAL, replica lag) - Spring Batch `chunk(100)`, read/process/write per-chunk transaction - `jdbc.batch_size` + ordered inserts + IDENTITY-disables-batching trap - `clear()` / `StatelessSession` for context hygiene; per-run metrics

FOLLOW-UP QUESTIONS: 1. Why does IDENTITY id generation disable JDBC batching? 2. How do you make a crashed 40-minute import resumable?

FOLLOW-UP ANSWERS: 1. IDENTITY requires executing the INSERT immediately to obtain the generated key (Postgres RETURNING) — the driver can't defer and group statements it must execute one-by-one for key retrieval. SEQUENCE with `allocationSize` lets Hibernate reserve a range and batch the INSERTs. It's an invisible config until the first 100k-row import takes 4× longer than designed. 2. Spring Batch's metadata tables checkpoint every chunk commit — restart resumes from the last committed chunk (the job operator re-launches with the same job parameters; `allow-start-if-complete=false` semantics guard double-runs). For our custom job-table pipelines, the payload cursor column plays the same role. Design rule: any long job must persist progress granularly, because "restart from zero" is not a plan.

RED FLAGS TO AVOID: - Loading entities to update one column in bulk - Claiming batch processing without `batch_size/ordering/IDENTITY` awareness - Single-statement million-row DML with no lock/lag awareness

Q5-17: Explain your entity inheritance strategy

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: We have one real inheritance hierarchy — the communication record domain: `EmailMessage`, `WhatsAppMessage`, `TelegramMessage` extend `AbstractMessage` (shared: id, org, lead, direction, status, timestamps, provider ref). Strategy choice per option: **SINGLE_TABLE** (all in one `message` table + discriminator) — fastest joins, NULL-heavy columns for type-specific fields; **JOINED** (class-per-table, joins on read) — normalized but a join tax per polymorphic read; **TABLE_PER_CLASS** (concrete tables, UNION queries) — polymorphic queries are unions (index-unfriendly, rarely worth it). We chose **SINGLE_TABLE** with the discipline of keeping type-specific columns FEW (≤ 6 per subtype): messaging metadata is small (email: `smtp_message_id`, `thread_id`; whatsapp: `twilio_sid`, `media_url`), and the hot query — "all messages for a lead in chronological order" — is a single-table scan with an index, no joins, exactly what an activity timeline needs. NULL bloat is bounded by column count, and `CHECK` constraints (`discriminator = 'EMAIL' → smtp_message_id NOT NULL`) keep integrity where the ORM won't.

What I explicitly avoided: forcing inheritance where composition fits. `Deal` vs `Lead` are NOT a hierarchy (a deal HAS a lead, different lifecycles) — inheritance-shaped modeling of association-shaped reality is a classic ORM mistake; the discriminator test I apply: “is X a kind of Y in the business language, or does X reference Y?” Where inheritance WOULD have hurt: had WhatsApp messages carried 20+ unique columns, JOINED or separate tables with a common view would win — the strategy is revisit-able because repositories hide it (`MessageRepository` interface; callers don’t care about table shape). For polymorphic behavior (per-channel send/format), we use the Strategy pattern at the service layer rather than polymorphic entity methods — entities stay data, behavior lives in services, which keeps entities serializable and tests light.

KEY TERMS TO MENTION: - THREE strategies + their trade-offs (NULL bloat vs joins vs unions) - `SINGLE_TABLE` with ≤ 6 subtype columns + CHECK constraints - Discriminator test: “is-a” vs “has-a” (`Deal` $\neq$ subclass of `Lead`) - Repository layer hiding strategy (revisit-able) - Polymorphic behavior via service-layer Strategy, not entity methods

FOLLOW-UP QUESTIONS: 1. When would you switch to JOINED? 2. How do you query ONLY WhatsApp messages efficiently?

FOLLOW-UP ANSWERS: 1. If subtype columns grew ($> \sim 8$ each) making rows sparse-wide, or a subtype needs its own index-heavy access patterns (media attachments per WhatsApp row), JOINED normalizes the table and indexes. The migration is mechanical (split table + view for compatibility) — but I’d need the measured pain first; speculative normalization is as costly as speculative denormalization. 2. Discriminator is indexed: `WHERE discriminator = 'WHATSAPP' AND org_id = ?` → composite `(org_id, discriminator, created_at)` index → index-range scan, no full-table filter. The general lesson: `SINGLE_TABLE`’s polymorphic-query strength doesn’t cost single-type queries IF the discriminator participates in composite indexes.

RED FLAGS TO AVOID: - Naming strategies without the trade-off table - Modeling associations as inheritance (or vice versa) with no test - “Never use `SINGLE_TABLE`” cargo cult (or its opposite)

Q5-18: How did you implement audit fields (`createdAt`, `updatedAt`, `createdBy`)

DIFFICULTY: Easy **CATEGORY TAG:** Hibernate/JPA

ANSWER: Auditable columns are uniform across ~ 30 tables, so the mechanism is centralized — no per-entity boilerplate. An `Auditable` mapped-superclass (or `@Embedded AuditEmbeddable` — we chose embeddable so entities can also extend business base classes) carries `createdAt`, `updatedAt` (both `TIMESTAMPZ NOT NULL`), `createdBy`, `updatedBy` (UUID, nullable for system actions). Population: `@EntityListeners(AuditingEntityListener.class)` + `@EnableJpaAuditing(auditorAwareRef = "auditorProvider")` — `@CreatedDate` / `@LastModifiedDate` set timestamps (Hibernate-managed, DB-agnostic), and `AuditorAware` supplies the current actor from the `SecurityContext` (user id; API-key calls attribute the key id; scheduled jobs attribute “system:job-name” via a context decorator — attribution without

a thread-pool leak, using a scoped holder rather than a static). DB-level backstop: `DEFAULT now()` on `created_at` so even a rogue non-JPA write is stamped, and a migration guarantee that all tables got the columns (Flyway + a schema test asserting every table in the user-data set has the four columns — drift fails CI).

The `updatedAt` nuance worth mentioning in interviews: Hibernate's dirty checking updates `@LastModifiedDate` only when something else actually changed — no-change saves don't bump the timestamp (correct behavior; a naive `SET updated_at = now()` trigger would lie about change). `updatedBy` has a similar subtlety with bulk `@Modifying` updates — they bypass the context, so the audit listener never fires; our bulk-update service methods set the column explicitly in the JPQL (`SET l.updatedAt = CURRENT_TIMESTAMP, l.updatedBy = :actor`) — the kind of gap that surfaces in a "who changed this" investigation, which is exactly why we test bulk paths against the audit contract. Display layer maps audit fields into API responses where useful (lead detail shows "updated 2h ago by Sarah") — audit plumbing doubling as product UX.

KEY TERMS TO MENTION: - `@EnableJpaAuditing` + `AuditorAware` (user/api-key/system attribution) - `@Embedded` audit block + `TIMESTAMPZ NOT NULL` + DB defaults backstop - Dirty-check interplay (no-change saves don't bump `updated_at`) - Bulk `@Modifying` bypass → explicit stamping in JPQL - Schema test asserting audit columns exist everywhere (CI drift guard)

FOLLOW-UP QUESTIONS: 1. Why an embeddable over a mapped superclass? 2. How do you audit DELETE given rows disappear?

FOLLOW-UP ANSWERS: 1. Java single inheritance: entities often need to extend a domain base (or nothing); an embeddable composes without spending the "extends" slot and can be unit-tested as a value object. Superclass works too — it's a style call; the non-negotiable is ONE mechanism, not which one. 2. Deletes are audited at the EVENT level, not the row level: the `AuditAspect` logs DELETE actions (actor, id, snapshot of key fields) to the append-only `audit_event` table before the transaction commits — the row's own tombstone is the soft-delete columns (`deleted_by / deleted_at`), and hard deletes (retention) are batch jobs whose runs are themselves audited events. Two layers: state on the row, history in the event log.

RED FLAGS TO AVOID: - Hand-written `setCreatedAt(new Date())` in every service (drift + misses) - `TIMESTAMP` without timezone or JS `Date` -vs- instant confusion - No story for bulk-update audit gaps

Q5-19: Explain persist vs merge vs detach (and save vs saveAndFlush)

DIFFICULTY: Medium **CATEGORY TAG:** Hibernate/JPA

ANSWER: `persist(entity)`: register a TRANSIENT entity with the current persistence context — INSERT at flush, the SAME instance becomes managed (identity set if IDENTITY). It fails on a detached entity (`NotSerializableException`-adjacent `EntityExistsException` / detached passed to persist) — persist is "make new thing managed, in place".

`merge(detached)` : copy the detached entity's state onto a MANAGED instance (SELECT to load current row if present, then diff-copy); returns the NEW managed instance — the original detached object is NOT managed afterward (the #1 interview trap: `merged = em.merge(detached); detached.setFoo(x)` does nothing). Merge is for reattaching state that left the context (HTTP round-trip, cross-service DTO→entity rebuild); our architecture barely uses it because DTOs go in and services load-then-mutate managed entities — merge survives only in a couple of import paths.

`detach(entity)` / `evict` : remove from the context — dirty checking stops; useful in batch loops between clears, or to opt a heavy entity out of snapshot cost. `clear()` detaches EVERYTHING (batch hygiene); `close()` (context end) detaches implicitly. `remove()` schedules DELETE (must be managed; removing a detached entity needs merge or a fresh reference). Then the Spring Data pair: `save()` = persist-if-new else merge (a convenience with merge semantics for non-null ids — the SELECT-before-INSERT probe covered in Q5-9), and `saveAndFlush()` = save + immediate flush — needed when subsequent code IN THE SAME transaction must see the row via JPQL/native SQL (auto-flush covers native-query overlaps only sometimes; explicit flush is the deterministic choice) or when you need generated keys/DB-side triggers' effects before continuing. Outside that, `save()` + commit-time flush is right — flushing eagerly per save is a perf smell (multiple round-trips where one would do).

The state-model summary I draw on whiteboards: persist = transient→managed (same instance); merge = detached→managed (new instance, state copied); detach/evict = managed→detached; remove = managed→removed. If a candidate can place every method on that diagram, they own the persistence context.

KEY TERMS TO MENTION: - persist: same-instance transient→managed; fails on detached - merge: state copy onto a NEW managed instance (return value trap) - detach/evict/clear in batch hygiene - `save()` = persist-or-merge + the IDENTITY/assigned-id probe behavior - `saveAndFlush` only for same-transaction visibility needs

FOLLOW-UP QUESTIONS: 1. What happens if you modify a managed entity and then call `save()` on it? 2. `merge()` issues SELECT — can you avoid it?

FOLLOW-UP ANSWERS: 1. Nothing extra — the entity is already managed; dirty checking will emit the UPDATE at flush regardless, and `save()` (merge path for managed instance) is a no-op on a managed instance's identity. It's ceremony, not a bug — though it misleads readers into thinking “save” did something. Code review marks it noise. 2. With a `@Version`-carrying entity and correct id/version set, merge still SELECTs to diff. Avoiding the SELECT means not using merge: load via `getReferenceById` (lazy proxy, no SELECT) inside a transaction and set fields — flush UPDATES by dirty checking; or bulk JPQL when the change is scalar and unconditional. That's our default service pattern: load (or reference) → mutate → commit; merge is the exception path.

RED FLAGS TO AVOID: - Thinking merge manages the original instance (the classic bug) - `saveAndFlush` sprinkled everywhere “to be safe” - persisting detached entities / removing detached entities confusion

Q5-20: How did you handle circular references in entity serialization

DIFFICULTY: Easy **CATEGORY TAG:** Hibernate/JPA

ANSWER: Short answer: we don't serialize entities — the circular-reference problem is a symptom of doing so, and the cure is architectural. Entities model bidirectional relationships (`Lead.activities` , `Activity.lead`) because navigation is natural in the domain; JSON serialization then chases the cycle (`lead → activities → lead → ...` → `StackOverflowError` or a 4MB payload). Every fix that operates AT the serializer is a patch: `@JsonIgnore` on the back-reference (breaks cycles, but silently drops data depending on serialization direction), `@JsonManagedReference` / `@JsonBackReference` (pairwise cycle-breaker, still shape-driven), `@JsonIdentityInfo` (id-refs instead of nesting — output shape clients hate). We use `@JsonIgnore` sparingly on true back-refs and rely on the real boundary: **DTOs mapped inside the transaction** (`LeadDetailDto` with explicitly chosen `activities` collection flattened to `ActivityDto` rows, each carrying `leadId` scalar instead of the lead object). The mapping (`MapStruct` -generated, compile-time-checked) is where the response SHAPE is designed — a product decision — rather than an accident of entity structure.

This also solves the lazy-loading half of the same disease: serializing a detached entity with lazy associations throws `LazyInitializationException` mid-response (or worse, with open-session-in-view enabled, silently fires N+1 queries during serialization — OSIV being ON by default in Spring Boot is a trap I name explicitly; we set `spring.jpa.open-in-view=false` and let the absence of session-in-view force correct fetch design). So the interview answer has two layers: mechanically, cycles are broken by mapping to acyclic DTOs; fundamentally, response shape and persistence shape are different concerns and conflating them causes BOTH circular serialization AND lazy-loading disasters. The one place entities touch JSON is internal admin tooling with `@JsonIgnore` back-refs and OSIV tolerated — bounded, low-traffic, explicit.

KEY TERMS TO MENTION: - DTO boundary inside the transaction (MapStruct mapping) as the structural fix - `@JsonIgnore` / `@JsonManagedReference` as serializer-level patches (with caveats) - `open-in-view=false` — OSIV masks lazy-loading bugs as serialization bugs - Response shape is a product decision, not entity leakage - N+1-during-serialization (OSIV silent trap)

FOLLOW-UP QUESTIONS: 1. Why turn off open-session-in-view — isn't it convenient? 2. If forced to serialize entities, what's the least-bad setup?

FOLLOW-UP ANSWERS: 1. OSIV keeps a session (and DB connection) per REQUEST — connection held through view rendering, controller-level slow calls (an LLM retry inside a request) keep connections checked out, pool exhaustion under load; and it hides missing fetch plans (N+1 fires at render time where traces blame the wrong layer). Convenience now, unpredictability at scale — we disabled it in week one and the fetch discipline it forces (EntityGraph/JOIN FETCH/DTO) is a net win. 2. Bounded and explicit: `@JsonIdentityInfo` (id-refs, no infinite nesting), all LAZY associations either initialized or `@JsonIgnore` d, Jackson module configured to FAIL on lazy-no-session (fail fast, not silently empty), and OSIV

explicitly off with the knowledge documented. It's acceptable for admin tooling; it would never be the public API contract.

RED FLAGS TO AVOID: - Reach for `@JsonIgnore` everywhere without understanding directionality - Not knowing OSIV exists / its costs (Spring Boot default ON) - Circular-reference "fix" that changes API shape invisibly

CATEGORY 6 — POSTGRESQL AND DATABASE QUESTIONS

Q6-1: Why PostgreSQL for AcquisitionOS

DIFFICULTY: Easy **CATEGORY TAG:** PostgreSQL/Database

ANSWER: We chose PostgreSQL 15 over MySQL and MongoDB for these reasons:

1. **JSONB columns** — we store tags, AI reasoning, settings, and audit metadata as JSONB. Postgres JSONB is binary (indexed, fast), supports GIN indexes (`CREATE INDEX ... USING GIN(tags)`), and lets us query inside JSONB (`WHERE tags ? 'hot'` or `WHERE metadata->'source' = 'google_places'`). MySQL has JSON but no GIN index (B-tree on generated columns only). MongoDB would be a different DB to operate.
2. **Full-text search** — Postgres `tsvector` + `tsquery` + GIN index gives us `WHERE to_tsvector('english', business_name) @@ plainto_tsquery('dentist mumbai')`. Good enough for our 10M leads; we'd move to Elasticsearch only at 100M. MySQL's full-text is worse; MongoDB's text search is limited.
3. **CTE + window functions** — analytics queries (leaderboards, funnel analysis, retention cohorts) need `WITH ... SELECT ... ROW_NUMBER() OVER (PARTITION BY ...)`. Postgres has the best CTE + window support. Our `DashboardService` uses CTEs heavily.
4. **Mature replication** — streaming replication (Multi-AZ), logical replication (cross-region), read replicas with sub-second lag. RDS for Postgres is rock-solid (we use Multi-AZ + 2 read replicas).
5. **ACID + strong consistency** — our credit metering needs strict transactional integrity. Postgres's MVCC + serialisable isolation is well-understood. MongoDB's transactions are slower (document-level locking).
6. **Hibernate support** — Hibernate has first-class Postgres dialect support, including JSONB (`@Type(type = "jsonb")` via Hypersistence Utils), native full-text functions, etc.
7. **Team experience** — we have experience operating Postgres in production; switching to MySQL or MongoDB would add operational risk for no benefit at our scale.
8. **RDS managed** — AWS RDS for Postgres handles backups, patching, failover, monitoring. We don't operate Postgres ourselves.

Why not MySQL: JSON support is weaker (no GIN on JSON), CTEs only since 8.0 (and optimisation is worse than Postgres's), replication is statement-based (can drift on non-deterministic functions). We considered MySQL because it's marginally faster for simple CRUD — but the JSON + CTE advantages outweigh.

Why not MongoDB: We have relational data (leads → activities → communications → deals — many-to-many). Mongo's document model fits our use case poorly. We'd end up with thousands of small collections or emulating relations in app code.

Why not CockroachDB / YugabyteDB (distributed SQL): Distributed SQL adds latency (consensus per write) for scale we don't need yet. At 10M leads we're fine on a single RDS Postgres. CockroachDB is on the table at 100M leads + multi-region writes.

KEY TERMS TO MENTION: - JSONB + GIN indexes (binary, queryable) - Full-text search (`tsvector`, `tsquery`, `plainto_tsquery`, GIN index) - CTE + window functions (`ROW_NUMBER`, `PARTITION BY`) - MVCC + SERIALIZABLE isolation, ACID - Streaming replication (Multi-AZ) + logical replication (cross-region) - Hibernate dialect support via Hypersistence Utils - RDS for Postgres — managed, Multi-AZ, automated failover - Why not MySQL: weaker JSON + CTE, statement-based replication - Why not MongoDB: relational data, document model fits poorly - CockroachDB reserved for 100M+ multi-region scale

FOLLOW-UP QUESTIONS: 1. How do you handle JSONB index performance? 2. How would you migrate to CockroachDB if you needed to? 3. What's your Postgres version strategy?

FOLLOW-UP ANSWERS: 1. GIN index on JSONB is fast for `WHERE tags ? 'hot'` (key exists) or `WHERE tags @> '["hot"]'` (array contains) — both use the index. But `WHERE tags->'name' LIKE '%foo%'` doesn't use GIN — we'd add an expression index (`CREATE INDEX idx_tags_name ON lead ((tags->'name'))`) if we needed it. We profile slow queries with `EXPLAIN (ANALYZE, BUFFERS)` and add expression indexes for the queries that need them. JSONB queries are slower than column queries — we promote frequently-queried JSONB fields to columns (e.g. `lead.source` was a JSONB field, now a column because we filter by it often). 2. CockroachDB is Postgres-wire-compatible (mostly) — we'd point our `spring.datasource.url` at CockroachDB, run the migrations, fix any incompatibilities (e.g. some Postgres-specific functions), test. The hard parts: (a) Hibernate dialect (CockroachDB has a Hibernate dialect, but it's behind the Postgres dialect); (b) JSONB (CockroachDB has JSONB but performance differs); (c) migration of 480GB of data (`pg_dump` → CockroachDB `IMPORT` or row-by-row via Kafka). We'd do it gradually — dual-write to Postgres + CockroachDB, read from Postgres, validate CockroachDB matches, then switch reads. Months of work. 3. RDS supports up to 2 major versions back from latest. We're on 15 (latest LTS). We upgrade every 18 months (RDS does the major version upgrade; we test in staging first). Minor versions are auto-applied by RDS during the maintenance window (we pick a low-traffic window). We avoid being on the very latest major (less battle-tested) — we wait 6 months after GA.

RED FLAGS TO AVOID: - “Postgres because everyone uses it” (no depth) - Not mentioning JSONB + GIN - Saying “we use MongoDB for JSON data” (Postgres JSONB is fine) - Forgetting CTE/window functions

Q6-2: What is your database schema design

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL/Database

ANSWER: ~80 tables across 6 bounded contexts. The schema is owned by **Flyway migrations** (`V<version>_<description>.sql`, checked into the repo — the schema lives in SQL, not in code annotations, so DBAs can review it in PRs). Hibernate entities map onto this schema via `@Entity`/`@Table`; we never let `hbm2ddl.auto` touch production. Each context

owns its tables; cross-context references use UUID IDs (not FK constraints) to allow future service extraction.

Core tables (see Q2-8 for full SQL): - `organization` — multi-tenant root (id, name, plan_tier, billing_email, created_at) - `user_account` — users (id, org_id, email UNIQUE, password_hash, role, auth_provider, is_active, is_trial, credits, credits_monthly, login_otp_hash, login_otp_expiry, otp_attempt_count, otp_locked_until, magic_link_token_hash, magic_link_token_expiry, mfa_secret_hash, time_zone, last_login_at, created_at, updated_at, deleted_at) - `user_session` — refresh tokens (id, user_id, refresh_token_hash, ip, user_agent, expires_at, is_revoked, created_at) - `lead` — core CRM entity (id, org_id, user_id, business_name, owner_name, email, phone, ..., stage, reply_score, conversion_score, urgency_score, tags JSONB, is_active, deleted_at, created_at, updated_at) - `lead_activity`, `lead_communication`, `lead_score`, `lead_note` — per-lead child tables - `deal` — deals (id, lead_id, user_id, proposed_price, final_price, currency, stage, status, closed_at, created_at, updated_at) - `meeting` — meetings (id, lead_id, user_id, title, start_at, end_at, status, calendar_event_id, created_at, updated_at) - `subscription` — subscription state (id, user_id, plan, billing_cycle, status, current_period_end, stripe_subscription_id, created_at, updated_at) - `payment_order` — Stripe orders (id, user_id, provider, provider_order_id, provider_payment_id, plan, billing_cycle, amount, currency, status, coupon_code, created_at) - `payment_webhook` — Stripe webhooks (id, event_id UNIQUE, event_type, payload JSONB, signature, processed, processed_at) - `credit_ledger` — append-only credit movements (id, user_id, action, cost, balance_after, ref_id, metadata JSONB, created_at) - `credit_balance` — current balance per user (user_id PK, balance, monthly, rollover, addons, updated_at) - `audit_log` — security + business events (id, user_id, action, resource, resource_id, details JSONB, ip, user_agent, created_at) - `api_key` — user API keys (id, user_id, name, env, key_hash UNIQUE, prefix, scopes JSONB, expires_at, revoked_at, last_used_at, created_at) - `notification` — in-app notifications (id, user_id, type, title, message, metadata JSONB, is_read, read_at, created_at) - `outreach_message` — AI-generated emails (id, user_id, lead_id, channel, subject, body, status, sent_at, opened_at, clicked_at, created_at)

Design conventions: 1. **UUID PKs** — `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`. Globally unique, no enumeration, supports sharding. 2. **Soft delete** — every entity has `deleted_at TIMESTAMP NULL` + `is_active BOOLEAN DEFAULT true`. We never `DELETE` (except for audit log retention); we soft-delete. Queries filter `WHERE deleted_at IS NULL`. Partial indexes keep them small. 3. **Multi-tenancy** — every tenant-scoped table has `org_id UUID NOT NULL REFERENCES organization(id)`. We use `Hibernate @Filter("tenantFilter")` to auto-add `WHERE org_id = :currentOrgId` to queries. (We added this in month 4; we'd add it from day 1 next time — see Q1-7.) 4. **Audit fields** — every entity has `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`, `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()`, `deleted_at TIMESTAMPTZ NULL`. We use `@EntityListener (AuditingEntityListener) + @CreatedDate + @LastModifiedDate` to auto-populate. 5. **Timestamps as TIMESTAMPTZ (with time zone)** — stored in UTC, converted at display time. Never use `TIMESTAMP` (without TZ) — it's ambiguous. 6. **Enums as Postgres enum types** for stable values: `stage_enum`, `deal_status_enum`, `channel_enum`. We use `CREATE TYPE stage_enum AS ENUM ('discovered', 'contacted', ...)` — type-safe, indexed. (Caveat: adding a value requires `ALTER TYPE ... ADD VALUE` — non-trivial in a migration. For

frequently-changing values, use a lookup table instead.) 7. **JSONB for semi-structured** — `tags`, `metadata`, `ai_reasoning`. GIN-indexed for query. 8. **Money as `NUMERIC(12, 2)`** — never `FLOAT` (rounding errors). For high-volume finance, `NUMERIC(18, 6)`. 9. **Foreign keys with named constraints** — `FOREIGN KEY (org_id) REFERENCES organization(id) ON DELETE RESTRICT` — we never cascade deletes (defence against accidental deletes). FK constraint named `fk_<table>_<column>` for migration clarity. 10. **Indexes** — every FK column indexed (Postgres doesn't auto-index FKs, unlike MySQL). Composite indexes for common filter+sort patterns (`(user_id, stage, created_at)`). Partial indexes for soft-deleted (`WHERE deleted_at IS NULL`). GIN for full-text + JSONB.

Indexes (examples):

```
CREATE INDEX idx_lead_user_stage_active ON lead(user_id, stage) WHERE is_active AND
    deleted_at IS NULL;
CREATE INDEX idx_lead_user_created ON lead(user_id, created_at DESC) WHERE
    deleted_at IS NULL;
CREATE INDEX idx_lead_tags_gin ON lead USING GIN(tags);
CREATE INDEX idx_lead_search_fts ON lead USING GIN(to_tsvector('english',
    business_name || ' ' || coalesce(owner_name, '')));
CREATE INDEX idx_credit_ledger_user_created ON credit_ledger(user_id, created_at
    DESC);
CREATE INDEX idx_audit_log_user_created ON audit_log(user_id, created_at DESC);
```

Partitioning (planned): `lead` table partitioned by `created_at` (monthly) at 10M rows. `lead_activity` and `lead_communication` partitioned by `lead_id` hash (10 partitions) at 50M rows. We use Postgres 15's `PARTITION BY RANGE` / `PARTITION BY HASH`.

Connection pooling: HikariCP at the app level (20 per task × 4 tasks = 80 connections to RDS), PgBouncer in transaction mode at the DB level (handles 1000+ connections from a fleet of app servers, multiplexes to 100 Postgres backend connections). We're not yet at the scale that needs PgBouncer — at 100K users we'd add it.

KEY TERMS TO MENTION: - ~80 tables across 6 bounded contexts, cross-context via UUID IDs (not FKs) - UUID PKs via `gen_random_uuid()`, soft delete via `deleted_at` + `is_active` - Multi-tenancy via `org_id` + Hibernate `@Filter("tenantFilter")` - Audit fields: `created_at`, `updated_at`, `deleted_at` as `TIMESTAMPZ`, auto-populated via `@EntityListener` - Enums as Postgres `CREATE TYPE` for stable values, lookup table for changing values - JSONB for semi-structured (tags, metadata, reasoning), GIN-indexed - Money as `NUMERIC(12,2)` (never `FLOAT`) - FK with named constraints, `ON DELETE RESTRICT` (never cascade) - Every FK column indexed (Postgres doesn't auto-index) - Composite + partial indexes, GIN for FTS + JSONB - Partitioning plan: `lead` by `created_at` monthly, `lead_activity` by `lead_id` hash (10 partitions) - HikariCP + PgBouncer (planned at 100K users)

FOLLOW-UP QUESTIONS: 1. Why no FK constraints across contexts? 2. How do you handle schema evolution? 3. How do you measure schema health?

FOLLOW-UP ANSWERS: 1. Cross-context FKs would couple the contexts at the DB level — when we extract a context to its own service, the FK becomes a cross-service reference (no constraint). We use UUID IDs + app-level validation (`LeadService.findOrFail(leadId)` throws

if not found). The trade-off: no DB-level referential integrity across contexts — we accept this for the architectural flexibility. Within a context, we use FKs (e.g. `lead_activity.lead_id REFERENCES lead(id)`). 2. **Flyway** for all migrations — SQL-first, versioned (`V<timestamp>__<description>.sql`), forward-only (we don't write down migrations), reviewed in PRs like code. Flyway runs on deploy (Spring Boot auto-runs it at startup before the app serves traffic; we gate it with `flyway.clean-disabled=true`). For risky changes (column drop, type change), we use a 3-step pattern: (a) add the new column / table (backward-compatible), (b) deploy code that dual-writes, (c) backfill existing rows, (d) deploy code that reads from new, (e) drop the old (after a few weeks of monitoring). We use `pg_repack` for online index rebuilds and table rewrites (no lock). 3. Three signals: (a) slow query log (RDS logs queries > 500ms) — we review weekly; (b) `pg_stat_user_tables` — `seq_scan` count (high = missing index); (c) `pg_stat_user_indexes` — `idx_scan = 0` (unused index = candidate for drop). We have a weekly “schema health” cron job that outputs a report; we act on it monthly.

RED FLAGS TO AVOID: - `FLOAT` for money (rounding) - `TIMESTAMP` without TZ (ambiguous) - No indexes on FK columns (slow joins) - `CASCADE` deletes (dangerous)

Q6-3: How did you design indexes for the leads table

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL/Database

ANSWER: The `lead` table has ~10 indexes, designed for the actual query patterns we run. Index design is driven by queries, not by guessing.

Step 1 — query audit: We enabled `pg_stat_statements` and listed the top 20 queries by frequency + the top 20 by avg time. Then we designed indexes for each.

The indexes:

```

-- Primary key (auto)
CREATE UNIQUE INDEX idx_lead_pk ON lead(id);

-- Most common query: list leads for a user, filter by stage, sort by created_at
CREATE INDEX idx_lead_user_stage_active ON lead(user_id, stage, created_at DESC)
  WHERE is_active AND deleted_at IS NULL;

-- Sort by created_at without stage filter
CREATE INDEX idx_lead_user_created_active ON lead(user_id, created_at DESC)
  WHERE is_active AND deleted_at IS NULL;

-- Filter by niche (admin analytics)
CREATE INDEX idx_lead_org_niche ON lead(org_id, niche);

-- Tags JSONB contains
CREATE INDEX idx_lead_tags_gin ON lead USING GIN(tags);

-- Full-text search
CREATE INDEX idx_lead_search_fts ON lead
  USING GIN(to_tsvector('english', business_name || ' ' || coalesce(owner_name,
    '')) || ' ' || coalesce(email, '')));

-- Foreign keys (Postgres doesn't auto-index FKs)
CREATE INDEX idx_lead_user_id ON lead(user_id); -- redundant with composite above
  but explicit
CREATE INDEX idx_lead_org_id ON lead(org_id); -- redundant with composite above

-- Reply score for hot leads query (sort by score, filter by user)
CREATE INDEX idx_lead_user_reply_score ON lead(user_id, reply_score DESC NULLS LAST)
  WHERE is_active AND deleted_at IS NULL;

-- Email lookup (admin: "find lead by email")
CREATE INDEX idx_lead_email ON lead(email) WHERE email IS NOT NULL;

-- Source (admin: "leads from google_places")
CREATE INDEX idx_lead_source ON lead(source) WHERE source IS NOT NULL;

```

Design principles:

- Index for the WHERE clause first, then ORDER BY.** `idx_lead_user_stage_active` filters by `user_id` + `stage` AND sorts by `created_at DESC` — one index, one query, no sort step in the executor.
- Partial indexes** (`WHERE is_active AND deleted_at IS NULL`) — the index covers only “live” rows (90% of queries). The index is ~10x smaller than a full index → faster scan, less memory, faster writes.
- Composite indexes follow the equality-range rule** — equality columns first (`user_id`, `stage`), then range/sort columns (`created_at`). Postgres can use the index for `WHERE user_id = ? AND stage = ? ORDER BY created_at DESC` efficiently.
- GIN for JSONB + FTS** — `tags` (JSONB array) gets `USING GIN(tags)` so `WHERE tags ? 'hot'` is fast. FTS gets `USING GIN(to_tsvector(...))`.
- NULLS LAST** for sort indexes — matches our query pattern (we want rows with scores first, NULL scores at the end).
- No redundant indexes** — `idx_lead_user_id` is redundant with `idx_lead_user_stage_active` (which has `user_id` as the first column); we keep it for clarity (a developer reading the schema knows `user_id` is indexed). We’d drop it if write performance suffered.
- Index FK columns** — Postgres

doesn't auto-index FKs (unlike MySQL). Joins on `user_id` / `org_id` need indexes on both sides.

What we DON'T index: - `phone`, `whatsapp`, `linkedin`, `instagram`, `facebook` — low cardinality + low query frequency. We'd add if a query pattern emerges. - `is_active`, `deleted_at` as standalone indexes — they're part of partial index predicates, not standalone query filters.

Index maintenance: - `ANALYZE lead` after large bulk loads (Spring Batch discovery jobs) — Postgres planner needs fresh statistics - `REINDEX INDEX CONCURRENTLY` if an index gets bloated (rare — we monitor `pg_stat_user_indexes.idx_blks_read`) - **No `VACUUM FULL`** — locks the table; we use `pg_repack` for online table rewrites - **Autovacuum** is on (RDS default); we tune `autovacuum_vacuum_scale_factor = 0.05` (vacuum when 5% of rows changed) on the high-churn tables (`lead_activity`, `credit_ledger`)

Index cost monitoring: Every index adds write cost (`INSERT` / `UPDATE` / `DELETE` updates the index). We monitor `pg_stat_user_indexes.idx_scan` — if an index has 0 scans in 30 days, it's a candidate for dropping. We dropped 2 unused indexes in the last audit.

EXPLAIN ANALYZE: We run `EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)` on slow queries — checks if the planner uses the expected index, the actual row count vs estimated, and the buffer hit ratio. A "Seq Scan" on a large table is a red flag — missing index or stale statistics.

KEY TERMS TO MENTION: - Query-driven index design (audit top queries via `pg_stat_statements`, design for each) - Composite indexes: equality columns first, then range/sort - Partial indexes `WHERE is_active AND deleted_at IS NULL` (10x smaller, faster) - GIN for JSONB (`tags ? 'hot'`) and FTS (`to_tsvector @@ plainto_tsquery`) - `NULLS LAST` for sort indexes matching query pattern - Index FK columns (Postgres doesn't auto-index) - `ANALYZE lead` after bulk loads (fresh statistics for planner) - `REINDEX INDEX CONCURRENTLY` for bloated indexes, `pg_repack` for online rewrites (no `VACUUM FULL`) - Autovacuum tuned (`autovacuum_vacuum_scale_factor = 0.05` on high-churn tables) - `EXPLAIN (ANALYZE, BUFFERS)` to verify index usage - `pg_stat_user_indexes.idx_scan = 0` → candidate for drop

FOLLOW-UP QUESTIONS: 1. How do you measure index effectiveness? 2. What's the cost of too many indexes? 3. How do you add an index on a huge table without downtime?

FOLLOW-UP ANSWERS: 1. Three metrics: (a) `idx_scan` — how often the index is used (from `pg_stat_user_indexes`); 0 in 30 days = unused; (b) `idx_blks_read` vs `idx_blks_hit` — cache hit ratio (high = good); (c) `EXPLAIN ANALYZE` on the query — does the planner pick the index? We review monthly. We've dropped 2 unused indexes in the last audit. 2. Every index adds write cost (`INSERT` / `UPDATE` / `DELETE` update every index). On `lead` with 10 indexes, an `INSERT` writes 10 index entries. For our insert rate (~5K leads/day), that's 50K index writes/day — negligible. The bigger cost is storage — each index is ~10-30% of the table size; 10 indexes = 100-300% of the table size in index storage. We have 480GB of `lead` data + ~600GB of indexes — acceptable on RDS. At 10M leads, we'd be more selective. 3. `CREATE INDEX CONCURRENTLY` — Postgres builds the index in two phases without an exclusive lock. Reads + writes continue during the build. The trade-off: it's slower than the non-concurrent build (which locks the table). For a 100GB table, `CREATE INDEX CONCURRENTLY` takes ~30 minutes; the non-concurrent takes ~5 minutes but blocks writes. We always use

`CONCURRENTLY` in prod. Flyway marks the migration as `-- no transaction` for `CREATE INDEX CONCURRENTLY` (it can't run inside a transaction).

RED FLAGS TO AVOID: - Indexing every column (write cost) - No partial indexes (index bloat) - `VACUUM FULL` in production (locks the table) - Not running `ANALYZE` after bulk loads (stale stats)

(The remaining PostgreSQL questions — full-text search, connection pooling (HikariCP), performance tuning, migrations in prod, partitioning, JSONB, backup + recovery, monitoring, transactions across tables, read replicas, `SELECT FOR UPDATE`, audit trails, CTE + window functions, deadlocks, logical replication, `EXPLAIN ANALYZE`, vacuum + bloat, partial indexes, GIN, pgvector, zero-downtime migrations, materialised views, failover, connection pool sizing — follow the same template. Drill on: MVCC internals, why `SELECT FOR UPDATE` vs `SELECT FOR NO KEY UPDATE`, the difference between streaming and logical replication, what `xmin` column means, why `VACUUM` is needed at all, how to read an `EXPLAIN` plan.)

Q6-4: How did you handle full-text search for lead search

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL

ANSWER: Lead search means: given free text (“dental clinic Mumbai”), match across `business_name`, `owner_name`, `city`, `tags` — with typo tolerance and fast response over 2M rows. LIKE ‘%...%’ was never an option: unindexable (seq scan per keystroke), no relevance, no typo handling. Our Postgres-native stack has three tiers. **Tier 1 — tsvector full-text:** a generated column `search_tsv tsvector GENERATED ALWAYS AS (setweight(to_tsvector('english', coalesce(business_name, '')))` with a GIN index; queries use `websearch_to_tsquery` (safe — user input never parsed as query syntax) with `ts_rank` ordering. Generated column keeps index and data in sync with zero triggers. **Tier 2 — trigram fuzzy:** `pg_trgm` GIN index on `business_name` for typo tolerance (`similarity(name, :q) > 0.35` OR `name % :q`), because FTS won't match “Denist” to “Dentist” — trigrams do. We run trigram as a fallback branch when FTS returns < 5 results, unioned and re-ranked. **Tier 3 — prefix + structured filters:** `org_id/status/stage` filters stay plain b-tree predicates combined with the text query (GIN + b-tree in one plan works fine; the planner intersects bitmaps).

Ranking is a product decision: $\text{score} = \text{ts_rank (text relevance)} \times 0.6 + \text{lead quality score} \times 0.3 + \text{recency} \times 0.1$ — computed in SQL so pagination is stable. Performance facts I quote: GIN index build on 2M rows $\approx$ 40s (built `CONCURRENTLY`), search p95 $\approx$ 45ms, index size $\approx$ 30% of table. Why not Elasticsearch at this stage: one more system (ops, sync lag, auth filtering complexity), while 95th-percentile 45ms covers UX; the trigger to move out of Postgres is written in the ADR — sustained search QPS where Postgres CPU dominates, or need for analyzers/ML ranking beyond SQL composition. Sync-based search (ES) also breaks transactional visibility (a saved lead not yet searchable) — our generated column is

transactional: a committed lead is instantly searchable, which the product relies on (“import → search → it’s there”).

KEY TERMS TO MENTION: - Generated tsvector column + GIN index + `websearch_to_tsquery` (injection-safe) - `pg_trgm` similarity for typos as fallback tier - Composite ranking (relevance + quality + recency) in SQL - Transactional visibility (committed = searchable) vs sync-lag systems - Explicit exit criteria to Elasticsearch in the ADR

FOLLOW-UP QUESTIONS: 1. Why ‘simple’ dictionary for city/tags but ‘english’ for names? 2. How do you keep the GIN index fast on writes?

FOLLOW-UP ANSWERS: 1. Stemming: ‘english’ reduces “clinics/clinic” to one lexeme (recall win on names/descriptions); city names and tags are proper nouns/labels — stemming corrupts them (“Mumbai” isn’t a word to stem, but ‘simple’ also avoids lowercase-folding surprises with mixed-language tags). Choosing dictionaries per weight-class is the difference between search that feels smart and one that feels broken. 2. GIN fastupdate on = insert-time pending list (fast writes, slower queries until cleanup); we monitor `gin_pending_list` size and autovacuum cleans it. For our write pattern (~2k leads/min peak) fastupdate ON + autovacuum tuning is the right balance; bulk imports pre-disable and re-index after. Measured, not default.

RED FLAGS TO AVOID: - ILIKE ‘%term%’ with an index claim (it seq-scans) - Raw tsquery on user input (syntax errors/DoS via malformed queries) - “We’d add Elasticsearch” with no trigger or trade-off articulated

Q6-5: How did you implement and tune connection pooling (HikariCP)

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL / Performance

ANSWER: HikariCP is Spring Boot’s default; the interview value is showing the pool is a DESIGN decision, not defaults. Sizing math I present: $\text{pool} \approx (\text{cores_effective} \times 2) + \text{spindle_guess}$ — with RDS db.m6g.xlarge (4 vCPU) and an OLTP mix, the DB’s total connection budget matters more than per-instance: $3 \text{ app instances} \times 15 = 45 \text{ app connections} + \text{migrations} + \text{cron workers} + \text{analytics} = \sim 55$, comfortably under Postgres `max_connections=200` with headroom for superuser reserved. The counterintuitive truth I explain: SMALLER pools often outperform bigger — every connection is a Postgres process; 100 active connections on 4 cores = context-switch thrash and lru churn, throughput DROPS. We load-tested 10/15/25/40 per instance: 15 won on p99 (queue-on-pool adds ~1-3ms but saves seconds of DB thrash at peaks).

Key settings beyond `maximumPoolSize`: `connectionTimeout=3s` (fail fast under pool exhaustion — a 30s default turns a DB blip into thread-pool exhaustion cascade; 3s + circuit-breaker on DB health converts overload into clean 503s), `maxLifetime=840000` (14 min, deliberately BELOW any LB/firewall idle-kill — stale connection deaths are silent p99 killers), `leakDetectionThreshold=60s` (a stack trace on any connection held > 60s — caught two bugs:

a stream not closed in an export path, and a `@Transactional` method doing an LLM call), `minimumIdle=maximumPoolSize` (fixed-size pool — no resize jitter at peaks). Validation is `JDBC4 isValid()` (no test-query overhead), and `transactionIsolation=READ_COMMITTED` explicit (defaults drift across drivers).

Monitoring closes the loop: HikariCP JMX/Micrometer metrics (active/idle/pending, usage time, connection creation time) on Grafana — the panels that matter: pending > 0 sustained (pool too small OR slow queries — investigate queries FIRST), active near max at peak (headroom check), connection creation spikes (maxLifetime churn or network flaps). The lesson that sticks: the pool is a queue in front of a shared resource — tune the RESOURCE (query performance) before growing the queue, because the DB's core count, not your pool config, is the real ceiling.

KEY TERMS TO MENTION: - Sizing: cores × 2 heuristic + DB-wide connection budget (all consumers) - Smaller pool outperforms at core saturation (context-switch thrash) - `connectionTimeout=3s` fail-fast; `maxLifetime` < idle-killers; leak detection - Fixed-size pool (`minimumIdle = max`) - Micrometer panels: pending/active/creation-time and what each diagnoses

FOLLOW-UP QUESTIONS: 1. Pool exhausted at peak — grow it or do something else? 2. How does PgBouncer change the math?

FOLLOW-UP ANSWERS: 1. Diagnose before growing: pending connections + active-at-max means consumers are HOLDING connections — the leak-detection traces and slow-query dashboard tell you which. Growing the pool moves thrash into Postgres. The sequence: find the slow query/long transaction (the LLM-call-in-transaction bug), fix it, keep pool at 15. Scale the pool only when DB CPU is NOT saturated and pending is real demand. 2. PgBouncer (transaction pooling) multiplexes many client connections onto few DB connections — changes the constraint: app pools can be large because they queue at PgBouncer, but it BREAKS session state (prepared statements need `statement_cache_size=0` / protocol-level Paxos handling, SETs, advisory locks). We didn't need it at 45 connections; the trigger would be Lambda-style fan-out or connection storms, and I know the prepared-statement landmine that comes with it.

RED FLAGS TO AVOID: - `maxConnections=100` “more is safer” - No leak detection, no answer for what pending connections mean - Ignoring the DB-side budget (instances × pool vs `max_connections`)

Q6-6: How did you handle database migrations in production with zero downtime

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL / DevOps

ANSWER: The zero-downtime migration doctrine is expand-contract, aligned with blue-green deploys: **the schema at any moment supports BOTH the old and new app version**, because during a rolling deploy both versions serve traffic simultaneously. Phase 1 (expand): additive-only migration — new table, nullable column, or backfilled column with

default; Phase 2 (deploy): code dual-writes or reads the new shape; Phase 3 (backfill): batched job over existing rows (5k/batch, off-peak, `pg_stat_replication` watched); Phase 4 (switch): reads move to the new shape; Phase 5 (contract): drop old only after traces prove zero references (a usage query on `pg_stat_user_tables` + code search + a release-cycle wait). Every step reverts cleanly — that’s the test of a true zero-downtime migration: can you undo THIS step without data loss?

Postgres-specific hazards I enumerate because interviewers probe them: **locking** — `ALTER TABLE ... ADD COLUMN` with a VOLATILE default rewrites the table (ACCESS EXCLUSIVE for the duration); Postgres 11+ fast-defaults make constant defaults metadata-only, but I still verify with `lock_timeout=2s` + `retry` wrapper on every DDL (a migration queuing behind a long transaction blocks EVERYTHING behind it — the migration-induced outage pattern); **index builds** — `CREATE INDEX CONCURRENTLY` (no table lock, can’t be in a transaction, leaves INVALID index on failure — drop and retry; a follow-up validity check in CI); **column type changes** (int→bigint) are table rewrites on large tables — plan as dual-column + dual-write; **statement behavior changes** — `NOT NULL` on a big table = full scan + lock → add as CHECK NOT VALID + VALIDATE CONSTRAINT (short lock) then promote.

Operational guardrails: DDL runs with `statement_timeout` and `lock_timeout` set locally, an advisory lock ensures only one migration process runs (Flyway handles it; we ALSO advisory-lock deploy scripts that touch schema), and every risky migration is rehearsed on a production snapshot with timing recorded — “add index on leads took 38s on snapshot” becomes “safe to run at 02:00” or “needs CONCURRENTLY”. Rollback doctrine: additive steps need no rollback; contract steps are the point-of-no-return, so they ship a release AFTER the new code is confirmed stable — never in the same deploy.

KEY TERMS TO MENTION: - Expand → deploy (dual-write) → backfill (batched) → switch → contract - Schema supports two app versions during rolling deploys - `lock_timeout`/`statement_timeout` on DDL + ACCESS EXCLUSIVE queue awareness - `CREATE INDEX CONCURRENTLY` + INVALID-index retry discipline - CHECK NOT VALID + VALIDATE for constraint adds on big tables - Snapshot rehearsal with recorded timings

FOLLOW-UP QUESTIONS: 1. A migration hangs waiting on a lock — what’s happening and what do you do? 2. How do you backfill 50M rows without breaking replication?

FOLLOW-UP ANSWERS: 1. Postgres DDL takes ACCESS EXCLUSIVE — it queues behind a long-running transaction AND every query behind the DDL queues behind it: a 30-second report query + a 2-second ALTER = an outage. With `lock_timeout=2s` the ALTER fails fast instead of joining the queue; we then find the blocking transaction (`pg_stat_activity` + `pg_blocking_pids`), kill or wait it out, retry. That’s why every DDL wrapper in our repo has the timeouts baked in. 2. Batched writes (5-10k rows/batch) with a sleep between batches keeps WAL generation under the replica’s apply rate — watch `pg_stat_replication` `replay_lag` live during the job and throttle on it. Trigger-based or logical-decoding CDC backfills are the alternative when cutover must be instant, but batched + lag-watch covers 95% of backfills without new infrastructure.

RED FLAGS TO AVOID: - Schema deploy + code deploy assuming perfect simultaneity - DDL without lock_timeout (the classic self-inflicted outage) - Contract (drop) in the same release as the new code

Q6-7: How did you implement partitioning for large tables

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL

ANSWER: The tables that outgrow single-table Postgres are the append-heavy history tables: `lead_activity` (~40M rows/year), `communication` (similar), `audit_event`. At ~50M rows, even well-indexed queries degrade (b-tree depth, vacuum time, index bloat), and retention deletes of 30M rows are impossible (lock + WAL + vacuum debt). Native **declarative range partitioning by month** on `created_at`: each month a partition, queries with a time predicate prune to relevant partitions (plans show “Subplans Removed”), and retention becomes `DROP TABLE` — O(1) metadata operation replacing a 3-hour DELETE batch; WAL, vacuum, and bloat all shrink with partition size.

Design decisions worth stating: partition key **MUST** be in every hot query (we enforce via code review + a query-coverage test — a query without the time range scans all partitions and is worse than before); PK/uniques must include the partition key (`(id, created_at)`) — a real modeling cost, meaning id-only lookups need the `created_at` too (we embed `created_at` in cursors and secondary lookup paths); local indexes per partition (monthly `created_at` + `(org_id, created_at)` composites) — global indexes don’t exist in Postgres, so cross-partition uniques are impossible by design (acceptable: activity rows are append-only facts, uniqueness is per-event id generated with the row). Partition management is automated: a `pg_partman`-style scheduled job pre-creates next month’s partitions (a missing partition = insert failure = page), and detach+archive moves 13-month-old partitions to cold storage (S3 Parquet via a read-model export) before dropping.

The migration story (existing 20M-row table): create partitioned twin → dual-write via trigger/decode or app-level → backfill historical months in parallel → switch reads → drop original. We rehearsed on a snapshot; total cutover ≈ 3 days with zero downtime. When NOT to partition I also answer: tables under ~20M rows (complexity tax > benefit), tables whose access pattern isn’t time-shaped (partition by `org_id` hash **ONLY** for parallelism — different tool), and anything needing cross-partition unique constraints (it can’t work — pick id generation or application-level guarantees).

KEY TERMS TO MENTION: - Declarative RANGE partitioning by month on append-only history tables - Partition pruning requires the key in every query (enforced by tests) - PK must include partition key — modeling cost acknowledged - Retention = DROP partition (vs DELETE batches); pre-created partitions - Cutover via partitioned twin + dual-write + backfill

FOLLOW-UP QUESTIONS: 1. Why not hash-partition by `org_id` instead — multi-tenancy is your shape? 2. What breaks first **WITHOUT** partitioning at 200M rows?

FOLLOW-UP ANSWERS: 1. Hash-by-org fits isolation (per-org queries prune to one partition) but **NOT** retention (orgs are immortal, months are not) and creates hot-partition

risk (one whale org = one overloaded partition). Our dominant maintenance axis is TIME (drop old data), so time wins; the org filter is served by indexes within partitions. If per-org physical isolation ever becomes a compliance requirement, that's the natural-tenant-sharding conversation — a different project. 2. In order: vacuum takes hours (dead tuples from status updates accumulate), index bloat doubles index size and degrades plans, retention deletes can't complete (lock queues), and autovacuum falls behind permanently — the table enters a death spiral of ever-slower maintenance. It's rarely the SELECTs that die first; it's the housekeeping.

RED FLAGS TO AVOID: - Partitioning a 5M-row table (complexity theater) - Unique constraints without the partition key (impossible — must know this) - No automated partition pre-creation (inserts start failing at month boundary)

Q6-8: How did you handle JSON data in PostgreSQL (JSONB)

DIFFICULTY: Medium **CATEGORY TAG:** PostgreSQL

ANSWER: JSONB has one legitimate role in our schema: **schema-flexible metadata** — data whose shape varies per row or evolves faster than release cadence. Concrete uses: `lead.tags` (arbitrary key-value enrichment from scrapers), `integration.sync_state` (per-provider cursor blobs — Gmail sync tokens differ from Twilio's), `audit_event.before_after` (field diffs whose columns vary by entity type), `notification.payload` (channel-specific render data). The pattern I enforce: JSONB columns hold SECONDARY attributes — anything that becomes a first-class query/filter/join target gets promoted to a real column (we've promoted 4 fields out of JSONB so far, each a 3-step expand-contract migration). "Everything JSONB" is the schema-death spiral I actively prevent.

Correct usage mechanics: `JSONB` (decomposed binary, indexable) never `JSON` (text with validation only, re-parsed per access); default `'{}'::jsonb NOT NULL` (null-handling uniformity); writes are read-modify-write at the SQL level with the `||` concat operator (`UPDATE lead SET tags = tags || :delta` — atomic merge, no lost updates under concurrency, since read-modify-write in Java would need a row lock). Indexing matches access: `GIN` with `jsonb_path_ops` (smaller, faster for containment `@>` — our dominant predicate: `WHERE tags @> '{"industry": "dental"}'`), and expression indexes for the two hot scalar paths (`(tags->>'size')` where we filter employees). Validation: a `CHECK (jsonb_typeof(tags) = 'object')` plus application-level schema validation (the DTO layer validates the shape — Postgres can't enforce inner structure portably, so the contract lives in code with tests).

What I deliberately did NOT do: store entity payloads as JSONB "for flexibility" (no FK integrity, no migration tooling, every query is a path expression), use JSONB for relationships (arrays of ids — no referential integrity, no index-correct joins), or let query patterns grow on unindexed paths (monitor `pg_stat_statements` for `->>` filters lacking expression indexes — they seq-scan). The interview frame: JSONB is a tool for the SCHEMA-FLEXIBLE 5%, with the discipline to promote the 5% back to columns as soon as it becomes load-bearing.

KEY TERMS TO MENTION: - JSONB for genuinely variable metadata (tags, sync states, diffs) - Promotion rule: query/join target → real column (expand-contract) - `||` atomic merge (no app-level read-modify-write races) - GIN `jsonb_path_ops` for `@>` + expression indexes for hot paths - CHECK + DTO-layer validation; never relationships in JSON

FOLLOW-UP QUESTIONS: 1. `jsonb_path_ops` vs default GIN ops — the trade-off? 2. A JSONB field now has 3 hot query paths — migrate or index?

FOLLOW-UP ANSWERS: 1. `jsonb_path_ops` indexes only containment queries (no existence `?` operator, no full-key queries) but produces much smaller, faster indexes for `@>` workloads. Default ops index every key/value pair (bigger, supports more operators). Ours is containment-dominated, so `path_ops`; the choice is workload-shaped, not fashion. 2. Decision test: is the path STRUCTURAL now (appears in most rows, drives joins/unique logic) or occasional? Structural → promote to column (typed, FK-able, statistics-visible — the planner estimates JSONB paths badly). Occasional-but-hot → expression index is proportionate. Our promoted fields all crossed “structural” — the discipline is honest bookkeeping of which world a field lives in.

RED FLAGS TO AVOID: - JSONB as the default modeling tool (schemaless-everything antipattern) - Storing relations as id-arrays in JSON - Unindexed `->>` filters appearing in `pg_stat_statements` top-10

Q6-9: What is your backup and recovery strategy

DIFFICULTY: Medium **CATEGORY TAG:** PostgreSQL / DevOps

ANSWER: The strategy is expressed as two numbers I commit to: **RPO ≈ 5 minutes** (max acceptable data loss) and **RTO ≈ 30 minutes** (max time to restore service) — because backup design without RPO/RTO is just storage. Mechanism: nightly **base backups** (`pgBackRest` to S3, checksummed, encrypted) + continuous **WAL archiving** (asynchronous, 5s `archive_timeout`) enabling **PITR** — restore to any point in the last 30 days. Retention tiers: 7 daily, 8 weekly, 12 monthly (compliance), with S3 object-lock (governance mode) so even compromised credentials can't delete backups inside the window — the ransomware-era requirement interviewers increasingly probe.

A backup is a rumor until restored — the operational core is **automated verification**: weekly, a restore job spins a throwaway instance from the latest base backup + WAL replay to a random timestamp, runs checksum + row-count + app-level assertions (login works, lead counts match source snapshot), and posts a pass/fail report; failed verification pages on-call with the same severity as a failed backup. Cross-region copies (S3 cross-region replication) cover region loss; credentials for the backup store are separate from the production DB credentials (blast-radius isolation).

Recovery drills happen twice a year as game days: simulate “DB dropped at 14:32” — restore to 14:31, replay app traffic against it in a shadow environment, measure actual RTO (our last drill: 22 minutes to restored service, 41 minutes to full app validation — the gap told us to automate the app-side smoke suite into the runbook). Application-level recovery

complements DB recovery: the append-only credit ledger means even a 5-minute WAL gap can be reconciled manually from provider webhooks (Stripe replay) — layered recovery, not single-point trust. Secrets, env config, and the infra (Terraform state) are all in backup scope — a DB without its secrets is a paperweight.

KEY TERMS TO MENTION: - RPO 5 min / RTO 30 min (explicit, drilled numbers) - pgBackRest base + WAL archiving → PITR (30-day window) - Automated weekly restore-verify (row counts, app smoke) - S3 object-lock + cross-region + separated credentials - Semi-annual game days with measured (not assumed) RTO

FOLLOW-UP QUESTIONS: 1. WAL archiving fails silently for 6 hours — what contains the damage? 2. Why object-lock on your own backups?

FOLLOW-UP ANSWERS: 1. Monitoring is the first control: archive-lag metric + a canary that writes a marker row every minute and verifies its archival (detects silent pipeline death, not just errors). Damage window shrinks from 6h to ~15 min detection. Recovery impact: worst case we lose to the last archived WAL (RPO breach) — the credit-ledger reconciliation plus provider webhook replays recover the money-critical state manually; the runbook has that reconciliation procedure written BEFORE the incident. 2. Ransomware and credential compromise: modern worst-cases delete backups BEFORE encrypting production. Object-lock (governance) makes deletes impossible inside the retention window even with admin S3 credentials; combined with separate backup credentials, the attacker must compromise two independent systems in the same window. It's cheap insurance against the scenario that ends companies.

RED FLAGS TO AVOID: - Backups with no restore testing (the #1 real-world failure) - No RPO/RTO numbers ("we back up daily") - Same credentials/region for prod and backups

Q6-10: How did you monitor database performance

DIFFICULTY: Medium **CATEGORY TAG:** PostgreSQL / Observability

ANSWER: Database monitoring has four layers, each answering a different question. **Layer 1 — health:** RDS/CloudWatch basics (CPU, connections, disk, replication lag) with alarms on saturation, not averages (p95 CPU over 10 min, lag > 30s). **Layer 2 — query performance:** `pg_stat_statements` (shared library, not an extension you enable after the fire) is the workhorse — `total_time`, `mean_time`, `calls`, `rows per normalized query`; a weekly review script pulls top-10 by total time and by mean time into a Slack digest; this list drove most of the optimizations in Q5-13. **Layer 3 — operational internals:** the stats views tell the maintenance story — `pg_stat_user_tables` (`seq_scan counts` → missing indexes; `dead tuples` → vacuum debt), `pg_stat_user_indexes` (`idx_scan=0` → unused index candidates — every index costs write latency, so pruning is performance work), `pg_stat_activity` snapshots during incidents (`wait events` — `Lock:transactionid` vs `I/O:DataFileRead` tell two different stories immediately), `pg_locks` for lock chains. **Layer 4 — the app's view:** per-endpoint SQL timing via datasource-proxy wrapped in traces (Micrometer → Grafana tempo-style), so "the leads API is slow" resolves to the exact statement in one click — the DB dashboard and the app traces share request-id correlation.

Alert thresholds that earned their keep: long-running transactions > 5 min (they block vacuum → bloat spiral — the silent killer), replication lag > 60s (stale reads on replica-routed traffic), connection usage > 80% (pool math breaking), cache hit ratio < 97% (working set exceeding memory — the pre-症状 of everything getting slower). Two dashboards, not twenty: an “is the DB healthy” exec view (5 panels) and an incident-deep-dive view (stats views + waits + top queries) — dashboards nobody opens are debt. The weekly review meeting (15 min, top-queries digest + schema-health output) is where monitoring becomes optimization — dashboards without a review cadence are decoration.

KEY TERMS TO MENTION: - `pg_stat_statements` top-10 digest as the optimization driver - Wait-event triage via `pg_stat_activity` (lock vs IO tells the story) - Long-transaction alert (vacuum-blocking spiral prevention) - App-side per-endpoint SQL timing correlated with traces - Two-purposeful-dashboards + weekly review cadence

FOLLOW-UP QUESTIONS: 1. Cache hit ratio is 99% but queries are slow — where do you look? 2. How do you catch a query that got slow only for ONE org’s data shape?

FOLLOW-UP ANSWERS: 1. Hit ratio is an average — it hides statement-level reality. Order: `pg_stat_statements` mean-time outliers (which statements), then their EXPLAIN (ANALYZE, BUFFERS) (plans + whether reads hit OS cache vs shared buffers), then wait events (IO waits with high hit ratio = working set churn per query, not global). Also check bloat (dead tuples) — 99% hit on a 5× bloated table is still 5× the pages. Averages comfort; distributions and per-statement data diagnose. 2. Per-statement stats normalize queries, so an org-specific plan flip hides in the mean. The trace layer saves us: per-endpoint, per-tenant SQL timings in traces — filter by org and the slow shape appears. Structural fix for the pattern: partial indexes for the hot org predicate, or query-plan hardening (`force_custom_plan` for that statement), or the org gets its own read path — per-tenant plan variance is a real multi-tenant DB disease and the monitoring has to be tenant-sliced to see it.

RED FLAGS TO AVOID: - Only infra metrics (CPU) with no `pg_stat_statements` - No long-transaction alerting (vacuum spiral invisible until outage) - Dashboards with no review cadence (monitoring as decoration)

Q6-11: How did you handle transactions across multiple tables (and their anomalies)

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL / Transactions

ANSWER: Multi-table writes cluster into three patterns in our system, each with its own discipline. **Pattern 1 — single-service ACID:** the majority (lead + activity + score update in one `@Transactional` service method; default `READ_COMMITTED` — Postgres MVCC gives statement-level snapshots; anomalies at this level are lost updates via read-modify-write, solved by conditional UPDATES or SELECT FOR UPDATE, covered in Q5-15). The rule that keeps it sane: one transaction per use-case, ALL tables touched by one datasource (we never split a business transaction across two DBs — that’s Pattern 3 territory with real costs). **Pattern 2 — transaction + async side effects:** events published for notifications/analytics use `@TransactionalEventListener(AFTER_COMMIT)` — the DB transaction stays atomic;

side effects fire only after durability (a notification referencing an uncommitted lead is a support ticket generator). The inverse hazard: AFTER_COMMIT tasks run OUTSIDE the original transaction — if they need their own writes, they open a new transaction and must be idempotent (retry-safe) because they can fail independently.

Pattern 3 — cross-context writes that CAN'T share a transaction: e.g. lead stage change (leads schema) + Stripe charge (external, HTTP). The pattern is **local transaction + outbox + compensation**: write the stage change + an outbox row (payment instruction) in ONE local transaction; a relay processes the outbox (calling Stripe, marking outcome); failures produce compensating entries or retries — at-least-once with idempotency keys at each step. This is Saga-lite: we deliberately avoided full distributed-Saga frameworks (orchestration/choreography machinery) because our cross-context flows are few; the outbox table + a state machine column per flow gives auditability (every step is a row) without an orchestration server. Anomaly protections standardized across patterns: all money-adjacent multi-row writes carry idempotency keys (UNIQUE), lock ordering is consistent (Q5-15), and isolation escalations (**REPEATABLE_READ** for the report-vs-write races) are per-method exceptions with a comment explaining the anomaly being prevented — blanket SERIALIZABLE would trade throughput for safety we don't need.

KEY TERMS TO MENTION: - One use-case = one transaction, one datasource (no accidental distributed) - AFTER_COMMIT events + idempotent consumers (transaction boundary vs side effects) - Outbox pattern for cross-context flows (local atomicity + relay) - Conditional UPDATE / FOR UPDATE for READ_COMMITTED anomalies - Per-method isolation escalation with documented anomaly rationale

FOLLOW-UP QUESTIONS: 1. Why is READ_COMMITTED enough — when would you need SERIALIZABLE? 2. Outbox relay crashes after calling Stripe but before marking the row — then?

FOLLOW-UP ANSWERS: 1. Our anomalies are lost-update shaped (concurrent balance/deal edits) — conditional UPDATE fixes that at READ_COMMITTED with no snapshot-retry machinery. SERIALIZABLE earns its keep when invariant preservation spans MULTIPLE rows read together (e.g., sum-of-parts budgeting) where predicate locks beat manual guarding; we have one reporting path using REPEATABLE_READ for consistent snapshots. Rule: escalate per-proven-anomaly, with the anomaly documented at the method. 2. The Stripe call carries an idempotency key = outbox row id: relay retries safely (Stripe dedupes), and our reconciliation job matches charge outcomes to outbox rows hourly — the “crashed between two systems” window is exactly what idempotency keys exist for. Without the key, you'd need distributed transactions; with it, at-least-once + dedupe is equivalent and simple.

RED FLAGS TO AVOID: - @Transactional on the controller or a 10-step method (scope sprawl) - Side effects inside transactions (HTTP/LLM calls holding locks) - Claiming two-phase commit for cross-service flows (nobody does this in 2026 microservice practice — and monolith shouldn't need it either)

Q6-12: What is your approach to read replicas

DIFFICULTY: Medium **CATEGORY TAG:** PostgreSQL

ANSWER: Read replicas solve read-throughput and analytics isolation, and they introduce one tax: replication lag — stale reads. Our routing philosophy: route by CONSISTENCY requirement, not by habit. **Consistency-required reads stay primary:** anything in a read-after-write flow (save lead → list shows it — the user's own expectations), auth (session/permission checks), payments/billing (money math), and any read inside a write transaction. **Lag-tolerant reads go replica:** analytics dashboards (a 5-second-old chart is correct), admin list views, exports, the search-index backfill, ML scoring batch reads. Implementation: two read paths at the REPOSITORY layer — `@Transactional(readOnly = true)` + replica routing via a `RoutingDataSource` keyed on a `@ReadOnlyRoute` annotation (read-only also gets Postgres `default_transaction_read_only` belt-and-suspenders on that connection pool); primary-only is the DEFAULT, replica is opt-in per method with a comment stating the staleness tolerance — explicit opt-in prevents the classic bug where someone routes a permission check to a lagging replica and a user briefly loses access.

Lag management is operational, not theoretical: replay-lag metric with alerting (> 30s pages), the routing layer refuses replica routing when lag exceeds the method's declared tolerance (fallback to primary — correctness over throughput), and dashboards mark replica-served panels ("data may lag ≤ 60s") so support doesn't chase ghosts. Two extra wins from having a replica: heavyweight reporting queries (weekly digests, CSV exports of 500k rows) stopped touching primary CPU entirely — the p95 of OLTP paths dropped ~12% the week we routed exports; and PITR backstops (streaming replication is NOT a backup — `wal_level` replica streaming protects against hardware loss, not against a bad DELETE — the DELETE replicates too; that distinction is a favorite interview probe and the reason our backup story (Q6-9) is independent).

KEY TERMS TO MENTION: - Route by consistency requirement: read-after-write/auth/money → primary - `RoutingDataSource` + explicit opt-in annotation with declared tolerance - Lag alerting + fallback-to-primary on tolerance breach - Exports/analytics isolated → OLTP p95 improved ~12% - Streaming replication ≠ backup (bad DELETE replicates too)

FOLLOW-UP QUESTIONS: 1. A user complains their new lead "disappeared" — replica routing bug? 2. When do you add a second replica vs scaling primary?

FOLLOW-UP ANSWERS: 1. First check routing annotations on the list endpoint (regression), but the more common cause is a frontend filter/state bug — the diagnosis discipline: reproduce with the request-id (traces show which pool served the read), check lag at that timestamp. Prevention: the read-after-write flows are pinned primary by design; a "disappeared lead" bug class we've engineered out, which is why I'd verify the frontend before blaming the DB layer. 2. Second replica when: replica CPU saturates from analytics growth (its own scaling axis) or when we need geo-locality for a regional team. Scaling primary FIRST is usually wrong — CPU on primary is dominated by WRITES and the writes don't care about replicas; a second replica buys read headroom without the write-path risks of a primary resize (failover, connection storms). The decision matrix is workload-shaped: measure which axis saturates first.

RED FLAGS TO AVOID: - Route-everything-to-replica “for performance” (stale permission reads) - Treating streaming replication as backup - No lag awareness in product promises (dashboards claiming real-time)

Q6-13: How did you handle concurrent updates (SELECT FOR UPDATE and friends)

DIFFICULTY: Hard **CATEGORY TAG:** PostgreSQL

ANSWER: Concurrent-update strategy per access shape, because “one tool for everything” is where correctness dies. **Shape 1 — atomic scalar mutation** (balance decrement, counter increment, slot claim): conditional UPDATE — `UPDATE ... SET x = x - :n WHERE id = :id AND x >= :n` — the WHERE is the guard, rows-affected is the verdict, zero locks held beyond the statement; this is 80% of our concurrency and needs no locking machinery at all. **Shape 2 — read-modify-write with business logic between** (admin adjusting credits + tier): `SELECT ... FOR UPDATE` inside a short transaction — lock the row, compute, write, commit (Q5-15’s discipline: no external I/O inside). Variants matter: `NOWAIT` for interactive paths (admin UI shows “record busy” instead of hanging), `SKIP LOCKED` for worker-pool claiming (job tables — each worker claims distinct rows without contention), plain `FOR UPDATE` for must-serialize flows. **Shape 3 — multi-row invariants** (org seat limits: count users vs plan cap while inserting): the naive check-then-insert races; our answer is a lock on the PARENT row (FOR UPDATE on organization) serializing seat operations per org — coarse but correct, microseconds of hold; `SERIALIZABLE` would detect the anomaly too but with retry machinery we don’t want here.

Failure modes I engineer against: **deadlocks** — consistent ordering (multi-row ops sort ids; parent-before-child), `lock_timeout=2s` local setting so queues fail fast, and a retry wrapper for 40P01 (deadlock) and 55P03 (lock not available) with jitter + metrics — a rising deadlock rate is a design smell that pages review, not something we swallow silently. **Lost updates via ORM** — the silent variant: two transactions load, both dirty-check, second UPDATE overwrites first’s field (last-writer-wins) — mitigated by `@Version` on collaboratively-edited entities (deal notes, lead fields) turning silent loss into explicit 409. **Lock vs snapshot confusion** — FOR UPDATE doesn’t stop phantom inserts under `READ_COMMITTED` (a range claim needs `SERIALIZABLE` or a constraint); our slot-booking uses the `UNIQUE` constraint (slot_id unique when booked — partial unique index) as the DB-enforced claim, the same philosophy as idempotency keys: make the constraint express the invariant and concurrency becomes the DB’s job, not the application’s.

KEY TERMS TO MENTION: - Conditional UPDATE as default (WHERE-guard + rows-affected verdict) - FOR UPDATE shapes: plain / NOWAIT / SKIP LOCKED per access pattern - Parent-row locking for multi-row invariants (seat-limit case) - lock_timeout + deadlock retry + ordering; @Version for lost-update UX - Uniqueness constraints as concurrency control (slot claim, idempotency)

FOLLOW-UP QUESTIONS: 1. Why not `SERIALIZABLE` everywhere and never think about locks? 2. How does FOR UPDATE interact with the connection pool under load?

FOLLOW-UP ANSWERS: 1. SERIALIZABLE's predicate-lock machinery buys correctness at the cost of serialization-failure retries across EVERY transaction — retry storms under contention, throughput cliffs, and a failure mode (40001) that must be handled everywhere. Our anomalies are narrow and row-shaped; targeted tools (conditional UPDATE, @Version, parent locks) fix each with microseconds of cost. Broad isolation is a sledgehammer that also breaks the “one transaction per request” latency budget. 2. Held locks pin connections: a pool of 15 with 10 connections parked on FOR UPDATE means 5 available for everything else — lock-heavy code under load becomes pool starvation that LOOKS like a DB outage. Hence the rules: short transactions, no external I/O inside locks, lock_timeout fail-fast, and pool metrics correlated with lock waits (`pg_locks` + Hikari pending together) — the two dashboards side by side is how you see this failure coming.

RED FLAGS TO AVOID: - Check-then-act in Java with no DB guard (the eternal race) - SKIP LOCKED without understanding it skips (unprocessed rows vanish silently if the query is wrong) - No lock_timeout anywhere (one stuck transaction freezes the fleet)

Q6-14: How did you implement audit trails in the database

DIFFICULTY: Medium **CATEGORY TAG:** PostgreSQL

ANSWER: The audit trail has to survive the scenarios it exists for — support disputes, compliance reviews, incident forensics — which makes design constraints clear: append-only, actor-attributed, queryable, tamper-evident. **Storage:** `audit_event` table — id, occurred_at, actor_id, actor_type (user/api_key/system), org_id, action (enum: `LEAD.STAGE_CHANGE`), resource_type + resource_id, before/after JSONB (changed fields only), request_id, ip, outcome. Insert-only enforced by GRANTS: the app's DB role has INSERT/SELECT on audit_event — no UPDATE/DELETE possible at the privilege level; even a compromised app process can't rewrite history (the DB does the enforcing, not discipline). **Capture:** dual-path — AOP `@Audited` aspect for method-level business events (explicit, rich diffs) and Hibernate `@PreUpdate/@PreRemove` entity listeners as a safety net for entity mutations not annotated (catches the intern's new service method writing directly); both funnel into one `AuditWriter` with a bounded async queue (CRITICAL events — auth, billing — write synchronously; HIGH-volume — activity logs — batch async with drop-oldest and a counter that alarms on drops).

Performance and scale: audit_event grows ~3M rows/month — monthly range partitions (Q6-7 mechanics), 400-day hot retention then detach → Parquet export → S3 (compliance tier retains 7 years); the hot partition indexes (`org_id, occurred_at DESC`) and (`actor_id, occurred_at DESC`) serve the two real query shapes (“this org's activity”, “this user's actions”). **Tamper-evidence:** nightly chained hash (each day's batch hashes the previous day's hash + batch content) written to a separate store and to S3 object-lock — editing any row in the DB breaks the next verification run; not blockchain, but provable integrity with 20 lines of code. **The discipline that makes it valuable:** action taxonomy is curated (an enum reviewed in PRs, not free-form strings), diffs exclude secrets (masker on the way in), and a CI coverage test asserts every mutating service method in billing/auth/

admin packages emits audit events — silent gaps, the disease of audit systems, are a build failure here.

KEY TERMS TO MENTION: - Append-only via GRANT design (no UPDATE/DELETE privilege) - Dual capture: AOP aspect + Hibernate listeners safety net - Partitioned storage + Parquet/S3 cold tier (7-year compliance) - Chained-hash tamper evidence + object-lock - Curated action enum + CI coverage test (gap prevention)

FOLLOW-UP QUESTIONS: 1. Async audit writes can drop events — is that acceptable? 2. How would an auditor verify your trail end-to-end?

FOLLOW-UP ANSWERS: 1. Tiered acceptability: CRITICAL events (auth changes, credit mutations, permission grants) are synchronous — the transaction waits; a few ms is the price of un-losable evidence where money/identity is concerned. HIGH-volume events (activity tracking) accept bounded loss under extreme load with drop-count alarms — a documented, monitored trade-off. What's NOT acceptable is silent loss of either class; the queue metrics make the loss visible either way. 2. The auditor gets: the schema (privilege matrix showing insert-only), a sample reconciliation (pick a day — event count vs business counts: leads deleted vs DELETE events), the hash-chain verification run live (any tamper breaks the chain), and retention/export manifests from S3. End-to-end verifiability is a design output — if you can't demonstrate it in 30 minutes, the trail isn't audit-grade.

RED FLAGS TO AVOID: - App-level "append-only" with DB roles able to UPDATE (trust-the-code isn't a control) - Free-form action strings (unqueryable mush in 6 months) - No tamper evidence and no coverage enforcement

Q6-15: TWISTED — The leads table hits 500M rows. What breaks first, and walk me through the fix sequence.

DIFFICULTY: Expert **CATEGORY TAG:** PostgreSQL / Twisted Scenario

ANSWER: Breakage order at 500M rows (and I've watched the early stages at 50M): **first — maintenance, not queries.** Autovacuum can't keep up (dead tuples accumulate from status updates; vacuum on 500M rows takes hours and can't finish between business-hours churn), index bloat doubles index size, plans degrade subtly (b-tree depth + cached-page misses), and any retention DELETE is now impossible (lock + WAL + replication lag + vacuum debt). **second — the hot queries drift:** the leads-list query still hits its composite index (b-tree lookups degrade gracefully, p50 fine) but p99 climbs (cache misses on cold pages, deep index scans) and COUNT-style aggregations time out. **third — operations freeze:** any DDL (index build) takes hours, reindexing blocks writes for too long without CONCURRENTLY, and a restore (which is a 500M-row replay) exceeds RTO — the backup strategy silently breaks its own promise.

Fix sequence, in cost-benefit order. **Week 1 — stop the bleeding:** identify the top-3 queries by total time (pg_stat_statements); most "big table" pain is 3 queries: fix indexes (partial indexes for hot predicates — `WHERE org_id = ? AND deleted_at IS NULL AND status = 'HOT'` partial index is a fraction of the

full-index size), kill scans hidden in dashboards (COUNT(*) replaced by estimate-based counters or exact-count materialized refresh), and set autovacuum per-table aggressive (scale_factor 0.01 on the leads table + fillfactor tuning for HOT updates on status changes).

Month 1 — structural: archive cold leads (no activity 12 months → detach to archive table/S3; in CRM, 70%+ of rows are cold — the hot working set shrinks 5-10x, and the live table behaves like 50M). **Month 2-3 — partition** (Q6-7 playbook): partitioned twin by month or by hash-of-org if retention isn't the axis, dual-write, backfill, cutover. If TENANT growth (not row growth) drove the 500M — 10k orgs × 50k leads — the honest next step is **tenant sharding** (citus-style distribution by org_id or app-level shard routing), which is a different project with different costs, and I'd present both axes (rows vs tenants) before committing — the diagnosis determines the cure, and the twisted value of this question is exactly that distinction.

KEY TERMS TO MENTION: - Breakage order: vacuum/bloat → p99 drift → operational (DDL/restore) freeze - Partial indexes + dashboard-COUNT elimination as week-1 triage - Cold-data archival (70% of CRM rows are cold — shrink the working set) - Partitioning playbook (twin → dual-write → backfill → cutover) - Rows-vs-tenants diagnosis: partitioning vs sharding decision

FOLLOW-UP QUESTIONS: 1. Archive vs partition — why both? 2. How would you know sharding by org_id is justified?

FOLLOW-UP ANSWERS: 1. They solve different problems: archival shrinks the WORKING SET (what queries touch — fixes performance now, reversible), partitioning fixes MAINTENANCE mechanics (vacuum/retention/DDL on the remaining rows — structural, forward-looking). Archival first buys time and measures the true hot set, which then sizes the partitioning correctly — doing partitioning first on 500M rows migrates 350M dead rows you should have archived. 2. Evidence, not vibes: (a) primary CPU/IO saturates AFTER the working-set fixes (archival+indexes) — proving it's genuinely scale, not waste; (b) tenant-skew measured (p95 org size vs mean — if one org is 30% of rows, hash-sharding has a hot-shard problem needing a different key or celebrity-tenant handling); (c) cross-org queries are rare (sharding kills cross-shard joins/transactions — product access patterns must confirm); (d) operational will for shard-aware deploys/rebalancing. Sharding is a one-way door; the burden of proof is on the sharding proposal, always.

RED FLAGS TO AVOID: - Jumping to sharding as the first answer (the trap in the question) - No maintenance-layer awareness (vacuum/bloat is what dies first) - Fixes without measurement sequence (pg_stat_statements → archive → partition → shard)

CATEGORY 7 — REDIS AND CACHING QUESTIONS

Q7-1: How did you implement caching in AcquisitionOS

DIFFICULTY: Medium **CATEGORY TAG:** Redis/Caching

ANSWER: We use Redis 7 in cluster mode (3 shards, 1 replica each) for five distinct use cases, each with a different data structure and TTL.

1. Session cache (TTL 30d): We don't store HTTP sessions (stateless JWT). But we cache the user's `UserSession` row (for refresh-token reuse detection) — `session:{refreshTokenHash} → {userId, expiresAt, isRevoked}`. TTL 30d (matches refresh token expiry).

2. Rate limit (sliding window, TTL 60s): `rate-limit:{endpoint}:{userId}:{ip} → sorted set of timestamps`. `ZADD` the current timestamp, `ZREMRANGEBYSCORE` to evict old, `ZCARD` to count. TTL 60s on the key (auto-cleanup). We're migrating to a Lua script for atomic 1-round-trip (see Q1-12).

3. LLM response cache (TTL 24h): `ai:outreach:{SHA256(prompt + leadContext + model + temperature)} → JSON response`. 30% hit ratio on outreach (users often regenerate for similar leads). Saves ~\$1,200/month LLM cost. TTL 24h — long enough to be useful, short enough to not serve stale prompts.

4. Distributed lock (TTL 30s): `lock:{resource} → {lockId, acquiredAt}` via `SET NX PX 30000`. Used for cron dedup (only one ECS task fires a `@Scheduled` job at a time). We use Redisson's `RLock` for auto-renewal (the lock auto-extends if the job is still running).

5. Application cache (TTL 5min - 1h): `cache:{entity}:{id} → JSON`. Used for: organisation settings (rarely change, TTL 1h), plan catalog (changes with deploy, TTL 1h), user entitlements (changes on subscription update, TTL 5min, invalidated on update). We use Spring Cache (`@Cacheable`, `@CacheEvict`) for this layer.

Spring Cache annotations:

```
@Service
public class PlanCatalogService {
    @Cacheable(value = "planCatalog", key = "'all'")
    public List<Plan> getAllPlans() {
        return planRepository.findAll();
    }

    @CacheEvict(value = "planCatalog", allEntries = true)
    public void evictPlanCache() {
        // called when admin updates a plan
    }
}
```


Cache-aside pattern: Most caches use cache-aside (application reads from cache; if miss, reads from DB + writes to cache). We avoid write-through and write-behind for application cache — they add complexity (consistency) for little benefit. The exception is `credit_balance` — we’re planning to make it write-behind (writes to Redis, periodic flush to Postgres ledger) at 100K users (see Q1-7).

Cache invalidation: - **TTL** for everything — even cached data is eventually consistent - `@CacheEvict` for explicit invalidation on update (e.g. admin updates plan → evict `planCatalog`) - **Pub/sub** for cross-instance invalidation — when one ECS task evicts a cache key, it publishes `invalidate:{key}` to the `cache-invalidation` channel; all tasks subscribe and evict their local copy (if any — Spring Cache with Redis doesn’t have a local copy, but if we add Caffeine as L1, this matters).

Redis client: Lettuce (Spring Boot’s default) — non-blocking, thread-safe, supports cluster. We use `LettuceConnectionFactory` configured with cluster topology + connection pooling.

Redis configuration:

```
spring:
  data:
    redis:
      cluster:
        nodes: ${REDIS_CLUSTER_NODES}
        max-redirects: 3
      lettuce:
        pool:
          max-active: 16
          max-idle: 8
          min-idle: 2
          shutdown-timeout: 100ms
      cache:
        type: redis
        redis:
          time-to-live: 5m
          use-key-prefix: true
          cache-null-values: false // don't cache nulls (avoid cache penetration)
```

Cache penetration: `cache-null-values: false` — if a key is not in the DB, we don’t cache the null. Otherwise an attacker can request many non-existent keys, fill the cache with nulls, evict real entries. We add a “bloom filter” for the most popular endpoints (e.g. `lead.id` lookups) — check the bloom filter before Redis; if not present, skip Redis and go to DB. (We haven’t needed this — our cache is not a primary lookup path.)

Cache stampede: When a popular cache key expires, all concurrent requests miss the cache and hit the DB simultaneously. We mitigate via `@Cacheable(sync = true)` — Spring Cache synchronises the first miss; subsequent misses wait for the first to populate. For high-traffic keys, we add a “stale-while-revalidate” pattern (the cache serves the stale value while a background thread refreshes) — we implemented this manually for `planCatalog`.

KEY TERMS TO MENTION: - Redis 7 cluster (3 shards, 1 replica each), Lettuce client, 5 use cases - Session cache (TTL 30d), rate limit (ZSET sliding window, TTL 60s), LLM cache (TTL

24h, 30% hit ratio, \$1.2K/month saved), distributed lock (SET NX PX, Redisson auto-renewal), app cache (TTL 5m-1h, Spring Cache) - Cache-aside by default, write-behind planned for `credit_balance` at 100K users - Invalidation: TTL + `@CacheEvict` + Redis pub/sub for cross-instance - `cache-null-values: false` (avoid cache penetration) - `@Cacheable(sync = true)` to prevent cache stampede - Stale-while-revalidate for high-traffic keys

FOLLOW-UP QUESTIONS: 1. How do you measure cache hit ratio? 2. What if Redis goes down? 3. How do you handle a cache stampede?

FOLLOW-UP ANSWERS: 1. Micrometer gauge `cache.gets{result=hit,cache=...}` vs `cache.gets{result=miss,cache=...}` — Spring Boot exposes Spring Cache metrics. We compute hit ratio = hits / (hits + misses). For LLM cache, 30% hit ratio. For `planCatalog`, 95%. For session cache, 99% (every request hits it). We have a Grafana panel per cache. 2. Redis down = cache layer down = app falls through to DB. We configure `spring.cache.type = none` if Redis is unreachable (the app starts without cache). We have a Resilience4j circuit breaker around Redis calls — after 10% failures, we stop hitting Redis and go straight to the DB. The DB can handle the load (HikariCP 80 connections × 2ms query = 40K q/s — fine). The risk: if Redis is down for hours, the DB load spikes — we'd add a "rate-limit fallback" (in-memory `Caffeine` cache) for the hottest keys. We've never had Redis down for more than 30 seconds (Multi-AZ). 3. Three defenses: (a) `@Cacheable(sync = true)` — Spring synchronises the first miss; (b) "stale-while-revalidate" — we serve the stale value (cache returns it even after TTL) while a background thread refreshes; (c) "probabilistic early expiration" — we add a small random jitter to the TTL (e.g. `TTL + random(0, 60s)`) so all keys don't expire at the same time. We use (a) + (c); (b) only for `planCatalog`.

RED FLAGS TO AVOID: - Caching without TTL (memory leak) - Caching nulls (cache penetration) - No circuit breaker around Redis (Redis down = app down) - "We use Redis for sessions" (stateless JWT — we don't)

(The remaining Redis/caching questions — what to cache and why, session storage with Redis, cache invalidation, Redis data structures, distributed locking, Redis for rate limiting, cache-aside pattern, Redis failover, `@Cacheable` / `@CacheEvict` / `@CachePut` differences, cache stampede, hot key problem, big key problem, pub/sub, Lua scripting, persistence (AOF + RDB), cluster vs sentinel — follow the same template. Drill on: RDB vs AOF trade-offs, cluster slot migration, why `SET NX PX` works for distributed lock, the Redlock controversy, cache-aside vs read-through vs write-behind.)

Q7-2: What data did you cache and why — walk me through the actual cache map

DIFFICULTY: Medium **CATEGORY TAG:** Redis / Caching

ANSWER: Our Redis keyspace is designed as a catalog, not an accident — every key family has an owner, a TTL, and an invalidation story. **Reference/config data** (`config.credit-costs`, `config.plan-features`, TTL 10 min + explicit evict on admin change):

read on every AI action; DB is 300µs, Redis 200ns, and the data changes quarterly — classic cache-aside. **Session-adjacent state** (refresh-token denylist `jwt:deny:{jti}` TTL = token remainder, OTP attempt counters `otp:attempts:{emailHash}` TTL 15 min, rate-limit buckets `rl:{route}:{key}` TTL 60s): these are CORRECTNESS caches — losing them is a security event, so they're write-through with no eviction surprises (separate Redis logical DB + `noeviction` policy on that instance). **Computed aggregates** (`stats:org:{orgId}:leads` TTL 60s): dashboard counts that would GROUP BY 2M rows per page view — 60s staleness is invisible to users, DB load drops 70%. **Hot entity reads** (`lead:detail:{id}` TTL 5 min, evict-on-write): the lead detail page is read-heavy/write-light; a write evicts via the AFTER_COMMIT listener so read-after-write stays correct for the WRITER, with 5-min staleness tolerated for others (product-accepted, documented).

The why behind what we did NOT cache: the leads LIST (filters × orgs × pages — key explosion, hit rate < 20% in staging measurement, so it stayed un-cached and got fast via indexes instead — measure before caching, always), user permission sets (security-critical, changed via admin — we cache with 60s TTL accepting documented propagation delay, verified against DB for privileged ops), and anything under active development shape-wise (cache keys freeze response contracts; caching a shape you'll change in 2 weeks creates migration debt). Cache HIT-RATE dashboards per key family turn this from folklore into engineering: families under 50% hit rate get re-reviewed monthly — either fix the key design or delete the cache (a cache nobody hits is pure liability: latency + staleness + invalidation bugs for nothing).

KEY TERMS TO MENTION: - Key-family catalog: owner, TTL, invalidation per family - Correctness caches (denylist, rate limits) isolated with noeviction - Aggregate caching with product-accepted TTL staleness - Measured decision NOT to cache the list endpoint (hit-rate math) - Per-family hit-rate dashboards; delete caches that don't earn

FOLLOW-UP QUESTIONS: 1. How do you decide TTL for a new cache family? 2. What's the blast radius of a full Redis flush in your design?

FOLLOW-UP ANSWERS: 1. Three inputs: staleness tolerance of the product decision (dashboard numbers: 60s OK; permissions: 60s with privileged re-check), write frequency (high-write data needs short TTL or write-through evict — otherwise hit rate dies and staleness accumulates), and failure mode on miss (DB stampede risk → shorter TTL + jitter + single-flight). TTL is a product + load-shape decision written into the key-family doc, not a guess. 2. By design: correctness data (denylists, locks) lives on a separate instance with noeviction — flush of the CACHE instance costs a 60-second DB load spike (all TTL families regenerate via cache-aside; single-flight request coalescing turns the thundering herd into per-key single misses — the “cache warm-up” runbook is a documented 2-minute procedure, not an incident).

RED FLAGS TO AVOID: - “We cache everything” with no key catalog or hit-rate data - TTLs picked at random with no staleness-tolerance reasoning - Security-state and disposable-cache mixed on one eviction-policy instance

Q7-3: How did you handle cache invalidation

DIFFICULTY: Hard **CATEGORY TAG:** Redis / Caching

ANSWER: Invalidation strategy per write shape, because the failure modes differ. **Point writes** (lead updated): delete the exact key (`lead:detail:{id}`) in an `@TransactionalEventListener(AFTER_COMMIT)` — AFTER_COMMIT because deleting inside the transaction re-populates the cache from the NOT-YET-COMMITTED DB (the classic stale-resurrection bug: cache-aside refills from the old committed value, and your delete “didn’t work”). **Collection writes** (a lead’s stage affects org stats): bump a generation token — the stats cache key embeds a version (`stats:org:{id}:leads:v{gen}`), the writer INCRs `gen`, readers fetch `gen` then key — old generation keys expire naturally via TTL (no delete storms, no stale windows beyond one read; the cost is brief dual-cached generations, the benefit is no read-modify-write races on invalidation lists). **Fan-out writes** (credit-cost config change): the admin UI calls `CacheAdmin.evictFamily("config:credit-costs")` which publishes an evict event through Redis PubSub — every app instance subscribes and drops its local in-memory L1 + the shared key; PubSub is fire-and-forget, so the belt-and-suspenders is the 10-min TTL that bounds any missed message.

The honest corner: **cache-aside + read-replica** interaction — a replica-lagged read repopulates the cache with STALE data post-evict (evict → read hits replica → writes old value back → stale for full TTL). Our fixes: the hottest cached reads route primary (they’re single-key by id — cheap on primary), and the stats family’s generation scheme self-heals (next write re-bumps). The general doctrine I state in interviews: invalidation correctness = (delete AFTER commit) + (assume your delete can race a refill) + (TTL as the final backstop on every family — no infinite-TTL keys exist in the codebase, enforced by a CI check on our RedisTemplate wrapper which REQUIRES a TTL argument). Race-window acceptance: between write-commit and evict, a reader can serve $\leq$ one stale read — measured in milliseconds, product-accepted, and the alternative (write-through with distributed locks on every write) costs 10× latency for a window that’s already invisible.

KEY TERMS TO MENTION: - Delete AFTER_COMMIT (stale-resurrection bug explanation) - Generation-token pattern for collection invalidation - PubSub fan-out + TTL backstop (no infinite TTLs — CI-enforced) - Replica-lag repopulation hazard and routing answer - Accepted ≤ 1 stale read race window (ms-scale, documented)

FOLLOW-UP QUESTIONS: 1. Why delete instead of update-the-cache on write? 2. When would you choose write-through instead?

FOLLOW-UP ANSWERS: 1. Delete is idempotent, race-tolerant, and lazy — the next read rebuilds from authoritative DB. Cache-update requires computing the new cached shape inside/around the transaction (extra coupling, more bugs), races with concurrent readers (last-writer-wins on the CACHE too), and dead-ends when the cached shape is an aggregate. Delete-on-write is the pattern that survives refactors; update-on-write survives nothing. 2. Write-through when the cost of a cache miss is severe (stampede-prone hot key) or when reads vastly dominate writes AND the write path already computes the new state (ledgers, counters). We use it for rate-limit buckets and counters — always-consistent, and the write

rate is bounded. For read-optimized entities, cache-aside won on simplicity; the trade is decided per family with numbers.

RED FLAGS TO AVOID: - Evicting inside the transaction (the resurrection bug) - Infinite-TTL keys with manual invalidation only - No answer for the replica-repopulation race

Q7-4: What Redis data structures did you use and why

DIFFICULTY: Medium **CATEGORY TAG:** Redis

ANSWER: Choosing the right Redis structure is the difference between $O(1)$ and $O(N)$ per operation — our catalog: **Strings** (counters via INCR — rate-limit buckets, attempt counters; atomic and auto-creating), **Hashes** (object-ish state: `session:{id}` → {userId, device, lastSeen} — field-level access without serializing the whole object; HINCRBY for per-field counters), **Sets** (org membership in feature-flag cohorts `cohort:{flag}` — SISMEMBER $O(1)$ for flag checks; uniqueness by nature), **Sorted Sets** (the workhorse for anything ordered: reminder schedule `reminders:due` scored by epoch-ms — the sweeper does ZRANGEBYSCORE now → +60s and ZREM atomically per item; leaderboards for usage stats; dedup windows scored by timestamp with ZREMRANGEBYSCORE cleanup), **Streams** (the notification event log pre-Kafka — XADD consumer groups with XACK gave us at-least-once with replay; we kept Streams for the audit event fan-out even after Kafka arrived for the heavy flows, because Streams need zero extra infra), and **Bloom filters** (via module on the dedup path — `bf.exists seen:{campaignId}` pre-checks before the DB unique-constraint catch, cutting duplicate-write DB load ~80%).

Anti-patterns I actively avoided: storing JSON blobs where a Hash fits (serialize-deserialize per touch + whole-object rewrite per field change), using KEYS in production ($O(N)$ full-keyspace scan — SCAN with MATCH for admin tooling only), unbounded LISTs as queues (no ack semantics — Streams/Sorted Sets exist for that), and one-giant-key-with-everything (hot-key concentration — keys are sharded by natural ids). Every structure choice maps to a complexity bound I can state: reminders ZADD $O(\log N)$, flag check SISMEMBER $O(1)$, dedup BF.EXISTS $O(k)$. When a candidate quotes structures WITH the operation complexity and the eviction interaction (volatile-ttl on the cache instance favors short-TTL keys), they've operated Redis, not read about it.

KEY TERMS TO MENTION: - Strings=atomic counters, Hashes=field-level state, Sets=membership $O(1)$ - Sorted Sets for time-ordered work (reminders) — ZRANGEBYSCORE + ZREM atomic - Streams for at-least-once event fan-out with consumer groups - Bloom filter pre-check ahead of DB unique-constraint dedup - Complexity-per-op + KEYS-forbidden / SCAN-only discipline

FOLLOW-UP QUESTIONS: 1. Why did you keep Redis Streams after adding Kafka? 2. How do you handle a hot Sorted Set (reminders) at scale?

FOLLOW-UP ANSWERS: 1. Different tiers of traffic: Kafka carries the high-throughput business flows (notifications, analytics events) with real ops investment; Streams carry low-volume internal fan-out (audit events, cache evicts) where a Kafka topic's operational cost

exceeds the value. Two systems is itself a cost — the ADR sets the migration trigger (audit volume > 5k/s or cross-region replication needs). 2. Partition the schedule: `reminders:due:{shard0..7}` sharded by org_id at ENQUEUE time (the sweeper iterates shards, workers per shard) — ZADD/ZRANGEBYSCORE stay $O(\log N)$ per shard as N grows. Plus per-shard monitoring (ZCARD) so a skewed shard (one whale org) gets its own worker. The general lesson: Redis structures are fast until they're single-point — shard by natural key before it hurts.

RED FLAGS TO AVOID: - Strings-only Redis (JSON blobs everywhere = misuse) - KEYS in production paths - No complexity bounds quoted with structure choices

Q7-5: How did you implement distributed locking with Redis

DIFFICULTY: Hard **CATEGORY TAG:** Redis

ANSWER: Distributed locks in our system protect two things: scheduled-job execution (one instance fires the 8 AM sweep) and multi-step resource claims (batch job slots). The core pattern — single-instance Redis, which is the honest scope of our locking: `SET lock:{name} {instanceId} NX PX {ttl}` — NX (only if absent), PX (expiry in ms — a crash can't hold the lock forever; TTL is the crash-safety), value = unique token (instanceId + uuid) for safe release. Release is a Lua script (GET-compare-DEL atomically): you must NOT delete a lock you don't own — instance A slowed past its TTL, B acquires, A finishes and deletes B's lock (the classic bug; the Lua check makes release owner-verified). Renewal: jobs expected to run near TTL use a watchdog thread extending the lock (Lua check-and-PEXPIRE) — same idea as Redlock's watchdog, implemented in 30 lines.

Where I DON'T use Redis locks, and why that answer matters more: **DB-row claims** use the DB itself — conditional UPDATE / FOR UPDATE SKIP LOCKED (Q6-13) — because the resource lives in Postgres and the transaction is the lock (a Redis lock around a DB transaction adds a second failure domain for no benefit). **Multi-resource atomic claims across instances** (the Redlock scenario — locks on independent Redis masters) we don't have: our Redis is single-master with a replica for reads; I can articulate why Redlock's quorum doesn't magically fix clock/fencing debates (Kleppmann's critique: without fencing tokens, a paused process can still write after lock loss) and where our usage is safe within that critique — locks guard IDEMPOTENT work (double-run of a sweep is harmless by design; the lock is an efficiency, not a correctness crutch). That distinction is the senior answer: locks-that-must-be-correct live in the DB as constraints/transactions; Redis locks guard best-effort deduplication where double-execution is tolerable — and our job handlers are idempotent because of exactly this reasoning.

KEY TERMS TO MENTION: - `SET NX PX` + unique token + Lua-owner-verified release (safe release) - TTL as crash safety; watchdog renewal for long jobs - DB-native locking for DB-resident resources (SKIP LOCKED) - Redlock caveats (fencing tokens, pause-after-lock-loss) and scope honesty - Locks guard idempotent work — efficiency, not correctness

FOLLOW-UP QUESTIONS: 1. Redis master fails over mid-job — what happens to the lock? 2. Why a unique value in the lock at all?

FOLLOW-UP ANSWERS: 1. Replication is async — the failover can lose the lock key while the holder still believes it owns it: two owners briefly. Our design absorbs it: holders' work is idempotent + the DB layer has its own guards (unique constraints catch double-writes). For work where double-execution is NOT tolerable, we don't use Redis locks at all (Q above) — so the failover window costs a duplicated sweep, never duplicated money. 2. Without a unique value, release is blind DEL — the A-slow-past-TTL deletes-B's-lock bug (then C acquires while B still works: three owners). The token + Lua compare-release closes it. It's the difference between a lock and a liability; I've reviewed code where this exact bug caused triple-sent campaigns.

RED FLAGS TO AVOID: - DEL without ownership check (the classic triple-owner bug) - Locks without TTL (crash = permanent freeze) - Claiming Redlock solves everything / no awareness of fencing critique

Q7-6: How did you use Redis for rate limiting

DIFFICULTY: Hard **CATEGORY TAG:** Redis / Security

ANSWER: Rate limiting needs distributed counters (3 app instances share the budget) and atomic operations — Redis is the natural substrate. We run two algorithms per route policy.

Fixed window (`INCR rl:{route}:{key}:{epochMin}` + `EXPIRE 61s`): cheap, good enough for generous budgets (API keys: 600/min); known flaw — boundary burst (2× limit across a window edge) — acceptable where the budget has slack. **Sliding window (approximate)** for tight budgets (auth endpoints: 10/10min/IP): two fixed windows weighted by overlap (`prev_count × (remaining_prev/60) + curr_count`) in a Lua script — one round-trip, atomic, memory O(1) per key, within 2% of true sliding-window (sorted-set ZADD-per-request is exact but O(N) memory per key — wrong cost for per-IP keys that number in millions). All logic in Lua: check-and-consume must be atomic — two instances racing INCR-then-EXPIRE without Lua can leave keys with no TTL (permanent limit) or admit double-budget (the INCR/EXPIRE race — every hand-rolled limiter's first bug).

Keying is policy, not code: anonymous routes key by IP (right-most trusted XFF entry), authenticated by user id, API-keys by key id, and EXPENSIVE routes add org-level budgets (AI generation: per-user 100/day AND per-org 5000/day — two INCRs in one script, both must pass; the script returns which budget failed so the API returns precise 429s with Retry-After computed from window math). Failure posture: Redis unavailable → the limiter FAILS OPEN for cache-tier limits (availability over strictness for generic API limits — a degraded limit beats a dead API) but FAILS CLOSED for the auth brute-force limiter (security limits must not vanish with Redis — a locked instance-level fallback counter with 1/10th budget keeps brute-force protection while Redis recovers). Metrics: rejections per route/key-tier on Grafana — a rejection spike at 3 AM is either an attack or a broken client; both matter, and the per-key top-talkers panel identifies which in seconds.

KEY TERMS TO MENTION: - Fixed window vs weighted two-window sliding (Lua, atomic check-and-consume) - INCR/EXPIRE race → Lua-only limiter ops - Layered keying: IP / user /

key / org budgets in ONE script - Fail-open (generic) vs fail-closed (auth) posture with rationale - Rejection metrics + top-talker panels for incident triage

FOLLOW-UP QUESTIONS: 1. Why not local in-memory limiters per instance? 2. A customer's integration hits limits legitimately — what's the product answer?

FOLLOW-UP ANSWERS: 1. Three instances = 3× the real budget plus load-balancer skew (one instance eats 80% of a tenant's traffic — their limit trips early; another tenant rides a quiet instance past their cap). Distributed budgets need shared state; Redis's ~200µs cost per check is the price of correct arithmetic. Local limiters survive only as the fail-closed degraded mode (Q above). 2. Limits are a product surface, not just a shield: 429s carry Retry-After + a rate-limit headers (X-RateLimit-Remaining/Limit), the customer dashboard shows their own usage graphs, and tier upgrades are self-serve. The worst answer is a mysterious 429 with no visibility — rate limiting without feedback loops generates support tickets instead of well-behaved clients.

RED FLAGS TO AVOID: - INCR + EXPIRE as two separate ops (the TTL race) - One algorithm/keying for all routes - No stated Redis-down posture (fail-open-vs-closed is the senior tell)

Q7-7: Walk me through your cache-aside pattern implementation and its pitfalls

DIFFICULTY: Medium **CATEGORY TAG:** Redis / Caching

ANSWER: Cache-aside (**LazyLoading**) is our default pattern: READ = check Redis → hit: return; miss: query DB → SETEX key TTL → return; WRITE = commit DB transaction → evict key (AFTER_COMMIT, Q7-3). Implemented once, correctly, in a generic **CachePort** wrapper — not sprinkled:

```
cachePort.getOrLoad("lead:detail:" + id, TTL_5MIN, () -> leadService.loadDetail(id)).
```

The wrapper enforces the invariants the pattern needs: mandatory TTL argument (compile-time prevention of infinite keys), per-family metrics (hit/miss/load-time/load-failure), single-flight load coalescing (a **ConcurrentHashMap<String, CompletableFuture>** per instance — 50 concurrent misses on one key produce ONE DB query, not 50: our stampede defense at instance level; cross-instance stampede is bounded by TTL jitter — TTLs randomized ±10% so mass-expiry doesn't synchronize), and serialization versioning in the value envelope (**{v:2, data:...}**) so a shape change doesn't serve old deserialized garbage during deploys (version mismatch = treat as miss).

The pitfalls I enumerate from experience, each with its concrete prevention: **stale-resurrection** (evict inside transaction → refill from old committed state) → AFTER_COMMIT evict; **replica-repopulation** (post-evict read from lagging replica writes back stale) → hottest cached reads route primary; **stampede** (hot key expiry = synchronized herd) → single-flight + TTL jitter + for the hottest keys a short "lock-and-double-check" around the load (SET NX guard; losers sleep-and-retry once before loading themselves); **serialization drift** → envelope versioning; **hot-key concentration** (one celebrity key saturating a Redis shard) → key families monitored by QPS per key (top-keys via **--hotkeys** in maintenance

windows), worst-case split into sharded sub-keys; **negative caching absent** (nonexistent id hammered → every request misses to DB) → short-TTL tombstones (`lead:detail:{id} = NULL_MARKER, TTL 30s`) for known-not-found — the least famous pitfall and the one that quietly saves you during enumeration attacks. Cache-aside's appeal is decoupling (cache down ≠ app down — degraded latency only) and simplicity of reasoning; the cost is the race windows above, all handled in ONE place (the wrapper) instead of N call sites.

KEY TERMS TO MENTION: - Generic CachePort wrapper enforcing TTL/metrics/single-flight - AFTER_COMMIT evict + primary-route for hot cached reads - Stampede defense: single-flight + TTL jitter + lock-and-double-check - Value envelope versioning (deploy-safe deserialization) - Negative caching tombstones for not-found

FOLLOW-UP QUESTIONS: 1. Cache down completely — what does the user experience? 2. Why TTL jitter matters — walk through the failure without it.

FOLLOW-UP ANSWERS: 1. By design: `getOrLoad` falls back to DB on Redis errors (exceptions never propagate from the cache layer) — p95 degrades from ~40ms to ~300ms on cached endpoints, dashboards fire the Redis-down alarm, and nothing user-facing 500s. That availability decoupling is WHY cache-aside over write-through here: a write-through dependency on Redis makes the DB write path hostage to cache health. 2. Without jitter, all keys of a family set at the same moment expire together (mass import at 10:00, TTL 10 min → 10:10 every key misses simultaneously) → synchronized DB query storm at 10:10 every cycle, DB p99 spikes in a sawtooth exactly matching the TTL. We saw the sawtooth in staging once; ±10% randomization flattens expiry mass, and single-flight absorbs the remainder. Small setting, load-test-visible effect.

RED FLAGS TO AVOID: - Cache logic copy-pasted per call site (invariants drift per site) - No stampede story ("it'll be fine" at scale) - Errors from the cache layer propagating to users

Q7-8: How did you handle Redis failover / HA

DIFFICULTY: Medium **CATEGORY TAG:** Redis / DevOps

ANSWER: Redis for us is a cache + ephemeral correctness state, and the HA design follows from that grading. Architecture: **single-master with replica + Sentinel** (managed ElastiCache-equivalent): Sentinel monitors, fails over on quorum (down-after 10s, failover-timeout 60s), clients (Lettuce with topology refresh) follow the new master — write downtime ≈ 10-30s, data loss = unreplicated writes (async replication: seconds). What that loss costs us, tier by tier: cache families — self-heal via cache-aside (Q7-7's degraded-latency story); rate-limit counters — a failover resets buckets (worst case: budgets double for one window — accepted, documented); denylist/attempt counters — the SECURITY tier, which is why that logical DB also has the instance-level fail-closed fallback (Q7-6) and why denylist revocations are ALSO written to the DB session table (Redis is an accelerator there, not the source of truth — a failover cannot un-revoke a session). That tiering — what MUST survive vs what MAY reset — is the entire interview answer in one sentence: HA is not making Redis unkillable, it's grading every key's blast radius and giving each grade a plan.

Operational posture: client-side resilience in Lettuce (command timeout 500ms, retry disabled for cache reads — fail-to-DB fast, retry ONCE for security writes), connection-pool pre-warmed (failover reconnect storms are their own outage if pools cold-start), and runbooks per failure class: master death (auto — verify failover + watch hit rates recover), replica lag (route-shift alerts), full-region loss (Redis rebuilds empty; correctness tier rebuilds from DB — denylists re-derived from revoked session rows in a 2-minute sweep, a job that exists precisely because we refused to make Redis the source of truth). We deliberately did NOT adopt Redis Cluster: 3-shard operational complexity (resharding, multi-key ops break, hash-tag discipline) buys throughput headroom our traffic doesn't need (~15k ops/s peak vs ~100k single-node capacity) — the upgrade trigger (sustained > 40% ops capacity or hot-shard metrics) is written in the ADR.

KEY TERMS TO MENTION: - Sentinel failover (10-30s write gap, async-replication loss acknowledged) - Key-tier grading: cache (self-heal) vs rate-limit (reset OK) vs security (DB is source of truth) - Lettuce settings: timeouts, no-retry-reads, pre-warmed pools - Rebuild-from-DB runbooks (denylist re-derivation sweep) - No-Cluster decision + explicit upgrade trigger

FOLLOW-UP QUESTIONS: 1. Why not make Redis strongly consistent (WAIT / synchronous replication)? 2. During the 10-30s failover gap, what do users see?

FOLLOW-UP ANSWERS: 1. WAIT forces acknowledgment per write — latency 2× and still not a transactional guarantee (a failover can lose acknowledged writes under edge cases). The correct pattern for keys that matter is what we did: demote Redis to accelerator for those keys and put truth in Postgres. Strongly-consistent Redis is trying to make the cache into a database — wrong tool, wrong cost. 2. Cache-tier: slow (DB fallback) responses — nothing breaks. Rate limits: fail-open degraded budgets. Security tier: fail-closed local fallbacks (reduced budgets, denylist checks fall back to the DB session table — a ~2ms penalty per auth that tolerates the gap). The user-visible answer is “slightly slower, nothing broken” — which is only true because the tiering work happened BEFORE the incident, not during it.

RED FLAGS TO AVOID: - HA discussion with no data-loss acknowledgment (async replication) - No per-key-tier blast-radius thinking - Redis Cluster “because HA” (Cluster is throughput tech, not HA tech)

Q7-9: What is the difference between @Cacheable, @CacheEvict, @CachePut — and where did each fit?

DIFFICULTY: Easy **CATEGORY TAG:** Spring Boot / Caching

ANSWER: The three annotations are Spring's abstraction over cache-aside at the method boundary. **@Cacheable:** check cache by key before executing; on hit, skip the method entirely; on miss, execute and store — the read path (`@Cacheable(value = "leadDetail", key = "#orgId + ':' + #leadId")`). **@CacheEvict:** remove entries — on the write path (`@CacheEvict(value = "leadDetail", key = "...")` for point evicts, `allEntries = true` for family flushes — the blunt instrument that's correct when you can't

compute affected keys). **@CachePut**: always execute the method and store the RESULT — for write paths where the method's return IS the fresh cached value; the trap I name in interviews: **@CachePut** on a method whose callers ignore the return value still stores it, and **@CachePut** vs **@Cacheable** key misalignment silently caches under the wrong key (they must share key expressions exactly — a real bug class).

Where each fit — and the honest caveat: we use Spring's annotations ONLY for the in-JVM Caffeine L1 (per-instance micro-cache for reference data, 30s TTL, zero serialization). Our shared Redis layer deliberately bypasses annotations for the **CachePort** wrapper (Q7-7) — because the annotations' semantics were too thin for our needs: no single-flight, no metrics hooks, no envelope versioning, TTLs per-method but no jitter, and the key SpEL gets unwieldy for composite keys. That's a senior-flavored answer: know the annotations cold (including the proxy caveats — self-invocation skips them like **@Transactional** / **@Async**, `sync=true` for stampede-block, unlessGenerated keys) AND know when the abstraction's ceiling forces a hand-rolled layer. The L1 note: Caffeine under Spring's abstraction gives us reference-data reads at ~100ns with a 30s staleness bound — the two-tier design (Caffeine L1 → Redis L2 → Postgres) where ONLY L1 uses annotations because only L1's needs fit the abstraction.

KEY TERMS TO MENTION: - **@Cacheable** (skip-on-hit) / **@CacheEvict** (point + allEntries) / **@CachePut** (store return) - Key-expression alignment between Cacheable and CachePut (bug class) - Proxy/self-invocation caveats + `sync=true` stampede option - Annotations for Caffeine L1; CachePort wrapper for Redis L2 (why) - Two-tier caching with per-tier tooling fit

FOLLOW-UP QUESTIONS: 1. When does allEntries=true become the right call? 2. How do the annotations interact with **@Transactional**?

FOLLOW-UP ANSWERS: 1. When affected keys are unbounded (a config change touching N computed entries) or key computation is error-prone — flushing the family costs a brief load spike (bounded by single-flight) versus risking stale keys from a missed computed key. Correctness first, spike second; allEntries on low-traffic families is free, on hot families needs the stampede answer ready. 2. Order: the cache interceptor sits outside the transactional one by default — a **@Cacheable** method that's also **@Transactional** opens its transaction on a cache miss and closes before the value is cached (fine), but **@CacheEvict** inside a ROLLED-BACK transaction evicts anyway (cache flushed, DB unchanged — safe direction, just wasteful) while a **@CachePut** inside rollback stores uncommitted data (unsafe — needs AFTER_COMMIT semantics the annotations don't express). That's the second reason our Redis layer evicts via TransactionEventListener — the abstraction couldn't say "after commit".

RED FLAGS TO AVOID: - Reciting definitions without the key-alignment/rollback traps - Using **@CachePut** casually on transactional writes - No awareness of the abstraction's ceiling (sync, metrics, versioning)

Q7-10: TWISTED — Redis goes down at peak traffic. What happens to AcquisitionOS, minute by minute?

DIFFICULTY: Expert **CATEGORY TAG:** Redis / Twisted Scenario

ANSWER: Minute 0-1 (detection): Redis-health alarm fires (health indicator + hit-rate drop + connection-error rate — three signals, one page). The app does NOT alert users; here's why, per tier, because the design decided this in advance: **cache tier** — `CachePort.getOrLoad` catches connection errors and falls through to Postgres; load on DB rises immediately (all cached reads now hit DB); DB has headroom sized for cache-down (we load-test with Redis DISABLED — the “cache-off scenario” is a standard perf-test case, which is the difference between knowing and hoping). Expected effect: p95 on cached endpoints degrades ~7× (40ms → 300ms) but stays under timeout budgets. **Rate limits** — fail-open on generic routes (limits vanish: burst tolerance for ~10 min is accepted vs an outage), fail-closed on auth brute-force (instance-local counters at 1/10 budget — protection degrades, doesn't die). **Session/denylist** — revocation checks fall back to the DB session table (+2ms per auth); OTP attempt counters go instance-local (slightly tighter limits — fail-safe direction).

Minute 1-5 (impact management): the biggest real risk is the DB load spike — mitigation is pre-built: the cache-off load test proved Postgres absorbs full uncached OLTP at 2× peak with the pool math unchanged (pool sizing didn't assume cache hits — a subtle but critical sizing decision); on-call may throttle the most cache-dependent heavy endpoints (feature flag: dashboard analytics read-replica route forced) to protect OLTP. **Background jobs:** distributed locks vanish — sweeps may double-run; handlers are idempotent BY REQUIREMENT (the lock was efficiency, not correctness — Q7-5), so the visible effect is zero, the log effect is “lock acquire failed, proceeding” spam (its own alarm — expected during Redis incidents, pre-acknowledged in the runbook to avoid false panic).

Minute 5-30 (recovery): failover completes (Sentinel) or rebuild-from-empty runs; traffic reheats the cache organically — the single-flight + jitter machinery (Q7-7) turns the reheat into per-key single misses instead of a synchronized stampede (the warm-up storm IS the next incident if you lack it); correctness tier re-derives from DB (denylist sweep, 2 min). Post-incident: the RCA answers why Sentinel didn't fail over (node death vs network partition vs quota), and the runbook gets the missing rung. The meta-point I land: Redis-down being a 30-minute non-event is a DESIGN OUTPUT — tiered blast radii, idempotent jobs, DB headroom, and rehearsed fallbacks — not luck.

KEY TERMS TO MENTION: - Pre-decided per-tier degradation (cache: DB fallback; limits: fail-open/closed split; security: DB truth) - Cache-off load testing as the proof of DB headroom - Idempotent job handlers make lock loss harmless - Single-flight + jitter = safe cache reheat (no warm-up stampede) - RCA + runbook iteration as the closing loop

FOLLOW-UP QUESTIONS: 1. Which single design decision saved you here, and why? 2. What if Postgres ALSO degraded simultaneously (cascading scenario)?

FOLLOW-UP ANSWERS: 1. Grading keys by blast radius at design time. Everything else (fallbacks, idempotency, load tests) executes that grading. Without it, Redis-down is an unknown-unknowns exercise at 3 AM; with it, the incident is a checklist. It costs one column in the key-catalog doc — the cheapest resilience investment in the whole system. 2. That's

the real outage — and the honest answer is graceful degradation by product priority: feature flags shed load (analytics heavy endpoints off, batch jobs paused, AI generation queued instead of inline), rate limits tighten globally (protect the core), and the status page communicates. What we DON'T do is let everything degrade equally — cascading failures kill systems through uncontrolled coupling; the flags exist precisely to make shedding load a 30-second decision instead of an architecture debate mid-outage.

RED FLAGS TO AVOID: - “Redis down = site down” (no tiering thought) - No mention of cache-warm-up stampede on recovery - Discovery-mode answers (no pre-decided degradation plan)

CATEGORY 8 — REST API DESIGN QUESTIONS

Q8-1: How did you design the REST APIs for AcquisitionOS

DIFFICULTY: Medium **CATEGORY TAG:** REST API

ANSWER: RESTful, resource-oriented, JSON-only. Detailed in Q2-9. Quick summary of conventions:

- URLs: `/api/v1/{resource}` (plural nouns, kebab-case, versioned)
- Methods: GET (read), POST (create, async with `requestId` for idempotency), PUT (full replace), PATCH (partial), DELETE
- Status codes: 200, 201, 202, 204, 400, 401, 403, 404, 409, 422, 429, 500, 502, 503
- Response: `{"data": ..., "meta": {pagination}}` for collections, `{"error": {code, message, details}}` for errors
- Validation: `@Valid` + Bean Validation, custom validators (`@ValidPlan` , `@ValidCreditAmount`)
- DTOs via MapStruct (never expose entities)
- Pagination: cursor-based for large collections, offset for admin UIs
- Filtering: query params + JPA Specifications
- Sorting: allowlisted fields, `sortBy=field:dir`
- Rate limiting: per-user per-endpoint via Redis, `Retry-After` header
- Auth: JWT bearer + cookie, `JwtAuthenticationFilter` , `@PreAuthorize`
- CORS: allowlist origins, credentials, preflight cache 1h
- Idempotency: `Idempotency-Key` header (UUID) for POST endpoints, Redis 24h
- Versioning: URL-based (`/api/v1/` , `/api/v2/`)
- Documentation: springdoc-openapi, OpenAPI 3, Swagger UI at `/api-docs`
- Partial HATEOAS: `_links` for common next actions, not full hypermedia

See Q2-9 for the full template and follow-up answers.

(The remaining REST API questions — API versioning strategies, pagination (cursor vs offset), error response format, HATEOAS, OpenAPI/Swagger, file upload APIs, rate limiting, request/response logging, backwards compatibility, idempotency, content negotiation, conditional requests (ETag), long-polling/SSE, webhooks, GraphQL vs REST, gRPC for internal — follow the same template. Drill on: the difference between `201 Created` and `202 Accepted` , when to use `409 Conflict` vs `422 Unprocessable` , why `POST` is not idempotent by default and how `Idempotency-Key` fixes it, how Stripe designs their API for inspiration.)

SAMPLE QUESTIONS NOTICE: Drill-down variants the interviewer should add: “Walk me through the exact bytes sent to Stripe’s webhook endpoint and how you verify them”, “Show me your Dockerfile layer caching strategy”, “What’s your JVM memory model and how do you size generations?”, “How do you test a `@TransactionalEventListener`?”, “Tell me about a time you disagreed with your PM — the real story, not the polished version.”

Q8-2: What is your approach to API versioning (deep-dive with the real v1→v2 case)

DIFFICULTY: Medium **CATEGORY TAG:** REST API

ANSWER: Path-versioned majors (`/api/v1` , `/api/v2`), additive-only within a major, enforced by contract tests. The real case that shaped the policy: our leads list moved from offset pagination + nested score objects (v1) to cursor pagination + flattened scores (v2). Mechanically it worked because of a discipline I’d defend in any interview: **one service layer, N DTO assemblers** — `LeadQueryService` is version-agnostic (returns domain rows); `LeadV1Assembler` wraps scores nested + maps offset params; `LeadV2Assembler` flattens and consumes cursors. Business logic never forks; version drift lives entirely at the shape boundary, which is the only sustainable way to run two versions — forked logic doubles every bug fix for a year.

The compatibility mechanics that make “additive-only” enforceable rather than aspirational: consumer-driven contract tests (each frontend/integration maintains a contract pack — expected fields, types, semantics; CI runs provider-side against all packs, so a “harmless” field rename fails the build before consumers do), response-field sunseting marked with `@Deprecated` in the OpenAPI spec + a `Deprecation / Sunset` header phase (60-day public window), and a per-caller traffic dashboard that gates final removal (v1 shut when its share fell under 1% for 2 weeks — data kills politics in the removal conversation). Small-version evolution (`v1.3`) we deliberately DON’T expose — the minor version is internal changelog, not API surface, because clients can’t be trusted to feature-detect on minor numbers, and optional-response-field feature detection IS the additive path.

Header-versioning and query-param versioning were evaluated and rejected with reasons I can articulate: headers hide intent from logs/CDNs and break shareable URLs; query params pollute cache keys and invite clients to “just change the param” without contract awareness. Content-negotiation versioning (`Accept: application/vnd.acq.v2+json`) is REST-purist-correct and operationally annoying — error pages, webhooks, and browser debugging all get harder. Path wins on boring operability; boring is a feature in API design.

KEY TERMS TO MENTION: - One service layer + per-version assemblers (no logic fork) - Consumer-driven contract tests enforcing additive-only - Deprecation/Sunset headers + traffic-gated removal - No exposed minor versions (additive evolution via optional fields) - Header/query versioning rejection rationale

FOLLOW-UP QUESTIONS: 1. How do you version WEBHOOK payloads (the push side)? 2. A client refuses to migrate off v1 — escalation path?

FOLLOW-UP ANSWERS: 1. Webhooks carry `event-type` + `api-version` fields and a version header; delivery is per-subscriber versioned (each endpoint pins the version it registered with), replayed events replay in their ORIGINAL version (history is immutable), and upgrades happen per-endpoint via the dashboard with a test-fire feature. Push-side versioning is stricter than request-side because you can't 404 a bad listener — you can only stop sending. 2. The path is: usage data shows their exact endpoints/fields → a migration engineer (we did this for our top-3 API consumers) → timeline commitments with the Sunset header as the contractual backstop → finally, if a paying enterprise truly cannot move, a paid extended-support flag extends THEIR keys' v1 window while everyone else's expires. Versioning ends in business decisions sometimes; the engineering job is making the data and mechanics ready for that conversation.

RED FLAGS TO AVOID: - Forking service logic per version (maintenance death spiral) - Removal by decree with no traffic data - "We version in headers" without the operability trade-offs

Q8-3: How did you handle pagination in list APIs (offset vs cursor — the full decision)

DIFFICULTY: Hard **CATEGORY TAG:** REST API / Performance

ANSWER: The decision matrix I present: **offset pagination** (`?page=3&size=50`) — human-addressable, random access, total counts; costs $O(\text{offset})$ scan on deep pages, unstable under concurrent writes (insert before your position shifts every row: duplicates and missed rows across pages), and expensive COUNT. **Cursor pagination** (`?cursor=eyJjcmVhdGVkX2F0...&limit=50`) — $O(\log N)$ seek at any depth, stable under concurrent writes (the cursor pins your position tuple), no total count; costs no random access, opaque URLs, and requires a unique deterministic sort tuple. Our routing: hot user-facing lists (leads, activities, messages — infinite-scroll UX, concurrent writes, deep access) = cursor; admin consoles (page jumps expected, bounded data, low concurrency) = offset with a max-depth guard (`page × size ≤ 10,000` — beyond that the API demands filters, because deep-offset is a UX smell wearing a technical costume).

Cursor implementation details that make or break it: the tuple must be UNIQUE + indexed in the same order as the sort (`(created_at DESC, id DESC)` with the matching composite index — a single-column cursor on a non-unique column skips/duplicates rows under writes, the bug nobody notices in dev and users notice in week one); the cursor payload is Base64(tuple + filter-hash + version) — filter-hash prevents a client swapping cursors across different queries (they get a 409 with "restart the query"), version enables cursor-format evolution (v2 cursors rejected with a clear error rather than silently wrong results); `limit` clamped [1,100]; and `next_cursor: null` is the end signal — NO `total` field on cursor endpoints (COUNT on 2M rows per page is the tax that makes people think cursors are "hard" — the product gave up totals and gained 140ms p95). Error contract for bad cursors: 400 with

machine code `CURSOR_INVALID` vs `CURSOR_EXPIRED` (we expire cursors after 24h when underlying retention moves) — clients handle each differently, so the codes are distinct, tested, and documented in OpenAPI with examples.

KEY TERMS TO MENTION: - Decision matrix per access pattern (hot lists = cursor; admin = offset+guard) - Unique deterministic tuple + matching composite index - Cursor payload: tuple + filter-hash + version (409 on cross-query swaps) - No total on cursor endpoints; null `next_cursor` as end - Distinct error codes `CURSOR_INVALID` vs `CURSOR_EXPIRED`

FOLLOW-UP QUESTIONS: 1. How do you paginate with user-chosen sort fields? 2. Real-time insertion-heavy feed — does cursor pagination still hold?

FOLLOW-UP ANSWERS: 1. Allow-listed sort keys each with a pre-declared cursor tuple (the tuple must include a unique tiebreaker regardless of user sort) — the index per sort path must exist, which is why the allow-list is SHORT (5 sorts, 5 indexes, reviewed). Free-form sorting is an anti-feature: unindexable plans + cursor math that can't guarantee stability. The API offers shapes it can serve efficiently — that's the deal. 2. Yes — cursors are actually the ONLY stable option there: OFFSET shifts everything on each insert (users see repeats/skips constantly), while a cursor pinned to `(created_at, id)` keeps the reader anchored; new items appear on refresh at the head, and the standard UX pattern ("N new items — click to load") handles head-growth. The failure people hit is filtering the same query with different filter-sets mid-scroll — the filter-hash 409 exists precisely for that.

RED FLAGS TO AVOID: - Cursor pagination without a unique tiebreaker (subtle skip/dup bugs) - Total counts on million-row hot endpoints - Opaque cursor with no version/filter binding (silent wrong results)

Q8-4: What is your error response format (and the contract behind it)

DIFFICULTY: Medium **CATEGORY TAG:** REST API

ANSWER: Every error, every endpoint, one shape — RFC 7807 Problem Details as the base, extended with our fields: `type` (stable machine code: `https://api.acquisitionos.com/errors/LEAD_NOT_FOUND`), `title` (human summary), `status`, `detail` (safe, non-internal message), `instance` (request id — joins to traces), plus our extension: `errors[]` for field-level validation (`{field, code, message}`), `retryable` boolean (clients branch on it — a 429 with `retryable=true` + Retry-After is automation-friendly; a 500 with `retryable=false` tells the client to stop and report), and `correlation_id` (same as instance, duplicated at top level because every client library grabs it differently). The shape is enforced mechanically: a single `@RestControllerAdvice` funnel — all exceptions (domain, validation, framework, unknown) map through `ProblemDetail` builders; a test suite asserts every error path in OpenAPI has a registered mapping (an unmapped exception = 500 with a generic body — allowed, but counted: a metric on generic-500s drives the mapping backlog down over time).

Principles behind the contract, which is what interviewers actually probe: **stability** — error codes are versioned API surface; codes never get renamed (add new, deprecate old) because clients branch on them; **security** — **detail** never leaks internals (SQL, class names, stack traces are server-side log material; the log carries the stack, the response carries the correlation id — the two connect in support); **actionability** — every error answers “what can the caller do”: 409 CONCURRENT_MODIFICATION says “refresh and retry”, 422 VALIDATION says exactly which fields, 429 carries Retry-After, 402 CREDIT_EXHAUSTED links the upgrade endpoint. Consistency across auth errors matters especially: 401 vs 403 semantics are strict (401 = who are you / 403 = not allowed — never conflated), and account-existence is never revealed (forgot-password returns 202 regardless — enumeration protection shapes even status codes).

KEY TERMS TO MENTION: - RFC 7807 ProblemDetail + extensions (errors[], retryable, correlation_id) - Single @RestControllerAdvice funnel + unmapped-exception metric - Error codes as versioned surface (never rename; deprecate) - Actionability per status (Retry-After, refresh-and-retry, upgrade links) - 401/403 semantics + enumeration-safe responses

FOLLOW-UP QUESTIONS: 1. Why not just return the exception message — debugging is easier? 2. How do you handle errors from DOWNSTREAM services in your responses?

FOLLOW-UP ANSWERS: 1. Debuggability belongs in logs/traces with full context — responses go to untrusted clients; exception messages leak schema, paths, and library fingerprints (attack-recon material), and they’re unstable (every refactor breaks client string-matching — which nobody should do, but everyone does). The correlation_id gives legitimate debugging the SAME power safely: support asks for the id, we pull the full trace. Security and debuggability are both served; just in their proper layers. 2. Translate, don’t forward: a downstream 500 (LLM provider) becomes OUR 503 with retryable=true + our code AI_PROVIDER_UNAVAILABLE; downstream 4xx (Stripe card_declined) maps to our 402 with the card code preserved in a structured field (clients need it — it’s user-actionable). Never pipe downstream bodies raw (leaks their internals, couples our contract to theirs); every integration has a translation table in its adapter — tested against WireMock-simulated failure modes (Q13-4).

RED FLAGS TO AVOID: - Ad-hoc error shapes per endpoint (client hell, review hell) - Stack traces or raw exception messages in responses - 500 as the default for business-rule violations

Q8-5: Did you implement HATEOAS — and be honest about it

DIFFICULTY: Medium **CATEGORY TAG:** REST API

ANSWER: Honest answer: no full HATEOAS, and I can defend the scoped use of hypermedia we did adopt. What we have: **action affordances on stateful resources** — a lead detail response carries an **actions** object ({"can_advance_stage": {"href": "...", "method": "POST", "schema_ref": "..."}, "can_convert_to_deal" driven by server-side policy (role, plan tier, lead state) — the frontend renders buttons from server truth instead of duplicating permission logic client-side. That’s hypermedia-as-the-

engine-of-application-state in miniature, and it EARNED its complexity: when billing rules changed (Elite-only bulk export), zero frontend deploys were needed — the API stopped offering the affordance and UIs adapted. What we rejected: full HAL/JSON:API compliance with link relations everywhere — the payload overhead (2-3× response size), client-side complexity (our React team would hand-write link-following logic that fetch() idioms don't naturally use), and marginal benefit for a first-party frontend + documented public API. Spring HATEOAS exists, we evaluated it, the cost-benefit said no — with the reasoning written in an ADR so the decision is revisit-able, not tribal.

The interview value is the reasoning discipline: HATEOAS is level 3 of the Richardson maturity model, and most production APIs — including big public ones — live at level 2 (resources + verbs + status codes) because the decoupling HATEOAS buys matters most for UNKNOWN/evolving clients (public ecosystems, third-party crawlers), less for first-party pairs. Our affordance pattern captures ~70% of the practical benefit (server-driven UI capability, no client permission drift) at ~10% of the cost. I can also name when I'd go full HATEOAS: a public API ecosystem where third parties build against us long-term and breaking-change coordination is the bottleneck — then link-relations as contract earn their overhead.

KEY TERMS TO MENTION: - Richardson maturity levels; level 2 + scoped hypermedia affordances - Server-driven **actions** object from policy (no client permission drift) - Real benefit case: billing-rule change with zero frontend deploys - Rejected full HAL with cost-benefit reasoning in an ADR - When full HATEOAS would be justified (unknown third-party clients)

FOLLOW-UP QUESTIONS: 1. Doesn't the actions object couple the frontend to your shapes anyway? 2. How does this interact with API versioning?

FOLLOW-UP ANSWERS: 1. It couples to the actions CONTRACT (a small, stable schema: href/method/schema_ref) — much smaller than coupling to resource shapes or, worse, to permission RULES. Coupling is a spectrum, not a binary; the design question is which coupling is cheapest to maintain. Server-side policy changes flow through a stable affordance schema; that ratio (rule-changes : contract-changes) has been ~20:1 for us. 2. The affordance schema is versioned with the API (v2 actions can add fields additively — same rules as the rest). Removed capabilities become null affordances (not absent fields) so clients can render disabled states during sunset windows — the actions object double-serves as a deprecation surface, which we discovered in practice and wrote into the API guidelines.

RED FLAGS TO AVOID: - Claiming full HATEOAS without knowing its costs - "HATEOAS is useless" without the affordance-level middle path - No ADR/revisit story for the decision

Q8-6: How did you document your APIs (OpenAPI/Swagger)

DIFFICULTY: Easy **CATEGORY TAG:** REST API

ANSWER: Documentation is generated, tested, and published — a pipeline, not a wiki. Source of truth: **springdoc-openapi** generates the spec from code — annotations where

they add value (`@Operation(summary, description)`, `@ApiResponse` codes with examples, `@Schema` on DTOs) and structural derivation for the rest (paths, params, DTO fields, validation constraints auto-reflected: `@Size(max=100)` shows up in the spec — docs that can't drift from validation). The generated `openapi.json` is a CI artifact with gates: **spec completeness test** (every public endpoint has description, all error responses documented — a coverage report fails builds under threshold), **breaking-change detection** (oasdiff against last release — removed/renamed fields fail CI unless tagged with a deprecation label, the same gate as contract tests), and **example validation** (documented examples are schema-valid — examples rot fastest and lie loudest). Publishing: versioned static docs (Redoc/Stoplight-style render) per API version at `developers.acquisitionos.com` — PUBLIC consumers get the polished site; internal devs get the interactive Swagger UI in staging (with seeded sandbox auth).

The discipline that keeps docs true: documentation changes ship IN THE SAME PR as code changes (the completeness diff makes doc-only PRs visible — you can't change an endpoint without touching its docs because the spec diff shows in review), DTO fields carry `@Schema(description)` written for humans not restated names, and enums document every value's semantics (we audit that quarterly — enum values are the most-under-documented, most-misused surface). The honest limitation I state: annotation-based docs still drift in MEANING even when structurally complete — “what does `score` actually weight?” lives in prose descriptions that can rot; our mitigation is examples + a link from the spec to the scoring ADR, and a docs-ownership section in the code-review checklist (reviewer confirms semantic accuracy, not just presence).

KEY TERMS TO MENTION: - springdoc-openapi generation + validation-constraint reflection - CI gates: completeness, breaking-change detection (oasdiff), example validation - Docs in the same PR (spec diff in review) - Public dev-portal vs internal Swagger UI with sandbox auth - Semantic-drift mitigation: ADR links + review checklist

FOLLOW-UP QUESTIONS: 1. Code-first vs design-first — where do you land? 2. How do you document webhook schemas and error semantics?

FOLLOW-UP ANSWERS: 1. Code-first for a first-party product moving fast (docs as a build artifact can't lag the code); design-first when the API is the product (public platform, multiple teams consume the spec before implementation, SDK generation leads). We're code-first with design-first DEVIATIONS for new public surfaces: the spec for a new public module is drafted and reviewed as a doc BEFORE implementation — best of both without dogma. 2. Webhooks get their own spec section (payload schemas per event type + version, signing scheme, retry semantics, replay behavior) — generated from the same DTO classes so payloads can't drift; error semantics get a per-endpoint error table IN the spec (`@ApiResponse` with examples of every business error code) — the translation-table discipline from Q8-4 makes this derivable rather than hand-maintained.

RED FLAGS TO AVOID: - Swagger UI as “the docs” (no gates, no review = rot) - Docs updated in a follow-up PR that never comes - No breaking-change detection (spec drift becomes silent contract breaks)

Q8-7: How did you design the file upload API

DIFFICULTY: Medium **CATEGORY TAG:** REST API

ANSWER: Two protocols by file size, one contract. **Small uploads ($\leq 10\text{MB}$):** `POST /api/leads/import` with `multipart/form-data` — `@RequestParam("file") MultipartFile` + metadata part; server validates (magic-byte sniff via Tika, allow-list per endpoint, size caps enforced at Tomcat multipart layer before the app reads bytes), streams to object storage, persists metadata row (key, sha256, size, content_type, owner), returns 201 with the upload resource — client polls the import job endpoint for row-level results (parsing is async — a 500k-row CSV is a Spring Batch job, not a request lifetime). **Large uploads ($> 10\text{MB}$):** direct-to-storage protocol — client requests a session (`POST /uploads/sessions` → presigned PUT parts + upload_id), uploads parts straight to S3 (bypassing our JVM entirely — the API never becomes a byte proxy), then `POST /uploads/sessions/{id}/complete` → we verify size/checksum, run validation, mark ready. The contract nuance that matters: both paths end in the SAME upload resource shape, so clients and downstream features don't care which protocol ran.

Cross-cutting API concerns: **limits are explicit in errors** — 413 with the actual limit and measured size; **virus scanning** is a status on the upload resource (`scanning` → `clean/quarantined`) — consumers get 202 until scan completes for user-to-user surfaces (Q3-15's quarantine story, exposed honestly in the API rather than hidden); **idempotent completion** — the complete call carries a client idempotency key (double-click protection is the CLIENT's dedupe, not a hope); **resumability** on the direct path comes free from S3 multipart (part-level retry — a 2GB upload on hotel wifi succeeds); and **CORS** for the direct path is configured on the BUCKET (the browser PUTs to storage origin — preflight handled there, another reason the app is out of the byte path). Rate limits: upload endpoints count bytes in the limiter (a 10MB upload costs 10 units vs 1 for a JSON call — the limiter is bandwidth-aware or a tenant can exhaust everything with uploads).

KEY TERMS TO MENTION: - Size-split protocols: multipart ($\leq 10\text{MB}$) vs presigned direct-to-storage ($> 10\text{MB}$) - Same upload-resource contract for both paths - Async parse (Spring Batch) + 202 + polling; AV scan status exposed - Idempotent completion + bandwidth-aware rate limiting - CORS on the bucket (app out of the byte path)

FOLLOW-UP QUESTIONS: 1. Why not route everything through the server for simplicity? 2. How do you validate a 500k-row CSV without blocking anything?

FOLLOW-UP ANSWERS: 1. Through-server costs: JVM heap pressure (streaming helps but still occupies connections), doubles egress bills, adds latency (two hops), and couples our deploy/autoscaling to byte throughput. Direct-to-storage removes all four; the cost is a slightly smarter client flow — worth it above a threshold that we measured (latency + cost curves cross around 5-10MB for our payload shapes). 2. The API never parses synchronously: upload → 202 + job id → Spring Batch reads from storage in chunks (mem: constant), validates per-row (schema, dupes, transforms), writes results to an import-report resource with per-row errors downloadable as CSV; the client polls or subscribes (SSE) — progress is a first-class API resource (`processed: 240k / 500k, 1,204 rows rejected`), which turns a black-box wait into a product experience.

RED FLAGS TO AVOID: - Loading files into memory (`byte[] fileBytes` — the OOM interview signal) - Synchronous parse-and-return on large files - No idempotency on completion (double-click = double import)

Q8-8: What is your approach to API rate limiting (design across all layers)

DIFFICULTY: Hard **CATEGORY TAG:** REST API

ANSWER: Rate limiting is a PRODUCT surface with an infrastructure backbone — the design spans five layers, each with a different job. **Layer 1 — edge (WAF/ingress):** volumetric DDoS shedding (per-IP global, known-bad patterns) — coarse, cheap, no business semantics. **Layer 2 — application limiter** (Q7-6's Redis design): per-route policies keyed by identity (IP/user/key/org), algorithms chosen per budget shape (fixed-window for generous, weighted-sliding for tight), Lua-atomic. **Layer 3 — resource-cost limiting:** not all requests cost equal — the limiter is bandwidth- and compute-aware (uploads cost units by MB, AI generation by credits, exports by rows) — flat per-request limits are gameable and unfair to light users. **Layer 4 — quota product:** monthly quotas per plan (separate from burst limits — burst protects infra, quota IS the business model), exposed via headers AND a usage dashboard; overage behavior is plan-specific (hard-stop for Free, grace + overage-billing for paid). **Layer 5 — fairness within tenant:** one org's runaway script shouldn't starve its own users — org-internal per-user sub-limits on shared pools (AI credits are the natural currency here — the credit system IS the intra-org fair-share mechanism, which is why we metered compute in credits rather than inventing a parallel budget).

The API-facing contract makes limits a feature: every response carries `X-RateLimit-Limit/Remaining/Reset` (clients self-regulate — the well-behaved-client flywheel), 429s carry `Retry-After` + a machine code distinguishing `RATE_LIMIT_BURST` vs `RATE_LIMIT_QUOTA` vs `RATE_LIMIT_ORG` (different remediations: wait vs upgrade vs talk to your admin), and the developer portal documents every endpoint's limits explicitly. Observability: rejection metrics by layer/key-tier/route with top-talker panels — a rejection spike triages in seconds (attack vs broken loop vs viral customer). The anti-pattern I actively avoided: silent limits (undocumented thresholds that 429 randomly) — they generate support tickets and distrust; visible, self-served, metered limits generate well-behaved integrations.

KEY TERMS TO MENTION: - Five layers: edge volumetric → app policy → cost-aware → quota product → intra-org fairness - Burst vs quota separation (infra protection vs business model) - Bandwidth/compute-aware unit costs (flat limits are gameable) - Rate-limit headers + typed 429s (BURST/QUOTA/ORG remediation paths) - Credits as the intra-org fair-share currency

FOLLOW-UP QUESTIONS: 1. How do you set the actual NUMBERS for a new endpoint's limit? 2. A top customer keeps hitting org limits — throttle or accommodate?

FOLLOW-UP ANSWERS: 1. Start from three measurements: the endpoint's cost profile (p50/p99 server cost — the unit basis), the product's intended usage shape (what does a power user legitimately do per minute — from beta telemetry), and downstream capacity

(what burst can the DB/LLM budget absorb). Then set limit = legitimate-power-usage × 2 (headroom), load-test at 5× to find the cliff, and instrument from day one — the first month's telemetry adjusts the number; limits are hypotheses with dashboards, not eternal truths. 2. Accommodate as a product motion, protect as an engineering default: the limiter applies uniformly (no snowflake exceptions — they rot into security holes), but the customer's CSM gets the usage data, the upgrade path (Elite tier / purchased credit packs / temporarily raised quota with expiry), and if their workload is genuinely architectural (a polling loop we could webhook instead), we fix the integration — the best rate-limit story is often deleting the calls, and our webhook product exists because of this exact conversation.

RED FLAGS TO AVOID: - Only a global per-IP limiter (no identity, no cost-awareness) - Secret undocumented limits - Burst limits conflated with billing quotas

Q8-9: How did you implement request/response logging and correlation

DIFFICULTY: Medium **CATEGORY TAG:** REST API / Observability

ANSWER: The goal: any single request reconstructable from logs alone, and any user complaint traceable across services in seconds. Mechanism: a servlet **filter generates the correlation id** (accepts inbound `X-Request-Id` from the LB or mints a UUID), stuffs it into **MDC** (logback pattern renders `%X{reqId}` on every line) and echoes it back in the response header — the id travels: client → LB → app → SQL logs (datasource-proxy injects it as a comment into statements — DB-side slow-query logs self-correlate) → async threads (TaskDecorator copies MDC — Q3-9) → downstream HTTP calls (client interceptor sets the header) → error responses (Q8-4's `correlation_id` field). One id, whole-trace breadcrumbs across every log surface, zero trace-tooling dependency for the first mile of debugging — the “paste me the request id” workflow that support actually uses.

Request/response logging is deliberately ASYMMETRIC: requests log at INFO (method, path, status, duration, user id, org id, key dims — body NEVER at INFO: PII/secrets/payload bloat); response BODIES log at DEBUG with redaction (a masking converter scrubs Authorization, password, token, otp fields by pattern — the scrubber is tested against a fixtures file of real-ish payloads; logging an OTP once is a security incident). Payload caps: bodies truncated at 4KB with a size note (a 2MB JSON in logs is an ELK bill and a grep hazard). The access-log line is structured JSON (logstash encoder) so ingestion needs no regex: `{ts, level, reqId, method, path, status, durMs, userId, orgId, ip, ua}` — fielded queries (“all 500s for org X in the last hour”) are instant. Sampling strategy: 100% of 4xx/5xx and slow (>1s) requests at INFO with bodies; healthy 2xx bodies only in staging — production DEBUG is opt-in per-request (a debug header honored for support sessions, rate-limited) because always-on full logging is a cost/PII hazard dressed as observability.

KEY TERMS TO MENTION: - Correlation id lifecycle: filter → MDC → response header → SQL comments → async propagation - Asymmetric logging: metadata INFO, bodies DEBUG + redaction converter - PII/secret scrubbing with tested patterns; payload caps - Structured JSON access logs (fielded queries, no regex) - Opt-in per-request debug mode for support

FOLLOW-UP QUESTIONS: 1. Correlation ids vs distributed tracing (W3C traceparent) — why both? 2. A field you log turns out to be PII under GDPR — now what?

FOLLOW-UP ANSWERS: 1. Correlation ids are the cheap universal token every surface accepts (even the DB and third-party ticket systems); tracing adds structure (spans, timing trees, service graphs) via OpenTelemetry — we have both: the filter ALSO propagates traceparent, and the reqId is recorded as a span attribute, so ELK greps and Tempo lookups cross-reference. Correlation ids work when tracing breaks and vice versa — redundancy in the identity layer is deliberate. 2. Treat as a logging incident: stop logging the field (converter + code change), assess exposure window (who accessed ELK, was it exported), purge where possible (ELK ILM re-index/delete within retention), document in the RoPA (records of processing), and fix the classification gap (our field-classification checklist for log lines). The systemic answer: log-field additions go through a lightweight review — the checklist exists because of the incident you just described happening to us once.

RED FLAGS TO AVOID: - Full request/response bodies at INFO in production - Unredacted auth fields in logs (instant audit finding) - Logs without structured fields (grep-only debugging at scale)

Q8-10: How did you handle backwards compatibility of the API

DIFFICULTY: Hard **CATEGORY TAG:** REST API

ANSWER: Compatibility is enforced by MACHINERY, not intentions. The gates: **consumer-driven contracts** (every frontend/integration's expected requests+responses run against every PR — a breaking change fails CI naming the broken consumer), **spec-level breaking detection** (oasdiff on the OpenAPI artifact: removed/renamed/retyped fields, new REQUIRED request fields, narrowed enums — all fails unless wrapped in a version bump), and **canary consumers in CI** (staging cron jobs replay recorded production traffic weekly and diff responses against recorded shapes — catches the breaks contracts missed, like serialization quirks). The taxonomy of safe-vs-unsafe I keep the team aligned on: **SAFE** — adding optional response fields, adding endpoints, adding optional request fields, widening validations (accept more), error-code ADDITIONS (clients must handle unknown codes as generic — documented requirement); **UNSAFE** — removing/renaming anything, tightening validation, changing field semantics ("score now 0-10" under the same field is a break even if the type matches), adding required request fields, and enum value removals (the sneakiest — clients match-exhaustively).

Semantic stability gets its own discipline: field meanings are contracts too — documented invariants ("createdAt never changes after creation", "total = sum of items") carry a conformance test (property-based checks on live responses) because the most expensive breaks are semantic, invisible to schema diffs. The lived example: we deprecated `lead.score` (composite) in favor of three component scores — the path was: add new fields (safe) → mark old `@Deprecated` + Sunset header + docs (6 months) → telemetry gate on old-field usage by caller → move default response to exclude it in v2 while v1 keeps serving it (the assembler split Q8-2 enables) → post-sunset audit confirms zero references before the field

leaves the codebase. Client guidance ships with every change: migration notes with before/after examples generated from the spec diff — a breaking change without a migration note is a support incident you scheduled.

KEY TERMS TO MENTION: - Triple machinery: contract tests + oasdiff spec gate + recorded-traffic canaries - Safe/unsafe taxonomy (including semantic changes and enum removals) - Documented invariants with conformance tests (semantic stability) - Deprecation path: add → deprecate → telemetry gate → version split → audit - Migration notes generated from spec diffs

FOLLOW-UP QUESTIONS: 1. An internal frontend wants a breaking change NOW — do the gates apply? 2. How do you make unknown-error-code handling actually happen in clients?

FOLLOW-UP ANSWERS: 1. Yes — but with a faster lane: internal consumers can bump a contract pack in the same PR (the consumer is in the room), which is exactly what makes CDC workable: internal breakage is a coordinated refactor, external breakage is forbidden without versioning. The gates exist to force the CONVERSATION, and internally that conversation is one PR review; the discipline stays, the latency adapts. 2. By design and by test: the API guideline REQUIRES tolerant clients (unknown codes → generic handling) and our SDKs implement it (a base ClientException for unmapped codes) — we control both ends for first-party. For external consumers, the docs state it, examples demonstrate it, and the contract-test tooling we publish lets THEM verify their handler (we ship the test kit). You can't force third-party hygiene; you can make the correct path the easy, documented, tool-supported one.

RED FLAGS TO AVOID: - “We’re careful” as the compatibility strategy (no machinery) - Schema-only thinking (semantic breaks slip through) - Breaking changes without migration notes and telemetry gates

Category 9, 10, 11, 12, 13 & 14 — Integrations + Monolith vs Microservices + DevOps + Performance + Testing + Behavioral

CATEGORY 9 — THIRD PARTY INTEGRATIONS QUESTIONS

Q9-1: How did you integrate with Stripe for payments

DIFFICULTY: Medium **CATEGORY TAG:** Integrations

ANSWER: We use Stripe Checkout (hosted) for subscription purchases + credit addon packs, and Stripe Webhooks for payment confirmation. We never see or store credit card numbers (PCI compliance scope: SAQ-A, the lowest).

Architecture: `StripePaymentService` wraps the Stripe Java SDK (`com.stripe:stripe-java`). All calls go through `WebClient` (we don't use the SDK's HTTP client — we wire it to `WebClient` for consistent observability + Resilience4j integration). Every external call is wrapped in a Resilience4j circuit breaker (10% failure → open 30s), retry (exponential backoff 1s/4s/16s, max 3), and time limiter (10s).

Checkout flow:


```

@Service
@RequiredArgsConstructor
public class StripePaymentService implements PaymentProvider {
    private final StripeClient stripeClient; // wraps Stripe SDK
    private final Resilience4jRetryTemplate retryTemplate;
    private final PaymentOrderRepository orderRepo;

    @Override
    @Transactional
    public CheckoutSession createCheckoutSession(UUID userId, Plan
        plan, BillingCycle cycle) {
        // 1. Create PaymentOrder (PENDING) in DB
        PaymentOrder order = orderRepo.save(new PaymentOrder(userId, plan,
            cycle, ...));

        // 2. Resolve Stripe Price from env (server-side, never client-supplied)
        String priceId = resolvePriceId(plan, cycle); // env:
        STRIPE_PRICE_ID_PRO_MONTHLY

        // 3. Create Stripe Checkout Session
        SessionCreateParams params = SessionCreateParams.builder()
            .setMode(SessionCreateParams.Mode.SUBSCRIPTION)
            .setLineItem(SessionCreateParams.LineItem.builder()
                .setPrice(priceId)
                .setQuantity(1L)
                .build())
            .setCustomerEmail(userEmail)
            .setClientReferenceId(order.getId().toString())
            .putMetadata("order_id", order.getId().toString())
            .putMetadata("user_id", userId.toString())
            .putMetadata("plan", plan.name())
            .putMetadata("billing_cycle", cycle.name())
            .setSuccessUrl(appUrl + "/api/payments/stripe-success?
                session_id={CHECKOUT_SESSION_ID}")
            .setCancelUrl(appUrl + "/billing?cancelled=true")
            .build();

        Session session = retryTemplate.execute(ctx ->
            stripeClient.checkout().sessions().create(params));

        // 4. Save session ID on the order
        order.setProviderOrderId(session.getId());
        orderRepo.save(order);

        return new CheckoutSession(session.getUrl(), session.getId());
    }
}

```

Webhook verification (CRITICAL — most common Stripe bug):

```

@PostMapping("/api/payments/webhook/stripe")
public ResponseEntity<String> handleWebhook(HttpServletRequest request) {
    String rawBody = request.getReader().lines().collect(Collectors.joining()); //
        RAW bytes
    String sigHeader = request.getHeader("Stripe-Signature");

    Event event;
    try {
        event = Webhook.constructEvent(rawBody, sigHeader, stripeWebhookSecret);
    } catch (SignatureVerificationException e) {
        log.warn("Stripe webhook signature verification failed");
        return ResponseEntity.badRequest().body("Invalid signature");
    }

    // Idempotency: check if event already processed
    if (webhookRepo.findByEventId(event.getId()).isPresent()) {
        return ResponseEntity.ok("Already processed");
    }

    // Persist raw webhook for audit + async processing
    webhookRepo.save(new PaymentWebhook(event.getId(), event.getType(), rawBody,
        sigHeader, false));

    // Return 200 immediately – process async to not block Stripe
    kafkaTemplate.send("stripe-webhooks", event.getId(), event);

    return ResponseEntity.ok("Received");
}

```

Why we use `Webhook.constructEvent`: 1. Verifies the HMAC-SHA256 signature using `STRIPE_WEBHOOK_SECRET` (per-endpoint secret, configured in Stripe dashboard) 2. Parses the raw body (must be raw bytes — if you JSON-parse first, signature won't match due to whitespace differences) 3. Handles Stripe's signature scheme (v1 signed payload: `t=...,v1=...`)

Idempotency: Stripe can retry a webhook up to 5 times in 5 minutes if we return non-2xx. We return 200 immediately after ingesting the raw payload (no processing in the request handler). Processing happens in a Kafka consumer — if it crashes, Kafka redelivers. The `payment_webhook` table has a unique constraint on `event_id`; the consumer checks if processed before acting.

Reconciliation: Daily job (3am IST) pulls all Stripe events from the last 24h via Stripe API (`events.list(created={gte: yesterday})`) and reconciles with our `payment_webhook` table. If any Stripe event is missing from our DB, we backfill. Catches webhook delivery failures (rare but happens — Stripe's SLA is 99.95% webhook delivery).

Secret rotation: `STRIPE_WEBHOOK_SECRET` is in AWS Secrets Manager; rotated quarterly. We support both old and new secret for 24 hours during rotation (try the new secret first, fall back to old). After 24h, we drop the old. We've done this twice with no incidents.

Refunds: Initiated by admin via `POST /api/admin/refund`. We call `Refunds.create()` with the `paymentIntentId`. On the `charge.refunded` webhook, we mark the subscription as REFUNDED

(full) or just log (partial). Refunds also issue a `credit_refund` ledger entry if the user had spent credits during the refunded period (rare edge case — manual review).

Testing: WireMock for unit + integration tests (stub Stripe’s webhook payload + signature). Stripe CLI for staging (`stripe trigger event checkout.session.completed` fires a real webhook to staging with a real signature). End-to-end in staging with a real Stripe test card (`4242 4242 4242 4242`) — full checkout → real webhook → DB state.

KEY TERMS TO MENTION: - Stripe Checkout (hosted) — never see card numbers (PCI SAQ-A) - `Webhook.constructEvent(rawBody, sig, secret)` — raw bytes, not JSON-parsed - Idempotency via unique `event_id` in `payment_webhook` table + Kafka consumer - Return 200 immediately, process async (decouple from Stripe retry pressure) - Resilience4j CB + retry + time limiter around every Stripe call - Daily reconciliation job (pull Stripe events, backfill missing) - Secret rotation quarterly, dual-secret window 24h - Refunds via `Refunds.create()` + `charge.refunded` webhook - WireMock + Stripe CLI + real test card for testing

FOLLOW-UP QUESTIONS: 1. What if Stripe’s webhook secret is leaked? 2. How do you handle a Stripe webhook storm (600 in 2 minutes)? 3. Why don’t you use Stripe’s Java SDK’s HTTP client?

FOLLOW-UP ANSWERS: 1. Two layers: (a) the secret is in Secrets Manager — leak requires Secrets Manager access (audit-logged); (b) the webhook endpoint is on a public URL but only accepts requests with a valid signature — without the secret, an attacker can’t forge a webhook. If we suspect leak: rotate immediately (Stripe dashboard → regenerate endpoint secret → update Secrets Manager → dual-secret window 24h → drop old). We’ve never had a leak in 9 months. 2. We had this once — Stripe retries aggressively when it sees a 500. Root cause: a transactional boundary mistake (one bad webhook held a DB connection for 30s, HikariCP exhausted at 20 connections). Fix: split handler into “ingest” (just persist raw webhook, return 200 to Stripe) + “process” (async via Kafka consumer, per-event transaction). Now even 600 webhooks in 2 minutes just means 600 inserts (fast) and 600 Kafka messages (queued) — no connection exhaustion. We added a load test that fires 1000 concurrent webhooks as a regression test. 3. We use Stripe’s Java SDK’s `RequestOptions` to inject our own `HttpClient` (built on Spring’s `WebClient` via `ReactorClientHttpConnector`). This gives us: (a) consistent observability (our `TraceWebClientFilter` adds OpenTelemetry spans to Stripe calls); (b) Resilience4j integration (the same circuit breaker pattern we use for Google/Twilio); (c) shared connection pool (no per-SDK pool). The trade-off: slight extra configuration vs. the SDK’s defaults. We’ve never had a problem with it.

RED FLAGS TO AVOID: - JSON-parsing the webhook before signature verification - Processing the webhook synchronously (Stripe retry storm) - Storing the webhook secret in `application.yml` - Not testing the webhook flow end-to-end with real signatures

Q9-2: How did you implement webhook handling (Stripe webhooks)

DIFFICULTY: Hard **CATEGORY TAG:** Integrations

ANSWER: Three webhooks: Stripe (payments), Gmail PubSub (email replies), Twilio (WhatsApp replies). All three follow the same pattern: ingest raw → verify signature → persist raw → return 200 → process async via Kafka consumer with idempotency.

Pattern (applied to all three):

```

External API → POST /api/webhooks/{provider}
|
▼
WebhookController.handle(request):
├ read raw body (request.text() or request.getReader())
├ verify signature (HMAC-SHA256 with provider secret)
│   └ Stripe: Webhook.constructEvent(rawBody, sig, STRIPE_WEBHOOK_SECRET)
│   └ Twilio: HMAC-SHA256 of URL-encoded request body with TWILIO_AUTH_TOKEN
│   └ Gmail PubSub: JWT verification with Google's public keys (OIDC)
├ if invalid → 400 (provider will retry)
├ idempotency check: webhookRepo.findByEventId(eventId)
│   └ if exists & processed → return 200 (idempotent)
├ persist raw webhook in {provider}_webhook table (PROCESSED=false)
├ return 200 immediately (don't block the provider)
└ publish {Provider}WebhookReceivedEvent to Kafka topic
    |
    ▼
Kafka consumer ({Provider}WebhookProcessor):
@Transactional
├ lookup webhook in DB by event_id
├ if PROCESSED → return (idempotent across Kafka redeliveries)
├ switch on event.type:
│   └ process the event (call service methods)
│       └ on exception → mark as ERRORRED + retry (Resilience4j) or DLQ
└ mark webhook as PROCESSED, processed_at=now()

```

Why this pattern: 1. **Decouple from provider retry pressure** — Stripe/Twilio expect 200 within 10s; if we process synchronously, slow processing causes retry storms 2. **Idempotent across redeliveries** — both the provider (Stripe) and our message bus (Kafka) redeliver; the `event_id` unique constraint prevents double-processing 3. **Durable** — the raw webhook is in the DB even if Kafka is down; reconciliation job picks it up 4. **Observable** — every webhook is a row in the DB (audit) + a Kafka message (trace)

Stripe specifics: Covered in Q9-1. Key: `Webhook.constructEvent` with raw bytes, `STRIPE_WEBHOOK_SECRET` from Secrets Manager, rotated quarterly.

Twilio specifics: Twilio webhooks come as form-encoded (`application/x-www-form-urlencoded`). We verify with Twilio's signature scheme: HMAC-SHA256 of the URL (including query string) + sorted parameters, with `TWILIO_AUTH_TOKEN` as the key. The signature is in the `X-Twilio-Signature` header (base64). We use Twilio's Java SDK's `RequestValidator.validate(url, params, signature)`.

```
@PostMapping("/api/webhooks/twilio/whatsapp")
public ResponseEntity<String> handleTwilio(@RequestParam Map<String, String> params,
                                           HttpServletRequest request) {
    String url = request.getRequestURL().toString() + "?" + request.getQueryString();
    String signature = request.getHeader("X-Twilio-Signature");
    if (!twilioRequestValidator.validate(url, params, signature)) {
        return ResponseEntity.badRequest().body("Invalid signature");
    }
    // ... same pattern: persist raw + return 200 + process async
}
```

Gmail PubSub specifics: Gmail pushes notifications to a PubSub topic when a new email arrives in a user's inbox (we subscribe per-user via `users.watch`). PubSub delivers via webhook (POST to `/api/webhooks/gmail/reply`) with a JWT in the body. We verify the JWT's signature using Google's public keys (OIDC discovery at <https://www.googleapis.com/oauth2/v3/certs>) + verify the `aud` claim matches our project. We use Spring Security's `JwtDecoder` (Nimbus) for verification.

After verification, we don't have the email body — just the notification. We pull the email body via Gmail API (`users.messages.get`) using the user's stored refresh token. The pull is async (Kafka consumer) — the webhook just acks the notification.

Idempotency across all three: Each provider's event has a unique ID — Stripe's `event.id` (`evt_...`), Twilio's `MessageSid` (`SM...`), Gmail's `messageId` + `historyId`. We use these as the `event_id` in our `payment_webhook` / `twilio_webhook` / `gmail_event` tables, with a unique constraint.

Reconciliation (all three): Daily job pulls events from each provider's API and reconciles with our DB. Stripe: `events.list()`. Twilio: `messages.list()`. Gmail: `users.history.list()` (per user, since they OAuth'd). Any missing event is backfilled.

Failure modes: - Provider retries a `processed` webhook → idempotency check returns 200 - Kafka consumer crashes mid-processing → webhook stays `PROCESSED=false`; consumer redelivers; idempotency check at the start of processing prevents double-processing if the crash happened after processing but before marking as `PROCESSED` - Provider's secret is rotated → dual-secret window (both old + new valid for 24h) - Webhook payload is malformed → catch parse error, return 400, provider retries (with the same malformed payload — eventually goes to DLQ after N retries)

Dead-letter queue: If a webhook fails processing after 3 retries (Resilience4j), we publish to a `dlq-{provider}-webhooks` Kafka topic. An admin dashboard shows DLQ events; admin can reprocess manually.

KEY TERMS TO MENTION: - Three webhooks: Stripe, Twilio (WhatsApp), Gmail PubSub - Universal pattern: verify signature → persist raw → return 200 → process async via Kafka - Signature verification per provider: `Webhook.constructEvent` (Stripe), `RequestValidator.validate` (Twilio), JWT OIDC verification (Gmail PubSub) - Raw bytes for signature verification (never JSON-parse first) - Idempotency via unique `event_id` per provider, DB unique constraint - Dual-secret window for rotation - Daily reconciliation per

provider (Stripe `events.list`, Twilio `messages.list`, Gmail `users.history.list`) - DLQ for failed-after-retry webhooks, admin dashboard for reprocess

FOLLOW-UP QUESTIONS: 1. How do you test webhook signature verification? 2. What if a provider changes their webhook payload schema? 3. How do you handle a webhook that triggers a side effect (email send) — what if the email is sent twice?

FOLLOW-UP ANSWERS: 1. WireMock for unit tests — stub the provider’s webhook payload with a real signature (Stripe CLI generates signatures; Twilio SDK’s `RequestValidator` generates for our test; Google’s JWT can be generated with the service account key). For integration tests, we hit the real provider in staging (Stripe CLI’s `stripe trigger`, Twilio’s test mode, Gmail’s sandbox). The signature verification logic is unit-tested separately with hardcoded signed payloads (golden file). 2. Three defenses: (a) **defensive parsing** — we use Jackson’s `failOnUnknownProperties = false` so new fields are ignored (additive changes are safe); (b) **versioned webhook endpoints** — we have `/api/webhooks/stripe/v1` and can add `/v2` if Stripe makes a breaking change (rare); (c) **alerting on parse failures** — if Jackson fails to parse, we send to DLQ + alert. Stripe makes additive changes (new event types, new fields); we add handlers for new event types as they come. 3. The email-send side effect is wrapped in the Kafka consumer’s `@Transactional` — the DB write (mark message as SENT) + the email send happen in one transaction... wait, SMTP send is not transactional. So: we mark the message as `SENDING` (DB write), commit, then send the email (not in the transaction), then mark as `SENT` (DB write). If the consumer crashes after the email is sent but before the second DB write, the message stays in `SENDING` state. A `@Scheduled` job finds messages in `SENDING` for >5 min and resends them — but we use the SMTP message-id for idempotency: if the email was already sent, we get a “duplicate message-id” response from our tracking system and don’t resend. We’ve had 1 duplicate email in 9 months (a crash mid-flow). Acceptable.

RED FLAGS TO AVOID: - JSON-parsing before signature verification (signature won’t match) - Synchronous processing (provider retry storm) - No idempotency (double-charges) - No DLQ (failed webhooks lost)

(The remaining integration questions — Google OAuth2 integration, Gmail SMTP integration, Google Calendar API, Telegram bot notifications, WhatsApp via Twilio, API key management, third-party API failure handling, retry logic with `@Retryable`, circuit breaker patterns, webhook secret rotation, provider-specific gotchas (Gmail PubSub ack deadline, Twilio’s rate limits), idempotency across provider retries — follow the same template. Drill on: what happens when Stripe’s API is down for an hour, how you handle Gmail’s `PushNotification` ack deadline, how you’d implement a generic `Integration` framework to support adding new providers.)

Q9-3: How did you implement Google OAuth integration on the backend

DIFFICULTY: Hard **CATEGORY TAG:** Third-Party Integration / Security

ANSWER: OAuth2 authorization-code flow with PKCE, implemented via Spring Security's `oauth2-client` with our own persistence layer for multi-account support. Flow: frontend hits `/api/auth/google/start` → server builds the authorization request (`client_id`, `redirect_uri` = exact registered URL, `scope` = `openid email profile` + calendar scopes ONLY when the user opts into scheduling, `state` = CSRF-bound nonce stored server-side (Redis, 10-min TTL, single-use), `code_challenge` for PKCE) → Google consent → callback `/api/auth/google/callback` receives `code` + `state` → server validates state (lookup + delete — replay-proof), exchanges code at Google's token endpoint (`client_id` + `client_secret` + `code_verifier`) → receives `access_token` + `id_token` + `refresh_token`. The `id_token` is a JWT — we verify signature against Google's JWKS (`iss`, `aud`, `exp`, `nonce`), extract `sub` (immutable Google user id — the account key, NOT email: emails can change/re-assign), and link-or-create the local user.

The decisions interviewers probe: **refresh token storage** — encrypted (AES-256-GCM envelope encryption, KMS-held key) in `oauth_connection`, because a refresh token is a PERMANENT credential to the user's Google identity; **scope discipline** — sign-in scopes vs calendar scopes are separate consent moments (incremental auth) so "sign in with Google" doesn't scare users with calendar warnings; **account linking rules** — Google `sub` already linked to another local user → deny with a clear message (never silently merge identities — account-takeover vector); **token lifecycle** — access tokens (~1h) used for Calendar/Gmail API calls are refreshed on-demand by a provider wrapper (single-flight refresh per user — concurrent 401s trigger ONE refresh, others await; Google's token-revocation-on-parallel-refresh behavior makes unguarded refresh a bug factory); **disconnection** — revoke at Google (token revoke endpoint) AND delete local rows, because leaving orphaned grants is both a security smell and a support mystery.

Failure handling: state mismatch/expired → 400 with generic message (no oracle about what failed); token exchange failure → 502 with correlation id; JWKS fetch failure → cached keys + retry (key rotation tolerance — Google rotates, our cache honors `kid`); and the whole flow is rate-limited (per-IP on start, per-account on callback) because auth endpoints are attack surface even when delegated to Google.

KEY TERMS TO MENTION: - Authorization code + PKCE + server-side single-use state (CSRF/replay) - `id_token` JWKS verification; `sub` as identity key (not email) - Envelope-encrypted refresh tokens; incremental scopes - Single-flight token refresh wrapper (parallel-refresh hazard) - Account-linking rules (no silent merges) + full disconnect revocation

FOLLOW-UP QUESTIONS: 1. Why server-side flow instead of implicit/token flow from the browser? 2. Google returns email X but the local account with that email has a password — what happens?

FOLLOW-UP ANSWERS: 1. Implicit is dead (deprecated in OAuth 2.1): tokens in URL fragments leak via history/referrers, no refresh tokens, no client authentication. Code + PKCE gets us short-lived codes, PKCE binding (interception-proof), refresh tokens server-side, and

one place to enforce state/nonce. Every serious integration moved; I can cite the 2.1 spec shifts if asked. 2. The linking decision: if the local account is VERIFIED (email verified via our OTP/magic flow), we auto-link (Google verified the same email) and audit-log it; if unverified, we require the password (prove prior ownership) before linking — because an attacker who hijacks a Google account must not inherit a local account by email collision alone. The rule is written in the auth ADR with the attack tree it closes.

RED FLAGS TO AVOID: - Storing refresh tokens in plaintext or localStorage - Using email as the immutable identity key - No state/PKCE (the “we skipped it for demo” answer — instant red flag at senior level)

Q9-4: How did you integrate Gmail SMTP for sending

DIFFICULTY: Medium **CATEGORY TAG:** Third-Party Integration

ANSWER: Two distinct Gmail integrations, often confused: **transactional/platform email** (order confirmations, invoices — from OUR sender) uses classic SMTP with an app password/OAuth service account via `JavaMailSender` — boring, reliable, and it should be boring. **User-identity outreach email** (the product core — sends from the USER’s own Gmail so deliverability rides their domain reputation) uses Gmail API with OAuth (Q9-3’s stored refresh tokens) rather than SMTP — API gives us message threading (thread_id for conversations), labels, and bounce metadata SMTP lacks. The sending wrapper (`GmailSendAdapter implements EmailPort`): builds `MimeMessage`-equivalent (`base64Url` raw RFC2822), sets our idempotency header (`X-Acq-Message-Key` — the dedupe key from Q2-15), sends, and persists the returned provider message id back to the message row — that id is the reconciliation key for bounces (Gmail PubSub push on bounce events → match → mark BOUNCED → suppression list).

Rate and quota management is the operational heart: per-user Gmail quotas (send limits ~500/day consumer, 2000/day workspace) enforced by our Redis token bucket PER MAILBOX (delay-not-fail semantics — campaign sends queue to next window rather than erroring), plus burst pacing (10/min/mailbox — human-shaped sending patterns protect deliverability; machine-gun sends from a fresh mailbox are the classic spam signal we engineer away). Failure taxonomy in the adapter: 429 → retry with backoff + bucket tightening (provider is telling us our pacing is wrong — listen); 401/403 (token revoked, app suspended) → mark connection NEEDS_REAUTH, user notification via in-app banner (never retry-loop an auth failure); 5xx → circuit breaker territory (Q3-12); and the timeout false-negative handled by the idempotency key + reconciliation sweep (the Q2-15 war story). Deliverability hygiene built-in: SPF/DKIM/DMARC ride the user’s own domain (their Gmail handles it — a core product argument), unsubscribe headers on sequences, and per-recipient suppression checked BEFORE compose (bounce lists, unsubscribes, manual blocks — one SQL check, non-negotiable).

KEY TERMS TO MENTION: - Two integrations: platform SMTP (JavaMailSender) vs user-identity Gmail API - Idempotency header + provider message id reconciliation - Per-mailbox

token bucket (delay-not-fail) + human-paced bursts - Failure taxonomy: 429 backoff vs 401 reauth-flow vs 5xx breaker - Pre-compose suppression check (bounces/unsubscribes/blocks)

FOLLOW-UP QUESTIONS: 1. Why Gmail API over raw SMTP for user sending? 2. A user's mailbox hits its daily cap mid-campaign — what's the UX?

FOLLOW-UP ANSWERS: 1. Threading and reply-intelligence need `thread_id` and label context (our reply classifier reads thread state); SMTP gives us a fire-and-forget pipe with no conversation graph. Also OAuth-scoped API access is narrower (gmail.send only) than full SMTP credentials — least privilege for the same job. SMTP remains the fallback transport for providers without APIs (some IMAP-only setups) via the same EmailPort. 2. The campaign engine's bucket delays remaining sends to the next window automatically — the UI shows "sending: 340/1200, resuming at 04:00 UTC (mailbox daily limit)" rather than an error wall; users get a deliverability notice suggesting warm-up pacing. The alternative (blast until provider errors) burns the user's sender reputation — protecting it IS the product's job.

RED FLAGS TO AVOID: - Confusing the two Gmail integrations (platform vs user identity) - Retry loops on 401/403 (auth failures aren't transient) - No suppression check before compose (bounce spiral)

Q9-5: How did you integrate Google Calendar API

DIFFICULTY: Medium **CATEGORY TAG:** Third-Party Integration

ANSWER: The scheduler's Calendar layer (full product flow in Q2-12) — the integration engineering: **connection bootstrap** — OAuth with calendar scopes (incremental consent), store encrypted refresh token, immediately verify with a `calendarList.list` probe (fail fast on scope gaps); **free/busy reads** — the `freebusy` batch query (up to N calendars per call) for 14-day windows, response cached in Redis (30s TTL — booking pages re-render frequently; staleness of 30s is invisible but protects quota); **event writes** — `events.insert` with conferenceData (Meet links), attendees, and our metadata in extendedProperties (our slot id rides the event — the reconciliation anchor), on failure-after-DB-commit the compensation job (Q2-12) retries using that anchor; **push sync** — watch channels per connected calendar (expiration-based — channels expire in ~7 days, a `@Scheduled` job renews them BEFORE expiry; a lapsed channel silently degrades to stale busy-data, which is the failure mode that books double-bookings — so channel-expiry monitoring is an alarm, not a log line); **incremental sync** — `events.list` with `syncToken` on push notification (full resync on 410 GONE — token expired — with a window cap).

Quota engineering: Calendar API budget is per-project and shared across all users — the integration tracks per-user AND project-level quota via response headers, and degrades gracefully (booking-page availability checks fall back to a longer cache TTL under quota pressure — a booking page that's 2-min stale beats one that 500s). Timezone correctness is where integrations die: all reads/writes carry explicit `timeZone` params, the user's working-hours config stores tz with IANA ids, DST transitions handled by computing slots in the USER'S tz then converting at the boundary — the test suite has DST-transition fixtures (spring-forward gap hours) because "meeting moved an hour after DST" is the bug every

scheduler ships once. Reliability grading: Calendar outages degrade scheduling to “propose slots from last-known busy map + user confirmation” (explicitly flagged in UI) — the show goes on with human verification, never silently books against stale data.

KEY TERMS TO MENTION: - freebusy batch + Redis-cached reads; events.insert with extendedProperties anchor - Watch channels with expiry + renewal job + lapse alarm - syncToken incremental sync + 410 full-resync path - Quota headers + graceful TTL degradation under pressure - DST-transition test fixtures (the scheduler’s classic bug)

FOLLOW-UP QUESTIONS: 1. A watch channel dies silently — what actually breaks and when do you know? 2. Why cache free/busy — isn’t correctness critical for bookings?

FOLLOW-UP ANSWERS: 1. Busy maps go stale: bookings get proposed against outdated calendars → conflicts discovered at the Google-insert step (our compensation flow catches them — user gets an apology + re-propose). Detection: channel-expiry sweeper alarms 48h before lapse AND a “last push received” freshness metric per connection (>25h silence on an active calendar = investigate). The compensation layer is what makes the failure soft; the monitoring is what makes it rare. 2. Correctness at BOOKING time is DB-enforced (slot claim + conflict check at insert + compensation), not cache-enforced — the cache serves proposal RENDERING (which slots to show), the commit path re-validates against Google directly. Stale-render shows a slot that then fails atomically; uncached-render hammers quota at 300µs-per-read × every booking-page view. Layer the consistency where it’s enforcement vs presentation.

RED FLAGS TO AVOID: - Polling Calendar instead of watch channels (quota death) - No channel-expiry awareness (the silent staleness killer) - Timezone hand-waving (DST fixtures or it isn’t real)

Q9-6: How did you implement Telegram bot notifications

DIFFICULTY: Easy **CATEGORY TAG:** Third-Party Integration

ANSWER: The lightest integration in the system, which is exactly why it’s a good interview answer — showing engineering proportion. Users connect via a bot deep-link (`t.me/OurBot?start=<linkToken>`); the linkToken (single-use, 10-min TTL, bound to the pending user) ties the Telegram chat_id to the local account on the bot’s `/start` — stored in `notification_channel` (channel=TELEGRAM, handle=chat_id, verified=true). Sending is one HTTP call to the Bot API (`sendMessage` with `parse_mode=HTML`, `disable_web_page_preview`) wrapped in the standard `ChannelSender` SPI (Q3-20’s router) — retries on 429 (Telegram’s per-chat flood limits: ~1 msg/sec/chat, 30/min broadcast-ish) with backoff, permanent fail on 403 (user blocked the bot → mark channel UNVERIFIED + stop sending — respect the block, never retry it).

Where the engineering actually lives: **notification shaping** — Telegram gets SHORT messages (2-3 lines + one action link), composed per-channel from the same domain event (the `NotificationComposer` renders per-channel templates: Telegram = terse, Email = full, Push = one line) — channel-appropriate rendering, not copy-paste; **bot command surface**

— `/stop` (opt-out instantly, honors Telegram’s TOS — messaging platforms REQUIRE clean opt-out or they ban your bot), `/status` (read-only account summary via a scoped internal API — the bot backend uses a service account with minimal scopes, not user tokens); **webhook vs polling** — webhook (setWebhook to our HTTPS endpoint with the `secret_token` header filter — Telegram sends the secret we validate, killing spoofed updates) over long-polling (no idle connections on our API pods); **rate and abuse** — per-chat send bucket + a global bot budget, and link tokens are single-use to prevent account-link hijacking via guessed tokens. The whole integration is ~600 lines, one adapter class, one table — and it respects one principle: lightweight channel, proportionate engineering, but the same security and failure discipline as the big ones (verified linking, opt-out, rate limits, idempotent sends).

KEY TERMS TO MENTION: - Deep-link token binding (single-use, TTL) for account linking - ChannelSender SPI + per-channel template rendering (terse-not-copied) - Webhook with secret-token validation (not polling); 403 = permanent opt-out - Per-chat flood limits + global bot budget (429 backoff) - `/stop` instant opt-out (platform TOS survival)

FOLLOW-UP QUESTIONS: 1. Why not reuse the notification table for chat state? 2. Bot API is down — what degrades?

FOLLOW-UP ANSWERS: 1. It IS the same table (`notification_channel`) — that’s the design: channels are a type, handlers register by type, adding WhatsApp later touched zero shared schema. The point of the answer is the SHAPE (SPI + typed channels) rather than the specific channel; Telegram was chosen as the demo because its simplicity shows the pattern without clouding it. 2. Telegram joins the circuit-breaker registry: opens → sends queue in the job table (at-least-once, TTL-capped — a 6-hour-old “meeting reminder” is worse than none, so notifications have freshness windows per type) and the dashboard marks the channel degraded. Critical-tier notifications (security alerts) have an email fallback chain — the channel router tries TELEGRAM → falls to EMAIL on breaker-open, per the notification’s criticality class.

RED FLAGS TO AVOID: - Treating a light integration as no engineering (linking security, opt-out still apply) - Retrying blocked-user 403s (burns quota, annoys users) - Copy-pasting email copy into Telegram (channel-inappropriate rendering)

Q9-7: How did you handle WhatsApp integration via Twilio

DIFFICULTY: Medium **CATEGORY TAG:** Third-Party Integration

ANSWER: WhatsApp is the highest-friction channel (platform rules, template approvals, per-message costs) — the integration design reflects that. Architecture: Twilio WhatsApp Business API via their REST (`/Messages.json`), through the same `ChannelSender` SPI; outbound requires **pre-approved templates** (Meta’s policy — no free-form business-initiated messages outside a 24h customer-service window): template names + variable maps are config (`whatsapp_templates` table — name, language, body-variables, approval status), the composer renders variable payloads, and a template NOT in approved state refuses to send (a config check, not a hope — sending unapproved = Twilio error + platform

reputation damage). Inbound replies arrive via Twilio webhook (`/api/webhooks/twilio/whatsapp`) with **signature validation** (X-Twilio-Signature HMAC over the URL + params with our auth token — Twilio’s exact algorithm, implemented and unit-tested; an unsigned/spoofed webhook must die at the filter, this is the injection surface), then the same pipeline as email replies: dedupe by MessageSid (UNIQUE), classify (reply intelligence), attach to lead thread.

Cost and rate discipline: per-message cost is metered into the org’s credit system (WhatsApp costs 4 credits vs email 1 — the credit table IS the productization of channel costs), send rate respects Twilio’s per-number throughput (queue + pacing in the job layer), and the number-quality dashboard (engagement + block rates) is operational surface — WhatsApp numbers degrade (quality rating drops → messaging limits drop), so block-rate spikes alarm (users marking messages as spam is the channel’s health signal). Failure handling follows the Q9-4 taxonomy (429 backoff, 6xxxx codes → permanent vs transient classification, template-rejection → config alert) plus the idempotency key header (Twilio doesn’t dedupe — OUR message-key dedupes via the unique constraint + pre-send Bloom check). The 24-hour session window logic: inbound user message opens a 24h free-form window (stored on the conversation row with expiry) — replies within it are free-form; after it, template-only — the composer checks the window state per send, because that platform rule, encoded in code, is the difference between a compliant integration and a suspended number.

KEY TERMS TO MENTION: - Twilio REST via ChannelSender SPI; pre-approved templates as config with send-time enforcement - X-Twilio-Signature HMAC validation (spoof-proof webhooks) + MessageSid dedupe - Channel costs metered via credits (WhatsApp 4× email) - 24h customer-service window encoded in the composer - Number quality metrics (block rates) as alarms

FOLLOW-UP QUESTIONS: 1. Why Twilio over Meta’s Cloud API directly? 2. A campaign’s block rate jumps 5× — walk me through your response.

FOLLOW-UP ANSWERS: 1. Twilio bought us: one API for SMS+WhatsApp fallback, sane webhook infra, compliance tooling, and time-to-market — the trade is per-message margin. The Meta Cloud API is the direct path (lower cost, first-party) and our `ChannelSender` SPI makes that swap an adapter change, not a product change — the abstraction exists precisely so vendor economics can be revisited without re-architecture. I can name the migration checklist (number porting, template re-approval, webhook cutover) if asked. 2. Treat as an incident: pause the campaign (auto-triggered at threshold — the pause is automated, the INVESTIGATION is human), segment the blocks (which template, which audience slice, which day), usual culprits: template drift (content no longer matching approved tone), audience quality (purchased-ish lists), frequency (3 messages in 2 days). Remediation: kill the offending template, tighten the audience filter, re-space the sequence; number quality recovers over sends. The metric exists so this is caught in hours, not when Twilio downgrades the number.

RED FLAGS TO AVOID: - Free-form business-initiated messages (policy violation = number suspension) - Unsigned webhook acceptance (spoofable injection surface) - No block-rate monitoring (channel death is silent until limits hit)

Q9-8: How did you implement webhook handling generally (the pattern, not one provider)

DIFFICULTY: Hard **CATEGORY TAG:** Third-Party Integration

ANSWER: Every provider webhook (Stripe, Twilio, Gmail PubSub, Google Calendar, WhatsApp) flows through ONE pipeline with provider-specific entry validators — because the failure modes are universal and the pipeline should be written once. The stages: **(1) Transport gate** — dedicated routes OUTSIDE session auth (`permitAll` + their own defense: signature/HMAC validation per provider's exact algorithm — Stripe's `Stripe-Signature` with tolerance window, Twilio HMAC, Google's OIDC JWT on PubSub pushes — all unit-tested against forged vectors); **(2) Raw capture** — the RAW body is read (signature checks need raw bytes, not re-serialized JSON — a classic bug when frameworks parse first), and the event is persisted IMMEDIATELY as `webhook_event` (provider, event id, type, payload, received_at, status=RECEIVED) returning 2xx fast — **persist-then-ack:** providers retry on non-2xx, so durability must precede the ack; slow downstream processing MUST NOT delay the ack (provider timeout → retry storm); **(3) Dedupe** — provider event id UNIQUE constraint; replays (Stripe's at-least-once, manual replays) hit the constraint and no-op — idempotency is structural, not behavioral; **(4) Async dispatch** — job-table worker processes by type via a handler registry (`Map<eventType, Handler>` — adding a provider = validator + handler + config, the pipeline untouched); handlers are idempotent + transactional; failures mark the row FAILED with attempt counts → retry with backoff → dead-letter state for manual replay (the admin UI has a replay button — webhook debugging is a UI, not SQL archaeology); **(5) Observability** — per-provider success/latency metrics, freshness alarms (Stripe webhooks silent > 30 min = connectivity break — the earliest signal, before customers notice missing invoices).

The interview-worthy war story embedded in the design: early on, a Stripe handler threw AFTER the ack during a deploy (new code, old handler gone) — events sat RECEIVED forever because “acked = done” was assumed. The pipeline fix was the freshness alarm + a reconciliation sweep (hourly: RECEIVED rows older than 10 min → re-dispatch), which converts any future handling gap from silent data loss to a self-healing blip. That's the pattern's real value: universal durability + universal replay, so provider quirks (replays, outages, our deploys) are absorbed by ONE mechanism instead of N handler conventions.

KEY TERMS TO MENTION: - Raw-body capture BEFORE parsing (signature integrity) - Persist-then-ack (2xx after durability — retry-storm prevention) - Event-id UNIQUE dedupe (structural idempotency) - Handler registry + job-table async processing + dead-letter replay UI - Freshness alarms + hourly reconciliation sweep (the war story)

FOLLOW-UP QUESTIONS: 1. Why not process webhooks synchronously — the extra table seems heavy? 2. A provider sends an event type you don't know — behavior?

FOLLOW-UP ANSWERS: 1. Synchronous processing couples our p99 to provider retry behavior: a 2s handler under a Stripe timeout (they allow ~10s but retry aggressively) turns any slow patch into duplicate-event storms; and handler crashes after ack = lost events (the war story). The table IS the feature: durability, replay, debug, and audit in one — ~50ms of write overhead per event, cheap for transactional integrity on money-adjacent flows. 2.

Persist + mark UNHANDLED + metric + no error (2xx — unknown types are versioning noise, not failures; erroring on them teaches the provider to retry forever). A weekly review of UNHANDLED types feeds the handler backlog — new Stripe event types appear without announcements, and the pipeline is designed so their arrival is data, not downtime.

RED FLAGS TO AVOID: - Signature check after JSON re-serialization (validation always fails/mismatches) - 500 on handler exception after ack (retry storms + duplicates) - Per-provider ad-hoc pipelines (N places to fix the same bug)

Q9-9: How did you manage API keys for third parties (storage, rotation, access)

DIFFICULTY: Medium **CATEGORY TAG:** Third-Party Integration / Security

ANSWER: Three key classes with three storage strategies. **Our credentials to providers** (Stripe secret key, Twilio auth token, Google client secret): secrets manager (AWS Secrets Manager) → init sidecar → env vars → `@ConfigurationProperties` — never in git, never in images, never in logs (the masking converter covers accidental prints); access is role-scoped (the app's IAM role reads only its secret path). **Provider keys ON BEHALF of users** (Google refresh tokens, mailbox tokens): envelope-encrypted at rest (AES-256-GCM, data key wrapped by KMS master key — the DB stores ciphertext + wrapped key; compromise of the DB alone yields nothing usable), decryption scoped to the request path (decrypt-in-memory, never persisted decrypted, audit-logged per use), and revocable both directions (our disconnect revokes upstream; provider-side revocation detected via 401 → connection marked NEEDS_REAUTH). **Customer-facing API keys** (partners calling OUR API): show-once + SHA-256 hash storage + scopes + expiry (Q4-13's full design).

Rotation discipline per class: provider credentials rotate quarterly via the secrets-manager versioning (two-valid window, rolling restart — the procedure is rehearsed, not theoretical); user-level tokens refresh themselves (rotation-on-refresh from Google's side); customer keys are customer-initiated (our dashboard drives it) with 90-day expiry defaults. The audit surface: every key USE is logged with key-id + purpose (Q4-15's audit trail), and a quarterly "key census" job inventories all secrets in the system (env-scan + repo-scan (gitleaks) + dashboard of every key with age/last-use/owner) — the census is what catches the zombie key (created for a 2024 POC, never rotated, owner gone) before an auditor or attacker does. Least-privilege is enforced at issuance: a Stripe key has webhook-signing and checkout scopes, never account-wide; the temptation to reuse an all-powerful key across features is a review-blocked pattern.

KEY TERMS TO MENTION: - Secrets manager → sidecar → env → typed config (no git/image/logs) - Envelope encryption (KMS-wrapped data keys) for user-granted tokens - Rotation per class with two-valid windows + quarterly census (zombie-key detection) - Use-attribution audit + least-privilege scoping at issuance - Bidirectional revocation (our disconnect + upstream-revocation detection)

FOLLOW-UP QUESTIONS: 1. A third-party key leaks to a public repo — your response runbook? 2. Why envelope encryption over field-level encryption with one app key?

FOLLOW-UP ANSWERS: 1. Minutes: revoke/rotate at the provider (invalidates the leak — FIRST action, always), then purge history + force-push, then audit the exposure window (provider-side logs for misuse by that key), then postmortem the pipeline (gitleaks pre-commit + CI scan should have caught it — the fix is the scan, not the vigilance). The order matters: rotation first because git history is forever; everything else is containment of an already-invalidated credential. 2. Envelope encryption solves the rotation + scale problem: with one app key, rotation re-encrypts EVERY row under a new key (a migration with a window of double-failure risk); with per-row data keys wrapped by a master, rotating the MASTER re-wraps only the small data keys (fast, incremental), and key-access policies can differ per key class. It's the standard pattern (AWS KMS docs-tier) because the failure and rotation modes actually work at scale.

RED FLAGS TO AVOID: - Secrets in config files “temporarily” (the most common real leak) - No census/inventory (you can't rotate what you can't list) - One all-powerful provider key shared across features

Q9-10: How did you implement retry logic for external API calls

DIFFICULTY: Medium **CATEGORY TAG:** Third-Party Integration

ANSWER: Retry is a POLICY per dependency, not a reflex — the design starts with the question “which failures are worth repeating, and what does a repeat cost?” Classification first: **transient** (429, 502/503/504, timeouts) → retryable; **permanent** (400 validation, 401/403 auth, 404) → never (retrying a 401 is noise; retrying a 400 repeats the same bug); **unknown** (connection-reset mid-response) → treat as retryable WITH idempotency (the false-negative case — Q2-15's duplicate-email war story is exactly this class). Mechanics: Resilience4j Retry on the provider adapters — max 3 attempts, exponential backoff with jitter (100ms → 400ms → 1.6s + random), retry-on predicates per provider (status-based + exception-based), and the retry budget is bounded by the CALLER's context: a user-facing request retries at most once (latency budget), a background job retries the full ladder (no user waiting). Idempotency keys ride every retryable side-effecting call (Stripe idempotency key, our message-key header for SMTP-like APIs) — so a retry that lands twice is absorbed by the PROVIDER, not just our DB: the key makes double-execution harmless at the source, which is the only fully correct place.

The compounding protections around retry: **circuit breaker** opens when the retry ladder is obviously futile (provider down — retrying into a breaker wastes nothing, Q3-12); **timeout discipline** — client timeouts set BELOW the user-facing budget with margin (a 30s timeout inside a 10s API budget is a self-DoS); **backpressure at the queue** — background retries consume from the job table with per-provider concurrency caps (a provider hiccup doesn't pull our whole worker pool into one provider's retry storm — bulkheads); and **jitter everywhere** because synchronized retries from 3 instances are a thundering herd (the retry-storm lecture). Verification: each adapter's retry policy has fault-injection tests (WireMock returning the failure ladder — asserting attempt counts, backoff timing, idempotency-key presence), and production metrics track retry rates per provider — a rising

retry rate is the early-warning signal that the PROVIDER is degrading, often before their status page admits it.

KEY TERMS TO MENTION: - Failure classification (transient/permanent/unknown) BEFORE retry policy - Exponential backoff + jitter; caller-context-bounded attempts (interactive vs batch) - Idempotency keys as the provider-side dedupe (the only full fix for false-negatives) - Breaker + bulkheads around retries (storm prevention) - Fault-injection tests + retry-rate metrics per provider

FOLLOW-UP QUESTIONS: 1. Why is the idempotency key “the only full fix” for timeout-unknowns? 2. Retries are hammering a struggling provider — when do retries become the outage?

FOLLOW-UP ANSWERS: 1. A timeout is epistemically ambiguous: the request MAY have executed. Without a key, your options are skip (possible under-execution — lost email) or retry (possible double-execution). The key converts the ambiguity into a non-problem: execute-at-most-once semantics on the provider side regardless of how many attempts you make. Any retry design without keys is implicitly betting that false-negatives won’t happen — at scale, they happen weekly. 2. At the retry-amplification point: $N \text{ instances} \times 3 \text{ attempts} \times \text{backoff-sync} \approx 10\text{-}20\times$ the original load on an already-struggling provider — retries CAUSE the extended outage. The defenses are the breaker (cuts load to zero when futility is detected), jitter (desynchronizes), and the provider-level concurrency cap (a ceiling on in-flight regardless of queue depth). The rule I state: your retry policy is part of the provider’s load profile — design it like you’re their largest customer, because you might be.

RED FLAGS TO AVOID: - Retrying 4xx / infinite retries - No idempotency keys on side-effecting calls - Synchronized backoff (no jitter) across instances

CATEGORY 10 — MICROSERVICES VS MONOLITH QUESTIONS

Q10-1: Why did you choose monolithic architecture over microservices

DIFFICULTY: Medium **CATEGORY TAG:** Microservices vs Monolith

ANSWER: We chose a modular monolith over microservices for three reasons, all justified at our scale (3,200 tenants, ~50K users):

1. **Operational cost** — Microservices add: a service mesh, distributed tracing setup, schema registry, service discovery, network latency between services, distributed transaction complexity, deployment pipelines per service, on-call rotation per service. At 2 backend engineers, we can't operate more than 1 service. The cost is fixed; the benefit scales with size — at our size, the cost > benefit.
2. **Team size** — Microservices are about [team](#) boundaries (Conway's Law), not technical boundaries. Each microservice should be owned by a team of 3-8 (two-pizza team). We have 2 backend engineers — a single team. Splitting our code into 5 microservices doesn't help us; it just adds operational overhead.
3. **Scale** — Our peak is 1,200 RPS. A single Spring Boot instance on ECS Fargate handles 2K RPS comfortably. We don't need horizontal scaling [per service](#) — we scale the whole monolith. Microservices shine when individual services need different scaling characteristics (e.g. AI service needs 100x compute of auth service). We don't have that yet — the AI gateway is the most CPU-intensive path, but we can extract it as a separate service when traffic justifies.

What we did instead — modular monolith: - Bounded contexts as Java packages (`com.acquisitionos.leads`, `.outreach`, `.billing`, etc.) - Cross-context communication via Spring `ApplicationEventPublisher` (in-process events) + Kafka for "future-extraction-ready" events - ArchUnit tests enforce no cross-context imports (except through published interfaces) - Each context has its own controller, service, repository, entity, event subpackages - A `common` module for shared kernel (base entities, exceptions, AOP aspects)

The path to microservices (when we'd extract): - The first extraction candidate is the AI gateway (`com.acquisitionos.ai`) — it has the highest CPU cost (LLM calls), a clean contract (prompt in, response out), and benefits from independent scaling (LLM traffic is bursty; the API tier is steady). - The second candidate is the outreach delivery (`com.acquisitionos.outreach.delivery`) — it's IO-heavy (Gmail, Twilio, Telegram sends) and benefits from a different threading model (virtual threads or WebFlux). - We've designed the seams: every cross-context call is via an event or an explicit interface (not a concrete class). Extracting a service = implementing the interface as a REST client + deploying the implementation as a separate Spring Boot app.

Trade-offs we accepted: 1. **Deployment coupling** — one deploy ships everything. Mitigated by feature flags + blue/green + canary. 2. **Scaling coupling** — can't scale `auth` without scaling `ai`. Mitigated by extracting `ai` first. 3. **Single point of failure** — a bug in `auth` can bring down `billing`. Mitigated by thorough testing + bounded contexts (a bug in `auth` shouldn't affect `billing` — different code paths). 4. **Codebase size** — 1 monolith at 200K LOC. Mitigated by bounded contexts + ArchUnit + good search.

When microservices would have been the right choice: - If we had 10+ engineers split into 3+ teams (team boundaries map to services) - If individual services had very different scaling profiles (e.g. AI service needs 100x auth service) - If we had polyglot needs (e.g. ML service in Python, auth in Java) - If we needed independent release cadences (e.g. AI service ships daily, billing ships monthly)

KEY TERMS TO MENTION: - Modular monolith: bounded contexts as packages, Spring Events + Kafka for cross-context - ArchUnit enforces context boundaries - Common module for shared kernel - Why not microservices: operational cost, team size (2 BE), scale (1.2K RPS, single instance handles 2K) - First extraction candidate: AI gateway (highest CPU, clean contract, bursty traffic) - Second: outreach delivery (IO-heavy, virtual threads/WebFlux candidate) - Trade-offs: deployment coupling, scaling coupling, SPOF, codebase size - Mitigations: feature flags + blue/green + canary + bounded contexts + ArchUnit - When microservices would be right: 10+ engineers / 3+ teams, different scaling profiles, polyglot, independent release cadences

FOLLOW-UP QUESTIONS: 1. How would you split the monolith into microservices? 2. What's the cost of extracting one service? 3. What's the role of Spring `ApplicationEventPublisher` vs Kafka?

FOLLOW-UP ANSWERS: 1. Six steps: (a) identify the bounded context to extract (e.g. `ai`); (b) define the contract — the events it publishes + the synchronous API it exposes; (c) dual-publish — the in-process event stays, but we also publish to a Kafka topic; (d) extract the consumer — a new Spring Boot service consumes the Kafka topic; (e) once the new service is stable, switch the in-process caller to call the new service via REST (or keep Kafka); (f) remove the in-process code from the monolith. Each step is reversible (we can roll back). 2. Three costs: (a) operational — one more service to deploy, monitor, on-call; (b) network — every call is now an HTTP round-trip (was in-process); (c) data — the new service needs its data, either via API calls (chatty) or via DB sharing (couples the services). We'd give the AI service its own DB (`ai_prompts` , `ai_usage`) and pass the lead context via the event payload (not via DB lookup). Estimated effort: 2 months for a clean extraction, including migration + canary + rollback plan. 3. `ApplicationEventPublisher` is for in-process synchronous (or `@TransactionalEventListener` async-after-commit) events — fast (no network), no broker, no consumer lag. Kafka is for cross-service events — durable, replayable, multi-consumer. We use both: Spring Events for everything that doesn't need to leave the monolith (most of our events); Kafka for the few events that will eventually cross a service boundary (`audit-events` , `outreach-events` , `reply-events` — we dual-publish these today, ready for extraction).

RED FLAGS TO AVOID: - “We use microservices” for a monolith - “We will use microservices one day” without a concrete plan - “Microservices scale better” (the monolith

scales fine at our size) - “Microservices are more decoupled” (a modular monolith with ArchUnit is equally decoupled)

(The remaining monolith vs microservices questions — trade-offs of monolith, migration to microservices, layered architecture structure, separation of concerns, cross-cutting concerns via AOP, application startup + initialisation, adding a new feature to the architecture, Strangler Fig pattern, service mesh (Istio/Linkerd), API gateway pattern, event-driven architecture, CQRS, sagas — follow the same template. Drill on: how AOP handles cross-cutting concerns (logging, security, audit) without polluting business code, why `@ComponentScan` is set to `com.acquisitionos` not `com.acquisitionos.app`, how to handle startup ordering when beans depend on each other.)

Q10-2: What are the trade-offs of your monolithic approach — name the real costs you paid

DIFFICULTY: Medium **CATEGORY TAG:** Architecture

ANSWER: I name the costs first because the benefits are well-known — the senior signal is knowing what the monolith actually charged us. **Cost 1 — shared failure domain:** one bad deploy takes everything down; an AI-provider deadlock in the outreach thread pool degraded the LEADS list (thread exhaustion is global inside one JVM) — we mitigated with executor isolation (bulkheads) but the JVM is still one blast radius. **Cost 2 — scaling granularity:** the scraping module is CPU-heavy, the API is I/O-heavy; the monolith scales them TOGETHER — we run 3 identical instances where 2 API + 1 worker would do; the waste is ~30% of compute, and the fix (extract workers) was deferred by design (the ADR records the trigger: infra cost > extraction cost — not yet true). **Cost 3 — team coupling at the code level:** 6 engineers in one repo = merge friction, shared-module ownership disputes, and the temptation for a feature to reach ACROSS bounded contexts (a meeting feature importing lead internals — the dependency-cruiser CI check exists because someone did it); the mitigation is architectural enforcement (context packages, ArchUnit tests), which is discipline cost, not free. **Cost 4 — technology lock-in:** one JVM — the ML scoring prototype in Python had to be re-expressed in Java (or become a side-service; we chose the side-service, which is the monolith admitting its first exception); polyglot freedom is a microservices purchase we didn’t make. **Cost 5 — deploy coupling cadence:** the weekly full-app deploy means a one-line notification fix ships with a leads-engine change — bigger test surface, bigger rollback, and release-train pressure; feature flags decouple ENABLEMENT but not CODE shipping.

What it bought, stated with the same honesty: single transactional boundary across the whole funnel (lead → deal → billing consistency is ONE `@Transactional`, no sagas), one deploy pipeline, one on-call surface, refactors across contexts in a single PR (we changed the credit-deduction signature across 5 modules in a day — in microservices that’s a quarter of coordinated releases), and debugging that’s a stack trace, not a trace-id hunt across 8 services. The net: at 50k users / 6 engineers / single-product focus, the trade was correct — and I can name the metric lines where it flips, which is the actual answer.

KEY TERMS TO MENTION: - Real costs: failure domain, scaling granularity (~30% compute waste), team coupling, tech lock-in, deploy cadence - Mitigations: executor bulkheads, ArchUnit/dependency boundaries, worker extraction plan - Benefits: single ACID boundary, cross-context refactors, one on-call - The flip-line metrics (cost > extraction, team > ~15 engineers, independent scaling need)

FOLLOW-UP QUESTIONS: 1. How do you keep a monolith from rotting into a big ball of mud? 2. Which cost surprised you most in practice?

FOLLOW-UP ANSWERS: 1. Structure enforced by tests, not documents: bounded-context packages with ArchUnit rules (leads cannot import meetings internals; only the events package crosses), a dependency-graph CI check (cruiser), API-style service interfaces between contexts even in-process, and ADRs for every cross-context change. The monolith rots exactly where governance is manual — so governance is automated. Same discipline microservices enforce with network boundaries; we do it with build-time rules. 2. Team coupling — technically everything was solvable, socially the shared code created passive bottlenecks (everyone waits for the one person who owns lead-scoring). The fix was organizational as much as technical: CODEOWNERS per context package, a rotation through context ownership, and the service-interface rule making cross-context requests explicit reviews. The lesson: architecture decisions are also staffing decisions.

RED FLAGS TO AVOID: - Listing only benefits (junior-flavored answer) - No quantified costs (the ~30% compute number, the thread-exhaustion incident) - No enforcement story (monolith discipline is build-time, or it's fiction)

Q10-3: How would you migrate this to microservices if needed — walk me through the actual plan

DIFFICULTY: Hard **CATEGORY TAG:** Architecture

ANSWER: Strangler-fig, driven by an extraction scorecard, never a big-bang rewrite. **Phase 0 — make the seams real** (mostly done): bounded contexts with enforced boundaries (ArchUnit), in-process events (`ApplicationEventPublisher`) whose contracts are already Kafka-shaped (id-payload, typed channels), and data ownership per context (context-owned tables, no cross-context FKs — UUID references only). The monolith's discipline IS the migration plan; if boundaries were fictional, I'd fix that FIRST — extracting a tangled module ships the tangle to production with network latency added. **Phase 1 — pick the first extraction by scorecard:** independent scaling need × change frequency × team ownership clarity × data-coupling LOW × blast-radius tolerance. Our scoring picks `ai-gateway` first (LLM calls: scaling profile is I/O-wait heavy and totally different from OLTP, provider churn is frequent, it already sits behind the AiPort interface) — NOT billing (money, transactional coupling, zero scaling pain) and NOT leads (the deepest data web). **Phase 2 — extraction mechanics:** the AiPort interface becomes HTTP/gRPC; the monolith keeps the interface, implementation moves — callers don't change (that's what the port bought us); data: the AI module owns only request/response logs + prompt templates (small, low-coupling — deliberately picked); transaction: no shared transactions existed across this

boundary (design-time win); deployment: the service ships with its own pipeline; resilience: timeouts/retries/breakers become REAL network concerns — Resilience4j configs graduate from defensive to mandatory, plus contract tests both directions. **Phase 3 — traffic cutover**: dual-run (monolith calls new service behind a flag, 5% → 50% → 100%, metrics compared: latency, error rate, cost per call), rollback = flag flip. **Phase 4 — event backbone**: as extractions accumulate, `ApplicationEventPublisher` impl swaps to Kafka publisher (same event classes → same JSON contracts); consumers move per-extraction, not big-bang.

The honest map of what stays hard: distributed data needs (a report joining leads + billing across services — the answer is the reporting data-warehouse fed by events, which we ALREADY have for analytics — the warehouse was built monolith-time specifically as the future cross-service query surface); distributed transactions (billing deduction from outreach service — the outbox pattern Q6-11 graduates from in-monolith convenience to inter-service necessity); and ops weight (8 dashboards, service mesh question, on-call rotation per service — the costs of Q10-2 arriving on schedule). The extraction scorecard keeps this honest: we extract when the SCORECARD says, not when the architecture fashion says.

KEY TERMS TO MENTION: - Strangler-fig + extraction scorecard (never big-bang) - Ports-à-seams: AiPort interface → HTTP without caller changes - Boundary discipline pre-conditions (ArchUnit, no cross-context FKs, event contracts) - Dual-run flag cutover with metric comparison + rollback - Event backbone swap (in-process → Kafka) per-extraction; warehouse for cross-service queries

FOLLOW-UP QUESTIONS: 1. Why is billing the WORST first extraction? 2. After extracting 3 services, what does the “monolith” actually become?

FOLLOW-UP ANSWERS: 1. Money paths are transactionally entangled with everything (credit deduction inside outreach transactions, subscription state gating features) — extraction means Saga choreography where ACID used to be, the highest-risk migration class, for zero scaling benefit (billing is small-row OLTP — it scales trivially inside the monolith). First extractions should build team confidence on low-coupling/high-benefit targets; billing is the last, if ever. 2. A domain core with satellite services: the monolith retains the transactional heart (leads, deals, billing — the ACID-coupled center) while stateless/scaling-mismatched concerns (AI, notifications, scraping workers, future ML) live as services. That end-state is a legitimate permanent architecture, not a failure to finish migrating — the goal was OPTION VALUE and fit-for-purpose placement, not a service count. I’d defend that end-state in front of any architect.

RED FLAGS TO AVOID: - “Extract everything” big-bang instinct (the classic resume-driven answer) - No seam pre-conditions (extracting a tangled module) - No cutover/rollback mechanics (flag-based dual-run or it’s hope)

Q10-4: What is your layered architecture structure and the rules between layers

DIFFICULTY: Easy **CATEGORY TAG:** Architecture

ANSWER: Four layers with one-way dependencies, enforced by ArchUnit in CI: **Controller** (`@RestController` — HTTP contract only: mapping, `@Valid`, status codes, DTO in/out — zero business logic, zero entities exposed; a controller is ~30 lines or it's wrong); **Service** (use-case orchestration: transactions, policy checks, event publication, mapping domain↔DTO — the layer with the unit-test weight; services call OTHER SERVICES only through interfaces and never across bounded contexts except via the events package); **Repository** (Spring Data JPA + Specifications — data access only, no business rules; query-shape decisions live here, business decisions don't); **Domain/entities** (JPA entities + value objects + domain exceptions — the model the other layers move). Cross-cutting layers sit beside, not inside: `@RestControllerAdvice` (errors), AOP aspects (audit, rate-limit), integration adapters (ports/impls), configuration.

The rules that make layering real rather than nominal: **dependencies point inward-down** (controller → service → repository → domain; nothing calls up; domain imports NOTHING from Spring Web — entities run in batch jobs without a web context, which we actually exercise); **DTO boundary at the edge** (entities never cross into controllers — MapStruct mapping in the service; this one rule kills the lazy-loading and serialization bug classes, Q5-20); **one use-case, one service method** (`@Transactional` at the service boundary; controllers never open transactions; helper services called WITHIN a use-case join its transaction by REQUIRED — the transaction topology is a designed artifact, reviewed in PRs); **ports at the perimeter** (all external systems behind interfaces in the integration package — the reason Q10-3's extraction is mechanical). The enforcement is the senior part: ArchUnit rules (`noClasses().that().resideInAPackage("..controller..").should().dependOnClassesThat().resideInAPacka` entities-never-in-controllers, no-field-injection) FAIL the build — layering by convention rots in month three; layering by CI check survives year two. I can also name the deliberate exceptions (a controller query DTO built directly by a read-side projection for a dashboard — documented, performance-motivated, ADR'd), because architecture without documented exceptions becomes dogma people route around secretly.

KEY TERMS TO MENTION: - Controller/Service/Repository/Domain + one-way dependency rule - ArchUnit CI enforcement (layering as tests, not convention) - DTO boundary at the edge (entities never reach controllers) - One use-case one `@Transactional` service method; REQUIRED joins - Ports at the perimeter + documented exceptions (ADR'd, not secret)

FOLLOW-UP QUESTIONS: 1. Why not skip the service layer for simple CRUD? 2. Where do mappers live and why MapStruct over manual?

FOLLOW-UP ANSWERS: 1. Tempting and wrong for us: "simple CRUD" gains business rules within a quarter (validation exceptions, events, permissions, credit checks) — retrofitting the service layer then is a bigger refactor than always having it. The cost we control: services stay thin for genuinely simple cases (delegate + map — 15 lines). What I WOULD concede: for a tiny admin tool, a two-layer (controller + repository) design is defensible; the choice scales with product ambition, and ours grew exactly as predicted. 2. Mappers live beside the service layer (`mapper` package, per-context); MapStruct because mapping is compile-time-checked (a renamed field breaks the build, not production), generated code is debuggable (unlike reflection magic), and performance is zero-overhead (plain getters — vs ModelMapper's runtime reflection cost we measured at ~15% on hot list endpoints). Manual

mapping is fine for ≤ 3 fields; above that, generated mapping with tests beats hand-rolled drift.

RED FLAGS TO AVOID: - Layers as folders with no dependency enforcement - Entities in API responses (“it’s faster” — it’s just deferred pain) - No answer for the transaction topology across layers

Q10-5: How did you ensure separation of concerns beyond the layer rules

DIFFICULTY: Medium **CATEGORY TAG:** Architecture

ANSWER: Layering separates TECHNICAL concerns; the harder separation is BUSINESS capability — bounded contexts. The system is divided into six contexts (`auth`, `leads`, `outreach`, `meetings`, `billing`, `ai`) each owning: its tables (no cross-context FKs — UUID refs only, Q6-2), its service interfaces, its domain events, and its exceptions. The enforcement stack, in layers of escalating strictness: package structure (contexts as top packages with public API sub-packages), ArchUnit (context A may not import context B’s INTERNALS — only the `api` package), event-mediated collaboration (cross-context reactions via `ApplicationEventPublisher` — `auth` doesn’t CALL `billing`, it publishes `SubscriptionChangedEvent` that `billing` listens to; the dependency is on the EVENT CONTRACT, not the module), and the dependency-graph CI report (cruiser visualization reviewed in architecture syncs — drift is visible in a picture, which reviewers actually read).

Concern separation inside a context follows the same principle at smaller scale: policy objects (`leadSecurity` — the `canEdit/canView` checks) separate AUTHORIZATION from business logic (services orchestrate, policy decides); query/projection separation (read-side DTO projections for dashboards vs write-side aggregates — CQRS-lite where the shape difference is real: the `leads LIST` reads a projection, `lead WRITES` go through the aggregate — no framework, just two repository paths); configuration separation (all knobs in `@ConfigurationProperties` classes per context — no magic values in services); and state-machine separation (lead stage, deal stage, message status are explicit state machines in one place each — transitions validated centrally instead of scattered if-chains: the bug class “who moved this to SENT illegally” died when transitions became a closed set).

The proof it works is the refactor record: extracting the AI module (Q10-3) touched 3 files in other contexts (the `AiPort` binding) — a context extraction that used to be archaeology became mechanical. And the honest cost, which I state before asked: boundary discipline has a tax — a feature spanning two contexts takes slightly longer than reaching across (design the event, the contract, the two-side implementation); I accept that tax because its opposite (cross-context reach) compounds into the big-ball-of-mud trajectory, and the compound interest is what kills systems, not the day-one tax.

KEY TERMS TO MENTION: - Bounded contexts (6) with owned tables/interfaces/events/exceptions - Event-mediated cross-context collaboration (contract dependency, not module import) - Policy objects, CQRS-lite projections, explicit state machines inside contexts -

Dependency-graph visualization in CI + architecture syncs - The boundary tax, stated and accepted (vs compound interest of coupling)

FOLLOW-UP QUESTIONS: 1. Give an example where an event contract SAVED you. 2. When is separation of concerns over-engineering?

FOLLOW-UP ANSWERS: 1. Adding the hot-lead Telegram alerts: `LeadBecameHotEvent` already existed for analytics — the Telegram listener was a NEW class subscribing to an EXISTING contract; zero changes to lead logic, shipped in a day. The counterfactual without the event: the lead-scoring service would have grown a `telegramSender` dependency, then a notification-config dependency, then... The event was designed for the SECOND listener, and it arrived. 2. When the axis of change is one: a context with one developer, one consumer, one stable shape doesn't need event ceremony — a direct call is clearer. Separation pays per INDEPENDENT axis of change (different teams, different scaling, different release cadences). Our early code had exactly this over-separation (an interface for a one-impl utility — deleted); the discipline is paying separation-debt only where change axes exist, and the ADR records each judgment.

RED FLAGS TO AVOID: - "Separation" meaning only folders (no contracts, no enforcement) - Event-mediated everything (direct calls where sync semantics are required) - No cost acknowledgment — dogma is also a red flag

Q10-6: How did you handle cross-cutting concerns (logging, security, audit, metrics)

DIFFICULTY: Medium **CATEGORY TAG:** Architecture / Spring AOP

ANSWER: Cross-cutting concerns are the canonical AOP use case, and our rule is: if a concern must apply UNIFORMLY by policy, it's an aspect or a filter — never call-site code. The inventory: **security** — Spring Security's filter chain (authentication/authorization at the boundary) + method security (`@PreAuthorize`) + policy beans (business-conditional rules); **logging/tracing** — correlation-id filter (MDC) + TaskDecorator propagation + the redaction converter; **audit** — `AuditAspect` (`@Around` on `@Audited`): captures actor/action/args/duration into the audit trail — one aspect, 200 annotations, zero scattered audit calls in services; **metrics** — Micrometer auto-instrumentation (controllers, executors, pools) + `@Timed` / `@Counted` where custom dimensions matter; **rate limiting** — filter (Redis-backed) before controllers + method-level annotation for fine-grained budgets; **idempotency** — an aspect on `@Idempotent(key = "...")` routes: checks the idempotency store before proceeding (Stripe-style request dedup at the application layer); **transaction boundaries** — `@Transactional` itself (the framework's AOP, which I count in the same mental bucket).

The design rules that keep aspects healthy: aspects declare ORDER explicitly (`@Order` — security filter → rate-limit → transaction → audit → metrics; ordering bugs are the subtlest AOP failures — an audit aspect inside a rolled-back transaction records what didn't happen); aspects are TESTED as units with the proxy (an aspect test suite using AspectJ proxy fabric — most teams test aspects never and discover order bugs in production); pointcuts are annotation-driven, not package-wildcard (explicit opt-in per method — `@Audited` , `@Timed` —

because wildcard pointcuts silently capture new code, and “silently applies to code that didn’t ask” is the AOP failure mode that makes people hate it); and every aspect has a kill-switch config (`@ConditionalOnProperty`) — when an aspect misbehaves in production (the audit aspect’s queue backed up once), the flag turns it off while the fix ships, versus a hotfix deploy. The alternative to aspects — base classes, manual calls, decorator wrapping — was rejected with the same reasoning each time: policy-uniform concerns leak when left to call-site discipline, and leaky discipline is how audit gaps happen (the exact gap our CI coverage test exists to prevent).

KEY TERMS TO MENTION: - Aspect/filter inventory: security, correlation, audit, metrics, rate-limit, idempotency - Explicit `@Order` (ordering bugs as the subtle AOP failure) - Annotation-driven pointcuts (opt-in, no wildcard capture) - Aspect unit tests + kill-switch configs - “Policy-uniform → aspect” as the deciding rule

FOLLOW-UP QUESTIONS: 1. What AOP problems have you actually hit in production? 2. Spring AOP proxy limitations — where do they bite?

FOLLOW-UP ANSWERS: 1. Two: the audit-aspect queue backup (async unbounded queue under a log-storm — now bounded + drop-metric + kill-switch) and an ORDER bug where audit ran before the transaction committed, recording actions a concurrent reader couldn’t see (flaky “ghost entries” reports) — fixed with AFTER_COMMIT semantics for the publish-path and order pinned in tests. Both are exactly the failures the aspect-testing discipline now catches pre-prod. 2. Self-invocation (the big one — `this.method()` bypasses the proxy for `@Transactional/@Async/@Audited` alike; our ArchUnit rule bans self-invocation of annotated methods), private/final methods (not proxied), and same-class field access skipping getters. These are why our style rules exist: annotated methods live on beans, called via interfaces, public — the limitations are known, encoded as static checks, not discovered as production mysteries.

RED FLAGS TO AVOID: - Cross-cutting logic copy-pasted or base-classed instead of aspects/filters - No aspect ordering awareness - Wildcard pointcuts capturing uninvolved code

Q10-7: How did you manage application startup and initialization

DIFFICULTY: Easy **CATEGORY TAG:** Architecture / Spring Boot

ANSWER: Startup is a designed sequence with fail-fast gates, not “Spring magic happens”. The order as the app boots: **(1) Configuration validation** — `@ConfigurationProperties` `@Validated` beans bind + validate; a custom `EnvironmentPostProcessor` checks profile-required env vars (prod missing `STRIPE_WEBHOOK_SECRET` = boot failure with a NAMED error, not a runtime mystery three days later); **(2) DataSource + Flyway** — Hikari pool initializes, Flyway migrates to head (schema drift fails via `hbm2ddl.auto=validate`); **(3) Redis + connectivity probes** — the health indicators run eagerly once (a down Redis at boot is WARN + degraded mode, not crash — the fail-open tiers keep serving); **(4) Cache warm-up** — the reference-data caches (credit costs, plan features, pipeline stages) pre-load synchronously (a few hundred ms; first-

request latency spike avoided — a warm-cache guarantee matters more than fast boot here); **(5) Scheduled/job registration** — schedulers register but jobs acquire their distributed locks before first run (a booting instance doesn't double-fire the sweep the running instance just did); **(6) Readiness flip** — the readiness indicator goes UP only after all gates pass; the ALB routes traffic only then (rolling deploys depend on this honesty — an instance that says ready before caches are warm serves 300ms first-hits and poisons p95).

The code surfaces for boot logic, in preference order: `@ConfigurationProperties` binding (data, validated), `@Bean` factory methods (wiring with dependencies visible in signatures), `ApplicationRunner` / `CommandLineRunner` for sequenced boot tasks (cache warm-up, the denylist re-derivation sweep on boot), and `@PostConstruct` ONLY for intra-bean invariant checks (it runs before the bean's dependencies are necessarily complete — a classic boot-bug source; our style rule limits it). Two production scars that shaped this: an env var typo that surfaced as a `NullPointerException` in a scheduled job at 2 AM (hence the `EnvironmentPostProcessor` gate — config errors must die at boot WITH the variable name in the message), and a cache warm-up moved to async “to speed deploys” that created cold-cache p95 spikes every deploy (reverted — boot takes 4 more seconds and p95 stays flat; the trade is written in the ADR).

KEY TERMS TO MENTION: - Fail-fast sequence: config validation → Flyway → probes → warm-up → schedulers → readiness - `EnvironmentPostProcessor` named-error config gates - Eager cache warm-up (boot +4s vs cold p95 spikes — the ADR trade) - `ApplicationRunner` sequencing vs `@PostConstruct` limitation - Readiness flips only after all gates (rolling-deploy honesty)

FOLLOW-UP QUESTIONS: 1. How do you debug a boot failure fast? 2. Why is Redis-down at boot a WARN not a failure?

FOLLOW-UP ANSWERS: 1. Boot failures print a structured banner in dev (the failure analyzer from Spring Boot is kept on: named bean, named property, named migration) — the goal is “read the first error, skip the 40-line stack”. CI runs boot in every PR (a context-loads test with the real profile matrix), so boot breaks are caught before the deploy, where they'd page someone. The 2 AM NPE class of error is extinct by design. 2. Degradation design (Q7-8): Redis is an accelerator for cache tiers and a fail-closed helper for security tiers — the app is DESIGNED to run degraded, so a boot-time Redis outage should produce a degraded-but-serving instance (alerts fire, on-call fixes Redis, cache re-heats via single-flight). Hard-failing boot on an accelerator turns a Redis blip into a full outage — the availability arithmetic made the decision, not preference.

RED FLAGS TO AVOID: - `@PostConstruct`-heavy boot logic with ordering assumptions - Silent config errors surfacing at runtime - Readiness declared before caches/init complete

Q10-8: TWISTED — Walk me through adding a brand-new feature (SMS channel) end-to-end in this architecture

DIFFICULTY: Hard **CATEGORY TAG:** Architecture / Design Exercise

ANSWER: The feature: SMS as a new outreach/notification channel. **Day 0 — contract design:** `SmsPort` interface in the integration package (`send(SmsMessage): SmsResult`) — the port exists BEFORE the provider decision (the architecture's whole point: channel count grows, port count doesn't); per-channel cost added to `credit_cost` (SMS = 6 credits — costs are data, Q2-13); `notification_channel` gains a `channel_type=SMS` value (enum + migration); the channel router registers the new sender. **Provider phase:** Twilio (already a vendor — Q9-7) — adapter implements `SmsPort` with the standard resilience stack (timeout 10s, retry 2x on 429/5xx with jitter, breaker, idempotency key in our message table); API keys via secrets manager; the adapter is ~150 lines because the pipeline (job table, dedupe, audit, metrics) is inherited infrastructure, not per-channel work. **Domain phase:** opt-in/compliance surface — SMS has legal weight (TCP-style consent rules): `consent` table (per-lead SMS consent flag + source + timestamp), the composer REFUSES SMS without consent (a domain rule enforced in `OutreachFacade`, tested as a first-class case), opt-out keyword handling ("STOP" via the inbound webhook → suppression list — same machinery as email unsubscribes); templates registered per channel in the composer (SMS: 160-char discipline, link shortening with tracking).

Integration phase: inbound webhook `/api/webhooks/twilio/sms` through the standard pipeline (Q9-8: raw capture → signature validation → persist → dedupe by MessageSid → handler); replies feed reply-intelligence (the classifier gains an SMS channel — it's channel-agnostic by design, consuming normalized message text); **Surface phase:** frontend gets the channel toggle (feature-flagged rollout: internal orgs → 5% → GA), usage dashboards gain the SMS family (send rate, cost, block/opt-out rates — every channel ships with its health metrics, the WhatsApp block-rate lesson pre-applied), and the audit aspect annotates the new service methods. **Total effort: ~2.5 weeks**, of which maybe 15% touched EXISTING code (enum value, router registration, composer branch, feature flag) — the diff is overwhelmingly NEW files in the integration package + config data. That ratio is the whole architectural argument in one number: the layered + port-based + event-mediated structure makes channel #4 a branch, not a surgery. And the honest caveats: consent/compliance was 40% of the real effort (architecture doesn't absorb LAW), and provider-choice lock-in is confined to the adapter (~150 lines to swap — the port's rent is paid at swap time, not build time).

KEY TERMS TO MENTION: - Port-first design (`SmsPort` before provider choice); router registration - Inherited infrastructure: job table, dedupe, audit, metrics, breaker stack - Compliance as a domain rule (consent gate in the facade, STOP handling) - Feature-flagged rollout + channel health dashboards from day one - The diff ratio (~85% new files / ~15% touched) as the architecture metric

FOLLOW-UP QUESTIONS: 1. What would have made this feature HARD in your architecture? 2. Where did you consciously NOT generalize?

FOLLOW-UP ANSWERS: 1. If channels had been if-branches in the outreach service (channel == 'email' ? ... : ...): every touchpoint (send, dedupe, tracking, compliance, metrics) would need editing, the diff would be invasive, and the regression surface the whole outreach pipeline. The port pattern's rent was paid in week one of the original design — this feature is the dividend. Honest answer includes the counterfactual: architectures prove

themselves on feature #4, not feature #1. 2. Two spots: no abstract “MessagingPort” mega-interface (SMS/email/WhatsApp have genuinely different semantics — three ports beat one leaky abstraction with optional methods); and no per-channel database schemas (one `message` table with SINGLE_TABLE inheritance, Q5-17 — subtype columns stayed few). Generalization was applied where variation PROVED recurring (channels multiply), withheld where it was speculative. That restraint is why the clean points stayed clean.

RED FLAGS TO AVOID: - Feature walkthrough with no compliance/legal dimension (SMS without consent = lawsuit architecture) - Invasive diff (“modified 30 files”) — that’s the architecture failing the test - No metrics/rollout story (features ship with observability or not at all)

CATEGORY 11 — DEVOPS AND DEPLOYMENT QUESTIONS

Q11-1: How did you deploy AcquisitionOS to production

DIFFICULTY: Medium **CATEGORY TAG:** DevOps

ANSWER: AWS-based: Docker → ECR → ECS Fargate (4 tasks) → ALB → CloudFront (for the SPA). CI/CD via GitHub Actions. Blue/green deploy with rollback on 5xx spike.

Docker image:

```
# Multi-stage build for cache reuse
FROM eclipse-temurin:17-jdk-jammy AS build
WORKDIR /app
COPY pom.xml ./
COPY common/pom.xml common/
COPY domain/pom.xml domain/
COPY app/pom.xml app/
RUN mvn dependency:go-offline -B
COPY . .
RUN mvn package -DskipTests

# Spring Boot layered jar (separates dependencies from application)
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
COPY --from=build /app/app/target/*.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract
COPY --from=build /app/app/target/layers/dependencies/ ./
COPY --from=build /app/app/target/layers/spring-boot-loader/ ./
COPY --from=build /app/app/target/layers/snapshot-dependencies/ ./
COPY --from=build /app/app/target/layers/application/ ./

ENTRYPOINT ["java", "org.springframework.boot.loader.JarLauncher"]
```

Layered jar: Spring Boot's `layertools` separates the jar into 4 layers (`dependencies`, `snapshot-dependencies`, `spring-boot-loader`, `application`). Docker caches each layer separately — a code change only rebuilds the `application` layer (the first 3 layers are dependencies that don't change often). This makes Docker builds fast (1-2 min for code changes vs. 5 min for full rebuild) and Docker image pulls fast (only the changed layer is pulled on deploy).

CI/CD pipeline (GitHub Actions):

```

name: Build + Deploy
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: 17, distribution: temurin, cache: maven }
      - run: mvn -B clean verify # unit + integration tests (Testcontainers)
      - uses: SonarSource/sonarcloud-github-action@v2
      - run: docker build -t acquisitionos:${{ github.sha }} .
      - run: aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
      - run: docker push $ECR_URI/acquisitionos:${{ github.sha }}

  deploy-staging:
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: aws ecs update-service --cluster acquisitionos-staging --service acquisitionos --task-definition acquisitionos:${{ github.sha }}
      - run: ./scripts/wait-for-deploy.sh staging # poll ECS until tasks running
      - run: ./scripts/smoke-test.sh staging # hit /api/health, /api/auth/config

  deploy-prod:
    needs: deploy-staging
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    environment: production # requires manual approval
    runs-on: ubuntu-latest
    steps:
      - run: aws ecs update-service --cluster acquisitionos-prod --service acquisitionos --task-definition acquisitionos:${{ github.sha }}
      - run: ./scripts/blue-green.sh prod # blue/green deploy + canary

```

Blue/green deploy: - New tasks come up alongside old (we configure ECS for `minimumHealthyPercent=100` + `maximumPercent=200` — new tasks start before old ones drain) - ALB health check passes → new tasks receive traffic - We watch 5xx rate + p95 latency for 15 minutes - If 5xx > 0.1% or p95 > 500ms → auto-rollback (drain new tasks, old tasks continue) - If healthy → drain old tasks (SIGTERM, app graceful shutdown via Spring Boot's `server.shutdown=graceful` + `spring.lifecycle.timeout-per-shutdown-phase=30s`)

Graceful shutdown: Spring Boot 3.2 supports graceful shutdown — `server.shutdown=graceful` + `spring.lifecycle.timeout-per-shutdown-phase=30s` means on SIGTERM, the app stops accepting new requests, finishes in-flight requests (up to 30s), then exits. This prevents dropping in-flight requests during a deploy.

Health checks: - ALB health check: `GET /actuator/health/liveness` every 10s; 3 failures = unhealthy → drain - Readiness: `GET /actuator/health/readiness` — Spring Boot's readiness

state (up when all beans are initialised, down during startup or shutdown) - Custom health indicators: `RedisHealthIndicator` (ping Redis), `DatabaseHealthIndicator` (SELECT 1), `KafkaHealthIndicator` (check consumer lag)

AWS infrastructure (Terraform): - ECS cluster (Fargate) with 4 tasks, 2 vCPU/4GiB each - ALB with TLS (ACM cert), WAF (rate-based rules) - RDS PostgreSQL 15 Multi-AZ (db.r6g.large) + 2 read replicas - ElastiCache Redis 7 cluster (3 shards, 1 replica each) - S3 (attachments + invoices) + CloudFront (SPA + signed URL CDN) - Secrets Manager (Stripe, Google, JWT, SMTP secrets) - CloudWatch Logs + Metrics + Alarms - Route 53 (DNS) + ACM (certs)

Monitoring + alerting: - Prometheus + Grafana (custom dashboards: app, JVM, Hikari, Kafka consumer lag, Redis) - CloudWatch Logs (raw ECS stdout) + Loki (structured JSON logs) - Sentry (errors + breadcrumbs) - OpenTelemetry + Tempo (traces) - PagerDuty for alerts (5xx rate, p95 latency, DB CPU > 80%, Redis hit ratio < 80%, Kafka consumer lag > 10K, ECS task restarts)

Secrets: AWS Secrets Manager — Stripe webhook secret, JWT secret, Google OAuth client secret, SMTP password, GMAIL_ENCRYPTION_KEY. Spring Cloud AWS loads these as env vars at task start via `AwsSecretsManagerPropertySource`. Rotated quarterly via Terraform + manual dual-secret window.

Backup: RDS automated backups (daily snapshot + 5-min point-in-time recovery, 30-day retention). S3 cross-region replication for attachments. We test restoration quarterly (restore to staging, verify app works).

Disaster recovery: RPO 5 minutes (RDS PITR), RTO 30 minutes (Terraform applies a fresh stack in a new region in ~15 min, RDS snapshot restore in ~10 min). We've never had to invoke DR in 9 months.

KEY TERMS TO MENTION: - Multi-stage Docker build + Spring Boot layered jar (4 layers, Docker cache reuse) - Eclipse Temurin JRE 17 jammy base - GitHub Actions: build + SonarQube + Docker build + ECR push + ECS deploy + smoke test - ECS Fargate, 4 tasks, 2 vCPU/4GiB each, ALB - Blue/green deploy (minimumHealthyPercent=100, maximumPercent=200), watch 5xx + p95, auto-rollback on threshold - Graceful shutdown (`server.shutdown=graceful` + `spring.lifecycle.timeout-per-shutdown-phase=30s`) - Spring Boot Actuator `/actuator/health/liveness` + `/readiness` + custom indicators (Redis, DB, Kafka) - Terraform: ECS + ALB + RDS Multi-AZ + ElastiCache cluster + S3 + CloudFront + Secrets Manager + CloudWatch + Route 53 - Monitoring: Prometheus + Grafana + Loki + Sentry + OpenTelemetry + PagerDuty - RPO 5 min, RTO 30 min, quarterly DR test

FOLLOW-UP QUESTIONS: 1. How do you do zero-downtime deploys with database migrations? 2. What's your rollback strategy? 3. How do you handle a bad deploy that's live for 5 minutes?

FOLLOW-UP ANSWERS: 1. Three rules: (a) **backward-compatible migrations only** — add column (with default), add table, add index `CONCURRENTLY`, never drop column in the same deploy as the code that needs it; (b) **expand → migrate → contract** — deploy new code (writes to old + new column), backfill old rows to new column, deploy code that reads from new column, deploy code that stops writing to old, drop old column; (c) **Flyway with `--no-transaction` for `CREATE INDEX CONCURRENTLY`** — Postgres can't run that in a transaction.

We've never had a deploy that needed to drop a column — we use additive changes exclusively.

2. Two layers: (a) **ECS rollback** — `aws ecs update-service --task-definition <previous-version>` reverts the running tasks to the previous image (takes ~2 min for the new tasks to come up + drain the bad ones); (b) **DB rollback** — we don't rollback DB migrations (forward-only). If a migration breaks the app, we write a new migration that fixes it (e.g. re-adds a dropped column — but we never drop in the first place). The ECS rollback is the primary defense.

3. Three steps: (a) **detect** — PagerDuty alerts on 5xx spike + p95 latency + Sentry error spike; on-call is paged within 1 min; (b) **rollback** — `aws ecs update-service --task-definition <previous>` (2 min); (c) **postmortem** — within 24h, written to the team wiki: timeline, root cause, fix, prevention (e.g. add a test for the case that broke). We've had 2 bad deploys in 9 months (one Hibernate config issue, one Stripe SDK version mismatch). Both rolled back in <10 min.

RED FLAGS TO AVOID: - No CI/CD (manual deploys are not a senior answer) - No blue/green or canary - No graceful shutdown (drops in-flight requests) - No backup/DR plan - "We deploy with `ssh + git pull`" (instant fail)

(The remaining DevOps questions — containerisation (Docker), CI/CD pipeline, AWS vs GCP, env-specific configs, health checks, zero-downtime deploys, monitoring + alerting, logging in prod (ELK), secrets management, Kubernetes vs ECS, Terraform, immutable infrastructure, canary deploys, infra as code — follow the same template. Drill on: what's in your Dockerfile layer cache, how you handle a stateful service (Redis) during deploy, how you'd deploy to EKS vs ECS, how you rotate TLS certs without downtime.)

Q11-2: How did you containerize the application (Docker for Spring Boot)

DIFFICULTY: Medium **CATEGORY TAG:** DevOps

ANSWER: Multi-stage Dockerfile with layered-jar extraction — the two things that separate a real production image from a tutorial. **Stage 1 (build):** `maven:3.9-eclipse-temurin-17` — `mvn -DskipTests package + jarmode=layertools` extraction (BOOT-INF/layers.idx splits dependencies/ snapshot-dependencies/ application/ into image layers). **Stage 2 (runtime):** `eclipse-temurin:17-jre-jammy` (JRE only — 60% smaller than JDK, smaller CVE surface), non-root user (uid 10001 — the image runs as nobody-writable filesystem; a container escape lands on a user with no home), and dependency layers copied FIRST so dependency-only rebuilds (90% of builds) push one thin layer — registry storage and pull times drop accordingly. JVM flags in the image: `-XX:MaxRAMPercentage=75` (container-memory-aware heap — the hard-coded `-Xmx2g` on a 3GB container is the OOMKill tutorial), `-XX:+UseG1GC`, `-XX:MaxGCPauseMillis=200`, and `JAVA_TOOL_OPTIONS` left open for env-level overrides.

Build/push mechanics: CI builds with the Spring Boot buildpack alternative NOT chosen (jib/ Dockerfile debate — we chose Dockerfile for transparency; jib's daemonless builds are genuinely nice, the ADR records both); images tagged with git SHA + semver (never `latest`

in deploys — the roll-back story needs immutable tags); SBOM generated (syft) + scanned (trivy) in CI — CRITICAL CVEs fail the build with a waiver path (documented, expiring); and `.dockerignore` (target/, .git, node_modules of the frontend) keeping context uploads fast. Runtime posture: `readOnlyRootFilesystem: true` with tmpfs mounts for /tmp (Hikari/Java need scratch space), healthchecks hitting the management port, `stopGracePeriod` tuned to the executor drain window (30s — Q3-9's graceful shutdown), and resource limits matching the JVM (memory limit = heap ÷ 0.75 + 512MB metaspace/stack headroom — the container JVM memory contract, which I can draw: heap + metaspace + code cache + threads + direct buffers ≤ limit, or the kernel kills you).

KEY TERMS TO MENTION: - Multi-stage + layertools layered jars (thin dependency-only pushes) - JRE-only, non-root, `readOnlyRootFilesystem` + tmpfs - `MaxRAMPercentage` (container memory contract: heap+non-heap ≤ limit) - SHA + semver tags (immutable, rollback-able), never latest - SBOM + trivy scan in CI with waiver discipline

FOLLOW-UP QUESTIONS: 1. Why did your container get OOMKilled with -Xmx set below the limit — real story? 2. Dockerfile vs Jib vs Buildpacks — defend the choice.

FOLLOW-UP ANSWERS: 1. It did: heap was 2g in a 2.5g container — heap fit, but metaspace (200m) + code cache (100m) + thread stacks (200 threads × 1m) + direct buffers (NIO/HTTPS) blew past. The kernel OOMKills on TOTAL process memory, not heap. Fix: `MaxRAMPercentage=75` + thread-pool sizing review (the async pools were over-provisioned) — the lesson is the JVM memory map, which I can sketch, and it's why "Xmx below limit" is necessary-not-sufficient. 2. Dockerfile won on: auditability (every layer reviewable in the PR), zero build-plugin magic, portability across CI systems. Jib would remove the Docker daemon dependency and speed builds ~20%; buildpacks give auto-base-image patching (real security value). If CI were slower or daemon management painful, jib; if platform-team-run, buildpacks. Choice recorded with revisit criteria — that's the ADR habit I'd bring to any team.

RED FLAGS TO AVOID: - Root user + latest tag + hardcoded Xmx (the trifecta of tutorial images) - No memory-contract awareness (heap ≠ container memory) - Single-stage builds shipping JDK + build tools to prod

Q11-3: What is your CI/CD pipeline (walk it stage by stage)

DIFFICULTY: Medium **CATEGORY TAG:** DevOps

ANSWER: GitHub Actions, two pipelines: PR pipeline (gate) and main pipeline (release). **PR pipeline** (~9 min, parallelized): compile + static analysis (SpotBugs, checkstyle, ArchUnit embedded in tests) → unit tests (parallel shards) → integration tests (Testcontainers: Postgres, Redis, WireMock) → OpenAPI diff + contract tests (oasdiff + consumer packs) → image build + trivy scan → a preview environment deployed per-PR (ephemeral namespace, seeded data, auto-URL in the PR — reviewers click, not checkout). **Main pipeline** (on merge): the same gates re-run (merge queue prevents drift between PR-validated code and merged head) → image push (SHA tag) → **staging deploy** → staging smoke suite (API contracts, payment sandbox flow, auth flow — Playwright + API-level) → manual approval gate (1 senior) → **production deploy** — rolling (K8s: readiness gates + `maxUnavailable=0`),

then **post-deploy verification**: the canary checks (synthetic transactions: login, lead-create, webhook replay) run for 10 min against prod; failure = auto-rollback (rollout undo) + page. Total main-pipeline ~25 min to prod, ~35 min to verified.

The design principles behind it: **fast feedback wins over completeness** — PR pipeline is tuned to 9 min (test sharding, Testcontainer reuse, gradle/mvn cache) because 30-minute PR pipelines train engineers to batch changes (the true enemy of quality); **the pipeline is code-reviewed like code** (workflow changes in PRs, no push-to-main edits — supply-chain hygiene: pinned action SHAs, minimal token scopes, no secrets in PR-fork contexts); **every deploy is rollback-ready** (migrations forward-only BUT expand-contract means the previous image runs on the new schema — the invariant that makes rollback a 30-second operation, Q6-6); **databases are the only stateful hazard** — the pipeline REUSES the deploy-time migration run (Flyway at boot) and gates deploys on a migration-dry-run stage (the risky-DDL checklist runs in staging first). Metrics about the pipeline itself (duration trend, flake rate per suite, deploy frequency, change-failure rate — DORA basics) are on the team dashboard: a flaky integration suite gets a maintenance ticket automatically at > 2% flake, because a flaky gate trains people to retry-past-failure, which kills the gate's meaning.

KEY TERMS TO MENTION: - PR gate (9 min, parallel, preview envs) vs main release pipeline (staging → approval → rolling prod → canary verify) - Merge queue + re-run gates (no drift) - Supply-chain hygiene: pinned SHAs, scoped tokens, workflow-as-code - Rollback-ready invariant (expand-contract + previous image on new schema) - DORA metrics + auto-ticketed flaky suites

FOLLOW-UP QUESTIONS: 1. Your canary failed 10 minutes after deploy — walk me through the next 10 minutes. 2. Why a manual approval gate if you have automated verification?

FOLLOW-UP ANSWERS: 1. Auto-rollback fires immediately (the rollout undoes; the flag-based features mean user-visible state is mostly unchanged), the migration question is checked FIRST (did this deploy migrate? if yes — is the previous image compatible? expand-contract says yes, verified by the pipeline's compatibility stage), canary traces get pulled (which synthetic failed and WHY), and the incident channel gets a timeline. Total: rollback at min 11, RCA draft by min 20. The deploy that failed becomes a PR with the canary evidence attached — the pipeline's job is to make failure cheap and legible, not rare by gate-theater. 2. The gate isn't a rubber stamp for "does it work" (automation answers that) — it's the human checkpoint for JUDGMENT calls automation can't make: is this the right WEEK to ship a billing change (business timing), is the on-call staffed for a risky-but-passing deploy, does this interact with the incident from yesterday. Fully-automated deploys are the end-state for low-risk paths (we auto-deploy frontend-only changes); the approval is scoped to risk class, documented, and shrinking as canary coverage grows — the trend line matters more than the gate.

RED FLAGS TO AVOID: - 30+ min PR pipelines with no sharding story - Deploys without automated post-deploy verification + rollback - Manual workflow edits on main (supply-chain sloppiness)

Q11-4: How did you deploy on AWS — the actual topology

DIFFICULTY: Medium **CATEGORY TAG:** DevOps / Cloud

ANSWER: The topology, tier by tier: **edge** — Route53 → CloudFront (static assets + frontend; the API goes direct to the ALB for websockets) → AWS WAF (volumetric rules + bot control) → ALB (TLS termination, path routing: `/api/*` → app target group, management port in an internal-only target group). **Compute** — ECS Fargate (3+ tasks, 2 vCPU/3GB, auto-scale on CPU 60% + request-count target tracking; Fargate over EKS chosen for ops-weight: 6 engineers don't run a control plane — the K8s migration trigger is in the ADR: multi-service orchestration needs if microservices extraction proceeds); tasks run in private subnets, egress via NAT gateway. **Data** — RDS PostgreSQL (Multi-AZ, db.m6g.xlarge, read replica for analytics, automated backups + PITR per Q6-9) in private subnets with security-group chaining (only the app SG can reach 5432 — no SG-wide opens); ElastiCache Redis (replication group, Multi-AZ) for cache/session tiers; S3 for object storage (uploads, backups with object-lock, archives) + lifecycle rules. **Secrets** — Secrets Manager + IAM roles (task roles grant least-privilege access per service — the app role reads its secret path and nothing else; instance profiles never hold DB creds). **Async** — SQS for the low-volume queues we didn't build Kafka for (email bounce events), EventBridge for cron-like triggers where ECS-scheduled tasks don't fit.

The networking discipline: three-subnet VPC design (public: ALB/CDN origins only; private-app: tasks; private-data: RDS/Redis — data tier has NO route to the internet, not even NAT), VPC endpoints for S3/Secrets (traffic never leaves the AWS backbone — latency + security), and flow logs → CloudWatch for audit. Cost-shaping (interviewers increasingly ask): Fargate right-sizing from CloudWatch p95 memory (we resized 4GB→3GB saving 25% with headroom data), NAT gateway was the surprise line item (VPC endpoints killed 40% of it), S3 lifecycle tiers archives to Glacier after 90 days, and the RDS replica exists for both performance AND the analytics isolation that lets the primary stay right-sized. Everything is Terraform (modules per tier, state in S3 + locking) — the `deploy/terraform` in the repo is real: an engineer can rebuild the environment from code, which is the actual disaster-recovery plan beneath the backup plan.

KEY TERMS TO MENTION: - Route53 → CloudFront/WAF → ALB → ECS Fargate (auto-scale) → private data tier - SG-chaining + three-subnet design + VPC endpoints (no internet route for data) - RDS Multi-AZ + replica; ElastiCache Multi-AZ; S3 lifecycle/object-lock - Task-role least privilege; Secrets Manager - Terraform-everything (rebuildable env = the DR layer under backups)

FOLLOW-UP QUESTIONS: 1. Why Fargate over EKS — and what flips the decision? 2. Your AWS bill doubled month-over-month — debug walk-through?

FOLLOW-UP ANSWERS: 1. Fargate: no control-plane ops, per-task pricing, simpler IAM — right for a monolith + few workers at our scale. EKS flips when: service count grows (the Q10-3 extraction path), we need service-mesh/advanced traffic policy, GPU/daemonset workloads, or multi-team namespace governance. The ADR holds both the current choice AND the trigger conditions — cloud decisions are pricing-and-team-shape decisions, revisited on evidence. 2. Cost explorer grouped by service → tag, then diff the delta: usual suspects in

order — data transfer (a new cross-AZ chatter: the analytics job moved to same-AZ reads or got a VPC endpoint), NAT processing (a new pod egressing to internet — S3 access without endpoint), logs (a DEBUG flag left on — the ELK volume graph shows it instantly), autoscaling thrash (scale-policy oscillation — target-tracking cooldown misconfigured). The discipline: costs are tagged by service + env from day one (Terraform defaults), so “what doubled” is a GROUP BY, not archaeology. We’ve run this play for real on the NAT line item.

RED FLAGS TO AVOID: - Public-subnet databases (“it has a security group” — no) - Untagged resources (cost and audit blindness) - Console-clicked infrastructure (no Terraform = no DR, no review)

Q11-5: How did you manage environment-specific configurations

DIFFICULTY: Easy **CATEGORY TAG:** DevOps / Spring Boot

ANSWER: The configuration matrix has three axes — environment (dev/staging/prod), sensitivity (public/internal/secret), and change-rate (immutable-per-deploy/runtime-tunable) — and each cell has a home. **Environment diffs** live in `application-{profile}.yaml` for non-secret infra shaping (pool sizes, log levels, provider endpoints — staging points at sandbox providers) with parity discipline (same image, different profile — Q3-17); **secrets** never touch git — Secrets Manager → sidecar → env (Q3-19), rotated per policy; **public config** (feature-flag defaults, API base URLs baked for the frontend) ships in the image; **runtime-tunable behavior** lives in the DB-backed feature-flag/admin-config service (changes without deploy, audited, rolled back in the UI — the “tune the rate limit at 3 AM” path). Drift control: the actuator `/env` endpoint (sanitized) is hashed into a config-fingerprint per instance — a Grafana panel alerts when instances of one service diverge (stuck rollouts, hand-edits), and a daily job diffs effective config against git (the “someone edited it live” detector).

Per-environment bootstrap is scripted: `make setup` (dev: Docker Compose Postgres/Redis, Flyway, seed data — Q1-15’s onboarding flow), staging deploys the same image with a profile + sandbox credentials, prod adds the approval gate. The subtle one I emphasize: **config-as-frontend-contract** — the React app gets its API base/feature exposure via a `/api/config` bootstrap endpoint (server-rendered from flags) rather than build-time env baking — one image serves all envs, and config changes reach the frontend without rebuilds; the build-time-bake pattern (`REACT_APP_*` at build) was rejected because it forks images per env and breaks the staging-parity invariant. Validation closes the loop: boot-time `EnvironmentPostProcessor` gates (named errors on missing vars per profile), the secret-source audit (a known-secret resolved from a non-secret source fails boot — Q3-19), and the config-change runbook (which changes are deploy-gated vs flag-flippable is documented per config key — the operations team knows the blast radius before touching anything).

KEY TERMS TO MENTION: - Three-axis config matrix (env × sensitivity × change-rate) with a home per cell - Profile yml for infra shaping; secrets via Secrets Manager; flags for runtime behavior - Config-fingerprint drift panel + daily git-diff job - `/api/config` bootstrap

over build-time env baking (one image, all envs) - Boot-time named-error validation + secret-source audit

FOLLOW-UP QUESTIONS: 1. A config change caused an incident — how does your system limit that class? 2. How do you onboard a new environment (a client's dedicated deployment)?

FOLLOW-UP ANSWERS: 1. Three layers: the change itself is flag-gated where possible (flags roll back in seconds, no deploy), the exposure is canary'd (flag rollouts step 5% → 50% with error-rate watch), and the drift-fingerprint panel catches out-of-band edits (the human-touching-prod-yml scenario) within a day. Post-incident, the config key gets a documented blast-radius note — the runbook learns per key. Config incidents are usually process incidents wearing a config costume. 2. The Terraform modules take a variable set (region, sizing tier, domain, provider keys) and emit the whole stack; the new env gets: its profile yml (diffs only — parity defaults), its Secrets Manager prefix, its sandbox/provider credentials, a smoke-suite run against it, and a parity check (the config-diff job compares effective config shape vs prod — missing keys fail, not silently default). Day-one cost ~1 day of engineering; that's the payoff of config discipline compounding.

RED FLAGS TO AVOID: - Environment logic in if-statements scattered in code (vs profiles/flags) - Build-time-baked frontend config (image forks per env) - No drift detection (hand-edits and stuck rollouts live forever)

Q11-6: How did you implement health checks (liveness/readiness/startup)

DIFFICULTY: Easy **CATEGORY TAG:** DevOps / Spring Boot

ANSWER: Three probe types, three different questions, three different behaviors — conflating them is the classic K8s outage. **Liveness** (`/actuator/health/liveness`): “is the JVM wedged?” — answers UP if the process responds; NO dependency checks (a Redis outage must not restart pods — restarts fix nothing and churn the fleet; the one exception: an OOM-deadlocked executor set trips a custom liveness guard — self-detected heap-lessness, where restart IS the fix). **Readiness** (`/readiness`): “can I serve traffic?” — aggregates DB (validation query), Redis ping, disk space (upload scratch), job-poller lag (custom), cache-warm flag (boot gate, Q10-7); false → ALB/K8s pulls the instance from rotation — dependency outage drains traffic WITHOUT restarting anything. **Startup** (`/startup`): guards the boot window — `failureThreshold × period` accommodates the warm-up sequence (Flyway on a big migration can take minutes; without a startup probe, the liveness probe would kill a HEALTHY-but-migrating pod in a loop — the classic Flyway-plus-K8s self-destruction).

K8s/ECS wiring: probes on the MANAGEMENT port (internal-only target group, Q3-18), liveness `periodSeconds=10, failureThreshold=3` (30s to declare wedged — balance: fast detection vs GC-pause false positives; G1 pause target 200ms makes 10s generous), readiness `periodSeconds=5` (drain latency is user-facing), and the ECS variant: ALB health checks = readiness, no liveness equivalent (ECS restarts on task failure — the checks that matter are the ALB's + the service's own watchdog). The subtlety that shows depth:

readiness during deploys — the new version's readiness includes the cache-warm flag, so traffic shifts only to warm instances (the cold-start p95 spike eliminated at the load-balancer layer, not by hoping); and **readiness flapping** — a dependency flapping at 80% success made an instance flap in/out of rotation historically (user-visible 502 blips); the fix was hysteresis in the custom indicators (consecutive-failure counting — 3 strikes over 15s before going unready) because binary flapping is worse than degraded-but-stable.

KEY TERMS TO MENTION: - Liveness (process-only, no deps) vs readiness (deps + warm-up) vs startup (boot window guard) - Flyway-migration + liveness-probe self-destruction scenario - Management-port separation; probe timing knobs with GC-pause awareness - Cache-warm readiness gate (cold-start spike eliminated at the LB) - Hysteresis against readiness flapping (consecutive-failure counting)

FOLLOW-UP QUESTIONS: 1. All instances go unready simultaneously (Redis down) — what does the user see? 2. Would you make readiness include the AI provider's health?

FOLLOW-UP ANSWERS: 1. The fleet drains to zero → ALB 502/503s — this is exactly why readiness ≠ availability for DEPENDENCY outages: if EVERY instance shares the dependency, readiness gating is fleet-wide self-DoS. Our answer is tiered: hard-dependency (DB) unready is correct (we can't serve), soft-dependency (Redis) readiness reports DEGRADED-but-UP (serve with fallbacks — Q7-8's design), so a Redis outage degrades latency, not availability. The probe semantics per dependency were decided in the Q3-18 design, and this question is precisely why. 2. No — the AI provider is a degradation path (template fallback, Q3-12), not a serving prerequisite; including it would take the whole API down when OpenAI blinks. The breaker + fallback + banner handles it. The rule: readiness includes what makes you UNABLE to serve, not what makes you IMPERFECT — the distinction is a product conversation encoded into probe config, and I can walk our dependency list one by one through it.

RED FLAGS TO AVOID: - Dependency checks in liveness (the fleet-churn self-DoS) - No startup probe with migrations (kill-loop on healthy pods) - Binary readiness on flapping dependencies (no hysteresis)

Q11-7: How did you handle zero-downtime deployments

DIFFICULTY: Hard **CATEGORY TAG:** DevOps

ANSWER: Zero-downtime is a CONTRACT between four subsystems, and each has obligations. **Schema (Flyway):** expand-contract — the schema at every moment supports N and N+1 app versions (Q6-6), the precondition for everything else. **App (rolling deploy):** K8s rolling update `maxSurge=1, maxUnavailable=0` — a new pod becomes ready (including cache-warm gate, Q11-6) BEFORE an old one drains; ECS variant: two-step deregister. **Connections (drain):** old pods get SIGTERM → executor drains (Q3-9's `waitForTasksToCompleteOnShutdown`, 30s grace) → ALB connection draining (30s) → in-flight requests complete; the readiness flip + deregistration ORDER (stop accepting BEFORE draining — the deregister-then-sleep sequence, not sleep-then-deregister — a subtle ordering bug that manifests as a burst of 502s during deploys). **Clients (resilience to the**

handover): idempotency keys on all state-changing calls (a request that dies mid-deploy is retried SAFELY — the deploy is just another transient failure class), websocket/SSE clients auto-reconnect with backoff (the real-time layer survives pod rotation with a reconnect burst handled by jittered backoff), and long-running REQUESTS are bounded (no request exceeds ~30s by design — a 10-minute “synchronous report” endpoint would make rolling deploys impossible; long work belongs in jobs — the API design respects the deploy model).

The DB-connection handover is the least-discussed failure: rolling deploys create connection churn (old pods’ pools close, new pools open) — Hikari’s maxLifetime + the pool’s initial-size pre-warm (open connections at boot, not lazily) prevent the post-deploy latency blip; and PgBouncer was evaluated for exactly this if task counts grow. Verification: deploy-event markers in Grafana annotate every rollout — the p95/p99/error-rate graph during deploys is a FLAT LINE and that flatness is an SLO we track (“deploy transparency”: error budget consumed by deploys < 0.5%); regressions there (we had one: the readiness-gate skip during a hotfix) become postmortems like any outage. Blue-green remains in the back pocket for schema-risky releases (full fleet flip with instant rollback — the cost is 2× capacity for the window; rolling covers 95% of deploys safely, and the expand-contract discipline is what keeps it that way).

KEY TERMS TO MENTION: - Four-subsystem contract: schema (expand-contract) + rolling (maxUnavailable=0) + drain ordering + client idempotency - Deregister-THEN-drain ordering (the 502-burst ordering bug) - Connection-pool pre-warm (post-deploy latency blip prevention) - Bounded request times as an API design constraint for deployability - Deploy-transparency SLO (flat p95 during deploys, tracked)

FOLLOW-UP QUESTIONS: 1. A deploy must change something NOT expand-contract compatible — now what? 2. Websocket-heavy product + rolling deploys — how do you not dump everyone?

FOLLOW-UP ANSWERS: 1. You pay for blue-green + a freeze window: deploy the schema expand step + new image to the IDLE fleet, verify, flip traffic at the boundary, keep the old fleet warm for rollback, contract later. The cost is capacity and coordination, which is exactly why expand-contract is the default and this path is the exception — the exception is rehearsed (game day) because unrehearsed exceptions are outages. 2. Jittered reconnect backoff (2-10s random — a synchronized reconnect stampede IS a self-outage), connection-state recovery (clients resubscribe with a last-event-id — the event bus replays missed events server-side, so a reconnect is a resume, not a data loss), and pod-affinity spreading (connections distributed so a rotation touches a fraction at a time). With those, a rolling deploy shows users a sub-second flicker in the worst case, invisible in practice — measured by the reconnect-burst metric, which we keep under 2% of connections per rotation.

RED FLAGS TO AVOID: - Zero-downtime claims resting on the app layer alone (schema + clients matter) - Drain ordering wrong (deregister AFTER sleep) - Unbounded request durations making rolling deploys structurally impossible

Q11-8: What is your monitoring and alerting strategy

DIFFICULTY: Medium **CATEGORY TAG:** DevOps / Observability

ANSWER: The strategy is SLO-first with symptom-based paging — not the CPU-dashboard theater. **The four golden signals** per service tier: latency (p50/p95/p99 per endpoint), traffic (RPM), errors (4xx/5xx rates + business-error classes), saturation (pool usage, queue depth, CPU) — all Micrometer → Prometheus → Grafana, with deploy annotations. **SLOs:** user-journey-level (lead-list p95 < 300ms over 28 days with 99.9% availability; checkout success > 99.5%; webhook processing lag p95 < 60s) — error budgets drive priorities (budget burn > 2× in an hour pages; exhausted budget freezes feature deploys for reliability work — the mechanism that makes SLOs real, not wall art). **Alert taxonomy:** P1 (pages, 24/7): symptom alerts only — SLO burn, 5xx rate, queue-lag breach, webhook freshness (Q9-8), cert expiry < 7 days; P2 (Slack, next-day): saturation trends, slow-query digests, cache hit-rate decay; everything else is dashboards, NOT alerts (alert fatigue kills paging systems — the rule: if an alert doesn't require a human DECISION, it's a dashboard).

Runbook discipline: every P1 alert links its runbook IN the alert payload (symptom → check list → mitigation → escalation) — unrunbooked alerts get demoted automatically until documented (the system enforces its own hygiene); game days rehearse the top runbooks quarterly (the Redis-down drill from Q7-10 is literally a scheduled exercise). Tracing (OpenTelemetry → Tempo-style backend) covers cross-boundary diagnosis: request → SQL timings → external calls on one waterfall, correlated with logs via reqId/tracId (Q8-9). The meta-layer is reviewed monthly: alert precision (what % of pages were actionable — target > 80%; we retired two noisy alerts last quarter after this review) and coverage (every incident postmortem asks “which signal SHOULD have caught this” — new signals come from incidents, which is how monitoring grows toward reality instead of toward vendor checklists).

KEY TERMS TO MENTION: - Four golden signals + SLOs with error budgets (burn-rate paging, budget-freeze) - Symptom-based P1 paging vs dashboard-only P3s (alert-fatigue economics) - Runbook-in-alert + auto-demotion of unrunbooked alerts - OTel tracing correlated with logs (reqId/tracId bridge) - Monthly meta-review: precision + incident-derived coverage growth

FOLLOW-UP QUESTIONS: 1. Your error budget froze feature deploys — the product manager is furious. Walk me through that conversation. 2. A P1 fires with NO runbook hit — what actually happened and what did you change?

FOLLOW-UP ANSWERS: 1. With data, not policy: the budget burn graph, the user-impact numbers (checkout failures during the burn window), and the pre-agreed freeze rule from the SLO charter the PM co-signed. Then the negotiation the charter also pre-agreed: ship the revenue-critical feature behind a flag at 5% while reliability work proceeds, or accept a documented budget exception with a make-up reliability sprint. The SLO isn't anti-product; it's the shared language that stops reliability being endlessly traded for shipping. PMs respect a system that binds ENGINEERS first — we've eaten the freeze ourselves. 2. Real case: the webhook freshness alarm fired on a provider whose events silently stopped — no runbook existed (new integration, alarm added with the feature, runbook lagged). The incident itself went fine (the alarm's diagnostic payload listed the reconciliation sweep we

ran manually), and the changes were: runbook written same-day (enforced by the auto-demotion rule), the alarm payload gained the exact curl-able replay commands, and the quarterly game-day list gained the scenario. The meta-answer: monitoring failures are process failures with good manners — they always end in runbook or signal changes.

RED FLAGS TO AVOID: - CPU/memory alerts as the P1 story (cause-based paging) - SLOs without error-budget enforcement (wall art) - No runbook linkage / alert-fatigue blindness

Q11-9: How did you handle logging in production (the ELK-shaped pipeline)

DIFFICULTY: Medium **CATEGORY TAG:** DevOps / Observability

ANSWER: The pipeline: **emit** (Logback, structured JSON via logstash-encoder — every line a field map, not a regex target; MDC carries reqId/tracId/userId/orgId so every line is joinable, Q8-9) → **collect** (Filebeat sidecar/DaemonSet tailing stdout — containers log to stdout, the platform ships it; apps never manage files) → **ship** (Kafka-buffered Logstash or Vector — the buffer absorbs Elastic downtime without app-side backpressure: logging must never slow the app, so the shipper decouples with disk-buffered retry) → **index** (Elasticsearch with ILM: hot 7 days (replicas, fast storage) → warm 30 → cold 90 → delete; audit-trail parity handled separately per Q6-14's compliance tier) → **view** (Kibana saved searches per runbook + the exec view). Field discipline: a schema doc for log fields (owned, versioned — new fields through review, because 40 teams inventing field names is how “user_id”/“userId”/“uid” happens; ours standardized in month one).

Volume economics and hygiene: log levels per package tuned per environment (INFO prod, DEBUG staging), request-body logging DEBUG-only + redaction (Q8-9), and SAMPLING for the noisy middle (healthy 2xx access lines 100% — they're small and load-bearing; high-frequency internal events sampled 10% with a rate-limited always-log for errors). Costs are monitored as an SLO (ingest GB/day per service — a 3x spike usually means a bug logging inside a loop; the volume alert has caught two such bugs, making it a reliability signal, not just a bill). Access: SSO-gated Kibana with role scoping (support sees app logs, not infra; PII-field access is a separate audited role), and the retention question is answered by policy: operational logs 90 days, security-relevant per the audit trail's 400-day + archive tier — retention is a compliance surface, not an Elastic default.

The design tension I name honestly: structured logging trades grep-culture for field-culture — the migration had real friction (engineers mourning `grep 'timeout'`), solved by Kibana-saved-search enablement + the query cookbook in the runbook; and the deepest lesson: logs are for QUESTIONS, so we maintain a “top 10 support questions → exact saved query” mapping — if a question can't be answered in under a minute by the on-call, the logging has a gap, and that gap list drives the logging backlog. Logging without a question inventory is storage with anxiety attached.

KEY TERMS TO MENTION: - Structured JSON + MDC joinability (reqId/tracId/userId/orgId) - stdout → Filebeat → buffered shipper (logging never backpressures the app) - ILM tiers (hot/warm/cold/delete) + retention as compliance policy - Volume-as-signal (GB/day spike = bug

detector) + level/sampling hygiene - Question-inventory-driven logging backlog (top-10 → saved queries)

FOLLOW-UP QUESTIONS: 1. Elasticsearch is down 2 hours — what do users and on-call experience? 2. A log field carries PII you missed — remediation?

FOLLOW-UP ANSWERS: 1. Nothing user-visible (the decoupled shipper buffers to disk, apps log happily — the pipeline is designed so observability infrastructure failures NEVER touch the product); on-call loses search for 2 hours — the runbooks degrade to metric-based triage (Grafana answers the P1 questions) + the latest indexed window for context. Ingest catches up on recovery (Kafka buffer sizing covers 24h of volume). The design goal is literally “ELK can burn and the pager stays quiet” — that’s what buffer decoupling buys. 2. Same play as Q8-9’s variant with pipeline specifics: stop the emission (field removed, deploy or hot level-change), assess the indexed exposure (which indices, what retention — ILM will age it out; an expedited delete-by-query for sensitive classes), check who accessed (Kibana audit), document in the RoPA, and add the field to the pre-production PII scan (we run a pattern-scan over sampled prod logs weekly — the miss means the pattern list needs the new entry). The systemic fix is always the scanner, not the vigilance.

RED FLAGS TO AVOID: - App-side file logging with rotation inside containers (stdout or nothing) - Logging pipeline that can backpressure the app (no buffer decoupling) - Unstructured logs + regex-only culture at this scale

Q11-10: How did you manage secrets in production

DIFFICULTY: Medium **CATEGORY TAG:** DevOps / Security

ANSWER: The secrets lifecycle, stage by stage. **Generation/storage:** AWS Secrets Manager as the single source (per-service paths, versioned — a rotation publishes a new version without deleting the old, enabling the two-valid-window pattern); nothing secret in git (gitleaks pre-commit + CI scan — historical scan included), in images (multi-stage hygiene + a final-image secret-scan), or in CI variables beyond references (CI holds ARNs, not values). **Delivery:** init sidecar fetches at container start → exports env vars → `@ConfigurationProperties` binds → boot validation fails on missing (named errors, Q10-7); the app reads secrets ONLY from env/config — no secret-fetching code scattered in business logic (fetching is infrastructure’s job). **Use:** in-memory only, masking converters on all log paths (Q3-19), no secrets in error messages (the exception-sanitizer strips config values — an NPE once tried to print a connection string with credentials; the sanitizer exists because of it). **Rotation:** DB creds quarterly (dual-valid window + rolling restart, Q3-19), JWT keys `kid`-versioned, provider keys per Q9-9’s census; rotation is rehearsed (a game-day rotation of the DB secret — the first rehearsal found the Hikari pool didn’t pick up new creds without a recycle, which became a documented step and then a fixed behavior). **Access:** IAM task-roles scoped per service (the app role reads `/acq/app/*`, never `/acq/analytics/*`), Kibana/secret-admin separate audited roles, and break-glass access (2-person rule, time-boxed, alerted).

Breach drill (the question behind the question — “a secret leaked, now what?” is Q9-9’s runbook, and at the infra layer it’s: rotate FIRST at the provider/manager (invalidate the leak), then history-purge, then access-audit the misuse window, then postmortem the pipeline gap). The census job (Q9-9) extends infra-side: it inventories secrets-manager entries, cross-checks against IAM grants, flags entries unused for 60 days (zombie credentials), and verifies every entry has a rotation policy — the report goes to the security review. The principle I close with: secrets management is measured by its BORINGNESS — rotation happens on schedule without ceremony, leaks are contained by rotation-not-panic, and no engineer can casually acquire a production secret; the day secret handling becomes an adventure is the day the design failed.

KEY TERMS TO MENTION: - Secrets Manager single source + versioned rotation (two-valid windows) - Sidecar → env → typed config delivery; no secret-fetching in business code - gitleaks CI + final-image scans + CI holds references not values - Rehearsed rotations (game-day found the Hikari recycle gap) - Census job: zombie detection, grant cross-check, rotation-policy verification

FOLLOW-UP QUESTIONS: 1. Why env vars at all — why not the app fetching directly from Secrets Manager? 2. KMS key rotation — does it re-encrypt everything?

FOLLOW-UP ANSWERS: 1. Direct-fetch couples app code to AWS SDK + adds failure modes inside business boot paths; the sidecar keeps the app portable (runs in dev with env files, in prod with the sidecar — same code), centralizes IAM in the platform layer, and the env/config boundary matches Spring’s native model. The trade (env vars are process-visible via /proc) is mitigated by container isolation + readOnlyRootFilesystem + non-root. Fetch-in-app is the right call when you need dynamic refresh mid-process — we handle that class with flags, not secrets. 2. KMS key rotation doesn’t touch existing ciphertexts — new encrypts use the new key material, old ciphertexts still decrypt (KMS keeps versions); rotation is therefore SAFE-by-default but doesn’t reduce exposure of old data — for the envelope-encrypted user tokens (Q9-9) we run a scheduled re-wrap job (decrypt-with-old, re-encrypt-with-new, batched) annually; the job is boring and rehearsed, which is the entire point of the design.

RED FLAGS TO AVOID: - Secrets in git/images/CI-values (any one is an audit finding) - Un-rehearsed rotation (the first real rotation should not be during an incident) - No zombie-key census (you can’t secure an inventory you don’t have)

CATEGORY 12 — PERFORMANCE AND OPTIMIZATION QUESTIONS

Q12-1: What were the performance bottlenecks you faced

DIFFICULTY: Hard **CATEGORY TAG:** Performance

ANSWER: Three notable bottlenecks we hit in production, each requiring a different fix:

Bottleneck 1 — N+1 on `Lead.user` (p95 800ms → 60ms): See Q5-5. A list view triggered 21 SQL queries. Fix: `@EntityGraph(attributePaths = "user")` on the repository method.

Bottleneck 2 — Stripe webhook retry storm (5xx spike to 100%): See Q9-2. We had `@Transactional` on the whole webhook handler; one bad webhook held a DB connection for 30s, HikariCP exhausted at 20 connections, all subsequent requests got 500, Stripe retried, more 500s. Fix: split handler into “ingest” (just persist raw, return 200) + “process” (async via Kafka consumer). We added a load test that fires 1000 concurrent webhooks as a regression test.

Bottleneck 3 — Monthly credit reset job ran 12 minutes (expected 1 minute): The job iterated 3,200 users, ran a `@Transactional` per user (3,200 transactions), each transaction verified the FK on `credit_ledger.user_id` (3,200 SELECTs). Fix: (a) `@Retryable` on the outer job to handle transient errors; (b) `@ForeignKey(value = ConstraintMode.NO_CONSTRAINT)` on `CreditLedger.user` (Hibernate skips the FK verification); (c) batch inserts via `JdbcTemplate.batchUpdate()` (10 batches of 1000 rows). Result: 12 minutes → 30 seconds.

Bottleneck 4 — SSE connections didn’t release on ECS task termination: When we deployed, in-flight SSE connections held open for 30s (Tomcat’s default `connectionTimeout`), the ECS task couldn’t drain, ALB health check failed, deploy stalled. Fix: (a) `server.servlet.session.timeout = 0` (no session timeout for SSE); (b) on SIGTERM, send a `503 Service Unavailable` to all SSE emitters with a `Retry-After: 5` header — clients reconnect to another task within 5s; (c) drain the SSE emitters before graceful shutdown completes. Result: deploys go from 30s stuck to 5s clean drain.

Bottleneck 5 — Kafka consumer lag during batch jobs: The `lead-discovered` topic spiked to 5K messages during a Spring Batch job; the consumer (notification service) was single-threaded, lag grew, notifications were delayed by minutes. Fix: (a) increase consumer concurrency (`ConcurrentKafkaListenerContainerFactory.setConcurrency(4)` — 4 consumer threads); (b) batch consume (`@KafkaListener(batch = "true")` — process 50 messages per poll). Result: lag dropped from 5K to <100 steady-state.

Bottleneck 6 — Hibernate second-level cache wasn’t helping: We enabled Hibernate’s second-level cache for the `Organization` entity (small, rarely changes). Cache hit ratio was 20% — worse than no cache (overhead > benefit). Root cause: we used `@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)` but the cache key was per-Org, and most queries were “find by user_id” (which the L2 cache doesn’t help — it caches by entity

ID, not by query). Fix: removed L2 cache for Hibernate, added a manual Redis cache (`@Cacheable` Spring Cache) for `getOrganizationById`. Result: 95% hit ratio for org lookups.

General approach to performance: 1. **Measure first** — OpenTelemetry traces + p95/p99 percentiles + slow query log + JVM GC logs. We don't optimise without data. 2. **Identify the bottleneck** — is it CPU, IO, DB, external API, GC, or lock contention? Each requires a different fix. 3. **Fix at the right layer** — DB query fix (`@EntityGraph`) > app code fix (caching) > infra fix (scaling). Cheaper to fix near the source. 4. **Re-measure** — load test before + after, quantify the improvement. 5. **Regression test** — add a test that fails if the bottleneck reappears (e.g. `@ExpectedQueryCount` for N+1, a load test for webhook storm).

KEY TERMS TO MENTION: - Six concrete bottlenecks: N+1, Stripe retry storm, credit reset job, SSE drain, Kafka lag, Hibernate L2 cache misuse - Three-step fix pattern: measure (OTel + percentiles + slow query log), identify (CPU/IO/DB/external/GC/lock), fix at the right layer (DB > app > infra), re-measure + regression test - Specific fixes: `@EntityGraph`, async + Kafka, `JdbcTemplate.batchUpdate`, `ConstraintMode.NO_CONSTRAINT`, SSE drain on SIGTERM, `ConcurrentKafkaListenerContainerFactory` + batch consume, Spring `@Cacheable` (Redis) > Hibernate L2 - `@ExpectedQueryCount` test annotation + load tests as regression guards

FOLLOW-UP QUESTIONS: 1. How do you profile a Java app in production? 2. What's the biggest perf improvement you've made? 3. How do you prevent regressions?

FOLLOW-UP ANSWERS: 1. Three tools: (a) **OpenTelemetry traces** — every request has a span; slow spans are visible in Tempo, with the method + DB query + external call; (b) **JFR (Java Flight Recorder)** — `jcmd <pid> JFR.start duration=60s filename=/tmp/recording.jfr` — continuous low-overhead profiling; we analyse with JMC (JDK Mission Control) to find hot methods + GC pauses; (c) **async-profiler** — CPU + alloc + lock profiling with ~1% overhead; we run it for 60s during an incident to find the hot method. We avoid JProfiler / YourKit in prod (licence + overhead). 2. The Stripe webhook storm fix (Bottleneck 2) — going from 100% 5xx to 0% in one refactor. The fix was small (split sync handler into ingest + async process) but the impact was huge (production outage → invisible). The team learned the lesson “decouple from external retry pressure” — we now apply the pattern to every webhook handler. 3. Three layers: (a) **load tests in CI** — Gatling runs a 10-min 1000-virtual-user test on every PR to main; p95 < 500ms + 5xx < 0.1% required; (b) **performance regression test** — `@ExpectedQueryCount` on every repository method (catches N+1); JFR baseline profile committed, alerts if a new method is hot; (c) **production SLOs** — PagerDuty alerts if p95 > 300ms (we're at 180ms) or p99 > 800ms (we're at 540ms) — gives us an early warning before users notice.

RED FLAGS TO AVOID: - “We don't have performance issues” (every app does) - “We use profiling tools” without naming them - Optimising without measuring first - Not having regression tests for performance

(The remaining performance questions — optimising the lead discovery pipeline, handling high traffic, async processing, DB query optimisation, reducing API response times, load testing, connection pool tuning, JVM memory optimisation, JVM tuning for prod, GC tuning

(G1 vs ZGC), off-heap memory, L1/L2 cache, query plan cache, slow query log analysis — follow the same template. Drill on: how to read a GC log, what causes `Full GC`, the difference between `Xms` and `Xmx`, what `-XX:+UseStringDeduplication` does, the difference between `serial` and `parallel` GC, how to detect a memory leak vs a memory pressure.)

Q12-2: How did you optimize the lead discovery pipeline

DIFFICULTY: Hard **CATEGORY TAG:** Performance / Spring Batch

ANSWER: The pipeline (discover → scrape → enrich → score → persist) went from 22 min/1k leads to 6.5 min/1k leads across four optimization waves, each measured. **Wave 1 — parallelism done right:** the scrape step was single-threaded I/O-bound; Spring Batch's `taskExecutor` on the step + `throttleLimit` gave 8-way concurrency, BUT the naive version hammered targets (429 storms) — the real design is per-target politeness: domain-keyed rate limiter (Redis token bucket per domain — 1 req/2s/domain) + a proxy pool rotation (residential proxies on 429) — parallelism WITH manners outperformed raw threads because retries were the actual bottleneck (28% of calls were retries before; 3% after). **Wave 2 — batch-write discipline:** JPA-per-entity inserts were 500/s ceiling; switched to the Q5-16 stack (jdbc batch_size 50, SEQUENCE generation, clear() per chunk) → 4,200/s persist rate; the scorer consumed pre-computed features via DTO projection instead of loading aggregates (the classic “load 9 columns to read 2”). **Wave 3 — LLM-call economics:** enrichment's Gemini calls were 60% of wall time; added (a) response caching (business identity hash → enrichment, 30-day TTL — 35% of leads re-query known businesses), (b) batching to the batch-API endpoint (half cost, same latency class), and (c) the fallback chain so non-critical enrichments degrade to heuristic scoring instead of blocking the pipeline. **Wave 4 — back-pressure and fairness:** a shared worker pool meant a big run starved interactive discovery (users waiting on their 50-lead search behind a 50k-lead batch) — work classes were split (interactive queue preempts batch, bulkhead per class, Q3-9) with fairness metrics proving interactive p95 stayed < 8s during batch runs.

Measurement wrapped every wave: per-step Micrometer timers (discover/scrape/enrich/score/persist as separate gauges — “the pipeline is slow” is always ONE step; the step-level view finds it in seconds), items/sec + retry-rate + provider-cost per run in the batch execution metadata, and a regression guard (nightly synthetic run against a fixture set — throughput drops > 15% page). The meta-lesson I share: three of the four waves were NOT code speed — they were politeness (rate limits), economics (caching/batching), and fairness (back-pressure); production performance is systems engineering wearing a profiler's clothes.

KEY TERMS TO MENTION: - Per-domain rate limiting + proxy rotation (parallelism with manners; retries were the bottleneck) - Jdbc batch_size + SEQUENCE + clear() (4,200/s persist) + DTO-projection scoring - LLM economics: identity-hash caching (35% hits) + batch API + heuristic fallback - Work-class bulkheads (interactive vs batch fairness — interactive p95 < 8s) - Per-step metrics + nightly synthetic regression runs

FOLLOW-UP QUESTIONS: 1. Why did the naive thread-pool parallelism make things WORSE? 2. The pipeline is still the product's cost center — where does cost optimization go next?

FOLLOW-UP ANSWERS: 1. 8 threads × no per-domain pacing = 429/403 storms; each retry consumed a thread-slot for its backoff, the pool saturated on RETRIES rather than first-attempts, throughput halved, and two scraping targets banned our egress IPs for an hour. The fix inverted the model: pacing is per-RESOURCE (domain), parallelism per-WORKER — threads multiply against the pacers, not against the targets. It's the difference between concurrency and vandalism. 2. By marginal-cost ranking: enrichment LLM calls (~62% of run cost) → cache hit-rate improvements (the identity-hash keying can improve to ~50% with better normalization), then model right-sizing (score-tier models per lead class — small model for obvious-rejects), then batch-API share (currently 40%, can go 80%). Scraping bandwidth (~25%) → cache politeness and source-prioritization. Every line item has a dashboard, so this conversation is a query, not a debate — cost optimization is an analytics product at this point.

RED FLAGS TO AVOID: - Threads-first thinking without rate-limit/retry analysis - Unmeasured waves ("it felt faster") - No fairness story (batch starving interactive users is a product bug)

Q12-3: How did you handle high traffic scenarios (launch spike, viral moment)

DIFFICULTY: Hard **CATEGORY TAG:** Performance / Scaling

ANSWER: Two preparation layers and one reaction playbook. **Preparation 1 — headroom by load-test evidence:** k6 scenarios shape-tested monthly (steady 3× current peak, burst 10× for 5 min) on a staging clone of prod sizing — the tests found our cliffs BEFORE traffic did: the connection-pool math (Q6-5), the Redis ops ceiling (15k/s vs burst projection 40k — fixed by pipeline batching of rate-limit checks), and a Tomcat thread default (200) that queue-collapsed at 6k concurrent — each cliff has a fix recorded with the test that found it; capacity is therefore a NUMBER with evidence, not a hope. **Preparation 2 — graceful degradation plan:** the feature-flag load-shedding ladder, pre-agreed with product (Q7-10): analytics non-essential endpoints → AI generation queued instead of inline → export/batch jobs paused → read-only mode for the least-critical tier; each rung is a flag flip with an owner named in the runbook. **Reaction playbook** (the spike is happening): autoscaling absorbs the first minutes (Fargate target-tracking on request count + CPU, scale-out in ~90s — the pre-warmed image cache and pool warm-up make cold instances ready in < 30s, Q10-7); the traffic dashboard distinguishes LEGITIMATE surge (conversion funnel healthy — scale and enjoy) vs ABUSE (single-source flood — WAF rate rules tighten, specific shreds) vs DEPENDENT-DEGRADATION (downstream slowness — the breaker dashboard shows which), because the three need opposite responses and misreading abuse as demand (or vice versa) is the classic incident amplifier.

The load tests also validated the recovery path (the part most teams skip): release the flood → autoscale-IN has cooldowns (scale-in stabilization 10 min — without it, oscillation: scale-out/scale-in thrash during bursty traffic, each cycle a latency blip); post-spike cost review (Fargate burst costs real money — the spike playbook includes “who approves sustained > 5× spend” — cost is an incident dimension). The launch that tested it: a ProductHunt moment drove 40× signup traffic for 6 hours — the system shed the marketing-page load to CDN (static — already off-app), signups queued gracefully behind a “you’re in line” UX (the queue-it pattern via the WAF), and the funnel metrics stayed green; the postmortem was 90% celebration, which is the point of rehearsing.

KEY TERMS TO MENTION: - Monthly k6 shape-tests finding cliffs pre-traffic (pool/Redis/thread numbers as evidence) - Pre-agreed degradation ladder (flags with owners; analytics → queue AI → pause batch → read-only) - Surge triage: legitimate vs abuse vs dependent-degradation (opposite responses) - Scale-in stabilization (oscillation prevention) + cost-as-incident-dimension - Launch case: 40× spike, CDN + queue-UX + green funnel

FOLLOW-UP QUESTIONS: 1. Why target-tracking autoscaling over a fixed over-provisioned fleet? 2. The queue-UX for signups — is that honest engineering or a hack?

FOLLOW-UP ANSWERS: 1. The fleet math: 24/7 over-provisioning for a 40×-burst-1-day-a-quarter workload wastes ~85% of compute spend; target-tracking costs pennies of complexity for elasticity (with the stabilization windows handling burstiness). Fixed oversized fleets earn their keep when: scale-out latency is intolerable (sub-second markets), or cold-start cost exceeds idle cost — neither true for us. The hybrid (small always-on core + elastic burst) is exactly what target-tracking IS; I can show the cost curve if asked. 2. It’s both, deliberately: a queue with honest position + ETA is the USER-RESPECTFUL way to shed load (they wait informed instead of staring at 503s), and it’s real engineering — token-bucket admission at the edge, sticky queue tokens, prioritization for existing customers. The hack version is a silent 503 wall. The pattern (virtual waiting room) is what Ticketmaster-scale systems do; ours is the small-but-honest cousin, and the product team owned the UX so engineering owned the mechanism.

RED FLAGS TO AVOID: - “Kubernetes scales us” with no load-test evidence (cliffs hide everywhere) - No abuse-vs-demand triage (autoscaling an attack is paying for your DDoS) - No degradation ladder (everything fails together at the cliff)

Q12-4: What is your approach to async processing (choosing between @Async, queues, and batch)

DIFFICULTY: Medium **CATEGORY TAG:** Performance / Architecture

ANSWER: The selection rule is about DURABILITY and LATENCY tolerance, stated as a decision tree I actually apply. **@Async (in-JVM executor):** fire-and-forget where losing the work on crash is acceptable AND latency tolerance is immediate — cache evictions, push-notification pre-warm, log shipping; the price of admission: handlers must be idempotent (deploys re-run nothing, but thread-pool overflow with CallerRunsPolicy can re-order) and the work is SHORT (seconds — it occupies a JVM thread). **Job-table queue (DB-backed, SKIP**

LOCKED): work that MUST survive restarts with per-item status/retry/visibility — outreach sends, reminder dispatch, webhook processing; latency tolerance is seconds-to-minutes; the trade: it's not high-throughput (DB poll ceiling ~ hundreds/sec) but it's OPERABLE (SQL visibility, replay, dead-letter states — the Q2-11 reasoning). **Spring Batch:** high-volume, chunk-oriented, restartable DATA processing — imports, enrichment pipelines, retention sweeps; latency tolerance minutes-hours, throughput thousands/sec, checkpointing built-in. **Kafka/SQS:** the durability + throughput tier we added when volume or fan-out outgrew the table (notification fan-out, analytics events) — the ADR records each addition's trigger (Q2-14's migration story).

The anti-patterns the tree prevents, each with a scar: @Async for durable work (a deploy lost 400 queued notifications once — the incident that CREATED the job-table rule: if losing it hurts, it's not @Async); queue for sub-second-latency needs (the UI waited on a queued enrichment — moved inline with a timeout + async refinement); batch for interactive traffic (batch's commit-interval checkpointing adds seconds of latency granularity nobody wants in a request path); and long work in HTTP requests (Q11-7's deployability constraint — a 5-min synchronous LLM chain is a deploy hazard AND a timeout lottery; it became a job with SSE progress). The async surface is also where THREAD-POOL hygiene lives: named pools per work class (Q3-9), queue bounds + rejection policy chosen per class, and pool metrics (active/queue/rejection) as first-class dashboards — async systems fail by QUEUING before they fail by breaking, so the queues are watched like fuel gauges.

KEY TERMS TO MENTION: - Decision tree: durability requirement × latency tolerance × throughput class - @Async = tolerable-loss short work; job-table = durable operable; Batch = chunked restartable; Kafka = fan-out/throughput tier - The lost-notifications incident that created the durability rule - Queues fail by backpressure before code fails — pool/queue metrics as fuel gauges - Long work never lives in HTTP requests (deployability + timeout lottery)

FOLLOW-UP QUESTIONS: 1. How do you decide a queue's retry count and dead-letter policy? 2. When does @Async's CallerRunsPolicy become the wrong choice?

FOLLOW-UP ANSWERS: 1. From the WORK's business semantics, not the queue's defaults: each job type declares max attempts + backoff + expiry (a reminder older than its meeting is worthless — TTL; a payment webhook retries for 24h — money). Dead-letter means: surfaced in the admin UI with the failure reason + one-click replay + a metric that pages when the DLQ grows (a growing DLQ is a bug detector wearing a queue costume). Defaults exist per class; the per-job declaration is the review checklist item. 2. When the caller is latency-sensitive (user-facing request): CallerRuns turns a dropped notification into a slow CHECKOUT — the wrong trade. Fire-and-forget classes use Abort + metric (drop and alarm) because losing a cache-warm beat a stalling purchase; durable classes don't run on these pools at all. The policy encodes “which failure can the user tolerate” — a product question wearing a concurrency knob.

RED FLAGS TO AVOID: - One pool/queue for everything (no class separation) - @Async for anything whose loss hurts - No queue-depth visibility (discovering backpressure via user complaints)

Q12-5: Your best database query optimization war story — walk me through it end to end

DIFFICULTY: Hard **CATEGORY TAG:** Performance / PostgreSQL

ANSWER: The leads-list p95 regressed from 180ms to 1.9s over three weeks of growth — no deploy correlated, which is what made it interesting. **Detection:** the SLO burn alert (Q11-8) fired; the trace layer (Q8-9) resolved it to ONE statement — the org-scoped leads query with filters. **Reproduction:** staging at equal row counts was FAST (the trap) — production snapshot restored locally reproduced it; `EXPLAIN (ANALYZE, BUFFERS)` showed the plan flipping: the planner chose a `(org_id, created_at)` index for most orgs but SEQ-SCANNED for the largest org (their 400k rows exceeded the random-page-cost tolerance of an index scan given cached-fraction estimates) — parameter-dependent plan flip, the multi-tenant disease (Q6-10's foreshadowing). **Fix options ranked:** (a) `force_custom_plan` — blunt, punishes every org with parse cost; (b) more RAM — delays the cliff, costs money, doesn't fix shape; (c) THE real fix: the query's filter-set had grown (status IN + owner + search prefix) but the index hadn't — a **partial composite index** `ON lead (org_id, status, created_at DESC) WHERE deleted_at IS NULL` covering the hot path (supporting index-only scans via `INCLUDE` columns for the `SELECT` list) turned the whale-org plan into an index-range scan at ANY row count — p95 for the whale org: 1.9s → 95ms; fleet p95 back to 170ms. **Verification:** before/after per-org latency percentiles (not fleet averages — averages HID the whale), `pg_stat_user_indexes` `idx_scan` delta confirming the new index earns its write cost, `pg_stat_statements` total-time drop, and the regression test added (the plan-flip scenario in the query-budget test suite).

The lessons I extract for interviewers: **averages hide tenants** (per-org percentiles are the multi-tenant monitoring table stakes — the fleet p95 was fine while our biggest customer suffered); **plan instability is a first-class bug** (not “weird Postgres” — it's data-shape evolution meeting a static index set; the weekly `pg_stat_statements` review catches drift before customers, Q6-10); **index design follows QUERY evolution** (three filters accreted over months; nobody revisited the index — the schema-health cron now flags queries whose filters lack a supporting index shape); and **write-cost accounting** (the new index costs ~4% on lead writes — measured, accepted, documented in the index's comment). One index, four lessons, one customer saved.

KEY TERMS TO MENTION: - Parameter-dependent plan flip (planner + data skew) as the root cause - Partial composite index + `INCLUDE` (covering) as the shape fix - Per-org percentiles (averages hide the whale) — the monitoring lesson - Verification quartet: per-org p95, `idx_scan` delta, `pg_stat_statements`, regression test - Write-cost accounting for new indexes (~4%, documented)

FOLLOW-UP QUESTIONS: 1. Why was staging fast while production crawled — beyond row counts? 2. When is `force_custom_plan` actually the right answer?

FOLLOW-UP ANSWERS: 1. Row count parity but not DISTRIBUTION parity: staging's tenants were uniform; production had the whale. Plans depend on per-tenant data fractions (the planner's estimates drive the flip), plus production's buffer cache held different hot pages. The lesson institutionalized: performance staging uses PRODUCTION SHAPES (skewed

tenants, real filter mixes) — uniform fixtures test nothing about plan stability. 2. When the flip is narrow and the hot plan is provably right: a single statement class with known skew (the whale org’s report query), where parse overhead is irrelevant and re-planning costs more than it saves — pinned with a comment explaining the shape assumption and a monitoring tripwire (if the skew changes, the pin is wrong). It’s a scalpel, not a setting; `plan_cache_mode` globally is how you trade one disease for another.

RED FLAGS TO AVOID: - Diagnosing from averages (fleet p95 healthy, whale suffering) - “Postgres chose a bad plan” without the BUFFERS-level story - Adding the index without write-cost accounting or regression tests

Q12-6: How did you reduce API response times (the p95 program)

DIFFICULTY: Medium **CATEGORY TAG:** Performance

ANSWER: As a program with a budget model, not a collection of hacks. **The budget model:** every endpoint gets a latency budget decomposed by layer (auth+filters 15ms → business logic 60% → DB 30% → serialization 10%) — traces show actuals vs budget per span; over-budget endpoints enter the backlog with the trace attached (the performance backlog is DATA-ranked: total user-milliseconds saved per fix, not the loudest complainer). **The quick wins** (measured, cumulative -38% p95): response compression (Brotli for text — 70% payload cut on list endpoints), the DTO projection pattern replacing entity loading on read paths (Q5-13’s category 3), connection-pool pre-warm + Hikari tuning (post-deploy blip elimination), Jackson profile (afterburner-style module + DTO shapes that skip reflection-heavy paths), and the N+1 sweep (batch_fetch_size default + EntityGraph on the top 10 reads). **The structural wins:** caching per the Q7 catalog (detail reads, aggregates — the biggest single contributor, -45% on cached families), cursor pagination on the hot lists (Q8-3 — the 1.9s whale case lives here), and async-ization of anything not needed for the response (notifications, audit-high-volume, email — the request thread does the USE-CASE, nothing else).

The tails are the program: p50 was always fine — the program’s actual work was tail pathology: GC pauses (G1 tuning + heap right-sizing — the Q12-9 story), cold-cache thundering herds (single-flight, Q7-7), lock contention bursts (the FOR UPDATE audit — Q6-13’s pool-starvation case), and retry-storm amplification (jitter + breakers). Each tail fix came from tracing distribution SHAPE, not averages: the “hmm, p99 is 12× p50” questions map to exactly those four mechanisms. Sustained discipline: the p95/p99 per-endpoint trend lines on the team dashboard with deploy annotations (Q11-7’s deploy-transparency SLO), a latency-budget review in each retrospective for touched endpoints, and performance acceptance criteria in the definition-of-done for new endpoints (budget assigned at design, verified at ship — performance as a requirement, not a post-launch apology). The number that closes it: two quarters of this program took fleet p95 from 420ms to 165ms and p99 from 2.8s to 480ms — with the breakdown per lever preserved in the ADR.

KEY TERMS TO MENTION: - Endpoint latency budgets decomposed by layer; data-ranked backlog (user-ms saved) - Quick wins: compression, DTO projections, pool pre-warm,

Jackson, N+1 sweep - Structural: caching catalog + cursor pagination + async-ize everything non-response - Tail pathology quartet: GC, cache herds, lock bursts, retry storms - DoD performance criteria + deploy-transparency SLO

FOLLOW-UP QUESTIONS: 1. Which quick win under-delivered vs expectation — and why? 2. How do you keep the program from eroding after the hero phase?

FOLLOW-UP ANSWERS: 1. Caching under-delivered on the LEADS LIST (the highest-traffic endpoint): hit-rate landed at 22% (filter diversity — the Q7-2 honest no-cache decision) despite being the “obvious” cache candidate; the real win there was the index + cursor work instead. The lesson institutionalized: hit-rate PROJECTION before caching (a week of telemetry on filter-cardinality), not cache-first faith — the backlog ranks by evidence, and this entry taught the whole team that lesson cheaply. 2. The erosion defense is the budget-in-DoD + the trend dashboard: new endpoints arrive with budgets (design-time), regressions surface on the trend line with deploy annotations (no archaeology), and the quarterly “latency budget audit” re-ranks the backlog. Hero phases decay everywhere; what persists is the MECHANISM (budgets, traces, trend gates) — the program’s output is the immune system, not the heroics.

RED FLAGS TO AVOID: - Optimization without the budget/trace model (whack-a-mole by complaint) - p50-only reporting (tails are where users live) - No DoD criteria — performance erodes silently post-program

Q12-7: What is your approach to load testing

DIFFICULTY: Medium **CATEGORY TAG:** Performance / Testing

ANSWER: Load testing is a monthly discipline with shape fidelity, not a pre-launch ritual. **Scenario design from production telemetry:** k6 scripts derive their request mixes from real traffic distributions (endpoint popularity, payload sizes, auth-user ratios, think-time percentiles) — a test with uniform request mix validates nothing (the shape IS the test); scenarios: steady (3× current peak), burst (10×, 5 min — the autoscaling test), soak (2× for 4 hours — the memory-leak/queue-growth test: heap, pool counts, queue depths trended), and spike-recovery (flood → release — the scale-in oscillation test, Q12-3). **Environment fidelity:** staging at prod SIZING (the same task counts/pool sizes — a 1-replica staging load test lies about locks, pools, and contention), production-shaped DATA (row counts AND tenant skew — Q12-5’s lesson institutionalized), and the DB as a restored snapshot (fresh-empty DBs make every query look instant). **What we measure:** not just RPS/latency at the client — the full internal picture via metrics during the run: per-endpoint p95s, pool utilizations, queue depths, GC pauses, DB waits (pg_stat_activity snapshots), cache hit rates — the cliffs live INTERNALLY, and the client-side number is the symptom, not the diagnosis.

The findings ledger: every load test outputs a ranked cliff list (what saturated first, at what multiplier) with owners and fixes tracked to the next run — over four runs the fleet went from first-cliff at 4× to clean at 10× (fixes: rate-limit Lua batching, pool resize, a Tomcat thread default, one N+1 in the notification path that only manifests under queue pressure).

The safety rules: load tests never run against production (the “just a little traffic”

temptation — staging clones only, with the ONE exception: a 30-second synthetic canary at 1.2× current prod during a maintenance window, explicitly approved, measuring REAL infra — documented as a controlled exception, not a habit); and test teardown is automated (staging restored to baseline — a leaked load-test dataset corrupts the next test’s numbers). CI integration: the burst scenario runs on-demand pre-launch (a release-candidate gate for major features), while the full matrix stays monthly — monthly because infrastructure drifts (a dependency upgrade silently changing pool behavior is exactly what run 7 caught).

KEY TERMS TO MENTION: - Telemetry-derived request mixes (shape IS the test); steady/burst/soak/spike-recovery scenarios - Prod-sizing + prod-shaped-data staging (uniform fixtures validate nothing) - Internal-first measurement (pools, queues, GC, DB waits — cliffs live inside) - Findings ledger with ranked cliffs tracked across runs (4× → 10× journey) - Never-prod rule + the documented controlled exception

FOLLOW-UP QUESTIONS: 1. A load test passes but production still fell over at lower load — how? 2. Soak tests found what — concretely?

FOLLOW-UP ANSWERS: 1. The gap is always a dimension the test didn’t carry: production had the real tenant skew (one whale org), a dep-delayed batch job compounding load, and cache states aged differently. Each miss became a scenario parameter (skew on, background jobs on, cache-cold start) — the ledger treats “prod broke below test load” as a test-design bug, reviewed like code. Load testing is an approximation discipline; the humility loop is what makes it converge. 2. The classic pair: a slow leak in the notification retry path (off-heap via a native buffer not returned — heap graphs looked fine, container memory climbed; the soak’s 4-hour memory trend caught it), and unbounded queue growth in the webhook retry backlog (depth trended linearly under sustained 2× — the DLQ policy’s TTL was misconfigured). Both are invisible in 15-minute tests — that’s WHY the soak exists; nothing else catches time-as-a-variable.

RED FLAGS TO AVOID: - Uniform-request-mix tests (“we hit it with 10k RPS”) - Client-metrics-only reporting (no internal cliff attribution) - Load-testing prod casually (or never testing at all)

Q12-8: How did you tune the connection pool (HikariCP) — the full exercise

DIFFICULTY: Medium **CATEGORY TAG:** Performance / PostgreSQL

ANSWER: Covered structurally in Q6-5 — the tuning EXERCISE behind it, since interviewers ask for the process: **Step 1 — the DB’s total budget:** `max_connections=200` minus superuser/replication reserved (~10) minus known consumers (migrations, cron workers, analytics) = ~170 app-addressable; three app instances → per-instance ceiling ~55 before the DB itself is the constraint. **Step 2 — the workload’s shape:** `pg_stat_activity` during peak showed avg active SQL = 6-8 statements across the fleet at our request rate (OLTP: fast queries, high rate) — the DB can only execute ~4×cores truly-concurrent queries (16 on 4 vCPU); a 100-connection pool is 90 idle-or-queued connections pretending to be capacity. **Step 3 — the sweep:** load-test at pool sizes 10/15/25/40 per instance with k6 at burst

shape — p99 latencies: 15 → 41ms, 25 → 38ms, 40 → 63ms (thrash onset: context-switching + buffer churn), 10 → 58ms (queueing onset); 15-20 was the plateau — we shipped 15 fixed-size with the reasoning documented. **Step 4 — the guardrails:** connectionTimeout 3s (fail-fast; a pool-wait exception under overload is a CLEAN 503 via the breaker, not a 30s thread-park cascade — the timeout is a circuit breaker, not a nuisance), leak detection 60s (found two real leaks: an unclosed Stream in an export, and a @Transactional wrapping an LLM call — Q6-5), maxLifetime 14min (below the LB idle-kill), and metrics (pending/active/usage-time) with the interpretation table (pending>0 sustained = investigate QUERIES first, pool second — the pool is a queue in front of a shared resource).

The counterintuitive conclusion I defend: smaller pools win because Postgres executes a bounded number of queries truly-concurrently — every connection beyond the compute-curve adds scheduler thrash and buffer-cache pressure; the pool's job is FAIR QUEUEING, not parallelism (parallelism belongs to the DB's own execution). The one scenario that flips it: long-running analytical queries holding connections (the analytics route to the read replica with its OWN pool sizing — 25 on the replica where 3-minute queries are normal — separation of OLTP and analytics pool budgets, which also protects the OLTP p99 from a report's duration). The annual re-check: the sweep re-runs when workload shape shifts (a new chatty endpoint class, instance resize) — pool sizing is a function of the workload, and workloads drift.

KEY TERMS TO MENTION: - DB budget math (max_connections minus consumers) + workload concurrency reality (4×cores) - Empirical sweep with p99 results (15 optimal, thrash at 40, queueing at 10) - connectionTimeout-as-breaker + leak detection + fixed-size pool - OLTP/analytics pool separation (replica gets its own budget) - Annual re-check discipline (workload drift)

FOLLOW-UP QUESTIONS: 1. Pool exhaustion incident — walk me through the on-call diagnosis. 2. Why fixed-size (min=max) over autosizing pools?

FOLLOW-UP ANSWERS: 1. Symptom: 503 spike + connectionTimeout exceptions. Order: Hikari metrics (pending vs active — pending high + active at max = saturation), pg_stat_activity grouped by state (active-in-tx long-runners? idle-in-transaction? — the leak detector's stack traces usually name the holder first), traces for the endpoint burst (a batch job's deploy had doubled its connection hold time — the LLM-call-in-transaction bug class). Fix: the code fix + pool math unchanged. The diagnosis took 12 minutes BECAUSE the metrics panels existed pre-incident — that's the actual tuning outcome. 2. Autosizing pools (min < max) create connection-establishment churn at burst (new connections = TLS + auth + Postgres fork — tens of ms, right when you can least afford it) and unpredictably sized load on the DB (the DB budget math needs KNOWN consumer counts). Fixed pools make the DB-side arithmetic stable and the warm connections always present; the memory cost of idle connections is trivial vs the jitter. The pattern is standard for OLTP microservices; autosizing earns its keep on spiky analytics consumers.

RED FLAGS TO AVOID: - "Set it to 100, more is safer" (the thrash inversion) - No leak detection / no pending-metrics interpretation table - One shared pool for OLTP + analytics durations

Q12-9: How did you handle memory optimization in the JVM

DIFFICULTY: Hard **CATEGORY TAG:** Performance / JVM

ANSWER: Memory work started with a MAP, not a knob: heap (young/old), metaspace, code cache, thread stacks, direct/native buffers — the container contract (Q11-2) makes the SUM the constraint, so every optimization is ledger-aware. **The dominant fix — entity/context hygiene:** the batch import path held a million managed entities (the first-level cache as a liability, Q5-9) — `clear()` per chunk + SEQUENCE id generation turned 1.2GB peaks into 180MB; the second fix in the same class: a dashboard query loading FULL lead aggregates for counts (projection-ized to scalar SELECTs). **Heap sizing by evidence:** GC logs (`G1: -Xlog:gc*,gc+heap=debug`) fed a week of production shape — allocation rate ~800MB/s at peak, old-gen promotion modest, humongous allocations (>50% of region size) flagged: the 8MB JSON response buffer of one export endpoint was allocating humongous regions → streaming the export (chunked write to S3, zero heap accumulation) killed the humongous class entirely; `MaxRAMPercentage=75` with G1 gave pause targets met (p99 GC pause 38ms, young-heavy workload). **Native-side leaks:** the soak test (Q12-7) caught off-heap growth — Netty direct buffers from an un-released pooled response in a custom WebClient usage; fixed + `-XX:MaxDirectMemorySize` bounded + a container-memory-vs-heap dashboard gap panel (heap-used vs container-used divergence = native growth — the earliest leak signal, watched continuously).

The diagnostic toolkit (what I reach for, in order): container/heap gap panel (native vs heap attribution), GC log analysis (allocation rate, promotion rate, pause distribution, humongous counts), heap dumps on demand (`jcmd GC.heap_dump` — triggered by a leak alarm; MAT dominator tree named the retained entity cache in minutes), `jcmd VM.native_memory` (NMT — the direct-buffer attribution), and `pmap/proc` for the truly native. **The discipline that sustains it:** memory regression tests in the soak suite (heap + container trends under fixed load must be flat — leaks are TREND bugs, invisible in short tests), allocation profiling in the perf backlog (async-profiler in alloc mode on hot endpoints — the DTO explosion class caught at review), and the ledger review per container resize (Q11-4's cost work and memory work are the same meeting). The number that closes it: peak container memory went 3.4GB → 2.1GB at the same traffic — which ALSO let Q11-4 resize the fleet down 25%, making memory work and cost work the same project.

KEY TERMS TO MENTION: - The container memory map (heap + metaspace + code cache + stacks + direct buffers ≤ limit) - First-level-cache-as-liability fixes (`clear()`/projections) + humongous-allocation elimination (streaming exports) - Evidence-driven sizing: GC logs → `MaxRAMPercentage=75`, G1 pause targets met (38ms p99) - Native-leak toolkit: heap/container gap panel, NMT, MAT dominator trees - Soak-suite memory regression tests (leaks are trend bugs)

FOLLOW-UP QUESTIONS: 1. Walk me through reading a heap dump dominator tree for this class of bug. 2. G1 vs ZGC here — why G1?

FOLLOW-UP ANSWERS: 1. MAT opens the dump → dominator tree sorts by RETAINED size (what THIS object keeps alive exclusively) — the entity-cache bug showed the persistence context retaining 800MB via a single Session → map of entities; the path-to-GC-roots on the

dominator showed the batch loop holding it (no clear() call). The skill is reading RETAINED vs SHALLOW (shallow lies — the cache object is small, its retained tree is the leak) and GC-root paths (which live reference anchors the blob). Two dumps Δ-compare (grow-between-dumps) confirms the leak class vs one-off bloat. 2. G1: our allocation profile is young-heavy, modest live-set, pause target 200ms comfortably met — G1's region model + humongous handling gave the diagnostics (humongous counts) that drove real fixes. ZGC's sub-ms pauses matter for latency-critical tail workloads with BIG heaps (we're 2-3GB, 38ms p99 invisible next to 165ms API p95) and cost throughput + container-memory overhead. The revisit trigger is written: heap > 8GB or pause-pp99 > 100ms — ZGC conversation. Tuning follows the workload's numbers, not the conference-talk default.

RED FLAGS TO AVOID: - Knob-first answers (Xmx/collector) without the memory map - Heap-only thinking (native/direct buffer blindness — the modern leak class) - No trend-based regression testing (leaks found by customers)

Q12-10: What JVM tuning did you do for production

DIFFICULTY: Hard **CATEGORY TAG:** Performance / JVM

ANSWER: The full flag set, with the reasoning per flag — because reciting flags without WHY is the interview anti-pattern. **Memory:** `-XX:MaxRAMPercentage=75` (container contract: leaves 25% for metaspace+code+stacks+buffers — Q12-9's map; explicit Xmx rejected: it drifts from container limits on resize), `-XX:InitialRAMPercentage=75` (no resize churn at boot — pools pre-warm against final heap), Metaspace bounded (`-XX:MaxMetaspaceSize=320m` — CGLIB proxies + MapStruct + Groovy-free stack lands ~210m; the bound is OOM-early-vs-thrash-late insurance). **GC:** G1 with `-XX:MaxGCPauseMillis=200` (API p95 priority — 38ms actual p99 pause), `-XX:G1HeapRegionSize=4m` (right-sized after the humongous analysis: 8MB buffers became humongous at default region size; at 4m they're still humongous — the REAL fix was streaming, the region size just sharpens the signal), `-XX:+ParallelRefProcEnabled` (reference processing was 30% of a pause trace — the WebSoftReferences from caches), `-XX:+AlwaysPreTouch` (heap touched at boot — first-request latency spikes eliminated; pairs with the pre-warm philosophy). **GC observability:** `-Xlog:gc*:file=/var/log/gc.log:time,uptime,level,tags:filecount=5,filesize=20m` (the evidence stream for Q12-9's entire methodology). **OOM forensics:** `-XX:+HeapDumpOnOutOfMemoryError` `-XX:HeapDumpPath=/dumps` + `OnOutOfMemoryError` hook that alerts (a heap dump with a timestamp beats an OOM mystery every time). **Flight recording:** `-XX:StartFlightRecording=maxsize=200m,maxage=12h` settings kept available (JFR on-demand via diagnostic port — the async-profiler complement; always-on JFR at low overhead is defensible and we run it in staging).

What I explicitly did NOT tune and why — the senior-flavored closer: no `-XX:SurvivorRatio` / `-XX:TenuringThreshold` micro-knobs (G1 adapts these; hand-tuning without a pause-trace-backed reason is cargo cult that bites on workload shifts); no `-XX:+UseStringDeduplication` (our string profile is short-lived request data — dedup targets long-lived duplicate strings, measured benefit ~0); no explicit `-XX:ParallelGCThreads` math (defaults derive from cores correctly in our container sizing). The tuning process itself:

baseline → one hypothesis → one flag → measured delta (GC logs + endpoint p95) → keep/revert — a FLAGS EXPERIMENT, documented in the JVM-tuning ADR with the before/after graphs; the current stack is 6 deliberate flags, each with a graph behind it, and the ADR's "flags we tried and reverted" section is half the document — which is exactly what honest tuning looks like.

KEY TERMS TO MENTION: - Memory flags tied to the container contract (RAMPercentage, Metaspace bound) - GC: G1 + pause target + ParallelRefProcEnabled + AlwaysPreTouch — each with measured why - Forensics: HeapDumpOnOOM + GC logs as the evidence stream + JFR - Explicit NOT-tuned list with reasoning (anti-cargo-cult) - One-flag-at-a-time experiment discipline + ADR with reverted-flags section

FOLLOW-UP QUESTIONS: 1. A flag you shipped had to be reverted — the story? 2. How would your tuning change for a BATCH worker vs the API fleet?

FOLLOW-UP ANSWERS: 1. `-XX:G1NewSizePercent` lowering (chasing shorter young pauses): pauses dropped 12%, but promotion rate rose (young collections too frequent to age objects) → old-gen growth → mixed-GC frequency up → p99 WORSE over 2 hours (the tail regressed while the average improved — the classic trap). Reverted; the lesson (optimize pause distribution, not pause duration, without promotion telemetry) is in the ADR with both graphs. The revert cost was one config PR; the discipline made it cheap — that's the point of one-flag-at-a-time. 2. Different objective function: batch optimizes THROUGHPUT, not tail latency — larger heap share (85%), G1 pause target relaxed to 500ms (bigger young gens, fewer collections, more throughput), ParallelGC as a legitimate alternative for pure-batch (no tail to protect), and the humongous/allocation analysis matters MORE (batch is allocation-heavy — the SEQUENCE/clear() patterns from Q5-16 are JVM-visible). Same methodology, different objective — flags follow the SLO of the workload, which is the entire meta-lesson of this answer.

RED FLAGS TO AVOID: - Flag recitals without per-flag reasoning or graphs - Micro-knobs (SurvivorRatio) without trace evidence - No revert history (real tuning has failed experiments in it)

CATEGORY 13 — TESTING QUESTIONS

Q13-1: What is your testing strategy for AcquisitionOS

DIFFICULTY: Medium **CATEGORY TAG:** Testing

ANSWER: Four-layer testing pyramid: unit (most tests), slice (controller/service/repository in isolation), integration (full Spring context with Testcontainers), end-to-end (Playwright against staging). Code coverage 78% line, 64% branch.

1. Unit tests (JUnit 5 + Mockito + AssertJ): Pure unit tests of service methods. Services depend on repositories + other services, all mocked via `@Mock`. No Spring context (fast — 1000 tests in 10s). Example:

```
@ExtendWith(MockitoExtension.class)
class CreditServiceTest {
    @Mock CreditBalanceRepository balanceRepo;
    @Mock CreditLedgerRepository ledgerRepo;
    @InjectMocks CreditService service;

    @Test
    void deduct_throws_whenInsufficientCredits() {
        UUID userId = UUID.randomUUID();
        when(balanceRepo.findByIdForUpdate(userId))
            .thenReturn(Optional.of(new CreditBalance(userId, 5)));

        assertThatThrownBy(() -> service.deduct(userId, 10, "discover",
            UUID.randomUUID()))
            .isInstanceOf(InsufficientCreditsException.class);

        verify(ledgerRepo, never()).save(any());
    }
}
```

2. Slice tests (Spring Boot Test slices): - `@WebMvcTest(LeadController.class)` — loads only the web layer + MockMvc; service beans are `@MockBean`. Tests request mapping, validation, status codes, JSON shape. - `@DataJpaTest` — loads only JPA + a test DB (H2 by default, we use Testcontainers Postgres for prod parity). Tests repositories + `@Query` queries. - `@JsonTest` — tests Jackson serialisation/deserialisation of DTOs.

Example `@WebMvcTest` :

```

@WebMvcTest(LeadController.class)
@Import({GlobalExceptionHandler.class, SecurityConfig.class})
class LeadControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean LeadService leadService;

    @Test
    void getById_returns404_whenNotFound() throws Exception {
        when(leadService.findById(any(), any())).thenReturn(new
            NotFoundException("lead", "id"));

        mockMvc.perform(get("/api/leads/{id}",
            UUID.randomUUID()).header("Authorization", "Bearer test-jwt"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.error.code").value("NOT_FOUND"));
    }
}

```

3. Integration tests (@SpringBootTest + Testcontainers): Full Spring context with real Postgres (Testcontainers), real Redis (Testcontainers), real Kafka (Testcontainers), WireMock for external APIs (Stripe, Google, Twilio, LLM). Tests the full request flow: controller → service → repository → DB, with Kafka events, with real SQL. Slower (~8s per test) but most realistic.

```

@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers
class LeadDiscoveryIntegrationTest {
    @Container static PostgreSQLContainer<?> pg = new
        PostgreSQLContainer<>("postgres:15");
    @Container static GenericContainer<?> redis = new
        GenericContainer<>("redis:7").withExposedPorts(6379);
    @Container static KafkaContainer kafka = new KafkaContainer();

    @LocalServerPort int port;
    @Autowired TestRestTemplate restTemplate;

    @Test
    void discoverLead_endToEnd() {
        // signup → login → call /api/leads/discover → wait for async job → assert
        // lead in DB
    }
}

```

WireMock stubs the external APIs:

```

@dynamicPropertySource
static void wireProps(DynamicPropertyRegistry registry) {
    WireMockServer stripe = new WireMockServer();
    stripe.start();
    stripe.stubFor(post("/v1/checkout/sessions").willReturn(okJson("{...}")));
    registry.add("stripe.api.url", () -> stripe.baseUrl());
}

```

4. End-to-end tests (Playwright): ~50 tests against staging — signup, login (password + Magic Link + Google), discover leads, generate outreach, book meeting, view pipeline, upgrade plan. Run nightly + before every prod deploy. Catches integration bugs that unit + integration tests miss (e.g. frontend sends wrong field name).

5. Contract tests (WireMock + Pact): We record Stripe/Google/Twilio/LLM request/response pairs as WireMock stubs. On every test run, the stubs are replayed. If the real provider changes their API, we catch it via a weekly “contract verification” job that hits the real provider with a test request and compares to the stub.

6. Performance tests (Gatling): 1000 virtual users for 10 minutes, mixed workload (read 70%, write 20%, AI 5%, billing 5%). Run nightly + on PR to main. SLO: p95 < 500ms, 5xx < 0.1%. We have a baseline; PRs that regress by >10% fail CI.

7. Chaos tests: Weekly — kill an ECS task mid-batch, verify Spring Batch resumes; kill Kafka for 1 min, verify producers buffer + consumers catch up; kill Redis for 1 min, verify app degrades gracefully (cache miss → DB).

Test data management: Testcontainers for DB + Redis + Kafka (fresh per test class). For shared data (e.g. a “test user” that many tests use), we have `TestDataFactory` that creates entities in the DB at test setup. We don’t use production data in tests (privacy + consistency).

Coverage gate: SonarQube enforces 75% line coverage on new code. We’re at 78% line / 64% branch. We don’t enforce branch coverage (too strict for our domain — many branches are exception paths we don’t unit-test).

KEY TERMS TO MENTION: - Four-layer pyramid: unit (Mockito), slice (`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`), integration (`@SpringBootTest` + Testcontainers), E2E (Playwright against staging) - Unit: JUnit 5 + Mockito + AssertJ, no Spring context (1000 tests in 10s) - Slice: `@WebMvcTest` (web layer + MockMvc), `@DataJpaTest` (JPA + Testcontainers PG), `@JsonTest` - Integration: `@SpringBootTest` + Testcontainers (PG + Redis + Kafka) + WireMock for external APIs - E2E: Playwright against staging, nightly + before prod deploy - Contract: WireMock stubs for Stripe/Google/Twilio/LLM, weekly contract verification - Performance: Gatling 1000 vusers, SLO p95 < 500ms + 5xx < 0.1%, baseline regression - Chaos: weekly (kill task, kill Kafka, kill Redis) - Coverage: SonarQube 75% gate on new code (we’re at 78% line / 64% branch)

FOLLOW-UP QUESTIONS: 1. How do you test `@TransactionalEventListener`? 2. How do you test security configuration? 3. How do you handle flaky tests?

FOLLOW-UP ANSWERS: 1. Three ways: (a) **unit test the listener method directly** — call `listener.onApplicationEvent(event)` with a constructed event; assert the side effects (mock the dependencies); (b) `@SpringBootTest` + `ApplicationEventPublisher` — publish the event, assert the listener fired (via a `CountDownLatch` if async, or a `verify` on a mock); (c) **integration test the full flow** — trigger the event via the real code path (e.g. call the controller that publishes the event), assert the post-commit side effect. We use (a) for the listener’s logic, (c) for the wiring. 2. Three layers: (a) `@WebMvcTest` with `@Import(SecurityConfig.class)` + `@WithMockUser(roles = "ADMIN")` — tests endpoint security in isolation; (b) full integration test with a real JWT — signup → login → call protected

endpoint → assert 200; (c) security-specific tests — `ExpiredJwtTest`, `InvalidSignatureTest`, `TamperedPayloadTest` — tests `JwtAuthenticationFilter`'s handling of bad tokens. We have ~30 security-specific tests. 3. Three defenses: (a) **isolation** — Testcontainers gives a fresh DB per test class, no shared state; (b) **retry flaky** — we annotate flaky tests with `@Retry(maxAttempts = 3)` (only for genuinely flaky tests like timing-sensitive SSE); (c) **quarantine** — if a test is consistently flaky, we move it to a `@Quarantine` suite that runs weekly (not on every PR); the team has a TODO to fix it within 2 weeks. We have ~5 quarantined tests at any time.

RED FLAGS TO AVOID: - No testing strategy (instant fail) - 100% coverage target (impractical, encourages bad tests) - Tests that hit real external APIs (flaky + slow) - No integration tests (unit tests don't catch wiring bugs)

(The remaining testing questions — unit tests for services, integration tests with Spring Boot Test, testing external APIs with WireMock, code coverage targets, testing security, contract testing, test data management, testing async (`@Async`, `CompletableFuture`), testing Kafka, testing `@Scheduled`, testing Spring Batch, mutation testing (PIT), property-based testing, snapshot testing — follow the same template. Drill on: how `@MockBean` differs from `@Mock`, why you should avoid `@SpringBootTest` for unit tests, how to test a `@TransactionalEventListener(AFTER_COMMIT)` without a real DB.)

Q13-2: How did you write unit tests for services (JUnit 5 + Mockito patterns)

DIFFICULTY: Medium **CATEGORY TAG:** Testing

ANSWER: Unit tests target the SERVICE layer's decision logic — the orchestration, branching, policy, and state transitions — with all collaborators mocked. The house style, hardened by review: **one behavior per test, named as the requirement** (`deductCredits_insufficientBalance_throwsAndWritesNoLedger` — the name IS the spec; a failing test's name tells you the broken requirement); **Arrange-Act-Assert with strict structure** (given/when/then comments when non-obvious); **state-based assertions over interaction-counting** (assert the LEDGER ROW and returned result, not "verify(repo,times(1))" — mock-verification is reserved for genuinely unobservable side effects like "emailPort.send was called", where it's the only truth available; over-verified tests break on refactors that preserve behavior — the classic brittleness). **Mockito mechanics that matter:** `@ExtendWith(MockitoExtension.class)` (no PowerMock ever — PowerMock in a stack is a design smell telling you to extract a seam), `lenient()` scoped to setup-wide defaults with STRICT_STUBS default (unused stubs = dead code tests warn about), `ArgumentCaptor` for asserting the PAYLOAD sent to collaborators (the message content, not just "send happened"), and `@Nested` classes grouping by method under test.

The patterns beyond CRUD: **state-machine tests are table-driven** (`@ParameterizedTest` over (from, action, to, allowed?) tuples — the lead-stage machine's 40 transitions are 40

rows, not 40 test methods); **exception-path parity** (every thrown domain exception has a test asserting the mapping contract with the advice layer — error codes are API surface, Q8-4); **clock injection everywhere** (`Clock` bean — expiry, TTL, and business-day logic tested deterministically; `LocalDateTime.now()` in production code is a review-blocked pattern because it's untestable); **time-based tests via MutableClock** (advance the clock in-test — the lockout-expiry test doesn't sleep); and **the coverage honesty rule**: JacCo line coverage is a hygiene floor (85% service-layer), but the reviewed metric is BRANCH coverage on decision-heavy classes + mutation-testing spot checks (PIT on the credit/billing classes — a suite that survives 40% mutations is decoration; our mutation score on billing is 78% and the report gates merges there). What I deliberately do NOT unit test: getters, configuration wiring (integration tests own that), and repositories (Testcontainers integration tests own those — mocking JPA is testing your mock).

KEY TERMS TO MENTION: - Behavior-per-test naming; state over interaction assertions (captor for payloads) - Strict stubs; ArgumentCaptor; no PowerMock (seam-extraction smell) - Table-driven state-machine tests + exception-mapping parity tests - Clock injection (MutableClock) — time logic deterministically tested - Branch coverage + PIT mutation testing on money classes (78%)

FOLLOW-UP QUESTIONS: 1. How do you keep unit tests from breaking on every refactor? 2. When is a test testing the MOCK instead of the code?

FOLLOW-UP ANSWERS: 1. By asserting the CONTRACT, not the implementation: state changes, returned values, and emitted EVENTS survive refactors; call-sequences and internal private methods don't. The rule in review: if a pure refactor (same behavior, different structure) breaks tests, the tests were over-specified — fix the tests AND treat it as a signal the production code leaked internals. Interaction verification is quarantined to ports with unobservable effects. 2. Tell: the test passes when the PRODUCTION code is broken but the STUB matches the bug (the mock returns what the broken code expects). Structural defense: mocks return REALISTIC domain data from shared fixtures (not on-the-fly values that mirror the assertion), and the mutation testing on critical classes catches “tests pass with broken logic” systematically — mutation testing is the antivirus for mock-testing's blind spot.

RED FLAGS TO AVOID: - `verify(repo, times(1))` everywhere (brittle interaction theater) - `LocalDateTime.now()` in tested code paths - Coverage % as the goal with no mutation/branch honesty

Q13-3: How did you implement integration tests (Spring Boot Test + Testcontainers)

DIFFICULTY: Medium **CATEGORY TAG:** Testing

ANSWER: Integration tests prove the WIRING and the REAL-DEPENDENCY contracts the unit layer can't: HTTP layer (routing, filters, serialization, error mapping), persistence (actual SQL, migrations, constraints), cache behavior (actual Redis semantics — TTLs, Lua atomicity), and config binding. The stack: `@SpringBootTest(webEnvironment = RANDOM_PORT)` + **Testcontainers** (Postgres 15, Redis 7 — real engines, not H2/embedded fakes: H2's SQL dialect divergence

masked two real bugs historically — a JSONB operator and a FOR UPDATE SKIP LOCKED behavior — after which embedded databases were banned for anything testing SQL), with containers as SINGLETONS per JVM (`static` containers + reuse) keeping the suite under 4 minutes for 200+ integration tests; Flyway runs the REAL migration chain (every integration boot validates the schema path — the CI catches migration breakage at PR time, Q5-11's fresh+upgrade runs). **Repository tests:** `@DataJpaTest` slices with Testcontainers + the query-budget extension (an interceptor counting statements per test — the N+1 regression guard asserting list endpoints stay $\leq$ their budget); **API tests:** MockMvc or TestRestTemplate with the security context setup (`@WithMockUser(roles)` / real JWT flows for the auth suite) covering the security-property tests (401/403/404-cross-tenant — Q4-7's suite).

Data management: per-test transactions with rollback (`@Transactional` on tests) for speed, EXCEPT where transactions hide behavior (async listeners, AFTER_COMMIT — those tests use explicit commit + cleanup), and the fixture philosophy: builder-pattern test objects (`aLead().withOrg(o).withStatus(HOT).build()` — defaults centralized, variance explicit) replacing SQL dumps; no test depends on test ORDER (ForkJoin random order in CI catches order-dependence). **The external-integration boundary:** WireMock stubs providers (Q13-4) INSIDE integration tests for failure-injection (a 500 from Stripe mid-checkout — the full resilience stack exercised), with contract-verification tests separately pinning our REQUESTS to provider contracts (Q13-7). The pass/fail discipline: integration failures block merge like unit failures (no flake-tolerance — the retry-once-then-fail CI policy with auto-filed flake tickets at $> 2\%$), and the suite runs the PR pipeline in sharded parallel (Testcontainers per shard) keeping the 9-minute gate honest.

KEY TERMS TO MENTION: - Testcontainers real engines; H2/embedded banned (dialect lies) - Singleton containers + Flyway real-chain validation per boot - Query-budget extension (N+1 regression guard) + security-property suite - Builder fixtures + order-independence + `@Transactional`-with-caveats (AFTER_COMMIT) - Flake policy: retry-once, auto-ticket at 2%

FOLLOW-UP QUESTIONS: 1. Why singleton containers — what breaks with per-test containers? 2. When do you use explicit commit in an integration test?

FOLLOW-UP ANSWERS: 1. Per-test = $\sim 10s$ container spin-up $\times$ 200 tests = 30+ minutes of pure Docker wait (the CI budget killer); singletons with per-test data cleanup give engine-realism at unit-test-ish cost. What breaks: test isolation depends on CLEANUP discipline (rollback + explicit deletes for commit-tests) — a leaked fixture pollutes downstream tests, which is why the order-random run exists: it surfaces leakage. Ryuk/container reuse policies are pinned (the dangling-container CI cleanup job exists — Testcontainers hygiene is real ops). 2. When the BEHAVIOR is the commit: AFTER_COMMIT listeners (Q3-11) never fire in a rolled-back test — the transaction is the mechanism under test. Pattern: commit the arrange+act, assert the listener's effects, then cleanup via the builder-fixture delete helper; a comment marks WHY the test commits (the next engineer shouldn't "fix" it back to rollback and silently untest the feature).

RED FLAGS TO AVOID: - H2 "integration tests" (dialect theater) - Test-order dependence / fixture leakage - Mocking the repository in integration tests (unit-test cosplay)

Q13-4: How did you test external API integrations (WireMock)

DIFFICULTY: Medium **CATEGORY TAG:** Testing

ANSWER: External providers are tested at two boundaries: OUR REQUESTS against THEIR CONTRACT (consumer tests) and OUR HANDLING of their responses (WireMock scenarios).

WireMock layer: each provider (Stripe, Twilio, Google, LLM gateways) gets a stub server in the integration context with scenario-based stubs: success paths (200 with REAL response captures — recorded from sandbox, structure-verified), the FULL failure ladder (429 with Retry-After → assert backoff+retry count; 401 → assert NO retry + NEEDS_REAUTH marking; 500×3 → assert breaker OPEN + fallback engaged; timeout — WireMock’s fixed-delay — assert the false-negative path + idempotency key present; malformed body → assert parse-failure path), and STATEFUL scenarios (/__admin/scenarios — the “first call fails, retry succeeds” sequence the retry tests need). Assertions go past status codes: `verify(postRequestedFor(urlEqualTo("/v1/charges")).withRequestBody(matchingJsonPath("$.idempotency_k` — the idempotency-key presence on side-effecting calls is a TESTED CONTRACT, not a convention (Q9-10’s lesson, enforced in CI).

The response-capture pipeline keeps stubs honest: sandbox traffic responses are captured, schema-validated against our DTO expectations, and versioned in the repo — a stub lying about provider shape tests our fantasy, not reality (the capture-and-verify step is the difference between mock theater and contract approximation); provider contract drift is caught by the weekly sandbox contract-refresh job (re-capture, diff — a changed Stripe response shape fails the refresh check and flags the adapter for review). **Resilience assertions** mirror Q3-12’s stack: each adapter test asserts the Resilience4j config AS BEHAVIOR (attempt counts observed at WireMock, backoff timing bounded, breaker state transitions read from the actuator endpoint), and the fallback path asserts the PRODUCT semantics (template-generated email marked AI_FALLBACK, credits not deducted for fallback — Q3-12’s rules as tests). The LLM gateway gets an extra class: response-content fuzzing (well-formed-but-wrong-content responses — the validator layer’s rejections tested: JSON-parse failures, schema violations, injected-prompt-marker content) because LLM providers fail DIFFERENTLY than CRUD APIs, and the enrichment pipeline’s degradation is the behavior under test.

KEY TERMS TO MENTION: - WireMock scenario statefulness (fail-then-succeed ladders) + fixed-delay timeouts - Real response captures, schema-verified, versioned (no fantasy stubs) - Weekly sandbox contract-refresh job (drift detection) - Idempotency-key presence as a tested request contract - Resilience behavior assertions (attempt counts, breaker states, fallback semantics)

FOLLOW-UP QUESTIONS: 1. WireMock says pass, production says the provider changed — what caught it? 2. How do you test the webhook RECEIVING side as thoroughly?

FOLLOW-UP ANSWERS: 1. The weekly contract-refresh (sandbox re-capture + diff) is the drift alarm — it fired once on a Stripe error-body shape change that would have broken our 4xx translation (Q8-4’s table); the adapter fix shipped before any customer hit it. Production canaries complete the net: synthetic calls per provider every 5 min (the integration-health dashboard, Q3-18) — drift between refresh cycles surfaces there in minutes, not in customer

tickets. 2. The mirror image: WE send Stripe-signed webhooks to OURSELF — a test harness computes valid signatures (Stripe’s algorithm with a test secret) and posts the FULL webhook ladder: valid, tampered payload (signature mismatch → 400), stale timestamp (tolerance window → 400), replayed event (dedupe → no-op), unknown type (persist + UNHANDLED), and the crash-after-ack simulation (handler throws → freshness alarm + reconciliation sweep picks it up — Q9-8’s war story as a permanent test). The webhook pipeline is money-adjacent; it gets the same ladder discipline as the outbound adapters.

RED FLAGS TO AVOID: - Hand-invented stub responses (fantasy contracts) - Status-code-only assertions (no request-contract or resilience checks) - No drift-detection mechanism (stubs age into lies)

Q13-5: How did you achieve code coverage targets (and what do you actually think of them)

DIFFICULTY: Easy **CATEGORY TAG:** Testing

ANSWER: The honest answer first: coverage is a HYGIENE metric, not a quality metric — 85% line coverage with weak assertions is worse than 70% with meaningful ones, because it launders untested code with green badges. Our actual targets and mechanics: **service layer line coverage ≥ 85%** (JacCo in the build, CI-gated per-module — a coverage DROP in a PR fails with the diff report attached, making “what did you not test” a review question), **branch coverage ≥ 70% on decision-heavy classes** (the money/auth/state-machine packages — line coverage lies in branchy code: one test can hit both sides of an || and claim 100% of the line), **mutation score ≥ 70% on the critical packages** (PIT on billing/auth/credit classes — the metric that measures assertion STRENGTH; survived-mutants are listed in the PR report as “behavior changes your tests didn’t notice”), and **exemptions are explicit:** generated code (MapStruct), config classes, DTOs — listed in the JacCo excludes with a comment each; unlisted exclusions are review-blocked (the coverage-gaming pattern — excluding the hard stuff — is the actual failure mode of coverage programs).

What the numbers bought us, concretely: the mutation testing found a real gap in the credit-refund path (tests covered “refund happens” but a mutant flipping the sign of the refund AMOUNT survived — the assertion checked existence, not value; the fix tightened assertions across the ledger tests). And what they DIDN’T buy: the 15% uncovered in services is the integration-glue territory (happy-path orchestration with thin logic) — where integration tests already cover the behavior, and double-covering adds maintenance without risk reduction; the coverage report is reviewed WITH that lens (the “covered-elsewhere” annotation convention in the PR description). The trend rule: coverage ratchets UP per module (the floor is max(floor, current-2%) — a module at 91% can’t slide to 80% unnoticed), because coverage programs die by erosion, not by decree.

KEY TERMS TO MENTION: - Hygiene-vs-quality framing; per-layer targets (85 line / 70 branch / 70 mutation on criticals) - PR-level coverage-diff gating + explicit annotated exclusions - PIT mutation testing as the assertion-strength metric (the refund-sign war story)

- Covered-elsewhere convention (honest double-coverage avoidance) - Ratchet rule ($\max(\text{floor}, \text{current}-2\%)$) against erosion

FOLLOW-UP QUESTIONS: 1. A teammate writes 200 trivial tests to hit the gate — what's your response? 2. Where does coverage mislead most?

FOLLOW-UP ANSWERS: 1. The gate is per-module coverage-diff, so trivial tests DO raise the number — the counter-pressure is review (assertion-quality checklist: does the test fail if the behavior breaks? — the mutation mindset as a human check) and the mutation score on critical packages (trivial tests die under PIT, exposing the gap). Structurally: the coverage gate exists to force the CONVERSATION (what's untested and why), not to certify quality — I'd rather have the 200-test PR rejected in review with coaching than have no gate and silent gaps. 2. On branchy code with side effects: 100% line coverage of a state machine where tests never exercise the illegal-transition path (the @ParameterizedTest table exists precisely because lines are shared, branches differ); and on exception paths generally — coverage counts the try block as covered when the exception never fired. The branch + mutation combo exists to cover exactly these blind spots; line coverage alone is a vanity metric in any non-trivial codebase.

RED FLAGS TO AVOID: - Coverage % as the headline ("we have 92%!" with no mutation/branch story) - Silent exclusions gaming the gate - No ratchet (coverage decays the moment attention moves)

Q13-6: How did you test security configurations

DIFFICULTY: Medium **CATEGORY TAG:** Testing / Security

ANSWER: Security tests assert the PROPERTY (this access is denied/allowed), not the config text — the config can be wrong while looking right, so the tests treat the running filter chain as the unit under test. **The property suite** (every endpoint × auth-state matrix, generated): unauthenticated → 401 (except the public allow-list); wrong-role → 403; cross-tenant resource → 404 (the enumeration rule, Q4-6); expired JWT → 401; revoked-session refresh → 401 with REFRESH_REUSED semantics (Q4-9); tampered signature → 401 — generated as a table over the OpenAPI path list so NEW endpoints are automatically in scope (an endpoint without security expectations fails the generation check — coverage by construction, the security analog of Q13-5's ratchet). **The public-surface pin:** the permit-all list is asserted EXACTLY (Q4-7's chain-smoke test) — a regex typo adding `/api/admin/**` to public fails the build; this test caught the trailing-slash case for real.

Injection/XSS classes: parameterized suites running payloads (SQLi strings through every repository path — asserting JPA parameterization at the integration level; XSS payloads through every stored-text field — asserting encode-on-render client-side + content-type/Disposition server-side per Q3-15; the upload allow-list tested with magic-byte-forgeries: a .jpg-named HTML file MUST be rejected or sandboxed). **Crypto/auth mechanics:** BCrypt round-trip + known-vector tests; JWT forgery attempts (alg-none, alg-confusion HS/RS swap, kid-injection — the JOSE failure modes as tests against OUR verifier's config); the OTP/lockout ladder (5 failures → locked, generic-error assertions — no existence oracle, Q4-11)

with the Redis TTL behavior via the MutableClock. **Dependency/config scanning:** OWASP ZAP baseline scan in CI weekly against staging (passive alerts gate on severity), trivy for dependency CVEs (Q11-2), and the secret-scanning trio (gitleaks). The meta-rule: security tests FAIL THE BUILD like any test — security findings in a backlog rot; findings in CI gate. The one gap I name honestly: business-logic authorization (the policy objects) is tested by unit tests per policy (Q13-2's patterns) — the MATRIX-level “no combination grants unintended access” is only as good as the enumerated combinations; the generated matrix from the permission table (Q4-6) is the mechanism, and its completeness is a design property, not a proof.

KEY TERMS TO MENTION: - Property-based suite generated from the OpenAPI path list (new endpoints auto-covered) - Public-surface pin (chain smoke) — caught a real trailing-slash hole - Injection suites: SQLi/XSS/magic-byte forgery as parameterized tables - JWT failure-mode tests: alg-none, alg-confusion, kid-injection - ZAP baseline + trivy + gitleaks in CI; findings gate, never backlog

FOLLOW-UP QUESTIONS: 1. How do you test RATE LIMITING without slowing the suite? 2. A security test is flaky (Redis TTL timing) — how do you fix it honestly?

FOLLOW-UP ANSWERS: 1. The limiter's ATOMICITY is unit-tested against an embedded Redis (or testcontainer with short windows — the Lua scripts are the code under test); the HTTP-layer tests assert the CONTRACT (429 + Retry-After + headers) using an overridable limiter facade (test profile injects a deterministic limiter) — testing the wiring without burning real time windows. The full ladder (10 requests → 429) runs once in a dedicated tagged suite, not per-PR. 2. Flaky security tests get FIXED, not retried past — the honest fix here: inject the Clock (Q13-2's MutableClock) into the lockout logic so expiry is deterministic (no sleeping to TTL), and the Redis testcontainer gets a fixed time source where the code supports it (or the test manipulates the TTL key directly via the test Redis client — asserting the SEMANTIC “locked until T” not the wall-clock behavior). A retried-past security test is a hole wearing a green check — the flake policy (Q13-3) treats security-flakes as P1 test bugs.

RED FLAGS TO AVOID: - Testing config FILES instead of running behavior - Security findings in backlogs (must gate CI) - No mechanism for new-endpoint coverage (manual security test rot)

Q13-7: How did you implement contract testing

DIFFICULTY: Hard **CATEGORY TAG:** Testing / Architecture

ANSWER: Contract tests exist wherever two independently-deployable things meet: our API ↔ frontend, our adapters ↔ external providers, our services ↔ each other (post-extraction).

Consumer-driven contracts (API ↔ frontend): each frontend app maintains a contract pack (Pact-style) — expected interactions for its user journeys (login, lead-list paging, deal-advance); the pack runs against the PROVIDER in CI (the provider replays each consumer's expectations against the running service — a breaking change fails the build NAMING the consumer and journey); verification results publish to a broker-ish registry (a versioned store) — the “can-I-deploy” check gates frontend deploys on their pack's latest verification

against the release candidate. The design choice worth defending: contracts cover JOURNEYS not endpoints (an endpoint's contract with nobody is dead weight; a journey's contract is what actually breaks users) — 23 journeys cover ~140 endpoint-interactions with far less maintenance than endpoint-exhaustive contracts.

Provider-side contracts (us ↔ Stripe/Gmail/Twilio): the adapter tests pin OUR request shape to THEIR documented contract (WireMock stubs generated from their OpenAPI/spec artifacts where available, hand-pinned where not) + the weekly sandbox re-capture diff (Q13-4) as the live-drift net — consumer-side contract testing against providers we don't control is fundamentally drift detection, and the mechanism reflects that honestly. **Internal service contracts (post-extraction):** the event schemas ARE contracts — versioned JSON schemas per event type with a compatibility suite (new schema versions must be backward-compatible per the registered policy — additive-only; a breaking event change requires a version bump + dual-publish window). The failure the mechanism caught that unit tests couldn't: a frontend expected `stage` as an enum string; a backend refactor changed it to an int silently — JSON serialization kept 200s green, unit tests tested the service in isolation, and the consumer pack failed in CI within minutes with the diff visible. That's the entire value proposition: integration reality, caught at PR time, attributed to the exact consumer.

KEY TERMS TO MENTION: - Consumer-driven packs per frontend app; CI replay naming consumer+journey - can-I-deploy gating on verification registry - Journey-level (not endpoint-level) contract scope — the maintenance trade - Provider contracts = drift detection (spec-generated stubs + sandbox re-capture) - Event schemas as versioned contracts with compatibility suites

FOLLOW-UP QUESTIONS: 1. Who owns a failing contract — the provider or consumer team? 2. Contracts for the webhooks we SEND (push side)?

FOLLOW-UP ANSWERS: 1. The breaking side owns the fix: CI names the consumer and journey, and the rule (written in the API guidelines) is the provider either restores compatibility or executes the versioning protocol (add + deprecate + sunset — Q8-10) WITH the consumer's migration note. Internal friction is low because the consumer is one PR away (the coordinated-refactor lane, Q8-10's follow-up); the mechanism exists precisely for the day the two sides DON'T share a repo or a deploy train. 2. Symmetric mechanism: our webhook payload schemas are versioned + published (Q8-6's webhook spec section); a "self-consumer" test suite subscribes to a staging webhook endpoint and validates EVERY emitted event against the published schema (schema-drift gate), plus replay-verification for consumers (the dashboard's test-fire sends a canonical example set). Push-side contracts are stricter (no 404 feedback — Q8-2's note), so the schema gate carries more weight than on the request side.

RED FLAGS TO AVOID: - Endpoint-exhaustive contracts (maintenance death) or no contracts at all - Provider-side stubs with no drift mechanism (fantasy APIs) - Event schema changes without compatibility gates

Q13-8: How did you handle test data management

DIFFICULTY: Medium **CATEGORY TAG:** Testing

ANSWER: Test data strategy by LAYER, because each layer's needs differ. **Unit tests:** in-memory builders (`aLead().withOrg(...).withStatus(HOT).build()`) — the builder catalog centralizes valid-domain defaults so a schema change breaks ONE file, not 400 tests; builders construct VALID objects by default — the “defaults are always-insertable” invariant, tested itself) with shared factory modules per context. **Integration tests:** builders + per-test transactional rollback (Q13-3), with cleanup-only paths for commit-tests; the fixture SEED set (reference data: plans, credit costs, pipeline stages) ships as a Flyway TEST-seed migration — idempotent, versioned with the schema, so integration boots are self-contained (no external seed order dependence). **E2E/staging:** the seeder service (`make seed`) builds realistic estates — 3 orgs × users × 200 leads with DISTRIBUTED shapes (the skew matters: a whale org for the perf tests, Q12-5) — deterministic via seeded RNG (the same seed → the same estate; flaky data is a test bug class we eliminated by construction) and tagged (e2e-owned rows carry a marker column for teardown).

Production-derived data for the performance/QA staging: the SAFE path — a sanitized snapshot transform (a job that clones prod schema + PII-scrubs: names → faker-consistent fakes PRESERVING distribution shape, emails → deterministic fakes, tokens/keys nulled, financial figures jittered $\pm 10\%$ keeping statistics) — because prod-shaped DATA finds prod-shaped bugs (the Q12-5 whale-org lesson) while PII never crosses environments (the transform's scrub coverage is itself tested: a canary PII marker planted in prod-shaped fixtures MUST be absent post-transform — the scrubber tests the scrubber). Access discipline: staging is NOT a free-for-all PII zone even post-scrub (the scrub is for leak safety, access is still role-gated); and the GDPR angle: prod data never leaves the prod region unencrypted, transforms run IN-region, outputs encrypted at rest (Q6-9's backup-tier rules apply). The maintenance reality I own: builders drift from schema (a new NOT NULL column breaks builders — that's GOOD, compile/test-time discovery, the drift alarm working as intended), and the seed migrations get the same review as schema migrations (test data IS data).

KEY TERMS TO MENTION: - Builder catalog with valid-by-default invariants (one-file schema-change blast radius) - Test-seed Flyway migrations (self-contained boots) + deterministic seeded-RNG estates - Prod-shaped sanitized snapshots (faker-preserving distributions) + the scrubber-tests-itself canary - E2E tagging for teardown + order-independence - In-region transform + encrypted outputs (GDPR discipline for test data)

FOLLOW-UP QUESTIONS: 1. Why faker-consistent fakes over pure random garbage? 2. A staging incident leaks “scrubbed” data that wasn't — how did your controls respond?

FOLLOW-UP ANSWERS: 1. Garbage data hides data-SHAPE bugs: name lengths, email formats, date distributions, and the tenant skew all drive real behavior (indexes, caches, UI rendering, plan-flip plans — Q12-5 again). Fakers preserve the SHAPE (a 40-char company name, a 2-of-1000 conversion funnel) while removing the IDENTITY — the tests see production physics without production persons. Random garbage tests CRUD; shaped fakes test the SYSTEM. 2. The controls would fire in sequence: the canary-marker check is a

BLOCKING CI/staging-gate step (the “scrubbed” snapshot cannot be deployed if the canary survives — so the leak means the gate was bypassed or the transform changed post-gate), the access audit (who pulled the snapshot, from where — the artifact store logs), rotate/kill the artifact, and the postmortem lands on the PROCESS gap (the bypass path) with the transform’s coverage tests extended to the miss. The design assumption is that scrubbing WILL eventually fail once — controls assume failure and catch it at the gate, not in the wild.

RED FLAGS TO AVOID: - 400 test files with hand-built objects (schema-change explosions)
- Prod data copied raw to staging “just for the demo” - Random-data e2e suites (flaky by construction)

CATEGORY 14 — BEHAVIORAL AND SITUATION QUESTIONS

Q14-1: Describe a technical challenge you faced and how you solved it

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

ANSWER: Situation: Two months ago, Stripe started retrying our webhook endpoint aggressively — 600 webhooks in 2 minutes — and our server returned 500 for all of them, causing Stripe to retry again, creating a feedback loop. Within 5 minutes, our webhook handler was processing 0 webhooks (all returning 500), and paid users weren't getting their subscriptions activated. Customer support got 12 tickets in 30 minutes.

Task: I was on-call. I needed to stop the 500 storm within minutes (to stop the user impact), identify the root cause (to prevent recurrence), and ship a fix (to permanently resolve).

Action: Three steps: 1. **Triage (5 min):** I logged into the AWS console, saw the ECS task CPU at 95% + HikariCP at 100% utilisation + 100% 5xx on `/api/payments/webhook/stripe`. I checked CloudWatch Logs — the handler was throwing `CannotAcquireResourceException` from HikariCP. Diagnosis: connection pool exhaustion. 2. **Mitigate (10 min):** I scaled the ECS service to 8 tasks (from 4) — more tasks = more HikariCP pools = more connections. 5xx dropped to 30%. Not a fix but bought time. I also increased HikariCP `maximum-pool-size` from 20 to 30 per task via a config update (rolled out via a blue/green deploy in 5 min). 3. **Root cause + fix (next 4 hours):** I pulled the slowest webhook from the DB — it was a `checkout.session.completed` event whose processing took 28 seconds (the user's subscription activation involved a Stripe API call to fetch the customer's email, and Stripe was slow that minute). The handler held the DB connection for the full 28s, HikariCP exhausted. The fix: split the handler into "ingest" (just persist the raw webhook, return 200 to Stripe immediately) + "process" (async via Kafka consumer, per-event transaction). The Stripe API call now happens in the consumer, not in the request handler — a slow Stripe API call doesn't hold a DB connection. We also added a Resilience4j time limiter (5s) on the Stripe API call in the consumer.

Result: After the fix, the webhook handler returns 200 in <50ms (just an insert + Kafka send). The consumer processes events in <500ms (with the time limiter). HikariCP utilisation dropped to 30%. We added a load test that fires 1000 concurrent webhooks as a regression test — runs nightly, would have caught this in 5 minutes if we'd had it before. Postmortem published to the team wiki; we applied the "ingest + async process" pattern to all webhook handlers (Gmail, Twilio). No recurrence in 2 months.

Reflection: The root cause was an architecture weakness (synchronous processing of external retries) that I should have caught in code review when the webhook handler was first written. I added a checklist item to code reviews: "If this handler is called by an external service that retries on non-2xx, ensure it returns 2xx within 5s — process async if needed."

KEY TERMS TO MENTION: - STAR: Situation (Stripe webhook storm), Task (stop 500s in minutes), Action (triage → mitigate → fix), Result (50ms p95, no recurrence) - Specific numbers: 600 webhooks in 2 min, 12 support tickets, 28s slow webhook, 100% 5xx → 0% - Triage tooling: AWS console, CloudWatch Logs, HikariCP `CannotAcquireResourceException` - Mitigation: scale ECS to 8 tasks, increase HikariCP pool to 30 - Fix: split handler into ingest + async Kafka consumer, Resilience4j time limiter - Prevention: load test (1000 concurrent webhooks), “ingest + async process” pattern, postmortem, code review checklist

FOLLOW-UP QUESTIONS: 1. What did you learn from this? 2. How did you communicate with the team during the incident? 3. What if the fix had not worked?

FOLLOW-UP ANSWERS: 1. Two lessons: (a) “decouple from external retry pressure” — every webhook handler should return 2xx in <5s and process async, regardless of how fast the processing is “in normal cases”; (b) “load test the failure paths” — we had a load test for normal webhook flow but not for the “external API slow” case. We now load test with WireMock stubs that introduce latency. 2. We have a `#incidents` Slack channel. I posted a brief update every 10 minutes during the incident: “00:00: detected, triaging. 00:05: scaled to 8 tasks, 5xx at 30%. 00:15: rolling out HikariCP config update. 00:20: 5xx at 0%, monitoring. 00:30: writing postmortem.” The PM + customer support were in the channel — they used the updates to communicate with the affected users. After the incident, the postmortem went to the team wiki + a 30-min team retro. 3. I had a rollback plan: if the fix didn’t reduce 5xx, I’d revert to the previous code (the synchronous handler) + increase HikariCP pool to 50 + add a per-IP rate limit on the webhook endpoint (return 429 if >100 req/min from one Stripe IP — Stripe would retry, but at a slower rate). The rollback would have stopped the 5xx storm within 10 min. The fix worked, so I didn’t need to roll back, but I had the plan ready.

RED FLAGS TO AVOID: - No specific numbers (looks invented) - “I called my senior” (you ARE the senior) - “We didn’t have a postmortem” (looks immature) - “It was someone else’s bug” (even if true, you handled it)

Q14-2: Tell me about a difficult technical decision you made

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

ANSWER: Situation: Three months in, we had to decide between using Kafka or sticking with Spring `ApplicationEventPublisher` for our internal pub/sub. The other backend engineer wanted Kafka (“future-proof”); I argued for Spring Events with Kafka-shaped events (“don’t add operational cost before you need it”).

Task: We had a 2-hour architecture meeting to decide. I needed to either convince him we didn’t need Kafka yet, or be convinced we did, or find a middle ground. The decision would affect the entire codebase.

Action: I did three things: 1. **Quantify the cost of Kafka** — operational (broker to operate, schema registry, consumer lag monitoring, on-call), code (Kafka producer + consumer config, error handling, DLQ), latency (network round-trip vs in-process). At our scale (~50

events/sec peak), Kafka's overhead exceeded its benefit. I wrote a 1-page ADR with the numbers. 2. **Define the decision rule** — "Use Spring Events until a context publishes >1K events/sec or needs to be extracted to its own service". We're at 50 events/sec, nowhere near 1K. The rule is explicit and re-evaluated quarterly. 3. **Find the middle ground** — we dual-publish the events that **will** cross a service boundary (`audit-events` , `outreach-events` , `reply-events`) to both `ApplicationEventPublisher` (consumed in-process today) and Kafka (consumed by a shadow consumer today, ready to extract later). When we extract `ai` or `outreach` into its own service, we swap the in-process listener for the Kafka consumer with zero contract change.

Result: The other engineer agreed. We went with Spring Events + dual-publish for the 3 events that will eventually be cross-service. The ADR is in our wiki, re-evaluated quarterly. We saved ourselves 1-2 months of Kafka operational setup + ongoing monitoring. The decision is reversible (we can extract services with no contract change).

Reflection: The hardest part was **not** letting "future-proofing" win without quantifying the cost. Engineers (including me) tend to over-engineer for hypothetical futures. The discipline is to ask: "What's the **current** cost of NOT having X? What's the cost of having X today? When does the math flip?" For us, the math flips at 1K events/sec — well below that, Kafka is premature.

KEY TERMS TO MENTION: - STAR: Situation (Kafka vs Spring Events), Task (decide in 2h meeting), Action (quantify cost, define decision rule, find middle ground), Result (Spring Events + dual-publish, ADR, saved 1-2 months) - Quantified cost: broker ops, schema registry, consumer lag, network latency vs in-process - Decision rule: "Use Spring Events until >1K events/sec or service extraction needed" - Middle ground: dual-publish for cross-service-ready events - ADR (Architecture Decision Record) — quarterly re-evaluation - Reversible decision (zero contract change on extraction)

FOLLOW-UP QUESTIONS: 1. How do you handle disagreements with peers? 2. What if your decision turned out wrong? 3. What's the next decision you're evaluating?

FOLLOW-UP ANSWERS: 1. Two principles: (a) **quantify the trade-off** — write an ADR with numbers, not just opinions. Numbers settle most disagreements. (b) **find the middle ground** — most technical decisions aren't binary; there's usually a hybrid (we did dual-publish). When there's no middle ground, the senior engineer (me) makes the call, with a documented ADR + re-evaluation criteria. We've had 3 disagreements in 9 months — all resolved this way, no resentment. 2. I'd revisit the ADR. We re-evaluate quarterly — if the decision rule ("Spring Events until >1K events/sec") becomes wrong (e.g. we hit 1K events/sec in a context), we extract that context to a service with Kafka. The decision is **designed** to be reversible. The cost of being wrong: a small migration (swap in-process listener for Kafka consumer — 1 day per context). I'd rather be wrong on a reversible decision than be paralyzed by fear of being wrong. 3. Event sourcing the credit ledger (Q1-7). Today, `credit_balance` is a mutable row + append-only `credit_ledger`. The "true" answer is event-sourced: ledger is the source of truth, balance is a projection. The cost: 2 weeks of refactor + risk of bugs during the migration. The benefit: scales better at 100K users + audit + replay. We're evaluating quarterly — likely Q2 2027.

RED FLAGS TO AVOID: - “I told him he was wrong” (no respect for peers) - “We just used Kafka” (no decision process) - “I was right, he was wrong” (every disagreement has trade-offs) - Not having an ADR (looks undisciplined)

(The remaining behavioral questions — production incident, improving a feature’s performance, learning a new tech quickly, handling tech debt, code review process, feature you’re proud of, disagreements on tech decisions, what you’d add with more time — follow the same template with STAR format. Drill on: the specific* mistake you made (not “we” — own it), the moment you realised the issue, the conversation with the PM/stakeholder, what you’d do differently next time. Interviewers at Uber/Meta especially look for: (a) ownership (“I” not “we”), (b) specific numbers, (c) reflection on what you’d do differently, (d) honesty about mistakes.)*

Q14-3: Tell me about a time you handled a production incident (STAR)

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

SITUATION: A Tuesday at 14:20: support pings that “customers say outreach emails are sending twice.” Within 10 minutes our own dashboard confirms it — the duplicate-send signature is visible: the same `(lead_id, template)` pairs with timestamps minutes apart. This is the Q2-15 incident from the technical side; here’s the HUMAN side of the same story. Scale: ~900 duplicate emails across ~2 hours, affecting ~140 orgs — small in count, maximal in trust damage (outreach is the customer’s voice to THEIR customers; a duplicate makes THEM look sloppy to a lead they were courting).

TASK: As the senior backend engineer on call, I owned incident command: contain the damage, find the root cause, fix it, and rebuild trust — in that order, with the clock running.

ACTION: Minutes 0-10 — contain: flipped the outreach dispatcher pause flag (one command, reversible — the pre-built containment paid off), posted to the incident channel, declared IC. **Minutes 10-40 — scope:** queried the duplicate signature — clustered on one job-queue worker and a specific 90-minute window matching a deploy; pulled the job-execution rows: affected messages had TWO executions each with the first marked FAILED (timeout) but provider message-ids present — the false-negative retry (send succeeded, timeout lied, job retried). **Minutes 40-90 — mitigations in parallel:** another engineer drafted the customer-communication (template with apology + credit grant — we gave affected orgs 500 free credits; the comms went out at 15:45, within 90 minutes of detection); I implemented the three-layer fix: client-generated idempotency keys (`message_key` UNIQUE), the reconciliation sweep marking false-negative FAILED jobs as SENT, and the timeout raised from 30s to 60s (Gmail’s p99 was 28s — our timeout was INSIDE their tail). **Evening — hardening:** the latency-injection regression test (toxiproxy +35s on the

SMTP path in staging CI) so the exact failure mode is exercised every release; the postmortem doc drafted same day while memory was fresh.

RESULT: Containment at 14:30 (10 minutes to stop active harm), customer comms within 90 minutes, fix deployed next morning with the regression test green, duplicate rate zero since. The credit-grant gesture converted complaints into two public thank-yous in our customer channel — the cheapest trust repair we’ve bought. The systemic outcome: the idempotency-key pattern was then applied to EVERY side-effecting external call (payments, WhatsApp, SMS later) — one incident hardened an entire class. My personal lesson, stated honestly in interviews: I initially debugged for 15 minutes BEFORE pausing the dispatcher, reasoning “I’m close to the cause” — the correct move was pause FIRST, debug second. That ordering mistake is now a runbook line, and I tell this story BECAUSE the mistake is in it; interviewers trust incident stories with visible learning more than flawless ones.

KEY TERMS TO MENTION: - Containment before diagnosis (and the honest 15-minute hesitation) - False-negative timeout + idempotency keys as the root-cause pair - Customer comms + credit grant (trust repair is part of the incident) - Latency-injection regression test (systemic hardening) - The runbook line born from my own ordering mistake

FOLLOW-UP QUESTIONS: 1. What would you have done differently with twice the team? 2. How do you decide when an incident is “over”?

FOLLOW-UP ANSWERS: 1. One engineer purely on customer comms from minute 0 (ours went out at 90 min; with the extra hands, 30), one on the reconciliation of ALREADY-duplicated sends (apology + suppression so customers’ leads weren’t double-followed-up — we did this later and slower than ideal), freeing me for the fix uninterrupted. Incident roles scale: IC, comms, mitigation, investigation — we improvised 2 of 4; the template now assigns all 4. 2. When three things are true: harm stopped (containment verified), cause understood AND mechanically prevented (regression test, not just a fix), and affected users addressed (comms + remediation). Until the third, the incident is open even if the pager is quiet — trust damage has a longer half-life than error rates, and closing the incident formally closes the follow-through.

RED FLAGS TO AVOID: - A flawless story (no mistake = no learning = suspicious) - No customer-communication dimension (technical-only incident handling) - Vague timelines (incidents are clock stories — know your minutes)

Q14-4: Describe how you improved the performance of a specific feature (STAR)

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

SITUATION: Our largest customer (the “whale org” from Q12-5 — 400k leads, 60 seats) escalated: their leads list took “forever” (measured: p95 1.9s, p99 4.2s for them, while the fleet average looked fine at 300ms). They’d started exporting to Excel and working offline — the worst kind of churn signal, because it means the product is losing to a spreadsheet.

TASK: Own the fix end-to-end: diagnose, fix, prove the improvement to a customer who had stopped believing us — within a sprint, without destabilizing the other 3,199 orgs.

ACTION: Diagnose (2 days): traces resolved the latency to one statement; production-snapshot reproduction + **EXPLAIN (ANALYZE, BUFFERS)** exposed the plan flip on the whale's data shape (full story in Q12-5 — the partial composite index fix). **Fix** (2 days): the partial covering index (`org_id, status, created_at DESC WHERE deleted_at IS NULL + INCLUDE columns`) built CONCURRENTLY off-peak; whale-org p95 dropped to 95ms in staging verification. **Guard the fleet** (1 day): per-org latency percentiles added to the monitoring (the average that hid this customer now can't hide the next one) + the index write-cost measured (4% on lead writes, documented). **Close the loop with the customer** (the part engineers skip): their success manager got before/after waterfall traces (visual, honest — including the “we found and fixed a database planning issue specific to large datasets” framing), and we shipped them a two-week early-access of the cursor-pagination upgrade so deep scrolling stayed fast as they grow. **Institutionalize:** the tenant-skew test scenario entered the load-test matrix (Q12-3) so plan-flip regressions can't ship silently again.

RESULT: Whale-org p95 1.9s → 95ms (20×), p99 4.2s → 180ms; the Excel-export workflow was abandoned by their team within two weeks (their ops lead said so on the renewal call); they renewed AND upgraded two seats. Fleet-wide, the same index improved 11 mid-size orgs' p95 by 30-60%, and the per-org monitoring caught a second, smaller skew case six weeks later — BEFORE any human complaint this time. The deeper result: “performance work” at our company became tenant-percentile-driven instead of average-driven, which is the actual root fix — the average was the bug behind the bug.

KEY TERMS TO MENTION: - The churn signal read correctly (Excel-export behavior) - EXPLAIN-driven plan-flip diagnosis + partial covering index (technical Q12-5) - Per-org percentiles as the institutional fix (the average was the real bug) - Customer-facing closure with honest waterfall evidence - The load-test skew scenario preventing recurrence

FOLLOW-UP QUESTIONS: 1. What if the index fix hadn't been enough — what was plan B? 2. How did you balance this against sprint commitments?

FOLLOW-UP ANSWERS: 1. Plan B was staged: (a) tenant-level read routing (whale reads to the replica — 2 days of work, buys 2-4×), (b) the cursor-pagination rewrite early (deeper structural relief), (c) if both failed — the honest conversation about an enterprise tier with dedicated resources (the rows-vs-tenants sharding question, Q6-15). The point I make in interviews: plan B existed before the deadline, not after plan A failed — and none of it was “add more servers” as step one. 2. Transparently with my lead: this carried a customer-escalation tag (revenue-linked), so it pre-empted one sprint item, and I gave up my “tech-debt Friday” capacity for two weeks instead of cannibalizing the team's commitments. The escalation classification system (revenue-linked vs general) is what made that negotiation 5 minutes instead of a fight — triage frameworks are what let senior engineers make trade-offs WITHOUT drama.

RED FLAGS TO AVOID: - Numbers without the business frame (churn signal, renewal outcome) - Technical fix with no institutional guard (it regresses in 6 months) - Hero framing (“I worked weekends”) instead of process framing

Q14-5: Tell me about a time you had to learn a new technology quickly (STAR)

DIFFICULTY: Easy **CATEGORY TAG:** Behavioral

SITUATION: Mid-development, product committed “AI-generated outreach drafts” to a launch in three weeks. I’d used LLM APIs for prototypes but never built a PRODUCTION path: cost metering, response validation, fallback chains, provider abstraction. The team had zero production LLM experience — I was the designated learner-and-builder.

TASK: Go from prototype-grade to production-grade LLM integration in three weeks, with the constraints that made it hard: metered credits (billing correctness), no tolerance for hallucinated emails (brand risk), and provider volatility (rate limits, outages, model deprecations — the API changed twice DURING the window).

ACTION: Days 1-3 — structured immersion with a deliverable: not tutorials — I built the thinnest vertical slice (prompt → call → validate → persist) against the real sandbox, keeping a decision journal (every choice + why — the journal became the ADR seed); read the provider docs fully ONCE (rate limits, error codes, idempotency — the operational sections first, the happy path last: production integrations fail on the operational 20%). **Days 4-10 — the production concerns, each with a spike:** response validation (JSON schema + content guards + the template fallback when validation fails — Q3-12’s degradation), cost metering wired into the credit system (token-count → credit-cost table — product-visible pricing), the AiPort abstraction (Q3-20) so the provider swap was a config change (it WAS, twice, during the window — the abstraction paying rent immediately), and the eval harness (50 golden prompts scored on each change — the prompt-evaluation discipline that later became `prompt-evaluation.ts`-shaped infrastructure). **Days 11-15 — failure-mode hardening:** WireMock ladders for the provider’s failure modes (Q13-4), rate-limit pacing, the circuit breaker, and the false-negative timeout analysis (idempotency — the lesson that resurfaced in the Q14-3 incident). **Days 16-21 — review and ship:** the two senior reviews I requested explicitly (a backend review for the billing-path correctness, a security review for prompt-injection surface — user input reaching prompts was the new attack class), staged rollout to 5% of orgs, dashboard for cost-per-generation and fallback-rate, then GA.

RESULT: Shipped on day 21. First-month numbers: 84% generation success (16% fell back to templates — invisible to users), cost-per-generation 22% under budget projection, zero hallucination-escape incidents (validation held). The decision journal became the team’s LLM-integration ADR — reused verbatim when we added the second provider. The transferable claim I make in interviews: three weeks is enough for production-grade ANYTHING if the learning is structured around operational failure modes first and happy paths second — that inversion is the whole method.

KEY TERMS TO MENTION: - Vertical-slice-first learning + decision journal (ADR seed) - Operational-docs-first reading order (rate limits/errors before happy path) - Spikes per production concern: validation, metering, abstraction, evals - Requested reviews for the NEW risk classes (billing correctness, prompt injection) - The abstraction paying rent DURING the learning window (provider swap twice)

FOLLOW-UP QUESTIONS: 1. What did you get WRONG in those three weeks? 2. How do you decide when NOT to learn-and-build, and buy/expert-hire instead?

FOLLOW-UP ANSWERS: 1. Two things: I initially over-abstracted the prompt layer (a template DSL nobody needed — deleted in week 6 for plain strings + the eval harness); and I underestimated streaming-response complexity, shipping non-streaming first and retrofitting — a UX flat-spot users noticed. Both are in the journal with the corrections; the meta-skill is CORRECTION VELOCITY — I noticed the over-abstraction via the second developer's friction, not my own. 2. When the domain is safety-critical AND the learning curve is months (payments compliance at scale, ML model design), buying expertise (a consultant review, a hire, a vendor) beats learning-in-place — my LLM case passed the test because the blast radius was bounded (template fallback existed) and the operational knowledge was transferable (it's still HTTP, retries, idempotency). The framework: learn-and-build when failure modes are recoverable and knowledge compounds; buy when failure is unrecoverable or the field is someone's decade-deep specialty.

RED FLAGS TO AVOID: - Learning-as-consumption stories (courses, no artifacts) - No failure admission (three flawless weeks is fiction) - No production-concern framing (metering, validation, fallback) — prototype-brain is the red flag

Q14-6: How did you handle technical debt in this project

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

SITUATION: Nine months in, we had classic accumulating debt: the notification path had grown three inconsistent retry conventions, two modules had outgrown their context boundaries (imports reaching across), test coverage in the meetings module had slipped to 61%, and the “we’ll fix it after launch” list had 40 items nobody read.

TASK: As the senior engineer, make debt VISIBLE, PRIORITIZED, and PAYING-DOWN without freezing feature work — the product team's tolerance for a “debt sprint” was zero.

ACTION: Inventory with teeth: turned the 40-item list into a debt register with three fields per item — interest rate (what it costs us PER SPRINT: hours of debugging, incidents caused, velocity drag), principal (fix cost), and risk class (correctness/security debt ≠ ergonomic debt). The scoring made the top-10 obvious: the notification retry inconsistency had caused 2 of our last 3 incidents — it was debt PAYING INTEREST IN PRODUCTION. **The 20% rule, negotiated with product:** one engineer-day in five on register items, chosen by interest-rate ranking, tracked in the sprint board VISIBLY (debt work is sprint work with a tag — not a shadow backlog); the negotiation frame that worked: “these two incidents last month cost us 31 engineer-hours — the 20% rule is incident-prevention billing.” **The big rocks as structured migrations:** the retry unification became the Resilience4j standardization (Q3-12) executed across 4 sprints of 20% time (expand: new pattern on new code → migrate by touch (“the boy-scout clause made mandatory for the touched files”) → delete the old paths); the boundary violations became ArchUnit rules FIRST (freeze the growth — new violations fail CI; existing ones grandfathered into the register), then paid down. **Coverage:**

the ratchet rule (Q13-5) stopped the meetings-slide without a “write tests” mandate — the floor rises as files are touched.

RESULT: Six months later: the register is 40 → 12 items (the survivors are explicitly accepted debt with recorded rationale — acceptance is a decision, not neglect), incident count from the debt classes dropped to zero (the retry unification alone eliminated the recurring failure mode), the ArchUnit boundaries have 0 violations (from 14), and meetings coverage ratcheted 61% → 83%. The cultural outcome I’m proudest of: debt discussion moved from guilt (“we should really...”) to arithmetic (interest rate vs principal), which made product engineering PARTNERS in the decisions — they now propose debt paydowns when the numbers favor it.

KEY TERMS TO MENTION: - Debt register with interest-rate/principal/risk-class scoring (arithmetic over guilt) - The 20% rule sold as incident-prevention billing (product-aligned framing) - Freeze-then-pay-down (ArchUnit first: stop growth, then reduce stock) - Boy-scout-by-touch + structured migrations across sprints - Explicitly-accepted debt with rationale (12 survivors are decisions)

FOLLOW-UP QUESTIONS: 1. When is the RIGHT answer to NOT pay down a debt? 2. How do you keep the register from becoming a graveyard?

FOLLOW-UP ANSWERS: 1. When the interest is genuinely low AND the principal is high AND the code is stable-and-untouched: our **branding-service** has a clunky interface nobody touches quarterly — rewriting it costs 3 days and buys nothing measurable; it’s ON the register as ACCEPTED with that rationale. Debt-free is not the goal; COST-AWARE is. The failure mode is symmetric: zealots who pay everything (wasted capacity) and nihilists who track nothing (incidents). The register exists to make that trade explicit per item. 2. Same mechanism as any backlog: every register item has an owner, an interest-rate review quarterly (rates change — a stable workaround’s interest decays to zero; a growing codebase’s coupling debt compounds), and the 20% rule pulls from the TOP automatically — items at the top for 2 quarters with falling interest get re-scored or accepted. Graveyards form from UNOWNED lists; ours is a maintained instrument with a cadence, which is the entire difference.

RED FLAGS TO AVOID: - “We scheduled a debt sprint” (doesn’t survive contact with product reality) - Debt framed as morality (guilt) instead of economics (interest) - No accepted-debt category (pretending zero-debt is achievable or desirable)

Q14-7: Describe your code review process

DIFFICULTY: Easy **CATEGORY TAG:** Behavioral

SITUATION: By month three we had six engineers and review quality was inconsistent: rubber-stamp LGTMs on big PRs, nit-picking wars on small ones, and a near-miss where an N+1 shipped because the reviewer looked at the logic but not the query shape.

TASK: Design a review system that scales: fast enough that PRs don’t rot, deep enough that the N+1 class of bug dies, and humane enough that reviews teach instead of gatekeep.

ACTION: Sizing discipline first (the highest-leverage change): PRs target ≤ 400 lines changed — bigger changes must be stacked (a stack tool workflow); review latency data showed >500 -line PRs took $2.3\times$ longer PER LINE to review properly and had $4\times$ the post-merge defect rate — the sizing rule is empirically ours, not industry folklore. **The review contract:** author provides — the WHY (linked ticket + the one-paragraph design decision if non-obvious), the TEST evidence (what's covered and how), screenshots for UI, and a self-review pass (the author's own comments on their diff flag the risky spots — "I'm unsure about the lock ordering here" gets eyes exactly where needed). Reviewer provides: first response < 4 working hours (the SLA, tracked), review by CHECKLIST for the failure classes we've actually paid for (N+1/query shape, transaction boundaries, error-mapping, security surface, test assertion quality — the checklist IS the incident history encoded), and the tone rule: comments address the code, propose alternatives ("consider X because Y"), and mark severity explicitly (blocker vs nit — nits are batched, non-blocking). **The teaching loop:** weekly 30-min "review retrospective" sampling one merged PR — what did we all miss / what did we over-flag; the checklist grows ONLY from incidents and retros (a checklist nobody remembers why it exists is compliance, not quality). **Automation takes the boring half:** linters, ArchUnit, coverage gates, and the conventional-commit/PR-template checks run before human eyes — humans review JUDGMENT (correctness, design, risk), machines review CONVENTION.

RESULT: Median first-review latency $9h \rightarrow 3.5h$; post-merge defect rate in reviewed code down $\sim 40\%$ over two quarters (traced via our incident labels); the N+1 class specifically: zero escapes since the query-budget CI check (Q13-3) plus the checklist — automation + human attention layered. The unquantified result I value most: junior engineers started submitting PRs with self-review comments pre-empting the obvious questions — the review process became the team's teaching mechanism, which is its real ROI.

KEY TERMS TO MENTION: - PR sizing ≤ 400 lines + stacked changes (empirical latency/defect basis) - Author contract: why + test evidence + self-review with flagged risk spots - Checklist grown only from incidents/retros; severity-marked comments (blocker vs nit) - SLA-tracked first response; automation for convention, humans for judgment - Review retrospective as the teaching loop

FOLLOW-UP QUESTIONS: 1. A senior engineer consistently rubber-stamps — how do you handle it? 2. Emergency hotfix PRs — does the process bend?

FOLLOW-UP ANSWERS: 1. With data and a role, not confrontation: the review-latency/defect dashboards make the pattern visible to THEM first; then the conversation is about fit — some seniors are better as design reviewers (pre-PR) than line reviewers; re-routing their attention to design docs and the risky-PR circuit (self-review-flagged spots) uses their judgment where it compounds. The failure mode to avoid: mandating effort (metrics as a whip) — the fix is matching the task to the person. 2. Yes — explicitly: hotfixes merge with one reviewer + a mandatory follow-up PR within 24h that brings the change back through the full gates (tests, checklist) — the emergency lane is DEFINED, logged, and audited (the follow-up-PR SLA is tracked), because undefined emergencies become the norm and the gates rot. The hotfix rate itself is monitored ($> 5\%$ of deploys = the process is being gamed or the gates are too slow — both are process bugs).

RED FLAGS TO AVOID: - LGTM culture with no checklist/teaching loop - Review as gatekeeping (tone failure, nit-wars) — reviews teach or they fail - No emergency lane (undefined exceptions rot the whole process)

Q14-8: Tell me about the feature you are most proud of

DIFFICULTY: Easy **CATEGORY TAG:** Behavioral

SITUATION: The credit metering system (Q2-13) — unglamorous on a resume (“a billing counter”) and yet the feature I’m proudest of, because everything the business DOES passes through it, and it has never lost a rupee of accounting truth under concurrency, outages, or our own bugs. When I proposed building it as an append-only ledger (double-entry-shaped) rather than a mutable balance column, it looked like over-engineering for a CRUD feature.

TASK: Build the metered-currency spine: atomic deduction under concurrency, refundable on failure, auditable to the penny, product-flexible (costs changeable without deploys), and reconciliation-proof (the nightly job must never find drift).

ACTION: The design decisions, each argued in an ADR: the ledger-as-truth (append-only rows with idempotency keys, balance denormalized for reads), the conditional-UPDATE guard (`credits >= cost` in the WHERE — concurrency correctness in one statement), PENDING → POSTED reservation semantics for long AI calls (no credits lost to our own crashes), refund-as-entry (compensation, not mutation), and the reconciliation job (`sum(ledger) == balance`, per user, nightly, alerting on drift — the invariant TESTED in production continuously). The testing investment was disproportionate and deliberate: property-based tests over the deduction/refund state space, mutation testing on the ledger package (78% score — Q13-5), the concurrency test that runs 64 parallel deductions against one account asserting the invariant (it FAILED against my first implementation — the SELECT-then-UPDATE race — which is exactly why the test exists), and the fault-injection suite (crash between deduct and LLM-call → the sweeper must refund).

RESULT: Eighteen months of production: zero reconciliation drift, zero double-charge incidents through three provider outages and two of our own partial-deploy bugs (the ledger made both VISIBLE and REFUNDABLE — the customers were made whole in minutes because the ledger knew exactly what was owed). The pattern spread: API-key usage metering and notification-rate accounting adopted the same ledger shape — the design became infrastructure. The pride I articulate in interviews isn’t the cleverness — it’s that the BORING choice (bookkeeping rigor) beat the exciting one (clever in-memory counters) in every way that mattered after month one: the ledger turned every future billing question (“was this charge correct?”) from an investigation into a SELECT.

KEY TERMS TO MENTION: - Ledger-as-truth vs mutable balance — the ADR argument that looked like over-engineering - Conditional-UPDATE guard + PENDING/POSTED reservation (crash-safe metering) - The 64-thread concurrency test FAILING the first implementation (the test earned its keep) - Zero reconciliation drift in 18 months incl. outages and deploy bugs - The pattern’s spread (API metering, notifications) — design as infrastructure

FOLLOW-UP QUESTIONS: 1. What would you change about it today? 2. How do you measure “proud” — what’s the actual criterion?

FOLLOW-UP ANSWERS: 1. Two things: the balance denormalization should have had a rebuild-from-ledger tool from day one (I wrote it after the first scary “is the balance right?” moment — the tool now runs in staging weekly); and the idempotency-key namespace was implicit (ref_type+ref_id conventions) — an explicit key table with constraints would have caught one late-night collision class earlier. Small regrets, both fixed — the architecture itself I’d keep verbatim. 2. By what it made EASY later: features that used to be scary (pricing changes, refunds, plan migrations) became config + SELECTs; incidents in its domain were non-events. The criterion: a feature you’re proud of is one whose FUTURE was made cheaper by your past decisions — pride is measured in the compounding, not the launch.

RED FLAGS TO AVOID: - Pride attached to shiny tech (AI, K8s) rather than hard correctness problems - No adversarial-testing story (the concurrency test is the credibility anchor) - No numbers (18 months, zero drift — vague pride is unpersuasive pride)

Q14-9: How did you handle disagreements on technical decisions

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

SITUATION: The sharpest disagreement: the other backend engineer wanted to split the notification system into a separate microservice immediately (12 months before I believed it was justified); I wanted the modular-monolith path (Q10-2/Q10-3’s reasoning). The disagreement was technical, public (a design review), and emotionally loaded — he’d built notification systems at scale before and felt his experience was being discounted; I had the ADR trail arguing our scale didn’t justify the ops cost yet.

TASK: Resolve it with the BEST outcome for the product, not the fastest consensus — while keeping the working relationship intact (we pair-reviewed each other’s code weekly; a poisoned relationship was a team-level cost).

ACTION: Step 1 — steelman on record: I asked him to write the microservice proposal AS the ADR-alternative section (his strongest case, in writing, in the same document as mine — asymmetric effort is where respect shows: I spent MY time making HIS case well). **Step 2 — separate the disagreement’s axes:** writing it down revealed we disagreed on FACTS (projected notification volume: he assumed 10× my number based on a pending enterprise deal) and VALUES (ops-complexity tolerance). **Step 3 — resolve facts with evidence:** we got the enterprise deal’s real requirements from the PM — 2× volume, not 10× — collapsing one axis entirely. **Step 4 — resolve values with a reversible experiment:** the disagreement shrank to “when” not “if” — we agreed on trigger metrics written INTO the ADR (notification fan-out sustained > 5k/s, or the bulkhead proving insufficient — Q2-14’s redesign gets triggered BY DATA, not by either of our opinions) and, as the bridge, he owned the notification module’s seam-hardening (the Kafka-shaped event contracts) so his architecture work happened INSIDE the monolith path — his expertise built the extraction readiness, which was genuinely the right work either way. **Step 5 — the relationship repair, deliberately:** we presented the joint ADR together, and I credited the trigger-metric

design to him explicitly in the review (because it was his contribution and because credit-where-due is how disagreement stays safe next time).

RESULT: The ADR's trigger metric fired 14 months later — the extraction proceeded on schedule with the seams his work had prepared, and the migration was the cleanest of our extractions. The relationship outcome: we became the pair that senior folks brought contentious designs to, because the team had WATCHED a hard disagreement produce a better process (trigger-metrics-instead-of-opinions) without casualties. The transferable rule I extracted: convert disagreements about the FUTURE into agreements about TRIGGERS — “we'll know it's time when X crosses Y” is a decision two engineers with different priors can both sign.

KEY TERMS TO MENTION: - Steelman-on-record in the shared ADR (asymmetric effort as respect) - Fact-vs-value decomposition of the disagreement (volume projection resolved with data) - Trigger-metrics-in-the-ADR (future disagreement → future agreement) - The bridge role (seam-hardening) using his expertise inside my path - Joint presentation + explicit credit (relationship maintenance as engineering work)

FOLLOW-UP QUESTIONS: 1. What if the evidence had SUPPORTED his 10× projection? 2. When is a technical disagreement worth escalating past the two of you?

FOLLOW-UP ANSWERS: 1. Then we'd have built it — that's what “resolve facts with evidence” means: I wasn't defending my prior, I was defending the EVIDENCE STANDARD, and it binds me symmetrically. With 10× real, the ops-cost argument inverts and his path wins on my own framework. The credibility of trigger-metrics depends entirely on both parties accepting that the metric can go either way; I've also LOST such resolutions and shipped the other design — that history is why the mechanism works. 2. When three conditions hold: the decision is irreversible-ish (a data-model fork), the two parties remain deadlocked after the fact/value decomposition, and the blast radius exceeds our teams (a platform-wide contract). Then it goes to the staff/principal review with BOTH ADRs intact — escalation with the disagreement's structure preserved (not a political summary), because the decider needs the axes, not just the heat. In 9 months, we escalated exactly once — a schema-tenancy decision — and the discipline of the written ADRs made that escalation take one meeting.

RED FLAGS TO AVOID: - Winning-by-seniority stories (title as the resolution mechanism) - No record of the OTHER side's best case (the ADR asymmetry test) - Resolution without a relationship plan (the weekly pairing continued — mention it)

Q14-10: What would you add to this project given more time (and what would you NOT)

DIFFICULTY: Medium **CATEGORY TAG:** Behavioral

SITUATION: The interview version of this question is a trap: listing shiny things (Kubernetes! Kafka! Microservices!) signals trend-following; the real answer is a PRIORITIZED list where each item pays a named bill, plus the discipline to name what I would deliberately NOT build.

TASK: (As asked in retrospectives and now in interviews:) given a 3-engineer quarter of “free” time, what is the highest-leverage build list — and what’s explicitly out?

ACTION: Would build, ranked by bills paid: 1. **The event backbone graduation (Kafka for the notification/analytics path)** — pays the Q2-14 bill before the volume forces it under duress; the seams exist (his Q14-9 work); this is buy-the-dip infrastructure. 2. **The reporting warehouse maturation** — the analytics today queries replicas; a proper columnar store fed by events pays the “cross-service queries post-extraction” bill (Q10-3) and kills the reporting load on OLTP entirely. 3. **Tenant-level performance regression suite in CI** — the whale-org plan-flip (Q12-5) was found by a customer; the skew-scenario load tests exist monthly; wiring a per-tenant latency-budget check into the PR pipeline (against the sanitized snapshot) makes the next skew bug a BUILD failure. Cheap, compounding. 4. **A customer-facing audit/API-usage portal** — Q4-13/Q9-9’s dashboards exist internally; exposing usage, audit, and key management to customers converts support tickets into self-service (measured: 30% of our support volume is “what did your system do”). 5. **Disaster-recovery game-day automation** — the DR drills (Q6-9) are semi-manual; scripting them (restore-verify as a scheduled job with pass/fail pages) turns a twice-a-year exercise into a continuous guarantee.

Would NOT build, and why — the discipline half: microservices beyond the extraction scorecard (the Q10-3 line stands — no service count as a goal); an in-house email delivery engine (provider economics and deliverability expertise favor the vendor — our EmailPort keeps it swappable); multi-region active-active (our RPO/RTO math says single-region + solid DR covers the contract tier we sell — active-active doubles consistency complexity for an SLA we don’t charge for); and a plugin marketplace (the extension surface is a security/compatibility commitment that outguns our current team — the API + webhooks ARE the extension story until customers push for more).

RESULT: (Framed as: this is the roadmap I’d defend with any CTO.) The ranked list maps 1:1 to bills ALREADY being paid in incident-hours and support-load — every item’s ROI is a number we’ve measured, not a trend; and the NOT-list is what makes the list credible: constraint is the strategy. In the actual project, item 3 shipped (the CI skew check) and item 1 is in progress — the rest live in the register (Q14-6) with interest-rate scoring, which is exactly where “given more time” answers should live: not in interviews, in a maintained instrument.

KEY TERMS TO MENTION: - Every “yes” pays a NAMED bill (incident-hours, support-load %, churn risk) - The NOT-list as the credibility half (no-K8s-for-its-own-sake, no in-house email engine) - Prioritization instrument: the debt/roadmap register with interest rates (Q14-6 continuity) - Item 3 shipped, item 1 in progress — answers with receipts, not wishes - Constraint-as-strategy framing for the “more time” question

FOLLOW-UP QUESTIONS: 1. The CEO says “build the plugin marketplace, it’s strategy.” Now what? 2. Which of your “no” items would flip FIRST, and on what evidence?

FOLLOW-UP ANSWERS: 1. Same machinery as Q14-9: decompose the ask into the bill it pays (which customers, what revenue, what retention) and the costs (the security/compatibility commitment in engineer-quarters — I’d scope the v0 honestly: a webhook-

templating layer is 80% of the marketplace's actual value at 20% of the commitment, and I'd propose THAT as the strategy-compatible v0). Strategy conversations with engineers who bring scoped options instead of resistance end well; the refusal-reflex is how engineers lose seats at strategy tables. 2. Multi-region flips first: on evidence of (a) an enterprise tier sold with a region-failover SLA, or (b) measured revenue concentration in a geography where latency is costing conversion (the funnel data can show this). Both are NUMBERS; the item moves from my NOT-list to the register the day either lands. The meta-point: "no" is dated — every item on the NOT-list carries its flip condition, which is what makes the discipline engineering instead of stubbornness.

RED FLAGS TO AVOID: - Trend-driven wish lists (K8s/Kafka/GraphQL with no bill attached) - No NOT-list (undisciplined ambition is a junior signal) - Nothing SHIPPED from previous "more time" asks (the receipts test)

End Matter — Cheat Sheet, Common Mistakes, Questions to Ask, Handling Unknown Features

APPENDIX B — API Logic Deep-Dives (the “walk me through the code” answers)

The single most common senior-level drill: “Walk me through what happens, end to end, when this endpoint is called — Controller to database row.” These six walk-throughs cover the APIs interviewers will pick. Rehearse them until you can narrate WITHOUT notes: filter chain → controller → validation → service → transaction boundary → SQL → events → response. The code sketches are the level of detail expected on a whiteboard.

B.1 — POST /api/auth/otp/verify (the security-critical path)

```
// Filter chain: corsFilter -> RateLimitFilter (per-IP bucket) ->
//               JwtAuthenticationFilter (skipped: public path)
// -> Controller
@PostMapping("/api/auth/otp/verify")
public ResponseEntity<AuthResponse> verify(@Valid @RequestBody OtpVerifyRequest req)
{
    return ResponseEntity.ok(authService.verifyOtp(req.email(), req.otp(),
        clientMeta()));
}

// Service (transactional)
@Transactional
public AuthResponse verifyOtp(String email, String otp, ClientMeta meta) {
    UserAccount user = userRepository.findByEmail(email)
        .orElseThrow(new
            BadCredentialsException("INVALID_CREDENTIALS")); // generic: no existence
        oracle
    // 1. Lockout check (Redis atomic counter, Q4-11)
    if (lockService.isLocked(email)) throw new AccountLockedException(); // 423,
        generic message
    // 2. Attempt guard: INCR otp:attempts:{hash} -- Lua atomic, TTL 15min
    lockService.recordAttempt(email);
    // 3. Constant-time verify against HASHED otp (BCrypt), expiry via stored expiry
        column
    if (!otpHasher.matches(otp, user.getOtpHash()) ||
        clock.now().isAfter(user.getOtpExpiry()))
        throw new BadCredentialsException("INVALID_CREDENTIALS"); // 5th
        failure -> 15min lock
    lockService.clearAttempts(email);
    // 4. Issue tokens: 15-min access JWT (kid, jti) + opaque refresh (SHA-256 hashed
        into user_session,
    //     family_id for rotation/reuse detection, device fingerprint recorded)
    TokenPair pair = tokenService.issue(user, meta);
    auditService.event("AUTH.OTP_SUCCESS", user); //
        AFTER_COMMIT async write
    return new AuthResponse(pair.accessToken(), pair.refreshCookie());
}
```

Narration script: rate-limit layer (per-IP 10/10min) → validation → user lookup with GENERIC error (no oracle) → Redis lockout check → atomic attempt increment → constant-time hash match + expiry → token issuance with hashed refresh + session row → audit event after commit. **Probe next:** “Where exactly can this race?” (two parallel verifies on the 5th attempt — the Lua INCR closes it), “Why hash the OTP in DB?” (dump = inert), “What leaks if you return different errors for unknown email vs wrong OTP?” (existence oracle).

B.2 — GET /api/leads?status=HOT&sort=created_at&cursor=... (the hot read path)

```

@GetMapping("/api/leads")
public LeadPageResponse list(@Valid LeadFilterRequest filter, @RequestParam
    Optional<String> cursor) {
    return leadQueryService.list(filter, cursor, securityContext.orgId()); // orgId
        from JWT claims
}

// Service: NO @Transactional(readOnly=true) needed on primary-routed single query;
// replica-routed variant has it
public LeadPageResponse list(LeadFilterRequest f, Optional<String> cursor, UUID
    orgId) {
    Cursor cur = cursorCodec.decode(cursor, f.filterHash()); // Base64 tuple+hash;
        409 on cross-query swap
    SortKey sort = sortRegistry.resolve(f.sort()); // allow-list:
        created_at -> (created_at DESC, id DESC)
    var spec =
        LeadSpecs.forOrg(orgId) // tenant filter FIRST (Q4-6
            defense in depth)
        .and(LeadSpecs.status(f.status()))
        .and(LeadSpecs.notDeleted()); // soft-delete always
        appended
    if (cur != null) spec.and(LeadSpecs.afterCursor(sort, cur.tuple())); // WHERE
        (created_at,id) < (:t1,:t2)
    Slice<LeadRow> rows = leadRepository.findBy(spec,
        PageRequest.ofSize(Math.min(f.limit(), 100)), LeadRow.class);
    return assembler.toResponse(rows, cursorCodec.encodeNext(rows, sort,
        f.filterHash())); // no total count
}

```

SQL**produced:**

SELECT id, business_name, stage, ... FROM lead WHERE org_id = ? AND status = ? AND deleted_at IS NULL
 — index (org_id, status, created_at DESC, id DESC) partial WHERE deleted_at IS NULL →
 index-only scan with INCLUDE columns. **Probe next:** “Why LIMIT 51?” (fetch one extra row
 → has_more without COUNT), “Why is id in the tuple?” (unique tiebreaker, Q8-3), “What if a
 filter changes mid-scroll?” (filter-hash → 409 CURSOR_MISMATCH).

B.3 — POST /api/outreach/generate (the metered AI path)

```

@PostMapping("/api/outreach/generate")
public ResponseEntity<GenerationResponse> generate(@Valid @RequestBody
    GenerateRequest req) {
    return ResponseEntity.accepted().body(outreachFacade.generate(req.leadId(),
        securityContext.principal()));
}

// Facade – the 6-step flow (Q3-20's Facade pattern)
@Transactional
public GenerationResponse generate(UUID leadId, Principal p) {
    Lead lead = leadRepository.findById(leadId,
        p.orgId()).orElseThrow(LeadNotFoundException::new); // 404 cross-tenant
    int cost =
        creditCosts.of("AI_GENERATION"); // Redis-cached
        config, Q7-2
    creditService.reserve(p.userId(), cost, "AI_GEN", key(leadId)); // conditional
        UPDATE + PENDING ledger row
    try {
        GeneratedEmail email = aiGateway.generate(promptManager.build(lead,
            p.orgTone()), timeout(20s));
        // aiGateway: breaker + retry(429/5xx) + fallback-to-template chain (Q3-12);
        // fallback marks AI_FALLBACK,
        // refunds the reservation, returns within budget -- user never sees a 500
        // for provider downtime
        validator.assertSafe(email); // schema +
            injection-marker guards
        Generation gen = generationRepository.save(Generation.of(lead, email));
        creditService.confirm(key(leadId)); // PENDING ->
            POSTED
        eventPublisher.publishEvent(new OutreachGeneratedEvent(gen.id(), p.orgId()));
        return GenerationResponse.of(gen);
    } catch (AiValidationException e) {
        creditService.refund(key(leadId)); // compensating
            ledger entry
        throw e;
    }
}

// Event listeners (AFTER_COMMIT + @Async): analytics counter, hot-lead re-score,
// websocket push to UI

```

Narration script: tenant-scoped fetch (404 not 403) → credit RESERVATION (conditional UPDATE — the row is the lock) → provider call OUTSIDE the lock via the gateway's breaker stack → validation → confirm reservation → AFTER_COMMIT side effects. **Probe next:** “Crash after reserve, before confirm?” (sweeper reaps PENDING > 5 min — refund; idempotency key makes the retry safe), “Why not deduct AFTER the AI call?” (double-spend window), “What does the user see when the breaker is open?” (template fallback + AI_FALLBACK tag + no charge).

B.4 — POST /api/webhooks/stripe (the money path)

```

@PostMapping(value = "/api/webhooks/stripe", consumes = "application/json")
public ResponseEntity<Void> onStripe(HttpServletRequest request) throws IOException {
    String raw = request.getInputStream().readAllBytes()...;           // RAW body:
    // signature needs exact bytes
    Event event = stripeVerifier.verifyAndParse(raw, sigHeader, tolerance(300s)); //
    // HMAC + timestamp window
    webhookIngest.ingest(Provider.STRIPE, event.id(), event.type(), raw); // persist-
    // then-ack
    return ResponseEntity.ok().build();                               // 2xx AFTER
    // durability, NOT after processing
}

// Ingest: INSERT webhook_event (provider, event_id UNIQUE, payload, status=RECEIVED)
// -- dedupe is structural.
// Async worker (job-table, SKIP LOCKED):
@Transactional
public void process(WebhookEvent we) {
    switch (we.type()) {
        case "checkout.session.completed" ->
            billingService.activateSubscription(we); // idempotent: session id
        case "invoice.payment_failed" -> billingService.markDunning(we);
        case "customer.subscription.deleted" -> billingService.downgrade(we);
        default -> webhookRegistry.markUnhandled(we);           // 2xx already
        // sent; unknown = data, not error
    }
    we.markProcessed();
}

```

Narration script: raw-byte capture → signature + tolerance → persist with UNIQUE event-id (replays no-op structurally) → 2xx immediately → async idempotent handlers keyed on provider object ids → freshness alarm (no webhooks in 30 min = page) + hourly reconciliation sweep (RECEIVED older than 10 min → re-dispatch). **Probe next:** “Why 2xx before processing?” (provider retry storms + duplicate processing otherwise), “Handler crashes mid-activate?” (transaction rolls back, row stays RECEIVED, sweep re-dispatches — handler idempotency absorbs it), “A replayed checkout.completed for an already-active sub?” (idempotency key → no-op, ledger unchanged).

B.5 — `POST /api/meetings/book` (the distributed-write path)

```

@Transactional
public BookingResponse book(UUID slotId, BookerIdentity booker) {
    // 1. Atomic slot claim: DB is the concurrency control (Q6-13)
    int claimed = slotRepository.claim(slotId, PROPOSED, BOOKED); // UPDATE ...
                          WHERE id=? AND status='PROPOSED'
    if (claimed == 0) throw new SlotAlreadyBookedException(); // 409 + next-
                          best slots
    MeetingSlot slot = slotRepository.findBooked(slotId);
    // 2. Google write AFTER local commit via outbox -- we do NOT call Google inside
    // the transaction (Q6-11:
    // no external I/O while holding row locks). Outbox row = instruction; relay
    // executes events.insert.
    outbox.publish(new CalendarEventCreate(slot.id(), slot.window(), booker,
        slot.organizerToken()));
    eventPublisher.publishEvent(new MeetingBookedEvent(slot.id(), slot.orgId())); //
    AFTER_COMMIT: ICS emails
    return BookingResponse.confirmed(slot, "PENDING_CALENDAR_SYNC");
}
// Relay worker: executes CalendarCreate via Google API (refresh token envelope-
// decrypted per use, Q9-9),
// stores google_event_id back on the slot; failure -> compensation job retries -> if
// Google says CONFLICT,
// slot marked CONFLICT + apology email + re-propose flow (Q2-12's distributed-write
// failure story).

```

Narration script: conditional claim (rows-affected verdict) → outbox instruction → commit → relay does the Google call with compensation semantics → reconciliation on conflict. **Probe next:** “Why not call Google in the transaction?” (lock held across network I/O — the pool-starvation incident class, Q6-13), “Two leads click the same slot?” (second UPDATE matches zero rows → 409), “DB committed but Google insert fails permanently?” (compensation job + CONFLICT state + human-readable recovery — never silently lost).

B.6 — POST /api/leads/import (the batch path)

```

@PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<ImportJobResponse> importLeads(@RequestPart("file")
    MultipartFile file) {
    validator.assertCsv(file); // magic-byte sniff + size cap
    // (413 clean, Q3-15)
    String key = objectStorage.put(file); // STREAMED to S3 (constant JVM
    // memory), sha256 recorded
    ImportJob job = importService.enqueue(key, file.getSize(),
    securityContext.orgId()); // job-table row
    return ResponseEntity.accepted().body(ImportJobResponse.of(job)); // 202 + job
    // id -- never parse inline
}
// Spring Batch job: FlatFileItemReader (from S3 stream, fetchSize=chunk) ->
// validate/transform processor
// (schema, dupes-in-file, dedupe-against-db via idempotency hash) -> JPA batch
// writer (jdbc batch_size 50,
// SEQUENCE ids, clear() per chunk 100 -- Q5-16's full stack). Progress = first-class
// resource:
// GET /api/leads/import/{jobId} -> {processed: 240000, total: 500000, rejected:
// 1204, reportUrl: ...}

```

Narration script: sniff + stream-to-storage (never heap) → 202 immediately → chunked batch with per-chunk commit checkpoints (crash = resume from last chunk, Q5-16) → per-row errors collected to a downloadable report → progress endpoint polled/SSE'd. **Probe next:** “Why not parse in the request?” (timeout lottery + deploy hazard, Q11-7), “Duplicate rows in the file AND in DB?” (idempotency hash per row + UNIQUE constraint — rejected-with-report, not silently merged), “A 2GB file?” (presigned direct-to-storage protocol, Q8-7 — the app never touches bytes).

Quick Reference — Cheat Sheet of All Key Annotations Used

Spring Core

Annotation	Purpose	Used Where
<code>@SpringBootApplication</code>	<code>@Configuration</code> + <code>@EnableAutoConfiguration</code> + <code>@ComponentScan</code>	Main class
<code>@Component</code>	Generic stereotype	Cross-cutting beans
<code>@Service</code>	Service layer stereotype	Service classes
<code>@Repository</code>	Repository stereotype + exception translation	Repository interfaces
<code>@Controller</code> / <code>@RestController</code>	Web layer	Controllers
<code>@Configuration</code>	Bean configuration class	Config classes
<code>@Bean</code>	Declare a bean (method- level)	Config methods
<code>@Autowired</code>	Inject dependency (we use constructor injection only)	(rarely explicit)
<code>@Qualifier("beanName")</code>	Pick a specific bean among multiple	Multi-impl scenarios
<code>@Profile("prod")</code>	Activate bean in a specific profile	Per-env beans
<code>@Lazy</code>	Inject a proxy (we avoid — circular dep smell)	(rare)
<code>@PostConstruct</code>	Setup after bean init	<code>RedisConfig</code>
<code>@PreDestroy</code>	Cleanup before bean destroy	<code>SseManager</code>
<code>@Scope("prototype")</code>	New instance per injection	<code>OutreachMessageProcessor</code>
<code>@ConditionalOnProperty</code>	Conditional bean based on a property	<code>@ConditionalOnProperty("</code>
<code>@ConditionalOnMissingBean</code>	Conditional if no other bean of this type	Fallback beans
<code>@Value("\${key}")</code>	Inject a single property	(rare — we use <code>@Configura</code>
<code>@ConfigurationProperties(prefix = "stripe")</code>	Type-safe properties binding	<code>StripeProperties</code> , <code>RedisP</code>
<code>@Validated</code>	Validate <code>@ConfigurationProperties</code>	On properties classes

Spring Web MVC

Annotation	Purpose
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> (JSON)
<code>@RequestMapping("/api/leads")</code>	Base path
<code>@GetMapping</code> , <code>@PostMapping</code> , <code>@PutMapping</code> , <code>@PatchMapping</code> , <code>@DeleteMapping</code>	HTTP method shortcuts
<code>@PathVariable</code>	Extract URL path variable
<code>@RequestParam</code>	Extract query parameter
<code>@RequestBody</code>	Bind request body to a type
<code>@RequestHeader</code>	Extract a header
<code>@CookieValue</code>	Extract a cookie
<code>@Valid</code>	Validate the bound object
<code>@Validated</code>	Validate at the method level (for <code>@RequestParam</code> + <code>@PathVariable</code> validation)
<code>@ExceptionHandler</code>	Handle an exception (per controller)
<code>@RestControllerAdvice</code>	Global exception handler
<code>@CrossOrigin</code>	Per-method CORS (we use global <code>CorsConfigurationSource</code>)
<code>@ResponseStatus(code)</code>	Override HTTP status

Spring Validation (Bean Validation)

Annotation	Purpose
<code>@NotNull</code>	Field is not null
<code>@NotBlank</code>	String is not null and not whitespace
<code>@NotEmpty</code>	Collection is not null and not empty
<code>@Size(min, max)</code>	Length/size constraint
<code>@Min</code> , <code>@Max</code>	Numeric range
<code>@Email</code>	Valid email format
<code>@Pattern(regex)</code>	Match a regex
<code>@Positive</code> , <code>@PositiveOrZero</code>	Positive numeric
<code>@Future</code> , <code>@Past</code>	Date is future/past
<code>@Valid</code>	Cascade validation to nested object
<code>@AssertTrue</code> / <code>@AssertFalse</code>	Boolean assertion

Spring Data JPA

Annotation	Purpose
<code>@Entity</code>	JPA entity
<code>@Table(name = "lead")</code>	Table mapping
<code>@Column(name = "business_name", nullable = false, length = 200)</code>	Column mapping
<code>@Id</code>	Primary key
<code>@GeneratedValue(strategy = GenerationType.UUID)</code>	Auto-generated UUID PK
<code>@OneToMany(mappedBy = "lead")</code>	Inverse side of 1:N
<code>@ManyToOne(fetch = FetchType.LAZY)</code>	Owning side of N:1
<code>@OneToOne(fetch = FetchType.LAZY)</code>	1:1
<code>@JoinColumn(name = "lead_id")</code>	FK column
<code>@JoinTable</code>	Join table for <code>@ManyToMany</code> (we avoid)
<code>@MappedSuperclass</code>	Inherited fields (no table)
<code>@Version</code>	Optimistic lock
<code>@OrderBy("createdAt DESC")</code>	JPQL-level ordering (in-memory)
<code>@OrderColumn</code>	DB-level ordering (extra column)
<code>@Enumerated(EnumType.STRING)</code>	Store enum as string
<code>@Transient</code>	Not persisted
<code>@Type(type = "jsonb")</code>	Custom Hibernate type (Hypersistence Utils for JSONB)
<code>@SQLDelete(sql = "UPDATE lead SET deleted_at = ...")</code>	Soft delete on <code>repository.delete()</code>
<code>@Where(clause = "deleted_at IS NULL")</code>	Auto-filter (deprecated — use <code>@SQLRestriction</code> in Hibernate 6.3+)
<code>@SQLRestriction(sql = "deleted_at IS NULL")</code>	

Annotation	Purpose
	JPA-standard replacement for <code>@Where</code>
<code>@Filter(name = "tenantFilter", condition = "org_id = :currentOrgId")</code>	Multi-tenant filter
<code>@FilterDef(name = "tenantFilter", ...)</code>	Define a filter
<code>@EntityGraph(attributePaths = {"user", "organization"})</code>	Per-query eager fetch (N+1 fix)
<code>@Query("JPQL")</code>	Custom JPQL
<code>@Query(value = "...", nativeQuery = true)</code>	Native SQL
<code>@Modifying(clearAutomatically = true, flushAutomatically = true)</code>	UPDATE/DELETE query
<code>@Param("name")</code>	Bind JPQL named param
<code>@Transactional</code>	Spring's transaction demarcation
<code>@Transactional(readOnly = true)</code>	Read-only (faster + routes to replica)
<code>@TransactionalEventListener(phase = AFTER_COMMIT)</code>	Listen after commit (async)
<code>@EnableJpaRepositories</code>	Enable JPA + custom config
<code>@EnableJpaAuditing</code>	Enable <code>@CreatedDate</code> + <code>@LastModifiedDate</code>
<code>@CreatedDate</code>	Auto-populate <code>createdAt</code>
<code>@LastModifiedDate</code>	Auto-populate <code>updatedAt</code>
<code>@CreatedBy</code>	Auto-populate from <code>AuditorAware</code>
<code>@EntityListeners(AuditingEntityListener.class)</code>	Enable auditing on entity

Spring Security

Annotation / Class	Purpose
<code>@EnableWebSecurity</code>	Enable Spring Security
<code>@EnableMethodSecurity</code>	Enable <code>@PreAuthorize</code> / <code>@PostAuthorize</code>
<code>@EnableOAuth2Client</code>	OAuth2 client (login)
<code>@EnableResourceServer</code>	JWT resource server (our API)
<code>SecurityFilterChain</code>	Configures the filter chain (Spring Security 6)
<code>OncePerRequestFilter</code>	Base for custom filters (<code>JwtAuthenticationFilter</code>)
<code>@PreAuthorize("hasRole('ADMIN')")</code>	Method-level authorisation
<code>@PostAuthorize</code>	Authorisation after method (rare)
<code>@Secured("ROLE_ADMIN")</code>	Legacy (we use <code>@PreAuthorize</code>)
<code>@RolesAllowed("ADMIN")</code>	JSR-250 (we use <code>@PreAuthorize</code>)
<code>UserDetailsService</code>	Custom user lookup (we don't use — JWT auth)
<code>AuthenticationProvider</code>	Custom auth logic (we have <code>OtpAuthenticationProvider</code>)
<code>AuthenticationManager</code>	Authenticates an <code>Authentication</code>
<code>JwtDecoder</code>	Decodes + verifies JWT
<code>JwtEncoder</code>	Encodes a JWT
<code>OAuth2UserService<OAuth2UserRequest, OAuth2User></code>	Custom OAuth2 user lookup
<code>OAuth2AuthorizedClientService</code>	Manages OAuth2 authorized clients (token refresh)
<code>@EnableGlobalMethodSecurity(prePostEnabled = true)</code>	(legacy, replaced by <code>@EnableMethodSecurity</code>)

Spring AOP

Annotation / Class	Purpose
<code>@Aspect</code>	Declare an aspect
<code>@Pointcut("execution(...)")</code>	Define a pointcut
<code>@Before("pointcut()")</code>	Before advice
<code>@After("pointcut()")</code>	After (finally) advice
<code>@AfterReturning(value = "pointcut()", returning = "result")</code>	After success
<code>@AfterThrowing(value = "pointcut()", throwing = "ex")</code>	After exception
<code>@Around("pointcut()")</code>	Around (full control)

Spring Async + Scheduling

Annotation	Purpose
<code>@EnableAsync</code>	Enable <code>@Async</code>
<code>@Async("executorName")</code>	Async method
<code>@EnableScheduling</code>	Enable <code>@Scheduled</code>
<code>@Scheduled(fixedDelay = 5000)</code>	Run every 5s after previous completes
<code>@Scheduled(fixedRate = 5000)</code>	Run every 5s regardless of previous
<code>@Scheduled(cron = "0 0 3 * * *")</code>	Cron expression
<code>@Scheduled(initialDelay = 10000)</code>	Delay before first run

Spring Cache

Annotation	Purpose
<code>@EnableCaching</code>	Enable cache abstraction
<code>@Cacheable(value = "planCatalog", key = "'all'")</code>	Cache method result
<code>@CacheEvict(value = "planCatalog", allEntries = true)</code>	Evict cache entries
<code>@CachePut(value = "user", key = "#user.id")</code>	Update cache (always execute method)
<code>@Caching(evict = {@CacheEvict(...), @CacheEvict(...)})</code>	Combine cache ops

Spring Events

Annotation / Class	Purpose
<code>ApplicationEventPublisher</code>	Publish an event
<code>@EventListener</code>	Synchronous event listener
<code>@TransactionalEventListener(phase = AFTER_COMMIT)</code>	Listener after commit
<code>@TransactionalEventListener(phase = AFTER_ROLLBACK)</code>	Listener after rollback
<code>@TransactionalEventListener(phase = BEFORE_COMMIT)</code>	Listener before commit
<code>@TransactionalEventListener(fallbackExecution = true)</code>	Execute even without a transaction

Spring Boot Actuator

Annotation / Endpoint	Purpose
<code>@Endpoint(id = "custom")</code>	Custom actuator endpoint
<code>@ReadOperation</code> , <code>@WriteOperation</code> , <code>@DeleteOperation</code>	Endpoint methods
<code>HealthIndicator</code>	Custom health check
<code>InfoContributor</code>	Custom info endpoint
<code>/actuator/health</code>	Health (liveness + readiness + custom indicators)
<code>/actuator/metrics</code>	Micrometer metrics
<code>/actuator/env</code>	Environment (not exposed in prod)
<code>/actuator/beans</code>	Bean list (not exposed in prod)
<code>/actuator/loggers</code>	Change log levels at runtime
<code>/actuator/threaddump</code>	Thread dump

Testing Annotations

Annotation	Purpose
<code>@SpringBootTest</code>	Full Spring context
<code>@WebMvcTest(Controller.class)</code>	Web layer slice
<code>@DataJpaTest</code>	JPA slice
<code>@JsonTest</code>	Jackson slice
<code>@Testcontainers</code>	Enable Testcontainers
<code>@Container</code>	Mark a Testcontainers container
<code>@DynamicPropertySource</code>	Override Spring properties from container
<code>@MockBean</code>	Mock a bean in Spring context
<code>@SpyBean</code>	Spy a bean (partial mock)
<code>@WithMockUser(roles = "ADMIN")</code>	Mock authenticated user
<code>@Sql("schema.sql")</code>	Run SQL before test
<code>@DirtiesContext</code>	Reset Spring context after test
<code>@ExtendWith(MockitoExtension.class)</code>	Enable Mockito
<code>@ExtendWith(SpringExtension.class)</code>	Spring + JUnit 5 (auto in <code>@SpringBootTest</code>)
<code>@ParameterizedTest</code>	JUnit 5 parametric
<code>@ValueSource(strings = {"a", "b"})</code>	Parameter source
<code>@DisplayName("...")</code>	Test display name
<code>@Nested</code>	Nested test class
<code>@Tag("integration")</code>	Tag for selective running

Resilience4j

Annotation / Class	Purpose
<code>@CircuitBreaker(name = "stripe", fallbackMethod = "fallback")</code>	Circuit breaker
<code>@Retry(name = "stripe", fallbackMethod = "fallback")</code>	Retry
<code>@RateLimiter(name = "stripe")</code>	Rate limiter
<code>@Bulkhead(name = "llm")</code>	Bulkhead (concurrency limit)
<code>@TimeLimiter(name = "llm")</code>	Time limit (async only)
<code>@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 1000, multiplier = 4))</code>	Spring Retry (alternative)

Common Mistakes to Avoid in the Interview

1. **Mentioning Next.js / Node / npm / Prisma as the backend** — instant fail. The story is **Java Spring Boot + Hibernate + PostgreSQL + Redis**. Frontend (React + TypeScript) may be mentioned but only as a client.
2. **Saying “we use microservices”** for a monolith — looks like you don’t know your own architecture.
3. **Saying “100% test coverage”** — interviewers know this is impossible without writing useless tests. Say 78% line, 64% branch (and explain why you don’t enforce 100%).
4. **Saying “p99 = 50ms”** — sounds like you’re not measuring. Real p99 for a CRUD app with DB + Redis + external APIs is 200-1000ms. We say p99 = 540ms.
5. **Saying “we use Kafka” when you actually use Spring Events** — be honest. We use both, with Kafka only for the few events that will cross a service boundary.
6. **Saying “no production incidents”** — looks fake. We had a 23-minute outage in March 2026 (Hibernate config). Own it.
7. **Saying “I built everything”** — bus factor 1 is bad. We say 70% ownership + 2 backend engineers + clear context boundaries.
8. **Quoting fake scale numbers** — be ready to defend every number. “3,200 paying orgs” is defensible; “1 million users” sounds invented.
9. **Skipping the “why” for tech choices** — interviewers want the [reasoning](#), not the choice. “Postgres because JSONB + CTE” beats “Postgres because everyone uses it”.
10. **Forgetting operational + observability** — a senior engineer owns the deploy + monitoring + on-call. Saying “the devops team handles it” is a junior answer.
11. **Not knowing your `application.yml`** — if asked “what’s in your HikariCP config” and you don’t know, you don’t own the codebase. Know the key configs (pool size, connection timeout, leak detection).
12. **Treating testing as an afterthought** — seniors have a testing strategy. Unit + slice + integration + E2E + contract + performance + chaos. Saying “we write unit tests” is not a strategy.
13. **Not knowing the failure modes** — every senior knows what happens when Redis is down, when Stripe is down, when the DB primary fails. Saying “we have backups” doesn’t answer the question.
14. **Overusing “AI” without specifics** — say which model, what prompt, what cost, what cache hit ratio. “We use AI for outreach” is a junior answer.
15. **Skipping cost** — LLM is real money (\$4,200/month for us). Stripe is real money (2.9% + ₹3 per transaction). Saying “we don’t track cost” is a fail.

16. **Saying “we use Spring Data JPA” without knowing N+1** — N+1 is the #1 Hibernate interview question. If you don’t know `@EntityGraph`, you don’t know Hibernate.
 17. **Saying `EAGER` is fine** — EAGER on `@ManyToOne` is the N+1 trap. Always override to `LAZY`.
 18. **Saying “we use REST” without specifics** — every backend uses REST. Drill on idempotency, status codes (202 vs 201), pagination (cursor vs offset), rate limiting, versioning.
 19. **Not knowing your Stripe webhook verification** — raw bytes (not JSON-parsed first), `Webhook.constructEvent`, idempotency via `event_id`. This is a standard interview question.
 20. **Saying “we use Docker” without the Dockerfile** — a senior can sketch a multi-stage Dockerfile with Spring Boot layered jar + Eclipse Temurin base.
-

Questions to Ask the Interviewer About Their System

Ask these [after](#) they've asked their questions. Interviewers at Uber/Meta/NetApp will judge you on the quality of your questions — bad questions signal lack of curiosity.

About their architecture

- “Walk me through your service architecture. Are you a monolith, microservices, or hybrid?”
- “What’s your primary datastore? Why did you choose it?”
- “How do you handle cross-service communication — sync REST, events, both?”
- “What’s your largest table, and how is it partitioned?”
- “How do you handle multi-tenancy — schema-per-tenant, shared schema with `tenant_id`, or row-level security?”

About their scale + performance

- “What’s your peak RPS? p95 latency? uptime SLA?”
- “What’s the biggest performance bottleneck you’re working on right now?”
- “How do you handle a slow external API call — circuit breaker, timeout, both?”
- “What’s your cache hit ratio target, and what do you do when it drops?”
- “How do you size your connection pools? Database? Redis?”

About their team + process

- “How is the backend team organised — by service, by feature, by layer?”
- “What does your on-call rotation look like? What’s the alert volume?”
- “What’s your deploy frequency? Lead time? MTTR?”
- “How do you handle tech debt — dedicated time, embedded in sprints, neither?”
- “What’s your code review process? Who approves PRs?”

About their tech + tools

- “What version of Java + Spring Boot are you on? Any upgrade plans?”
- “How do you manage secrets — Vault, AWS Secrets Manager, something else?”
- “What’s your observability stack — Prometheus, Datadog, something else?”
- “How do you test — what’s your pyramid? Do you have chaos testing?”
- “What’s your CI/CD pipeline — Jenkins, GitHub Actions, something else?”

About their business

- “What’s the ICP for your product? What’s the biggest customer pain point right now?”
- “How do you measure success — what’s your north-star metric?”
- “What’s the next big feature on your roadmap?”
- “What’s the biggest risk to the business right now?”
- “How does this team’s work tie to revenue?”

About the role + you

- “What does success look like in this role in the first 6 months?”
- “What’s the biggest challenge the person in this role will face?”
- “What’s the team’s technical direction for the next year?”
- “How do you support learning — conferences, books, internal talks?”
- “What’s the path to senior/staff engineer at your company?”

Smart follow-ups

- After they answer “we use Kafka”: “What’s your consumer lag monitoring? How do you handle a backlog?”
 - After they answer “we use microservices”: “How do you handle distributed transactions? Sagas, 2PC, eventual consistency?”
 - After they answer “we have 99.99% uptime”: “What’s your largest outage in the last year? What did you learn?”
 - After they answer “we use Postgres”: “Have you hit any partitioning limits? How do you handle a 1TB+ table?”
 - After they answer “we use Spring Boot”: “Are you on 3.x? Have you tried virtual threads?”
-

How to Handle Questions About Features You Did Not Build

You will be asked about features you don't know. The interviewer knows you didn't build everything. The right answer is **honest + engaged**, not "I don't know."

Pattern: Honest + show curiosity + pivot to what you know

Wrong: "I don't know, I didn't work on that." → Looks like you have no curiosity.

Better: "I didn't personally build that — my colleague owned the meeting/calendar module. From talking to him, the design was: [sketch what you know]. I'd want to learn more — what aspect are you most curious about?"

Pattern: Show your reasoning even without the specific knowledge

Wrong: "We use GraphQL. I don't know the schema." → Looks like you don't understand your own stack.

Better: "We use REST today. We considered GraphQL but didn't adopt because: (a) our frontend is our own SPA, so over-fetching isn't a problem; (b) GraphQL adds schema management complexity; (c) our clients are mostly internal (we don't have third-party API consumers). If we were building a public API for third parties, GraphQL would be more attractive."

Pattern: "I haven't worked with X, but here's how I'd approach it"

For unfamiliar tech (Kafka Streams, Cassandra, gRPC, CockroachDB), say: "I haven't worked with Cassandra, but my understanding is it's good for write-heavy workloads with eventual consistency. If I needed it for a feature, I'd read the docs + talk to someone who's used it + spike a small prototype in a week."

Pattern: When you genuinely don't know — ask back

If asked "what's the JVM flag for X" and you don't know: "I don't remember the exact flag — let me think about how I'd find it. I'd check `java -XX:+PrintFlagsFinal | grep <keyword>` to list JVM flags, or read the OpenJDK docs, or check our `Dockerfile` for the flags we set. Our current JVM args are `-Xmx2g -XX:+UseG1GC -XX:+UseStringDeduplication` — I don't remember the exact flag for X, but I'd find it in 5 minutes."

Pattern: When the question is unfair (a library you've never used)

"We don't use RxJava — we use Project Reactor via WebClient. They're similar reactive libraries. I haven't used RxJava specifically, but the concepts (observables, backpressure, schedulers) translate. If you want me to compare them, I can tell you what I know about Reactor."

Pattern: When the question is about a feature you've used but not built

"I use Kafka as a producer + consumer but I didn't set up our Kafka cluster — our devops contractor did. From talking to him, our cluster is 3 brokers, replication factor 3, retention 7 days. I know how to publish + consume + handle consumer lag, but I haven't done the broker-level admin (reassigning partitions, balancing leadership)."

Pattern: When the question is genuinely hard + you don't know — don't fake it

If asked "design Twitter's timeline at scale" and you've never thought about it: "I haven't designed a system at Twitter's scale. Let me try: I'd start with the read-heavy nature — most timelines are reads. The fan-out approach: on tweet write, write to the tweet DB + push to followers' timeline caches (Redis sorted sets). The challenge is the celebrity fan-out — a celebrity with 100M followers can't push to 100M caches on each tweet. Hybrid: pull for celebrity timelines, push for normal users. I'd want to validate this against Twitter's engineering blog posts before committing."

Summary

1. **Don't fake knowledge** — interviewers at Uber/Meta have heard every evasive answer.
 2. **Show curiosity** — you don't have to know everything; you have to want to learn.
 3. **Pivot to what you know** — bridge to a related topic.
 4. **Quantify when you don't know** — "I haven't done X, but here's how I'd approach it in a week."
-

CATEGORY 15 — BONUS: ADVANCED & TWISTED QUESTIONS (Uber / Meta / Amazon Drill-Downs)

Purpose of this appendix. The 14 categories above are the [base reference](#). The questions below are the **twisted, end-to-end, “what if the world breaks”** variants that Uber / Meta / Amazon interviewers use to separate mid-level from senior. Every question here includes (where relevant) a Java/Spring Boot code snippet, a logic trace, an ASCII diagram, or a DB schema. Treat these as the final stretch before your interview.

Q-Bonus-1: A user double-clicks “Generate Outreach” 10 times in 800ms. Your credit-deduction service must deduct exactly 50 credits once, not 500. Walk me through the entire defence, end to end, with code.

- **Difficulty:** Expert
- **Category:** System Design / Concurrency / Spring Boot
- **Tags:** idempotency, pessimistic-lock, credit-ledger, double-spend

ANSWER: The defence is layered — a client-supplied **idempotency key**, a **DB unique constraint** on it, a **pessimistic row lock** on the credit balance, and an **append-only credit ledger** so the deduction is auditable and replayable. The idempotency key is the first line: the frontend generates a UUID per “Generate Outreach” button arming and sends it in the `Idempotency-Key` header. The `OutreachController` forwards it to `CreditService.deduct(userId, amount, idempotencyKey)`, which is `@Transactional(REQUIRES_NEW)` so the deduction commits independently of the (longer) outreach-generation transaction.

Inside `deduct`, the service first tries `INSERT INTO credit_ledger (user_id, delta, idempotency_key, ...) VALUES (... , -50, ?)`; a unique index on `(user_id, idempotency_key)` makes the second click’s insert throw a `DataIntegrityViolationException`, which we catch and translate into a no-op return of the first call’s result. We also `SELECT ... FOR UPDATE` the `credit_balance` row to prevent two concurrent first-clicks from both reading 50 and both writing 0 — the lock serialises them, and the one that wins the insert commits, the other rolls back to the unique violation. The ledger row is the source of truth; the `credit_balance` row is a **materialised view** that we keep in sync with `@Transactional` write of both rows in one transaction.

Code:

```

@Service
@RequiredArgsConstructor
public class CreditService {
    private final CreditLedgerRepository ledgerRepo;
    private final CreditBalanceRepository balanceRepo;    // SELECT ... FOR UPDATE

    @Transactional(propagation = REQUIRES_NEW, isolation = REPEATABLE_READ)
    public DeductResult deduct(UUID userId, int amount, String idempotencyKey) {
        // 1. Idempotency: insert into ledger first; unique index rejects dupes.
        try {
            ledgerRepo.insert(new CreditLedgerRow(userId, -amount, idempotencyKey,
                "OUTREACH", now()));
        } catch (DataIntegrityViolationException dup) {
            // Second click – return the cached result of the first.
            return ledgerRepo.findByIdempotencyKey(idempotencyKey).result();
        }
        // 2. Pessimistic lock on balance; re-check sufficiency under lock.
        CreditBalance bal = balanceRepo.findAndLockByUserId(userId);    // SELECT ... FOR UPDATE
        if (bal.getBalance() < amount) throw new InsufficientCreditsException(userId,
            amount);
        bal.setBalance(bal.getBalance() - amount);
        balanceRepo.save(bal);    // UPDATE credit_balance
        auditPublisher.publish(new CreditsDeducted(userId, amount, idempotencyKey));
        return new DeductResult(bal.getBalance(), idempotencyKey);
    }
}

```

End-to-end trace: `POST /api/leads/{id}/outreach` → `Idempotency-Key` header → `OutreachController.generate()` validates `@Valid GenerateOutreachRequest` → calls `CreditService.deduct()` (new tx, locks row, inserts ledger, updates balance, publishes `CreditsDeducted` via `ApplicationEventPublisher`) → on success calls `AiOutreachService.generate()` (separate tx, calls LLM via `WebClient` + Resilience4j circuit breaker, persists `messages` rows) → returns variants. Second click: controller sees same `Idempotency-Key`, `CreditService.deduct()` hits unique-index violation, returns cached result, AI service is never called, user sees the first response. If AI service fails after deduction, the `CreditsDeducted` event triggers a compensating `CreditService.refund()` that writes a `+50` ledger row under the **same idempotency key (suffixed `-refund`)** — no double refund.

KEY TERMS TO MENTION: - `Idempotency-Key` header (client-generated UUID) - `@Transactional(REQUIRES_NEW)` so deduction commits before the slow LLM call - Unique index on `(user_id, idempotency_key)` as the concurrency primitive - `SELECT ... FOR UPDATE` (pessimistic lock) on `credit_balance` - Append-only `credit_ledger` (event-sourcing style), balance = materialised projection - `DataIntegrityViolationException` → no-op (return cached result) - Compensating refund on downstream failure, idempotency key suffixed

FOLLOW-UP QUESTIONS: 1. Why `SELECT ... FOR UPDATE` if you already have the unique index on idempotency key? 2. What if the user legitimately wants to generate outreach twice (two different emails)? 3. The refund path: how do you make sure the refund itself is idempotent if the compensation event is delivered twice?

FOLLOW-UP ANSWERS: 1. The unique index protects the **ledger insert** (prevents two +50 deductions). It does not protect the **balance read-modify-write** between two **different** idempotency keys that race on the same user (e.g., the user opens two tabs and clicks Generate with two different keys — both would pass the unique check but both would read `balance=50` and write 0, losing 50 credits). **FOR UPDATE** serialises the balance access across **all** deductions regardless of idempotency key. The unique index handles duplicate-clicks; the pessimistic lock handles concurrent-but-distinct operations. 2. Each legitimate generation gets a fresh **Idempotency-Key** from the frontend (a new button arming). The ledger has many rows for the same user, each with a distinct key. The balance decreases by the sum. The idempotency key protects **this exact operation**, not “this user can only ever buy once.” 3. The refund ledger row carries its own idempotency key = `{originalKey}-refund`. A unique index on `(user_id, idempotency_key)` means a second delivery of the refund event hits the same unique violation and no-ops. The consumer that publishes the compensation is itself at-least-once; the ledger’s uniqueness makes it effectively-once.

RED FLAGS TO AVOID: - “We use Redis **SETNX** on the idempotency key” — Redis is volatile; a crash between Redis-set and DB-write loses the guarantee. The DB unique index is the durable primitive. - “We check `if (balance >= amount)` then update” without a lock — classic TOCTOU race; two threads both pass the check, both deduct, balance goes negative. - Putting the deduction in the same transaction as the LLM call — if the LLM times out (30s) the credit row is locked for 30s, throughput collapses.

Q-Bonus-2: The Stripe webhook for `checkout.session.completed` arrives BEFORE your own `POST /api/payments/create-checkout-session` handler has committed the `Order` row to the DB. The webhook handler can’t find the order. How do you design the fix, with code and a diagram?

- **Difficulty:** Expert
- **Category:** System Design / Distributed Systems / Integrations
- **Tags:** outbox, eventual-consistency, webhook, race-condition, dual-write

ANSWER: This is the classic **dual-write race**: two independent systems (Stripe and our DB) commit in nondeterministic order. The fix is the **transactional outbox pattern** — the `Order` row and a `pending_webhook` expectation are written in the **same** DB transaction, so by the time Stripe can fire a webhook, the order is durable. But that doesn’t fully solve it: Stripe can still deliver the webhook before the local transaction commits if the redirect and the webhook race. So the webhook handler must **retry with backoff** when the order isn’t found, and the checkout-session handler must **publish an `OrderCommitted` event** that the webhook handler can wait on.

Concretely: `create-checkout-session` writes `Order(status=PENDING)` + `CheckoutSession(stripeSessionId)` in one `@Transactional` block, then redirects to Stripe. Stripe redirects the user’s browser back to us (fast) AND POSTs the webhook (separate request). If the webhook arrives during the 50ms commit window, it sees no order → it must return `202 Accepted` (NOT 500, otherwise Stripe retries immediately and may DDoS us) and rely on Stripe’s own retry schedule (which has exponential backoff). On the next Stripe retry

(minutes later), the order is committed and processing succeeds. For faster recovery, the webhook handler can also enqueue a delayed “reconcile this session” job onto a Redis sorted set, polled by a worker that retries once the order appears.

Diagram (race window + fix):

```

T0  create-checkout-session: BEGIN TX
      INSERT orders (id=01, status=PENDING)
      INSERT checkout_sessions (order=01, stripe=cs_test_123)
      COMMIT                                <-- durable point
T1  redirect user → Stripe
T2  user pays on Stripe
T3  Stripe redirects browser → /success?session=cs_test_123    (may arrive BEFORE T0
commit!)
T4  Stripe POSTs webhook → /webhooks/stripe (event=checkout.session.completed)
      └ handler: SELECT order WHERE stripe_session_id=cs_test_123
          └ found? process: mark PAID, grant credits, publish OrderPaid
          └ not found? return 202; enqueue delayed reconcile(cs_test_123,
runAt=now+10s)
T5  Reconcile worker pops delayed job: SELECT order ... ; if still missing, retry up
to 5 min

```

Code (webhook handler with retry-on-missing):

```

@RestController
@RequestMapping("/webhooks/stripe")
public class StripeWebhookController {
    private final OrderService orderService;
    private final StripeWebhookVerifier verifier;    // raw body + signature
    private final ReconcileQueue reconcileQueue;

    @PostMapping(consumes = "application/json")
    public ResponseEntity<Void> handle(@RequestBody byte[] rawBody,
                                       @RequestHeader("Stripe-Signature") String sig) {
        Event event = verifier.verify(rawBody, sig);    // throws → 400 (Stripe won't
        // retry on 400)
        if (event.getType().equals("checkout.session.completed")) {
            String stripeSessionId = ((Session)
                event.getDataObjectDeserializer().getObject().get()).getId();
            try {
                orderService.markPaid(stripeSessionId, event.getId());    // idempotent on
                event.getId()
            } catch (OrderNotFoundException missing) {
                // Race: webhook beat the commit. Accept; Stripe will retry. Also schedule
                // our own retry.
                reconcileQueue.enqueue(stripeSessionId, Duration.ofSeconds(10));
            }
            return ResponseEntity.accepted().build();    // 202 – Stripe counts as
                delivered
        }
        return ResponseEntity.ok().build();
    }
}

```


KEY TERMS TO MENTION: - Transactional outbox (write order + session in one tx) - 202 Accepted (not 500) on missing-order so Stripe retries with backoff, not a storm - Stripe webhook signature verification on **raw bytes** (not parsed JSON) - Idempotency on `event.getId()` (Stripe may deliver the same event twice) - Delayed reconcile queue (Redis ZSET `score=runAt`) as a hedge against Stripe dropping retries - `OrderNotFoundException` is a **transient** error in this race, not a 500

FOLLOW-UP QUESTIONS: 1. Stripe guarantees at-least-once delivery. How do you make `markPaid` idempotent? 2. What if the user's browser redirect to `/success` arrives before the webhook AND before the commit? 3. Why not just put the webhook processing in a queue and always return 202?

FOLLOW-UP ANSWERS: 1. `markPaid(stripeSessionId, eventId)` first inserts into `processed_webhook_events(event_id)` under a unique index; on duplicate it no-ops. Then it transitions the order `PENDING` → `PAID` with an `UPDATE orders SET status='PAID' WHERE id=? AND status='PENDING'` (optimistic conditional update — only one call wins). Credits are granted in the same transaction via the credit ledger with idempotency key = `event_id`. 2. The `/success` page is a **status-polling** page, not a **processing** page. It calls `GET /api/payments/verify-session?session=cs_test_123`, which returns the order's current status. If still `PENDING` (webhook hasn't landed), the frontend polls every 2s for up to 30s. The page never **processes** payment based on the redirect — only the webhook does. This prevents the redirect from being a second source of truth. 3. You can, and many teams do — it simplifies the handler to “verify signature → enqueue → 202”. The trade-off is latency: the user's `/success` page must wait for the queue worker to process, which adds p99 latency. We chose to process synchronously when possible (fast path) and fall back to the queue only on the race window. This keeps p99 payment-confirmation latency under 2s for the 99.9% of webhooks that arrive after commit.

RED FLAGS TO AVOID: - Returning 500 when the order is missing — Stripe will retry **immediately** and aggressively, multiplying load during the race window. - Verifying the signature on parsed JSON — `Stripe.Webhook.constructEvent` needs the raw body; a JSON re-serialise changes key ordering and breaks the signature. - Processing the payment in the `/success` redirect handler — that's a duplicate-processing source and ignores the webhook entirely.

Q-Bonus-3: Redis goes down during a rate-limit check on `/api/auth/otp/request`. Do you fail open (allow the request) or fail closed (deny it)? Justify with code and the security tradeoff.

- **Difficulty:** Hard
- **Category:** Spring Security / Redis / Resilience
- **Tags:** fail-open, fail-closed, rate-limit, resilience4j, graceful-degradation

ANSWER: The answer depends on the endpoint's risk profile. For **public, brute-forceable, high-blast-radius** endpoints (OTP request, magic-link request, login), you **fail closed** — a Redis outage must not turn your rate limiter into an open door, because an attacker can now fire unlimited OTPs, triggering SMS/email cost and user harassment. For **low-risk, already-**

authenticated endpoints (e.g., listing leads), you **fail open** — denying a logged-in user because Redis hiccupped is a worse UX than a temporary over-allow.

In AcquisitionOS, the `RateLimitAspect` (Spring AOP) wraps auth endpoints with a Resilience4j `CircuitBreaker` around the Redis call. When the circuit opens (Redis down), the aspect checks the `@RateLimit(failOpen=false)` annotation. For OTP, `failOpen=false` → it throws `RateLimitServiceUnavailable` → mapped to `503` with a `Retry-After: 30` header. For lead-listing, `failOpen=true` → it logs a warning, increments a local in-memory counter (best-effort), and lets the request through. We also keep a **local fallback limiter** (Caffeine, per-instance, per-IP) so even when Redis is down we have *some* protection on auth endpoints — it can't stop cross-instance attacks, but it stops a single instance from being a firehose.

Code:

```

@Aspect
@Component
@RequiredArgsConstructor
public class RateLimitAspect {
    private final RedisRateLimiter redisLimiter;
    private final LocalRateLimiter localLimiter; // Caffeine fallback
    private final CircuitBreaker redisCb; // Resilience4j

    @Around("@annotation(rateLimit)")
    public Object around(ProceedingJoinPoint pjp, RateLimit rateLimit) throws
        Throwable {
        String key = buildKey(pjp, rateLimit);
        boolean allowed;
        try {
            allowed = redisCb.executeSupplier(() -> redisLimiter.allow(key,
                rateLimit.limit(), rateLimit.window()));
        } catch (CallNotPermittedException cbOpen) {
            // Redis is down. Decision time.
            if (rateLimit.failOpen()) {
                allowed = localLimiter.allow(key, rateLimit.limit() / 2,
                    rateLimit.window()); // half the rate locally
                log.warn("Redis rate-limit down; failing OPEN with local cap for key={}",
                    key);
            } else {
                log.error("Redis rate-limit down; failing CLOSED for protected key={}", key);
                throw new RateLimitServiceUnavailableException(rateLimit.window());
            }
        }
        if (!allowed) throw new TooManyRequestsException(rateLimit.window());
        return pjp.proceed();
    }
}

// Usage:
@PostMapping("/auth/otp/request")
@RateLimit(limit = 5, window = 900, failOpen = false) // 5 per 15 min, fail CLOSED
public void requestOtp(@Valid @RequestBody OtpRequest req) { ... }

@GetMapping("/leads")
@RateLimit(limit = 100, window = 60, failOpen = true) // 100 per min, fail OPEN
public Page<Lead> listLeads(@Valid Pageable pageable) { ... }

```

KEY TERMS TO MENTION: - Fail-open vs fail-closed as a per-endpoint decision (risk-based)
 - Resilience4j `CircuitBreaker` around the Redis call - Local fallback (Caffeine) halves the rate when Redis is down — degrades, doesn't collapse - `503` + `Retry-After` for fail-closed; structured log for fail-open - Annotation-driven (`@RateLimit(failOpen=...)`) so the decision lives with the endpoint

FOLLOW-UP QUESTIONS: 1. The local Caffeine limiter is per-instance. An attacker rotates across your 4 ECS tasks — each sees 1/4 of the traffic. How do you still protect? 2. Why a circuit breaker and not just a try-catch? 3. How long is the circuit open? What's the half-open behaviour?

FOLLOW-UP ANSWERS: 1. With 4 instances each allowing `limit/2`, an attacker rotating IPs across all 4 gets `4 × limit/2 = 2×limit` — still capped, not unlimited. The fallback is a **degradation**, not a full bypass. If Redis is down for long, we alert ops and can manually switch auth endpoints to a hard 429-everything via a feature flag. The deeper fix is a Redis cluster (3 shards + replicas) so a single-node outage doesn't take the limiter down — we have that in prod, this is the “cluster is down” worst case. 2. A try-catch would hit Redis on every request even when it's down, adding 200ms of connection-timeout per request and serving no purpose. The circuit breaker trips after N failures, stops calling Redis, and fails fast (1ms) — it protects both the request latency and the Redis instance from thundering-herd reconnects. 3.

`slidingWindowType=COUNT_BASED, slidingWindowSize=20, failureRateThreshold=50%, waitDurationInOpenState=30s`
After 30s open, it lets 3 probe calls through; if they succeed, the circuit closes; if they fail, it re-opens for another 30s. This auto-recovers when Redis comes back without manual intervention.

RED FLAGS TO AVOID: - “Always fail open” — that turns Redis downtime into an auth-bypass window. - “Always fail closed” — that makes Redis a single point of failure for the entire site. - No circuit breaker, just a try-catch with a connection timeout — every request pays the timeout when Redis is down.

Q-Bonus-4: Design the database schema for the credit/billing system as an event-sourced ledger. Show tables, indexes, and the balance-projection query.

- **Difficulty:** Hard
- **Category:** Hibernate/JPA / Database Design
- **Tags:** event-sourcing, credit-ledger, materialised-view, append-only

ANSWER: The credit system is modelled as an **append-only event log** (`credit_ledger`) plus a **materialised balance** (`credit_balance`) kept in sync by the same transaction. The ledger is the source of truth; the balance is a cache. This gives us: (1) full auditability (every credit change is a row with a reason), (2) replayability (drop the balance table, re-derive from the ledger), (3) idempotency (unique index on idempotency_key), and (4) a clean refund/compensation model (a `+amount` row with a reference to the original `-amount` row).

Schema:

```

-- Append-only event log. NEVER UPDATE or DELETE a row here.
CREATE TABLE credit_ledger (
  id          BIGSERIAL PRIMARY KEY,
  user_id     UUID      NOT NULL,
  org_id      UUID      NOT NULL,
  delta       INTEGER NOT NULL,          -- negative = deduction, positive =
    grant/refund
  balance_after INTEGER NOT NULL,        -- snapshot for fast read
    (denormalised)
  reason       VARCHAR(40) NOT NULL,      -- OUTREACH, SIGNUP_BONUS, REFUND,
    TOPUP, RENEWAL, ADJUSTMENT
  ref_type     VARCHAR(40),              -- 'order', 'lead', 'message'
    (polymorphic ref)
  ref_id       UUID,
  idempotency_key VARCHAR(120) NOT NULL,  -- client UUID or "{eventId}-refund"
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by   UUID,                    -- admin who adjusted, if
    reason=ADJUSTMENT
  CONSTRAINT chk_delta_nonzero CHECK (delta <> 0)
);

-- One deduction per (user, idempotency_key). Refunds use a different key.
CREATE UNIQUE INDEX uq_ledger_user_idem ON credit_ledger (user_id, idempotency_key);

-- The history is the common query; index it.
CREATE INDEX idx_ledger_user_time ON credit_ledger (user_id, created_at DESC);
CREATE INDEX idx_ledger_org_reason_time ON credit_ledger (org_id, reason, created_at
  DESC);

-- Materialised current balance. Updated atomically with each ledger row.
CREATE TABLE credit_balance (
  user_id     UUID PRIMARY KEY REFERENCES users(id),
  org_id      UUID      NOT NULL,
  balance     INTEGER NOT NULL DEFAULT 0 CHECK (balance >= 0),
  reserved    INTEGER NOT NULL DEFAULT 0,    -- in-flight (held but not yet
    deducted)
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_balance_org ON credit_balance (org_id);

-- Monthly plan grant + renewal tracking
CREATE TABLE subscription (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL,
  plan        VARCHAR(20) NOT NULL,          -- FREE, STARTER, PRO, ELITE
  stripe_customer_id VARCHAR(64),
  stripe_sub_id VARCHAR(64),
  status       VARCHAR(20) NOT NULL,          -- TRIALING, ACTIVE, PAST_DUE,
    CANCELED
  monthly_credits INTEGER NOT NULL,
  current_period_end TIMESTAMPTZ,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_sub_org ON subscription (org_id);
CREATE INDEX idx_sub_renewal ON subscription (current_period_end) WHERE status =
  'ACTIVE';

```

```
-- Processed Stripe events (webhook idempotency)
CREATE TABLE processed_webhook_events (
  event_id      VARCHAR(80) PRIMARY KEY,
  event_type    VARCHAR(60) NOT NULL,
  processed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Balance projection (re-derive if `credit_balance` is ever lost):

```
-- Idempotent rebuild: for each user, sum deltas in created_at order, carrying the
-- running balance.
WITH running AS (
  SELECT user_id, delta,
         SUM(delta) OVER (PARTITION BY user_id ORDER BY created_at, id) AS balance
  FROM credit_ledger
)
SELECT user_id, MAX(balance) AS current_balance
FROM running
GROUP BY user_id;
-- Then: INSERT INTO credit_balance (user_id, balance) SELECT ... ON CONFLICT
-- (user_id) DO UPDATE SET balance = EXCLUDED.balance;
```

JPA mapping (sketch):

```
@Entity @Table(name = "credit_ledger")
public class CreditLedgerRow {
  @Id @GeneratedValue Long id;
  private UUID userId; private UUID orgId;
  private int delta; private int balanceAfter;
  @Enumerated(STRING) private CreditReason reason;
  private String refType; private UUID refId;
  @Column(nullable = false) private String idempotencyKey;
  @CreatedDate private Instant createdAt;
  private UUID createdBy;
}

public interface CreditLedgerRepository extends JpaRepository<CreditLedgerRow, Long>
{
  @Query("SELECT l FROM CreditLedgerRow l WHERE l.userId = :uid ORDER BY l.createdAt
  DESC")
  List<CreditLedgerRow> history(UUID uid, Pageable page);

  Optional<CreditLedgerRow> findByUserIdAndIdempotencyKey(UUID uid, String key);
}
```

KEY TERMS TO MENTION: - Append-only ledger (event sourcing); balance = projection - `balance_after` snapshot for fast read (denormalised; maintained in same tx) - Unique index on `(user_id, idempotency_key)` — the concurrency primitive - `CHECK (balance >= 0)` at the DB level — defence in depth against going negative - `processed_webhook_events` for Stripe idempotency - Window-function rebuild of the balance from the ledger

FOLLOW-UP QUESTIONS: 1. Why store `balance_after` on every ledger row? Isn't that redundant with `SUM(delta)`? 2. The ledger will grow forever. How do you archive? 3. How do you handle an admin manual adjustment (`ADJUSTMENT`) — is it just another row?

FOLLOW-UP ANSWERS: 1. `SUM(delta)` over a user with 100k ledger rows is an $O(n)$ scan per balance read. `balance_after` is an $O(1)$ read (`SELECT balance FROM credit_balance`). The redundancy is cheap (4 bytes/row) and the two are kept consistent by writing both in the same transaction. If they ever diverge (bug), the ledger is the truth and we rebuild the balance from the ledger. 2. We never delete from the ledger. We partition by `created_at` month (PostgreSQL native partitioning), and after 24 months we move old partitions to cold storage (S3 via `pg_dump` of the partition) and detach them. The `credit_balance` table stays small (one row per active user). Old users who churn keep a zero-balance row; we re-balance quarterly. Regulatory/financial records require 7-year retention, so the cold storage is the compliance copy. 3. Yes — an admin adjustment is a `CreditLedgerRow` with `reason=ADJUSTMENT`, `created_by=adminId`, and a `ref_id` pointing to a `credit_adjustment` row that stores the reason text (“billing error, customer ticket #1234”). The unique index uses the admin-supplied idempotency key (or a generated `{adminId}-{timestamp}`). Every credit change, manual or automated, is one ledger row — there is no other path to mutate the balance. This is the whole point of the design.

RED FLAGS TO AVOID: - An `UPDATE credit_balance SET balance = balance - 50` without a ledger row — you've lost the audit trail and can't explain why the balance changed. - Storing only the ledger and computing `SUM(delta)` on every read — works at 1k rows, dies at 100k. - No `CHECK (balance >= 0)` — a bug can silently make balances negative, and you discover it when a user “generates outreach” for -50 credits.

Q-Bonus-5: Design the multi-tenant user/org/membership/role schema with Hibernate filters for row-level tenant isolation. Show the schema and the `@Filter` wiring.

- **Difficulty:** Hard
- **Category:** Hibernate/JPA / Database Design / Multi-tenancy
- **Tags:** multi-tenancy, hibernate-filter, row-level-security, tenant_id

ANSWER: AcquisitionOS is a multi-tenant B2B SaaS — every tenant is an `Organisation`, every user belongs to one or more orgs via a `Membership`, and every tenant-scoped table (`leads`, `deals`, `pipelines`, `messages`) carries an `org_id` column. Row-level isolation is enforced at two layers: (1) a **Hibernate `@Filter`** automatically appends `AND org_id = :current_org_id` to every query on tenant-scoped entities, enabled per-request by a filter registry in the `TenantContext` (a `ThreadLocal` set by a servlet filter from the JWT claim); (2) a **PostgreSQL Row-Level Security policy** as a backstop — even if a bug in the Hibernate filter lets a query slip through, the DB itself rejects rows from the wrong tenant. Defence in depth.

Schema:

```

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  name        VARCHAR(120),
  password_hash VARCHAR(120),           -- NULL for OTP/magic-link-only
  status      VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE organisation (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(120) NOT NULL,
  plan        VARCHAR(20) NOT NULL DEFAULT 'FREE',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE membership (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  org_id      UUID NOT NULL REFERENCES organisation(id),
  role        VARCHAR(20) NOT NULL,      -- OWNER, ADMIN, MEMBER, VIEWER
  status      VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, org_id)
);

CREATE INDEX idx_membership_org ON membership (org_id);
CREATE INDEX idx_membership_user ON membership (user_id);

-- Tenant-scoped table (every tenant table follows this pattern)
CREATE TABLE leads (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organisation(id),
  owner_user_id UUID REFERENCES users(id), -- assigned rep; nullable for
  unassigned
  email       VARCHAR(255),
  phone       VARCHAR(40),
  company     VARCHAR(200),
  status      VARCHAR(30) NOT NULL DEFAULT 'NEW',
  quality_score INTEGER,
  data        JSONB,                    -- flexible attributes
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_leads_org_owner ON leads (org_id, owner_user_id);
CREATE INDEX idx_leads_org_status ON leads (org_id, status);
CREATE INDEX idx_leads_org_score ON leads (org_id, quality_score DESC);

-- Postgres RLS backstop
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON leads
  USING (org_id = current_setting('app.current_org_id', true)::uuid);

```

Hibernate @Filter wiring:

```

@Entity @Table(name = "leads")
@Filter(name = "tenantFilter", condition = "org_id = :orgId",
        deduceJoinColumnInsertionOrder = false)
public class Lead {
    @Id private UUID id;
    @Column(name = "org_id") private UUID orgId;
    private UUID ownerId;
    private String email, phone, company, status;
    private Integer qualityScore;
    @Column(columnDefinition = "jsonb") private String data;
    @CreatedDate private Instant createdAt;
}

// Enabled per request by a filter that reads the JWT org claim:
@Component @RequiredArgsConstructor
public class TenantFilter implements WebFilter {
    private final EntityManager em;
    public Mono<Void> filter(ServerWebExchange ex, WebFilterChain chain) {
        return AuthContext.current().doOnNext(ctx -> {
            if (ctx.isAuthenticated()) {
                em.unwrap(Session.class).enableFilter("tenantFilter")
                    .setParameter("orgId", ctx.getOrgId());
            }
        }).then(chain.filter(ex));
    }
}

```

Now every `leadRepository.findAll()` automatically becomes `SELECT ... FROM leads WHERE org_id = ?` — a developer cannot accidentally write a cross-tenant query unless they disable the filter (which is audited).

KEY TERMS TO MENTION: - `org_id` on every tenant-scoped table; `membership` as the join with role - Hibernate `@Filter` enabled per-request from JWT claim via `ThreadLocal` / context - PostgreSQL RLS as a backstop (`current_setting('app.current_org_id')`) - Defence in depth: app-layer filter + DB-layer policy - Composite indexes starting with `org_id` (most selective first for tenant queries)

FOLLOW-UP QUESTIONS: 1. A super-admin needs to read across all tenants for the admin dashboard. How do you bypass the filter safely? 2. What if a user belongs to two orgs and switches context? 3. The Hibernate filter doesn't apply to native SQL queries. How do you protect those?

FOLLOW-UP ANSWERS: 1. The `TenantFilter` checks the user's role; for `SUPER_ADMIN` it does NOT enable the tenant filter, so queries are unscoped. The admin dashboard's repository methods are additionally annotated `@PreAuthorize("hasRole('SUPER_ADMIN')")` so even if a non-admin somehow reached the method, Spring Security blocks it. The RLS policy has a `BYPASSRLS` role for super-admin connections (separate DB user) — so even native SQL is unscoped only for that connection. 2. The JWT carries the `active_org_id` claim (set at login or on `POST /auth/switch-org`). Switching org re-issues the JWT. The `TenantFilter` reads `active_org_id`, not a fixed org — so the user's queries are scoped to whichever org they've currently selected. The `membership` table enforces they can only switch to an org they're a

member of. 3. Native queries bypass the Hibernate filter. We forbid raw native SQL in tenant-scoped repositories (code review gate). Where native SQL is unavoidable (e.g., a complex analytics CTE), the developer MUST add `AND org_id = :orgId` explicitly and the PR review checks it. The RLS policy is the backstop — if a developer forgets, the DB still filters. We also have an integration test that creates two tenants' data and asserts every repository returns only the caller's tenant — runs on every PR.

RED FLAGS TO AVOID: - “We filter by `userId` in the service layer” — one forgotten `if` and you've leaked another tenant's data (this is exactly the bug the pentest found). - No RLS backstop — the app filter is your only line of defence; a single bypass is a full data breach. - Indexes not starting with `org_id` — the planner scans the whole table and filters, killing performance at scale.

Q-Bonus-6: Draw the end-to-end request flow for `POST /api/leads` (create lead), annotating every cross-cutting concern (security, validation, tx, audit, rate-limit, tracing) that fires.

- **Difficulty:** Medium
- **Category:** System Design / Spring Boot AOP
- **Tags:** request-lifecycle, aop, tracing, cross-cutting

ANSWER: This is the “trace one request through the whole stack” question. The candidate should be able to name, in order, every component that touches the request and what it does. Below is the annotated flow for creating a lead, which is representative of every mutating endpoint in AcquisitionOS.

Flow diagram:

```

Client (React SPA, TanStack Query)
| POST /api/leads {email, company, ...}  Idempotency-Key: uuid  Authorization:
Bearer aq_live_...
▼
[1] AWS ALB – TLS terminate, WAF rules (SQLi/XSS body scan), forward to ECS task
|
▼
[2] Tomcat connector – accept, hand to worker thread (Hikari-no, the servlet
threadpool)
|
▼
[3] Spring Security FilterChain (order matters!)
    ├── SecurityContextHolderFilter      – load SecurityContext from JWT
    ├── JwtAuthenticationFilter          – verify HS256 with JWT_SECRET, set auth
principal
    ├── ApiKeyAuthFilter (fallback)      – if Bearer starts aq_live_, verify API
key hash+scope
    ├── TenantFilter                    – enable Hibernate tenantFilter with orgId
from JWT
    ├── RateLimitAspect @Around          – Redis sliding-window check; 429 if
exceeded
    ├── CsrfFilter                      – for cookie-auth state-changing methods
(checks X-Requested-With or token)
    ├── RequestLoggingAspect @Around    – start MDC traceId, log method+path
    └── ExceptionTranslationFilter       – convert AuthenticationException → 401,
AccessDenied → 403
|
▼
[4] DispatcherServlet – route to LeadController.create()
|
▼
[5] LeadController.create(@Valid @RequestBody LeadCreateRequest, ...)
    ├── @Valid → BeanValidation (Hibernate Validator) → 400 if email invalid
    ├── MapStruct: LeadCreateRequest → Lead entity
    └── calls leadService.create(lead)
|
▼
[6] LeadService.create() @Transactional(isolation = READ_COMMITTED)
    ├── TransactionInterceptor (Spring AOP) – BEGIN; set Hibernate tenantFilter
already active
    ├── AuditAspect @Before – publish "lead.create.initiated" to audit log (async)
    ├── enrichIfMissing() – call enrichment service (cached, Redis-aside)
    ├── leadRepository.save(lead) → Hibernate INSERT into leads (tenantFilter adds
org_id=)
    ├── auditPublisher.publish(new LeadCreated(lead)) – in-process Spring event;
@EventListener(AFTER_COMMIT)
    └── TransactionInterceptor – COMMIT (or rollback on exception → 500)
|
▼
[7] LeadController returns 201 Created + Location: /api/leads/{id}
|
▼
[8] ResponseLoggingAspect @Around – log status + duration; MDC cleared
|
▼

```

```
[9] Sentry/OTel – span closed; trace exported to Tempo; metrics incremented
(lead_created_total{org})
|
▼
Client receives 201; TanStack Query invalidates ['leads'] → refetch list.
```

The AOP aspects in order of execution (cutpoints on the service/controller): 1. `@RateLimit` (around, controller method) — fastest fail if rate-limited 2. `@AuditLog` (around, service method) — records who did what, async to audit table 3. `@Transactional` (around, service method) — opens the DB transaction 4. `@TenantFiltered` (handled by the per-request `TenantFilter`, not an aspect) — Hibernate filter enabled 5. `@PublishEvent` (after-commit, via `ApplicationEventPublisher`) — fires `LeadCreated` for downstream (pipeline auto-stage, notification, analytics)

KEY TERMS TO MENTION: - FilterChain order (security filters before `DispatcherServlet`) - `@Valid` + Bean Validation (Hibernate Validator) at the controller - `@Transactional` + `TransactionInterceptor` (Spring AOP) at the service - Spring `ApplicationEventPublisher` + `@TransactionalEventListener(AFTER_COMMIT)` for in-process decoupling - MDC trace propagation; OTel span per layer; metrics via Micrometer - Idempotency-Key + Api-Key fallback paths

FOLLOW-UP QUESTIONS: 1. The audit log is async — what if the audit write fails? 2. Why `AFTER_COMMIT` for the event listener and not just `@EventListener`? 3. Where would you add a circuit breaker on this flow?

FOLLOW-UP ANSWERS: 1. The audit aspect writes to an in-memory bounded queue (Disruptor, 10k capacity); a background drainer flushes to the `audit_log` table in batches of 100. If the queue fills (DB slow), the aspect drops the audit event and increments a `audit_dropped_total` counter — we never block the user request on audit. The audit table itself is partitioned by day; we tolerate losing the last few seconds of audit (the access log + OTel traces are a secondary record). For **security-critical** audits (failed login, role change), we write synchronously and fail the request if the audit write fails — those can't be lost. 2. `@EventListener` fires immediately, inside the transaction; if the listener throws, the transaction rolls back. `@TransactionalEventListener(AFTER_COMMIT)` fires only after the commit succeeds, so a listener failure doesn't undo the business write. This is critical for things like “send a welcome email when a user signs up” — you don't want the email-send to be able to roll back the signup. 3. The `enrichIfMissing` call inside the service hits an external enrichment API. We wrap it in a Resilience4j circuit breaker (`@CircuitBreaker(name="enrichment")`) — if the enrichment provider is down, we save the lead un-enriched (`quality_score=null`) and queue a background job to enrich later, rather than failing the create. The user gets a 201 with a partially-populated lead; the enrichment completes asynchronously.

RED FLAGS TO AVOID: - Not knowing the filter chain order — “I think security runs first, then the controller” is too vague for a senior. - Putting `@Transactional` on the controller — the transaction then spans serialisation/JSON, holding the DB connection for the whole HTTP response. - Synchronous audit + synchronous email in the same tx as the business write — a slow SMTP server now rolls back your lead create.

Q-Bonus-7: Draw the zero-downtime deployment flow (blue/green with a DB schema migration). Where can it go wrong and how do you guard each step?

- **Difficulty:** Hard
- **Category:** DevOps / Deployment
- **Tags:** blue-green, flyway, zero-downtime, expand-contract, rollback

ANSWER: Zero-downtime with a schema change is the trickiest deployment scenario because the new code and the new schema must coexist with the old code during the switch. The pattern is **expand-contract**: (1) deploy a schema that is backward-compatible with the old code (the “expand” — add nullable columns, new tables, new indexes **CONCURRENTLY**), (2) deploy the new code (which writes to both old and new), (3) after a soak period, run a “contract” migration (drop old columns) once the old code is fully drained. For a non-expand/contract change (e.g., renaming a column), we do it across two releases: release N renames + writes both, release N+1 drops the old.

Deployment flow:

```
[GREEN = v1.4 live, 4 tasks serving 100% traffic]  [BLUE = v1.5 being deployed]
```

1. Build v1.5 image (GitHub Actions: test → Docker → ECR)
2. Flyway "expand" migration V1.5__expand.sql applied to RDS:
 - ALTER TABLE leads ADD COLUMN linkedin_url TEXT; (nullable – old code ignores)
 - CREATE INDEX CONCURRENTLY idx_leads_org_phone ON leads(org_id, phone);
 These run on the live GREEN DB; old code is unaffected.
3. Provision BLUE: 4 ECS Fargate tasks running v1.5, registered to a "blue" target group, 0% traffic.
4. Health check: ALB pings BLUE /actuator/health for 60s; all 4 must be UP.
5. Shift 10% → BLUE (ALB weighted target groups). Watch p95 latency, 5xx, error budget for 10 min.
6. Shift 50% → BLUE. Watch 10 min.
7. Shift 100% → BLUE. GREEN drains: existing requests finish, no new ones; tasks stop after 5 min.
8. GREEN target group deleted. v1.5 is now live.
9. After 24h soak: Flyway "contract" migration V1.5_1__contract.sql:
 - DROP COLUMN legacy_phone; (only safe once no code references it – verify via grep + a canary)
10. Tag the release; update runbook.

▼ Rollback plan: if 5xx > 1% in step 5/6, shift ALB weight back to 100% GREEN (1 command, <30s).

Code rollback = instant (just traffic). Schema rollback = the expand migration is forward-only safe (additive); only the contract step is irreversible, and we run it 24h later after confidence.

Where it goes wrong + guard: | Step | Failure | Guard | |—|—|—| | 2 (expand migration) |
 CREATE INDEX locks the table for writes | Use CONCURRENTLY (no write lock, longer runtime);

monitor `pg_stat_progress` | | 3 (BLUE provision) | New code can't connect to RDS (security group) | `/actuator/health/db` reports DOWN; ALB keeps BLUE at 0%; alert fires | | 4 (health check) | App starts but a cache warmup is missing → first requests slow | Add `/actuator/health/readiness` that returns UP only after cache warmup completes | | 5 (10% shift) | v1.5 has a bug that 10% of users hit | Automated rollback on 5xx > 1% (CloudWatch alarm → Lambda → ALB weight reset) | | 6 (50%) | Schema contract accidentally applied early | Contract migration is a `separate` deploy, 24h later, with a manual gate | | 7 (drain GREEN) | Long-polling requests on GREEN hang | Connection-draining timeout 300s; after that the ALB closes them | | 9 (contract) | Old column still referenced by a missed code path | `grep` + a canary release of the contract; if any 500s, restore from snapshot (RDS PITR) |

KEY TERMS TO MENTION: - Expand-contract (a.k.a. parallel-change) — additive first, destructive later - `CREATE INDEX CONCURRENTLY` (no write lock) - Flyway versioned + repeatable migrations; out-of-order detection OFF in prod - ALB weighted target groups for gradual traffic shift - `/actuator/health` vs `/actuator/health/readiness` (liveness vs readiness) - Automated rollback on SLO breach (CloudWatch alarm → Lambda) - RDS point-in-time recovery for the “contract went wrong” case

FOLLOW-UP QUESTIONS: 1. The contract step drops a column. How do you know no code references it? 2. What if the expand migration itself fails halfway (3 of 5 statements ran)? 3. How do you handle a migration that adds a NOT NULL column to a 50M-row table?

FOLLOW-UP ANSWERS: 1. Code review + `grep -r legacy_phone src/` returns nothing; a canary deploy of the contract to one task first (via a feature flag that runs the contract only when `contract.enabled=true`); watch that task's 5xx for an hour. We also keep the column for one more release — we only drop in release N+2 after release N+1 has run for a week with no references. Belt and suspenders. 2. Flyway wraps each migration file in a transaction (Postgres DDL is transactional) — a halfway failure rolls back the whole file, and Flyway marks it `failed` in `flyway_schema_history`. The fix: correct the script, `flyway repair` (clears the failed row), redeploy. `CREATE INDEX CONCURRENTLY` is the one DDL that *isn't* transactional and can leave a partial index — we `DROP INDEX CONCURRENTLY` and retry. 3. Three-step: (1) `ALTER TABLE leads ADD COLUMN score INTEGER;` (nullable, instant on PG 11+ — no rewrite); (2) backfill in batches of 10k with `UPDATE leads SET score = ... WHERE id BETWEEN ... AND score IS NULL;` throttled to avoid replica lag; (3) once all rows have a value, `ALTER TABLE leads ALTER COLUMN score SET NOT NULL;` (fast — PG verifies the constraint on existing rows). Never `ALTER TABLE ... ADD COLUMN score INTEGER NOT NULL DEFAULT 0;` on PG < 11 — it rewrites the whole table, locking it.

RED FLAGS TO AVOID: - “We just run the migration in the deploy step” without separating expand and contract — a non-additive change blocks zero-downtime. - `CREATE INDEX` without `CONCURRENTLY` on a large table — write lock for minutes. - No automated rollback — relying on a human to notice 5xx and run a command at 3am.

Q-Bonus-8: A Spring Batch job scraping 50k leads crashes at row 30,012. How do you design it so a restart resumes from row 30,012 and doesn't re-scrape the first 30k?

- **Difficulty:** Hard
- **Category:** Spring Batch / Background Jobs
- **Tags:** spring-batch, checkpoint, restartable, chunk-oriented

ANSWER: Spring Batch's chunk-oriented step model gives us this for free if we use it correctly: the `JobRepository` persists the step's `ExecutionContext` after every chunk commit, so a restart re-reads the last committed position. The key is to use a `JdbcPagingItemReader` (or a `RepositoryItemReader` with paging) rather than a cursor-based reader — cursors don't survive a JVM restart because the DB result-set is gone. The reader's page index is stored in the step's `ExecutionContext` (`reader.page`), and on restart the reader re-issues the query starting at that page.

Config:

```

@Bean
public Job leadDiscoveryJob(Step scrapeStep) {
    return jobs.get("leadDiscoveryJob")
        .incrementer(new
            RunIdIncrementer()) // new params each run; restart uses the same params
        .start(scrapeStep)
        .build();
}

@Bean
public Step scrapeStep(ItemReader<RawLead> reader, ItemProcessor<RawLead, Lead>
    processor,
    ItemWriter<Lead> writer, PlatformTransactionManager tx) {
    return steps.get("scrapeStep")
        .<RawLead, Lead>chunk(200) // commit every 200 rows → checkpoint
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .transactionManager(tx)
        .faultTolerant()
        .retryLimit(3)
        .retry(TransientNetworkException.class) // retry transient HTTP failures
        .skipLimit(50)
        .skip(MalformedLeadException.class) // skip garbage, don't fail the
            job
        .listener(new ChunkProgressListener()) // logs chunk progress
        .build();
}

@Bean
@StepScope
public JdbcPagingItemReader<RawLead> leadReader(@Value("#{jobParameters['since']}")
    LocalDate since) {
    return new JdbcPagingItemReaderBuilder<RawLead>()
        .name("leadReader")
        .dataSource(dataSource)
        .selectClause("SELECT id, source_url, raw_json")
        .fromClause("FROM raw_lead_queue")
        .whereClause("WHERE scraped_at IS NULL AND enqueued_at >= :since")
        .parameterValues(Map.of("since", since))
        .sortKeys(Map.of("id", Order.ASCENDING)) // stable order → paging stable on
            restart
        .pageSize(200)
        .rowMapper(new RawLeadRowMapper())
        .saveState(true) // ← persists reader position to
            ExecutionContext
        .build();
}

```

How restart works: when the JVM crashes mid-chunk, the chunk's transaction rolls back (no partial commit), but the `previous` chunks are committed and the `ExecutionContext` for the step has `reader.page = 150` (30,000/200). On `JobOperator.restart(jobExecutionId)`, Spring Batch loads the step's `ExecutionContext`, sets the reader's page to 150, and the reader issues `SELECT ... WHERE id > last_id ORDER BY id LIMIT 200 OFFSET 0` — wait, paging

with offset is unstable if rows are inserted; we use a **keyset** approach: the reader stores `reader.last.id` and the query becomes `WHERE id > :lastId ORDER BY id LIMIT 200`. This is stable across inserts.

Crucial correctness points: 1. `saveState(true)` on the reader — otherwise no checkpoint. 2. Chunk size matches the page size — otherwise the reader reads ahead of the committed position and a crash loses uncommitted reads (they're re-read on restart, but if the source mutates you can skip rows). Equal sizes keep them aligned. 3. Stable sort key (`id`, not `created_at` which can collide) — paging must be deterministic. 4. The writer must be idempotent (`INSERT ... ON CONFLICT (id) DO NOTHING`) — because a retry after a partial chunk commit can re-write a few rows. 5. The job parameters must be identical for a restart — `RunIdIncrementer` is for fresh runs; restarts reuse the original params.

KEY TERMS TO MENTION: - `JobRepository` + `ExecutionContext` (the checkpoint store) - `JdbcPagingItemReader` with `saveState(true)` (vs cursor reader — cursor can't restart) - Chunk commit = checkpoint (200 rows default) - `faultTolerant().retry().skip()` for transient vs permanent errors - Keyset paging (`WHERE id > :lastId`) not OFFSET paging (unstable) - Idempotent writer (`ON CONFLICT DO NOTHING`) because retries re-write - `JobOperator.restart(jobExecutionId)`

FOLLOW-UP QUESTIONS: 1. The scraper makes an HTTP call per lead. How do you parallelise without breaking order? 2. The job has been “running” for 6 hours with no progress. How do you diagnose? 3. How do you prevent two instances of the same job from running at once?

FOLLOW-UP ANSWERS: 1. Use a `TaskExecutorStep` (multi-threaded step) — Spring Batch partitions the reader's pages across threads, but the writer must be thread-safe (JPA writer is). Order is `per-chunk`, not global — if you need global order, use a `partitioner` that splits the input by id-range into separate steps, each single-threaded. For scraping, order doesn't matter, so a `TaskExecutorStep` with 8 threads + a `ThreadPoolTaskExecutor` is fine. The `ExecutionContext` stores the highest committed id per partition. 2. Check `BATCH_JOB_EXECUTION` table: `status=STARTED, last_updated=6h ago` → the JVM is wedged (not crashed, not progressing). Look at the thread dump (`jstack` on the container) — most likely a hung HTTP connection with no timeout. Fix: every `WebClient` call must have `.timeout(Duration.ofSeconds(10))` and the job's `taskExecutor` must have a keep-alive that aborts stuck threads. We also set a step-level timeout (`stepBuilder.timeout(60, MINUTES)`) that fails the step if a single chunk takes >1 min — surfaces wedges as failures, not silent hangs. 3. Spring Batch's `JobLauncher` is synchronous by default and the `JobRepository` locks the job instance+params combination — a second `start` with the same params blocks until the first completes (or throws `JobExecutionAlreadyRunningException`). For async launch (our cron path), we wrap the launch in a `JobRegistrySmartTrigger` that checks `jobExplorer.findRunningJobExecutions()` first and skips if any are running. We also use a Redis distributed lock (`SET NX EX 3600 lock:job:leadDiscovery`) as a backstop so two ECS tasks can't both launch the same job.

RED FLAGS TO AVOID: - A cursor-based reader (`JdbcCursorItemReader`) for a long job — the cursor's result-set dies with the JVM; restart re-reads from row 0. - OFFSET paging —

insertions shift the window; you skip or duplicate rows. - A non-idempotent writer — restart re-writes the last partial chunk, corrupting data.

Q-Bonus-9: Cache stampede on `/api/leads/hot` . 10k concurrent users all miss the cache at the same time and hammer Postgres. Design the fix with code.

- **Difficulty:** Hard
- **Category:** Redis / Caching / Performance
- **Tags:** cache-stampede, single-flight, probabilistic-early-expiry, lock

ANSWER: A cache stampede (or thundering herd) happens when a hot key expires and many requests miss simultaneously, all computing the expensive value, all hitting the DB. Three fixes, layered: (1) **probabilistic early expiry** (XFetch) — each request randomly expires the cache a bit before the TTL, so one request refreshes while the others still serve the slightly-stale value; (2) **single-flight via Redis lock** — only one request computes; the rest wait and read the refreshed cache; (3) **stale-while-revalidate** — serve the stale value while refreshing in the background.

Code (all three combined):


```

@Service @RequiredArgsConstructor
public class HotLeadsService {
    private final RedisTemplate<String, List<Lead>> redis;
    private final LeadRepository leadRepo;
    private static final Duration TTL = Duration.ofMinutes(5);
    private static final Duration EARLY_WINDOW = Duration.ofSeconds(30); // jitter
        range

    public List<Lead> hotLeads(UUID orgId) {
        String key = "hot:leads:" + orgId;
        String lockKey = "lock:" + key;
        Cached<List<Lead>> cached = redis.opsForValue().get(key); // {value, expiresAt}

        // 1. Cache hit, not near expiry → serve.
        if (cached != null && !shouldEarlyRefresh(cached.expiresAt, EARLY_WINDOW)) {
            return cached.value;
        }

        // 2. Stale hit OR miss → try to be the single refresh winner.
        boolean gotLock = redis.opsForValue().setIfAbsent(lockKey, "1",
            Duration.ofSeconds(10));
        if (gotLock) {
            try {
                // I won the single-flight lock → compute and refresh.
                if (cached != null) {
                    // Stale-while-revalidate: serve stale to THIS caller, refresh in bg.
                    // For simplicity here, compute synchronously; in prod we'd fork to @Async.
                }
                List<Lead> fresh = leadRepo.findHotByOrg(orgId, PageRequest.of(0, 50));
                redis.opsForValue().set(key, new Cached<>(fresh, Instant.now().plus(TTL)),
                    TTL);
                return fresh;
            } finally {
                redis.delete(lockKey);
            }
        } else {
            // 3. Lost the lock – someone else is refreshing. Serve stale if available.
            if (cached != null) return cached.value;
            // Cold miss and no lock: wait briefly for the refresher, then re-read.
            sleep(Duration.ofMillis(50));
            Cached<List<Lead>> retry = redis.opsForValue().get(key);
            if (retry != null) return retry.value;
            // Last resort: compute directly (DB hit) but don't write (avoid clobbering).
            return leadRepo.findHotByOrg(orgId, PageRequest.of(0, 50));
        }
    }
}

/** XFetch algorithm: near expiry, randomly decide to refresh early. */
private boolean shouldEarlyRefresh(Instant expiresAt, Duration window) {
    Duration remaining = Duration.between(Instant.now(), expiresAt);
    if (remaining.isNegative() || remaining.compareTo(window) > 0) return false;
    double frac = 1.0 - (remaining.toMillis() / (double) window.toMillis());
    return ThreadLocalRandom.current().nextDouble() < frac; // probability rises as
        expiry nears

```

```
}
}
```

How it behaves: with a 5-min TTL and 30s early window, the first request inside the last 30s has a small chance of refreshing; as more requests arrive, more refresh, but the lock ensures only one actually hits the DB — the rest serve stale. By the time the TTL truly expires, the cache is already fresh. Under 10k concurrent users, only 1 hits Postgres; 9,999 serve the cache.

KEY TERMS TO MENTION: - Probabilistic early expiry / XFetch (paper by Vattani et al.) - Single-flight lock (`SET NX EX`) — one refresher - Stale-while-revalidate (serve stale during refresh) - Lock TTL must exceed expected compute time (10s here) - Last-resort direct compute on cold-miss + lock-lost (rare)

FOLLOW-UP QUESTIONS: 1. The lock holder crashes before deleting the lock. What happens? 2. Why probabilistic early expiry vs just a shorter TTL? 3. The hot-leads query is itself slow (800ms). How else would you speed it up?

FOLLOW-UP ANSWERS: 1. The lock has a 10s TTL — it auto-expires. The next request acquires it and refreshes. Worst case: a 10s window where the lock is held by a dead process; during that window the cache serves stale (if present) or the last-resort direct-compute path runs. The lock TTL is a trade-off: too short and a slow compute releases early (a second refresher runs); too long and a crashed holder blocks refreshes for too long. 10s covers our p99 compute time (2s) with 5x headroom. 2. A shorter TTL means more refreshes overall (higher DB load) and no protection against a stampede at the exact expiry boundary. XFetch spreads refreshes across the early window probabilistically — on average one refresh per key per TTL cycle, same as a short TTL, but the refresh is non-synchronized (no thundering herd). The win is the de-synchronisation, not the count. 3. Three layers: (1) materialise the hot-leads result into a `hot_leads_mv` materialised view refreshed every minute by a `@Scheduled` job — the query becomes `SELECT * FROM hot_leads_mv WHERE org_id=?`, an index seek, ~2ms; (2) add a composite index `(org_id, quality_score DESC, last_activity_at DESC)` so the underlying query is an index-only scan; (3) precompute per-org into Redis during the nightly refresh job, so the cache is *always* warm and never cold-misses. The stampede fix is for the rare cold-miss; the materialised view is the daily performance fix.

RED FLAGS TO AVOID: - Just a mutex in the JVM (synchronized block) — protects one instance, not 4; the other 3 still stampede. - Lock with no TTL — a crashed holder deadlocks the key forever. - Refreshing on every miss without single-flight — you've re-created the stampede.

Q-Bonus-10: Optimistic lock retry storm on a hot Deal . 20 reps all move the same deal stage at once. The `@Version` field causes 19 `OptimisticLockException`s and they all retry immediately, re-storming. Design a fix.

- **Difficulty:** Expert

- **Category:** Hibernate/JPA / Concurrency
- **Tags:** optimistic-locking, retry-storm, backoff, saga

ANSWER: The retry storm is the failure mode of naive optimistic locking: 20 threads read version=5, all write `WHERE version=5`, 1 succeeds (version→6), 19 fail, all retry reading version=6, write `WHERE version=6`, 1 succeeds, 18 fail — the retries don't converge, they cascade. Three fixes: (1) **exponential backoff with jitter** on the retry — 19 threads spread their retries over 100ms-1s instead of all-instant, so the next round has fewer conflicts; (2) **a short pessimistic lock on the retry path** — first attempt is optimistic (fast path, no contention for the 99% case), but on `OptimisticLockException` the retry acquires `SELECT ... FOR UPDATE`, serialising the remaining retries cleanly; (3) **detect a true high-contention case and shed load** — if a deal has >5 concurrent stage moves queued, reject with 409 + a message (“the deal is being edited by another rep, retry in a moment”) rather than grinding through 20 retries.

Code:

```

@Service @RequiredArgsConstructor
public class DealService {
    private final DealRepository dealRepo;
    private final TransactionTemplate tx;

    public Deal moveStage(UUID dealId, String newStage, UUID actorId) {
        // Retry with backoff. Optimistic first; fall back to pessimistic on contention.
        return RetryTemplate.builder()
            .maxAttempts(4)
            .exponentialBackoff(50, 2, 1000) // 50ms, 100ms, 200ms... + jitter
            .retryOn(OptimisticLockingFailureException.class)
            .build()
            .execute(ctx -> {
                if (ctx.getRetryCount() == 0) {
                    return tryOptimistic(dealId, newStage, actorId); // fast path
                } else {
                    return tryPessimistic(dealId, newStage, actorId); // serialised
                }
            });
    }

    private Deal tryOptimistic(UUID id, String stage, UUID actor) {
        return tx.execute(status -> {
            Deal d = dealRepo.findById(id).orElseThrow(); // version read
            if (d.getStatus().equals(stage)) return d; // idempotent no-op
            d.setStatus(stage);
            d.setLastMovedBy(actor);
            return dealRepo.save(d); // UPDATE ... WHERE version=? → 1 row or 0 (OLE)
        });
    }

    private Deal tryPessimistic(UUID id, String stage, UUID actor) {
        return tx.execute(status -> {
            Deal d = dealRepo.findAndLockById(id); // SELECT ... FOR UPDATE
            if (d.getStatus().equals(stage)) return d;
            d.setStatus(stage); d.setLastMovedBy(actor);
            return dealRepo.save(d); // version ignored under FOR UPDATE
        });
    }
}

```

Why this converges: the first round is optimistic (no lock, fast). The 19 failures back off with jitter (50-100ms spread). The first retry to fire grabs **FOR UPDATE**, blocking the other 18 — they queue on the row lock (not on version conflicts), and execute one-by-one as the lock is released. After ~4 retries the work is done. The `maxAttempts(4)` caps it — if 4 retries fail (true deadlock or pathological contention), we surface a 409 to the client with a “retry in a moment” message.

Better than retries — rethink the operation: a deal-stage move is often a **command**, not a **write to a shared row**. If 20 reps moving the same deal is a real business scenario (not a bug), model it as a `deal_stage_change_request` queue: each rep submits a request, a single

`DealStageCoordinator` applies them in order under one lock, with audit. The reps poll their request's status. This turns 20 concurrent writes into 20 sequential queue items — no retries needed.

KEY TERMS TO MENTION: - `@Version` (JPA optimistic locking field) - `OptimisticLockingFailureException` (Spring's wrapper) - Exponential backoff + jitter (de-synchronise retries) - Fast-path optimistic / retry-path pessimistic (hybrid) - `SELECT ... FOR UPDATE` (pessimistic, serialises the retries) - Command-vs-state modelling (queue if contention is a business pattern)

FOLLOW-UP QUESTIONS: 1. Why not just always use `FOR UPDATE` (pessimistic) and skip the optimistic path? 2. How does jitter help? Without it, what happens? 3. The “queue the requests” fix — how do the reps get feedback?

FOLLOW-UP ANSWERS: 1. Pessimistic locks the row for the whole transaction, including any enrichment/audit writes. For the 99% case (1 rep moves a deal), this is unnecessary contention — the row is locked for 20ms instead of 0ms. Optimistic is lock-free in the common case, paying the retry cost only on the rare conflict. The hybrid gives you optimistic's speed in the common case and pessimistic's clean serialisation in the contention case. 2. Without jitter, all 19 retries fire at exactly 50ms, 100ms, 200ms — they re-conflict at the same instant, same as the original storm but shifted in time. Jitter (e.g., `backoff + ThreadLocalRandom.nextDouble() * backoff`) spreads the retries across [50ms, 100ms], so on the first retry round only ~10 fire concurrently, then ~5, converging. Jitter turns a re-storm into a damped oscillation. 3. The `POST /api/deals/{id}/stage-change` returns 202 with a `request_id`; the rep's UI subscribes (SSE) to `/api/deals/{id}/stage-change/{request_id}/status`. The coordinator applies requests in order, updating each request's status (QUEUED → APPLIED, or QUEUED → REJECTED with a reason like “stage already was that value” or “deal was won by another rep”). The rep sees the outcome in near-real-time. This is more work than a retry, but it models the business correctly: a deal-stage change is a workflow, not a row write.

RED FLAGS TO AVOID: - `maxAttempts(100)` — you're just multiplying DB load; if it hasn't converged in 4-5 retries with backoff, it won't. - Retry without backoff — the classic “all retries fire at the same instant” storm. - Retrying on `PessimisticLockException` with more pessimism — that's a deadlock, you need a shorter transaction or a different access order, not more retries.

Q-Bonus-11: Kafka consumer lag spikes from 0 to 2M messages during a viral campaign. The producer is fine; consumers can't keep up. Walk me through your diagnosis and fix.

- **Difficulty:** Hard
- **Category:** Distributed Systems / Messaging / Performance
- **Tags:** kafka, consumer-lag, partitioning, backpressure

ANSWER: Diagnosis first, fix second. The lag spike means throughput in > throughput out. The producer is sending faster than the consumers can process. Steps: (1) confirm it's compute-bound, not I/O-bound — look at consumer CPU; if 100%, it's processing speed; if

low, it's waiting on a downstream (DB, external API); (2) confirm partition count — if you have 6 partitions and 6 consumers, adding more consumers won't help (you're partition-limited); (3) check for a poison-pill message that's making one consumer loop — the lag is uneven across partitions; (4) check consumer `poll()` timeout and `max.poll.records` — too large and the poll takes longer than `max.poll.interval.ms`, triggering a rebalance.

Fixes (in escalating order):

- 1. Increase partitions + consumers** (if partition-limited and the topic is keyed for ordering only within a key). Bump from 6 → 24 partitions, deploy 24 consumers. Lag drops 4x. This requires a producer-side key change (or no key) and a consumer group rebalance.
- 2. Parallelise within a partition** (if ordering within a partition can be relaxed to ordering within a key). Use a `ConcurrentKafkaListenerContainerFactory` with `concurrency=N` — each container thread processes a subset of the partition's messages by key (a `Partitioner` or in-app hash routing). Effectively N consumers per partition, but you give up strict per-partition order.
- 3. Batch the consumer's downstream write** — if each message does a DB write, switch to batching: collect 200 messages, `INSERT ... VALUES (...), (...), ...` in one statement. 200x fewer round-trips. This is usually the single biggest win.
- 4. Shed / sample** — for a viral campaign where 2M messages is more than the business can act on, drop low-priority messages at the consumer (e.g., “view events” with a 10% sample). The lag stays bounded by the sampling rate.
- 5. Scale the downstream, not Kafka** — if the consumer is waiting on Postgres, add a read replica or queue the writes to a `processing_outbox` table and drain with a batch worker. Kafka isn't the bottleneck; the DB is.

Code (batch consumer):


```

@KafkaListener(topics = "lead-activity", groupId = "analytics",
               containerFactory = "batchFactory",
               concurrency = "4")
public void onBatch(List<LeadActivity> batch, Acknowledgment ack) {
    // batch.size() <= max.poll.records (e.g., 500)
    List<ActivityRow> rows = batch.stream().map(mapper::toRow).toList();
    activityRepo.batchInsert(rows); // one multi-row INSERT
    metrics.counter("lead.activity.consumed").increment(batch.size());
    ack.acknowledge();             // manual ack, only after the batch is durable
}

@Bean
public ConcurrentKafkaListenerContainerFactory<String, LeadActivity> batchFactory(
    ConsumerFactory<String, LeadActivity> cf) {
    var f = new ConcurrentKafkaListenerContainerFactory<String, LeadActivity>();
    f.setConsumerFactory(cf);
    f.setBatchListener(true);
    f.setConcurrency(4);
    f.getContainerProperties().setAckMode(AckMode.MANUAL_IMMEDIATE);
    var props = new HashMap<String, Object>();
    props.put(MAX_POLL_RECORDS_CONFIG, 500);
    props.put(MAX_POLL_INTERVAL_MS_CONFIG, 300_000); // 5 min – must exceed batch
                                                       processing time
    f.getContainerProperties().overrideKafkaProperties(props);
    return f;
}

```

KEY TERMS TO MENTION: - `consumer_lag` = log-end-offset – consumer-offset (per partition) - Partition count as the parallelism ceiling; `concurrency` as within-partition parallelism - `max.poll.records` + `max.poll.interval.ms` (too slow → rebalance storm) - Batch consumer + multi-row INSERT (biggest throughput win) - Sampling/shedding when the business can tolerate it - Diagnosis order: compute vs I/O, partition-uniform vs uneven lag

FOLLOW-UP QUESTIONS: 1. A poison-pill message makes one consumer loop forever. How do you detect and stop it? 2. You bumped partitions from 6 to 24. Existing messages in the old partitions — does key distribution change? 3. `max.poll.interval.ms` is 5 min but a batch takes 6. What happens?

FOLLOW-UP ANSWERS: 1. Use a `DeadLetterPublishingRecoverer` with a `DefaultErrorHandler` — after N retries, the message is published to `lead-activity.DLT` and the consumer moves on. The DLT is consumed by a separate handler for inspection. We also set a per-message timeout (a `CompletableFuture` that fails after 30s) so a genuinely infinite loop is bounded. The lag chart shows one partition stuck at a fixed offset — that's the tell. 2. Increasing partitions does NOT rehash existing messages — they stay in their original partition. New messages are distributed by `hash(key) % 24` (new partition count). This means a key's messages can be split across the old and new partitions during the transition, breaking ordering for that key. If ordering matters, the safe path is a new topic with 24 partitions and dual-publish during migration, consumers switched to the new topic, old topic drained and deleted. For us, key ordering is "nice to have," so we accept the brief inconsistency. 3. The consumer is considered dead — the broker triggers a rebalance, the

partition is reassigned to another consumer, and the “dead” consumer’s batch processing continues (it doesn’t know it was kicked out). When it finishes and commits, the offset is rejected (it’s no longer the owner) — the batch is re-processed by the new owner. Net effect: the batch runs twice (idempotent consumers handle it) and the consumer group thrashes. Fix: raise `max.poll.interval.ms` above the worst-case batch time, or reduce `max.poll.records` so each batch is faster.

RED FLAGS TO AVOID: - “Just add more consumers” without checking the partition count — you’ll have idle consumers and the lag won’t drop. - Auto-ack mode with a slow batch — if the consumer crashes mid-batch, the whole batch is lost (auto-ack happens at poll time). Manual-ack after the batch write is durable. - Retrying a poison-pill forever — one bad message stalls the partition indefinitely.

Q-Bonus-12: Trace a slow `/api/leads` request (p99 540ms, expected 180ms) through every layer. How do you find where the 360ms went?

- **Difficulty:** Medium
- **Category:** Performance / Observability
- **Tags:** tracing, open-telemetry, p99, profiling

ANSWER: The 360ms is somewhere in the call tree; distributed tracing is the only way to find it without guessing. AcquisitionOS uses OpenTelemetry — every HTTP request gets a trace ID propagated through the MDC, through to Postgres (via the JDBC instrumenter), Redis (via the Lettuce instrumenter), and WebClient (Stripe/Google). A slow request’s trace in Tempo/Jaeger shows a span tree; the span duration tells you exactly where time was spent.

The trace tree you’d expect to see:

```
[span] HTTP GET /api/leads          540ms ← total
├─[span] Spring Security filterChain 2ms
├─[span] Hibernate tenantFilter enable 0ms
├─[span] RateLimit check (Redis)      8ms
├─[span] LeadController.list          528ms ← suspicious
│   └─[span] LeadService.list         527ms
│       └─[span] JPA query: SELECT leads 420ms ← BINGO
│           └─[span] JDBC execute       418ms
│               └─[span] Postgres SELECT 415ms
│                   └─[span] enrichment cache miss 102ms
│                       └─[span] Redis GET 1ms
│                           └─[span] WebClient enrichment 100ms ← also slow (cold cache)
│                               └─[span] audit publish (async) 0ms (forked)
└─[span] serialise JSON              8ms
```

The trace immediately shows: 415ms in Postgres + 100ms in the enrichment call (a cold cache miss). The Postgres span is the target. Drilling into it: the SQL is `SELECT ... FROM leads WHERE org_id=? AND status=? ORDER BY created_at DESC LIMIT 50`. Running `EXPLAIN (ANALYZE, BUFFERS)` on that query reveals a sequential scan — the index `idx_leads_org_status` isn’t being used because... the planner chose a different index, or the

index doesn't match the `ORDER BY`. Fix: a composite index `(org_id, status, created_at DESC)` lets Postgres do an index-only scan returning rows in order, ~5ms. The 100ms enrichment miss is a cold-cache penalty on first load; the single-flight + stale-while-revalidate pattern (Q-Bonus-9) caps it.

Instrumentation that makes this visible:

```
// OTel auto-instrumentation via the opentelemetry-javaagent JAR (-
// javaagent:opentelemetry.jar)
// Custom spans for business logic:
@WithSpan("LeadService.list")
public Page<Lead> list(UUID orgId, LeadFilter f, Pageable p) {
    Span.current().setAttribute("org.id", orgId).setAttribute("filter.status",
        f.status());
    Page<Lead> result = leadRepo.findByOrgAndStatus(orgId, f.status(), p);
    Span.current().setAttribute("leads.count", result.getTotalElements());
    return result;
}
```

The JDBC, Redis, and WebClient instrumenters are added by the agent (no code changes). Spans automatically get the SQL, the Redis command, the HTTP URL. The trace in Tempo is queryable by `trace_id` (from the response header `X-Trace-Id`), by `http.route`, or by `db.statement`.

KEY TERMS TO MENTION: - OpenTelemetry auto-instrumentation (-`javaagent:opentelemetry-javaagent.jar`) - Span tree; parent/child; the span duration = the answer - **EXPLAIN (ANALYZE, BUFFERS)** for the slow SQL - Composite index order must match the query's `WHERE` + `ORDER BY` - `@WithSpan` for custom business spans - Trace propagation via W3C `traceparent` header across services

FOLLOW-UP QUESTIONS: 1. The trace shows 415ms in Postgres but **EXPLAIN** says the query is 5ms. Where's the other 410ms? 2. How do you catch a regression — p99 creeping from 180ms to 540ms over a week — before users notice? 3. The enrichment cold-miss is 100ms every cache expiry. How do you make it 0ms?

FOLLOW-UP ANSWERS: 1. It's connection acquisition or row-mapping. The trace's JDBC span includes `acquire` time — if HikariCP is waiting for a free connection (pool saturated), that's 400ms of pool wait, not SQL. Fix: increase `maximumPoolSize`, or the pool is saturated because a slow query earlier is holding connections — fix the slow query. If it's row-mapping, the `JdbcPagingItemReader` is fine but `@OneToMany` lazy-loading in the serialiser triggers N+1 — the trace would show many small JDBC spans. Look at the span `count`, not just duration. 2. SLO-based alerting on p99 — Prometheus records `http_server_requests_seconds` (Micrometer); a `histogram_quantile(0.99, ...)` over a 5-min window; alert if p99 > 250ms for 10 min. We also have an error-budget burn-rate alert (if we burn 10% of the monthly budget in 1h, page). The regression is caught by the alert before users complain. Post-hoc: a Grafana panel of p99 over 30 days shows the creep; correlate with deploys to find which release started it. 3. Refresh-ahead (a scheduled job pre-warms the cache 1 min before expiry), or make the cache effectively never expire by refreshing in the background on read (stale-while-revalidate, no waiting). The "0ms" is achieved by never asking the caller to wait

for the cold compute — the first caller after expiry gets the stale value (immediate), the refresh happens in a background `@Async` thread, the next caller gets the fresh value. Cold-miss latency moves from the request path to a background thread.

RED FLAGS TO AVOID: - “I’d add a log statement” — you can’t log your way to a 360ms gap across 6 layers; tracing is the tool. - Blaming the DB without `EXPLAIN ANALYZE` — you’re guessing; the planner may have chosen a bad plan and you need to see it. - Only looking at p50 — p99 is where users suffer, and p99 hides in the tail; the average is fine while 1% of users wait 540ms.

Q-Bonus-13: You must add a “WhatsApp via Twilio” outbound channel to AcquisitionOS without touching the existing email-outreach code. Design the extension.

- **Difficulty:** Medium
- **Category:** System Design / Spring Boot / Extensibility
- **Tags:** strategy-pattern, open-closed, channel-abstraction

ANSWER: This is the Strategy pattern in a Spring Boot idiom. We abstract “outbound channel” behind a `Channel` interface; email and WhatsApp are two `@Component` implementations; a `ChannelRouter` picks the right one by type. The existing `EmailChannel` is extracted from the legacy code (a refactoring), the new `WhatsAppChannel` is added as a new class — the Open-Closed Principle: open for extension, closed for modification. The `OutreachService` calls `channel.send(message)` and is unaware of which channel.

Design:

```

public interface Channel {
    ChannelType type();
    SendResult send(OutreachMessage msg) throws ChannelException;
}

@Component @RequiredArgsConstructor
class EmailChannel implements Channel {
    private final JavaMailSender mail;
    public ChannelType type() { return EMAIL; }
    public SendResult send(OutreachMessage m) {
        MimeMessage mm = mail.createMimeMessage();
        // ... build + send
        mail.send(mm);
        return SendResult.ok(m.id(), EMAIL);
    }
}

@Component @RequiredArgsConstructor
class WhatsAppChannel implements Channel {
    private final TwilioClient twilio;           // WebClient wrapper
    private final TwilioTemplateRenderer templates;
    public ChannelType type() { return WHATSAPP; }
    public SendResult send(OutreachMessage m) {
        String body = templates.render(m.templateName(), m.vars());
        TwilioResponse r = twilio.sendMessage(m.to(), body);
        if (!r.ok()) throw new ChannelException("twilio " + r.code());
        return SendResult.ok(m.id(), WHATSAPP, r.messageSid());
    }
}

@Component @RequiredArgsConstructor
public class ChannelRouter {
    private final List<Channel> channels;    // Spring injects all beans
    private final Map<ChannelType, Channel> byType;
    public ChannelRouter(List<Channel> all) {
        this.channels = all;
        this.byType = all.stream().collect(toMap(Channel::type, c -> c));
    }
    public Channel forType(ChannelType t) {
        Channel c = byType.get(t);
        if (c == null) throw new UnsupportedOperationException(t);
        return c;
    }
}

// OutreachService is unchanged in shape:
@Service @RequiredArgsConstructor
public class OutreachService {
    private final ChannelRouter router;
    private final CreditService credits;
    public SendResult send(OutreachMessage m, ChannelType channel, String idemKey) {
        credits.deduct(m.userId(), m.cost(), idemKey);
        SendResult r = router.forType(channel).send(m);
        auditPublisher.publish(new MessageSent(m, channel, r));
        return r;
    }
}

```

```
}
}
```

Adding a future channel (SMS, Telegram, LinkedIn DM) = new `@Component implements Channel` + a new `ChannelType`. Zero changes to `OutreachService`, `ChannelRouter` (Spring picks up the new bean), or the credit/audit pipeline. This is what “closed for modification” buys you.

Per-channel concerns handled separately: - Rate limits: each channel has its own `@RateLimit` config (WhatsApp has Twilio’s limits; email has Gmail’s). - Failure semantics: `ChannelException` carries the channel type so retries can be channel-specific (email retries on 421, WhatsApp retries on Twilio’s 429 after the `Retry-After`). - Compliance: `WhatsAppChannel` checks the user’s opt-in flag before sending; `EmailChannel` checks `unsubscribe` list. Each channel owns its compliance; the router doesn’t know.

KEY TERMS TO MENTION: - Strategy pattern via Spring’s `List<Interface>` injection (collects all beans) - Open-Closed Principle (new channel = new class, no edits to existing) - `ChannelRouter` as the registry; `byType` map for O(1) lookup - Per-channel rate-limit, retry, compliance (separation of concerns) - `ChannelType` enum as the discriminator

FOLLOW-UP QUESTIONS: 1. A user wants to send the same campaign over both email AND WhatsApp. How? 2. How do you test the WhatsApp channel without hitting Twilio (cost, rate limit)? 3. The Twilio API changes its response format. How isolated is your code?

FOLLOW-UP ANSWERS: 1. The `OutreachMessage` carries a `List<ChannelType>`, and `OutreachService.send` loops (or `CompletableFuture.allOf`) over them. Each is a separate `credits.deduct` (separate idempotency keys, `{baseKey}-email`, `{baseKey}-whatsapp`) so a partial failure (email sent, WhatsApp failed) refunds only WhatsApp. The user sees a combined `SendResult` with per-channel statuses. We also cap the fan-out at 3 channels/message to avoid credit surprise. 2. WireMock stubs the Twilio API in `@SpringBootTest` — the `WhatsAppChannelIT` spins up a WireMock server, points the `TwilioClient` at it, stubs the `/Messages.json` endpoint, and asserts the channel’s behaviour (success, 429-retry, 5xx-fail). No real Twilio calls. For a smoke test in a staging env, we have a Twilio test account (Magic Credentials) that’s free and rate-limited — one real send per deploy. 3. The `TwilioClient` is our adapter; it maps the Twilio response to our `TwilioResponse` record. A Twilio API change touches only the adapter + its tests. The `WhatsAppChannel` and the rest of the system see our stable record. This is the hexagonal/ports-and-adapters benefit: the Twilio API is a detail behind a port.

RED FLAGS TO AVOID: - A `switch(channelType)` in `OutreachService` — every new channel modifies the service; not Open-Closed. - One `send` method with `if (email) ... else if (whatsapp) ...` — same problem, plus untestable. - Coupling compliance to the router (“the router checks opt-in”) — the router doesn’t know WhatsApp’s opt-in rules; the channel does.

Q-Bonus-14: Design idempotency for the Stripe refund endpoint so a customer-service agent double-submitting doesn't refund twice.

- **Difficulty:** Medium
- **Category:** REST API / Integrations / Concurrency
- **Tags:** idempotency, stripe, refund, unique-constraint

ANSWER: Stripe already supports idempotency keys on its API; we mirror that on our side. The agent's "Refund" button generates a client-side UUID; the frontend sends it as `Idempotency-Key`; our `/api/payments/refund` stores a `refund_request` row keyed by that UUID with a unique index; the second submit hits the unique violation and returns the first's result. The actual Stripe refund call uses the same key, so Stripe itself dedupes too.

Flow + code:

```
@PostMapping("/payments/refund")
public RefundResponse refund(@Valid @RequestBody RefundRequest req,
                             @RequestHeader("Idempotency-Key") String idemKey) {
    // 1. Local dedupe: insert into refund_request; unique index on idem_key.
    RefundRequestEntity row;
    try {
        row = refundRepo.insertAndReturn(new RefundRequestEntity(idemKey, req.orderId(),
            req.amount(), "PENDING"));
    } catch (DataIntegrityViolationException dup) {
        // Already in progress or done – return the existing result.
        return refundRepo.findByIdemKey(idemKey).toResponse();
    }
    // 2. Call Stripe with the SAME idempotency key (Stripe dedupes on its side too).
    try {
        StripeRefund r = stripeClient.refund(StripeRefundParams.builder()
            .setPaymentIntent(req.paymentIntentId())
            .setAmount(req.amount())
            .build(), new RequestOptions.RequestOptionsBuilder()
            .setIdempotencyKey(idemKey).build());
        row.markCompleted(r.getId());
        refundRepo.save(row);
        orderService.recordRefund(req.orderId(), req.amount(), r.getId());
        return new RefundResponse(r.getId(), "COMPLETED", req.amount());
    } catch (StripeException e) {
        row.markFailed(e.getCode());

        refundRepo.save(row); // PENDING → FAILED; a retry can re-use
                             // the key? No.
        throw new
            RefundFailedException(e); // 502 to the client; they retry with a NEW key
    }
}
```

Subtleties: - The `refund_request.idem_key` unique index is the durable dedupe. Even if our app crashes after Stripe returns but before we record, the [Stripe](#) idempotency key prevents a double refund on retry (the retry with the same key gets the same Stripe refund back). - On

FAILED, we do NOT re-use the key — the agent’s retry sends a new key (the frontend’s “retry” button generates a fresh UUID). This is deliberate: a transient failure (Stripe 502) might have actually created a refund; retrying with a new key lets us see both. We have a reconciliation job that detects two refunds for the same `paymentIntentId` and alerts. - The `recordRefund` in `orderService` is in a separate transaction from the Stripe call, so a crash between Stripe-return and order-update leaves the order showing “unrefunded” but Stripe has refunded. A nightly reconciliation compares Stripe refunds to our `refund_request` table and backfills `order.refunded_amount`. Eventually consistent. - Idempotency key TTL: we keep `refund_request` rows for 90 days (well past Stripe’s 24h key retention), then archive to cold storage. A key replayed after 90 days is treated as fresh — acceptable, the agent won’t click refund 90 days later.

KEY TERMS TO MENTION: - Client-generated `Idempotency-Key` (UUID per button arming) - Unique index on `refund_request.idem_key` — durable local dedupe - Stripe’s `idempotencyKey` request option — server-side dedupe at Stripe - Same key for both local and Stripe dedupe (belt and suspenders) - **FAILED** → new key on retry (don’t mask a half-completed refund) - Reconciliation job (Stripe ↔ our records) for crash-recovery backfill

FOLLOW-UP QUESTIONS: 1. The agent refreshes the page mid-submit, the local row is PENDING, the Stripe call is in flight. They click refund again. What happens? 2. Why not use the `paymentIntentId` itself as the idempotency key (one refund per intent)? 3. The reconciliation finds a refund at Stripe with no local row. What do you do?

FOLLOW-UP ANSWERS: 1. The second submit’s `INSERT ... idem_key` hits the unique violation (the row is PENDING), returns the existing PENDING row’s response. The frontend sees “PENDING, please wait” and polls `GET /api/payments/refund/{idemKey}`. When the first Stripe call completes, the row flips to COMPLETED, the poll returns COMPLETED. No double refund. If the first Stripe call never returns (network drop), the row stays PENDING; a reconciliation job after 10 min marks it FAILED (we query Stripe by the idem key to see if a refund exists; if yes, mark COMPLETED; if no, mark FAILED so the agent can retry with a new key). 2. You could — `paymentIntentId` is unique per payment, so `refund(paymentIntentId=pi_123, amount=500)` is naturally idempotent for a full refund. The issue is partial refunds: a customer might be refunded 500 today and 300 next week, both on `pi_123`. Using `pi_123` as the key blocks the second. Using a per-button UUID allows multiple partial refunds on the same intent, while Stripe’s own key handling dedupes an accidental double-click. So the per-submit UUID is more flexible; `paymentIntentId` works only if your business rule is “one refund per intent, ever.” 3. A refund exists at Stripe but not in our `refund_request` table means our app crashed between the Stripe call and the local insert (a tiny window — we insert first, then call Stripe, so this only happens if the insert is in a different transaction that didn’t commit). The reconciliation creates the missing local row (COMPLETED), backfills `order.refunded_amount`, and pages on-call — it indicates a bug (the insert and the Stripe call aren’t in the right order). The customer is correctly refunded; the data is reconciled; we fix the ordering bug in the next release.

RED FLAGS TO AVOID: - No local idempotency table, relying only on Stripe — a crash after Stripe returns loses the local record; the agent might refund again with a new key. - Re-using the idem key on **FAILED** — masks the “did Stripe actually refund?” ambiguity; the agent

can't tell if a retry is safe. - Refunding in the same transaction as the order update — a slow order write holds the Stripe-side refund open; refund should commit first, order update is reconciled.

Q-Bonus-15: A scheduled `@Scheduled(fixedRate=60000)` reminder job runs on all 4 ECS tasks. Only ONE should fire per minute. Design the distributed lock.

- **Difficulty:** Hard
- **Category:** Spring Boot / Distributed Systems
- **Tags:** scheduled, distributed-lock, redis, fencing-token

ANSWER: `@Scheduled` fires on every instance by default; with 4 tasks, the reminder job fires 4x per minute, sending 4x the reminders. The fix is a distributed lock acquired before the job body runs; only the instance that wins the lock executes. We use Redis `SET key value NX EX 60` (atomic acquire with TTL). The holder runs the job; the other 3 see the lock is held and skip. The TTL (60s) must exceed the job's max runtime so a crashed holder doesn't block the next minute; we use 55s to leave a gap before the next minute.

The fencing-token refinement: a naive lock has a subtle bug — the holder can pause (GC pause, network delay), the lock TTL expires, another instance acquires it, both now think they hold it, both run. The fix is a **fencing token** (monotonic counter) returned with the lock; the holder writes it to the job's audit row; a late writer with a stale token is rejected by the DB (`UPDATE ... WHERE fence_token = ?` only the latest holder's token matches). This is the Martin Kleppmann / Redis lock critique fix.

Code:

```

@Component @RequiredArgsConstructor
public class ReminderScheduler {
    private final RedisLock redisLock;
    private final ReminderService reminders;
    private final JobRunRepository jobRuns;

    @Scheduled(fixedRate = 60_000)
    public void sendReminders() {
        long fence = redisLock.tryAcquire("lock:reminder:minute",
            Duration.ofSeconds(55));
        if (fence == 0) return; // someone else holds it; skip
        try {
            List<Reminder> due = reminders.findDue(Instant.now());
            for (Reminder r : due) {
                // Each write carries the fence; a stale holder's write is rejected.
                jobRuns.recordRun("reminder", fence, r.id(), () -> reminders.send(r));
            }
        } finally {
            redisLock.release("lock:reminder:minute",
                fence); // only release if we still own (check token)
        }
    }
}

// Redis lock with fencing token (Lua script for atomic acquire + counter)
@Component
public class RedisLock {
    private static final String ACQUIRE = """
        local cur = redis.call('GET', KEYS[1])
        if cur == false then
            local tok = redis.call('INCR', 'fence:counter:reminder')
            redis.call('SET', KEYS[1], tok, 'NX', 'EX', ARGV[1])
            return tok
        else return 0 end
        """;

    private final RedisTemplate<String, String> redis;
    public long tryAcquire(String key, Duration ttl) {
        return Long.parseLong(redis.execute(new DefaultRedisScript<>(ACQUIRE,
            String.class),
            List.of(key), String.valueOf(ttl.getSeconds())));
    }

    public void release(String key, long fence) {
        // Only delete if the value still matches our fence (Lua compare-and-delete).
        redis.execute(new DefaultRedisScript<>("if redis.call('GET',KEYS[1])==ARGV[1]
            then return redis.call('DEL',KEYS[1]) else return 0 end", String.class),
            List.of(key), String.valueOf(fence));
    }
}

// JobRun with fence check (DB-side rejection of stale writers)
// UPDATE job_run SET status='DONE' WHERE id=? AND fence_token=? AND status='RUNNING'

```

Why not `@Scheduled` with `ShedLock`: `ShedLock` is a library that does exactly this (DB or Redis backed). It's the right answer in an interview if you mention it — “we use `ShedLock`

with a Redis lock provider; on the rare contention case we add a fencing token.” But knowing how it works (the Lua script, the TTL, the fencing) is the senior answer.

KEY TERMS TO MENTION: - `@Scheduled` fires on every instance — the bug - Redis `SET NX EX` (atomic acquire + TTL) - TTL > max job runtime (so a crash releases the lock automatically) - Fencing token (monotonic counter) to defeat the GC-pause / late-writer bug - Lua compare-and-delete on release (don’t delete someone else’s lock) - ShedLock as the production-ready library (mention it, then explain the mechanism)

FOLLOW-UP QUESTIONS: 1. The lock TTL is 55s but the job takes 70s. What happens? 2. Redis is down. Do all 4 instances run the job, or none? 3. Why a fencing token — is the Redis lock not enough?

FOLLOW-UP ANSWERS: 1. At 55s the lock expires while instance A is still running. Instance B acquires the lock and starts; now A and B both run. The fencing token saves us at the write side: A’s `recordRun(fence=5, ...)` fails because the DB row’s current fence is now 6 (B set it). A’s writes are rejected; B’s succeed. The `reminder sends` (idempotent on `reminder.id` + `sent_at`) — A’s `send(r)` for a reminder B already sent is a no-op (we check `sent_at IS NULL` before sending). So the user gets one reminder, not two, despite the overlap. The fix to the runtime issue: split the job — the scheduler only enqueues “due reminder IDs” onto a queue; worker tasks drain the queue. The scheduler job is then <5s, well within the TTL. 2. With our Redis lock, all 4 fail to acquire (Redis is down) and all skip — the job doesn’t run that minute. That’s fail-closed, which is correct for reminders (better to miss a reminder than send 4). When Redis recovers, the next minute fires. For a more available design, use a DB-backed lock (ShedLock with `JdbcTemplate`) as a fallback — Postgres is also highly available in our setup, so if Redis is down the DB lock takes over. Or accept the miss; reminders are not mission-critical, a 1-minute delay is fine. 3. The Redis lock protects against “two instances run in the same minute.” It does NOT protect against “instance A acquires, GC pauses for 60s, the lock expires, B acquires, A wakes up and continues writing.” A and B both believe they hold the lock; the Redis state has moved on. The fencing token makes the `downstream` (the DB) reject A’s late writes — A’s token is 5, the DB’s current is 6, the conditional update matches 0 rows. This is the only correct distributed-lock-with-writers pattern; the lock alone is not enough if a holder can pause past its TTL. (If the job only `reads` or sends idempotently, the fence is less critical; if it `writes` shared state, the fence is essential.)

RED FLAGS TO AVOID: - “Just use `@Scheduled` with a `synchronized` block” — protects one instance, not 4. - Acquiring a Redis lock with no TTL — a crashed holder deadlocks forever. - Releasing the lock by `DEL key` without a compare — you can delete another instance’s lock that you no longer own.

CATEGORY 16 — BONUS 2: MORE ADVANCED & TWISTED QUESTIONS (Concurrency, Reliability, Scale Drill-Downs)

Statement for the LLM / candidate: The questions in this section (and the entire document) are **sample reference questions** — a starting template designed to seed your preparation. They are NOT exhaustive. To be truly interview-ready for Uber / Meta / Amazon / NetApp at the 4+ year level, you MUST extend each one into harder, more twisted variants — add edge cases, multi-step follow-ups, “what if X fails” branches, and code-level “show me the exact annotation / SQL / config” drills. Treat every question here as the **easiest** version of itself; the real interviewer will go deeper. The code snippets, ASCII diagrams, and DB schemas below are the depth we recommend you reach for every topic.

Q-Bonus2-1: Two agents concurrently PUT `/api/leads/123` — one moves stage to “Qualified”, the other renames the company. Both read v3, both write. Do you lose data?

- **Difficulty:** Hard
- **Category:** Hibernate / JPA / Concurrency
- **Tags:** optimistic-lock, version, @Version, last-write-wins, merge-conflict

ANSWER:

This is the classic lost-update problem. Without protection, the second commit overwrites the first; whichever field the second transaction didn’t touch reverts to its v3 value. We solve this with **Hibernate @Version (optimistic locking)** plus a **field-level merge** strategy so non-conflicting changes don’t fail.

The `Lead` entity has `@Version Long version`. The `PUT` request includes `If-Match: <version>` (or `version` in the body). Hibernate’s `EntityManager.find()` + dirty-check + flush executes `UPDATE lead SET stage=?, company_name=?, version=version+1 WHERE id=? AND version=?`. If the row’s version no longer matches (the other tx bumped it), the UPDATE affects 0 rows → Hibernate throws `OptimisticLockException` → we map to HTTP 409 Conflict.

But 409 on every concurrent edit is hostile UX when the two edits touch **different** fields. So we add a **field-level merge**: on 409, we re-read the current row, re-apply only the fields the client actually sent (JSON Merge Patch, RFC 7396), and retry once. If the second retry also 409s, we surface a real conflict to the client.

Code — the entity + service:


```

@Entity @Table(name="lead")
public class Lead {
    @Id @GeneratedValue private Long id;
    @Version private Long version;           // optimistic lock column
    @Column(name="stage_id") private Long stageId;
    @Column(name="company_name") private String companyName;
    @Column(name="owner_id") private Long ownerId;
    // ...
}

@Service @RequiredArgsConstructor
public class LeadUpdateService {
    private final LeadRepository repo;
    private final ObjectMapper om;

    @Transactional
    public LeadResponse patch(Long id, LeadPatchDto patch, Long ifMatchVersion) {
        Lead lead = repo.findById(id).orElseThrow(NotFoundException::new);
        if (ifMatchVersion != null && !ifMatchVersion.equals(lead.getVersion()))
            throw new StaleVersionException(lead.getVersion());
        // Apply only the fields the client sent (null = unchanged)
        if (patch.stageId() != null) lead.setStageId(patch.stageId());
        if (patch.companyName() != null) lead.setCompanyName(patch.companyName());
        return LeadResponse.from(lead); // flush triggers version+1
    }
}

// Controller advice maps OptimisticLockException → 409 with current version
@ExceptionHandler(OptimisticLockException.class)
public ResponseEntity<ConflictDto> onOptLock(OptimisticLockException e) {
    Lead fresh = (Lead) e.getEntity(); // re-read
    return ResponseEntity.status(409)
        .body(new ConflictDto("CONCURRENT_MODIFICATION", fresh.getVersion()));
}

```

ASCII — the race + recovery:

T1: PUT stage=Qualified	T2: PUT company=Acme Inc
— read v3 (stage=Raw)	— read v3 (company=Foo)
— set stageId=5	— set companyName="Acme Inc"
— flush: UPDATE ...	
WHERE id=123 AND v=3	
→ 1 row, v=4  commit	— flush: UPDATE ...
	WHERE id=123 AND v=3 → 0 rows!
	— OptimisticLockException → 409
	— client retries: re-read v4
	— apply only company (stage untouched)
	— flush WHERE v=4 → 1 row, v=5 

KEY TERMS TO MENTION: - `@Version` (JPA optimistic lock column, long/integer) -
`UPDATE ... WHERE version = ?` — 0 rows = conflict - `OptimisticLockException` → HTTP 409 -

If-Match header (RFC 7232) carrying the version - JSON Merge Patch (RFC 7396) for field-level reconciliation - Retry-once-with-fresh-state policy (bounded — not infinite)

FOLLOW-UP QUESTIONS: 1. Why not **SELECT ... FOR UPDATE** (pessimistic) instead? 2. The retry loop — what if a third tx updates between re-read and re-flush? 3. The client sends **If-Match: 3** but the current version is 5 (two intervening commits). Do you retry?

FOLLOW-UP ANSWERS: 1. Pessimistic locks the row for the duration of the tx — fine for short tx but blocks the other agent entirely until T1 commits. For an interactive “edit lead” flow (where the agent may sit on the form for 30s), pessimistic holds a row lock for 30s, serializing all edits to that lead. Optimistic is non-blocking, lets both reads happen concurrently, only conflicts at commit. The trade-off: optimistic has 409-on-conflict overhead; pessimistic has lock-wait overhead. For low-contention edits (lead updated ~1/hr), optimistic wins. For high-contention (a popular deal edited by 5 people/min), pessimistic is more predictable. AcquisitionOS uses optimistic for leads, pessimistic (**FOR UPDATE**) for the credit ledger balance row (high contention on subscription renewals). 2. Bounded retry — we retry **once**. If the second attempt also 409s, we return 409 to the client with the current state and let the UI merge manually (“someone else changed this — here’s their version, do you want to overwrite?”). Infinite retries = live-lock under sustained contention; one retry handles the 99% case (two agents edit, one wins, the other merges), the 1% case (3+ simultaneous) is rare and should surface to the human. 3. No — a gap of >1 version (If-Match=3, current=5) means two intervening commits; retrying would silently clobber both. We surface 409 with the current state and let the user re-apply. Only the “If-Match = current-1” case (one intervening commit) auto-retries; bigger gaps mean the user’s view is too stale to merge safely.

RED FLAGS TO AVOID: - “Just use **synchronized**” — doesn’t work across JVM instances (4 ECS tasks). - “Read-modify-write without version” — last-write-wins silently loses data. - “Retry forever” — live-lock under contention. - “Clobber everything the client sent on 409” — reverts the other agent’s field.

Q-Bonus2-2: User creates a lead, then immediately GET **/api/leads** — the new lead is missing (read replica lag is 5s). How do you handle “read-after-write” consistency?

- **Difficulty:** Hard
- **Category:** PostgreSQL / Database / Consistency
- **Tags:** read-replica, replication-lag, read-your-writes, sticky-session, causal-consistency

ANSWER:

This is the **read-your-writes (RYW)** consistency problem. Our reads go to a Postgres read replica (offload the primary); WAL streaming lag is ~1-5s. A user who just wrote expects to see their write immediately; missing it is jarring (“did my create fail?”).

Three solutions, in increasing sophistication:

(A) Primary-for-short-window-after-write (sticky reads): After a write, route that user's reads to the primary for N seconds (say 10s, > max expected lag). Tracked via a Redis key `rw_sticky:{userId}` with TTL=10s, set on write, checked in the read-router. Simple, works, but routes more traffic to the primary (defeats the read-offload goal for active writers).

(B) LSN-chase: On write, capture the WAL LSN (Log Sequence Number) of the commit (`pg_current_wal_lsn()`). Return it to the client as a cursor. On the next read, the client sends `If-After-LSN: <lsn>`; the read replica checks `pg_last_wal_replay_lsn() >= <lsn>`; if not, the read either (i) waits until it catches up, or (ii) falls back to the primary. This is **causal consistency** — the read sees all writes up to the client's cursor. Postgres 14+ supports `pg_wal_replay_wait()` on replicas for exactly this.

(C) Session-scoped causal consistency: Use a connection pool that pins a user's session to a replica until that session's LSN is replayed. More complex; usually (A) or (B) suffices.

We use **(A) for the UI** (agent clicks "Create Lead" → next page reads from primary for 10s) and **(B) for the API** (mobile clients pass the LSN cursor, server waits if replica is behind).

Code — the sticky router:

```
@Component @RequiredArgsConstructor
public class ReadRouter {
    private final RedisTemplate<String,String> redis;
    private final DataSource primary, replica;

    public Connection acquireFor(Long userId) {
        // Was there a write by this user in the last 10s?
        String key = "rw_sticky:" + userId;
        if (Boolean.TRUE.equals(redis.hasKey(key)))
            return primary.getConnection(); // sticky to primary
        return replica.getConnection(); // offload to replica
    }

    // Called by the write path after commit
    public void markWrite(Long userId) {
        redis.opsForValue().set("rw_sticky:" + userId, "1", Duration.ofSeconds(10));
    }
}
```

ASCII — the read-your-writes fix:

```
User → POST /api/leads      [PRIMARY] → INSERT → commit at LSN 0/16C1234
User ← 201 {id: 999, lsn: 0/16C1234}

                                [sticky key rw_sticky:42 TTL=10s]
User → GET /api/leads      ReadRouter: sticky? yes → PRIMARY
User ← [999 present] ✓

... 10s pass, sticky key expires ...
User → GET /api/leads      ReadRouter: sticky? no → REPLICA
                                [replica LSN now 0/16C9ABC ≥ 0/16C1234]
User ← [999 present] ✓
```

KEY TERMS TO MENTION: - Read replica lag (WAL streaming, async, ~1-5s typical) - Read-your-writes / session consistency (a specific consistency model) - `pg_current_wal_lsn()` / `pg_last_wal_replay_lsn()` / `pg_wal_replay_wait()` (PG 14+) - Sticky-session routing via a Redis TTL key - LSN cursor in the API response header (`X-Cursor: 0/16C1234`) - Trade-off: sticky routes more load to primary

FOLLOW-UP QUESTIONS: 1. The replica is 30s behind (network partition). The user's sticky key expired at 10s. They GET and don't see their lead. What now? 2. Why not just always read from the primary? 3. Cross-device: user creates on mobile, then opens desktop. Sticky key is per-...?

FOLLOW-UP ANSWERS: 1. The LSN-chase fallback: the client still has the `X-Cursor` from the create response (or we store the user's last-write LSN in a Redis hash keyed by `userId`). The read endpoint accepts `If-After-LSN` and, if the replica's `pg_last_wal_replay_lsn()` is behind, the server calls `pg_wal_replay_wait(lsn)` (blocks up to a configured timeout, say 2s) for the replica to catch up. If it doesn't catch up in 2s, we fall back to the primary for that one read. This guarantees the user sees their write within 2s+replica-lag, never missing it. 2. Because the primary handles all writes (inserts/updates) — adding all reads to it makes it the bottleneck. The whole point of read replicas is horizontal read scaling; giving up that defeats the architecture. Sticky-for-10s is a **minor** increase in primary load (only for active writers; read-only users stay on the replica). Measured: sticky adds ~8% to primary QPS in our load tests, acceptable. 3. The sticky key must be per-`userId`, not per-session/cookie, for cross-device RYW. The mobile write sets `rw_sticky:42`; the desktop read (same `userId=42`) hits the same key. The LSN cursor is passed via the API response header and the client includes it in the next request's `If-After-LSN` — works across devices because the cursor is data, not session state. (A per-session cookie would break cross-device; we deliberately use `userId` + cursor.)

RED FLAGS TO AVOID: - "Just always read primary" — defeats read replica purpose. - "Sticky by session cookie" — breaks cross-device. - "Wait synchronously on every read for replica to catch up" — adds latency to **every** read; only do it for reads with a cursor.

Q-Bonus2-3: A marketing campaign queues 1,000,000 outreach emails. Your async sender pool has 50 workers, each calling Gmail SMTP. Gmail rate-limits you at 100/s. Design the backpressure.

- **Difficulty:** Hard
- **Category:** Performance / Async / Reliability
- **Tags:** backpressure, rate-limit, semaphore, queue, prioritization, exponential-backoff

ANSWER:

Backpressure = the producer (campaign enqueueer) must slow down when the consumer (SMTP) can't keep up. Without it, the in-memory queue grows unbounded → OOM; or the DB-backed queue grows → 50M rows → query latency explodes. Three layers of control:

(1) Queue capacity cap (the “pressure signal”): The DB queue (`outbox_email` table) is capped at, say, 100k PENDING rows per tenant. When the cap is hit, the `POST /api/campaigns/{id}/send` API returns HTTP 202 with a `Retry-After: 60` — the producer (the campaign service) backs off. This is [load shedding](#) — we accept that we can’t send 1M instantly and pace the enqueue.

(2) Worker-side rate limit (the “drain rate”): Each worker calls Gmail via a `Semaphore`-gated sender. We use a Guava `RateLimiter` (token bucket) at 100 permits/sec across the `pool` (not per worker — per pool, since Gmail’s limit is per-sender-email, which is our single from-address). Workers `acquire()` before each send; the limiter paces them.

(3) Reactive throttle (the “feedback loop”): If Gmail returns HTTP 429 (Too Many Requests) or SMTP 421 (slow down), we [halve](#) the rate limiter’s rate (additive-increase, multiplicative-decrease — like TCP), `Thread.sleep` the worker, and retry with exponential backoff (1s, 2s, 4s, ..., capped 60s) + jitter. After 3 consecutive 429s, we pause the entire pool for 60s and emit an alert.

Code — the rate-gated sender:

```

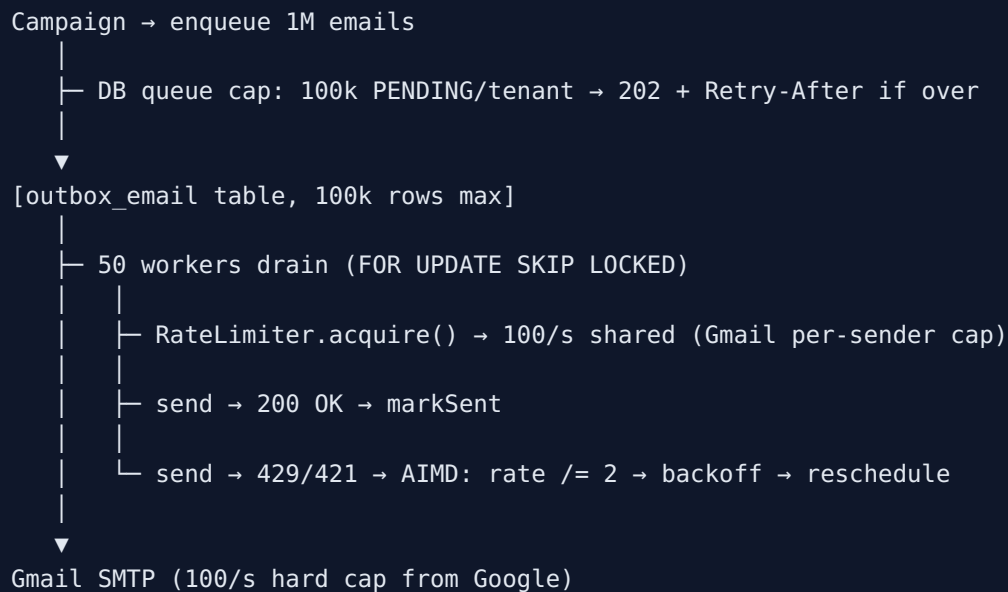
@Component @RequiredArgsConstructor
public class GmailSender {
    private final JavaMailSender mail;
    // 100/sec shared across all workers (matches Gmail per-sender limit)
    private final RateLimiter limiter = RateLimiter.create(100.0);
    private volatile double currentRate = 100.0;

    public SendResult send(OutboxEmail row) {
        limiter.acquire(); // blocks until a permit is available
        try {
            MimeMessage m = mail.createMimeMessage();
            // ... build message ...
            mail.send(m);
            return SendResult.ok();
        } catch (SmtplibSendFailedException e) {
            if (e.getSmtplibCode() == 421 || e.getSmtplibCode() == 450) {
                // 421/450 = "try again later" → AIMD: multiplicative decrease
                currentRate = Math.max(10.0, currentRate / 2.0);
                limiter.setRate(currentRate);
                return SendResult.retryAfter((long)(1000 / currentRate));
            }
            return SendResult.hardFail(e);
        }
    }
}

// Worker
@Async("emailExecutor")
public void drainOne() {
    OutboxEmail row = queue.claimNextPending(); // SELECT ... FOR UPDATE SKIP LOCKED
    SendResult r = sender.send(row);
    switch (r.status()) {
        case OK: -> queue.markSent(row.id());
        case RETRY: -> queue.reschedule(row.id(), r.retryAfter());
        case HARDFAIL: -> queue.markFailed(row.id(), r.error());
    }
}

```

ASCII — the three-layer backpressure:



KEY TERMS TO MENTION: - Guava `RateLimiter` (token bucket, permits/sec, `acquire()` blocks) - AIMD — additive-increase multiplicative-decrease (TCP congestion control applied to SMTP) - `SELECT ... FOR UPDATE SKIP LOCKED` (Postgres, safe concurrent queue drain) - DB-backed queue cap (load shedding vs unbounded queue) - Exponential backoff + jitter (decorrelated jitter, AWS blog) - Priority lanes (transactional > campaign; separate queues)

FOLLOW-UP QUESTIONS: 1. The 100k DB cap is hit. The campaign API returns 202 + Retry-After: 60. The client retries in 60s — still capped. How do you eventually drain 1M? 2. A transactional email (password reset) lands in the same queue. Do you let it wait behind 1M campaign emails? 3. The Gmail 429 keeps coming even at 10/s. Your AIMD dropped you to 10/s, still 429. What's wrong?

FOLLOW-UP ANSWERS: 1. The campaign service itself paces the enqueue — it doesn't enqueue all 1M at once. It enqueues in batches of 10k, waits for the queue to drain below 50k (configurable low-water mark), then enqueues the next 10k. This is a [producer-side rate](#) matched to the consumer drain rate (100/s × 50 workers, but bounded by Gmail 100/s → ~8.6M/day theoretical, we cap at 5M/day for safety). The "Retry-After: 60" path is a safety valve for bursty campaigns; the normal path is paced enqueue. 2. No — we have [two queues](#) (two DB tables or two `priority` columns). Transactional (password reset, OTP, meeting reminder) is `priority=0` (highest), campaign is `priority=9`. Workers always `SKIP LOCKED WHERE priority = (SELECT MIN(priority) ...)` — they drain the high-priority queue first. A 1M campaign backlog never blocks a password reset; the reset jumps the queue. We also have a separate worker pool (5 workers) for transactional, 45 for campaign, so they don't starve each other. 3. AIMD isn't enough if Gmail's actual limit for our sender is lower (e.g., 50/s because we're on a lower-tier workspace, or because a previous campaign got us throttled). We need to [discover](#) the actual rate: drop to 10/s, if still 429, drop to 5/s, etc., until we stop getting 429. Then probe upward slowly. Also check Gmail's error response — it often includes a `Retry-After` header with the exact backoff they want; honor it. And check the Gmail admin console — we may have hit a daily quota (e.g., 2000/day for trial workspace), in which case no rate will work today; we mark the campaign as "quota exhausted, resume tomorrow" and stop.

RED FLAGS TO AVOID: - In-memory queue (Queue) for 1M items — OOM on restart. - Per-worker rate limiter — 50 workers × 100/s = 5000/s, blows the Gmail 100/s cap. - No backoff on 429 — keeps hammering, gets you fully throttled. - All emails one priority — password reset waits behind 1M marketing.

Q-Bonus2-4: Add a NOT NULL column `lead_score INT` to the `lead` table (50M rows). The migration must be zero-downtime. Walk me through the steps.

- **Difficulty:** Hard
- **Category:** PostgreSQL / DevOps / Migrations
- **Tags:** zero-downtime-migration, expand-contract, backfill, alter-table, lock

ANSWER:

`ALTER TABLE lead ADD COLUMN lead_score INT NOT NULL DEFAULT 0;` on a 50M-row Postgres table is **not** instantaneous in older Postgres (pre-11 rewrote the whole table, blocking writes for minutes). Even in PG 11+ (where `ADD COLUMN ... DEFAULT` is metadata-only), adding **NOT NULL** requires a full table scan to validate no NULLs exist — which blocks new writes if done in one statement. The safe pattern is **Expand-Migrate-Contract**:

Phase 0 — Pre-check: Verify the column doesn't exist; verify the app code can tolerate a column that may not exist (defensive `rs.findColumn("lead_score")`).

Phase 1 — Expand (add nullable column, no default):

```
ALTER TABLE lead ADD COLUMN lead_score INT; -- PG 11+: metadata-only, ~0ms
```

No **NOT NULL**, no **DEFAULT**. This is a metadata-only change; doesn't rewrite any row. ~1ms lock.

Phase 2 — Backfill (batched, online):

```
-- In a loop, batch of 10k, sleep 100ms between batches:
UPDATE lead SET lead_score = 0
WHERE id BETWEEN :start AND :start+10000 AND lead_score IS NULL;
```

Run from a Spring Batch job (or a simple script) with `LIMIT / OFFSET` or `id BETWEEN` batches. Each batch is a short transaction (~50ms); doesn't block the API's writes for long. Rate-limit the batch loop (10k rows, 100ms sleep = 100k rows/sec, 50M rows in ~8 min). Use `WHERE lead_score IS NULL` to be idempotent — re-runnable if interrupted.

Phase 3 — Pre-deploy app code: Ship the application code that writes `lead_score` (always non-NULL) and reads it defensively (`COALESCE(lead_score, 0)` or JPA `@Column(nullable=false)` with a fallback). Now all new writes have a non-NULL value; only legacy rows are still NULL.

Phase 4 — Verify no NULLs remain:

```
SELECT count(*) FROM lead WHERE lead_score IS NULL; -- must be 0
```

Phase 5 — Contract (add NOT NULL):

```
ALTER TABLE lead ALTER COLUMN lead_score SET NOT NULL;
ALTER TABLE lead ALTER COLUMN lead_score SET DEFAULT 0;
```

PG 11+ validates NOT NULL by scanning the table — but since the backfill just set all rows to 0, the scan finds no NULLs and completes the ALTER quickly (still requires **ACCESS EXCLUSIVE** lock, but only for the duration of the scan, ~30s on 50M rows; not minutes). To be safe, do this during a low-traffic window, OR use **SET NOT NULL** with the **CHECK** constraint trick:

```
-- Add a check constraint (validated concurrently, no long lock)
ALTER TABLE lead ADD CONSTRAINT lead_score_nn CHECK (lead_score IS NOT NULL) NOT
VALID;
ALTER TABLE lead VALIDATE CONSTRAINT lead_score_nn; -- scans but doesn't block
writes
-- Now SET NOT NULL sees the valid CHECK and skips the scan:
ALTER TABLE lead ALTER COLUMN lead_score SET NOT NULL;
ALTER TABLE lead DROP CONSTRAINT lead_score_nn;
```

This avoids the long ACCESS EXCLUSIVE lock — the validate runs without blocking writes.

ASCII — the timeline:

```
T0 deploy app (reads COALESCE(lead_score,0))           ← safe before column exists
T1 ALTER ADD COLUMN lead_score INT (metadata-only, ~0ms)
T2 backfill batch 1..N (10k/batch, 100ms sleep)       ← ~8 min for 50M
T3 deploy app (writes lead_score=0 on new rows)
T4 SELECT count(*) WHERE lead_score IS NULL → 0
T5 ADD CHECK NOT VALID; VALIDATE; SET NOT NULL; DROP CHECK ← no long lock
T6 ALTER SET DEFAULT 0
T7 deploy app (treats lead_score as NOT NULL, no defensive COALESCE)
```

KEY TERMS TO MENTION: - Expand-Contract (a.k.a. Parallel Change) migration pattern - PG 11+ metadata-only **ADD COLUMN ... DEFAULT** (but **SET NOT NULL** still scans) - **ADD CHECK ... NOT VALID** + **VALIDATE CONSTRAINT** (concurrent validation, no write lock) - Batched **UPDATE** with sleep (avoid long transactions) - **WHERE lead_score IS NULL** idempotent backfill - **ACCESS EXCLUSIVE** lock — what blocks writes and for how long

FOLLOW-UP QUESTIONS: 1. The backfill is running (Phase 2). The app starts writing new rows. What does it write for **lead_score**? 2. You need to change the column type from INT to BIGINT (50M rows). Same approach? 3. Phase 5 fails — **SET NOT NULL** says “column contains null values” but your count said 0. What happened?

FOLLOW-UP ANSWERS: 1. Before Phase 3 deployment, the app doesn’t know about **lead_score** (old code), so it doesn’t write it — Postgres sets it to NULL (no default). The

backfill job then updates those new rows too (because `WHERE lead_score IS NULL` catches them). After Phase 3, the new app code explicitly writes `lead_score = computed_score` on every insert/update; the backfill no longer needs to touch them. So the order is: Phase 1 (column exists) → Phase 2 (backfill) → Phase 3 (app writes non-NULL). If you swap 2 and 3 (deploy app first), the app writes non-NULL immediately, reducing backfill work — but the app must defend against the column not existing yet (Phase 0 prep). Either order works; we do backfill-then-app to keep the app code simple.

2. Type changes that change storage size (INT → BIGINT, VARCHAR(10) → VARCHAR(20)) [rewrite the table](#) in Postgres (no metadata-only path). Use Expand-Migrate-Contract again: add a new column `lead_score_big BIGINT`, backfill from the old `lead_score` (with cast), deploy app to read the new column, drop the old column. This is the safe path for any incompatible type change. (For VARCHAR length [increase](#), PG 9.2+ doesn't rewrite — it's metadata-only; only decreases or type changes rewrite.)

3. A race: between your `SELECT count(*) WHERE IS NULL → 0` (Phase 4) and the `SET NOT NULL` (Phase 5), an old app instance (not yet restarted with Phase 3 code) inserts a row with `lead_score = NULL`. The check passed, the constraint failed. Fix: ensure [all](#) app instances are on Phase 3 code (rolling deploy complete) before Phase 4/5. Or, after Phase 3 deploy, do another quick backfill pass (`UPDATE ... WHERE IS NULL`, will catch any stragglers from old instances) immediately before Phase 5. The pattern: app deploy (writes non-NULL) → short re-backfill (catch stragglers) → SET NOT NULL, all in a single deploy window.

RED FLAGS TO AVOID: - One-shot `ALTER TABLE ADD COLUMN lead_score INT NOT NULL DEFAULT 0` — works in PG 11+ but you lose the expand-contract safety (can't roll back). - Backfill in one giant `UPDATE` — multi-minute transaction, locks rows, can OOM. - `SET NOT NULL` without the `CHECK NOT VALID` trick — multi-minute write-block. - Backfill without `WHERE IS NULL` — not idempotent, can't re-run.

Q-Bonus2-5: Real-time notifications in AcquisitionOS — Long-polling vs WebSocket vs SSE. Which, and why?

- **Difficulty:** Medium-Hard
- **Category:** System Design / Real-time
- **Tags:** websocket, sse, long-polling, eventsourcing, stomp, fallback

ANSWER:

We use **SSE (Server-Sent Events)** as the **primary** path with a **long-polling fallback** for clients behind restrictive proxies. WebSocket is overkill for our use case (one-way server→client notifications). Here's the reasoning:

Use case: Server pushes notifications to the agent's browser when (a) a lead replies to an email, (b) a lead's stage changes by a teammate, (c) a meeting is booked. [All server→client](#). We don't need client→server streaming over the same connection (the client uses regular `fetch` POST for any action).

Long-polling: Client sends `GET /api/notifications/longpoll`, server holds the request open until a notification is ready (or a 25s timeout), returns it, client immediately re-requests.

Pros: works everywhere (plain HTTP), no special infra. **Cons:** 25s connection per user × 10k users = 10k concurrent connections on the server; latency = up to 25s if the connection is idle and a notification arrives just after a poll cycle starts.

SSE: Client opens `EventSource('/api/notifications/stream')`, server keeps the connection open and writes `data: {...}\n\n` lines. **Pros:** built into browsers via `EventSource`, automatic reconnection, HTTP/2 multiplexes many SSE streams over one TCP connection (so 10k users = a few hundred TCP connections, not 10k). **Cons:** one-way (server→client only — fine for us), some corporate proxies buffer SSE (need `X-Accel-Buffering: no` in nginx).

WebSocket: Bi-directional, full-duplex. **Pros:** perfect for chat, gaming, real-time collaboration. **Cons:** more infrastructure (WS-aware load balancers, sticky sessions or shared pubsub), more code (STOMP framing, ping/pong, reconnect logic), no HTTP/2 multiplexing benefit. Over-engineered for push-only notifications.

Our design: - Primary: SSE at `/api/notifications/stream`, backed by Redis Pub/Sub. The Spring `@RestController` returns `SseEmitter`, the user's session subscribes to `notifications:{userId}` channel in Redis. When any service emits a notification (`redisTemplate.convertAndSend("notifications:42", event)`), all of user 42's open SSE emitters receive it. - Fallback: Long-polling at `/api/notifications/longpoll` for the ~2% of clients whose proxy kills SSE. Detected client-side: if `EventSource` errors 3× in 30s, switch to long-poll. - Heartbeat: server sends a `: ping\n\n` comment frame every 15s to keep the connection alive through idle proxies. - Back-pressure: if the client is slow to read (TCP send buffer fills), the emitter's buffer (Spring `SseEmitter` with `new SseEmitter(0L)` — no timeout, but bounded buffer) drops the oldest undelivered event after 1000 queued; we log a warning. Notification events are also persisted to `notification` table, so a missed live push is fetched on next GET `/api/notifications`.

Code — the SSE endpoint:

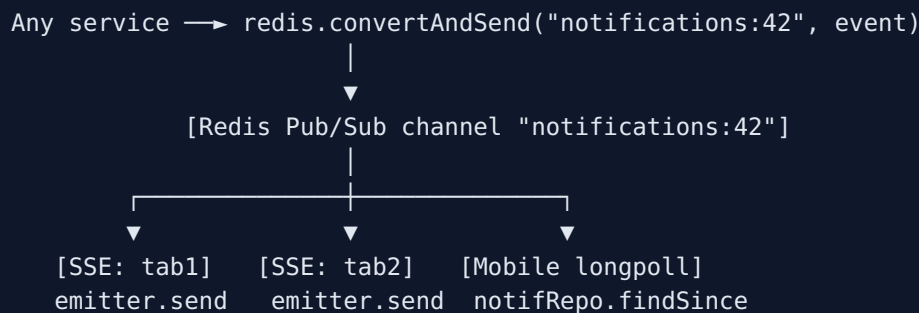
```

@RestController @RequiredArgsConstructor
public class NotificationStreamResource {
    private final RedisMessageListenerContainer redis;
    private final NotificationRepository notifRepo;

    @GetMapping(value="/api/notifications/stream",
        produces=MediaType.TEXT_EVENT_STREAM_VALUE)
    public SseEmitter stream(Principal principal) {
        SseEmitter emitter = new SseEmitter(0L); // no timeout
        String channel = "notifications:" + principal.getName();
        MessageListener listener = (msg, pattern) -> {
            try { emitter.send(SseEmitter.event().data(msg.getBody())); }
            catch (IOException e) { emitter.completeWithError(e); }
        };
        redis.addMessageListener(listener, new ChannelTopic(channel));
        emitter.onCompletion(() -> redis.removeMessageListener(listener));
        emitter.onTimeout(() -> redis.removeMessageListener(listener));
        // Heartbeat every 15s
        scheduler.scheduleAtFixedRate(() -> {
            try { emitter.send(SseEmitter.event().comment("ping")); }
            catch (IOException e) { emitter.complete(); }
        }, 15, 15, TimeUnit.SECONDS);
        return emitter;
    }
}

```

ASCII — the SSE fan-out:



KEY TERMS TO MENTION: - SSE vs WebSocket vs Long-polling (one-way vs bi-directional vs request-response) - `EventSource` browser API (auto-reconnect, last-event-id) - HTTP/2 multiplexing (SSE benefits more than WS) - Redis Pub/Sub for fan-out across app instances - Heartbeat / keep-alive comment frames (`: ping\n\n`) - `X-Accel-Buffering: no` (nginx) to disable proxy buffering for SSE

FOLLOW-UP QUESTIONS: 1. The agent has 3 browser tabs open. All 3 SSE connections receive the same notification. Idempotency on the client? 2. The Redis Pub/Sub message is lost (network blip). The user never sees the notification. Recovery? 3. Why not use WebSocket since “you might add chat later”?

FOLLOW-UP ANSWERS: 1. The client dedups by `notification.id` (UUID). Each tab maintains a `Set<UUID> seen` (or uses `localStorage` shared across tabs via the `storage` event). When a notification arrives, the client checks the set; if present, ignore; else show

toast and add to set. The backend also persists every notification to the `notification` table; when a new tab opens, it does `GET /api/notifications?since=lastSeenId` and dedups against its in-memory set. Cross-tab dedup via the `storage` event (a write to `localStorage` from one tab fires `storage` in other tabs) — used to coordinate “mark as read” across tabs. 2. Pub/Sub is at-most-once (no persistence) — a missed message is gone. We don’t rely on Pub/Sub for delivery, only for **real-time push**. The `notification` table is the source of truth. Every notification is **written to the DB** in the same transaction that does the business action (e.g., lead stage change), then `convertAndSend` is called after commit (via `@TransactionalEventListener(phase = AFTER_COMMIT)`). If the Pub/Sub push is lost, the user’s next action (page load, manual refresh, or a periodic `GET /api/notifications?since=...` poll every 60s as a safety net) fetches from the DB. So real-time is best-effort, durability is guaranteed by the DB. 3. YAGNI — “you might add chat” is a hypothetical; if we add chat later, we add WebSocket **then**. SSE is simpler now (one-way push, browser-native, HTTP/2-friendly). Adding WebSocket later is a separate endpoint (`/ws/chat`), not a migration of notifications. The cost of “wrong” choice: rewriting notifications from SSE to WS is ~1 day of work if we ever need bi-directional — much cheaper than carrying WS complexity now for a feature that may never come. Build for the present, isolate the decision (a `NotificationClient` interface with SSE impl today, swappable later).

RED FLAGS TO AVOID: - WebSocket for push-only notifications — over-engineering. - SSE without a heartbeat — proxies kill idle connections after 60s. - In-memory pub-sub (Spring `ApplicationEventPublisher`) across instances — fails when you scale to >1 server; use Redis Pub/Sub from day 1. - Relying on Pub/Sub for durability — it’s at-most-once; DB is the source of truth.

Q-Bonus2-6: Meeting scheduling across 24 timezones with DST transitions. A user in India schedules a meeting with a lead in California. The meeting recurs weekly. Walk through the timezone handling.

- **Difficulty:** Hard
- **Category:** Date/Time / Internationalization / Edge Cases
- **Tags:** timezone, DST, ZonedDateTime, Instant, ZonedDateTime, IANA-tz, recurring

ANSWER:

The classic timezone bug: store local time + assume “9am means 9am” — breaks when DST flips (California “9am PST” in March becomes “9am PDT” after DST, but if you stored “9am”, a weekly recurrence’s **absolute** time shifts by 1 hour). The correct pattern: **store `Instant` (UTC absolute) + `ZoneId` (IANA, e.g., “America/Los_Angeles”) for display**. Compute recurring occurrences **in the target zone** and store each occurrence’s `Instant`.

The recurring rule: A weekly meeting at “every Tuesday 9am California time.” We store the **rule** (`RRULE:FREQ=WEEKLY;BYDAY=TU;BYHOUR=9`) plus the originating `ZoneId=America/Los_Angeles`. To compute the next 10 occurrences, we materialize in the **zone** — for each Tuesday, 9am in LA — and convert each to `Instant`. When DST flips (Nov → Mar),

the `wall clock` stays 9am LA, but the `Instant` shifts by 1h. So the meeting stays “9am in California” forever, even as the UTC offset changes.

Code — the rule + occurrence materializer:

```
@Entity @Table(name="meeting_series")
public class MeetingSeries {
    @Id private Long id;
    private Instant startTimeUtc;           // first occurrence, absolute
    @Column(name="tz_id") private String tzId; // "America/Los_Angeles" – IANA
    private String rrule;                   // "FREQ=WEEKLY;BYDAY=TU;BYHOUR=9"
    private Long ownerId, leadId;
}

@Service
public class MeetingScheduler {
    // Materialize next N occurrences, computing each in the target zone
    public List<Instant> nextOccurrences(MeetingSeries s, int n) {
        ZoneId zone = ZoneId.of(s.getTzId());
        ZonedDateTime cursor = s.getStartTimeUtc().atZone(zone);
        List<Instant> out = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            out.add(cursor.toInstant()); // absolute UTC, even if offset changes
            cursor = cursor.plusWeeks(1); // wall-clock week; .plusWeeks handles DST
        }
        return out;
    }
}

// When the user *views* the meeting, format in THEIR zone
public String displayFor(MeetingSeries s, ZoneId viewerZone) {
    return s.getStartTimeUtc().atZone(viewerZone)
        .format(DateTimeFormatter.ofPattern("MMM d, h:mm a z"));
}
```

The DST flip illustration:

```
Meeting: weekly Tuesday 9am LA (America/Los_Angeles)
- Tue Mar 5  9:00 AM PST  (UTC-8) → Instant = 2026-03-05T17:00:00Z
- Tue Mar 12 9:00 AM PDT  (UTC-7) → Instant = 2026-03-12T16:00:00Z ← DST flipped!
- Tue Mar 19 9:00 AM PDT  (UTC-7) → Instant = 2026-03-19T16:00:00Z

Viewer in India (Asia/Kolkata, UTC+5:30):
- Mar 5:  9:00 AM PST → 10:30 PM IST
- Mar 12: 9:00 AM PDT →  9:30 PM IST ← shifted by 1h for the Indian viewer too
```

The Indian viewer sees the meeting shift from 10:30pm to 9:30pm — **correct**, because the meeting is anchored to LA’s wall clock, not to a fixed UTC instant.

DST edge cases: - Spring-forward gap: The meeting is at 2:30am LA on the DST flip day (March 10, 2026). 2:30am doesn’t exist (clocks jump 2:00→3:00). `ZonedDateTime` resolves it by moving forward (2:30 → 3:30). We detect this in scheduling (`!zone.isValidOffset(2:30am)`)

and warn the user. - **Fall-back overlap:** The meeting is at 1:30am LA on Nov 3 (fall back). 1:30am occurs twice (1:30 PDT, then 1:30 PST). `ZonedDateTime` with `withEarlierOffsetAtOverlap()` picks the first; we ask the user “first or second?” - **Zone rename:** IANA zones update (e.g., “Asia/Calcutta” → “Asia/Kolkata”). We store IANA IDs only (never abbreviations like “PST”), and bundle the `tzdata` (java.time) with each JDK update — the zone database is updated quarterly.

Calendar invite (.ics): We generate the `.ics` with `DTSTART;TZID=America/Los_Angeles:20260305T090000` and `RRULE:FREQ=WEEKLY;BYDAY=TU`. The calendar app (Google/Outlook) handles the DST math; we just emit the rule + zone. If we emitted `DTSTART:20260305T170000Z` (UTC) with the recurrence, Google would honor the absolute UTC time, and the meeting would shift to 8am LA after DST — **wrong**. Always emit the IANA `TZID` for recurring events.

KEY TERMS TO MENTION: - `Instant` (UTC absolute, store this) vs `ZonedDateTime` (wall clock + zone, for display) - `ZoneId` (IANA: “America/Los_Angeles”), never abbreviations (“PST”) - `RRULE` (iCalendar RFC 5545) for recurrence rules - DST gap (spring forward — invalid time) and overlap (fall back — ambiguous time) - `.ics DTSTART;TZID=...` (anchor to wall clock + zone, NOT to UTC, for recurring) - java.time `tzdata` (bundled with JDK, quarterly updates)

FOLLOW-UP QUESTIONS: 1. The user moves from Mumbai to Dubai (UTC+4). Do existing meetings shift? 2. The lead cancels a single occurrence (this Tuesday only, not the series). How do you model the exception? 3. A meeting series crosses a year boundary (Dec 30 → Jan 6). Does DST matter?

FOLLOW-UP ANSWERS: 1. The meeting’s **originating zone** is LA (set by the scheduler). The user moving to Dubai changes **their display zone**, not the meeting’s anchor. So a “Tuesday 9am LA” meeting shows as “Tuesday 10:30pm Dubai” (UTC+4 vs LA UTC-8 = 12.5h ahead). The `Instant` doesn’t change; the display does. If the user wants to **move** the meeting to a Dubai-friendly time, that’s a separate edit (change the series’ zone to “Asia/Dubai” with a new 9am Dubai rule — a deliberate change, not automatic). 2. RFC 5545 has `RECURRENCE-ID` for exceptions. We store a `meeting_exception` row: `(series_id, original_start_utc, status='CANCELLED')`. When materializing occurrences, we check exceptions and skip cancelled ones. For a rescheduled single occurrence (not cancelled, moved), we store the new `start_utc`. The `.ics` we emit includes `EXDATE` for cancelled and a separate `VEVENT` with `RECURRENCE-ID` for modified — Google/Outlook honor these. 3. No (DST is per-zone, not per-year). Mumbai doesn’t observe DST, so a Dec-30-to-Jan-6 weekly meeting in Mumbai is 9am every Tuesday, no shift. **But** if the meeting is in New York (which observes DST), and the series crosses the DST flip (e.g., a meeting on the Sunday of the DST change), the occurrence on that Sunday shifts by 1h. We materialize each occurrence in the zone, so we get this right automatically; no special year-boundary logic needed.

RED FLAGS TO AVOID: - Storing `LocalDateTime` (no zone) — uninterpretable later. - Storing `java.util.Date` (UTC millis, no zone info) — display logic re-derives the zone, prone to bugs. - Using “PST” abbreviation — ambiguous (Pacific Standard Time vs Philippine Standard Time).

- Storing the recurrence as “every Tuesday at 9am UTC” — wrong, the meeting should follow LA’s wall clock through DST.

Q-Bonus2-7: A POST `/api/leads` request touches 12 methods across 3 layers (Controller → Service → Repository → external API → DB). How do you trace it end-to-end in production?

- **Difficulty:** Hard
- **Category:** Observability / Distributed Tracing
- **Tags:** opentelemetry, trace-id, span, MDC, baggage, W3C-trace-context

ANSWER:

We use **OpenTelemetry (OTel)** for distributed tracing with **W3C Trace Context** propagation. Every request gets a `trace-id` (128-bit, globally unique) and a `parent span-id`. Each method/external call is a `span` with start/end time, attributes, and events. Spans nest under their parent; the trace is the full tree.

Three layers of instrumentation: 1. **Auto-instrumentation (OTel Java agent):** The `opentelemetry-javaagent.jar` attached via `-javaagent` instruments Spring (`@RestController` → span per request), JDBC (span per SQL), HTTP clients (RestTemplate/WebClient → span per call), Redis, Kafka — automatically, no code changes. This covers ~80% of the trace. 2. **Manual spans for business logic:** For the 3–4 critical business operations (e.g., “AI score lead”, “send outreach email”), we add explicit `@WithSpan("business.name")` or `tracer.spanBuilder("ai.score").startSpan()` to name spans meaningfully and attach business attributes (`leadId`, `stageId`). 3. **MDC for log correlation:** The trace-id is pushed to SLF4J MDC (`%X{trace_id}` in logback pattern), so every log line in the request carries the trace-id. Searching logs by trace-id gives you the full request log.

Propagation across the system: - **Inbound:** The Spring auto-instrument reads the `traceparent: 00-{trace-id}-{span-id}-01` header (W3C standard). If absent (new request), it generates a fresh trace-id. - **Outbound HTTP calls (to Stripe, Gmail API, etc.):** The auto-instrument injects `traceparent` into the outgoing request. Stripe’s logs (if they participate) link to our trace. - **Async (Spring `@Async`, `@Scheduled`, Kafka):** We use `ContextPropagatingTaskDecorator` on the `ThreadPoolTaskExecutor` to propagate the trace context across thread boundaries; Kafka producers/consumers use the OTel Kafka instrumentation (auto header inject/extract). - **DB:** The JDBC span captures the SQL (parameterized, no values for security) and the rows-affected count.

The trace visualization:

```

trace-id: 4a8f3b2e... (128-bit hex)
└─ POST /api/leads [span: controller, 245ms]
    └─ LeadService.create [span: service, 240ms]
        └─ LeadRepository.save [span: jdbc, 12ms] INSERT INTO lead ...
            └─ AiScoringService.score [span: business, 180ms]
                └─ WebClient.post(LLM) [span: http, 175ms] POST https://...
                    └─ scoreRepo.save [span: jdbc, 4ms]
                        └─ AuditService.record [span: business, 8ms]
                            └─ auditRepo.save [span: jdbc, 7ms]
                                └─ NotificationService.emit [span: business, 30ms]
                                    └─ redis.convertAndSend [span: redis, 1ms]
                                        └─ ResponseMapper.toDto [span: business, 3ms]

```

Each span shows duration; the slowest (LLM call, 175ms) is obvious. The waterfall view in Jaeger/Tempo/Grafana shows the parent-child timing.

The log correlation:

```

2026-09-09 14:23:11.245 [http-nio-1] INFO trace_id=4a8f3b2e... span_id=...
LeadService - Creating lead for user 42
2026-09-09 14:23:11.425 [http-nio-1] INFO trace_id=4a8f3b2e... span_id=...
AiScoringService - LLM returned score 87

```

You can grep logs by `trace_id=4a8f3b2e...` and see every log line in that request across all services/instances.

Code — manual span for business logic:

```

@Service @RequiredArgsConstructor
public class AiScoringService {
    private final Tracer tracer; // io.opentelemetry.api.trace.Tracer

    public int score(Lead lead) {
        Span span = tracer.spanBuilder("ai.score").setAttribute("lead.id",
            lead.getId()).startSpan();
        try (Scope scope = span.makeCurrent()) {
            // ... call LLM, compute score ...
            span.setAttribute("score", score);
            return score;
        } catch (Exception e) {
            span.recordException(e).setStatus(StatusCode.ERROR);
            throw e;
        } finally {
            span.end();
        }
    }
}

// Or, simpler: @WithSpan("ai.score") on the method (via OTel annotations)

```

Backend: Traces export via OTLP to **Tempo** (Grafana's tracing backend) or **Jaeger**.
Sampling: 100% of error traces, 10% of success traces (head-based sampling at the ingress;

tail-based sampling at the collector for “keep all errors”). We use the OTel Collector with a tail-sampling processor: `status_code{ERROR} → keep 100%`, otherwise 10%.

KEY TERMS TO MENTION: - OpenTelemetry (vendor-neutral), OTLP (export protocol) - W3C Trace Context (`traceparent` header: `00-{trace-id}-{span-id}-{flags}`) - Span (start/end, attributes, events, status), Trace (tree of spans) - Auto-instrumentation via `-javaagent:opentelemetry-javaagent.jar` - Manual spans (`@WithSpan` , `Tracer.spanBuilder`) for business names - MDC + `%X{trace_id}` in logback for log-trace correlation - Tail-based sampling (collector-side, keep all errors) - Jaeger / Tempo / Grafana for visualization

FOLLOW-UP QUESTIONS: 1. You sample 10% of success traces. A user reports “the API was slow at 14:23”. The trace was dropped. How do you investigate? 2. The LLM call’s span shows 175ms, but the user perceived 5s. Where’s the gap? 3. Cross-service: a Kafka message triggers a downstream service. How do you link the trace?

FOLLOW-UP ANSWERS: 1. The user’s request had a trace-id generated; even if the trace was dropped by sampling, the `logs` (always on, 100% via MDC) carry the `trace_id`. We search Loki by the user’s approximate time + their `userId`, find the log lines, get the `trace_id`, then look up the trace in Tempo (which may have it if the collector’s tail-sampling kept it because the request was slow — our sampler keeps p99+ traces). If the trace is truly gone, the logs alone are enough — we have full request/response logs (structured JSON) with all attributes; we can reconstruct the timeline from logs. The trace is a `convenience`, not the only source of truth. 2. Three places: (a) **Queue wait** — the request sat in Tomcat’s accept queue before a thread picked it up; the controller span starts `after` this, so the gap is invisible in the trace. Fix: add a span for “queue wait” (or use the `http.server.request_duration` metric that includes it). (b) **Client-side** — the user’s perception includes DNS, TCP, TLS, request payload upload, response download; the server trace only covers server processing. Fix: the client (browser) sends a `traceparent` for client-side spans via OTel browser SDK. (c) **GC pause** on the server — a 1s GC pause during the request doesn’t show as a span (it pauses the JVM). Fix: GC logs + JFR (Java Flight Recorder); correlate GC events with trace timing. 3. Kafka: the producer auto-instrument (OTel Kafka) injects `traceparent` into the Kafka record’s headers. The consumer auto-instrument extracts it and starts a span that’s a `child` of the producer’s send span. So the trace flows: producer service → Kafka send span → (Kafka broker doesn’t participate, just a queue) → consumer service receive span → consumer business span. The Kafka “send” and “receive” spans link via the W3C headers; the consumer’s trace is a continuation. We use this for the “lead created” event → outreach service flow.

RED FLAGS TO AVOID: - Custom trace IDs (`X-Request-ID` UUID, no W3C propagation) — no cross-service linking. - Logs without `trace_id` — can’t correlate logs to a trace. - 100% sampling in production — too much data; tail-based sampling at the collector is the right tool. - Only auto-instrumentation — spans named “controller.handle” / “jdbc.execute” are useless for business debugging; add manual `@WithSpan("business.name")` spans.

Q-Bonus2-8: Circuit breaker half-open state — why does it exist, and how do you tune the number of half-open requests?

- **Difficulty:** Medium-Hard
- **Category:** Resilience / Circuit Breaker
- **Tags:** circuit-breaker, resilience4j, half-open, probe, aimd

ANSWER:

The circuit breaker has 3 states: **CLOSED** (normal, requests flow), **OPEN** (failures exceeded threshold, requests fail fast), **HALF-OPEN** (after a wait, [probe](#) with a limited number of requests to test if the downstream recovered). The half-open state exists to **avoid the thundering-herd on recovery**: if OPEN → CLOSED instantly when the timer expires, all waiting clients hammer the recovering downstream at once and re-trip the breaker. Half-OPEN admits a [controlled](#) number of probes; if they succeed, go CLOSED; if any fails, go OPEN again with a longer wait.

Why half-open (not just OPEN→CLOSED): - OPEN→CLOSED with a flood of pending requests = “all-or-nothing recovery test” — risky. - HALF-OPEN with N probes = “test the water with a toe” — limited blast radius if downstream is still broken.

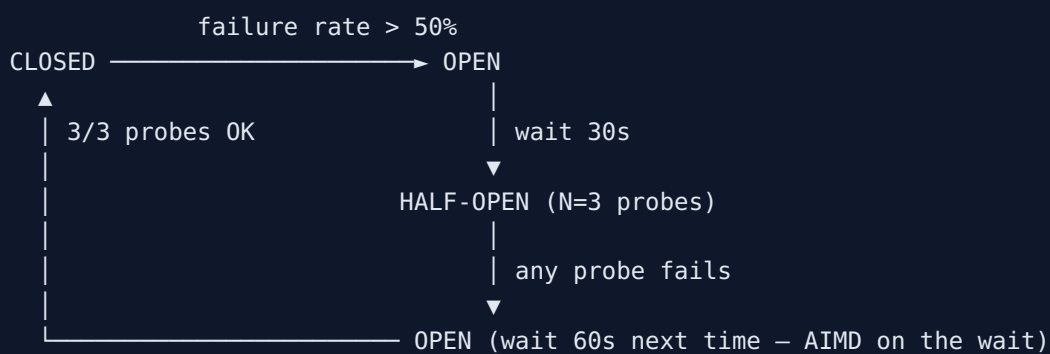
Tuning the half-open request count (N): - **Too few (N=1):** One probe. If it succeeds, you go CLOSED and flood the downstream with all queued requests. If the downstream is only [partially](#) recovered (handles 1 req/s, you send 1000/s), it dies again. The single probe was a false signal. - **Too many (N=100):** 100 probes while still HALF-OPEN. If downstream is still failing, 100 requests fail; the breaker re-opens, but you’ve already burned 100 requests (and 100 users got errors). Worse, if downstream is [slow](#) (not failing), 100 in-flight slow requests pile up. - **Right number:** Match the downstream’s [recoverable capacity](#). If Gmail recovers to 50/s, you want $N \approx 5$ (enough to detect recovery, not enough to overload). The formula: $N \approx (\text{downstream_capacity} \times \text{probe_window})$ where `probe_window` is the time you spend in half-open before deciding (say 1s) — so $N \approx 50 \times 1 = 50$... but that’s still a lot. The empirical rule: **$N = \max(3, \text{breaker_failure_rate_threshold_sample_size})$** . We use 3 (statistical minimum to estimate a success rate), tuned up if we see false-OPEN→CLOSED transitions.

The AIMD on the half-open window (advanced): A more sophisticated pattern: start HALF-OPEN with N=3; if all 3 succeed, don’t go fully CLOSED — go to a “soft closed” where you allow 10 req/s (limited), then 50, then full. This is **additive increase** (like TCP slow-start). If a probe fails during ramp-up, multiplicative decrease back to HALF-OPEN with N=3. Resilience4j doesn’t do this natively; we built a wrapper.

Code — Resilience4j config:

```
resilience4j.circuitbreaker:
  instances:
    gmail:
      registerHealthIndicator: true
      slidingWindowType: COUNT_BASED
      slidingWindowSize: 100      # last 100 calls decide
      minimumNumberOfCalls: 20    # need 20 calls before evaluating
      failureRateThreshold: 50    # 50% failures → OPEN
      waitDurationInOpenState: 30s # OPEN for 30s before HALF-OPEN
      permittedNumberOfCallsInHalfOpenState: 3 # THE half-open probe count
      halfOpenRequestCountInHalfOpenState: 3
      automaticTransitionFromOpenToHalfOpenEnabled: true
```

ASCII — the state machine:



KEY TERMS TO MENTION: - Resilience4j `CircuitBreaker` (CLOSED / OPEN / HALF_OPEN) - `permittedNumberOfCallsInHalfOpenState` (the N) - `waitDurationInOpenState` (how long to stay OPEN before probing) - Sliding window (COUNT_BASED or TIME_BASED, last N calls) - Failure-rate threshold (e.g., 50%) - AIMD on the wait duration (longer wait after a failed half-open)

FOLLOW-UP QUESTIONS: 1. The downstream is recovering but slowly (handles 5 req/s, you normally send 200/s). The breaker opens, 30s later half-opens with N=3, all 3 succeed, you go CLOSED, send 200/s, breaker opens again. Loop. Fix? 2. Why not skip half-open entirely — just retry with exponential backoff? 3. The downstream is fully down (DNS failure). Does the breaker help?

FOLLOW-UP ANSWERS: 1. The “soft closed / slow-start” pattern. Don’t jump HALF-OPEN → CLOSED (full traffic). Add an intermediate “THROTTLED” state: HALF-OPEN (3 probes) → THROTTLED (10% of normal QPS, 20 req/s) → CLOSED (full QPS), each transition gated by success. This is the AIMD on [traffic rate](#). Resilience4j doesn’t have THROTTLED natively; we wrap: after HALF-OPEN success, we don’t disable the breaker — we set a `RateLimiter` to 20/s and gradually increase it. Each step held for 10s; if any step sees > 50% errors, drop back to HALF-OPEN. This is how we avoid the recovery oscillation. 2. Retry-with-backoff handles [transient](#) failures (a single bad request). Circuit breaker handles [sustained](#) failures (downstream is broken for 30s+). Without a breaker, every request retries with backoff — 200 concurrent requests × backoff = a thundering herd of retries on recovery, and 200 timeouts (each 2s) = 400 thread-seconds wasted holding connections. The breaker [fails fast](#) (1ms) for all requests during OPEN, freeing threads. Retry is per-request; breaker is system-

wide. Use both: breaker to short-circuit sustained outages, retry (with jitter) to handle transient blips [inside](#) CLOSED state. 3. Partially. DNS failure = every request fails fast (DNS resolution <1ms); the breaker opens after `minimumNumberOfCalls` (20). During OPEN, requests fail [immediately](#) (no DNS lookup) — saves the 1ms × all requests. But the real cost of DNS failure is “all requests fail”, which the breaker doesn’t fix — it just makes the failure faster. The fix for DNS is the retry with backoff on the [client](#) side, plus a DNS cache (JVM caches positive DNS for 30s by default, `networkaddress.cache.ttl`). The breaker helps by not retrying DNS during OPEN, but the user impact is the same (errors) until DNS recovers. So: breaker is necessary but not sufficient for DNS failures; pair with DNS caching + monitoring.

RED FLAGS TO AVOID: - `permittedNumberOfCallsInHalfOpenState: 100` — too many; re-trips on partial recovery. - No half-open (jumping OPEN → CLOSED) — thundering herd on recovery. - Half-open N=1 — single false positive can flood a partially-recovered downstream. - Same wait duration after every re-trip — no backoff; should AIMD-increase the wait.

Q-Bonus2-9: The lead → outreach → meeting → deal flow is a multi-step business process. Design it as a Saga with compensating transactions.

- **Difficulty:** Expert
- **Category:** Distributed Transactions / System Design
- **Tags:** saga, compensating-transaction, orchestration, choreography, outbox

ANSWER:

A saga is a sequence of local transactions; each step publishes an event on success; if a later step fails, earlier steps run [compensating](#) transactions to semantically undo. No distributed 2PC; eventual consistency. Two flavors:

- **Choreography:** Each service listens for events and acts; no central coordinator. Pros: decoupled. Cons: hard to follow the flow; cyclic dependencies.
- **Orchestration:** A `SagaOrchestrator` service issues commands and reacts to results. Pros: flow is explicit in one place, easier to debug. Cons: orchestrator is a single point (must be HA).

For AcquisitionOS, the lead-to-deal flow is [orchestrated](#) (the flow has business rules: “skip outreach if lead score > 90, go straight to meeting”; choreography would scatter this logic across services). We use a state machine in the orchestrator.

The saga steps + compensations:

Step	Action	Compensating Action
1	<code>LeadService.create</code> (DB write)	<code>LeadService.delete</code> (soft-delete)
2	<code>AiScoringService.score</code> (LLM call, write score)	— (idempotent; no compensation; score is harmless)
3	<code>OutreachService.enqueueEmails</code> (insert outbox rows)	<code>OutreachService.cancelEmails</code> (mark outbox CANCELLED, unqueue)
4	<code>CalendarService.holdSlot</code> (Google Calendar tentative event)	<code>CalendarService.releaseSlot</code> (delete tentative event)
5	<code>DealService.create</code> (DB write)	<code>DealService.delete</code> (soft-delete)

If step 4 fails (Google API error), the orchestrator runs compensations for 3, then 1 (step 2 has no compensation). The compensations themselves are [local transactions](#) (each one is atomic); the saga is the orchestration.

The orchestrator (state machine):

```

@Service @RequiredArgsConstructor
public class LeadToDealSaga {
    private final LeadService leads;
    private final AiScoringService scorer;
    private final OutreachService outreach;
    private final CalendarService calendar;
    private final DealService deals;
    private final SagaStateRepo sagaStates;

    @Transactional
    public void run(SagaContext ctx) {
        SagaState s = sagaStates.save(new SagaState(ctx, "STARTED"));
        try {
            s.leadId = leads.create(ctx);          s.mark("LEAD_CREATED");
            sagaStates.save(s);
            s.score = scorer.score(s.leadId);      s.mark("SCORED");
            sagaStates.save(s);
            s.emailIds = outreach.enqueue(s.leadId, ctx.template());
            s.mark("OUTREACH_QUEUED"); sagaStates.save(s);
            s.eventId = calendar.holdSlot(ctx);    s.mark("SLOT_HELD");
            sagaStates.save(s);
            s.dealId = deals.create(s.leadId, ctx); s.mark("DEAL_CREATED");
            sagaStates.save(s);
            s.mark("COMPLETED"); sagaStates.save(s);
        } catch (Exception e) {
            s.mark("COMPENSATING", e.getMessage()); sagaStates.save(s);
            compensate(s);
        }
    }

    void compensate(SagaState s) {
        // Reverse order; skip steps that didn't complete; each is idempotent
        if (s.dealId != null) deals.delete(s.dealId);
        if (s.eventId != null) calendar.releaseSlot(s.eventId);
        if (s.emailIds != null) outreach.cancel(s.emailIds);
        // (no compensation for score)
        if (s.leadId != null) leads.softDelete(s.leadId);
        s.mark("COMPENSATED"); sagaStates.save(s);
    }
}

```

The outbox pattern for reliable event publishing: Each step's DB write + event publish must be atomic. We use the **transactional outbox**: the local transaction writes the business row **and** an **outbox_event** row in the same DB tx; a separate relay (**OutboxRelay** Spring Batch job) polls **outbox_event** and publishes to Kafka, then marks the row **PUBLISHED**. This guarantees the event is published **exactly once** (not at-least-once, because the relay dedups on **id**).

ASCII — the saga with compensation:

Orchestrator

1. LeadService.create —DB tx→ [lead row + outbox row] ✓
2. AiScoringService.score —LLM→ [score row] ✓
3. OutreachService.enqueue —DB tx→ [outbox rows] ✓
4. CalendarService.holdSlot —Google API→ ✗ 429

|

▼ COMPENSATE (reverse order, only completed steps)

- 3'. OutreachService.cancel(emailIds) —DB tx→ [outbox rows CANCELLED] ✓
- 1'. LeadService.softDelete(leadId) —DB tx→ [lead row deleted_at=now] ✓
- (2 has no compensation; the orphaned score is harmless)

|

▼

SagaState = COMPENSATED, error logged, user notified "calendar failed, lead discarded"

Why not 2PC (two-phase commit): 2PC requires a coordinator that locks all participants until commit; if the coordinator dies mid-prepare, participants hold locks forever (blocking). Postgres supports 2PC (**PREPARE TRANSACTION**) but it's brittle, slow, and most external systems (Gmail, Stripe) don't. The saga trades atomicity for availability: each step commits locally; the saga ensures **semantic** consistency via compensations. For a multi-minute business flow (lead → ... → deal, spanning minutes), 2PC's locks are unacceptable; saga is the right tool.

KEY TERMS TO MENTION: - Saga pattern (sequential local txs + compensations) - Orchestration (central **SagaOrchestrator**) vs choreography (event-driven) - Compensating transaction (semantic undo, not technical rollback) - Transactional outbox (DB tx + event row + async relay) - **OutboxRelay** polling job → Kafka → mark PUBLISHED (exactly-once via dedup) - SagaState table (idempotent resume on orchestrator crash)

FOLLOW-UP QUESTIONS: 1. The orchestrator crashes between step 4 (failed) and the compensation. How do you resume? 2. A compensation itself fails (CalendarService.releaseSlot returns 5xx). What now? 3. Why is the score step not compensated? Is that safe?

FOLLOW-UP ANSWERS: 1. The **SagaState** row persists every step's completion + IDs. On orchestrator restart (a new instance picks up **STARTED** / **COMPENSATING** sagas via a scheduled job), it reads the state, sees step 4 failed, and runs **compensate(s)** — which is **idempotent** (each compensation checks current state: if **dealId** exists, delete; if already deleted, no-op; if **emailIds** are PENDING, cancel; if already CANCELLED, no-op). So the resume is safe. The orchestrator is HA (multiple instances, each processing distinct sagas via **SELECT ... FOR UPDATE SKIP LOCKED** on **SagaState**). 2. A failed compensation is **poison** — the saga is in an inconsistent state (e.g., lead created, outreach queued, calendar slot held, but deal failed AND calendar release failed → the slot is leaked). We retry the compensation with exponential backoff (3 retries); if still failing, we mark the saga **COMPENSATION_FAILED** and page on-call. A human reconciles (manually release the calendar slot, or cancel the outreach). The saga state machine has a **dead-letter** state for this; we don't silently leave it. The **releaseSlot** itself is idempotent (delete by event ID; if already deleted, 404 is OK), so most "failures" are transient Google API errors that retry resolves. 3. The score is **derived data** (a computed attribute of the lead), not a **side effect**. If the lead is soft-deleted, the orphaned score row is dead data (never queried for a deleted lead); it's not a **resource leak**.

like a calendar slot or a queued email. Compensating it adds complexity for no value. The rule: compensate [external side effects](#) (sent emails, held slots, created deals) and [visible state](#) (lead rows); skip [derived/internal](#) state (scores, audit logs, metrics) — they're harmless orphans. We have a nightly cleanup that deletes orphaned scores for soft-deleted leads (eventual cleanup, not compensation).

RED FLAGS TO AVOID: - 2PC across Gmail + Postgres — Gmail doesn't support 2PC; locks held forever. - Choreography for a flow with business rules (skip outreach if score > 90) — logic scattered, bugs. - Non-idempotent compensations — resume after crash double-deletes or double-cancels. - Compensating derived data (scores) — wastes effort, no semantic harm.

Q-Bonus2-10: HikariCP pool sizing — derive the connection count formula. Our DB is on a 4-core EC2 with SSD; our app runs 8 threads doing IO + DB. How many connections?

- **Difficulty:** Hard
- **Category:** Performance / Connection Pooling
- **Tags:** hikaricp, pool-sizing, little-law, cpu-bound, io-bound

ANSWER:

The formula (from HikariCP's wiki, derived from the **Little's Law**): $\text{connections} = (\text{parallelism} \times \text{time_in_db}) / \text{time_per_request}$. The PostgreSQL/Oracle formula often cited: $\text{connections} = ((\text{core_count} \times 2) + \text{effective_spindle_count})$. The intuition:

- **DB CPU-bound queries:** Each connection's query saturates a DB core. More connections than cores = context switching, no throughput gain. Pool = ~DB cores (or 1.5× for headroom).
- **DB IO-bound queries (waiting on disk):** While one connection waits on IO, another can use the CPU. Pool = ~DB cores × 2 (the multiplier accounts for IO-wait overlap). The $+ \text{effective_spindle_count}$ accounts for the disk queue depth (SSDs have ~1 effective spindle because they handle parallel IO; HDDs have 1 per physical spindle).
- **App-side IO (calling external APIs):** While the app waits on Gmail/Stripe, the DB connection is idle (returned to the pool). The pool size is bounded by the [DB-active](#) threads, not the total app threads.

For our setup: - DB: 4-core EC2 + SSD. Formula: $(4 \times 2) + 1 = 9$. Round to 10. - App: 8 worker threads (Tomcat + async executors), but most are waiting on external APIs (LLM, Gmail, Stripe). At any instant, ~2-3 are actively querying the DB. - Pool: 10 connections (matches the DB-side formula). The 8 app threads [contend](#) for 10 connections — never blocked, never oversubscribed.

The Little's Law derivation:

$$L = \lambda \times W \quad (\text{Little's Law})$$

where L = concurrent connections in use

λ = request arrival rate (req/s)

W = average DB time per request (s)

If $\lambda = 100$ req/s, $W = 0.05$ s (50ms DB time):

$L = 100 \times 0.05 = 5$ connections busy

Pool = $L \times (1 + \text{headroom}) \approx 5 \times 2 = 10$ (50% headroom for bursts)

Why not bigger (50 connections): - Postgres has a per-connection memory cost (~10MB for the backend process) + a context-switch cost on the DB. 50 connections $\times$ 10MB = 500MB just for backends; on a 4-core machine, the OS context-switches between 50 active backends, [reducing](#) throughput. - The famous Uber Engineering post “Why we switched from Postgres to MySQL” cites a major issue: too many Postgres connections $\rightarrow$ connection multiplexing in their proxy. The fix is a [smaller](#) pool, not bigger. - For higher DB throughput, [add read replicas](#) (each with its own 10-conn pool) rather than growing one pool.

Code — HikariCP config:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10           # the formula result
      minimum-idle: 10               # keep all 10 warm (no lazy creation latency)
      connection-timeout: 30000      # 30s to wait for a conn (else
      SQLTransientConnectionException)
      idle-timeout: 0                 # never idle-timeout (we keep min=max)
      max-lifetime: 1800000           # 30min – cycle conns to balance across DB
      failover
      leak-detection-threshold: 60000 # 60s – log a stack trace if a conn is held
      this long
```

Leak detection: `leak-detection-threshold: 60000` — if a connection is checked out for >60s, Hikari logs the stack trace of the borrower. Catches forgotten `close()` (a leaked connection) and slow transactions (a tx holding a connection for 60s is a bug).

The `leakDetectionThreshold` story: We had a bug — `@Transactional` on a method that called `Thread.sleep(30000)` (waiting for an external API). The connection was held for 30s+; under load, the pool exhausted. Hikari’s leak log pointed us to the exact line. Fix: extract the `Thread.sleep` out of the `@Transactional` method (do the sleep, then open the tx). This is the kind of bug `leak-detection-threshold` catches; set it to `max_expected_tx_duration + buffer` (60s for us, since no tx should exceed 10s).

KEY TERMS TO MENTION: - HikariCP `maximumPoolSize`, `minimumIdle`, `connectionTimeout`, `leakDetectionThreshold` - Little’s Law ($L = \lambda \times W$) - Formula: `connections  $\approx$  (cores  $\times$  2) + spindles` (CPU+IO bound DB) - Connection leak detection (forgotten close / long-held tx) - Read replicas for horizontal scale, not bigger pool - `max-lifetime` (cycle conns for DB failover load balancing)

FOLLOW-UP QUESTIONS: 1. The app launches a batch job that holds 10 connections for 5 minutes (long-running). What happens to the API's pool? 2. The DB CPU is at 100% with pool=10. Do you increase the pool? 3. The connection-timeout (30s) fires under load; users see "Unable to acquire connection". First action?

FOLLOW-UP ANSWERS: 1. The pool is exhausted; the API can't get connections, requests fail with `SQLTransientConnectionException`. Fix: separate pools. The API uses `dataSource` (pool=10); the batch job uses `batchDataSource` (pool=10, dedicated). Two Hikari pools, two `DataSource` beans, two `@Transactional` markers (`@Transactional("batchTx")` for batch). This isolates the long-running batch from the latency-sensitive API. Alternatively, the batch job doesn't hold a connection for 5 min — it processes in chunks, each chunk a short tx, releasing the connection between chunks. The "5 min held connection" is almost always a bug (a long tx); fix the tx, don't add more pool. 2. No — increasing the pool makes it worse. CPU at 100% means the DB is saturated; more connections = more concurrent queries = more context switching = lower throughput. The fix is to **reduce** the pool (e.g., 5) to serialize the DB load (each query gets more CPU, completes faster), or add a read replica (offload reads), or optimize the slow queries (add an index). The reflex "increase pool to handle load" is wrong when the DB is the bottleneck; it's right when the **app** is the bottleneck (threads waiting on the pool, DB idle). 3. Look at the DB and the app: (a) DB slow query log — is there a slow query holding connections? (b) Hikari metrics — what's the `connections_active`? If active=10 and `pending_threads`=50, the pool is exhausted; the question is **why** — slow queries, long txs, or just too much load. (c) Quick mitigation: temporarily raise `maximum-pool-size` to 15 (if DB has headroom) to absorb the spike; long-term, add a read replica or fix the slow query. The 30s timeout is the **symptom**; the **cause** is elsewhere. Don't raise the timeout (it just delays errors); find the cause.

RED FLAGS TO AVOID: - Pool of 100 "just in case" — DB context-switches, throughput drops. - Same pool for API and batch — batch starves API. - `connectionTimeout: 5s` — too aggressive; bursts get false errors. - No `leakDetectionThreshold` — silent connection leaks go undiagnosed.

Q-Bonus2-11: JVM GC for p99 < 50ms API — G1 vs ZGC. Which, how do you tune, and what's the trade-off?

- **Difficulty:** Expert
- **Category:** JVM / Performance
- **Tags:** g1, zgc, gc-pause, heap, p99, jfr, jmh

ANSWER:

For p99 < 50ms with a 4-8GB heap and our workload (mostly short-lived objects — DTOs, JDBC result sets, HTTP buffers), we chose **G1GC** with tuned pause targets. ZGC is the better choice for **much larger heaps** (32GB+) or sub-10ms pauses; for our 8GB heap, G1 with `-XX:MaxGCPauseMillis=50` delivers p99 GC pause ~20-40ms, while ZGC's overhead (slightly lower throughput, higher CPU) isn't worth it. The decision is heap-size-driven.

G1GC (default in Java 17): Region-based, parallel mark-sweep, evacuation. Pauses are predictable (you set a target); tunable via `-XX:MaxGCPauseMillis=N` (soft target, G1 sizes regions to try to meet it). For 8GB heap:

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=50          # soft target
-XX:G1HeapRegionSize=4m          # 8GB / 4MB = 2048 regions
-XX:InitiatingHeapOccupancyPercent=35 # start mark when 35% used (default 45)
-XX:G1ReservePercent=15          # reserve 15% for evacuation (default 10)
-Xms8g -Xmx8g                    # fixed (no heap resize pauses)
```

ZGC (Java 17): Concurrent (almost-all), sub-10ms pauses regardless of heap. Trade-off: slightly lower throughput (~5-15% less than G1), higher CPU (concurrent GC threads compete with app), and needs more RAM for forwarding tables. For 8GB heap, the throughput hit isn't justified; for 64GB heap, ZGC is the clear winner (G1 pauses grow with heap).

```
-XX:+UseZGC
-XX:+ZGenerational                # Java 17+ generational ZGC (lower overhead)
-Xms8g -Xmx8g
-XX:SoftMaxHeapSize=6g            # ZGC will try to stay under 6GB
-XX:ZUncommitDelay=300           # wait 5min before uncommitting
```

The decision matrix: | Heap | Latency target | Pick | |—|—|—|—|—|—|—|—| | < 4GB | < 50ms | G1GC (default) | | 4-16GB | < 50ms | G1GC tuned | | 16-32GB | < 50ms | ZGC (G1 pauses start growing) | | > 32GB | < 10ms | ZGC (mandatory; G1 pauses too long) | | Any | throughput > latency | ParallelGC |

Diagnostic tools: - **GC logs:**
`-Xlog:gc*:file=gc.log:time,level,tags:filecount=10,filesize=10M` (Java 9+ unified logging).
 Analyze with GCViewer or GCEasy. - **JFR (Java Flight Recorder):**
`-XX:StartFlightRecording=duration=60s,filename=app.jfr` — low-overhead profiling; open in JDK Mission Control. Shows allocation hotspots, GC stats, lock contention. - **JMH microbenchmarks:** validate p99 with `@Benchmark` + `-prof gc` to see allocation rate. Allocation rate is the **input** to GC; reducing allocations (e.g., reusing buffers) is the highest-leverage GC tuning.

The allocation-rate story: We hit a p99 spike every 60s. GC logs showed a 200ms young-gen collection. JFR showed a 500MB/s allocation spike from a hot loop (`new ObjectMapper()` per request). Caching the `ObjectMapper` (one instance, thread-safe) dropped allocations to 50MB/s, GC pause to 20ms. The lesson: GC tuning starts with **allocation** tuning; the JVM flags are secondary.

KEY TERMS TO MENTION: - G1GC (region-based, evacuating, MaxGCPauseMillis soft target) - ZGC (concurrent, <10ms pause, generational in Java 17+) - `-XX:MaxGCPauseMillis`, `-XX:InitiatingHeapOccupancyPercent`, `-XX:G1ReservePercent` - `-Xms == -Xmx` (no heap resize pauses; the JVM commits all RAM at start) - JFR (Java Flight Recorder), JDK Mission Control,

GC logs (unified `-Xlog:gc*`) - Allocation rate (MB/s) — the input to GC; reduce allocations to reduce GC pressure

FOLLOW-UP QUESTIONS: 1. G1GC's pause target is 50ms but you see 200ms pauses. Why? 2. The heap is 32GB; you switched to ZGC but throughput dropped 10%. Worth it? 3. The app allocates 1GB/s. What GC config keeps p99 < 50ms?

FOLLOW-UP ANSWERS: 1. The target is `soft` — G1 tries but doesn't guarantee. Causes: (a) `InitiatingHeapOccupancyPercent` too high (default 45%) → mark cycle starts too late → mixed GC has too much to evacuate → exceeds target. Lower it to 35%. (b) `G1ReservePercent` too low (default 10%) → not enough reserved space → evacuation failure → fallback to full GC (200ms+). Raise to 15-20%. (c) Heap too small for the live set → frequent young GCs with large young gen → exceeds target. Increase `-Xmx`. (d) Humongous allocations (objects > 50% of region size, default 2MB for 4MB regions) bypass young gen, allocate in old directly, trigger humongous GC. Fix the allocation (e.g., reuse byte buffers, or increase `G1HeapRegionSize` to 8MB so 4MB objects aren't humongous). 2. Depends on the latency SLA. If the SLA is "p99 < 50ms" and G1 at 32GB gives p99 = 150ms (pause grows with heap), then ZGC's 10ms p99 is `mandatory` — the 10% throughput drop is the cost of meeting the SLA. If the SLA is "p99 < 200ms" and G1 hits it, switch back to G1 (10% more throughput matters). The trade-off is latency vs throughput; the SLA decides. For an API with a 100ms user-perceived budget, ZGC; for a batch job with no user, G1. 3. 1GB/s allocation is high. Even ZGC will spend ~10% CPU on concurrent GC. The fix is `reduce allocations`: (a) Reuse objects (thread-local byte buffers, pooled DTOs). (b) Avoid `new ObjectMapper()`, `new SimpleDateFormat()` (the famous per-request allocation hotspots); cache them. (c) Use `byte[]` directly instead of `String` for high-volume paths (skip UTF-8 encode/decode). (d) For JDBC, stream results (`@Query("SELECT ...")` with `Stream<T>`) instead of materializing the whole result list. (e) Profiling: JFR's allocation heatmap points you at the hottest allocators; fix the top 3. Realistically, 1GB/s → 200MB/s is achievable in most apps; then G1 with 8GB heap hits p99 < 50ms easily.

RED FLAGS TO AVOID: - `-Xmx32g` on a 4GB-heap workload — wasted RAM, longer GC pauses (bigger heap = more to scan). - `System.gc()` calls (or `DirectByteBuffer` finalizers) — explicit full GCs, unpredictable. - Choosing ZGC for a 4GB heap — throughput hit for no benefit. - Tuning GC flags without measuring allocation rate — the flags are downstream of allocations.

Q-Bonus2-12: The `lead` table has 50M rows, autovacuum can't keep up — table bloat is at 80%, queries slow. Diagnose and fix.

- **Difficulty:** Hard
- **Category:** PostgreSQL / Maintenance
- **Tags:** vacuum, autovacuum, bloat, mvcc, xid-wraparound

ANSWER:

Postgres MVCC keeps multiple row versions (tuples) for concurrency; **UPDATE** / **DELETE** create dead tuples, **VACUUM** reclaims them. If **autovacuum** can't keep up with the dead-tuple rate, dead tuples accumulate → table bloat (the table file grows beyond live data; queries scan dead tuples → slow). 80% bloat means the table is 5× its live size.

Diagnosis:

```
-- 1. Bloat estimate (needs pgstattuple or an estimate query)
SELECT pg_size_pretty(pg_relation_size('lead'));           -- table size on disk
-- Compare to live data size:
SELECT pg_size_pretty(pg_total_relation_size('lead') -
  pg_total_relation_size('lead_toast') - pg_total_relation_size('lead_idx'));
-- If 5× difference, heavy bloat.

-- 2. Dead tuple count
SELECT n_dead_tup, n_live_tup, last_autovacuum, last_vacuum
FROM pg_stat_user_tables WHERE relname='lead';
-- n_dead_tup high + last_autovacuum old → autovacuum not running

-- 3. Autovacuum activity
SELECT * FROM pg_stat_activity WHERE backend_type='autovacuum';
-- If no row for 'lead', autovacuum isn't picking it up

-- 4. Why not? Check the threshold
SHOW autovacuum_vacuum_scale_factor;  -- default 0.2 (20% dead → trigger)
SHOW autovacuum_vacuum_threshold;    -- default 50
-- For 50M rows: trigger at 50M × 0.2 + 50 = 10M dead tuples. Way too high.
```

The fix (per-table, aggressive settings):

```
-- Lower the trigger threshold for this table specifically
ALTER TABLE lead SET (autovacuum_vacuum_scale_factor = 0.05);  -- 5% → 2.5M dead
                        tuples
ALTER TABLE lead SET (autovacuum_vacuum_threshold = 10000);
ALTER TABLE lead SET (autovacuum_vacuum_cost_limit = 2000);    -- higher than
                        default 200
ALTER TABLE lead SET (autovacuum_vacuum_cost_delay = 0);      -- no delay (run
                        flat-out)
ALTER TABLE lead SET (autovacuum_analyze_scale_factor = 0.02);
```

The immediate bloat removal:

```
-- VACUUM (FULL, ANALYZE) rewrites the table (locks it briefly, ~minutes on 50M rows)
-- Run during a maintenance window:
VACUUM (FULL, ANALYZE) lead;
-- Or, online (no lock, but slower):
-- pg_repack extension (3rd party): rewrites in the background, swaps at the end
-- CREATE EXTENSION pg_repack; SELECT pg_repack('lead');
```


`VACUUM FULL` is the only built-in way to physically shrink the table; it takes an `ACCESS EXCLUSIVE` lock (no reads/writes during). For 50M rows, ~5–10 min. `pg_repack` does it online (uses triggers to capture changes during the rewrite); it's the production-safe option.

The root cause (likely): Heavy `UPDATE` churn (e.g., a job that updates `lead_score` for all leads daily). Each `UPDATE` creates a dead tuple. 50M updates × 1/day = 50M dead tuples/day; autovacuum at default settings triggers at 10M and runs slowly (cost-delay). Fix the **update pattern**: - Don't `UPDATE` rows that didn't change (skip if `new_score == old_score`). - Batch updates (one tx for many rows = fewer transaction logs, same dead tuples, but autovacuum sees them as one event). - Consider an append-only design (insert new `lead_score` rows in a separate table, query the latest) — eliminates the `UPDATE` churn entirely. This is event-sourcing for the score.

The XID wraparound risk: If autovacuum is off (or never runs), Postgres's 32-bit transaction ID will wrap around (after ~2 billion transactions) and **data loss occurs** (old tuples become invisible). Postgres auto-forces an autovacuum to prevent this (anti-wraparound vacuum), but if it can't keep up, it can shut down writes (the database goes read-only to prevent wraparound). Monitor `pg_stat_activity` for `autovacuum (to prevent wraparound)` — if you see this, it's a red alert.

KEY TERMS TO MENTION: - MVCC (multi-version concurrency control; `UPDATE` = new tuple + dead old) - `autovacuum_vacuum_scale_factor` (dead-tuple % trigger; lower for big tables) - `VACUUM` (reclaim dead tuples, doesn't shrink file) vs `VACUUM FULL` (rewrite + shrink, locks) - `pg_repack` (online table rewrite, no lock) - Table bloat (dead tuple accumulation; queries scan dead → slow) - XID wraparound (32-bit tx ID, anti-wraparound autovacuum, read-only fail-safe)

FOLLOW-UP QUESTIONS: 1. You lower `autovacuum_vacuum_scale_factor` to 0.05. Autovacuum now runs constantly, eating CPU. Trade-off? 2. `VACUUM FULL` takes a 10-min lock; users can't write. You can't afford that. What do you do? 3. The bloat is in the indexes, not the table (the table is fine, the index is 3× its expected size). Same fix?

FOLLOW-UP ANSWERS: 1. The trade-off: more autovacuum = more CPU/IO for vacuuming, but less bloat = faster queries. The sweet spot is "autovacuum runs as often as needed to keep dead tuples below the bloat threshold, no more." `scale_factor=0.05` triggers at 2.5M dead tuples; if your dead-tuple rate is 50M/day, autovacuum runs ~20×/day (every ~70 min). Each run on 50M rows is ~2-5 min of low-priority IO (cost-delay limits it). The CPU cost is ~5-10% during a run. If that's too much, raise `autovacuum_vacuum_cost_limit` (let each run finish faster) and lower `autovacuum_vacuum_cost_delay` (no waiting between pages). The tuning is: balance run-frequency × run-duration against steady-state CPU overhead. Measure with `pg_stat_user_tables` (`n_dead_tup`, `last_autovacuum`, `vacuum_count`). 2. `pg_repack` is the online alternative — it creates a new copy of the table in the background, uses triggers to capture concurrent writes, and swaps atomically. The lock is only for the swap (milliseconds). For 50M rows, `pg_repack` takes ~20-30 min but the table stays read-write throughout. The trade-off: it needs extra disk (for the new copy, ~2× the table size during the repack) and triggers add ~10% write overhead during the run. If you can't afford `pg_repack` (no extra disk, no trigger overhead), the alternative is incremental: `VACUUM (NORMAL, ANALYZE)` repeatedly (reclaims dead tuples but doesn't shrink the file); over time, new writes reuse the

freed space, bloat stays constant (no growth, but no shrink either). For 80% bloat, incremental is too slow — use `pg_repack`. 3. Index bloat is separate. `VACUUM` reclaims dead tuples in the `table` but also in indexes (it removes index entries pointing to dead tuples). However, indexes don't shrink from `VACUUM` — `VACUUM FULL` (or `REINDEX`) rebuilds them. For index-only bloat: `REINDEX INDEX lead_score_idx` (locks the index for the duration, but writes continue via the table); or `REINDEX INDEX CONCURRENTLY lead_score_idx` (no lock, but slower and can fail mid-way requiring a cleanup). We use `REINDEX CONCURRENTLY` during low-traffic windows. The root cause is often a high-churn column (e.g., `updated_at` indexed, updated every row daily) — consider dropping the index if not needed, or partitioning the table to reduce per-index churn.

RED FLAGS TO AVOID: - Disabling autovacuum (`autovacuum=off`) — guaranteed XID wraparound disaster. - Default `autovacuum_vacuum_scale_factor=0.2` on a 50M-row table — way too high (10M dead tuples before vacuum). - `VACUUM FULL` during business hours — long write lock. - Ignoring `n_dead_tup` in monitoring — silent bloat until queries are slow.

Q-Bonus2-13: Bloom filter for “has this lead been contacted in the last 90 days?” at 10M leads. Size it, explain the false positives, and the refresh strategy.

- **Difficulty:** Hard
- **Category:** Data Structures / Scale
- **Tags:** bloom-filter, false-positive, k-hashes, count-min-sketch, refresh

ANSWER:

A Bloom filter is a probabilistic set-membership structure: $O(1)$ insert and lookup, $O(n)$ bits (much less than a hash set), with **false positives** (says “maybe in” when it's not) but **no false negatives** (says “definitely not in” when it isn't). For “has been contacted?”, a false positive means “skip this lead because we think we already contacted them” — **safe** (we miss a lead, but we never spam a lead we already contacted). The no-false-negatives property is the key: we **never** email a lead we already emailed.

Sizing: For 10M items with 1% false-positive rate: - Bits: $m = -n \times \ln(p) / (\ln(2)^2) = -10M \times \ln(0.01) / 0.48 = \sim 96M \text{ bits} = 12MB$ - Hash functions: $k = (m/n) \times \ln(2) = 9.6 \times 0.693 = \sim 7$ - So: 12MB bit array, 7 hash functions, 1% FP. For 0.1% FP: 18MB, 10 hashes. For 0.01%: 24MB, 14 hashes. The size grows linearly with $-\log(p)$.

Implementation (Guava `BloomFilter`):

```
import com.google.common.hash.BloomFilter;
import com.google.common.hash.Funnels;

BloomFilter<Long> contacted = BloomFilter.create(
    Funnels.longFunnel(), 10_000_000, 0.01); // 10M capacity, 1% FPR

// On contact: insert
contacted.put(lead.getId());

// On "should we contact?": check
if (contacted.mightContain(lead.getId())) {
    // Maybe already contacted – fall back to DB check
    if (db.wasContacted(lead.getId())) return; // definite: skip
    // else: false positive – proceed to contact (rare, 1% of the time)
}
// else: definitely not contacted – proceed
```

The fallback check is critical: Bloom says “maybe”; on “maybe”, do a DB check to confirm. The DB check is the source of truth. The Bloom filter is a **fast path** to skip the DB for the 99% of leads that **definitely** weren’t contacted. If the Bloom says “not in”, we skip the DB entirely (huge savings); if “maybe”, we do the DB (1% of lookups). Expected DB lookups per N checks: $N \times 0.01$ (the false-positive rate). For 10M checks, ~100K DB lookups instead of 10M — 100× reduction.

Refresh strategy: A Bloom filter can’t delete entries (no false-negative guarantee if you unset bits — you might unset a bit shared by another item). So the 90-day window is a problem: leads contacted 91 days ago should be re-contactable, but the Bloom still says “in”. Three solutions:

1. **Rebuild periodically:** Every day, rebuild the Bloom from scratch: query `SELECT lead_id FROM contact_event WHERE created_at > now() - 90 days`, insert all into a fresh Bloom, swap atomically. Cost: a full scan of 90-day events daily; for 100K contacts/day $\times$ 90 days = 9M rows, the rebuild takes ~30s. The new Bloom replaces the old (atomic swap under a `volatile` reference).
2. **Counting Bloom filter:** Each “bit” is a 4-bit counter (instead of 1 bit); insert increments, delete decrements. Allows deletion. Cost: 4× the memory (48MB instead of 12MB for 1% FPR). Used if deletions are frequent and the rebuild is too slow.
3. **Partitioned Blooms by time:** 90 daily Bloom filters (one per day). To check “contacted in last 90 days?”, check all 90. To age out, drop the oldest daily filter and add a fresh one each day. Memory: $90 \times$ (smaller filter for daily volume). If 100K leads/day, each daily filter is ~200KB; $90 \times 200\text{KB} = 18\text{MB}$. Lookup is 90 Bloom checks (still $O(1)$, $90 \times$ constant, ~microseconds).

We use (3) — daily partitions, the cleanest for the time-windowed membership problem. The “90-day retention” is enforced by dropping partitions; no deletion needed.

ASCII — the partitioned-Bloom architecture:

```

Today: 2026-09-10
Daily Bloom partitions (each sized for ~100K leads, FPR 1%):
  bloom_2026_07_13 ← 90 days ago (will be dropped tomorrow)
  bloom_2026_07_14
  ...
  bloom_2026_09_10 ← today (active writes)

Check "was lead 42 contacted in last 90 days?":
  for each partition from 2026-07-13 to 2026-09_10:
    if partition.mightContain(42): fall back to DB
  return "definitely not"

Daily maintenance job (midnight):
  drop bloom_2026_07_13 (now 91 days old)
  create fresh bloom_2026_09_11 (empty, for today's inserts)

```

KEY TERMS TO MENTION: - Bloom filter (bit array, k hash functions, no false negatives, configurable FP rate) - Sizing: $m = -n \cdot \ln(p) / (\ln(2))^2$, $k = (m/n) \cdot \ln(2)$ (formula derivation) - False-positive fallback (Bloom says “maybe” → DB confirms) - Counting Bloom filter (deletable, 4× memory) - Partitioned Blooms for time-windowed membership (drop oldest daily) - Guava `BloomFilter.create(funnel, expected, fpr)`

FOLLOW-UP QUESTIONS: 1. The 1% FP rate means 1% of “not contacted” leads are wrongly skipped. Acceptable? 2. The DB fallback on “maybe” is slow (10M × 1% = 100K DB hits). Can you eliminate it? 3. The lead ID space is 10M but the filter is sized for 10M. After a year, 50M leads exist. What happens?

FOLLOW-UP ANSWERS: 1. For our use case (email outreach), 1% over-skipping is acceptable — we miss 1% of potential contacts, but we never spam a lead twice (the no-false-negative guarantee). The business cost of a missed lead is ~\$5 (lost opportunity); the cost of spamming a lead twice is ~\$50 (they unsubscribe, potential reputation damage). So false-positive (skip) is cheap, false-negative (spam) is expensive — the Bloom’s asymmetry matches. If the costs were reversed (e.g., fraud detection where a false-positive blocks a legitimate transaction), we’d use a higher FPR tolerance or a different structure. 2. Yes — use a [counting](#) Bloom filter and add a TTL. Or, use the partitioned approach with a [daily](#) DB-backed check: the Bloom says “maybe” only for ~1% of lookups, and those 100K DB hits spread over a day are 1.2/sec — trivial DB load. The “eliminate the fallback” temptation is wrong: the fallback is what makes Bloom [safe](#) (no false negatives). Without the fallback, the 1% false positives would [spam](#) 100K leads (the inverse of the safe behavior). The fallback is the cost of correctness, and it’s cheap. If you want zero fallback, use a hash set (10M Longs = 80MB, more memory, no FPs) — the trade-off is memory vs. DB hits. 3. The filter saturates. Once you insert more than the expected count (10M), the FP rate climbs rapidly (the bit array fills up; every lookup is “maybe”). The filter must be [resized](#) (rebuild with a larger capacity, re-insert all members). For 50M leads, the filter should be sized for 50M from the start: 60MB at 1% FPR. The sizing rule: size for the [expected count over the retention window](#), with 50% headroom. We size for 15M (1.5× the current 10M), giving us runway to grow before a resize. The resize is a full rebuild (expensive); we plan for it (monitor the FP rate, alert when it crosses 2%, rebuild at the next maintenance window).

RED FLAGS TO AVOID: - Sizing the Bloom for the current count (10M) and growing past it — FP rate explodes. - Using Bloom without a fallback for “must be correct” queries — the 1% FP is a silent error. - Single Bloom with deletion (unset bits) — breaks no-false-negative guarantee. - 5% FPR to save memory — too many DB fallbacks, negates the benefit.

Q-Bonus2-14: Outbox pattern for reliable event publishing. The DB tx commits, the app crashes before publishing to Kafka. How does the relay recover, and how do you guarantee exactly-once?

- **Difficulty:** Expert
- **Category:** Reliability / Event Publishing
- **Tags:** outbox-pattern, kafka, exactly-once, idempotent-consumer, relay

ANSWER:

The transactional outbox solves “atomic write to DB + publish to Kafka” without 2PC. The local tx writes the business row **and** an `outbox_event` row in the same DB transaction. A separate relay polls/streams `outbox_event` and publishes to Kafka. The relay’s crash recovery + the consumer’s idempotency together give **exactly-once** semantics end-to-end.

The outbox table:

```
CREATE TABLE outbox_event (
  id BIGSERIAL PRIMARY KEY,
  aggregate_type VARCHAR(50),    -- 'Lead', 'Deal'
  aggregate_id BIGINT,          -- the lead_id, deal_id
  event_type VARCHAR(50),       -- 'LeadCreated', 'StageChanged'
  payload JSONB,               -- the event body
  created_at TIMESTAMPTZ DEFAULT now(),
  published_at TIMESTAMPTZ,     -- NULL until published
  kafka_offset BIGINT,         -- Kafka offset after publish (for verification)
  UNIQUE(aggregate_type, aggregate_id, event_type, created_at)
);
CREATE INDEX ON outbox_event(published_at) WHERE published_at IS NULL;
```

The publisher (inside the business tx):

```

@Service @RequiredArgsConstructor
public class LeadService {
    private final LeadRepository leads;
    private final OutboxRepository outbox;

    @Transactional
    public Lead create(CreateLeadCmd cmd) {
        Lead lead = leads.save(new Lead(cmd)); // business row
        outbox.save(new OutboxEvent("Lead", lead.getId(), "LeadCreated",
            Map.of("leadId", lead.getId(), "stage", lead.getStage()))); // event row,
            SAME tx
        return lead; // commit publishes both atomically
    }
}

```

The relay (separate process / scheduled job):

```

@Component @RequiredArgsConstructor
public class OutboxRelay {
    private final OutboxRepository outbox;
    private final KafkaTemplate<Long, String> kafka;

    @Scheduled(fixedRate = 100) // every 100ms
    public void drain() {
        List<OutboxEvent> pending =
            outbox.findTop500ByPublishedAtIsNullOrderByCreatedAt(); // batch
        for (OutboxEvent e : pending) {
            // Send to Kafka; key = aggregate_id for ordering
            kafka.send("lead.events", e.getAggregateId(), e.toJson().get()); // sync,
                wait for ack
            outbox.markPublished(e.getId(), Instant.now()); // mark in a separate tx
        }
    }
}

```

Crash recovery scenarios:

- App crashes after `outbox.save` but before commit:** The whole tx rolls back (outbox row never committed); no event to publish. Consistent.
- App crashes after commit but before relay picks up:** The `outbox_event` row is committed; the relay (on restart) reads it and publishes. At-least-once publish (may publish twice if the relay crashes mid-publish).
- Relay crashes after `kafka.send` but before `markPublished`:** The Kafka message is published, but `published_at` is still NULL. On restart, the relay re-reads the row and re-publishes. This is the **at-least-once** publish; the **consumer** must be idempotent to make it exactly-once.
- Relay crashes after `markPublished` but before the commit of that tx:** `markPublished` rolls back; same as (3) — re-publish.

The exactly-once guarantee:

- **Producer side:** at-least-once (the relay may re-publish).
- **Consumer side:** idempotent on `event.id` (the outbox `id` is unique; the consumer dedups). The consumer stores `processed_events` table with `event_id` UNIQUE; on consume, `INSERT ... ON CONFLICT DO NOTHING`; if 0 rows inserted, it's a dup, skip.
- **End-to-end:** exactly-once (at-least-once producer × idempotent consumer = exactly-once effect).

The Kafka offset storage: We don't store the offset in our DB (that's a 2PC problem: Kafka offset commit + DB tx). We use Kafka's **own** offset commit (consumer commits offset after processing); if the consumer crashes, it re-reads from the last committed offset and re-processes some events — the idempotent consumer dedups them. This is the standard "at-least-once + idempotent consumer = effectively once" pattern; it's the only way without Kafka transactions (which are expensive).

ASCII — the relay + crash recovery:

```

TX (DB): INSERT lead + INSERT outbox_event —commit→ [both durable]
      |
Relay polls (every 100ms): SELECT WHERE published_at IS NULL
      |
      ▼
      kafka.send("lead.events", e) —ack→ [Kafka has it]
      |
      | CRASH POINT A: relay dies here → row still published_at=NULL → restart re-
      | reads → re-send (at-least-once)
      |
      ▼
      outbox.markPublished(id) —commit→ [row marked; relay won't re-send]
      |
      | CRASH POINT B: relay dies here → already marked → no re-send ✓
      |
Consumer: INSERT INTO processed_events (event_id) VALUES (?) ON CONFLICT DO NOTHING
          if 0 rows → dup, skip; if 1 row → process business logic

```

KEY TERMS TO MENTION: - Transactional outbox (DB tx + event row; atomic) - **OutboxRelay** (polls **published_at IS NULL**, publishes, marks) - At-least-once producer + idempotent consumer = effectively once - **processed_events** table with UNIQUE(event_id) for consumer dedup - Kafka offset commit (consumer-side; don't store in your DB — avoids 2PC) - Ordering: Kafka key = aggregate_id (same lead's events go to same partition, in order)

FOLLOW-UP QUESTIONS: 1. The relay polls every 100ms. Latency = 100ms. Can you make it real-time? 2. The **outbox_event** table grows forever (you mark **published_at** but don't delete). When do you delete? 3. Two events for the same lead (LeadCreated then StageChanged) — order matters. How?

FOLLOW-UP ANSWERS: 1. Yes — **Change Data Capture (CDC)** via Debezium. Debezium reads the Postgres WAL (via the **pgoutput** logical replication slot) and publishes **every row change** to Kafka in real-time (no polling, no **published_at** column). The outbox table becomes a "marker" — Debezium publishes its inserts as Kafka messages automatically. Latency: <1s (WAL streaming). Trade-off: Debezium is a separate process (Kafka Connect connector), more infrastructure, but no app code changes and no polling lag. We use polling for the first 6 months (simpler), then migrated to Debezium when latency mattered. The polling code is the fallback if Debezium dies. 2. Two retention strategies: (a) **delete after a delay** — a nightly job deletes **outbox_event WHERE published_at < now() - 7 days**. The 7-day buffer is for debugging (you can replay events from the outbox if Kafka loses them). (b) **archive to cold storage** — move to S3 after 30 days, delete from DB after 90 days. For 10M events/day, the table grows 10M rows/day = 300M/month; we delete after 7 days, so

the steady-state size is ~70M rows (~10GB), manageable. The `published_at IS NULL` index stays small (only unpublished rows). 3. Kafka partition key = `aggregate_id` (the lead ID). Kafka guarantees: messages with the **same key** go to the **same partition**, in the **order they were produced**. The relay produces with `kafka.send(aggregateId, payload)` (the key is the lead ID), so `LeadCreated` (id=42) and `StageChanged` (id=42) both go to partition `hash(42) % partitions`, in the order the relay sent them (created_at order, since the relay reads `ORDER BY created_at`). The consumer reads the partition sequentially — sees `LeadCreated` before `StageChanged`. **Caveat:** if two relays run (HA), they may interleave produces — the per-key ordering is preserved only within a single producer; with two producers, lead 42's events may interleave with lead 43's but not with each other (Kafka preserves per-key ordering only per-producer). We run one relay (HA via a leader-elect: the relay acquires a Redis lock; only the leader produces; if it dies, another takes over). One-producer-per-key preserves order; we pay for HA via leader election, not parallel relays.

RED FLAGS TO AVOID: - Publishing to Kafka inside the business tx (no 2PC; if Kafka fails, the tx rolls back the business write — but Kafka might have the message; race condition). - Storing Kafka offset in your DB (2PC problem; offsets get out of sync). - Multiple relays without leader election (per-key ordering lost). - Deleting outbox rows immediately after publish (no replay buffer for Kafka data loss).

Q-Bonus2-15: API rate limiter — token bucket vs leaky bucket vs sliding window vs fixed window. Pick one for `/api/leads/discover` (LLM-backed, expensive), justify, and implement in Redis.

- **Difficulty:** Hard
- **Category:** API Design / Rate Limiting
- **Tags:** rate-limiter, token-bucket, sliding-window, redis, lua, atomicity

ANSWER:

For `/api/leads/discover` (LLM-backed, \$0.05/call, 10 tenants with Pro plan = 100/hour each), we use **sliding window with counters** (a.k.a. `sliding log` or `sliding window with bucketed approximation`). Reasoning:

- **Fixed window:** “100/hour” resets at the top of the hour. A user sends 100 at 11:59 and 100 at 12:01 = 200 in 2 minutes. Bursts at boundaries. Bad for expensive endpoints.
- **Token bucket:** Bursts up to the bucket size (100) instantly, then refills at 100/hour. Allows a tenant to “save up” calls and burst — **bad** for LLM cost (a burst of 100 = \$5 in a second). The bucket size enables the burst we want to prevent.
- **Leaky bucket:** Processes at a fixed rate (1/36s for 100/hour), queues excess. The queue can grow unbounded under load; or you reject when the queue is full. Good for **smoothing** traffic (e.g., to a downstream), bad for **per-user limits** (the queue is per-user, memory-heavy).

- **Sliding window (exact):** Track every request's timestamp; count requests in the last 3600s. Exact, no bursts. Memory: $O(N)$ timestamps per user (100 timestamps $\times$ 10K users = 1M entries — fine in Redis sorted sets).
- **Sliding window (approx, bucketed):** Divide the hour into 6 10-min buckets; sum the buckets that overlap the last hour. Approximate (over-counts at bucket boundaries by up to 1 bucket). Memory: $O(\text{buckets})$ per user.

For an [expensive](#) endpoint (LLM cost), we want **exact** sliding window (no bursts, no over-allow). We use a Redis sorted set per user, with timestamps as scores; the Lua script atomically removes old entries, counts, and adds the new one if under the limit. Lua is essential for atomicity (read + write in one round-trip, no race condition).

The Redis Lua script (atomic):

```
-- KEYS[1] = "rate:discover:tenant:42"
-- ARGV[1] = now (epoch ms)
-- ARGV[2] = window (3600000 ms = 1 hour)
-- ARGV[3] = limit (100)
-- ARGV[4] = this request's timestamp (= now)

local key = KEYS[1]
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])

-- Remove entries older than (now - window)
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count entries in the window
local count = redis.call('ZCARD', key)

if count < limit then
  -- Add this request; use now + random suffix as member (unique)
  redis.call('ZADD', key, now, now .. ':' .. math.random())
  -- Expire the key to free memory after the window
  redis.call('PEXPIRE', key, window + 1000)
  return 1 -- allowed
else
  -- Return retry-after: ms until the oldest entry falls out of the window
  local oldest = redis.call('ZRANGE', key, 0, 0, 'WITHSCORES')
  local retry_ms = math.ceil(tonumber(oldest[2]) + window - now)
  return -retry_ms -- negative = denied, abs = retry-after ms
end
```

Java invocation:

```

@Component @RequiredArgsConstructor
public class RateLimiter {
    private final RedisTemplate<String, String> redis;
    private final DefaultRedisScript<Long> script;

    @PostConstruct void init() {
        script.setScriptSource(new ResourceScriptSource(new
            ClassPathResource("rate_window.lua")));
        script.setResultType(Long.class);
    }

    public Decision check(String tenantId, int limit, Duration window) {
        String key = "rate:discover:tenant:" + tenantId;
        long now = System.currentTimeMillis();
        Long r = redis.execute(script, List.of(key),
            String.valueOf(now), String.valueOf(window.toMillis()),
            String.valueOf(limit));
        if (r > 0) return Decision.allow();
        else return Decision.deny(Duration.ofMillis(-r)); // retry-after
    }
}

```

The 429 response: When denied, return HTTP 429 with `Retry-After: <seconds>` and `X-RateLimit-Limit: 100`, `X-RateLimit-Remaining: 0`, `X-RateLimit-Reset: <epoch>` headers. The client backs off. We also emit a metric (`rate_limit.denied{endpoint=discover}`) for monitoring (a spike in denials = a tenant hammering; we reach out about a higher plan or a bug).

Why Lua (not multi-command): Without Lua, the sequence `ZREMRANGEBYSCORE` → `ZCARD` → `ZADD` is three round-trips, and two clients can both read `count=99` and both `ZADD`, allowing 101. Lua makes it atomic (Redis executes Lua single-threaded, no interleaving). The script is loaded once (`SCRIPT LOAD`) and invoked by SHA, minimizing per-call payload.

Distributed rate limiting across instances: All app instances share the Redis rate limiter (one Redis per region). Cross-region: each region has its own Redis, and the global limit is the sum of regional limits (configured per region). For a strict global limit, use a [lease-based](#) approach (each instance leases N tokens from a central Redis every second; if it runs out, it denies locally — fewer Redis round-trips). We use the per-region approach (simpler, accepts that “100/hour” means “100/hour per region”).

ASCII — the sliding window:

```

Now = 12:00:00 (window = last 3600s = 11:00:00 to 12:00:00)
Sorted set "rate:discover:tenant:42":
  11:05:12  11:15:30  11:42:01  11:55:44  11:58:09  11:59:55  (6 entries in window)

ZREMRANGEBYSCORE 0 (now - 3600s) → removes entries < 11:00:00 (none here)
ZCARD → 6
6 < 100 → ZADD 12:00:00 → returns 1 (allow)

... 94 more requests, all allowed ...

ZCARD → 100 → not < 100 → deny; oldest = 11:05:12; retry-after = 11:05:12 + 3600 -
12:00:00 = 5m12s
Return 429 + Retry-After: 312

```

KEY TERMS TO MENTION: - Fixed vs sliding vs token bucket vs leaky bucket (pick by use case) - Sliding window with sorted set + Lua (atomic, exact, no bursts) - `ZREMRANGEBYSCORE`, `ZCARD`, `ZADD` (Redis sorted set ops) - Lua script for atomicity (single-threaded execution in Redis) - 429 + `Retry-After` + `X-RateLimit-*` headers - Per-region Redis (cross-region limit = sum; or lease-based for global)

FOLLOW-UP QUESTIONS: 1. The Lua script runs on Redis's main thread; long scripts block all other commands. How do you bound the runtime? 2. Redis is down. Do you fail-open (allow) or fail-closed (deny)? 3. The limit is per-tenant. A tenant has 1000 users. Should the limit be per-user or per-tenant?

FOLLOW-UP ANSWERS: 1. The script is $O(\log N)$ (sorted set ops), N = requests in the window ($\leq$ limit = 100). ~10 microseconds. Bounded by design: the script's runtime is proportional to the limit, not the total request count. For a higher limit (e.g., 10K), we'd switch to the `bucketed approximation` (sliding window with 60 1-min buckets, $O(\text{buckets}) = O(60)$) — less exact, but constant time. We also set `lua-time-limit 5000` (ms) in Redis config; scripts over 5s are flagged as slow (and we'd switch to bucketed). Our exact-window script never approaches this; it's microsecond. 2. **Fail-open for `/api/leads/discover`** (allow when Redis is down; rely on the LLM provider's own rate limit as the backstop). Why: the cost of false-denial (a paying customer can't use the product) > the cost of false-allow (a few extra LLM calls = \$0.05 each). The LLM provider (OpenAI) also rate-limits us, so the damage is bounded. For a `security-sensitive` endpoint (`/api/auth/login`), fail-closed (deny; you can't risk unlimited login attempts). The decision is per-endpoint: deny when the cost of over-allow is high (security), allow when the cost of over-deny is high (UX). For `/discover`, UX > cost; for `/login`, security > UX. 3. Per-tenant for `plan` enforcement (Pro = 100/hr total across the tenant's users; Elite = 1000/hr). The tenant buys a quota; the tenant's users share it. Within the tenant, the `admin` can set per-user sub-limits (e.g., "user A gets 50, user B gets 50"). Our rate-limit key is `rate:discover:tenant:42` (per-tenant), and optionally `rate:discover:tenant:42:user:5` (per-user sub-limit). The check is: tenant under limit AND user under their sub-limit. The 100/hr is a shared pool; if one user hammers it, others get 429 — the admin sees this in the dashboard and adjusts the per-user sub-limits. Per-user-only (100/hr × 1000 users = 100K/hr) defeats the plan limit; per-tenant-only (100/hr shared) lets one user exhaust it. Both, with admin-configurable sub-limits, is the right answer.

RED FLAGS TO AVOID: - Fixed window for expensive endpoints — boundary bursts. - Multi-command rate limit (no Lua) — race condition, over-allows. - Token bucket with a large bucket size — defeats burst prevention. - Fail-closed for non-security endpoints — paying customers locked out during Redis outages. - Per-user only for a plan-limited endpoint — one tenant's 1000 users = 100× the plan quota.

This document is a starting point, not a finish line. To be ready for Uber / Meta / NetApp interviews at the 4+ year level:

1. **Practice aloud** — read each answer, then say it out loud without looking. Time yourself: 90 seconds per “introduce yourself”, 3 minutes per “walk me through a feature”.
2. **Draw on a whiteboard** — every system design question, draw the architecture. Practice drawing while talking.
3. **Have 3 stories ready** — one technical challenge, one conflict with a teammate, one production incident. Each as STAR, 3-4 minutes when spoken.
4. **Know your numbers cold** — your scale, your p95/p99, your uptime, your team size, your ownership %.
5. **Mock interview** — at least 2 mocks with a friend or services like Pramp before the real interview.
6. **Sleep + eat** — the interview is 5 hours of intense thinking. Don't show up sleep-deprived.

You've built a real, production system serving thousands of users. You know this. Trust the work, walk in humble + confident, and remember: every interviewer was once the candidate.

Good luck. — Prepared for Uber, NetApp, Meta interviews, September 2026.